

Versatile Routing and Services with BGP



Volume II

Issue 1

Colin Bookham

1/17/2026

Contents

1	Introduction.....	6
	Objective	6
	Audience.....	7
	Document Structure	7
2	Getting Started.....	8
	Session Negotiation and Capabilities.....	8
	Update Messages.....	12
	Notification Messages	14
	Multi-Protocol BGP.....	15
3	MPLS/BGP IP-VPNs.....	18
	Network Topology	18
	Configuration	19
	AS_PATH Visibility	29
	Prefix Dissemination	30
	Automatic Route Filtering	31
	Route Refresh.....	32
	Outbound Route Filtering.....	33
	Soft Reconfiguration	34
	Route Target Constraint	35
	OSPF as a PE to CE Protocol	39
	Domains and Route-Types	40
	Backdoor Links and Sham Links.....	41
	Configuration and Operation	42
	Multi-AS Backbones (Inter-AS).....	49
4	Using BGP in VPLS	55
	BGP Auto-Discovery with LDP Signalling.....	58
	BGP Auto-Discovery and Signalling	61
	Multi-Homing with BGP-VPLS	64
5	BGP Signalling for VPWS.....	71
	Single-homed VPWS.....	72
	Multi-homed VPWS	76

Single-Homed Pseudowire	76
Active/Standby Pseudowire	82
6 Ethernet VPN	85
Ethernet VPN AFI/SAFI and Route-Types	86
Ethernet Auto-Discovery Route (Route-Type 1)	87
MAC Advertisement Route (Route-Type 2).....	90
Inclusive Multicast Ethernet Tag Route (Route Type 3)	93
Ethernet Segment Route (Route Type 4)	94
EVPN-VPLS	94
Proxy-ARP in EVPN.....	109
Static Protected MACs.....	113
E-Tree	115
EVPN-VPWS	124
Single-Homed EVPN-VPWS	126
Dual-Homed (single-active) EVPN-VPWS	129
Flexible-Cross-Connect (FXC).....	136
IP Prefix Advertisement (Route-Type 5).....	147
Interface-ful with SBD IRB model	150
Interface-ful with Unnumbered SBD IRB model	158
Interface-less IP-VRF-to-IP-VRF model	162
7 Labeled Unicast IPv4	168
Seamless MPLS	169
Transport Layer	170
Service Layer	176
BGP-LDP FEC Stitching.....	181
Inter-AS Option C	186
Carriers Carrier.....	191
IPv6 PE Router (6PE).....	197
8 Segment Routing and BGP	203
Node-SID Distribution using the BGP Prefix-SID	203
Advertisement of SR Policies in BGP	212
SR Policy Overview	213
SR TE Policy NLRI	215
Test Topology	216

SR Policy MPLS Local Block	216
Static SR Policy	217
Using BGP to Advertise SR Policies	221
Redundant SR Policies and Seamless-BFD	225
SRv6 Policies.....	232
Using SR Policies for Services and IPv4/IPv6 BGP.....	240
SRv6 in BGP Overlay Services	245
9 Reconvergence	253
Advertisement of Multiple Paths (ADD-PATH)	253
Best External	259
Next-Hop Tracking	261
Prefix Independent Convergence	265
Core PIC	266
Edge PIC.....	266
Minimum Route Advertisement Interval.....	272
Rapid Withdrawal	273
Rapid-Update	274
Update Propagation	274
Delayed Route Advertisement.....	275
TCP Maximum Segment Size (MSS)	281
Test Topology	282
Static TCP MSS.....	283
Path MTU Discovery	285
10 Multicast.....	292
Inter-Domain IPv4-IPv6 PIM.....	292
Multicast in MPLS/BGP IP-VPNs.....	298
Draft-Rosen	298
Multicast VPN	308
Wildcard S-PMSI.....	328
11 Graceful Restart and Error Handling	338
Graceful Restart Mechanism.....	339
Notification support for Grateful Restart.....	342
Error Handling	347
12 Security	351

FlowSpec	351
Attack Mitigation with Traffic Discard	355
Attack Mitigation with Redirect to VRF	359
Attack Mitigation using L3VPN.....	361
FlowSpec Validation.....	365
Remote Triggered Black-Holing.....	367
Generalised TTL Security Mechanism	369
Auto-Generation of Filters for BGP Peers	370
The TCP Authentication Option (TCP-AO)	374
Prefix Origin Validation.....	377
13 General Topics.....	387
Load-Balancing	387
IBGP Multipath.....	387
Multipath.....	391
Multipath with Weighted ECMP	395
EIBGP Multipath	400
BGP RIB Leaking	402
BGP Shortcuts	410
Peer Groups.....	413
Split-Horizon	414
Dynamic Peer Groups.....	418
Source-Address as Dynamic Peer Trigger.....	418
ICMPv6 Router Advertisement as Dynamic Peer Trigger	420
BGP Monitoring Protocol (BMP)	424
QoS Policy Propagation using BGP (QPPB)	434
Link State Information Distribution using BGP	439
Link-State NLRI.....	440
BGP-LS Attribute	442
Configuration and Operation	444
Advertising IPv4 Reachability over an IPv6 Core	452
Optimal Route Reflection (ORR).....	456
Route Policy Framework.....	465
Basic Path Attribute Manipulation.....	467
Nested Policies	471

Sub-Routines	474
Logical AND, OR, and NOT Operators	479
Parameterisation	481
Extended Community Filtering using Hex.....	487
BGP Conditional Advertisement.....	493
Policy Verification (BGP Policy-Test).....	497
Triggered Policy	499
Policy Accounting	499
BGP in Residential Networks.....	506
Appendix A: Path Selection Process	514
Path Selection Process.....	514
Always-Compare-MED.....	516
Deterministic MED	516
References	520
Glossary	523
Change Control	528

1 Introduction

As defined in the base specification for the Border Gateway Protocol (BGP), the primary function of a BGP speaking system is to exchange network reachability information with other BGP speakers, whilst including information on the list of Autonomous Systems that the reachability information traverses. This information can be used to construct a graph of AS connectivity for this reachability, while at the same time removing routing loops and providing operators the ability to implement local policy. The intention was clear. At its conception, BGP was to be used for exchanging Internet routes between Autonomous Systems/Internet Service Providers. As a result, the protocol was built with characteristics that above all provided a level of stability among the constant churn of the Internet routing table. The size of the Internet routing table has grown significantly since BGP was initially used to exchange routing information between AS's. At the time of writing there are around ~1,04 million prefixes in the Internet routing table (<https://www.cidr-report.org/as2.0/>), and while there have been some lessons learned and general hardening, BGP has proven its capabilities and stood the test of time.

During the last 10-15 years the use of BGP has evolved significantly. From a deployment perspective, operators have learned from experience and shared those experiences with the wider community to the mutual benefit of everyone. BGP is well understood and is considered a mature protocol. From a service delivery perspective, the evolution is two-fold. Firstly, the Internet is no longer perceived as a best-effort service. Instead, it has become a must-have, always-on service, for businesses, cloud service providers, and homes. Secondly, BGP's scalability and extensibility, together with its numerous hooks that allow for application of policy, have made it the Service Provider's protocol of choice for service enablement. So, whilst BGP remains the primary protocol for inter-domain route exchange, its use for delivery of intra-domain services has increased significantly. The base protocol has been extended many times to provide the ability to carry new reachability information. It thereby enables Service Providers to effectively deliver new services with minimum impact on their existing IP infrastructure using a known and deployed protocol.

In parallel to this, BGP is historically a slow-converging protocol, but fast-reconvergence upon failure has become an absolute requirement for delivery of high-profile services. Many potential customers use fast reconvergence upon failure as a measuring stick of network performance. Incidents that result in the failure of BGP have become totally unacceptable, and so the base protocol has had to become more robust than early implementations.

Objective

The purpose of this book is to provide the reader with an all-encompassing single reference guide to the BGP implementation within Nokia SR-OS. It aims to equip the reader with sufficient knowledge to feel competent and confident about the technology being

addressed, and to be able to maximise and optimise the user's implementation of BGP using SR-OS.

The book looks at how services can be delivered and how efficient routing can be achieved in both native IP networks and MPLS networks. It covers how BGP can be used to provide services such as Layer-2 VPNs and Layer-3 VPNs, as well as native or VPN-aware multicast and IPv6. At the core infrastructure layer, it looks at how BGP can be used to deliver scalable IP/MPLS networks using inter-AS and inter-domain scenarios. For each feature, function, or technology that the book covers, the aim is to provide an overview of what it is, and how it operates at a protocol level. The configuration requirements are then described together with CLI and debug outputs used to aid understanding. The objective is that the reader has a full understanding of the technology in question together with the knowledge of how it is implemented in SR-OS.

Audience

The book is primarily intended for IP design and engineering communities. Familiarity with SR-OS is not a requirement, although readers who are familiar with SR-OS will recognise the structure of configuration examples and Command Line Interface (CLI) outputs.

To keep this book to a manageable size, the basic operating of BGP as a path vector protocol is not discussed. Numerous other reference books provide this introductory information, and this knowledge is assumed as a prerequisite.

Document Structure

The document deals generally with SR-OS and focuses on 7x50 products, notably, 7750, 7950, and 7250 platforms, and is based on SR-OS release 25.10.R1. As it is not an FP-based product some features may not be supported on the 7250 IXR, and I have attempted to call out which features are either not supported or are implemented differently owing to the underlying hardware. However, please be aware that the document is only as current as the time it was written, and feature parity evolves. To that end it is recommended that readers check the relevant Release Notes for an updated list of features not supported on the 7250 IXR.

The aim has been to write the book so that each chapter can be read as a standalone chapter, if for example, some guidance is needed on how to implement and configure a particular service and/or function, or even just to learn how a particular technology works. To avoid repetition, I frequently refer to topics that have been previously described in other chapters.

2 Getting Started

Although this book does not discuss the operation of BGP as a path-vector protocol, it's worth a quick recap on how a BGP speaker processes and stores routes in the Routing Information Bases (RIBs). The RIB within a BGP speaker is made up of three distinct parts: the *Adj-RIB-In*, the *Loc-RIB*, and the *Adj-RIB-Out*. The Adj-RIB-In stores routing information learned from inbound Update messages advertised by peers to the local router. The routes in the Adj-RIB-In represent routes that are available to the path decision process. The Loc-RIB contains routing information the local router selected after applying policy to the routing information contained in the Adj-RIB-In. These are the routes that will be used by the local router. The Adj-RIB-Out stores information the local router selected for advertisement to its peers. This information will be carried in Update messages sourced by this router when advertising to peers. In summary, the Adj-RIB-In contains unprocessed routing information advertised by peers to the local router; the Loc-RIB contains the routes that have been selected by the local BGP speaker's best-path decision process, and the Adj-RIB-Out contains the routes for advertisement to peers in Update messages. This terminology is used extensively throughout the book and may interchangeably use Adj-RIB-In or simply RIB-In, and Adj-RIB-Out or simply RIB-Out.

Enabling BGP in its most basic form is a simple exercise. All that is required is an IP interface towards a BGP peer and some minimal BGP configuration. For conciseness, *Output 2-1* does not show the IP interface configuration. For exchange of IPv4 reachability the only parameters required are an AS number (ASN) defined within the global router context (or VPRN context), an IP address for the peer, and a peer AS number.

Output 2-1: Base BGP Configuration

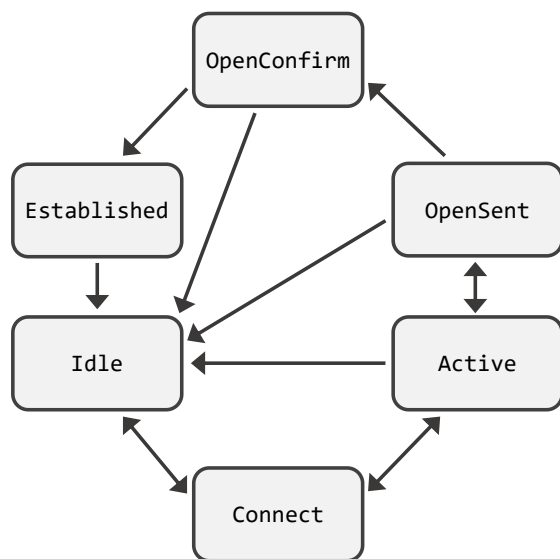
```
router "Base" {
    autonomous-system 64496
    bgp {
        group "EBGP-to-AS64511" {
            peer-as 64511
        }
        neighbor "192.168.1.2" {
            group "EBGP-to-AS64511"
        }
    }
}
```

Session Negotiation and Capabilities

A Finite State Machine (FSM) is maintained for each BGP peer, and there are six possible states in the FSM. Initially, the FSM for the BGP peer is in the *Idle* state. In this state, the router listens for a TCP connection initiated by the remote peer or initiates the TCP connection itself. The second state is the *Connect* state, where the FSM is waiting for the TCP three-way handshake to be completed. If the TCP connection is not successfully established the state is changed to *Active*, and a further attempt will be made to establish the TCP connection to the remote peer. (If the connection continues to fail, the FSM reverts

to the Idle state.) If the TCP connection is successfully established, the FSM completes the BGP initialization, generates an Open message towards the peer, and changes its state to *OpenSent*. If an Open message is also received from the remote peer and the parameters contained in the Open message are acceptable, the router generates a Keepalive message and changes its state to *OpenConfirm*. If the parameters of the Open message are not acceptable, a Notification message is sent with the appropriate error code, and the state is reverted to Idle. Whilst in the OpenConfirm state, if the router receives a Keepalive message from the remote peer, it moves to the *Established* state. In the Established state, peers can send Update messages to exchange routing information.

Figure 2-1: Finite State Machine



The Open message sent by each peer contains its AS number, Hold Time, BGP identifier, and some optional parameters. The notable optional parameter is the Capabilities parameter. The Capabilities parameter is defined in [2] and allows BGP speakers to exchange capability sets during the Open exchange. If both peers advertise a given capability the peers may use that advertised capability on the peering session. If either peer did not advertise the capability, it cannot be used.

The Capabilities parameter is encoded as a code, a length, and a value. Each capability is allocated a code by the Internet Assigned Numbers Authority (IANA) [<https://www.iana.org/assignments/capability-codes/capability-codes.xhtml>]. The length is set to 4, and as shown in Figure 2-2 the capability value field is encoded to list the Address Family Indicators (AFIs) and Subsequent Address Family Indicators (SAFIs) which are supported for the given capability.

Figure 2-2: Capability Value Field



The allocations and encodings for AFI and SAFI are again allocated by IANA. AFI allocations can be found at the following address <https://www.iana.org/assignments/address-family-numbers/address-family-numbers.xhtml> while SAFI allocations can be found at the following address <https://www.iana.org/assignments/safi-namespace/safi-namespace.xhtml>.

The output in *Debug 2-1* is taken from Open negotiation between an SR-OS router and a test device. The SR-OS router sends its Open message first with capability codes indicating support for Graceful Restart, Multi-Protocol (MP)-BGP, Route-Refresh, 4-byte ASN, and Add-Path capability.

- Graceful Restart is a high availability mechanism described in Chapter 11, and as such it is not covered in detail here. Suffice to say though that it indicates that it is not undergoing a restart, it has a restart time of 120 seconds, and it supports the IPv4 Address Family.
- The capability code for MP-BGP encodes a value of (0x0 0x1 0x0 0x1), which represents an Address Family Identify (AFI) of IPv4 (0x0 0x01) and a Subsequent Address Family Identifier (SAFI) of unicast (0x0 0x1). This indicates support only for IPv4 unicast MP-BGP, noting that IPv4 unicast NLRI can be encoded as either vanilla BGP or MP-BGP.
- The Route Refresh capability is advertised with a length of 0 and therefore carries no value field. When advertised to a peer, a BGP speaker conveys to the peer that the speaker has the capability to receive and properly handle the Route Refresh message as defined in [3].
- The capability code for 4-Octet ASN also encodes a value indicating its 4-byte Autonomous System number. In this case the router only has a 2-byte Autonomous System number (AS 64496), hence it is converted into a 4-byte Autonomous System number by setting the two high-order octets of the 4-octet field set to zero (0xfb 0xf0 = 64,996), known as a mappable 2-byte AS number.
- The Add-Path capability is encoded as shown in *Figure 2-3* where the supported AFI and SAFI are still advertised, but an additional 1-byte Send/receive field indicates whether the sender is a) able to receive multiple paths from its peer (value 1), b) able to send multiple paths to its peer (value 2), or c) both (value 3) for the advertised AFI/SAFI. In this case the Send/Receive value of 0x03 indicates the router can both send and receive multiple paths for IPv4 unicast Address Family.

Figure 2-3: Add-Path Capability



In response to the Open message from the router the test device peer sends its Open message indicating support MP-BGP, Route-Refresh, and 4-byte ASN. The MP-BGP capability advertises support for IPv4 unicast only (0x0 0x1 0x0 0x1). The Route Refresh capability is the same as described above, and as with the router, the tester only has a 2-byte AS number, hence the 4-byte ASN field represents its AS number (64511) mapped into the two low-order octets of the 4-byte ASN field (0xfb 0xff).

Debug 2-1: Open Exchange

```

7 2025/08/05 17:48:41.000 BST minor: DEBUG #2001 Base BGP
BGP: OPEN
Peer 1: 192.168.1.2 - Send (Passive) BGP OPEN: Version 4
  AS Num 64496: Holdtime 90: BGP_ID 192.0.2.1: Opt Length 30 (ExtOpt F)
  Opt Para: Type CAPABILITY: Length = 28: Data:
    Cap_Code GRACEFUL-RESTART: Length 6
      Bytes: 0x40 0x78 0x0 0x1 0x1 0x0
    Cap_Code MP-BGP: Length 4
      Bytes: 0x0 0x1 0x0 0x1
    Cap_Code ROUTE-REFRESH: Length 0
    Cap_Code 4-OCTET-ASN: Length 4
      Bytes: 0x0 0x0 0xfb 0xf0
    Cap_Code ADD-PATH: Length 4
      Bytes: 0x0 0x1 0x1 0x3

8 2025/08/05 17:48:41.000 BST minor: DEBUG #2001 Base BGP
BGP: OPEN
Peer 1: 192.168.1.2 - Received BGP OPEN: Version 4
  AS Num 64511: Holdtime 60: BGP_ID 192.0.0.111: Opt Length 16 (ExtOpt F)
  Opt Para: Type CAPABILITY: Length = 14: Data:
    Cap_Code MP-BGP: Length 4
      Bytes: 0x0 0x1 0x0 0x1
    Cap_Code ROUTE-REFRESH: Length 0
    Cap_Code 4-OCTET-ASN: Length 4
      Bytes: 0x0 0x0 0xfb 0xff

```

It is not necessary for the capability negotiation to be symmetric. Asymmetric capability negotiation is acceptable from the perspective of the peering session providing that the only optional capabilities used are the compatible ones of IPv4 MP-BGP, Route-Refresh, and 4-byte ASN. If for example the peer advertised an IPv6 prefix using MP-BGP this would result in a Notification message being sent by the receiving router, after which the integrity of the peering session would be dependent on supported and configured error handling capabilities. Although standard capabilities codes are maintained by IANA, some vendor-specific capability codes are still in use. During capability exchange these should be ignored by a BGP speaker if not recognised.

Output 2-2: Local/Remote Capabilities

```

A:admin@R1# show router bgp neighbor 192.168.1.2 | match 'Local|Remote'
Local AS          : 64496          Local Port        : 179
Local Address     : 192.168.1.1
Local Family      : IPv4
Remote Family     : IPv4
Local Capability   : RtRefresh MPBGP 4byte ASN ExtNhEncoding
Remote Capability  : RtRefresh MPBGP 4byte ASN
Local AddPath Capabi*: Send - ipv4 (2) label-ipv4 (2)
Remote AddPath Capab*: Send - None
Remote Ext-nh-encodi*: n/a

```

The Hold Times negotiated in the Open exchange do not have to be the same value at each peer for the BGP session to be established. The BGP speaker calculates the active Hold Time value by using the smaller of its configured value and the value received in the Open message. In the Open exchange shown in *Debug 2-1* the router uses the SR-OS default Hold Time of 90 seconds whilst the peer advertises a Hold Time of 60 seconds. This exchange results in both peers using a Hold Time of 60 seconds, with Keepalive messages exchanged every (60/3) 20 seconds.

Output 2-3: Active Hold Time

```
A:admin@R1# show router bgp neighbor 192.168.1.2 | match 'Hold Time'
Hold Time           : 90                Keep Alive           : 30
Min Hold Time       : 0
Active Hold Time     : 60                Active Keep Alive     : 20
Rem Idle Hold Time   : 00h00m00s
```

As previously described in the FSM, when a BGP speaker has sent an Open message it moves to the OpenSent state, and when it has received a corresponding Open message from its peer it moves to OpenConfirm state. If the BGP speaker is happy with the contents of the received Open message it responds with a Keepalive message. Once each BGP speaker has sent and received an Open message and Keepalive message, they move to the Established state and can then exchange reachability information.

Update Messages

Whilst it is not the intention to explicitly detail all BGP message formats, it’s useful to review the basic BGP Update format for the purpose of understanding the differences between that and the general format of Multi-Protocol BGP Update messages that will be covered in detail throughout the course of this book. *Figure 2-4* shows the basic Update message format. The Withdrawn Routes field contains a list of IP prefixes in the form {length, prefix} that are being withdrawn from service, whilst the Network Layer Reachability Information (NLRI) field contains a list of IP prefixes, again in the form {length, prefix} that can be reached from a given BGP speaker (subject to policy).

Figure 2-4: Update Message Format

Withdrawn Routes Length (2 octets)
Withdrawn Routes (variable)
Total Path Attributes Length (2 octets)
Path Attributes (variable)
Network Layer Reachability Information (variable)

The Path attributes field contains a sequence of attributes associated with an NLRI and each attribute can be placed into one of four categories: well-known mandatory, well-known discretionary, optional transitive, and optional non-transitive. Non-transitive simply refers to the fact that this attribute may be advertised into an AS but may not leave that AS. Mandatory attributes must be present in the Update message if NLRI is present (that is, the Update does not purely carry Withdrawn routes) and include the ORIGIN, AS_PATH, and NEXT_HOP attributes. Examples of well-known discretionary attributes include LOCAL_PREF and ATOMIC_AGGREGATE.

Debug 2-2: Update Message with NLRI

```
27 2025/08/06 08:31:22.036 BST minor: DEBUG #2001 Base Peer 1: 192.168.1.2
```

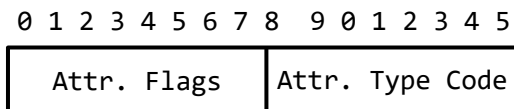
```

Peer 1: 192.168.1.2: UPDATE
Peer 1: 192.168.1.2 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 18
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 4 AS Path:
    Type: 2 Len: 1 < 64511 >
  Flag: 0x40 Type: 3 Len: 4 Nexthop: 192.168.1.2
  NLRI: Length = 4
    172.16.0.0/20

```

At the beginning of the path attribute there is a 2-octet field that contains an Attribute Flags octet followed by the Attribute Type Code octet.

Figure 2-5: Attribute Flags and Type

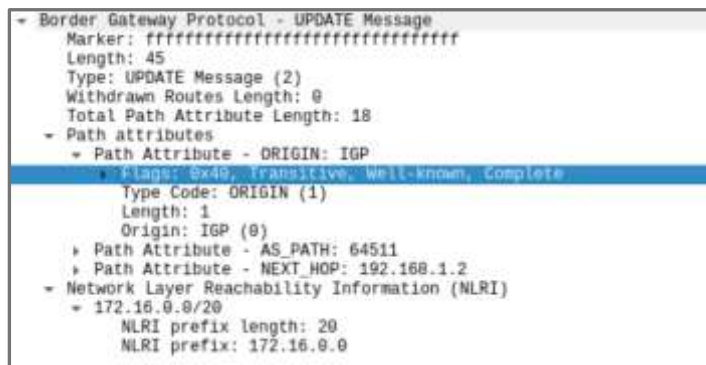


The Attribute Type Code is a value defining the type of Attribute. Within the Attribute Flags octet, the four high-order bits all carry information about the attribute:

- The high-order bit (bit 0) is the Optional bit and defines whether the attribute is optional (1) or well-known (0).
- The second high-order bit (bit 1) is the Transitive bit and defines whether an optional attribute is transitive (1) or non-transitive (0).
- The third high-order bit (bit 2) is the Partial bit and defines whether the information contained in the optional transitive attribute is partial (if set to 1) or complete (if set to 0). If a BGP speaker recognises the optional transitive attribute it sets the partial bit to 0. If the BGP speaker recognises the optional transitive attribute and would set the partial bit to 0 but the partial bit has already been set to 1 by some other AS, it must not be set back to zero by the processing speaker. In effect, when set the partial bit provides visibility that some BGP speaker along the path did not recognise the attribute.
- The fourth high-order bit (bit 3) is the Extended Length bit. It defines whether the attribute length is one octet (if set to 0) or two octets (if set to 1).

Figure 2-6 shows a packet trace of the Update message contained in *Debug 2-2* with particular attention drawn to the attribute flags field of the ORIGIN attribute. In this example the Flags field is set to 0x40 (binary 0100), therefore it is a well-known (bit 0 is set to 0) transitive (bit 1 is set to 1) attribute that is recognised by the BGP speaker and is therefore complete (bit 2 is set to 0). It is also a standard-length attribute with a one octet attribute length (bit 3 is set to 0).

Figure 2-6: IPv4 NLRI Update Message



Examples of optional non-transitive attributes include the MED, ORIGINATOR_ID, CLUSTER, MP_REACH, and MP_UNREACH attributes, whilst examples of optional transitive attributes include the AGGREGATOR and COMMUNITY attributes.

To withdraw a route from service once it has been advertised, the IP prefix previously advertised as NLRI in the Update message can be advertised in the Withdrawn Routes field of a subsequent Update message, or a replacement route with the same NLRI can be advertised. Equally, if the BGP session between two peers is closed all routes advertised to each other are implicitly removed. If an Update message carries only Withdrawn Routes and no NLRI, the mandatory attributes such as NEXT_HOP, ORIGIN, and AS_PATH need not be present.

Debug 2-3: Update message with Withdrawn Routes

```

33 2025/08/06 09:27:54.020 BST minor: DEBUG #2001 Base Peer 1: 192.168.1.2
Peer 1: 192.168.1.2: UPDATE
Peer 1: 192.168.1.2 - Received BGP UPDATE:
  Withdrawn Length = 4
    172.16.0.0/20
  Total Path Attr Length = 0
  
```

Notification Messages

A Notification message is sent when an error condition is detected and causes the BGP session to close. The Notification message contains fields for error codes, one or more error sub-codes associated with that error code, and a data field that provide some indication of the error condition. Error codes and sub-codes are contained in section 4.5 of [1], subsequently updated by [4].

Debug 2-4: Notification Message

```

48 2025/08/06 09:37:29.289 BST minor: DEBUG #2001 Base Peer 1: 192.168.1.2
Peer 1: 192.168.1.2: NOTIFICATION
Peer 1: 192.168.1.2 - Send BGP NOTIFICATION: Code = 6 (CEASE) Subcode = 2 (Administrative Shutdown)
  
```

Error conditions that require a Notification message to be sent are categorised into those experienced during processing of the generic BGP message header, those experienced in

processing of the Open message, and those experienced in processing of Update messages. When the BGP session is closed, the associated TCP connection is closed, the RIB-In entries with the peer are cleared, and all resources allocated to the associated peer are released. Errors in the BGP message header are uncommon and indicate a fundamental problem whilst errors in the Open message are typically due to misconfiguration of peer parameters. However, errors in Update messages are not uncommon, and have the potential to be extremely disruptive. The original BGP specification [1] called for a Notification message to be generated under certain conditions during error checking of attributes within Update messages. This was recognised as being too disruptive, hence [5] specified alternative measures to be implemented under these circumstances to avoid this level of disruption. This is discussed further in Chapter 11.

Multi-Protocol BGP

The Multiprotocol extensions to BGP defined in [6] provide the capability for BGP to carry routing information for multiple network layer protocols such as IPv6, VPN-IPv4, VPN-IPv6, EVPN, and Multicast-VPN to name but a few. To identify individual network layer protocols and be able to associate them with next hop information and the semantics of the NLRI, the extensions to Multiprotocol BGP specified the use of the AFI and the SAFI. For example, a VPN-IPv4 prefix is represented as AFI 1 (IPv4), SAFI 128 (MPLS-labeled VPN address).

Two optional non-transitive attributes were introduced to support Multiprotocol extensions to BGP; Multiprotocol Reachable NLRI and Multiprotocol Unreachable NLRI. The Multiprotocol Reachable NLRI (MP_REACH_NLRI) is used to carry the set of reachable destination prefixes together with the Next-Hop information to be used for forwarding to those destination prefixes. Each MP_REACH_NLRI UPDATE message contains a single Next-Hop address and a list of NLRIs associated with that Next-Hop address.

Figure 2-7: MP_REACH_NLRI Encoding

Address Family Indicator (2 octets)
Subsequent Address Family Indicator (1 octet)
Length of Next Hop Network Address (1 octet)
Network Address of Next Hop (variable)
Network Layer Reachability Information (variable)

At a minimum, an Update message that carries the MP_REACH_NLRI must also carry the Next-Hop, Origin and AS_PATH attributes (in EBGp and IBGP) and the LOCAL-PREF attribute (in IBGP).

In contrast, Multiprotocol Unreachable NLRI (MP_UNREACH_NLRI) are used to withdraw one or more unfeasible routes and have similar format as the MP_REACH_NLRI attribute without the requirement to signal Next-Hop information.

Figure 2-8: MP_UNREACH_NLRI Encoding

Address Family Indicator (2 octets)
Subsequent Address Family Indicator (1 octet)
Withdrawn Routes (variable)

Unlike the MP_REACH_NLRI, an Update message containing the MP_UNREACH_NLRI attribute is not required to carry any other path attributes.

The capability to support Multiprotocol BGP is negotiated in the Open exchange on an Address Family basis (capability code 1). By default, SR-OS signals the Multi-Protocol BGP capability for AFI/SAFI unicast IPv4 only. If other Address Families are added or removed at BGP/group/neighbour level the Open exchange is re-negotiated. To illustrate the encoding of the Multiprotocol BGP MP_REACH_NLRI *Debug 2-5* shows an Update message for a VPN-IPv4 prefix 172.31.3.0/24. Note the Address Family, Next-Hop information, and NLRI are all contained within the single MP_REACH_NLRI attribute. In this instance, and for this AFI/SAFI, the NLRI consists of the prefix, a Route-Distinguisher (RD), and an MPLS label. Other AFI/SAFI have NLRI consisting of other attributes.

Debug 2-5: Update with MP_REACH_NLRI Attribute

```
64 2025/08/06 13:18:40.664 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 97
    Flag: 0x90 Type: 14 Len: 32 Multiprotocol Reachable NLRI:
        Address Family VPN_IPV4
        NextHop len 12 NextHop 192.0.2.3
        172.31.3.0/24 RD 64496:1 Label 524283 (Raw label 0x7fffb1)
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 6 AS Path:
        Type: 2 Len: 1 < 64512 >
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.3
    Flag: 0x80 Type: 10 Len: 4 Cluster ID:
        192.0.2.10
    Flag: 0xc0 Type: 16 Len: 24 Extended Community:
        target:64496:1
        vrf-import:192.0.2.3:2
        source-as:64496:0
```

The same VPN-IPv4 prefix can also be seen in *Figure 2-9* which clearly shows the MP_REACH_NLRI as a path attribute containing the AFI (1) and SAFI (128), the Next-Hop, and the NLRI, which in this case consists of an MPLS label, an RD, and the IPv4 prefix.

Figure 2-9: Packet trace of Update message containing MP_REACH_NLRI

```

* Border Gateway Protocol - UPDATE Message
  Marker: ffffffffffffffffffffffffff
  Length: 120
  Type: UPDATE Message (2)
  Withdrawn Routes Length: 0
  Total Path Attribute Length: 97
  * Path attributes
    * Path Attribute - MP_REACH_NLRI
      * Flags: 0x00, Optional, Extended-Length, Non-transitive, Complete
      * Type Code: MP_REACH_NLRI (14)
      * Length: 32
      * Address family identifier (AFI): IPv4 (1)
      * Subsequent address family identifier (SAFI): Labeled VPN Unicast (128)
      * Next hop: RD=0:0 IPv4=192.0.2.3
      * Number of Subnetwork points of attachment (SNPA): 0
      * Network Layer Reachability Information (NLRI)
        * BGP Prefix
          * Prefix Length: 112
          * Label Stack: 524283 (bottom)
          * Route Distinguisher: 64496:1
          * MP Reach NLRI IPv4 prefix: 172.31.3.0
      * Path Attribute - ORIGIN: IGP
      * Path Attribute - AS_PATH: 64512
      * Path Attribute - LOCAL_PREF: 100
      * Path Attribute - ORIGINATOR_ID: 192.0.2.3
      * Path Attribute - CLUSTER_LIST: 192.0.2.10
      * Path Attribute - EXTENDED_COMMUNITIES

```

The introduction of Multiprotocol BGP was significant. BGP was already considered a very flexible protocol and relatively lightweight to support. With the introduction of Multiprotocol BGP AFI/SAFI and different NLRI, it had become extensible to support any other network layer as will be seen in the following chapters.

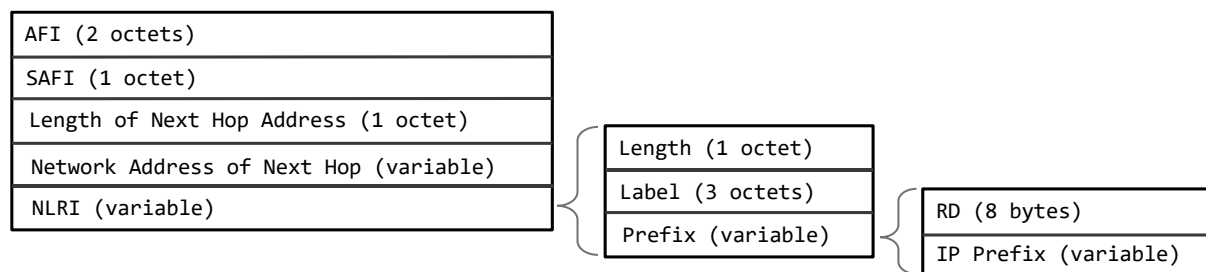
Update or MP_REACH and Withdraw or MP_UNREACH are referred to interchangeably throughout this book.

3 MPLS/BGP IP-VPNs

The framework for building BGP/MPLS based IP-VPNs is defined in [7] and relies on Multiprotocol BGP [6] and the optional transitive BGP Extended Communities [8] attribute 'Route Target'. Multiprotocol BGP is used for advertising of VPN-IPv4/VPN-IPv6 prefixes, and since both are labeled prefixes, they follow the encoding of labeled BGP [9], where the prefix is constructed of an 8-byte Route-Distinguisher (RD) followed by a 4-byte IPv4 prefix or 16-byte IPv6 prefix. The purpose of the RD is to allow the concatenation of RD and IPv4/IPv6 prefixes to create a unique VPN-IPv4/VPN-IPv6 prefix.

For VPN-IPv4 the AFI is 1 (IPv4) and for VPN-IPv6 the AFI is 2 (IPv6). Both VPN-IPv4 and VPN-IPv6 use a SAFI of 128 (MPLS-labeled VPN address).

Figure 3-1: VPN-IPv4/IPv6 NLRI Encoding



When a route is redistributed into VPN-IPv4 or VPN-IPv6, a Route-Target extended community is appended to the prefix. The Route-Target extended community is a transitive attribute used to define the set of sites belonging to a given VPN. When a VPN-IPv4/IPv6 prefix is received at a PE router it parses the Route-Target value and checks whether any locally configured VRFs have an import policy that matches the received value. If it does, the route is imported into that VPRN. If it does not, the route is not imported into any VPRNs. In short, associating a particular Route-Target attribute with a prefix allows that route to be installed into VRFs serving that VPN. If ten sites in a VPN all have a common export and import Route-Target value, then the result will be an 'any-to-any' VPN.

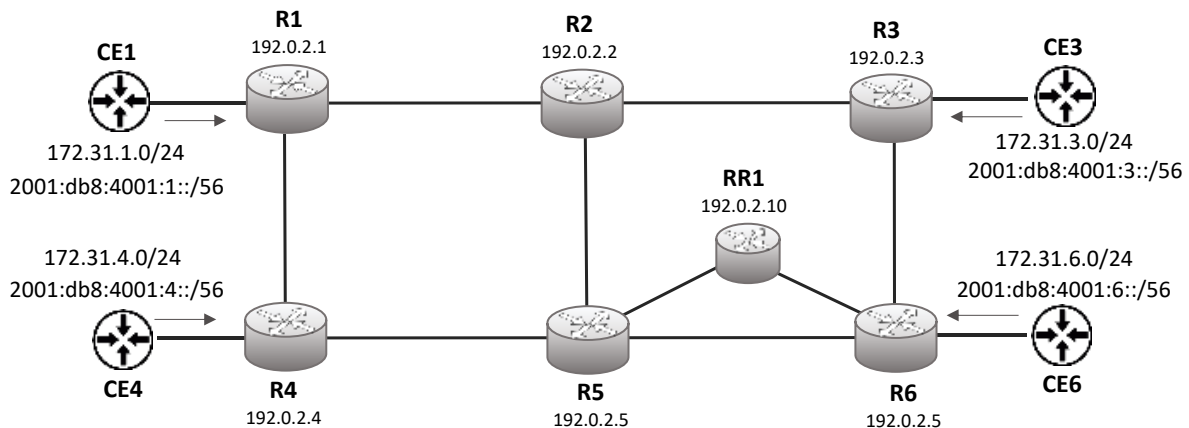
The term BGP/MPLS IP-VPN is somewhat outdated. Since the introduction of SRv6 there is no definitive requirement for an MPLS data-plane, so while the temptation may have been to simply use the term BGP IPVPNs for correctness, I have maintained the term BGP/MPLS IP-VPN throughout this section and the remainder of the document. The use of SRv6 as a data-plane for IP-VPNs is discussed and described in Chapter 8.

Network Topology

To illustrate the use of MPLS/BGP IP-VPN the network topology shown in *Figure 3-2* is used. Routers R1 to R6 and a Route-Reflector, RR1, are in a flat IS-IS Level 2 area. All routers belong to Autonomous System 64496 and Routers R1 to R6 peer in IBGP with RR1 as Route-

Reflector clients to simplify BGP prefix advertisement. All IGP link metrics are 100. Both LDP and Segment Routing (SR-ISIS) are enabled within the domain. There are four CE routers attached to parent PE routers that advertise IPv4 and IPv6 prefixes to their adjacent routers in IPv4 and IPv6 EBGP which will be used to verify the IP-VPN connectivity. All of the CE routers belong to AS 64512.

Figure 3-2: Network Topology for MPLS/BGP IP-VPN



Configuration

Before detailing any of the service configuration requirements for basic IP-VPN connectivity between the sites, the VPN-IPv4 and VPN-IPv6 Address Families should be enabled in IBGP to allow for correct dissemination of VPN IPv4/IPv6 prefixes. Recall that Address Families are negotiated during the Open exchange, hence the addition (or removal) of Address Families will cause an established BGP session to renegotiate.

Output 3-1: Peering for VPN-IPv4 and VPN-IPv6 Address Families

```
router "Base" {
  bgp {
    group "IBGP-Core" {
      family {
        vpn-ipv4 true
        vpn-ipv6 true
      }
    }
    neighbor "192.0.2.10" {
      group "IBGP-Core"
    }
  }
}
```

It is not the purpose of this book to attempt to detail every option of service-level configuration, however, the basic service configuration for an IP-VPN service (or VPRN) between the four sites is shown in *Output 3-2*, and includes EBGP peering for IPv4 and IPv6 Address Families between the PE and CE. Descriptions of the base VPRN parameters follow, while the interface and BGP parameters are described below the output. The configuration begins with the **service-id** and **customer** number which are fundamental constructs of SR-OS service configuration and are mandatory attributes. These are followed by the

autonomous-system number, which is required if BGP is used as the PE-CE routing protocol. It will be used as the source AS number in the Open exchange and will be the AS number inserted into the AS_PATH attribute when advertising routes externally. Within the **bgp-ipvpn** context there are options for either **mpls** or **segment-routing-v6** with this example using **mpls**. The **route-distinguisher** is a mandatory parameter when configuring a VPRN and the VPRN will not become operational until it is configured. When a VPRN is configured with a **route-distinguisher** but without any Route-Target parameters, the VPRN does not rely on any MPLS/BGP IP-VPN control plane for learning prefixes but simply creates a separate local routing context often referred to as 'VRF-lite'. The **route-distinguisher** command is followed by a value that can take three formats but typically uses the type 0 format of a 2-byte ASN subfield followed by a 4-byte assigned number subfield (the remaining 2-bytes are used to define the actual type).

To participate in the MPLS/BGP IP-VPN control plane the definition of Route-Target values is required for import and export of VPN-IPv4 prefixes. The simplest method is using the **vrf-target** context followed by an **import-community** and an **export-community** that have the same format as the **route-distinguisher**. The manipulation of the **import-community** and **export-community** Route-Target values allow for creation of topologies such as hub/spoke or extranet in addition to the straightforward any-to-any connectivity. An alternative to the **vrf-target** approach for defining Route-Target values is to use the **vrf-import** and **vrf-export** commands to reference policies constructed within the policy framework. This is the more widely used approach for production networks as it allows for finer granularity of exactly what routes are imported and exported into a VRF. It allows for the imposition and manipulation of attached path attributes/communities. If for some reason both **vrf-target** and **vrf-import/export** are configured, the **vrf-import/export** configuration overrides the **vrf-target** configuration.

When prefixes are learned in VPN-IPv4 or VPN-IPv6, the receiving PE router must resolve the BGP Next-Hop to a GRE or MPLS tunnel before the prefix is considered valid. The **auto-bind-tunnel** context allows the operator to define the preferred tunnel mechanism. The **resolution** command has options for **filter** or **any**. When **any** is selected, the system does a lookup in the tunnel-table and selects the tunnelling mechanism that is the most preferred (where most preferred is the lowest numeric value). When **resolution filter** is selected, the **resolution-filter** context is used to filter the tunnelling protocols to only those that are deemed suitable for this service. If there is no preference as to the tunnelling mechanism used for a service then the **resolution any** option can be configured, in which case the tunnel with the lowest metric in the tunnel table is used. In the example of *Output 3-2 resolution filter* is used and only **sr-isis** is configured, in which case it is the only tunnelling mechanism used to resolve BGP Next-Hops.

Output 3-2: VPRN basic configuration

```

vprn "one" {
  admin-state enable
  service-id 1
  customer "1"
  autonomous-system 64496
  bgp-ipvpn {

```

```

mpls {
    admin-state enable
    route-distinguisher "64496:1"
    vrf-target {
        import-community "target:64496:1"
        export-community "target:64496:1"
    }
    auto-bind-tunnel {
        resolution filter
        resolution-filter {
            sr-isis true
        }
    }
}
}
bgp {
    min-route-advertisement 1
    group "CE-EBGP" {
        as-override true
        split-horizon true
        send-communities {
            extended false
        }
    }
    neighbor "192.168.100.2" {
        group "CE-EBGP"
        authentication-key "JK/6NxCqPrX7NU7cD3w1Pms1rQvMqZw= hash2"
        peer-as 64512
        family {
            ipv4 true
        }
        import {
            policy ["RFC8212-allow-ebgp"]
        }
        export {
            policy ["VPN-IPv4/v6-to-EBGP"]
        }
    }
    neighbor "2001:db8:3001:2101::2" {
        group "CE-EBGP"
        authentication-key "JK/6NxCqPrX7NU7cD3w1Pu/cEXUQmKg= hash2"
        peer-as 64512
        family {
            ipv6 true
        }
        import {
            policy ["RFC8212-allow-ebgp"]
        }
        export {
            policy ["VPN-IPv4/v6-to-EBGP"]
        }
    }
}
}
interface "link-to-CE1" {
    ipv4 {
        primary {
            address 192.168.100.1
            prefix-length 30
        }
    }
    sap 1/1/c3/1:200 {
    }
    ipv6 {
        address 2001:db8:3001:2101::1 {
            prefix-length 126
        }
    }
}
}

```

```

    }
}
}

```

The interface towards the CE is straightforward and is dual-stack IPv4/IPv6 to allow for IPv4 and IPv6 to be forwarded over the IP-VPN. A VPN is referred to as an IPv6 VPN when each site of the VPN is IPv6 capable and is connected to a PE router of the Service Provider over an IPv6 interface. Typically, a site would be both IPv4 and IPv6 capable dictating a requirement for dual-stack operation on the PE and CE. This is not a requirement however, and single-stack IPv6 is equally possible. Running dual-stack on the PE-CE interface is as simple as configuring an IPv4 address and an IPv6 address. In this example the interface is configured with an IPv6 Globally Unique Address (GUA) and the link-local address is automatically generated using prefix fe80::/64, but it is equally possible to manually configure the link-local address. *Output 3-3* shows the IPv6 interface with the auto-generated IPv6 link-local address of fe80::fd:9e36:8816:937f/64.

Output 3-3: IPv6 Interface with Link-Local Address

```
A:admin@R1# show router 1 interface "link-to-CE1" ipv6
```

```

=====
Interface Table (Service: 1)
=====
Interface-Name      Adm      Opr (v4/v6)  Mode      Port/SapId
IP-Address          PfxState
-----
link-to-CE1         Up        Up/Up        VPRN       1/1/c3/1:200
2001:db8:3001:2101::1/126
fe80::fd:9e36:8816:937f/64
PREFERRED
PREFERRED
-----
Interfaces : 1

```

Note that the 126-bit PE-CE interface IPv6 address is a routable address that appears in the VPRN IPv6 routing table. Link-local addresses are not routable and therefore do not appear in the IPv6 routing table. The consequence of this is that if an operator wishes to address a connected or adjacent link-local address it must be suffixed with the appropriate interface name delineated with a % marker like so:

Output 3-4: VRF ping to CE1 link-local address

```

A:admin@R1# ping fe80::e3:b77a:e679:ba9d router-instance "one" count 1
MINOR: OAM #1972: The address is invalid or does not match the address type

A:admin@R1# ping fe80::e3:b77a:e679:ba9d%link-to-CE1 router-instance "one" count 1
PING fe80::e3:b77a:e679:ba9d%link-to-CE1 56 data bytes
64 bytes from icmp_seq=1 hlim=64 time=3.71ms.

---- fe80::e3:b77a:e679:ba9d%link-to-CE1 PING Statistics ----
1 packet transmitted, 1 packet received, 0.00% packet loss
round-trip min = 3.71ms, avg = 3.71ms, max = 3.71ms, stddev = 0.000ms

```

The use of 127-bit IPv6 prefixes for PE-CE addressing is recommended in [16] and resolves exposure to IPv6 Neighbour Discovery exploitation [17]. The use of 126-bit IPv6 prefixes also serves to largely mitigate the same issue(s) and is analogous to the use of 30-bit IPv4 addressing on the PE-CE link also used on this interface.

Continuing with *Output 3-2* and moving to BGP, the example uses two peers, one IPv4 and one IPv6 with each peer advertising only its associated Address Family. It is of course possible to use a single Address Family to advertise both IPv4 and IPv6, but a more common approach is to have separate peering sessions. Within the **bgp** context the first parameter is the **min-route-advertisement** interval (MRAI). The MRAI defines the interval between which routes are advertised to a given peer or set of peers and defaults to a value of 30 seconds. To allow for faster convergence, the **min-route-advertisement** is commonly modified, and in this example is set to a value of 1 second. The **group** context is used to define the parameters for a common set of peers and can be considered analogous to a 'peer-group'. Within this context the first parameter is the **as-override** command. When **as-override** is configured on a PE-CE EBGp session, the PE rewrites any iterations of the peer AS number with the VPRN **autonomous-system** number when the route is advertised towards that peer. This allows for multiple sites of an IP-VPN to belong to the same AS number without creating problems with AS number loops. The **split-horizon** command is used to avoid a route being advertised back towards a peer from which the route was received. By default, SR-OS only applies **split-horizon** behaviour to routes received from IBGP non-client peers, which means by default a route received from an EBGp peer will be advertised back towards that peer. As this is typically not a behaviour favoured by customers, **split-horizon** is applied to these PE-CE EBGp peering sessions. The final **group** level parameter is the **send-communities** context which has **extended false** configured. This stops the advertisement of any Extended Communities towards a peer by overriding them with null before the route is still advertised. Extended Communities are typically used within a Service Provider core and therefore many providers prefer to block any routes containing Extended Communities from being advertised externally as these may include sensitive information such as Route-Targets.

Within each **neighbor** context, the first parameter is to link the neighbor to a (peer) group, and this is done with the **group** command which references the previously configured **group**. The **authentication-key** does basic MD5 authentication of the TCP session and whilst widely used, it is deprecated due to its security weakness, with the recommended approach being the TCP Authentication Option (TCP-AO) [12], described in Chapter 13. This is followed by the **peer-as** which allows for configuration of the neighbouring AS number and defines the BGP session as either internal or external. In this example, all sites of the VPN belong to AS 64512 hence the session is external. The **family** context allows for configuration of the allowed Address Families on this peering session. The example is straightforward in that the IPv4 session allows only the **ipv4** Address Family whilst the IPv6 session allows only the **ipv6** Address Family. Finally, each of the IPv4 and IPv6 peering sessions has an **import-policy**, and an **export-policy** applied. When a newly configured EBGp neighbour is configured in Model-Driven mode, SR-OS applies the secure behaviour to reject all routes from being either imported or exported in compliance with [13]. The use of policy is therefore required to receive and advertise routes to and from an EBGp peer. On import the referenced policy 'RFC8212-allow-ebgp' simply has a **default-action** of **accept** for simplicity. The export policy 'VPN-IPv4/IPv6-to-EBGP' simply matches all routes in the VRF route-table learnt through **bgp-vpn** and allows them to be advertised into EBGp. Both import and export policies are shown in *Output 3-5* and override the default secure reject behaviour. If this default secure behaviour for importing and exporting routes is not desirable, the commands **ebgp-default-**

reject-policy export false and **ebgp-default-reject-policy import false** can be used at BGP, group, and neighbour level which simply allow routes to be imported and exported without explicit policy being applied.

Output 3-5: Route-Policies for BGP Neighbour Import and Export

```

policy-options {
  policy-statement "RFC8212-allow-ebgp" {
    default-action {
      action-type accept
    }
  }
  policy-statement "VPN-IPv4/v6-to-EBGP" {
    entry 10 {
      from {
        protocol {
          name [bgp-vpn]
        }
      }
      to {
        protocol {
          name [bgp]
        }
      }
      action {
        action-type accept
      }
    }
  }
}

```

With similar configuration applied at the four PE routers (R1, R3, R4, and R6) connecting the VPRN sites, *Output 3-6* shows the IPv4 route-table at router R1 for the IPv4 prefixes advertised by the CE routers. There are four routes, one from each of the advertising CEs, with one learned locally from BGP (CE1) and three learned remotely from BGP-VPN (CE3, CE4, and CE6). The routes that are learned remotely have Next-Hops of the corresponding PE routers (R3, R4, and R6) and are all resolved to SR-ISIS.

Output 3-6: IPv4 VRF Route-Table at router R1

```
A:admin@R1# show router 1 route-table 172.31.0.0/21 longer
```

```

=====
Route Table (Service: 1)
=====
Dest Prefix[Flags]                                Type   Proto   Age           Pref
  Next Hop[Interface Name]                        Metric
-----
172.31.1.0/24                                     Remote  BGP      01d02h09m    170
  192.168.100.2                                   0
172.31.3.0/24                                     Remote  BGP VPN   07h33m19s    170
  192.0.2.3 (tunneled:SR-ISIS:524301)             200
172.31.4.0/24                                     Remote  BGP VPN   07h33m19s    170
  192.0.2.4 (tunneled:SR-ISIS:524311)             100
172.31.6.0/24                                     Remote  BGP VPN   07h33m19s    170
  192.0.2.6 (tunneled:SR-ISIS:524323)             300
-----
No. of Routes: 4
Flags: n = Number of times nexthop is repeated
      B = BGP backup route available
      L = LFA nexthop available

```

S = Sticky ECMP requested

The BGP RIB-In can be viewed for each of the received VPN-IPv4 routes, and *Output 3-7* shows the RIB-In entry for prefix 172.31.3.0/24, truncated to show only the original attributes and not the post-policy modified attributes (in this example no attributes are modified). The `NextHop` is 192.0.2.3 (R3) and the resulting Next-Hop (`Res. Nexthop`) shows the IGP Next-Hop towards the BGP Next-Hop. This is shown as 192.168.0.2 which is the link towards R2. The `Route Dist.` field shows the configured Route-Distinguisher of 64496:1 and is used to prefix the IPv4 route to make a unique VPN-IPv4 route. The `VPN Label` is the advertised service label for this prefix. By default, SR-OS uses a label-per-VRF allocation mechanism, so any other IPv4/IPv6 prefixes advertised by R3 for this VRF will share the same label. The `Community` field shows the extended community Route-Target of target:64496:1 used to control prefix distribution for this VPN. The route was learned `From` 192.0.2.10 (RR1) and having come from a Route-Reflector the `Cluster` attribute is present using RR1's system address as is the `Originator Id` attribute. The `AS-Path` field contains a single AS, 64512, which would have been appended by CE3 when advertising the IPv4 prefix in EBGp towards R3. The last line shows the `VPRN Imported` field, and in this case the VPRN is 1. If the VPN-IPv4 route exists in RIB-In but the `VPRN Imported` field shows a value of None, then it is reasonably likely that the BGP Next-Hop cannot be resolved to an LSP of the protocol selected within the VPRN **auto-bind-tunnel** context.

Output 3-7: RIB-In entry for 172.31.3.0/24 from CE3

```
A:admin@R1# show router bgp routes 172.31.3.0/24 vpn-ipv4 detail
=====
BGP Router ID:192.0.2.1          AS:64496          Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                l - leaked, x - stale, > - best, b - backup, p - purge,
                w - unused-weight-only
Origin codes   : i - IGP, e - EGP, ? - incomplete
=====
BGP VPN-IPv4 Routes
=====
Original Attributes

Network       : 172.31.3.0/24
Nexthop       : 192.0.2.3
Route Dist.   : 64496:1          VPN Label       : 524283
Path Id       : None
From          : 192.0.2.10
Res. Nexthop  : 192.168.0.2
Local Pref.   : 100
Aggregator AS : None            Interface Name  : link-to-R2
Atomic Aggr.  : Not Atomic      Aggregator     : None
AIGP Metric   : None            MED            : None
Connector     : None            IGP Cost       : 200
Community     : target:64496:1
Cluster       : 192.0.2.10
Originator Id : 192.0.2.3        Peer Router Id  : 192.0.2.10
Fwd Class     : None            Priority        : None
Origin        : IGP
Flags         : Used Valid Best Group-Best
Route Source  : Internal
AS-Path       : 64512
```


Next Hop[Interface Name]	Metric		
-----	-----		
2001:db8:4001:100::/56	Remote	BGP	02d01h42m 170
2001:db8:3001:2101::2			0
2001:db8:4001:300::/56	Remote	BGP VPN	07h34m08s 170
192.0.2.3 (tunneled:SR-ISIS:524301)			200
2001:db8:4001:400::/56	Remote	BGP VPN	07h34m08s 170
192.0.2.4 (tunneled:SR-ISIS:524311)			100
2001:db8:4001:600::/56	Remote	BGP VPN	07h34m08s 170
192.0.2.6 (tunneled:SR-ISIS:524323)			300
-----	-----		
No. of Routes: 4			
Flags: n = Number of times nexthop is repeated			
B = BGP backup route available			
L = LFA nexthop available			
S = Sticky ECMP requested			

Output 3-9 shows the RIB-In entry for the VPN-IPv6 prefix 2001:db8:4001:300::/56 from CE3 (R3), again truncated to show only the original attributes and not the modified attributes. The first field of interest is the NextHop which is ::ffff:192.0.2.3. This is an IPv4-mapped IPv6 address [14], which is a special form of IPv6 address containing an embedded IPv4 address that allows IPv6-enabled applications to work with IPv4 devices. It uses the prefix ::ffff:0:0/96 where the :ffff is followed by the IPv4 address in dotted decimal notation. In this case the Next-Hop embeds R3's system address of 192.0.2.3. As previously described, NLRI advertised in Multiprotocol BGP should have a Next-Hop of the same Address Family as the advertised NLRI, hence the use of the IPv4-mapped IPv6 address in this manner allows for IPv6 (VPN-IPv6) routes to be advertised over an IPv4 core. At the receiving PE, the embedded IPv4 address can simply be extracted and be used to resolve in the tunnel-table as shown in *Output 3-8*. The remainder of the fields in the output are similar to the VPN-IPv4 prefix discussed above and are therefore not repeated here.

Because the IPv4 address can be mapped into the IPv6 address this works when carrying IPv6 over an IPv4 core, but how about carrying IPv4 over an IPv6 core? This involves protocol extensions and is discussed in Chapter 13 (*Advertising IPv4 Reachability over an IPv6 Core*).

Output 3-9: RIB-In entry for VPN-IPv6 prefix 2001:db8:4001:300::/56 from CE3

```
A:admin@R1# show router bgp routes 2001:db8:4001:300::/56 vpn-ipv6 detail
=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP VPN-IPv6 Routes
=====
Original Attributes

Network       : 2001:db8:4001:300::/56
Nexthop       : ::ffff:192.0.2.3
Route Dist.   : 64496:1                VPN Label      : 524283
Path Id       : None
From          : 192.0.2.10
Res. Nexthop  : 192.168.0.2
Local Pref.   : 100                    Interface Name : link-to-R2
```

Aggregator AS	: None	Aggregator	: None
Atomic Aggr.	: Not Atomic	MED	: None
AIGP Metric	: None	IGP Cost	: 200
Connector	: None		
Community	: target:64496:1		
Cluster	: 192.0.2.10		
Originator Id	: 192.0.2.3	Peer Router Id	: 192.0.2.10
Fwd Class	: None	Priority	: None
Origin	: IGP		
Flags	: Used Valid Best Group-Best		
Route Source	: Internal		
AS-Path	: 64512		
Route Tag	: 0		
Neighbor-AS	: 64512		
DB Orig Val	: N/A	Final Orig Val	: N/A
Source Class	: 0	Dest Class	: 0
Add Paths Send	: Default		
Last Modified	: 07h39m58s		
VRPN Imported	: 1		

Figure 3-4 shows a packet capture for the same VPN-IPv6 prefix, the encoding of which is similar to that of the previously described VPN-IPv4 prefix. Note however that the AFI/SAFI in this case is 2/128, where the AFI of 2 represents IPv6.

Figure 3-4: VPN-IPv6 Update message

```

* Border Gateway Protocol - UPDATE Message
  Marker: ffffffffffffffffffffffffffffffff
  Length: 120
  Type: UPDATE Message (2)
  Withdrawn Routes Length: 0
  Total Path Attribute Length: 97
  * Path attributes
    * Path Attribute - MP_REACH_NLRI
      * Flags: 0x90, Optional, Extended-Length, Non-transitive, Complete
      * Type Code: MP_REACH_NLRI (14)
      * Length: 48
      * Address family identifier (AFI): IPv6 (2)
      * Subsequent address family identifier (SAFI): Labeled VPN Unicast (128)
      * Next hop: RD=8:0 IPv6=:ffff:192.0.2.3
      * Number of Subnetwork points of attachment (SNPA): 0
      * Network Layer Reachability Information (NLRI)
        Label Stack=524283 (bottom) RD=64496:1, IPv6=2001:db8:4001:300::/56
    * Path Attribute - ORIGIN: IGP
    * Path Attribute - AS_PATH: 64512
    * Path Attribute - LOCAL_PREF: 100
    * Path Attribute - ORIGINATOR_ID: 192.0.2.3
    * Path Attribute - CLUSTER_LIST: 192.0.2.10
    * Path Attribute - EXTENDED_COMMUNITIES
      * Flags: 0xc0, Optional, Transitive, Complete
      * Type Code: EXTENDED_COMMUNITIES (16)
      * Length: 8
      * Carried extended communities: (1 community)
        * Route Target: 64496:1 [Transitive 2-Octet AS-Specific]
          * Type: Transitive 2-Octet AS-Specific (0x00)
          * Subtype (AS2): Route Target (0x02)
          * 2-Octet AS: 64496
          * 4-Octet AN: 1

```

With the control plane in place and both IPv4 and IPv6 routes advertised between the VPN sites, the data-plane is verified with a successful ping from CE1 to CE3 for both IPv4 and IPv6.

Output 3-10: Ping IPv4 and IPv6 from CE1 to CE3

```

*A:ce1# ping 172.31.3.1 source 172.31.1.1
PING 172.31.3.1 56 data bytes
64 bytes from 172.31.3.1: icmp_seq=1 ttl=62 time=9.81ms.
64 bytes from 172.31.3.1: icmp_seq=2 ttl=62 time=7.13ms.
64 bytes from 172.31.3.1: icmp_seq=3 ttl=62 time=6.22ms.
64 bytes from 172.31.3.1: icmp_seq=4 ttl=62 time=6.87ms.
64 bytes from 172.31.3.1: icmp_seq=5 ttl=62 time=6.15ms.

---- 172.31.3.1 PING Statistics ----

```

```

5 packets transmitted, 5 packets received, 0.00% packet loss
round-trip min = 6.15ms, avg = 7.24ms, max = 9.81ms, stddev = 1.34ms

*A:cel# ping 2001:db8:4001:300::1 source 2001:db8:4001:100::1
PING 2001:db8:4001:300::1 56 data bytes
64 bytes from 2001:db8:4001:300::1 icmp_seq=1 hlim=62 time=8.19ms.
64 bytes from 2001:db8:4001:300::1 icmp_seq=2 hlim=62 time=7.12ms.
64 bytes from 2001:db8:4001:300::1 icmp_seq=3 hlim=62 time=7.09ms.
64 bytes from 2001:db8:4001:300::1 icmp_seq=4 hlim=62 time=6.13ms.
64 bytes from 2001:db8:4001:300::1 icmp_seq=5 hlim=62 time=8.40ms.

---- 2001:db8:4001:300::1 PING Statistics ----
5 packets transmitted, 5 packets received, 0.00% packet loss
round-trip min = 6.13ms, avg = 7.39ms, max = 8.40ms, stddev = 0.824ms

```

Figure 3-5 shows a packet capture for both the IPv4 ICMP and IPv6 ICMPv6 Echo Requests. The top label of 14003 is the transport label and represents the segment routing Node-SID for R3. The bottom label (with bottom of stack=1) of 524283 is the service label advertised in both the VPN-IPv4 and VPN-IPv6 Update messages.

Figure 3-5: IPv4 and IPv6 ping from CE1 to CE3

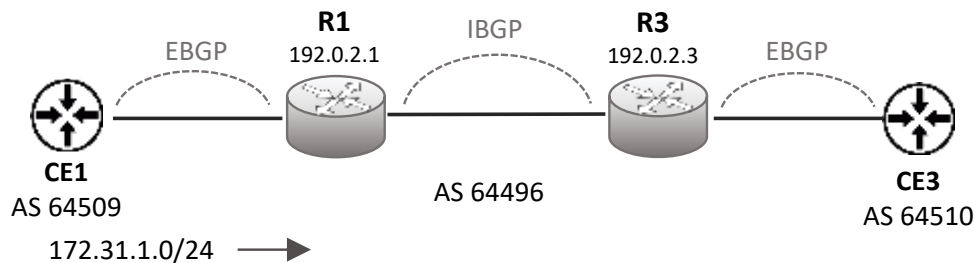
The figure consists of two screenshots of packet capture data. The top screenshot shows an IPv4 ICMP Echo Request (ping) from 172.31.1.1 to 172.31.3.1. It details the MultiProtocol Label Switching Header with two labels: a top label of 14003 (Node-SID for R3) and a bottom label of 524283 (Service Label). The bottom screenshot shows an IPv6 ICMPv6 Echo Request from 2001:db8:4001:100::1 to 2001:db8:4001:300::1, also showing the same MPLS label stack (14003 top, 524283 bottom).

AS_PATH Visibility

The **autonomous-system** number configured at BGP level is required if BGP is used as the PE-CE routing protocol and by default will be used as the source AS number in the Open exchange unless the **local-as** parameter is also configured, in which case the AS number defined as the **local-as** is used (this also applies to the use of **local-as** in the global BGP context). The use of the **local-as** parameter should be considered transitory and should only be used for the purpose of AS migration. The **local-as** parameter can be configured at BGP level, group level, or peer level, and when it is used, an AS number must also be present at VRF level. As Autonomous System re-numbers are still fairly common, it's worth understanding how it can be used.

Consider the topology in *Figure 3-6* where CE1 is in AS 64509 and advertises IPv4 prefix 172.31.1.0/24 to R1. Router R1 propagates the prefix to R3 in IBGP, which in turn propagates the prefix to CE3 in AS 64510. The AS_PATH seen at CE3 for prefix 172.31.1.0/24 with different configuration options is shown in *Table 3-1* and discussed below the figure.

Figure 3-6: AS_PATH Encoding



When the **local-as** is the same as the VRF-level **autonomous-system** number, there is no change to the AS_PATH encoding; a single AS number is imposed when advertising to EBGP peers and this is shown in row three. When the configured **local-as** and the VRF-level **autonomous-system** are different, by default both AS numbers will be imposed in the AS_PATH of routes advertised to EBGP peers as seen in row four. However, this behaviour can be modified by extending the **local-as** command to include setting the **prepend-global-as** parameter to **false** (by default it is set to **true**). In this scenario, only the **local-as** number is imposed in the AS_PATH as shown in row five.

Table 3-1: AS_PATH Encoding with VRF AS number and Local-AS options

Global ASN	R3 VRF ASN	R3 Local-AS	R3 (Local-AS) no-prepend-global-as	AS_PATH at CE3
64496	64496	None	N/A	64496 64509
64496	64497	None	N/A	64497 64509
64496	64497	64497	Disabled	64497 64509
64496	64497	64498	Disabled	64498 64497 64509
64496	64497	64498	Enabled	64498 64509

Prefix Dissemination

When a PE router belongs to a particular VPN it will learn some of that VPN's routes from attached CE routers using static or dynamic routing. These routes will be installed in the VRF associated with that CE router and will also be converted to VPN-IPv4/IPv6 routes for export into BGP such that other PE's belonging to that VPN can learn those routes. There are numerous methods through which these routes can be disseminated to other PE routers of the same VPN: some use an implicit flood model whilst others use an explicit send-only-if-required model. This section discusses the varying approaches that can be adopted for both prefix dissemination and route-table updates following a local policy change.

Automatic Route Filtering

To help scale BGP/MPLS based IP-VPNs PE routers do not retain prefixes in the RIB-In that are not associated with any configured VRFs by default. When a PE router receives a VPN-IPv4/IPv6 prefix with a Route-Target value that is not associated with any VRFs on that PE, the prefix is simply discarded (unless of course the PE is a Route-Reflector). This approach is known as Automatic Route Filtering (ARF) and ensures that PE routers only hold routes for VRFs that are explicitly configured on that PE. It is enabled by default and requires no configuration. In the example illustrated in *Debug 3-1* PE router R1 receives VPN-IPv4 prefix 64496:5:192.168.100.0/24 with extended community Route-Target 64496:5 but has no configured VRFs with that Route-Target value. It therefore silently discards the prefix as shown in *Output 3-11*. This approach uses an implicit flood model but relies on receiving PEs to not install routes that are not associated with any locally configured VRFs. ARF is the default behaviour in SR-OS.

Debug 3-1: VPN-IPv4 Prefix not explicitly configured at R1

```
81 2025/08/06 14:35:10.664 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 75
    Flag: 0x90 Type: 14 Len: 32 Multiprotocol Reachable NLRI:
        Address Family VPN_IPV4
        NextHop len 12 NextHop 192.0.2.3
        192.168.100.0/24 RD 64496:5 Label 524260 (Raw label 0x7ffe41)
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.3
    Flag: 0x80 Type: 10 Len: 4 Cluster ID:
        192.0.2.10
    Flag: 0xc0 Type: 16 Len: 8 Extended Community:
        target:64496:5
```

Output 3-11: ARF and resulting RIB-In

```
A:admin@R1# show router bgp routes 192.168.100.0/24 vpn-ipv4
=====
BGP Router ID:192.0.2.1          AS:64496          Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP VPN-IPv4 Routes
=====
Flag  Network                               LocalPref  MED
     Nexthop (Router)                       Path-Id    IGP Cost
     As-Path                                Label
-----
No Matching Entries Found.
```


Route Refresh

ARF is useful for optimising memory consumption, but what happens if a PE router's policy changes such that it now requires a VPN-IPv4/IPv6 prefix that it had previously discarded. What is needed is a mechanism to allow the PE router to re-evaluate all learned routes against the modified policy, and this is the purpose of Route Refresh. Route Refresh [10] capability is negotiated during the Open exchange and allows a BGP speaker to dynamically request a re-advertisement of the Adj-RIB-Out from a BGP peer. Once the peer re-advertises the Adj-RIB-Out it can be re-evaluated against the new policy. In SR-OS every time a VPRN import policy is modified, either through the route-policy framework or through modification of the VPRN **vrf-target** syntax, a Route Refresh is generated for the VPN-IPv4 and VPN-IPv6 Address Families as shown in *Debug 3-2*. This Route-Refresh is not specific to the VPRN being modified but rather will cause a Route-Refresh for all VPN-IPv4/IPv6 prefixes. Not shown for conciseness is the Adj-RIB-Out re-advertised to the speaker generating this refresh message.

Debug 3-2: Route-Refresh message for VPN-IPv4/IPv6 Address Families

```
1 2025/08/06 14:56:43.335 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: ROUTE REFRESH
Peer 1: 192.0.2.10 - Send BGP ROUTE REFRESH: Address Family AFI_IPV4: Sub AFI SAFI_VPN

2 2025/08/06 14:56:43.336 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: ROUTE REFRESH
Peer 1: 192.0.2.10 - Send BGP ROUTE REFRESH: Address Family AFI_IPV6: Sub AFI SAFI_VPN
```

When the Route Refresh message has been advertised, all triggered routes (in this case VPN-IPv4 and VPN-IPv6) are marked as 'stale' and must be refreshed within the system 'purge timer'. By default, this timer is set to 10 minutes and any prefixes not refreshed before the expiration of the purge timer are deleted. Forwarding of traffic for these routes held in 'purge' state is not impacted while the timer is running. There should be no requirement to modify this timer, but it is possible using the **purge-timer** command within the BGP context.

Route Refresh provides a dynamic way to refresh policy, but the method that it uses can be considered somewhat sub-optimal because the mechanism essentially says to its peer(s) "give me every prefix again and I will re-evaluate all of those prefixes against my new RIB-In policy". This consumes resources not only for the receiving PE, but also for the peer(s) that need to transmit the entire contents of the Adj-RIB-Out again. In environments with high IP-VPN provisioning activity, these Route Refreshes can be frequent and often overlapping (which causes the purge timer to be reset again) resulting in a high control plane load.

Route Refresh is the default method of prefix dissemination in SR-OS.

Outbound Route Filtering

Where the Route Refresh mechanism refreshes all prefixes of a given Address Family, a better approach could be to ask only for those prefixes a PE router knows that it needs, and this is the premise of Outbound Route Filtering (ORF) [11]. ORF allows a BGP speaker to send to a BGP peer a set of filters that the peer should apply on its Adj-RIB-Out.

ORF entries are carried within Route Refresh messages and are encoded as AFI/SAFI defining the NLRI type, an ORF-type defining the content of the ORF-value, an Action defining whether to add or remove the filter, and a match statement in the form of permit or deny (where a permit asks the peer to send the set of routes that match the signalled ORF entry). There are multiple ORF-types such as Address Prefix ORF, CP-ORF, VPN Prefix ORF, but SR-OS only supports the extended community (Route-Target) ORF-type. It is enabled under the main BGP process using the **outbound-route-filtering** context followed by the command **extended-community** and an option of either **accept-orf** or **send-orf**. In a typical MPLS/BGP IP-VPN environment involving Route Reflection, PE routers are configured to **send-orf** values, whilst Route-Reflectors are configured to **accept-orf** values. This causes the Route-Reflector(s) to apply filters to its Adj-RIB-Out such that only requested Route-Target values are advertised to its peers.

Output 3-12: ORF Configuration at PE router R1

```
router "Base" {
    bgp {
        outbound-route-filtering {
            extended-community {
                send-orf {
                }
            }
        }
    }
}
```

Once ORF is enabled, anytime the VPRN import policy is modified (again either through the route-policy framework, or through modification of the VPRN **vrf-target** syntax), Route Refresh messages are generated that firstly remove any existing filter policy and then apply the modified filter policy. The simple example shown in *Debug 3-3* shows a Route Refresh with ORF messages for AFI 1/SAFI 128 where the router wants to receive Updates for policy Route-Targets 64496:1, 64496:3, and 64496:4. The extended community is encoded as an 8-octet value where the first two octets (0.2) represent the Type and Sub-Type. The following two octets (251.240) represent the Global Administrator subfield denoted as an AS number (the AS number in use is AS 64496 which is represented in the debug output as 251.240, which in turn is hex 0xfb0 or 64496). The last four octets (0.0.0.x) represent the Local Administrator subfield, which are the VRF Route-Target values.

Debug 3-3: Route Refresh with ORF

```
229 2025/08/06 17:37:07.470 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: ORF
Peer 1: 192.0.2.10 - Send BGP (ROUTE_REFRESH) ORF: AFI 1, Sub AFI 128
    When-to-refresh: DEFER
    ORF Type: Extended Community
```

```
ORF Len: 1 Bytes
ORF Action: REMOVE-ALL
ORF Match: PERMIT

230 2025/08/06 17:37:07.470 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: ORF
Peer 1: 192.0.2.10 - Send BGP (ROUTE_REFRESH) ORF: AFI 1, Sub AFI 128
  When-to-refresh: IMMEDIATE
  ORF Type: Extended Community
  ORF Len: 27 Bytes
  ORF Action: ADD
  ORF Match: PERMIT
  Extended Community : 0.2.251.240.0.0.0.1
  ORF Action: ADD
  ORF Match: PERMIT
  Extended Community : 0.2.251.240.0.0.0.3
  ORF Action: ADD
  ORF Match: PERMIT
  Extended Community : 0.2.251.240.0.0.0.4
```

ORF allows for implementation of a more explicit ‘request only what you need’ model, but support of different ORF-types varies between vendors. For example, SR-OS supports extended community ORF-type, but other implementations have historically favoured Address-based ORF-type.

Soft Reconfiguration

One potential way to completely avoid the use of Route Refresh is using the so-called ‘soft reconfiguration’. Using soft reconfiguration, the router retains all the Multi-Protocol BGP prefixes that it receives regardless of whether they are imported into a VRF or not. For prefixes that are imported into a VRF, normal behaviour applies, but prefixes with Route-Target values that are not associated with any VRF on the router are retained in the RIB-In and marked as invalid/rejected. Thereafter, when local policy on the router is modified, it does not send any Route Refresh messages but simply scans the prefixes in the RIB-In against the modified policy.

Soft reconfiguration for MPLS/BGP IP-VPN is enabled with the **mp-bgp-keep true** command under the global BGP context. Because all Multiprotocol BGP routes are retained in the RIB-In, Route Refresh messages are no longer required and are no longer advertised, and this is the main advantage of the feature. The significant disadvantage is that memory is consumed by prefixes that the router doesn’t need. If memory is not an issue, soft reconfiguration is a good mechanism for reducing control plane activity.



Note: The **mp-bgp-keep** command blocks Route Refresh messages for VPN-IPv4/IPv6 and EVPN routes, but not for L2-VPN or Multicast routes.

Route Target Constraint

Constrained Route-Target distribution for BGP/MPLS IP-VPNs [15] builds on the concept of cooperative route filtering by propagating required Route-Target membership information. Route-Target membership information received by BGP speakers is then used to dynamically build outbound filters in order to propagate VPN-IPv4/IPv6 Update messages only to peers that have advertised the respective Route-Target, thus creating a controlled flooding distribution graph.

The ability to exchange Route-Target membership information is negotiated as a capability during the Open exchange. Route-Target membership NLRI is subsequently advertised using Multi-Protocol BGP (AFI 1, SAFI 132) and contains a prefix between 0 and 96 bits in length of the form {origin-as number, route-target}. A 'default' Route-Target is also defined, which is encoded as a zero-length prefix. The purpose of the default Route-Target is to indicate to a peer that this BGP speaker is willing to receive all VPN-IPv4/IPv6 prefixes and is used for example between a Route-Reflector and a client peer. After exchange of Route-Target membership information, a given VPN-IPv4/IPv6 prefix is only advertised to a peer if that peer has advertised either the default Route-Target membership NLRI, or a Route-Target membership NLRI containing any of the targets contained in the extended communities attribute of the VPN-IPv4/IPv6 prefix in question.

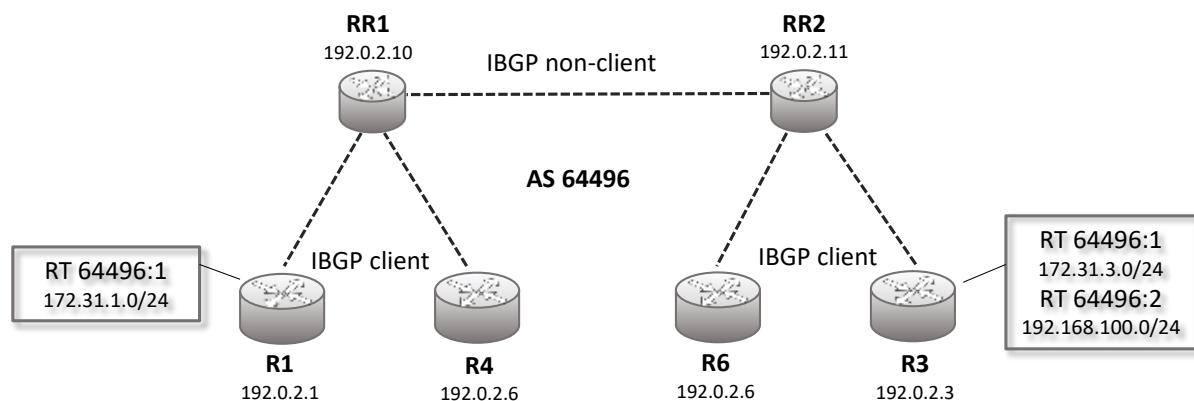
For inter-AS scenarios, the encoding of Route-Target membership information to include origin-as number allows for adoption of the conventional best-path selection algorithm (including AS_PATH) to build a distribution tree that includes only AS's that are either directly supporting a given VPN or are in the shortest inter-AS path between those AS's supporting that given VPN. This is based on the premise that Inter-AS BGP speakers select the best-path for the Route-Target Membership NLRI, and subsequently only advertise VPN-IPv4/IPv6 Updates in the inverse direction of that best-path distribution tree.

For intra-AS scenarios however, multiple routers may advertise the same {origin-as number, route-target} Route-Target Membership NLRI. In order to ensure the correct distribution tree is built, it is necessary to consider all available IBGP paths for a given RT prefix when building the outbound route filter, and not just the best path. In addition, when advertising Route-Target membership information sourced by the local AS to an IBGP peer, BGP attributes are modified as follows:

- a. When advertising Route-Target membership NLRI to a Route-Reflector client, the originator attribute should be set to the router-ID of the advertising router, and the Next-Hop attribute should be set to the local address of that IBGP session. This is so that the originator of a Route-Target Membership NLRI does not drop a Route-Target Membership NLRI reflected back to it and allows the Route-Reflector to use this Route-Target Membership NLRI to instruct the client what VPN routes it should advertise towards the Route-Reflector.
- b. When advertising Route-Target membership NLRI to a non-client peer, if the best-path is a route received from a non-client peer and an alternative path exists from a client, the attributes of the client path are advertised to the peer. This allows full propagation of Route-Target Membership NLRI throughout the IBGP mesh.

To illustrate the concept, consider the simplistic IBGP topology depicted in *Figure 3-7*. In this example, R1 and R4 are IBGP clients of RR1 whilst R3 and R6 are IBGP clients of RR2. RR1 and RR2 have a non-client peering session between them. Only R1 and R3 support IP-VPN services. R1 has a single VRF with Route-Target 64496:1 with prefix 172.31.1.0/24, whilst R3 has two VRFs; Route-Target 64496:1 with prefix 172.31.3.0/24 and RT 64496:2 with prefix 192.168.100.0/24.

Figure 3-7: RT Constraint example topology



To enable RT Constraint, all that is required from a PE perspective is to enable the **route-target** Address Family. This ensures that Route-Target Membership NLRIs are generated for all supported MPLS/BGP IP-VPN and EVPN services the router. From a Route-Reflector perspective, there is the optional additional step of signalling the default Route-Target to clients using the command **default-route-target** in order to receive all VPN-IPv4/IPv6 prefixes. An example of this configuration from RR1 is shown in *Output 3-13*.

Output 3-13: RT Constraint Configuration at RR1

```

bgp {
  group "CLIENTS" {
    peer-as 64496
    default-route-target true
    family {
      vpn-ipv4 true
      vpn-ipv6 true
      route-target true
    }
    cluster {
      cluster-id 192.0.2.10
    }
  }
  group "NON-CLIENT" {
    peer-as 64496
    family {
      vpn-ipv4 true
      vpn-ipv6 true
      route-target true
    }
  }
  neighbor "192.0.2.1" {
    group "CLIENTS"
    authentication-key "C3i5c6IP3VMhQFGff3RCM6O4qzEOtDo= hash2"
  }
  neighbor "192.0.2.4" {

```

```

        group "CLIENTS"
        authentication-key "C3i5c6IP3VMhQFGff3RCM5QVKE0RvXM= hash2"
    }
    neighbor "192.0.2.11" {
        group "NON-CLIENT"
        authentication-key "C3i5c6IP3VMhQFGff3RCM4OzeZYXAcU= hash2"
    }
}

```

With this topology and configuration, the Route-Target membership NLRI advertised/received by R1 can be observed. *Debug 3-4* shows the RT Membership NLRI generated towards RR1 with {origin-as 64496, route-target 64496:1}. This is followed by an End of RIB marker (empty NLRI) to indicate that the initial RT membership exchange is complete.

Debug 3-4: RT Constraint Membership NLRI at router R1

```

1026 2025/08/08 15:52:51.470 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 47
    Flag: 0x90 Type: 14 Len: 22 Multiprotocol Reachable NLRI:
        Address Family RTC_V4
        NextHop len 4 NextHop 192.0.2.1
        [RT-Const-V4] origin-as 64496, Target target:64496:1
    Flag: 0x40 Type: 1 Len: 1 Origin: 2
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100

1027 2025/08/08 15:52:51.470 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 7
    Flag: 0x90 Type: 15 Len: 3 Multiprotocol Unreachable NLRI:
        Address Family RTC_V4
        End-of-Rib marker (RTC_V4)

```

At R1 it is also possible to observe the RT Membership NLRI generated by the Route-Reflector RR1 and router R3. The first Update message contains the RT Membership NLRI generated by RR1 with the default route-target {origin-as 0, route-target 0.0:0:0}. The second Update message contains the RT Membership NLRI from R3 with {origin-as 64496, route-target 64496:1} and {origin-as 64496, route-target 64496:2}. Note that as described previously the Route-Reflector sets the Originator ID and Next-Hop attributes to itself.

Debug 3-5: RR1 and R3 RT Constraint Membership at router R1

```

1036 2025/08/08 15:52:51.664 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 42
    Flag: 0x90 Type: 14 Len: 10 Multiprotocol Reachable NLRI:
        Address Family RTC_V4
        NextHop len 4 NextHop 192.0.2.10
        [RT-Const-V4] origin-as 0, Target 0.0:0:0
    Flag: 0x40 Type: 1 Len: 1 Origin: 2
    Flag: 0x40 Type: 2 Len: 0 AS Path:

```

```

Flag: 0x80 Type: 4 Len: 4 MED: 0
Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.10

1037 2025/08/08 15:52:51.664 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 78
  Flag: 0x90 Type: 14 Len: 35 Multiprotocol Reachable NLRI:
    Address Family RTC_V4
    NextHop len 4 NextHop 192.0.2.10
    [RT-Const-V4] origin-as 64496, Target target:64496:1
    [RT-Const-V4] origin-as 64496, Target target:64496:2
  Flag: 0x40 Type: 1 Len: 1 Origin: 2
  Flag: 0x40 Type: 2 Len: 0 AS Path:
  Flag: 0x80 Type: 4 Len: 4 MED: 0
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
  Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.10
  Flag: 0x80 Type: 10 Len: 8 Cluster ID:
    192.0.2.10 192.0.2.11

```

In this very simple topology, it is possible to verify that the correct distribution tree has been built by checking how far the VPN-IPv4 prefix 192.168.100.0/24 with RT 64496:2 was propagated (i.e. its distribution tree). *Output 3-14* shows that at RR2 two VPN-IPv4 prefixes were received from router R3; prefix 172.31.3.0/24 with Route-Target 64496:1 and prefix 192.168.100.0/24 with Route-Target 64496:2. This is because RR2 advertised an RT Membership NLRI with the default route-target requesting all routes. However, as shown in *Output 3-15* RR2 does not propagate the VPN-IPv4 prefix 192.168.100.0/24 with Route-Target 64496:2 towards RR1 and only advertises the prefix 192.168.13.0/24 with Route-Target 64496:1.

Output 3-14: Route-Target 64496:2 propagation from R4

```

A:admin@rr2# show router bgp neighbor 192.0.2.3 vpn-ipv4 received-routes
=====
BGP Router ID:192.0.2.11      AS:64496      Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge
Origin codes  : i - IGP, e - EGP, ? - incomplete

=====
BGP VPN-IPv4 Routes
=====

```

Flag	Network Nexthop (Router) As-Path	LocalPref Path-Id	MED IGP Cost Label
*>i	64496:1:172.31.3.0/24 192.0.2.3 64512 No As-Path	100 None	None 300 524283 524283
*>i	64496:2:192.168.100.0/24 192.0.2.3 No As-Path	100 None	None 300 524261

```

-----
Routes : 2

```

Output 3-15: Route-Target 64496:2 propagation from R4

```

A:admin@rr2# show router bgp neighbor 192.0.2.10 vpn-ipv4 advertised-routes
=====
BGP Router ID:192.0.2.11      AS:64496      Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                  l - leaked, x - stale, > - best, b - backup, p - purge
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP VPN-IPv4 Routes
=====
Flag  Network                               LocalPref  MED
      Nexthop (Router)                      Path-Id     IGP Cost
      As-Path                               Label
-----
i     64496:1:172.31.3.0/24                  100        None
      192.0.2.3                             None        300
      64512                                 524283
-----
Routes : 1

```

If a BGP speaker no longer wishes to receive VPN-IPv4/IPv6 prefixes for a given Route-Target is simply withdraws the Route-Target membership NLRI using the MP_UNREACH_NLRI attribute. This triggers all peers to re-evaluate their RIB-OUTs to remove this Route-Target NLRI.

RT Constraint provides a powerful tool to build a prefix distribution tree. The effectiveness of the distribution tree largely depends on the BGP hierarchy and the sparseness of the VRFs deployed. RT Constraint with densely deployed VRFs on a pair of redundant Route-Reflectors essentially achieves the same function as ORF, so some thought needs to be given to the BGP topology and service base before considering deployment of RT Constraint.



Note: RT Constraint is fully supported for the VPN-IPv4, VPN-IPv6, L2VPN, MVPN, MDT-SAFI, and EVPN Address Families.

OSPF as a PE to CE Protocol

The predominant way for connected CE routers to exchange routing information with its adjacent PE router is BGP. However, there are scenarios where CE routers have a requirement to exchange routing information using OSPF. This may be because the administrators of that VPN have no experience with BGP, or because the CE routers are limited only to OSPF support, or simply because it is preferred for certain VPNs. The interaction between PE and CE and between OSPF and BGP is standardised in [80]. It requires that the PE runs a per-VRF instance of the OSPF routing protocol, which includes maintaining a link-state database, and installing routes into the VRF using the standard OSPF decision process. A PE installs a particular VPN-IPv4 route into a VRF, and if this is the preferred BGP path for the prefix then it is eligible for

redistribution into OSPF. In other words, only BGP routes that are in the route-table are eligible for redistribution into OSPF.

Domains and Route-Types

CE routers connecting to the VPN may be part of the same or different domains, and they may belong to the same or different areas. The PEs that form the VPN are considered the backbone area 0 (sometimes referred to as the super backbone), but this does not preclude CE routers from participating in area 0. The domain and area of a given CE router determines how routes received in VPN-IPv4 Updates are redistributed into OSPF:

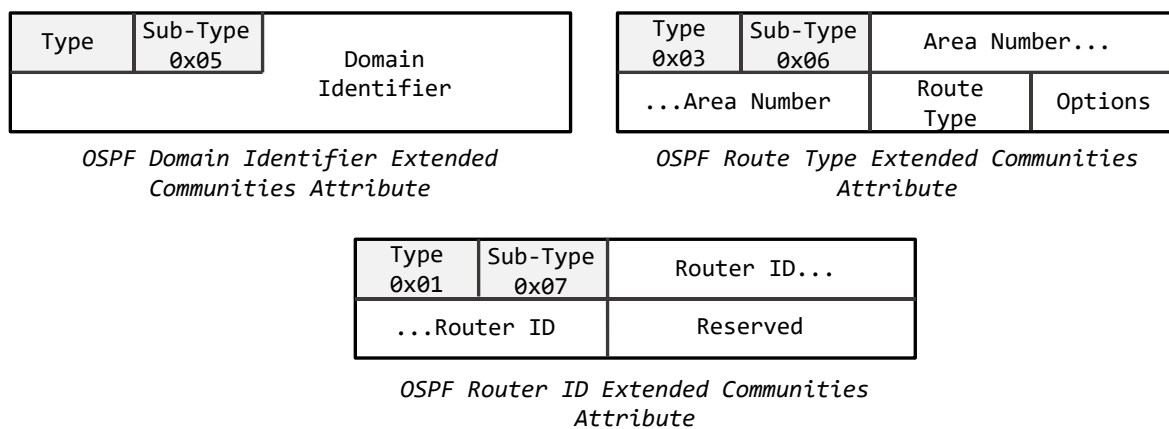
- If VPN sites are not in the same OSPF domain, then routes between those sites are re-advertised from BGP VPN into OSPF by the adjacent PE as Type 5 (External) LSAs. That is, the PE functions as an Autonomous System Border Router (ASBR). To allow a PE to determine a received BGP VPN route originated as an OSPF route, and to which OSPF domain it originated from, a domain identifier is encoded as an extended community (along with the OSPF route type and metric).
- If VPN sites are in the same OSPF domain, then routes between those sites are re-advertised from BGP VPN into OSPF by the adjacent PE as Type 3 (Inter-Area) LSAs. That is, the PE functions as an ABR. Again, the OSPF domain identifier extended community is used to determine that the sites belong to the same OSPF domain.
- If VPN sites are in the same domain and a route was originally advertised to a PE as an OSPF external route (Type 5) or an OSPF Not-So-Stubby-Area (NSSA) route (Type 7), it will be treated as an external route.
- AN OSPF Type 4 LSA (ASBR Summary LSA) received by a PE router should not be redistributed into BGP VPN-IPv4.

To allow a PE receiving a VPN-IPv4 route that was originated from OSPF to determine the source domain and area, three of extended community attributes are defined: the OSPF Domain Identifier Extended Communities attribute, the OSPF Route Type Extended Communities attribute, and the OSPF Router ID Extended Communities attribute. The intention of the three attributes is to be able to convert OSPF routes into VPN-IPv4 routes, distribute them across the BGP/MPLS IP-VPN core, and redistribute them back into OSPF with sufficient information to make it transparent to OSPF as though BGP had not been involved. The use of these Extended Communities is as follows:

- The OSPF Domain Identifier Extended Communities attribute is identified using the sub-type code 0x05 and is an optional attribute. Its purpose is to carry the OSPF instances primary domain identifier to identify how to redistribute the prefix when redistributing back from VPN-IPv4 into OSPF. If the domain identifier carried in the extended community is not the same as the locally configured domain identifier, the route is redistributed as an AS-external route. If the domain identifier carried in the extended community is the same as the locally configured domain identifier, the way the route is redistributed will depend on the Route Type extended community. If the OSPF instance has a non-NULL primary domain identifier this attribute must be present. If the OSPF instance has a NULL domain identifier, it may be omitted.

- The OSPF Route Type Extended Communities attribute is identified using sub-type code 0x06 and is a mandatory attribute. It contains a 4-byte Area Number where for AS-external routes the value is 0, and a non-zero value identifies the route as being internal to the OSPF domain. This is followed by a 1-byte Route Type which can be encoded as 1 or 2 for intra-area routes, 3 for inter-area routes, 5 for external routes (Area Number field must be zero), and 7 for NSSA routes. Lastly the 1-byte Options field is used only if the Route Type is 5 or 7, and if the least significant bit of the field is set indicates that the route carries a Type 2 metric.
- The OSPF Router ID Extended Communities attribute uses sub-type code 0x07 and is an optional attribute. It specifies the OSPF Router ID of the PE in the OSPF instance that installed the route into the VRF and advertised the route into VPN-IPv4.

Figure 3-8: OSPF Domain ID, Route Type, and Router ID Extended Communities



Backdoor Links and Sham Links

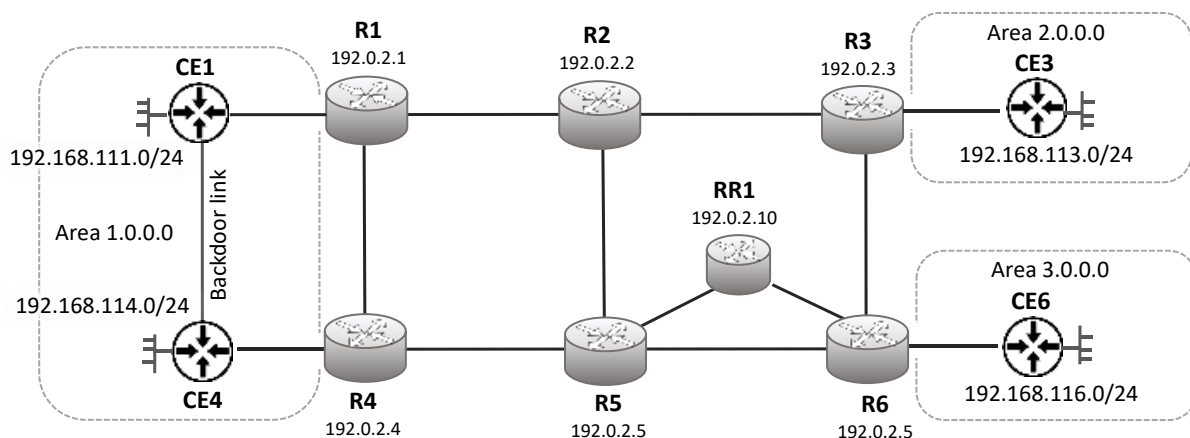
When using OSPF as the routing exchange mechanism for PE-CE routes, it may be that two sites of the VPN are connected using a link that also runs OSPF. The intention of this link is that it is only used when the VPN link is down and is referred to as a *backdoor* link. A backdoor link refers to a link between two sites of a VPN that are attached to different PE routers (for example, a multi-homed site). That is, CE1 is attached to PE1 and CE2 is attached to PE2, and a link exists between CE1 and CE2. Routes advertised between CE1 and CE2 may be advertised over the backbone link or across the VPN backbone. In the case of the latter, when the route is readvertised from VPN-IPv4 back into OSPF it will appear to the receiving CE router as an inter-area route. Hence the intra-area route from the backbone link will be preferred. If the customer would prefer to route such traffic over the VPN backbone, then it is necessary to make routes through the backbone appear as intra-area routes. This is the purpose of a *sham link*. A sham link is an optional feature that makes the Service Providers VPN network look like an intra-area link. That is, the VPN network appears as a link between CEs of the same OSPF domain and area. The sham link is a virtual link between two PEs of the same VRF each VRF associated with a 32-bit IPv4 sham link endpoint address. This sham link endpoint address must be advertised in VPN-IPv4 but cannot be advertised into the VRF OSPF instance. The sham link functions as an intra-area link that can be created for any area, including area 0. If the remote sham link endpoint address is not present in VPN-IPv4, the

sham link is considered down. When the sham link is operational, the participating PEs advertise the link into OSPF as a Type 1 Router LSA.

Configuration and Operation

The topology shown in *Figure 3-9* is used to demonstrate the use of PE-CE OSPF, and differs from *Figure 3-2* only in the way that the CE routers exchange routing information with their connected PE routers. CE1 and CE4 are connected to routers R1 and R4 respectively and have OSPF adjacencies with each of those PE routers, forming part of area 1.0.0.0. CE1 and CE4 also have a direct link between them that serves as a backdoor link that also participates in OSPF. CE3 has an OSPF adjacency to router R3 and is in area 2.0.0.0, while CE6 has an OSPF adjacency to R6 in area 3.0.0.0. All OSPF link metrics are configured as 100 including the attached LAN interfaces, with the exception of the CE1-CE4 backdoor link, which is configured with a metric of 200.

Figure 3-9: Test topology for PE-CE OSPF



The configuration for PE-CE OSPF at router R1 is shown in *Output 3-16*. The generic VPN service parameters have already been described and therefore are not repeated here. The **ospf 0** context provides the container for the OSPF configuration, where the **0** refers to the instance identifier as opposed to any reference to the area. A **router-id** is required for OSPF operation and in this case is the connected interface towards CE1. The **export-policy** command refers to the **policy-statement** shown in *Output 3-17*. As routes must be redistributed from VPN-IPv4 into OSPF this is a very simple policy that allows for this redistribution while tagging OSPF routes in the process. A tag is optional and may be used to identify routes advertised from VPN-IPv4 if required. It is not required for loop prevention as this is prevented using a different mechanism described later in this section. The **super-backbone** command indicates the type of LSA used when routes are redistributed into OSPF. When set to **true**, routes are redistributed as summary, external, or NSSA LSAs. When not enabled, routes are redistributed as external or NSSA LSAs only. As the objective is for the

core to appear ‘transparent’ to OSPF, it is enabled so that routes belonging to the same OSPF domain are redistributed as summary LSAs. The **vpn-domain** context allows for configuration of an **id** and **type** to identify the OSPF domain. The **type** command is a two-octet value to specify the extended community type and sub-type, and therefore the format of the extended community. The sub-type is always 05 for OSPF Domain Identifier. The type may be 00 for transitive two-octet AS-specific, 01 for transitive IPv4 Address-specific, 02 for transitive four-octet AS-specific, or 80 for generic transitive extended community. In this example, the type is 00 for transitive two-octet AS-specific extended community. The **id** field is simply an NSAP system ID and in this case uses the system address of R1. The same **vpn-domain id** is used on all PE routers; hence all sites are part of the same OSPF domain. Following this, the **area** context allows for configuration of generic OSPF parameters such as **interface-type**, which in this case is **point-to-point**, **authentication-type**, which uses MD5 authentication, and the OSPF **metric**, which as described is set to 100 for all PE-CE links.

Output 3-16: Configuration for PE-CE OSPF at R1

```

service {
    vprn "four" {
        bgp-ipvprn {
            mpls {
                ...etc
            }
        }
        interface "PE-to-CE" {
            ...etc
        }
        ospf 0 {
            admin-state enable
            router-id 192.168.84.1
            export-policy ["VPNIPv4-to-OSPF"]
            super-backbone true
            vpn-domain {
                id 1920.0000.2001
                type 0005
            }
            area 1.0.0.0 {
                interface "PE-to-CE" {
                    interface-type point-to-point
                    metric 100
                    authentication-type message-digest
                    message-digest-key 1 {
                        md5 "QjhhP/rLCjDpRb3n4dLf/dPseZxASkQ= hash2"
                    }
                }
            }
        }
    }
}

```

Output 3-17: Route-policy for redistribution of VPN-IPv4 to OSPF

```

policy-statement "VPNIPv4-to-OSPF" {
    entry 10 {
        from {
            protocol {
                name [bgp-vpn]
            }
        }
    }
}

```

```

    to {
        protocol {
            name [ospf]
        }
    }
    action {
        action-type accept
        tag 1001
    }
}
}

```

With similar configuration applied at all of the PE routers, adjacencies can be formed, and routes exchanged. *Output 3-18* shows the OSPF adjacency between R1 and CE1 with the state being **Full** to indicate that both routers have fully synchronised their link-state databases and are now considered fully adjacent neighbours.

Output 3-18: OSPF adjacency between R1 and CE1

```
A:admin@R1# show router 4 ospf neighbor
```

```

=====
Rtr vprn4 OSPFv2 Instance 0 Neighbors
=====
Interface-Name          Rtr Id          State    Pri  RetxQ  TTL
Area-Id
-----
PE-to-CE                192.168.111.1   Full     1    0      35
1.0.0.0
-----
No. of Neighbors: 1
=====

```

The prefix 192.168.111.0/24 at CE1 is learned through OSPF and is advertised into VPN-IPv4 as shown in *Debug 3-6*. The encoding of the Update is generic for VPN-IPv4 but includes the OSPF Domain ID and OSPF Route-Type Extended Communities. The Route-Type is encoded with an Area ID of 1, and a Route-Type of 1 to indicate that it was learned from a Type 1 Router LSA. The Domain-ID is encoded as two-octet AS-specific and therefore represents the configured **id** of 1920.0000.2001 as 6432:8193 (1920:2001). Although the encoding of the configured **id** is not entirely possible with a two-octet AS-specific extended community, providing all VPN sites of the same OSPF domain have the same value, they will be considered part of the same domain. One final point on the VPN-IPv4 Update is that the advertised MED value is derived from the OSPF metric to the prefix.

Debug 3-6: VPN-IPv4 route for prefix 192.168.111.0/24 from OSPF

```

1 2025/11/11 15:09:06.392 GMT minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 84
    Flag: 0x90 Type: 14 Len: 32 Multiprotocol Reachable NLRI:
        Address Family VPN_IPV4
        NextHop len 12 NextHop 192.0.2.1
        192.168.111.0/24 RD 64496:4 Label 524282 (Raw label 0x7fffa1)
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 200
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100

```

```

Flag: 0xc0 Type: 16 Len: 24 Extended Community:
  target:64496:4
  ospf-rt:1.0.0.0:1:0
  domain-id:6432:8193

```

A packet capture of the same VPN-IPv4 is shown in *Figure 3-10* to aid understanding of the encoding. The Update is largely the same as a generic VPN-IPv4 Update (shown in *Figure 3-3*) with the addition of the OSPF Domain ID and Route-Type Extended Communities. The former is a transitive two-octet AS-specific extended community type as defined by the **vpn-domain type** field, while the latter is a transitive opaque extended community type (non-configurable).

Figure 3-10: Packet capture of VPN-IPv4 Update with OSPF Extended Communities

```

= Border Gateway Protocol - UPDATE Message
  Marker: #
  Length: 107
  Type: UPDATE Message (2)
  Withdrawn Routes Length: 0
  Total Path Attribute Length: 84
  Path attributes
    = Path Attribute - MP_REACH_NLRI
      Flags: 0x98, Optional, Extended-Length, Non-transitive, Complete
      Type Code: MP_REACH_NLRI (14)
      Length: 32
      Address family identifier (AFI): IPv4 (1)
      Subsequent address family identifier (SAFI): Labeled VPN Unicast (128)
      Next hop: RD=0:0 IPv4=192.0.2.1
      Number of Subnetwork points of attachment (SNPA): 0
    = Network Layer Reachability Information (NLRI)
      = BGP Prefix
        Prefix Length: 112
        Label Stack: 524282 (bottom)
        Route Distinguisher: 64496:4
        MP Reach NLRI IPv4 prefix: 192.168.111.0
      Path Attribute - ORIGIN: IGP
      Path Attribute - AS_PATH: empty
      Path Attribute - MULTI_EXIT_DISC: 200
      Path Attribute - LOCAL_PREF: 100
      Path Attribute - EXTENDED_COMMUNITIES
        Flags: 0xc0, Optional, Transitive, Complete
        Type Code: EXTENDED_COMMUNITIES (16)
        Length: 24
        Carried extended communities: (3 communities)
          Route Target: 64496:4 [Transitive 2-Octet AS-Specific]
          OSPF Route Type: Area: 1.0.0.0, Type: Router [Transitive Opaque]
            Type: Transitive Opaque (0x03)
            Subtype (Opaque): OSPF Route Type (0x06)
            Area ID: 1.0.0.0
            Route type: Router (1)
            Options: 0x00 (Metric: Type-1)
              ... 0 = Metric type: Type-1
          OSPF Domain Identifier: 6432:8193 [Transitive 2-Octet AS-Specific]
            Type: Transitive 2-Octet AS-Specific (0x00)
            Subtype (AS2): OSPF Domain Identifier (0x05)
            2-Octet AS: 6432
            4-Octet AN: 8193

```

With all the routes exchanged between the VPN sites, *Output 3-19* shows the VRF route-table at router R1. The routes from CE3 (192.168.113.0/24) and CE6 (192.168.116.0/24) are known through protocol BGP VPN. The route from CE1 (192.168.111.0/24) is known through OSPF with metric 200, while the route from CE4 (192.168.114.0/24) is also known through OSPF with a metric of 400. This route learned in OSPF by CE1 through the backdoor link to CE4 and is thereafter advertised to R1. Consequently, router R1 learns CE4's prefix 192.168.114.0/24 through VPN-IPv4 (BGP VPN) with a preference of 170, and also through OSPF with a preference of 10. The route via CE1 is therefore preferred, and traffic from R1 to CE4 will transit through CE1. As the backdoor link is a low-speed link, this may not be the desirable traffic path for the customer, and therefore a sham link is required between R1 and R4.

Output 3-19: VRF route-table at R1

```

A:admin@R1# show router 4 route-table 192.168.96.0/19 longer

```

```

=====
Route Table (Service: 4)
=====
Dest Prefix[Flags]                                Type    Proto    Age          Pref
Next Hop[Interface Name]                        Metric
-----
192.168.111.0/24                                Remote  OSPF      19h06m05s    10
192.168.84.2                                    200
192.168.113.0/24                                Remote  BGP VPN    19h03m33s    170
192.0.2.3 (tunneled:SR-ISIS:524300)              200
192.168.114.0/24                                Remote  OSPF      19h09m23s    10
192.168.84.2                                    400
192.168.116.0/24                                Remote  BGP VPN    19h03m16s    170
192.0.2.6 (tunneled:SR-ISIS:524327)              300
-----
No. of Routes: 4
Flags: n = Number of times nexthop is repeated
       B = BGP backup route available
       L = LFA nexthop available
       S = Sticky ECMP requested
=====

```

As previously described, a sham link is used to create a virtual link between two PE routers that makes the VPN network look like a link in the same OSPF domain and area. The sham link endpoint addresses are VRF-specific and are advertised into VPN-IPv4 but not advertised into OSPF. If there is no VPN-IPv4 route to the remote endpoint address, there should be no reachability to that address, and the sham-link should be declared down. The sham link is conceptually the same as an OSPF virtual link used to connect a non-backbone area to the backbone area through a transit area. *Output 3-20* shows the configuration of the **sham-link** at router R1 with a similar configuration at R4. The **sham-link** endpoint address is configured as a loopback address with a 32-bit prefix mask. The loopback address is advertised into VPN-IPv4 (not shown) but is not advertised into OSPF. The **ospf area** context then allows for the **sham-link** configuration, which firstly references a locally configured interface. In this example it is the loopback interface representing the local **sham-link** endpoint. This is followed by the **ip-address** of the remote **sham-link** endpoint. In this example it is the IP address 192.168.80.254 which is a loopback address at router R4 also advertised into VPN-IPv4 but not OSPF. The **sham-link** then allows for some conventional parameters such as **metric** and **authentication-type**, **hello-interval**, and **dead-interval**.

Output 3-20: OSPF sham-link configuration at R1

```

service {
    vpn "four" {
        interface "loopback" {
            loopback true
            ipv4 {
                primary {
                    address 192.168.80.251
                    prefix-length 32
                }
            }
        }
    }
    ospf 0 {
        area 1.0.0.0 {
            sham-link "loopback" ip-address 192.168.80.254 {
                authentication-type message-digest
                metric 100
            }
        }
    }
}

```


configured between R1 and R4. The second two with link state ID 192.168.111.1 and 192.168.114.1 represent the prefixes advertised into OSPF by CE1 and CE4 respectively. This is followed by four Type 3 Summary LSAs: two for prefix 192.168.113.0 at CE3 and two for prefix 192.168.116.0 at CE6, with each prefix advertised by both R1 (advertising router ID 192.168.80.251) and R4 (advertising router 192.168.80.254). These four prefixes are Type 3 Summary LSAs that are readvertised from VPN-IPv4 into OSPF and are identified as belonging to the same OSPF domain courtesy of the OSPF Domain ID extended community. There are then a number of Type 5 External LSAs. The prefixes 192.168.84.x are the 30-bit subnets that are used for PE-CE interface addressing. The advertising PE router sees these as connected subnets (protocol direct) as opposed to a redistribution from OSPF, and as a result the OSPF Domain ID extended community is not attached to these VPN-IPv4 prefixes. In a similar vein, there are also External LSAs for the **sham-link** endpoint addresses 192.168.80.251 and 192.168.80.254. Again, these are seen as connected subnets and consequently do not carry the OSPF Domain ID.

Output 3-22: OSPF link-state database at R1

```
A:admin@R1# show router 4 ospf database

=====
Rtr vprn4 OSPFv2 Instance 0 Link State Database (type: All)
=====
Type      Area Id      Link State Id  Adv Rtr Id    Age  Sequence      Cksum
-----
Router    1.0.0.0      192.168.80.251 192.168.80.251 229  0x80000087 0xb7c
Router    1.0.0.0      192.168.80.254 192.168.80.254 69   0x80000055 0x515a
Router    1.0.0.0      192.168.111.1  192.168.111.1  83   0x80000110 0xcf42
Router    1.0.0.0      192.168.114.1  192.168.114.1  159  0x80000109 0xc442
Summary   1.0.0.0      192.168.113.0  192.168.80.251 524  0x8000000f 0xe78d
Summary   1.0.0.0      192.168.116.0  192.168.80.251 2014 0x8000000f 0x3f8d
Summary   1.0.0.0      192.168.113.0  192.168.80.254 1161 0x80000010 0xd39d
Summary   1.0.0.0      192.168.116.0  192.168.80.254 696  0x80000010 0x2b9d
AS Ext    n/a          192.168.80.254 192.168.80.251 1964 0x80000008 0xbb33
AS Ext    n/a          192.168.84.8   192.168.80.251 763  0x80000017 0x5d0
AS Ext    n/a          192.168.84.12  192.168.80.251 1709 0x80000015 0xe0f2
AS Ext    n/a          192.168.80.251 192.168.80.254 1880 0x80000008 0xfded
AS Ext    n/a          192.168.84.8   192.168.80.254 550  0x80000016 0x2ba5
AS Ext    n/a          192.168.84.12  192.168.80.254 205  0x80000016 0x3c9
-----
No. of LSAs: 14
=====
```

Attracting traffic over the **sham-link**, and subsequently over the VPN core, is determined by the metric setting of the link (versus the metric setting over the backdoor link). Recall from *Output 3-19* that prior to implementing the **sham-link**, R1's preferred path to 192.168.114.0/24 at CE4 was through OSPF via CE1 with a metric of 400. With the **sham-link** in place, *Output 3-23* again shows R1's route-table with the preferred path to the same prefix now known through BGP-VPN resolved through an SR-ISIS tunnel.

Output 3-23: VRF route-table at R1

```
A:admin@R1# show router 4 route-table 192.168.96.0/19 longer

=====
Route Table (Service: 4)
=====
```

Dest Prefix[Flags] Next Hop[Interface Name]	Type	Proto	Age Metric	Pref
192.168.111.0/24 192.168.84.2	Remote	OSPF	14h20m54s 200	10
192.168.113.0/24 192.0.2.3 (tunneled:SR-ISIS:524300)	Remote	BGP VPN	19h39m28s 200	170
192.168.114.0/24 192.0.2.4 (tunneled:SR-ISIS:524311)	Remote	BGP VPN	09h02m53s 100	170
192.168.116.0/24 192.0.2.6 (tunneled:SR-ISIS:524327)	Remote	BGP VPN	19h39m28s 300	170

No. of Routes: 4				
Flags: n = Number of times nexthop is repeated				
B = BGP backup route available				
L = LFA nexthop available				
S = Sticky ECMP requested				
=====				

When prefixes are advertised from VPN-IPv4 into OSPF, it's important to avoid a potential loop situation where the same prefix is subsequently advertised from OSPF back into VPN-IPv4. To achieve this, the Down bit (DN bit) in the LSA options field is set by a PE router whenever a route is redistributed from VPN-IPv4 into OSPF. Hence, if a CE router sends this Type 3 LSA to a different PE router, that PE router will not redistribute it further. If a PE router receives a Type 3, 5, or 7 LSA from a connected CE router with the DN bit set, the information contained in that LSA must not be used for OSPF route calculation. As a result, the LSA is not readvertised into a VPN-IPv4 route. This ensures that if a route derived from BGP VPN is redistributed into OSPF, it cannot be redistributed back into BGP as a VPN-IPv4 route.

The combination of the DN bit coupled with the OSPF Domain ID and Route-Type Extended Communities makes the VPN backbone transparent to different VPN sites. There is additional CPU/memory resource required on the PE routers to maintain a VRF-specific link-state database, but they must be balanced with the convenience and transparency for customers that favour OSPF.

Multi-AS Backbones (Inter-AS)

A Multi-As or Inter-AS IP-VPN is a VPN where the sites belonging to that VPN are connected to different Autonomous Systems and is typically used either to provide extended reach through a partnership/trust agreement, or as an interim means to interconnect Autonomous Systems of the same administrative domain following acquisition, or simply due to the internal organisation of a single Service Provider. Three methods of interconnecting Autonomous Systems are outlined in [7].

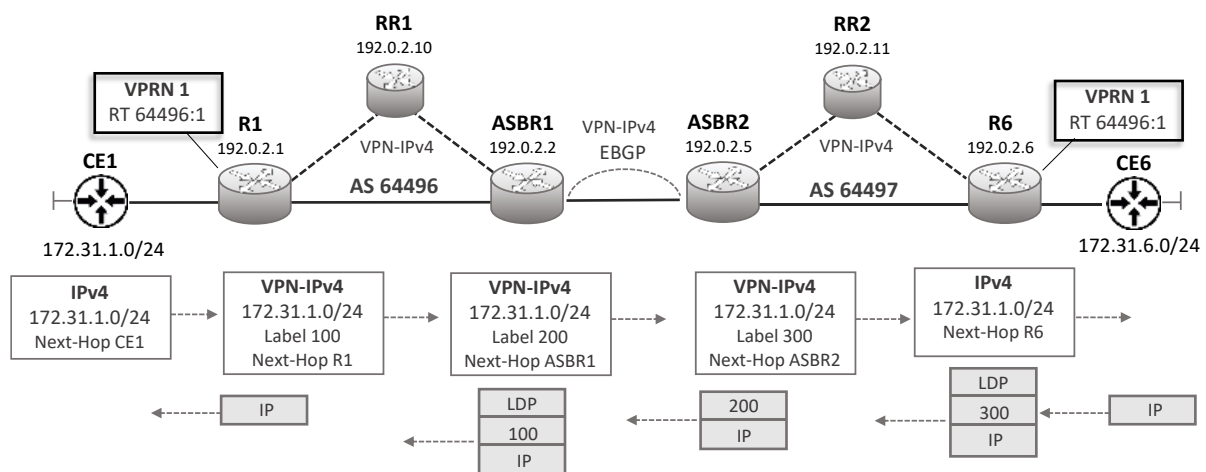
A 'Type A' interconnect consists of VRF-to-VRF connections at the Autonomous System Border Routers (ASBRs) and is commonly referred to as back-to-back VRFs. From the perspective of the local ASBR, each VRF-to-VRF connection (i.e. VLAN) looks and feels just like a PE to CE connection with route distribution between Autonomous Systems using IPv4/IPv6 BGP. Given the straightforward nature of its implementation, it does not require further discussion.

A 'Type C' interconnect involves EBGP redistribution of internal system addresses to the neighbouring AS using labeled unicast IPv4 routes. VPN-IPv4/VPN-IPv6 routes are thereafter advertised from PEs (or Route-Reflectors) in one AS to PEs (or Route-Reflectors) in the neighbouring AS using multi-hop EBGP. At the VPN-IPv4/IPv6 level the Next-Hop IP addresses remain intact, but in order to forward traffic that Next-Hop address must be recursively resolved to a labeled IPv4 route. Due to its inherent use of labeled IPv4 unicast routes, Type C interconnects are discussed in further detail in Chapter 7.

A 'Type B' interconnect involves EBGP redistribution of VPN-IPv4/VPN-IPv6 routes from AS to neighbouring AS. An ASBR learns VPN routes from within its own AS using IBGP (perhaps as a client of a Route-Reflector) and then uses EBGP to redistribute those labeled VPN routes to its adjacent ASBR. In the process of redistributing the routes into EBGP the ASBR imposes Next-Hop-Self on the VPN-IPv4/IPv6 Update and generates its own VPN label value when it advertises the Update upstream. This essentially means that the ASBR programmes a label-swap entry in its FIB and forwards traffic to the neighbouring ASBR using a single-level label stack (the VPN label). A key attribute of a Type B interconnect is that the ASBR does not need to have VRFs explicitly configured. This means however that the ASBR must have a mechanism to implicitly learn all VPN prefixes within its own AS and selectively advertise some of those prefixes to the neighbouring ASBR.

Figure 3-11 gives an example of the control plane and corresponding data plane used in Type B interconnects, and this same topology can also be used to illustrate the configuration requirements for Inter-AS Type B. Here CE1 is attached to R1 in AS 64496 and advertises prefix 172.31.1.0/24 which is propagated between neighbouring ASBRs to R6 in AS 64497, and then upstream to CE6. The label values are fictitious but are intentionally easy-on-the-eye to aid understanding.

Figure 3-11: Type B control and data plane example



There are no specific requirements on PE routers or Route-Reflectors to enable a Type B interconnect. There are however some specific configuration requirements on the ASBRs and *Output 3-24* illustrates an example of these taken from ASBR1. The **inter-as-vpn true**

command in the global BGP context does two significant things. Firstly, it allows the ASBR to implicitly accept all VPN routes regardless of Route-Target value into the RIB-In, thereby avoiding explicit configuration of VRFs (it is analogous to an ‘accept all’). These routes will be advertised to neighbouring ASBRs once they are considered valid, and subject to RIB-Out policy. Secondly, the ASBR attempts to resolve the BGP Next-Hop of these IBGP-received VPN routes to an MPLS LSP and considers a received VPN route as valid only after it has done so. If there is no LSP for the BGP Next-Hop, the route is considered invalid and is not advertised further by the ASBR.

By default, SR-OS uses LDP as a transport protocol to attempt to resolve these BGP Next-Hops, but this behaviour is configurable at global BGP level using the **next-hop-resolution labeled-routes transport-tunnel** context. Within this context, the **family** command is used to enable different tunnel selection criteria for VPN routes, and when entered opens a new context for the selected **family**. In this case, the selected family is **vpn** to encompass VPN-IPv4/IPv6 routes, however the **vpn** parameter will also include EVPN routes. Within the **family** context, the **resolution** command has options for **filter** or **any**. When **any** is selected, the system does a lookup in the tunnel-table and selects the tunnelling mechanism that is the most preferred (lowest numeric value). When **filter** is selected, the **resolution-filter** context is used to filter tunnelling protocols to only those that are permitted. If the requirement is to use only RSVP, then the filter list should contain only RSVP. If for example, both LDP and RSVP are configured, the system will use the preferred tunnel from that filtered list. As RSVP has a lower preference than LDP by default, it would be used in preference to LDP for binding Next-Hops to LSPs. As the example uses LDP, and LDP is the default configuration, the **labeled route transport-tunnel** context shown in *Output 3-24* would not be visible, however, it is included in this output for clarity.

Within the external group ‘EBGP’ context the **vpn-apply-import** and **vpn-apply-export** commands are set to **true** in order to allow global instance BGP import and export policies to be applied to VPN-IPv4/IPv6 routes. The corresponding import/export policies are implemented below that using the **import policy** and **export policy** commands whilst the associated route policies are shown in *Output 3-25*. The policies are simple in nature and are also symmetric but serve to illustrate how the policies can be applied. The policies should be completed with a **default-action action-type reject** not only to ensure that only the specified prefixes are advertised but also to ensure that MPLS resources are not unnecessarily consumed.

Output 3-24: Type B Interconnect configuration

```
router "Base" {
  autonomous-system 64496
  bgp {
    inter-as-vpn true
    next-hop-resolution {
      labeled-routes {
        transport-tunnel {
          family vpn {
            resolution filter
            resolution-filter {
              ldp true
            }
          }
        }
      }
    }
  }
}
```

```

    }
  }
}
group "EBGP" {
  vpn-apply-export true
  vpn-apply-import true
  peer-as 64497
  family {
    vpn-ipv4 true
  }
  import {
    policy ["ASBR-IMPORT"]
  }
  export {
    policy ["ASBR-EXPORT"]
  }
}
group "IBGP-Core" {
  peer-as 64496
  family {
    vpn-ipv4 true
  }
}
neighbor "192.0.2.10" {
  group "IBGP-Core"
  authentication-key "C3i5c6IP3VMhQFGff3RCM8pKOP5nZxU= hash2"
}
neighbor "192.168.0.14" {
  group "EBGP"
  authentication-key "JK/6NxQqPrX7NU7cD3w1PkmpZKMQDY8= hash2"
}
}

```

Output 3-25: ASBR VPN-IPv4 Route Policies

```

policy-options {
  community "from-AS64497" {
    member "target:64496:1" { }
  }
  community "to-AS64497" {
    member "target:64496:1" { }
  }
  policy-statement "ASBR-EXPORT" {
    entry 10 {
      from {
        community {
          name "to-AS64497"
        }
      }
      action {
        action-type accept
      }
    }
    default-action {
      action-type reject
    }
  }
  policy-statement "ASBR-IMPORT" {
    entry 10 {
      from {
        community {
          name "from-AS64497"
        }
      }
    }
  }
}

```

```

    }
  }
  action {
    action-type accept
  }
}
default-action {
  action-type reject
}
}
}

```



Note: The configuration example shown in *Output 3-24* uses Ethernet on the ASBR interconnect, and for this scenario the configuration shown above is sufficient. However, if Packet Over Sonet (POS) is used on the ASBR interconnect it requires that MPLS is enabled on the interface. The reason is that in order for SR-OS to be able to forward labeled packets over the interface MPLSCP has to be negotiated in PPP Network Control Protocol (NCP), and this requires the interface to be MPLS-enabled.

Referring again to *Figure 3-11*, ASBR1 has the responsibility of receiving VPN prefixes in IBGP and redistributing them into EBGp and vice versa. In the process of doing so it imposes Next-Hop-Self, generates a new label for the advertised Update message, and programmes a label-swap entry of the advertised label to the received label. When the ASBR forwards traffic into its own Autonomous System it is also necessary to push on a transport level label (i.e. in this example LDP) to reach a non-adjacent next hop. The received/advertised BGP labels are programmed on a {Next-Hop, Received Label} basis, as shown in *Output 3-26*.

Output 3-26: Inter-AS BGP labels at ASBR1

```
A:admin@ASBR1# show router bgp inter-as-label
```

```
=====
BGP Inter-AS labels
Flags: B - entry has backup, P - entry is promoted
=====
NextHop                Received      Advertised    Label
                        Label          Label         Origin
-----
192.0.2.1               524285        524283        Internal
192.168.0.14            524274        524284        External
-----
Total Labels allocated:  2
=====
```

The output above is not immediately straightforward to interpret so it's worthy of some explanation. Recall that R1 is advertising VPN prefix 172.31.1.0/24 into IBGP and that ASBR1 has policy that permits this route to be re-advertised in EBGp towards ASBR2 in AS 64497. A look at the Update message that ASBR1 received from R1 (192.0.2.1) for VPN-IPv4 prefix 172.31.1.0/24 in *Output 3-27* shows that the received label was 524285. Then looking at the Update that ASBR1 advertised towards ASBR2 for the same prefix in *Output 3-28* the advertised label is 524283. This accounts for the first row in *Output 3-26* together with an

indication that the label was originated internally (IBGP). The second row is the inverse operation for VPN-IPv4 prefix 172.31.6.0/24 sourced by R6 in AS 64497 and advertised towards ASBR1 by ASBR2 (192.0.2.14).

Output 3-27: ASBR1 received label from router R1

```
A:admin@ASBR1# show router bgp neighbor 192.0.2.10 received-routes vpn-ipv4
=====
BGP Router ID:192.0.2.2          AS:64496          Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP VPN-IPv4 Routes
=====
Flag  Network                               LocalPref  MED
      Nexthop (Router)                     Path-Id     IGP Cost
      As-Path                               Label
-----
*>i   64496:1:172.31.1.0/24                  100         None
      192.0.2.1                             None         100
      64512                                   524285
-----
Routes : 1
=====
```

Output 3-28: ASBR1 advertised label towards ASBR2

```
A:admin@ASBR1# show router bgp neighbor 192.168.0.14 advertised-routes vpn-ipv4
=====
BGP Router ID:192.0.2.2          AS:64496          Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP VPN-IPv4 Routes
=====
Flag  Network                               LocalPref  MED
      Nexthop (Router)                     Path-Id     IGP Cost
      As-Path                               Label
-----
i     64496:1:172.31.1.0/24                  n/a         None
      192.168.0.13                         None         100
      64496 64512                           524283
-----
Routes : 2
=====
```

By default, SR-OS uses a label-per-VRF mode of label distribution. As a result, if R1 were to advertise another VPN-IPv4 prefix belonging to the same VRF it would not consume any additional MPLS label resources at ASBR1 because the same label would be signalled and nothing changes in the {Next-Hop, Received Label} tuple. However, if another PE router were

added into the VPN and advertised another prefix (or even the same prefix), this would consume additional resources because the Next-Hop and label would be different. Equally, if a second VRF was configured at R1, this would consume resources because although the Next-Hop would be the same, the advertised label would be different.

4 Using BGP in VPLS

Prior to the conception of Ethernet VPN, it was possible to use BGP for auto-discovery and signalling of a VPLS, and there remains many deployments using this approach. In SR-OS the set-up, maintenance, and teardown of pseudowires forming a VPLS can be achieved using either an LDP or BGP control plane, although the functional behaviour of VPLS is the same across both models. The LDP control plane model deals exclusively with the pseudowire signalling aspects, whilst the BGP model [17] is broken into the sub-components of auto-discovery and pseudowire signalling, which can both be fulfilled within a single BGP Update message. The Auto-discovery and signalling sub-components are enabled independently in SR-OS, which leads to support of a further ‘hybrid’ model where auto-discovery is managed by BGP but pseudowire signalling is managed by LDP. Indeed, it’s even possible that within a single VPLS instance some parts of the service use LDP signalling and some use BGP signalling (for example a Hierarchical-VPLS implementation with a core using BGP and metro areas using LDP).

To identify other PE routers forming part of the VPLS (as part of the auto-discovery process) the Route-Target extended community [8] is used. The semantics of the use of Route-Targets is exactly the same as used for MPLS/BGP IP-VPN, and if a VPLS is fully meshed a single Route-Target will suffice. A VPLS NLRI is also introduced with L2VPN AFI (25) and VPLS SAFI (65) for the purpose of declaring VPLS membership and exchanging demultiplexers.

Figure 4-1: BGP VPLS NLRI

Length (2 octets)
Route Distinguisher (8 octets)
VE ID (2 octets)
VE Block Offset (2 octets)
VE Block Size (2 octets)
Label Base (3 octets)

To declare VPLS membership a PE router belonging to a given VPLS announces its VPLS NLRI with the relevant Route-Target and accepts VPLS NLRI from other PE routers that contain the same Route-Target. To exchange demultiplexers the fields VE ID, VE Block Offset, VE Block Size, and Label Base are used and the way that the demultiplexer is derived requires some explanation. When establishing a pseudowire between two endpoints an MPLS label is

exchanged that serves as a demultiplexer to identify traffic of that given pseudowire amongst a number of pseudowires that might be carried in a single (MPLS or GRE) tunnel. For a VPLS service that same demultiplexer requirement exists in order to do the following:

- a. Identify the specific VPLS instance to which the packet belongs for packet forwarding.
- b. Identify the ingress PE for the purpose of MAC learning and populating the VPLS Forwarding Database (FDB).

To facilitate MAC learning the demultiplexer label must be unique to a given ingress PE, and to achieve that the BGP VPLS control plane uses the concept of ‘label blocks’ defined by a label base (LB) and a VE block size (VBS), where SR-OS always uses a VBS of 8 for BGP-VPLS. The label base is a contiguous set of labels LB, LB+1, until the block reaches LB+VBS-1. When a PE advertises its VPLS NLRI to other PE routers in the same VPLS with a common label block, each receiving PE infers the label intended to be used for forwarding by adding its own unique VPLS Edge ID (VE ID) to the label base. Using this approach ensures that each receiving PE derives a unique demultiplexer that can be used at the advertising PE for both forwarding and MAC learning in the correct VPLS instance.

To assist in the administration of label blocks, it may be beneficial to have multiple smaller label blocks assigned to certain VE IDs (known as a ‘remote VE set’) instead of using a single larger label block assigned to all VE IDs in the VPLS. To achieve this, VE block offsets (VBOs) are used so that the label block becomes LB+VBO, LB+VBO+1, until the block reaches LB+VBO+VBS-1. A PE advertising VPLS NLRI may advertise multiple NRIs each with a different VBO and label block that defines the set of VE IDs, or remote VE set, that should use that block.

Suppose PE1 is part of a VPLS and sends a VPLS NLRI with VE ID ‘V’, VE Block Offset ‘VBO’, VE Block Size ‘VBS’, and label base ‘LB’. If PE2 is part of the same VPLS (determined by Route-Target) and has VE ID ‘W’ it implements the following to compute the label to be used:

- a. First PE2 verifies if VE ID W is part of PE1’s intended ‘remote VE set’. If $VBO \leq W < VBO + VBS$, W is part of PE1’s remote VE set. If not, PE2 ignores the message.
- b. If PE2 is part of PE1’s remote VE set it sets up a pseudowire to PE1 where the demultiplexer label to send traffic is computed as $(LB + W - VBO)$.

By way of example, assume PE25 advertises an Update message containing the following VPLS NLRI:

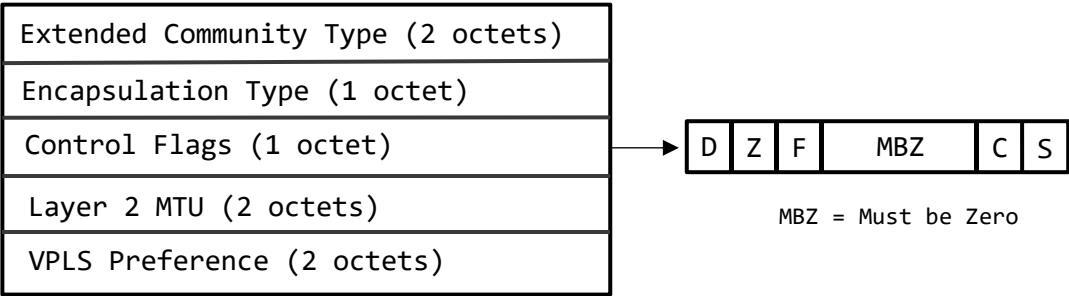
```
RD:64496:600, VE-ID=25, VBO=25, VBS=8, LB=131056
RD:64496:600, VE-ID=25, VBO=17, VBS=8, LB=131015
```

If PE24 with VE ID=24 is in the same VPLS (determined by Route-Target), PE24 must first determine if it is part of PE25’s remote VE set by identifying the NLRI for which $VBO \leq 24 < VBO + VBS$. This is the NLRI with label base 131015. PE24 must then set up a pseudowire with a demultiplexer label computed as $131015 (LB) + 24 (VE-ID) - 17 (VBO) = 131022$.

The BGP VPLS specification also uses the BGP extended community attribute “Layer2 Information extended community” for the purpose of signalling control information about

the pseudowires to be setup for a given VPLS. The extended community type/sub-type allocated by IANA is 0x800a and the Encapsulation Type is defined as value 19 for VPLS.

Figure 4-2: Layer 2 Information Extended Community



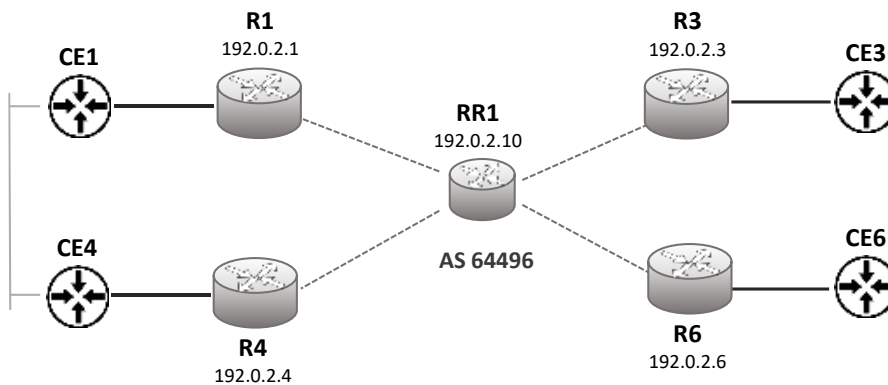
The Control Flags field contains a number of bits with the following meanings. The D and F bits are relevant to BGP multi-homing and will be discussed in further detail later in this chapter.

- C
- When C is set to 1, it defines that a Control word must be present when sending VPLS packets to this PE.
- S
- When S is set to 1, sequenced delivery of frames must be used when sending VPLS packets to this PE (not supported in SR-OS).
- D
- When D is set to 1, it indicates that all attachment circuits connecting a VPLS CE and PE are down.
- F
- Indicates when to flush MAC state. A designated forwarder must set the F-bit and a non-designated forwarder must clear the F-bit when sending BGP Multi-Homing advertisements. A state transition from 1 to 0 for the F-bit can be used by a remote PE to flush all the MACs learned from the PE that is transitioning from designated forwarder to non-designated forwarder.

The VPLS preference is used to control the selection of a particular site as Designated Forwarder (DF) when multiple PEs are assigned the same multi-homing Site ID. The VPLS preference field is specified because the well-known attribute LOCAL_PREF is non-transitive and therefore inadequate for Inter-AS operations. The higher VPLS preference value is preferred.

To illustrate the configuration requirements and control plane aspects of BGP-VPLS and its various models the simple logical topology depicted in *Figure 4-3* is used. In this figure R1, R3, R4, and R6 are all supporting sites of a VPLS instance. CE1, CE3, CE4, and CE6 form part of the same VPLS instance, with CE1 and CE4 representing a dual-homed site. BGP multi-homing is discussed later in this section as a mechanism to provide redundant access connectivity with a loop-free topology. All the PE routers are IBGP peered with RR1 in AS 64496.

Figure 4-3: BGP-VPLS Topology



Before discussing in some detail how BGP VPLS services are configured, the PE routers participating in the VPLS need to be configured to support the L2VPN Address Family for mutual exchange of VPLS NLRI. If this Address Family is being added it must be negotiated as a capability in an Open exchange. In short, the addition of the L2VPN Address Family causes the router to send a Notification message to the associated peers followed by an Open message containing the new capability.

Output 4-1: Addition of L2-VPN Address Family

```

router "Base" {
  bgp {
    group "IBGP" {
      peer-as 64496
      family {
        l2-vpn true
      }
    }
    neighbor "192.0.2.10" {
      group "IBGP"
      ...
    }
  }
}

```

BGP Auto-Discovery with LDP Signalling

For the purpose of BGP auto-discovery (AD) a VPLS requires two additional parameters, the VPLS-ID and the VSI-ID. The VPLS-ID is a unique network-wide identifier with the same value assigned for all VPLS Switch Instances (VSIs) belonging to the same VPLS and is encoded as a two-octet AS specific or IPv4 address-specific extended community attribute. This is the only auto-discovery parameter that requires explicit configuration and is defined using the **vpls-id** command in the **bgp-ad** node.

The VSI-ID is a unique identifier for each individual VSI and is carried in the VPLS NLRI. Where auto-discovery is used without BGP signalling a simplified version of the VPLS NLRI is used

where only the Route-Distinguisher field and the next 4-bytes (VE ID and VE Block Offset) are used to identify the VSI-ID (there is no requirement for Block Size and Label Base fields of the NLRI as LDP is used for signalling of demultiplexer labels). The VSI-ID requires no explicit configuration. The Route-Distinguisher is automatically derived from a concatenation of the global ASN and VPLS service ID, whilst the following 4-bytes are by default the system IP address but can be modified using the **vsi-id-prefix** syntax again in the **bgp-ad** node. When using BGP-AD the Route-Distinguisher and Route-Target values are automatically derived from the **vpls-id-prefix** configured under the **bgp-ad** node. If there is a requirement to change those values, the Route-target values and Route-Distinguisher can be explicitly configured under the **bgp** node of the VPLS service.

The minimum required configuration is shown in *Output 4-2*. Aside from the **vpls-id** already discussed, the other notable requirement is the **pw-template** configuration in the service context. This provides a template configuration defining the parameters of the pseudowire and contains local behavioural characteristics such as MAC-aging timers and path MTU, as well as signalled parameters such as **vc-type** and support for **control-word**. As can be seen, this example just contains a **split-horizon-group**. The **pw-template** is subsequently referenced within the **bgp** node of the VPLS service.

Output 4-2: BGP-AD with LDP signalling configuration

```

service {
    pw-template "1" {
        split-horizon-group {
            name "SHG"
        }
    }
    vpls "six" {
        admin-state enable
        service-id 6
        customer "1"
        bgp 1 {
            pw-template-binding "1" {
            }
        }
        bgp-ad {
            admin-state enable
            vpls-id "64496:6"
        }
        sap 1/1/c3/1:6 {
        }
    }
}

```

Once the **bgp-ad** node has been placed in an enabled state the auto-discovery process commences and the VPLS NLRI is advertised. *Debug 4-1* shows the VPLS NLRI generated by R3 as a result of the preceding configuration. As indicated above the BGP auto-discovery uses a simplified version of the VPLS NLRI containing Route-Distinguisher and system ID encoded into the VE ID and VE Block Offset fields, and this can be seen prefixed with [AD]. Also note that whilst the Layer2 Information extended community attribute is present it does not contain any Encapsulation Type field, Layer-2 MTU field, or Control flags field, or VPLS Preference.

Debug 4-1: VPLS NLRI for BGP-AD only

```

25 2025/08/16 08:38:24.832 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 81
    Flag: 0x90 Type: 14 Len: 23 Multiprotocol Reachable NLRI:
        Address Family L2VPN
        NextHop len 4 NextHop 192.0.2.3
        [AD] 192.0.2.3/32, RD 64496:6
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.3
    Flag: 0x80 Type: 10 Len: 4 Cluster ID:
        192.0.2.10
    Flag: 0xc0 Type: 16 Len: 16 Extended Community:
        target:64496:6
        l2-vpn/vrf-imp:64496:6

```

The attributes of the L2VPN VPLS NLRI received at a remote PE can be viewed using conventional RIB-In commands, but equally useful (if not more useful) is the service-level L2 route-table output shown in *Output 4-3*, not least because it contains the SDP ID of the pseudowire that was signalled in LDP as a result of the associated BGP Auto-Discovery Update message shown in *Output 4-4*. The pseudowire uses the Generalised PWid FEC Element (FEC 129) for pseudowire setup where the Address Group Identifier (AGI) can be used to signal membership information, and the Source Attachment Individual Identifier (SAII) and Target Attachment Individual Identifier (TAII) represent the VSI-IDs of the two PE routers forming the pseudowire.

Output 4-3: L2 Route-Table for BGP-AD

```

A:admin@R1# show service l2-route-table bgp-ad

=====
Services: L2 Route Information - Summary
=====
Svc Id      L2-Routes (RD-Prefix)          Next Hop      Origin
           Sdp Bind Id              PW Temp Id
-----
6           *64496:6-192.0.2.3             192.0.2.3    BGP-L2
           32767:4294967295             1
6           *64496:6-192.0.2.4             192.0.2.4    BGP-L2
           32765:4294967293             1
6           *64496:6-192.0.2.6             192.0.2.6    BGP-L2
           32766:4294967294             1
-----
No. of L2 Route Entries: 3
=====

```

Output 4-4: LDP SDPs signalled as a result of the BGP-AD NLRI

```

A:admin@R1# show service sdp

=====
Services: Service Destination Points
=====

```

SdpId	AdmMTU	OprMTU	Far End	Adm	Opr	Del	LSP	Sig
32765	0	9174	192.0.2.4	Up	Up	MPLS	L/B	TLDP
32766	0	9174	192.0.2.6	Up	Up	MPLS	L/B	TLDP
32767	0	9174	192.0.2.3	Up	Up	MPLS	L/B	TLDP

Number of SDPs : 4								

Legend: R = RSVP, L = LDP, B = BGP, M = MPLS-TP, n/a = Not Applicable								
I = SR-ISIS, O = SR-OSPF, T = SR-TE, F = FPE								
=====								

When a PE router no longer participates in the VPLS service an MP_UNREACH_NLRI is advertised containing the VPLS NLRI. This causes remote PEs to automatically commence pseudowire teardown by sourcing LDP Label Withdraw/Release messages.

Debug 4-2: BGP VPLS MP_UNREACH_NLRI

```
49 2025/08/16 09:00:16.227 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 21
    Flag: 0x90 Type: 15 Len: 17 Multiprotocol Unreachable NLRI:
        Address Family L2VPN
        [AD] 192.0.2.3/32, RD 64496:6
```

BGP Auto-Discovery and Signalling

BGP-VPLS for auto-discovery and signalling again requires that a **pw-template** is created and referenced in the **bgp** node of the VPLS service. The **bgp** node also provides the context for configuring **route-distinguisher** and **route-target** values which are signalled in the VPLS NLRI and extended community attributes respectively.

The **maximum-ve-id** command within the **bgp-vpls** context configures the maximum allowed value for the **ve id**; either configured locally or received in an incoming VPLS NLRI. Configuration of a **ve id** value higher than the **maximum-ve-id** value is blocked in CLI. If a VPLS NLRI is received with a **ve id** higher than the **maximum-ve-id**, the NLRI is dropped and an error message generated. The **ve-name** is a locally significant name for the VPLS instance, whilst the **ve id** is used to derive demultiplexer labels and is signalled as part of the VPLS NLRI. Note that before the **bgp-vpls** node can be placed into an enabled state a **route-distinguisher** value must be configured in the **bgp** context, or a **vpls-id** must exist in the **bgp-ad** node. *Output 4-5* shows the completed configuration at router R1.

Output 4-5: BGP-VPLS with AD and signalling

```
service {
    pw-template "1" {
        split-horizon-group {
            name "SHG"
        }
    }
    vpls "six" {
        admin-state enable
        service-id 6
    }
}
```

```

    customer "1"
    bgp 1 {
        route-distinguisher "64496:6"
        route-target {
            export "target:64496:6"
            import "target:64496:6"
        }
        pw-template-binding "1" {
        }
    }
    bgp-ad {
        admin-state enable
        vpls-id "64496:6"
    }
    bgp-vpls {
        maximum-ve-id 512
        ve {
            name "R1"
            id 1
        }
    }
    sap 1/1/c3/1:6 {
    }
}

```

Debug 4-3 shows the VPLS NLRI as received at R1 from R3. The notable difference between this VPLS NLRI and the one from *Debug 4-1* (where only Auto-Discovery was used) is that there are three different NRIs spread across three Update messages. The first Update contains the NLRI for Auto-Discovery and is effectively the same Update as shown in *Debug 4-1*. The second and third Update messages contain the NLRI for BGP VPLS encoding the Route-Distinguisher, VEID, VE block offset (VBO), VE block size (VBS), and the Label Base, with each Update containing different VE Block Offsets and a different Label Base. In SR-OS, a VPLS PE sends out a VPLS NLRI with a label block covering at a minimum its own **ve id**. If, however, it has received a VPLS NLRI containing other **ve id**'s it also advertises label blocks for them if additional label blocks are required. In this example routers R1, R3, and R6 are configured with **ve id**'s of 1, 3, and 6 respectively, but router R4 is configured with a **ve id** of 14. Consequently, R3 sends one VPLS NLRI with a VBO of 1 (which covers the **ve id**'s 1 and 6 for routers R1 and R6 respectively) and one VPLS NLRI with a VBO of 8 (which covers the **ve id** of 14 for R4).

Note also, that the Layer2 Information extended community contains the Encapsulation Type, MTU and Preference fields. It does not however contain any non-zero flags in the Control flags field. This will become important when Multi-homing is discussed in the next section.

Debug 4-3: BGP VPLS NLRI with AD and signalling

```

90 2025/08/18 09:16:06.819 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 81
    Flag: 0x90 Type: 14 Len: 23 Multiprotocol Reachable NLRI:
        Address Family L2VPN
        NextHop len 4 NextHop 192.0.2.3
        [AD] 192.0.2.3/32, RD 64496:6
    Flag: 0x40 Type: 1 Len: 1 Origin: 0

```

```

Flag: 0x40 Type: 2 Len: 0 AS Path:
Flag: 0x80 Type: 4 Len: 4 MED: 0
Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.3
Flag: 0x80 Type: 10 Len: 4 Cluster ID:
    192.0.2.10
Flag: 0xc0 Type: 16 Len: 16 Extended Community:
    target:64496:6
    l2-vpn/vrf-imp:64496:6

91 2025/08/18 09:16:06.820 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 86
    Flag: 0x90 Type: 14 Len: 28 Multiprotocol Reachable NLRI:
        Address Family L2VPN
        NextHop len 4 NextHop 192.0.2.3
        [VPLS/VPWS] preflen 17, veid: 3, vbo: 1, vbs: 8, label-base: 524253, RD 64496:6
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.3
    Flag: 0x80 Type: 10 Len: 4 Cluster ID:
        192.0.2.10
    Flag: 0xc0 Type: 16 Len: 16 Extended Community:
        target:64496:6
        l2-vpn/vrf-imp:Encap=19: Flags=none: MTU=1514: PREF=0

92 2025/08/18 09:16:08.819 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 86
    Flag: 0x90 Type: 14 Len: 28 Multiprotocol Reachable NLRI:
        Address Family L2VPN
        NextHop len 4 NextHop 192.0.2.3
        [VPLS/VPWS] preflen 17, veid: 3, vbo: 9, vbs: 8, label-base: 524227, RD 64496:6
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.3
    Flag: 0x80 Type: 10 Len: 4 Cluster ID:
        192.0.2.10
    Flag: 0xc0 Type: 16 Len: 16 Extended Community:
        target:64496:6
        l2-vpn/vrf-imp:Encap=19: Flags=none: MTU=1514: PREF=0

```

Output 4-6 shows the RIB-In at router R1 showing the VPLS NLRI received from R3, R4, and R6, and the associated dynamically created SDP IDs. *Output 4-7* shows the same SDP IDs within the VPLS service with the type 'BgpVpls' or 'BgpAd' together with the calculated (egress) demultiplexer labels.

Output 4-6: BGP VPLS L2-Route-Table

```
A:admin@R1# show service l2-route-table bgp-vpls
```

```

=====
Services: L2 Bgp-Vpls Route Information - Summary
=====
Svc Id      L2-Routes (RD)      Next Hop      Ve-Id

```


	Sdp Bind Id	PW Temp Id	
6	*64496:6	192.0.2.3	3
	32753:4294967276	1	
6	*64496:6	192.0.2.6	6
	32761:4294967286	1	
6	*64496:6	192.0.2.4	14
	32755:4294967278	1	
No. of L2 Bgp-Vpls Route Entries: 3			

Output 4-7: BGP-VPLS Service-Level SDPs

```
A:admin@R1# show service id 6 sdp
```

```
Services: Service Destination Points
```

SdpId	Type	Far End addr	Adm	Opr	I.Lbl	E.Lbl
32753:4294967276	BgpVpls	192.0.2.3	Up	Up	524261	524253
32754:4294967277	BgpAd	192.0.2.3	Up	Up	524269	524275
32755:4294967278	BgpVpls	192.0.2.4	Up	Up	524256	524233
32759:4294967282	BgpAd	192.0.2.4	Up	Up	524267	524265
32761:4294967286	BgpVpls	192.0.2.6	Up	Up	524264	524249
32766:4294967294	BgpAd	192.0.2.6	Up	Up	524268	524257

```
Number of SDPs : 6
```

When a PE router no longer participates in the VPLS service an MP_UNREACH_NLRI is advertised containing the VPLS NLRI. Other PEs participating in the VPLS receiving the UPDATE remove the originating PE from the VPLS and teardown any BGP-signalled SDPs.

Debug 4-4: BGP VPLS MP_UNREACH_NLRI for BGP AD and signalling

```
93 2025/08/18 10:05:06.799 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 59
  Flag: 0x90 Type: 15 Len: 55 Multiprotocol Unreachable NLRI:
    Address Family L2VPN
    [AD] 192.0.2.3/32, RD 64496:6
    [VPLS/VPWS] preflen 17, veid: 3, vbo: 9, vbs: 0, label-base: 0, RD 64496:6
    [VPLS/VPWS] preflen 17, veid: 3, vbo: 1, vbs: 0, label-base: 0, RD 64496:6
```

Multi-Homing with BGP-VPLS

When delivering business services, it is often a requirement for a Service Provider to give redundant connectivity to one or more sites, often referred to as 'multi-homing'. In the case of a VPLS service the service provider has a requirement to ensure that this multi-homed redundant access provides a loop-free topology to protect both the customer network and the Service Provider network infrastructure. The BGP-VPLS multi-homing solution detailed

in [19] is applicable to both BGP and LDP VPLS technologies and in the case of LDP VPLS can also be used without the use of the BGP Auto-Discovery solution. Using BGP to provide a loop-free multi-homing solution has significant benefits over other multi-homing technologies such as Spanning Tree Protocol or Multi-Chassis (active/standby) LAG because it is entirely transparent to the customer (there is no requirement for the CPE to run any control plane protocol), uses a standard and scalable protocol that is familiar to most if not all service providers, and in many cases just requires the addition of the L2-VPN Address Family if it is not already in use.

BGP multi-homing uses procedures for electing a Designated Forwarder (DF) among the PEs that are multi-homed to a customer site. The DF is responsible for forwarding traffic from the attachment circuit to the VPLS service and vice-versa. The non-DF does not forward any traffic thus creating a loop-free topology where only one attachment circuit is placed in forwarding state. The DF election process uses a Multi-Homed ID (MH-ID) to distinguish sites that are part of the same customer site, so the same MH-ID must be assigned on all VPLS PEs that are multi-homed to the same customer site. When two or more VPLS PEs advertise their NLRI with the same MH-ID they are identified as candidates for DF selection.

The MH-ID is carried in the VE-ID field of the VPLS NLRI shown in *Figure 4-1*, and when present the VE Block Offset, VE Block Size, and Label Base fields are all set to zero. In addition, each of the PE's supporting the multi-homed site must be provisioned with a unique Route-Distinguisher, which allows VPLS advertisements from different VPLS PEs to be distinct even if the advertisements have the same VE-ID. This is entirely possible where multi-homing is used.

The DF election algorithm is a two-stage process applicable to all VPLS NLRIs (not just MH NLRIs) that operates at both BGP and VPLS level. The first stage performs bucketisation of relevant and comparable advertisements, whilst the second stage picks a single winner from each bucket by repeatedly applying a tie-breaking algorithm on a pair of advertisements from that bucket. The result of the election is that any PEs elected as non-DF for a given Site-ID place their attachment circuits in non-forwarding state. Because a VPLS builds its forwarding tables based on the data-plane there is no explicit requirement to signal this non-forwarding state to the CPE, but SR-OS allows this state to be linked to a Y.1731 Maintenance End Point (MEP) on the attachment circuit and either suspend the generation of Continuity Check Messages (CCMs) or set the Interface-TLV to down towards the peer MEP. In addition to the MH-ID encoded in the VE-ID field of the MH NLRI, BGP multi-homing also uses the D and F bits of the Layer2 Information extended community to signal forwarding state. Their purpose was described earlier in this section, but some examples of these bits in use will be covered later in this section.

The topology shown in *Figure 4-3* is once again used to demonstrate the use of BGP multi-homing where R1 and R4 represent a dual-homed site consisting of CE1 and CE4. An example of the configuration applied at R1 is given in *Output 4-8* with a similar configuration applied at router R4, noting that each of the PE's supporting the multi-homed site must be provisioned with a unique Route-Distinguisher. Using different Route-Distinguishers allows the VPWS NLRI from different VPLS PEs to be distinct even if the advertisements have the

same VE-ID. The use of multi-homing also requires the provision of the **bgp-mh-site** object. The site context allows for provision of the **id** representing the MH-ID as well as other attributes such as the **boot-timer** and **activation-timer**. As previously described the site context must be the same across the PEs supporting a common multi-homed site. The site context also allows SAPs to be referenced that will be placed in forwarding or non-forwarding state depending on the outcome of the DF election. The **boot-timer** defines how long the system waits after a node reboot before executing the DF election algorithm. This delay is to allow for BGP sessions to be established and for NLRI to be exchanged so that the executing PE has received all relevant information before executing the algorithm. The **activation-timer** serves a similar function and is used to define a time period that the system will keep any local sites in standby status after they have become operational. Like the **boot-timer** the purpose is to allow some time to receive BGP Updates from remote PEs and therefore make a more informed decision when executing the DF election algorithm. Note that the **bgp-mh-site** "DUAL-HOME" in the below configuration is not put into an **admin-state** of **enable** at present. This is intentional and is done to make it possible to distinguish between VPLS and MH NLRI as they are described throughout the remainder of this section.

Output 4-8: BGP-VPLS with Multi-Homing Site

```

service {
    pw-template "1" {
        split-horizon-group {
            name "SHG"
        }
    }
    vpls "six" {
        admin-state enable
        service-id 6
        customer "1"
        bgp 1 {
            route-distinguisher "64496:6"
            route-target {
                export "target:64496:6"
                import "target:64496:6"
            }
            pw-template-binding "1" {
            }
        }
        bgp-ad {
            admin-state enable
            vpls-id "64496:6"
        }
        bgp-vpls {
            maximum-ve-id 512
            ve {
                name "R1"
                id 1
            }
        }
        bgp-mh-site "DUAL-HOME" {
            id 6
            boot-timer 120
            activation-timer 30
            sap 1/1/c3/1:6
        }
        sap 1/1/c3/1:6 {
        }
    }
}

```

```
}
}
```

Both R1 and R4 have the VPLS site context in an administratively disabled state when the VPLS is enabled and regular BGP VPLS NLRI is advertised as previously shown in *Debug 4-3*. The site context at R1 and R4 is subsequently enabled, and a MH NLRI is then sourced from both of the VPLS PEs. *Debug 4-5* shows the MH NLRI sourced from R1 where the VE Block Offset, VE Block Size, and Label Base fields are all set to zero and the VE-ID field carries the MH-ID of 6 configured at R1 and R4 under the **bgp-mh-site** context. In addition, the flags field of the Layer2 Information extended community shows that the DF bits are set, meaning that R1 is the elected DF.

Debug 4-5: MH NLRI sourced by R1

```
118 2025/08/18 12:55:16.323 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 72
  Flag: 0x90 Type: 14 Len: 28 Multiprotocol Reachable NLRI:
    Address Family L2VPN
    NextHop len 4 NextHop 192.0.2.1
    [MH] site-id: 6, RD 64496:6
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 0 AS Path:
  Flag: 0x80 Type: 4 Len: 4 MED: 0
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
  Flag: 0xc0 Type: 16 Len: 16 Extended Community:
    target:64496:6
    l2-vpn/vrf-imp:Encap=19: Flags=-DF: MTU=0: PREF=0
```

Using the service-level *l2-route-table* verification shown in *Output 4-9* it is possible to verify the presence of BGP VPLS NLRI from all PE routers in the VPLS domain bound to the associated BGP-signalled SDPs. In addition, note that R1 (192.0.2.1) is the DF for Site-ID 6 (DF bits set) while R4 (192.0.2.4) is the non-DF. As previously discussed, this means that R4 must place all attachment circuits associated with Site-ID 6 into a non-forwarding state. This can be observed in *Output 4-10* where the SAP is put into an operationally down state with the flag *'StandbyForMHPProtocol'*.

Output 4-9: Service-Level L2 route-table at R3

```
A:admin@R3# show service id 6 l2-route-table bgp-vpls multi-homing
```

```
=====
Services: L2 Multi-Homing Route Information - Summary
=====
```

Svc Id	L2-Routes (RD-Prefix)	Next Hop	SiteId	State	DF
6	64496:6	192.0.2.1	6	up(0)	set
6	64496:66	192.0.2.4	6	up(0)	clear

```
-----
No. of L2 Multi-Homing Route Entries: 2
=====

=====
Services: L2 Bgp-Vpls Route Information - Service 6
=====
```

L2-Routes (RD)	Next Hop	Ve-Id	Sdp Bind Id

```
-----
```

```
*64496:6          192.0.2.1          1          32736:4294967255
*64496:6          192.0.2.6          6          32735:4294967254
*64496:66         192.0.2.4          14         32732:4294967251
-----
No. of L2 Bgp-Vpls Route Entries: 3
=====
```

Output 4-10: R4 attachment circuit in non-forwarding state

```
A:admin@R4# show service id 6 sap "1/1/c3/1:6"
```

Service Access Points (SAP)

```
=====
Service Id       : 6
SAP              : 1/1/c3/1:6          Encap              : q-tag
Description      : (Not Specified)
Admin State      : Up                  Oper State           : Down
Flags            : StandByForMHProtocol
Multi Svc Site   : None
Last Status Change : 08/18/2025 12:54:47
Last Mgmt Change  : 08/18/2025 10:28:44
=====
```

To verify the switchover capabilities of the BGP multi-homing feature a failure can be simulated on the attachment circuit at the current DF, R1. Because the re-convergence times largely depend on BGP Update propagation times, SR-OS allows for the optional and configurable rapid propagation of BGP Updates (bypassing the conventional Minimum Route Advertisement Interval or MRAI timer) using the **l2-vpn true** command under the **rapid-update** context at global BGP level.

Output 4-11: Rapid-Updates for L2-VPN Address Family

```
router "Base" {
    bgp {
        rapid-update {
            l2-vpn true
        }
    }
}
```

When the attachment circuit is failed at the DF R1 it sources an MP_REACH MH NLRI as shown in *Debug 4-6* with the flags field of the Layer2 Information community set to D, but with the F bit clear. To avoid black-hole conditions remote BGP VPLS PEs must interpret DF bit transitions from 1 to zero as an implicit MAC flush-all-from-me indication. As a result, MAC addresses learnt from the transitioning VPLS PE are flushed and flooding/re-learning takes place until MAC tables are populated again. Router R1 also sends a second Update message containing an MP_UNREACH_NLRI for the MH-Site NLRI.

Debug 4-6: Attachment circuit failure at elected DF

```
119 2025/08/18 13:27:50.424 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 72
    Flag: 0x90 Type: 14 Len: 28 Multiprotocol Reachable NLRI:
        Address Family L2VPN
```

```

      NextHop len 4 NextHop 192.0.2.1
      [MH] site-id: 6, RD 64496:6
      Flag: 0x40 Type: 1 Len: 1 Origin: 0
      Flag: 0x40 Type: 2 Len: 0 AS Path:
      Flag: 0x80 Type: 4 Len: 4 MED: 0
      Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
      Flag: 0xc0 Type: 16 Len: 16 Extended Community:
        target:64496:6
        l2-vpn/vrf-imp:Encap=19: Flags=D: MTU=0: PREF=0

121 2025/08/18 13:27:51.891 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 26
  Flag: 0x90 Type: 15 Len: 22 Multiprotocol Unreachable NLRI:
    Address Family L2VPN
    [MH] site-id: 6, RD 64496:6

```

The BGP Update from R1 indicating a transition to non-DF (attachment circuit down) means that R4 which forms part of the same Site-ID must transition to DF. As a result, R4 sources an MH NLRI with the DF bits of the flags field in the Layer-2 Information community set as shown in *Debug 4-7*.

Debug 4-7: Transition to DF at router R4

```

120 2025/08/18 13:27:51.890 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 86
  Flag: 0x90 Type: 14 Len: 28 Multiprotocol Reachable NLRI:
    Address Family L2VPN
    NextHop len 4 NextHop 192.0.2.4
    [MH] site-id: 6, RD 64496:6
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.4
    Flag: 0x80 Type: 10 Len: 4 Cluster ID:
      192.0.2.10
    Flag: 0xc0 Type: 16 Len: 16 Extended Community:
      target:64496:6
      l2-vpn/vrf-imp:Encap=19: Flags=-DF: MTU=0: PREF=0

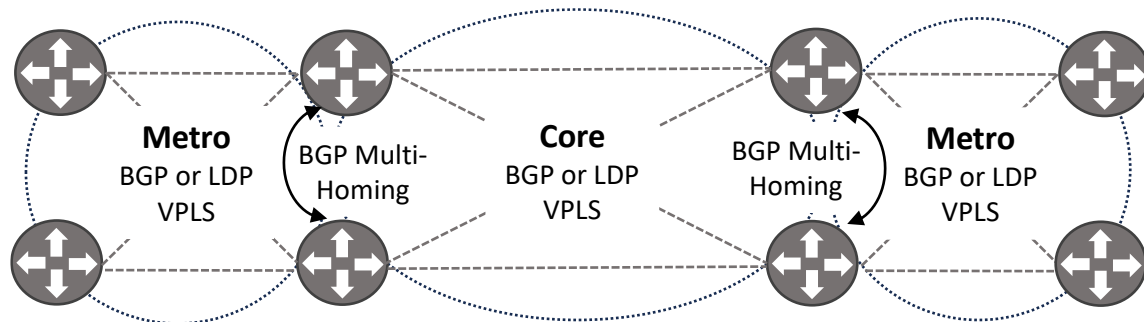
```

Note that once the attachment circuit at R1 recovers, the restoration is revertive and that this behaviour cannot be changed.

The topology used for the purpose of illustrating the BGP multi-homing process contained a single SAP at each of the multi-homing VPLS PEs and as such it represented a binary up/down condition for the Site-ID. However, what would happen if R1 and R4 had more than one SAP belonging to Site-ID 6 and at the DF, one of those SAPs failed? This would leave a situation where some of the SAPs of the Site-ID are operationally up and one of the SAPs of the Site-ID is operationally down. The action taken during this kind of scenario is controllable using the **failed-threshold** command in the **bgp-mh-site** context to indicate a number of objects [1..1000] that should be down in order for the site to be declared down.

The use of BGP multi-homing detailed in this section used redundantly connected SAPs to illustrate the concept of Site-ID and DF election to provide a redundant loop-free topology. It is, however, equally possible to use SDPs within the context of a site meaning that BGP multi-homing can also be used in a Hierarchical VPLS topology as shown in *Figure 4-4*. An example of this is to interconnect fully-meshed metro VPLS domains through a core VPLS domain, where each metro VPLS domain is represented as a multi-homed site to the core thus creating a loop-free topology.

Figure 4-4: Hierarchical VPLS with BGP Multi-Homing



5 BGP Signalling for VPWS

The widely adopted method for set-up, maintenance, and teardown of pseudowires uses LDP for signalling as defined in RFC4447. However, it is equally possible to use BGP for auto-discovery and signalling of point-to-point L2-VPNs using much of the mechanics previously described for VPLS together with some additional concepts from [20]. Whilst it is intended that individual sections of this book are readable within their own right, the overlap with BGP-VPLS is significant and many of the concepts are not repeated within this section. Therefore, readers of this section are advised to be familiar with the contents of Chapter 4.

The SR-OS implementation of BGP VPWS supports Ethernet pseudowires and provides for a redundant access capability using BGP Multi-Homing [19] where there are options to establish either a single pseudowire to the DF of a Site-ID, or by signalling two pseudowires to the DF/non-DF in an active/standby arrangement. The active/standby method provides for faster reconvergence upon failure than the single pseudowire method, primarily because the standby pseudowire is pre-signalled. The single pseudowire method can be considered a subset of that capability and aligns with the implementations supported by other vendors.

The VPLS NLRI with AFI (25) and VPLS SAFI (65) used for auto-discovery and signalling of VPLS services is extended for the VPWS NLRI to include a Circuit Status Vector (CSV) encoded as a TLV. The Type field of this TLV is 1 and the value field uses a bit-vector where each bit indicates the status of the Permanent Virtual Circuit (PVC) associated with the corresponding label in the label-block. For VPWS signalling SR-OS always signals a label block size of 1; hence only the high order bit is used and when set to 0 indicates that the local attachment circuit and pseudowire to the remote PE is up, whilst a value of 1 indicates that either or both of them are down. The use of the remaining fields is as previously described in Chapter 4.

Figure 5-1: VPWS NLRI including Circuit Status Vector

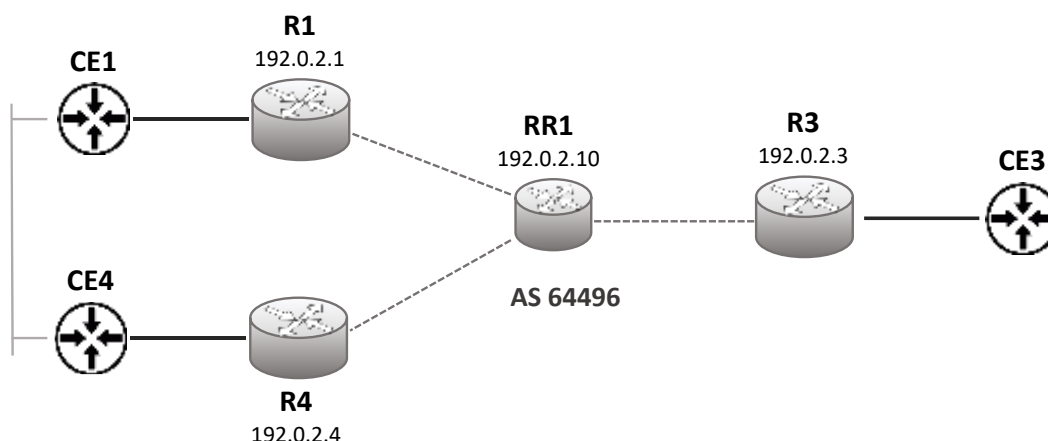
Length (2 octets)
Route Distinguisher (8 octets)
VE ID (2 octets)
VE Block Offset (2 octets)
VE Block Size (2 octets)
Label Base (3 octets)
Circuit Status Vector (4 octets)

The Encapsulation Type field of the Layer-2 Information extended community (*Figure 4-2*) is also changed for the purpose of VPWS to allow for Ethernet VLAN (value 4) and Ethernet Raw (value 5).

To illustrate the use of BGP VPWS the logical topology shown in *Figure 5-2* is used and shows a single homed and dual-homed scenario. First the simplest scenario of a non-redundant

single-homed pseudowire between R1 and R3 is described. Thereafter, an example of a multi-homed pseudowire between R1/R4 (representing a multi-homed site) and R3 is provided; first using a single pseudowire model and then using an active/standby pseudowire model.

Figure 5-2: BGP VPWS test topology



If PE routers have not already been configured to support the L2VPN Address Family, it must be added to allow for mutual exchange of L2VPN NLRI. As previously described, if this Address Family is being added to an existing BGP session, it must be negotiated as a capability in an Open message and therefore triggers a Notification/Open exchange.

Output 5-1: Addition of L2-VPN Address Family

```

router "Base" {
  bgp {
    group "IBGP" {
      peer-as 64496
      family {
        l2-vpn true
      }
    }
    neighbor "192.0.2.10" {
      group "IBGP"
      ...
    }
  }
}

```

Single-homed VPWS

Output 5-2 shows the basic provisioning requirement for a single-homed BGP VPWS as configured at R3 with a corresponding configuration at R1. The VPWS uses an SR-OS **epipe** service, and as with a BGP VPLS, BGP VPWS requires that a **pw-template** is created to define the characteristics of the pseudowire. This **pw-template** is then referenced in the **bgp 1** context of the **epipe** service to dynamically create a pseudowire with those characteristics. The **bgp 1** node also provides the context for configuration of **route-distinguisher** and

route-target values which are signalled in the VPWS NLRI and extended community attributes respectively. The **bgp-vpws** branch provides the configuration context for the **local-ve name** and **id**, and a **remote-ve** context to configure the remote **id**. The **local-ve name** is a locally significant name for the **epipe** service, whilst the **remote-ve id** is signalled as part of the VPWS NLRI. When a PE receives a VPWS NLRI to establish a BGP VPWS service, the VE ID the NLRI field is validated against the **id** in the **remote-ve** context and must match before the VPWS becomes operational.

Output 5-2: Single-homed VPWS configuration at R3

```

service {
    pw-template "7" {
        vc-type vlan
    }
    epipe "seven" {
        admin-state enable
        service-id 7
        customer "1"
        bgp 1 {
            route-distinguisher "64496:7"
            route-target {
                export "target:64496:7"
                import "target:64496:7"
            }
            pw-template-binding "7" {
            }
        }
        bgp-vpws {
            admin-state enable
            local-ve {
                name "R1"
                id 1
            }
            remote-ve "R3" {
                id 3
            }
        }
        sap 1/1/c3/1:7 {
        }
    }
}

```

Debug 5-1 shows the VPWS NLRI received at R1 from R3 when the **epipe** service has been administratively enabled. The NLRI is indicated as a VPWS NLRI by the presence of the CSV sub-TLV (csv: 0x00000000). The VBS is always set to a value of 1 for BGP VPWS, and the VBO is set to the **remote-ve id** configured in the **bgp-vpws** context of the advertising router. With a VBS of 1, the advertised label-base actually represents the label value that will be used for this service.

Debug 5-1: BGP VPWS NLRI received from R3

```

148 2025/08/19 07:31:21.818 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 90
    Flag: 0x90 Type: 14 Len: 32 Multiprotocol Reachable NLRI:
        Address Family L2VPN
        NextHop len 4 NextHop 192.0.2.3

```

```
[VPLS/VPWS] preflen 21, veid: 3, vbo: 1, vbs: 1, label-base: 524242, RD 64496:7,
csv: 0x00000000, type 1, len 1,
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 0 AS Path:
  Flag: 0x80 Type: 4 Len: 4 MED: 0
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
  Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.3
  Flag: 0x80 Type: 10 Len: 4 Cluster ID:
    192.0.2.10
  Flag: 0xc0 Type: 16 Len: 16 Extended Community:
    target:64496:7
    l2-vpn/vrf-imp:Encap=4: Flags=none: MTU=1514: PREF=0
```



Note: A dynamically created SDP is not signalled until a PE router has received a valid VPWS NLRI. When a VPWS is first enabled (and no NLRI has been exchanged), the PE router initiating the VPWS NLRI Update message will not have an operationally up spoke-SDP. Therefore, the first Update message may contain a CSV sub-TLV set to 1 (0x80). When the same PE router subsequently receives a VPWS NLRI from its pseudowire peer, it sources a second Update message with CSV sub-TLV set to 0. Typically, this first Update message with CSV set to 1 is sourced when the **bgp-vpws** node is placed in an administratively enabled state.

To validate the integrity of the VPWS, aside from checking for the presence of L2-VPN BGP routes in the RIB-In (using the '*show router bgp routes l2-vpn*' command) the service integrity can be checked by verifying the service-level L2-route-table for valid VPWS NLRI. This is shown in *Output 5-3* together with the associated dynamically created BGP-signalled SDPs. Alternatively, the service-level base information shown in *Output 5-4* can also be used for verification that the service is up and that SDPs have been correctly signalled.

Output 5-3: R1 service-level L2 Route-Table for BGP-VPWS entries

```
A:admin@R1# show service id 7 l2-route-table bgp-vpws

=====
Services: L2 Bgp-Vpws Route Information - Service 7
=====
L2-Routes (RD)      Next Hop      Ve-Id  Sdp Bind Id
-----
*64496:7            192.0.2.3    3      32751:4294967272
-----
No. of L2 Bgp-Vpws Route Entries: 1
=====
```

Output 5-4: R1 service-level base information

```
A:admin@R1# show service id 7 base

=====
Service Basic Information
=====
Service Id       : 7                Vpn Id          : 0
Service Type     : Epipe
MACSec enabled   : no
Name             : seven
Description      : (Not Specified)
Customer Id      : 1                Creation Origin  : manual
```

```

Last Status Change: 08/18/2025 19:58:24
Last Mgmt Change   : 08/18/2025 19:58:24
Test Service       : No
Admin State        : Up                Oper State          : Up
MTU                : 1514
Vc Switching       : False
SAP Count          : 1                  SDP Bind Count      : 1
Per Svc Hashing    : Disabled          Lbl Eth/IP L4 TEID: Disabled
Ignore MTU Mismatch: Disabled
Vxlan Src Tep Ip   : N/A
Force QTag Fwd     : Disabled
Lcl Switch Svc St  : sap
Oper Group         : <none>

```

Service Access & Destination Points

Identifier	Type	AdmMTU	OprMTU	Adm	Opr
sap:1/1/c3/1:7	q-tag	9212	9212	Up	Up
sdp:32751:4294967272 SB(192.0.2.3)	BgpVpws	0	9174	Up	Up

When using BGP VPWS with a dual-homed site using active/standby pseudowire the system automatically creates an 'endpoint' object to manage the active/standby status and put the appropriate pseudowire into a forwarding state. For ease of implementation the system also automatically creates an endpoint named `_tmnx_BgpVpws-<service id>` even if the VPWS is single-homed. This can be seen for the example VPWS service in *Output 5-5*. However, for a single-homed VPWS this endpoint object is irrelevant and can be ignored.

Output 5-5: Automatic endpoint creation for BGP-VPWS

```
A:admin@R1# show service id 7 endpoint
```

Service 7 endpoints

```

Endpoint name      : _tmnx_BgpVpws-7
Description        : Automatically created BGP-VPWS endpoint
Creation Origin    : bgpVpws
Revert time        : 0
Act Hold Delay     : 0
Standby Signaling Master : false
Standby Signaling Slave  : false
Tx Active (SDP)    : 32751:4294967272
Tx Active Up Time  : 0d 11:46:21
Revert Time Count Down : never
Tx Active Change Count : 2
Last Tx Active Change : 08/18/2025 19:58:24

```

Members

```
Spoke-sdp: 32751:4294967272 Prec:4 Oper Status: Up
```

To verify the use of the CSV field for signalling of VPWS integrity a failure of the attachment circuit at R3 is simulated. Recall that the one-octet CSV field uses a bit-vector where each bit indicates the status of the PVC associated with the corresponding label in the label-block. As SR-OS always signals a label block size of 1 only the high order bit is used, and this can be

seen in *Debug 5-2* which shows the CSV field set to a hex value of 0x80 (decimal 128) to reflect the attachment circuit failure.

Debug 5-2: Attachment circuit failure signalled with CSV field

```
149 2025/08/19 07:46:45.728 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 90
    Flag: 0x90 Type: 14 Len: 32 Multiprotocol Reachable NLRI:
        Address Family L2VPN
        NextHop len 4 NextHop 192.0.2.3
        [VPLS/VPWS] preflen 21, veid: 3, vbo: 1, vbs: 1, label-base: 524242, RD 64496:7,
        csv: 0x00000080, type 1, len 1,
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.3
    Flag: 0x80 Type: 10 Len: 4 Cluster ID:
        192.0.2.10
    Flag: 0xc0 Type: 16 Len: 16 Extended Community:
        target:64496:7
        l2-vpn/vrf-imp:Encap=4: Flags=D: MTU=1514: PREF=0
```



Note: The above example illustrates the requirements of a BGP VPWS using a dynamically created SDP that uses any LDP transport LSP that is available. If there is a requirement to reference an explicitly defined SDP (for example, in order to use an RSVP transport LSP) for the BGP VPWS, the **pw-template** must be configured with the **provisioned-sdp** command. When a valid VPWS NLRI is received from a peer, the system will thereafter look for explicitly defined SDPs with a corresponding far-end IP address. Explicitly defined SDPs must be configured for BGP signalling.

Multi-homed VPWS

As previously described SR-OS provides the capability to support redundant access with BGP Multi-Homing using either a single pseudowire to the DF or by signalling two pseudowires to the DF and non-DF in an active/standby arrangement managed by an endpoint object. This section discusses both methods in turn using R1 and R4 as the multi-homed site, and R3 as a single-homed site.

Single-Homed Pseudowire

Output 5-6 shows the configuration for the multi-homed VPWS at PE1 with a similar configuration applied at R4, noting that each of the PE's supporting the multi-homed site must be provisioned with a unique Route-Distinguisher allowing the VPWS NLRI from different VPLS PEs to be distinct even if the advertisements have the same VE-ID. The introduction of multi-homing also requires the addition of the **bgp-mh-site** context. The concept of sites will be familiar to readers of Chapter 4, and the command syntax does not

vary between VPLS and VPWS. The **bgp-mh-site** context contains the **id** representing the MH-ID signalled in the MH NLRI and must be common across the PEs supporting a multi-homed site. The **preference** field indicates a level of preference in the DF election where the higher value is preferred. In this example R1 has a preference of 100 whilst R4 has a preference of 50. To indicate a level of preference to transit BGP speakers (who may not interpret this service-level multi-homing preference value) the value that is configured in the **preference** field is also automatically populated into the LOCAL_PREF attribute and any attempt to modify the LOCAL_PREF attribute through policy will be ignored by the system. The **bgp-mh-site** context also contains local SAPs that are placed in forwarding or non-forwarding state depending on the outcome of the DF election.

The **boot-timer** defines how long the system waits after a node reboot before executing the DF election algorithm and allows for BGP sessions to be established and NLRI to be exchanged before executing the algorithm. The **activation-timer** defines how long the system will keep the local sites in standby status waiting for BGP Updates from remote PEs before running the DF election algorithm.

Output 5-6: R1 Multi-Homed VPWS configuration

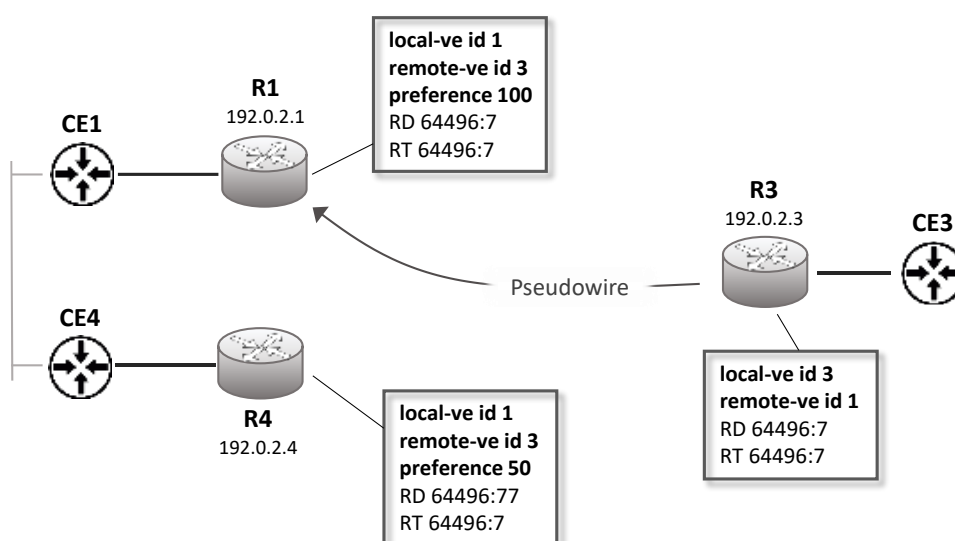
```

service {
  pw-template "7" {
    vc-type vlan
  }
  epipe "seven" {
    admin-state enable
    service-id 7
    customer "1"
    bgp 1 {
      route-distinguisher "64496:7"
      route-target {
        export "target:64496:7"
        import "target:64496:7"
      }
      pw-template-binding "7" {
      }
    }
    bgp-vpws {
      admin-state enable
      local-ve {
        name "R1"
        id 1
      }
      remote-ve "R3" {
        id 3
      }
    }
    bgp-mh-site "DUAL-HOME" {
      admin-state enable
      id 7
      sap 1/1/c3/1:7
      boot-timer 360
      activation-timer 30
      preference 100
    }
    sap 1/1/c3/1:7 {
    }
  }
}

```

As described during the single-homed BGP VPWS set-up, when a PE receives a VPWS NLRI to establish a BGP VPWS service, the VE ID in the NLRI field is validated against the **id** in the **remote-ve** context and must match before the VPWS becomes operational. This constitutes one of the main differences in provisioning for the single pseudowire method and the active/standby pseudowire method. In the single pseudowire method, there is a single **remote-ve id** configured at R3, which means that the **local-ve id**'s configured at R1 and R4 must be the same. The result is that R3 only sees a single VE ID and validates it against its configured **remote-ve id**. For the active/standby pseudowire method, two **remote-ve id**'s are configured at R3; hence each of the **local-ve id**'s configured at R1 and R4 are different and correspond to the **remote-ve id**'s configured at R3. Note however that the **local-ve name** has local significance only and can be the same or different at R1 and R4.

Figure 5-3: VE-ID configuration for single-pseudowire method



Debug 5-3 shows the VPWS NLRI advertised by R1 as the multi-homed VPWS is enabled. The preference in the L2 Information extended community reflects the configured **bgp-mh-site preference** and in turn is reflected in the LOCAL_PREF attribute. This preference influences the outcome of the DF election process and therefore determines the destination to which R3 signals its pseudowire. In short, R3's decision as to which PE it signals its pseudowire to is made on the basis of the VPWS NLRI - it does not use the MH NLRI at all because the information required for R3 to run its DF election is in the VPWS Update and should be the same in both the VPWS NLRI and MH Updates.

Debug 5-3: R1's VPWS NLRI received at R3

```

13 2025/08/19 09:03:01.480 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 86
  Flag: 0x90 Type: 14 Len: 28 Multiprotocol Reachable NLRI:
    Address Family L2VPN
    NextHop len 4 NextHop 192.0.2.1
    [VPLS/VPWS] preflen 21, veid: 1, vbo: 1, vbs: 1, label-base: 524242, RD 64496:7,
  csv: 0x00000000, type 1, len 1,
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  
```

```

Flag: 0x40 Type: 2 Len: 0 AS Path:
Flag: 0x80 Type: 4 Len: 4 MED: 0
Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.1
Flag: 0x80 Type: 10 Len: 4 Cluster ID:
    192.0.2.10
Flag: 0xc0 Type: 16 Len: 16 Extended Community:
    target:64496:7
    l2-vpn/vrf-imp:Encap=19: Flags=none: MTU=0: PREF=100

```

This is followed in *Debug 5-4* by an MH NLRI advertised by R1 and as received by R4 indicating that R1 (192.0.2.1) is the DF. Of course, because the VPWS and MH NLRI share the same Route-Target values it follows that both R4 and R3 would have received both Update messages. However, they are illustrated this way to make the point that a) R3 only needs to see the VPWS NLRI to determine the outcome of the DF election and therefore know where to signal its pseudowire, and b) the MH NLRI are only relevant to the Multi-Homed PEs.

Debug 5-4: R1's MH NLRI received at R4

```

111 2025/08/19 09:10:36.890 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 86
    Flag: 0x90 Type: 14 Len: 28 Multiprotocol Reachable NLRI:
        Address Family L2VPN
        NextHop len 4 NextHop 192.0.2.1
        [MH] site-id: 7, RD 64496:7
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.1
    Flag: 0x80 Type: 10 Len: 4 Cluster ID:
        192.0.2.10
    Flag: 0xc0 Type: 16 Len: 16 Extended Community:
        target:64496:7
        l2-vpn/vrf-imp:Encap=19: Flags=-DF: MTU=0: PREF=100

```

Using the service-level l2-route-table verification shown in *Output 5-7* it is possible to verify the presence of VPWS NLRI from the DF together with the associated dynamically created SDP binding. In addition, the output shows that that R1 (192.0.2.1) is the DF for Site-ID 7 (DF bits set) whilst R4 (192.0.2.4) is the non-DF. This means R4 must place all attachment circuits associated with Site-ID 7 into a non-forwarding state. This is shown in *Output 5-8* where the SAP is put into an operationally down state with the flag *'StandbyForMHPProtocol'*.

Output 5-7: Service-Level L2-Route-Table for BGP-VPWS Multi-Homing at R3

```

A:admin@R3# show service id 7 l2-route-table bgp-vpws multi-homing

=====
Services: L2 Multi-Homing Route Information - Summary
=====
Svc Id      L2-Routes (RD-Prefix)      Next Hop      SiteId      State  DF
-----
7           64496:7                    192.0.2.1    7           up(0)  set
7           64496:77                  192.0.2.4    7           up(0)  clear
-----
No. of L2 Multi-Homing Route Entries: 2
=====

```



```

=====
Services: L2 Bgp-Vpws Route Information - Service 7
=====
L2-Routes (RD)          Next Hop          Ve-Id  Sdp Bind Id
-----
*64496:7                192.0.2.1          1      32731:4294967243
-----
No. of L2 Bgp-Vpws Route Entries: 1
=====

```

Output 5-8: Attachment Circuit in Non-Forwarding state at R4

```

A:admin@R4# show service id 7 sap 1/1/c3/1:7

=====
Service Access Points(SAP)
=====
Service Id      : 7
SAP             : 1/1/c3/1:7          Encap           : q-tag
Description     : (Not Specified)
Admin State     : Up                  Oper State      : Down
Flags           : StandByForMHPProtocol
Multi Svc Site  : None
Last Status Change : 08/19/2025 09:10:35
Last Mgmt Change  : 08/19/2025 08:11:03
=====

```

To verify the failover capabilities for BGP multi-homing in VPWS it is possible to simulate a failure on the attachment circuit at the current DF, R1. As with BGP VPLS the re-convergence times will largely depend on BGP Update propagation times so use of the **l2-vpn true** command under the **rapid-update** context at global BGP level described in Chapter 4 (*Output 4-11*) should be considered if fast re-convergence times are a requirement.

Once the attachment circuit has failed at R1 it sources two Update messages. The first is a MH NLRI with the D bit set and the F bit clear since it can no longer function as DF for the multi-homed site. The second Update is a VPWS NLRI with the CSV field set (0x80) and the D bit of the Layer-2 Information extended community flags field set in order to reflect the attachment circuit failure. Both Updates are shown in *Debug 5-5*.

Debug 5-5: Attachment Circuit failure at R1

```

203 2025/08/19 09:24:21.918 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 72
  Flag: 0x90 Type: 14 Len: 28 Multiprotocol Reachable NLRI:
    Address Family L2VPN
    NextHop len 4 NextHop 192.0.2.1
    [MH] site-id: 7, RD 64496:7
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 0 AS Path:
  Flag: 0x80 Type: 4 Len: 4 MED: 0
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 0
  Flag: 0xc0 Type: 16 Len: 16 Extended Community:
    target:64496:7
    l2-vpn/vrf-imp:Encap=19: Flags=D: MTU=0: PREF=100

```

```

204 2025/08/19 09:24:21.918 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 76
    Flag: 0x90 Type: 14 Len: 32 Multiprotocol Reachable NLRI:
        Address Family L2VPN
        NextHop len 4 NextHop 192.0.2.1
        [VPLS/VPWS] preflen 21, veid: 1, vbo: 3, vbs: 1, label-base: 524250, RD 64496:7,
csv: 0x00000080, type 1, len 1,
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 0
    Flag: 0xc0 Type: 16 Len: 16 Extended Community:
        target:64496:7
    l2-vpn/vrf-imp:Encap=4: Flags=D: MTU=1514: PREF=100

```

The Update from R1 indicating a transition to non-DF (attachment circuit down) means that R4, which forms part of the same Site-ID, must transition to DF. As a result, R4 sources an MH NLRI with the DF bits of flags field in the Layer-2 Information community set as shown in *Debug 5-6*.

Debug 5-6: Transition to DF at R4

```

205 2025/08/19 09:24:23.897 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 86
    Flag: 0x90 Type: 14 Len: 28 Multiprotocol Reachable NLRI:
        Address Family L2VPN
        NextHop len 4 NextHop 192.0.2.4
        [MH] site-id: 7, RD 64496:77
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 50
    Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.4
    Flag: 0x80 Type: 10 Len: 4 Cluster ID:
        192.0.2.10
    Flag: 0xc0 Type: 16 Len: 16 Extended Community:
        target:64496:7
    l2-vpn/vrf-imp:Encap=19: Flags=-DF: MTU=0: PREF=50

```

Checking the service level L2-route-table again at R3 it is possible to verify that the multi-homing information shows the DF bits from R4 (192.0.2.4) are set whilst R1 (192.0.2.1) is signalling the F bit as clear, but the D bit set to indicate the attachment circuit is down. In addition, the VPWS route information displays the VPWS NLRI from R4 with the associated binding. As with VPLS Multi-Homing the failover mechanism is revertive, meaning that when the attachment circuit at R1 recovers it will again become DF.

Output 5-9: Service-Level L2-Route-Table for Multi-Homing at R3

```

A:admin@R3# show service id 7 l2-route-table bgp-vpws multi-homing

=====
Services: L2 Multi-Homing Route Information - Summary
=====
Svc Id      L2-Routes (RD-Prefix)      Next Hop      SiteId      State  DF

```

```

-----
7          64496:7          192.0.2.1      7          dn(1)  clear
7          64496:77         192.0.2.4      7          up(0)  set
-----
No. of L2 Multi-Homing Route Entries: 2
=====

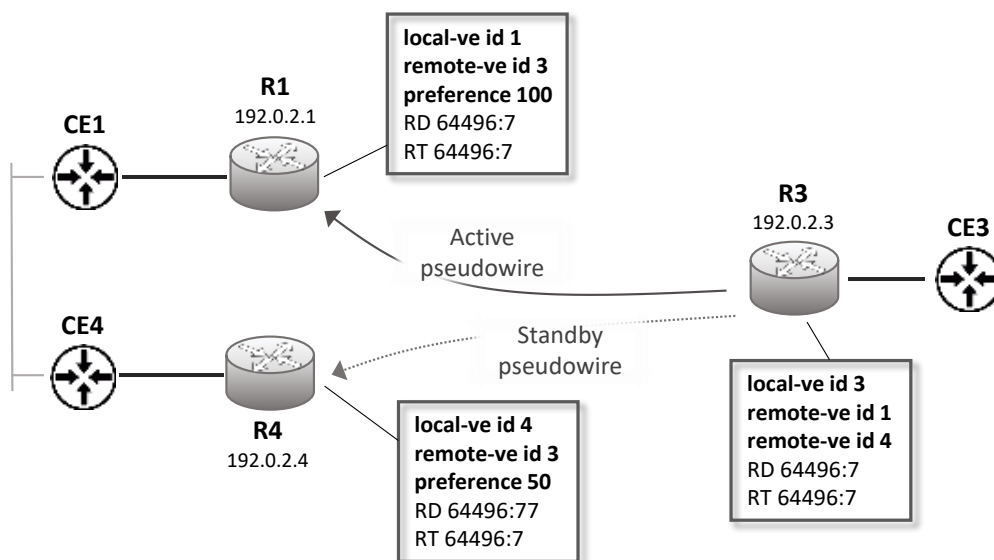
Services: L2 Bgp-Vpws Route Information - Service 7
=====
L2-Routes (RD)          Next Hop          Ve-Id  Sdp Bind Id
-----
*64496:77              192.0.2.4          1      32731:4294967242
-----
No. of L2 Bgp-Vpws Route Entries: 1
=====

```

Active/Standby Pseudowire

As indicated in the previous section, the notable difference in provisioning for the active/standby pseudowire method is that two **remote-ve id**'s are configured at R3 instead of a single **remote-ve id** used in the single pseudowire scenario. As a result, the **local-ve id**'s configured at R1 and R4 are different and correspond to the two **remote-ve id**'s configured at R3.

Figure 5-4: VE-ID Configuration for Active/Standby Pseudowire Method



Aside from the different VE IDs there are no other configuration changes at R1 and R4 for the active/standby pseudowire method. At R3 a second **remote-ve** context is configured which provides the ability to configure the second (VE) id.

Output 5-10: Active/Standby Pseudowire configuration at R3

```

epipe "seven" {
  admin-state enable
  service-id 7
  customer "1"
  bgp 1 {

```

```

        route-distinguisher "64496:7"
        route-target {
            export "target:64496:7"
            import "target:64496:7"
        }
        pw-template-binding "7" {
        }
    }
    bgp-vpws {
        admin-state enable
        local-ve {
            name "R3"
            id 3
        }
        remote-ve "R1" {
            id 1
        }
        remote-ve "R4" {
            id 4
        }
    }
    sap 1/1/c3/1:7 {
        admin-state enable
    }
}

```

All VPWS and Multi-Homing Update signalling used to establish the pseudowire and re-converge upon failure (DF transition) are the same as the single pseudowire method and is therefore not repeated here. The main difference is that both the active and standby pseudowire are both pre-signalled and the forwarding state controlled by a system-created endpoint. This is shown in *Output 5-11* with the endpoint name `_tmnx_BgpVpws-7` and description “*Automatically created BGP-VPWS endpoint*”. The active SDP is 32731:4294967241 which is the pseudowire to R1. Confirmation that R1 is the DF is given by the L2-Route-Table shown in *Output 5-12*.

Output 5-11: Active/Standby Pseudowire with Dynamic Endpoint

```
A:admin@R3# show service id 7 endpoint
```

```

=====
Service 7 endpoints
=====
Endpoint name           : _tmnx_BgpVpws-7
Description              : Automatically created BGP-VPWS endpoint
Creation Origin          : bgpVpws
Revert time              : 0
Act Hold Delay           : 0
Standby Signaling Master : false
Standby Signaling Slave  : false
Tx Active (SDP)          : 32731:4294967241
Tx Active Up Time        : 0d 00:27:34
Revert Time Count Down   : never
Tx Active Change Count   : 11
Last Tx Active Change    : 08/19/2025 10:06:22
-----
Members
-----
Spoke-sdp: 32731:4294967241 Prec:4                      Oper Status: Up
=====
=====

```

Output 5-12: L2-Route-Table at R3

```

A:admin@R3# show service id 7 l2-route-table bgp-vpws multi-homing

=====
Services: L2 Multi-Homing Route Information - Summary
=====
Svc Id      L2-Routes (RD-Prefix)      Next Hop      SiteId      State  DF
-----
7           64496:7                    192.0.2.1     7           up(0)  set
7           64496:77                   192.0.2.4     7           up(0)  clear
-----
No. of L2 Multi-Homing Route Entries: 2
=====

=====
Services: L2 Bgp-Vpws Route Information - Service 7
=====
L2-Routes (RD)      Next Hop      Ve-Id  Sdp Bind Id
-----
*64496:7            192.0.2.1     1      32731:4294967241
*64496:77           192.0.2.4     4      32730:4294967240
-----
No. of L2 Bgp-Vpws Route Entries: 2
=====

```

In summary both single pseudowire and active/standby pseudowire use common Update signalling to set up and maintain the pseudowire, but the significant difference is the presence of a pre-signalled standby pseudowire in the active/standby pseudowire model which can help to reduce re-convergence times upon DF transition. Both mechanisms are revertive upon failure restoration and this is non-configurable.

6 Ethernet VPN

VPLS is a reasonably mature and widely deployed technology. However, it has a number of deficiencies and limitations that Ethernet VPN attempts to tackle:

- a. Multi-homing solutions need to operate in active/standby fashion to maintain a loop-free topology. There is no solution for active/active operation.
- b. Due to the inherent flood-and-learn behaviour in a conventional learning bridge, flooding is commonplace, but undesirable.
- c. Flooding of Address Resolution Protocol (ARP) and IPv6 Neighbour Discovery packets is frequently undesirable, particularly in environments such as Internet Exchange Points (IXPs).

Similar to a MPLS/BGP IP-VPN, an Ethernet VPN (EVPN) instance comprises a number of CE devices that are connected to PE routers forming the edge of the Service Provider infrastructure. A BGP based Ethernet VPN (EVPN) [21] provides virtual Layer 2 bridged connectivity between this set of CE routers. PE routers providing this service are interconnected using MPLS LSPs or IP tunnelling techniques such as the Virtually eXtensible Local Area Network (VXLAN), Generic Routing Encapsulation (GRE), or SRv6, in exactly the same way that other Layer-2 or Layer-3 VPN services are provided.

In a conventional bridged service (VPLS), MAC learning is done in the data-plane, but for EVPN, MAC learning is done in the control plane. PE routers supporting the EVPN advertise MAC addresses learnt from connected CEs together with an MPLS label, VXLAN Network Identifier (VNI) or SRv6 SID, using Multi-Protocol BGP. Control plane learning offers significant advantages over data plane learning, such as reducing the requirement to flood and helping to improve reconvergence times for certain failure scenarios. One other major benefit of control plane learning is that it enables flow-based load-balancing of traffic to CEs that are dual-homed. Today dual-homed sites must work in active/standby mode using methods such as BGP Multi-Homing, Spanning-Tree Protocol (STP), or Multi-Chassis LAG to provide a loop-free topology, but EVPN allows active-active (known as All-Active) multi-homing using control plane learning.

The specification in [21] also provides the ability to forward customer traffic to/from a given customer attachment circuit (AC) without the need for a MAC lookup. This capability is ideal for providing point-to-point services and using a reduced control plane allows Ethernet VPN to additionally support Virtual Private Wire Services (VPWS). It also provides the benefits of providing all-active and single-active redundancy for VPWS.

In summary the EVPN control plane provides support for both EVPN-VPLS services and EVPN-VPWS services and these are discussed throughout this chapter. Before getting into the details however, it's important to understand some new terminology:

EVI	An EVI is an EVPN instance that spans all of the PE routers participating in the VPN. Each EVI is associated with a Route-Distinguisher and one or more Route-Targets, and the semantics
-----	--

	and use of these are exactly the same as with BGP-MPLS IP-VPNs. MAC addresses are advertised in much the same way that PE routers' supporting BGP-MPLS IP-VPN advertise IP reachability.
MAC VRF	A virtual routing and forwarding table for MAC addresses on a PE for a given EVI.
ESI	If a CE is multi-homed to two or more PEs, the set of Ethernet links that attaches the CE to the PEs is known as an Ethernet segment, identified by an Ethernet Segment Identifier (ESI).
Ethernet Tags	An Ethernet Tag identifies a broadcast domain within an EVI, and each EVI can contain one or more broadcast domains. The Ethernet tag is encoded in the EVPN NLRI. When a PE receives an EVPN NLRI containing an Ethernet TAG, it has to perform a mapping to the attached CE VLAN Identifiers (CE-VIDs). The function of this mapping depends on the type of service interface, which can be VLAN-based, VLAN bundle-based, or port-based. VLAN-based service interfaces have a 1:1 mapping between the CE-VID and the EVI with a single bridge domain per PE for the EVI. VLAN-bundle based service interfaces have a N:1 mapping between the CE-VID and the EVI but again a single bridge domain per PE for the EVI (different CEs connected to different PE ports use the same CE-VIDs for the same EVI). Port-based services are essentially the same as VLAN-bundle services with the exception that all VLANs on the port are part of the same EVPN. A variation of the VLAN-bundle and port-based service interfaces makes them 'VLAN aware' permitting an N:1 mapping between the CE-VID and EVI, but with multiple bridge domains per PE per EVI. The Ethernet tag is present only in the EVPN NLRI when VLAN aware service interface types are used.
Multi-Homing	Multi-homing can be in the form of Single-Active redundancy mode or All-Active redundancy mode. In Single-Active mode, only a single PE of all the PEs attached to a given Ethernet segment is allowed to forward traffic to/from that segment. In Active-Active mode, all PEs attached to an Ethernet segment are allowed to forward known unicast traffic to/from that segment.

Ethernet VPN AFI/SAFI and Route-Types

The EVPN control plane is implemented with Multi-Protocol BGP using AFI 25 (L2VPN) SAFI 70 (EVPN). It's structure and the way that routes are advertised is similar to that of BGP/MPLS IP-VPN. An EVI requires a Route-Distinguisher that is unique per MAC-VRF, and one or more Route-Targets that control which MAC-VRFs import the route at receiving PEs. The NLRI is encoded as shown in *Figure 6-1*. *Table 6-1* below it lists the five EVPN route types that are currently defined for unicast, and their use is discussed throughout this chapter. There are a

further three route types defined for multicast over EVPN, but they are beyond the scope of this chapter.

Figure 6-1: Ethernet VPN NLRI

Route Type (1 octet)
Length (1 octet)
Route Type specific (variable)

Table 6-1: Ethernet VPN NLRI Route-Types

Value	Route Type
1	Ethernet Auto-Discovery (A-D) Route
2	MAC Advertisement Route
3	Inclusive Multicast Ethernet Tag Route
4	Ethernet Segment Route
5	IP Prefix Route

The Route Types 1 to 4 are used to construct an Ethernet VPN (EVPN-VPLS or EVPN-VPWS) with one or more all-active or active-active Ethernet segments. This chapter starts by describing those together with configuration examples of both EVPN-VPLS and EVPN-VPWS services. The chapter will then move on to discuss the final Route Type, the IP Prefix Route, which is required for Inter-Subnet connectivity.

Ethernet Auto-Discovery Route (Route-Type 1)

The Ethernet A-D route NLRI is depicted in *Figure 6-2* and is used to aid fast convergence (through a process known as mass withdraw), *split-horizon filtering*, and *aliasing*. It may be accompanied by an ESI Label extended community, also shown in the figure.

Figure 6-2: Ethernet Auto-Discovery Route and ESI Label Extended Community

Route Distinguisher (8 octets)
Ethernet Segment Identifier (10 octets)
Ethernet Tag ID (4 octets)
MPLS Label (3 octets)

Type 0x06	Sub-Type	Flags	Reserved
Reserved	ESI Label		

ESI Label Extended Community

Ethernet Auto-Discovery Route

ESI Label Extended Community

The Ethernet A-D route can be advertised on a per-Ethernet Segment (ES) basis or on a per-EVI basis, and this dictates how the values of some of the fields in the NLRI are populated.

- An Ethernet A-D per-EVI is a service-level route used to advertise reachability to an ES and is used for aliasing.

- An Ethernet A-D per-ES is a system-level route used to advertise the ES capabilities (multi-homing mode) and for fast convergence (mass withdraw). It is a system-level route because it refers to the ES as a general entity. For example, if you assume that the ES is a LAG, then the LAG may have multiple VLANs (SAPs) attached, each belonging to different EVIs. The Ethernet A-D per-ES must be accompanied by an ESI Label extended community containing an ESI label used for the purpose of split-horizon filtering.

The low-order bit of the flags field in the ESI Label extended community is defined as the *Single-Active* bit. A value of 0 means that the multi-homed site is operating in All-Active mode, whilst a value of 1 means that the multi-homed site is operating in Single-Active mode. When operating in Single-Active mode, any other PEs advertising Ethernet A-D routes for the same ES operate in standby mode only. If the primary PE encounters a failure, its Ethernet-AD route may be withdrawn together with its MAC Advertisement routes, but in parallel to this the backup PE starts to learn local MAC addresses and advertises them in MAC Advertisement routes, which minimizes flooding during failover. When the Single-Active bit is set to 0 any PE receiving an Ethernet A-D route must treat the ES as operating in All-Active multi-homing mode, and as a result advertised MAC addresses are assumed to be reachable through all of the PE nodes belonging to the same ES. This happens even if one of the PEs has not advertised a MAC Advertisement route (for example, perhaps it has not yet learnt the attached CEs MAC address). This process is referred to as aliasing and is discussed in detail later in this section.

Table 6-2 lists the values used for the Ethernet A-D per-ES and Ethernet A-D per-EVI route variants.

Table 6-2: Ethernet A-D per-ES and per-EVI routes

A-D Route Field	Ethernet A-D Per-ES Route	Ethernet A-D Per-EVI Route
Route-Distinguisher	Type 1 RD, with Administrator subfield containing IP address followed by number unique to PE.	Set to the RD of the MAC-VRF or VPWS advertising the A-D route
Ethernet Segment ID	10-octets consisting of 1-octet Type field and 9-octet value field.	10-octets consisting of 1-octet Type field and 9-octet value field.
Ethernet Tag	Set to reserved value “MAX-ET” (0xFFFFFFFF).	VLAN Identifier of the Ethernet segment for VLAN-aware and VLAN-aware-bundle services. Set to 0 for VLAN-based and VLAN bundle-based services. Set to Eth-Tag of local AC for VPWS
MPLS label	Set to 0	Label value
Remarks	Must carry ESI Label extended community	

Split Horizon

The split horizon rules widely used in VPLS are inherited for EVPN. When a PE router receives a broadcast, multicast, or unknown unicast frame from an attached CE it sends it to all other attached CEs and also to all other PEs in the EVI. However, when a PE router receives an unknown unicast frame from another PE, it must forward it to attached CEs only if it is the Designated Forwarder (DF) for the egress ESI. It must never forward the frame to another PE. However, All-Active multi-homing presents an additional challenge. Consider the case of a CE that is multi-homed to two PE routers on an Ethernet segment operating in All-Active mode. If the CE sends a broadcast packet to the non-DF PE, that non-DF PE sends the frame to all other PEs in the EVI, including the DF for the Ethernet segment. The DF must drop that frame and not forward it back to the CE to avoid a loop. To achieve this behaviour the ESI Label contained in the ESI Label extended community attribute is used. Whenever a broadcast, unknown unicast, or multicast packet is forwarded onto the EVI from a non-DF PE, it is encapsulated with this ESI Label to identify the Ethernet segment of origin. When the packet reaches the DF PE, it uses the ESI Label to determine whether or not this frame can be forwarded onto the segment.



Note: As SRv6 has no concept of a label, the mechanism for implementing for an ES is notably different and is described in Chapter 8.

Fast Convergence

Whilst there are many advantages to using the BGP control plane to advertise EVPN routes, one potential disadvantage is that network reconvergence times may increase; particularly in a scaled environment where a lot of MAC Advertisement routes need to be withdrawn. To help speed up convergence times upon failure of an attached Ethernet segment, a PE router sends a withdraw message (MP_UNREACH) for the corresponding segment using an Ethernet A-D per-ES route. This triggers all other PEs to update the Next-Hop adjacencies for all the MAC addresses associated with the ES. If no alternate Next-Hop exists, the MAC addresses associated with that ES are marked as invalid. This approach is considered beneficial because advertising a single Update (MP_UNREACH) for the ES is significantly better than constructing a number of withdraws for all MAC addresses associated with that segment.

Aliasing

The Ethernet A-D route is also used for aliasing an ESI within a given EVI. Aliasing is the PE's capability to signal reachability for an ESI even when it has not learnt any MAC addresses locally on that Ethernet segment. Consider the case where a CE is multi-homed to multiple PE nodes using a LAG with All-Active redundancy. It is quite conceivable (because of the hash routine on the CE) that only a single PE learns the MAC addresses of the multi-homed site; for example, if data-plane learning is implemented on the attachment circuit(s). This leads to a situation where remote PEs receive MAC Advertisement routes only from a single PE of the PE's forming the multi-homed site and are therefore unable to effectively load-balance traffic destined towards the multi-homed site across all of the PE's supporting that site.

Using aliasing, PEs locally attached to an Ethernet segment signal the ESI in an Ethernet A-D route per-EVI even when no MAC addresses have been learnt. Remote PE's that receive MAC Advertisement routes thereafter consider the advertised MAC addresses as reachable through all PE's that have advertised a combination of the following:

- An Ethernet A-D per-EVI route with the same ESI, *and*
- An Ethernet A-D per-ES route with the same ESI and with the Single-Active bit set to 0 in the ESI Label extended community (indicating that the ESI is in All-Active mode).

A given remote PE should not use the Ethernet A-D per-EVI route for forwarding until it also receives the associated set of Ethernet A-D per-ES routes. When a PE has a frame to send to one of the PEs connected to the All-Active Ethernet segment, it uses the following rules:

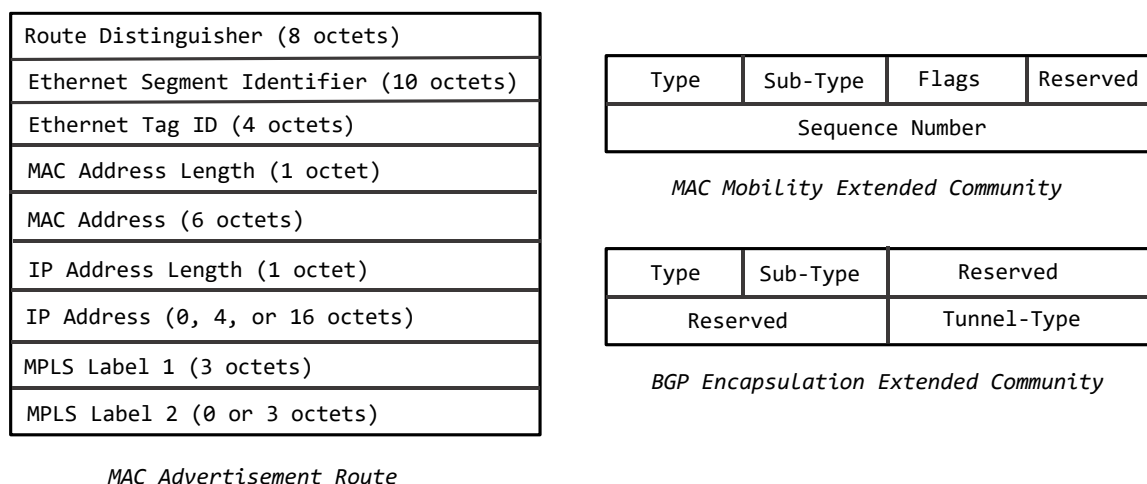
- If the PE sends the frame to a PE that advertised a MAC Advertisement route, it uses the MPLS label associated with the MAC Advertisement route.
- If the PE sends the frame to another PE in the ESI that has not sourced a MAC Advertisement route, the label used is the MPLS label advertised in the Ethernet A-D per-EVI route.

MAC Advertisement Route (Route-Type 2)

The MAC Advertisement route facilitates control plane MAC learning, which can be local or remote. Local Learning is learning of MAC addresses from directly connected CE routers using conventional data-plane learning, DHCP, or ARP. Remote learning is learning of MAC addresses that are behind CEs connected to other PEs. It is achieved by advertising the MAC addresses it learns locally into a MAC Advertisement route.

The MAC Advertisement route contains an RD, the ESI, Ethernet Tag ID, MAC, and MPLS label. It can optionally carry an IP address associated with the advertised MAC address. The MAC Advertisement route may also be accompanied by one or more extended community attributes. The MAC Mobility extended community allows for MAC addresses in an EVI to move to a different attachment circuit in the EVI without creating black-holes. The BGP Encapsulation extended community [31] provides a mechanism for a PE router to indicate the data-plane tunnelling mechanism to be used for a given MAC Advertisement route. In the absence of the BGP Encapsulation extended community, MPLS is assumed to be the default encapsulation, but can be configured for either MPLS or VXLAN using the **def-recv-evpn-encap** command at BGP, group, or neighbour level. The use of both Extended Communities is discussed in further detail throughout this sub-section.

Figure 6-3: MAC Advertisement Route



EVPN decouples the control plane from the data-plane used for the EVI, which allows for the use of various tunnelling mechanisms such as SRv6, MPLS, or VXLAN. Because these options exist, there needs to be a way to signal which tunnelling mechanism to use for a given MAC Advertisement route, and as previously outlined, the BGP Encapsulation extended community is used for this purpose. SR-OS currently supports MPLS, VXLAN, or SRv6 encapsulation for EVPN, and whilst the choice of tunnelling mechanism is carried in the BGP Encapsulation extended community, the demultiplexer value for MPLS and VXLAN is carried in the MPLS Label field. As one would expect, the demultiplexer value is different for each tunnelling technique:

- a. When EVPN is used with MPLS as a data-plane [21] the Tunnel Type field of the BGP Encapsulation extended community is set to a value of 10, and the MPLS label field of the MAC Advertisement route uses a conventional 20-bit service label as a MAC-VRF demultiplexer. By default, SR-OS uses a label-per-MAC-VRF model (analogous to the label-per-VRF model for IP-VPN), but configurable options exist for label-per-MAC or label-per-ESI.
- b. When EVPN is used with VXLAN as a data-plane [22] the Tunnel Type field of the BGP Encapsulation extended community is set to a value of 8, and the MPLS label field of the MAC Advertisement route uses a 24-bit VXLAN Network Identifier (VNI) as a demultiplexer. This VNI value is encapsulated as part of the VXLAN header.

For SRv6 encapsulation the associated forwarding information is carried in an SRv6 Service TLV, and this is described in Chapter 8.

Advertising a MAC address without an IP address permits for control-plane learning and implicitly means that flood-and-learn data-plane learning is not required. Advertising a MAC address with an associated IP address provides the ability to create an IP-to-MAC binding which can be beneficial in minimizing ARP or Neighbour Discovery (ND) procedures through use of proxy ARP. That is, if a PE receives an ARP request for an IP address for which it has an IP-MAC binding, it can locally respond to that ARP request rather than propagate it through the EVI.

The ability to dynamically associate IP and MACs can also be useful in a Data Centre (DC) environment or Internet Exchange (IX). The Network Virtual Overlay over Layer 3 (NVO3) working group in the IETF has defined a framework for constructing overlay networks that operate on top of an IP-based underlay network. An overlay network is a virtual network in which the separation of tenants is not made visible to the underlying physical infrastructure. One of the popular overlay approaches is VXLAN, which encapsulates Ethernet frames in UDP/IP. The Ethernet frames are invisible to the underlay network, which is responsible purely for forwarding at the IP layer. These VXLAN tunnels are extended between physical network functions (PNFs), virtual network functions (VNFs) or containerised network functions (CNFs) and provide traffic separation between tenants, except through a carefully controlled interface such as a firewall. The VXLAN tunnels also allow for address space isolation, so that address space can be re-used between tenants. A MAC Advertisement route provides the ability to learn tenant MACs and associated VXLAN Tunnel End Points (VTEPs) and automatically create VXLAN tunnel bindings for intra-DC traffic, or for inter-DC traffic using a VPLS or VPRN at the DC gateway.

The process of control plane learning has benefits, but what if a host moves from one Ethernet segment to another segment. This scenario is not unknown in Data Centre environments. If a host simply moves segments, the PE to which it is newly attached propagates a MAC Advertisement route when it's MAC has been locally learnt, but the PE to which it was previously attached very likely still has the same MAC address locally cached. This MAC Advertisement route would still be active, which results in two active MAC Advertisement routes, one of which is a blackhole. This scenario is called MAC mobility (or MAC move). To resolve this situation, the old MAC Advertisement must be withdrawn so that only one MAC Advertisement route is active. To achieve this, a MAC Mobility extended community attribute is used. When a PE router initially advertises a MAC address in a MAC Advertisement route no MAC Mobility extended community attribute is attached. When a PE router learns a local MAC address for which it had previously received a MAC Advertisement route with a different ESI (including zero-ESI value), it advertises the MAC address in a MAC Advertisement route and includes a MAC Mobility extended community attribute. When the same MAC address is advertised by two PE routers with different ESI values (again, including zero-ESI value) it is interpreted by other PEs as mobility, in which case the route with the MAC Mobility extended community is considered to be the most recent. (If the same MAC address is advertised by two PE routers with the same ESI value, it is interpreted as multi-homing.) If another PE should subsequently locally learn the MAC and advertise the MAC, a sequence number in the MAC Mobility extended community is incremented allowing PE routers to determine the most recent information. A PE router receiving a MAC Advertisement route with a different ESI and a higher sequence number in the MAC Mobility extended community thereafter withdraws the MAC Advertisement route it previously advertised. This allows for mobility whilst avoiding black-holes.

The low-order bit of the MAC Mobility Flags field is known as the *Sticky* or *Static* flag. When set to one it means that the MAC address is static and cannot move. This is used in SR-OS when advertising system-generated MAC addresses, or when provisioning static MACs via a SAP or SDP. These static MACs are used for MACs requiring protection, where the MAC

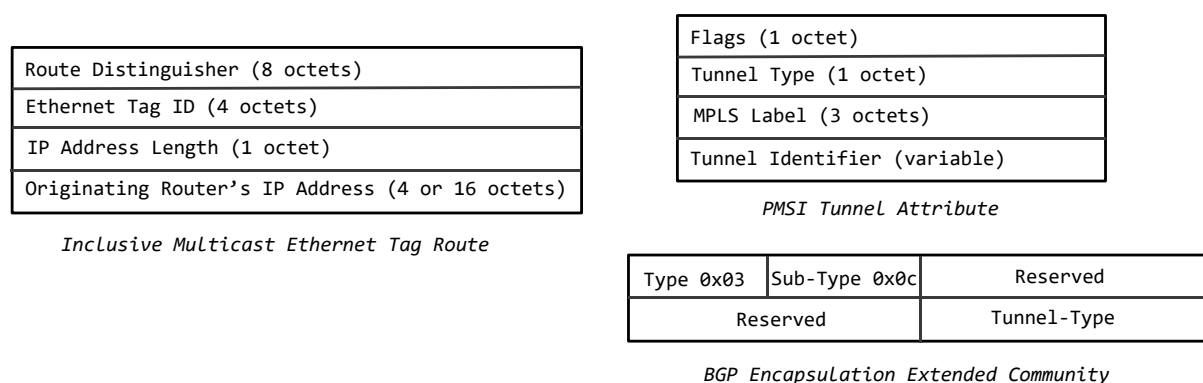
address cannot be learnt dynamically from anywhere else in the EVI. If a frame with a source MAC address matching one of the protected MACs is received in a different SAP/SDP-bind than the owner of the protected MAC, the frame will be discarded, and an alarm will be logged.

Of course, some measures need to be taken to ensure that a MAC address cannot continually move, triggering an endless exchange of MAC Advertisement routes and withdraws. SR-OS resolves this using a MAC duplication feature that monitors for a configurable number of moves within a configurable window before putting the MAC into a hold-down state. Configuration of the MAC duplication feature is discussed further later in this chapter.

Inclusive Multicast Ethernet Tag Route (Route Type 3)

When forwarding broadcast, unknown unicast, or multicast traffic, PEs in an EVI may use ingress replication, but to allow for optimisation of the flooding procedure, options exist to use P2MP or MP2MP LSPs to distribute the replication function. To enable this, each PE must advertise an Inclusive Multicast Ethernet Tag route carrying a P-Multicast Service Interface Tunnel (PMSI) Tunnel attribute to identify the type of Provider-Tunnel (P-tunnel) that will be used for this purpose. The PMSI Tunnel attribute is an optional transitive attribute that instructs the receiving PE or PEs how to construct the P-tunnel and was originally specified for use in Multicast-VPN (discussed in Chapter 10). In addition to the PMSI Tunnel attribute, each PE will also attach a BGP Encapsulation extended community to indicate the data-plane tunnelling mechanism to be used for broadcast, unknown unicast, and multicast traffic.

Figure 6-4: Inclusive Multicast Ethernet Tag Route, PMSI Tunnel Attribute, and BGP Encapsulation extended community Attribute



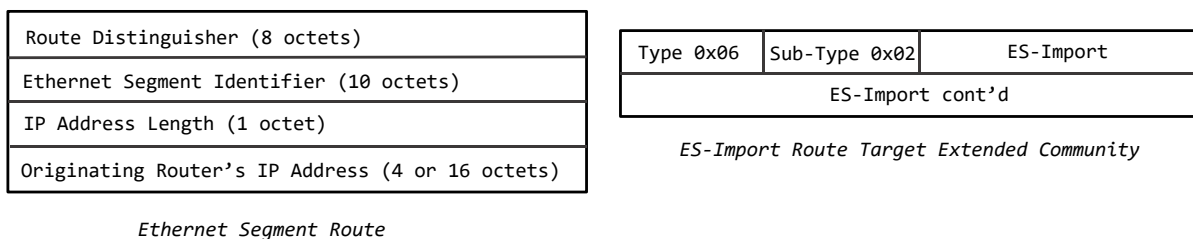
The RD field within the Inclusive Multicast Ethernet Tag route contains an EVI-specific RD value. Within the PMSI Tunnel attribute, the MPLS label field is used to carry a label that PE routers will use when forwarding broadcast, unknown unicast or multicast frames towards the PE that originated the route. (Again, SRv6 uses a different mechanism described in Chapter 8.) The Tunnel Identifier field is used to indicate what type of tunnelling technology will be used to build a point-to-point or multipoint-to-multipoint multicast tree for the purpose of forwarding broadcast, unknown unicast, and multicast frames. Options for the Tunnel Type include RSVP-TE, mLDP, PIM-SSM, PIM-SM, BIDIR, PIM, and Ingress Replication.

The Inclusive Multicast Ethernet Tag route is sent as soon as the EVI is placed into an administratively enabled state (regardless of the operational state of local SAPs/SDPs). This serves the additional function of auto-discovering other members of the EVI and associating the BGP Next-Hops within the EVI to the relevant (VXLAN/MPLS) tunnel.

Ethernet Segment Route (Route Type 4)

An Ethernet Segment route is used for automatic discovery of the PEs supporting the same ESI, and for Designated Forwarder (DF) election. The route is accompanied by an ES-Import Route-Target extended community attribute, which is used instead of a conventional Route-Target extended community. The Value field of the ES-Import Route-Target extended community is constructed by encoding the high-order 6-octet portion of the 9-octet ESI value (the ESI is actually 10-octets but uses a 1-octet ESI Type field and 9-octet ESI value). This ensures that the route is imported only by PEs that are multi-homed to the same Ethernet segment.

Figure 6-5: Ethernet Segment Route and ES-Import Route Target



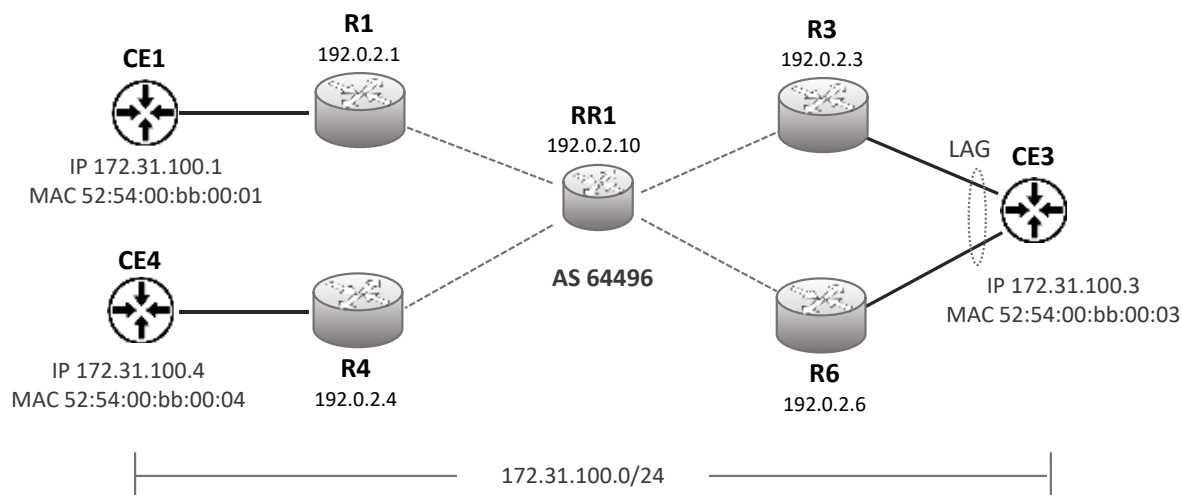
Regardless of whether the multi-homing operation is All-Active or Single-Active, a single PE that connects to the ESI for a given EVI is nominated as Designated Forwarder (DF). The DF is responsible for flooding broadcast, unknown unicast, and multicast traffic onto a given Ethernet segment towards the CE, whereas the non-DF will discard these packets. To provide for All-Active load-balancing, both DF and non-DF can forward frames with known MAC destinations. The DF election process within [20] defines a process that uses a *mod N* function of the VLAN ID to determine a DF with <ES, VLAN> granularity. This allows for multiple DF's to be elected and allows for load-balancing of multi-destination frames onto a segment. This is a process referred to in EVPN as *service-carving*.

EVPN-VPLS

To illustrate the use of EVPN and how the EVPN route types are utilised the topology shown in *Figure 6-6* is used. R1, R3, R4, and R6 all belong to AS 64496 and constitute the PE routers supporting EVPN. All the PE routers are IBGP peered with a Route-Reflector, RR1, and are configured to support the EVPN Address Family. The IGP in use within AS 64496 is IS-IS, and, as the example will use EVPN with an MPLS data-plane, all routers apart from RR1 are configured to run SR-ISIS. CE1, CE2 and CE3 constitute the CE routers that belong to the

EVPN instance. CE1 and CE2 are single-homed CE routers, whilst CE3 represents a multi-homed site constructed as a single LAG at CE3.

Figure 6-6: EVPN Test Topology



As one would expect, in SR-OS an EVPN-VPLS service is modelled as a VPLS service. *Output 6-1* shows the configuration parameters for the EVPN service with an MPLS data-plane as configured at R1 (R4 has a similar configuration). Within the VPLS service, the **bgp-evpn** node provides the context to define the operating parameters of the EVI. The first line within this context is the **evi** command with an instance number. This number should be unique to this EVPN instance. The **evi** value can be configured in the range 1-16777215 and provides automatic system-generated Route-Distinguisher and Route-Target values. The Route-Distinguisher uses a Type 1 RD format containing the system address in the 4-byte Administrator subfield, followed by the EVI value in the 2-byte Assigned Number subfield. The Route-Target uses the router's Autonomous System in the 2-byte Global Administrator field followed by the EVI value in the 4-byte Local Administrator field. The intention of the auto-generated values is to provide for a low-touch configuration approach. If the values are not suitable however, the service-level **bgp** context provides the option to manually provision both **route-distinguisher** and **route-target** values. The **mac-duplication** context allows the user to define the operating parameters for MAC duplication within the EVI. It allows for a configurable number of re-learns/moves (**num-moves**) within a configurable **window** expressed in minutes. When the threshold is exceeded, the MAC is put into a hold-down state, whereby the system stops sending and processing any MAC Advertisement routes until either a) the MAC is locally flushed or remotely withdrawn, or b) the **retry** timer expires, which flushes the MAC and restarts the process. It is recommended that the **retry** time should be at least twice that of the **detect** window. The **mac-duplication** feature is enabled by default with **num-moves** set to a value of 5 (moves), **window** set to a value of 3 (minutes) and **retry** set to a value of 9 (minutes).

The **bgp-evpn** node also provides the context to select the required data-plane tunnelling mechanism, with the options being **mpls**, **segment-routing-v6**, or **vxlan**. SRv6 is covered

separately in chapter 8 so is not discussed here. The configuration requirements differ slightly for each of the remaining two options:

- a. When **mpls** is selected, the PE router must resolve the BGP Next-Hop for EVPN routes to an MPLS tunnel before the prefix is considered valid. The **auto-bind-tunnel** context allows the operator to define the preferred MPLS tunnel mechanism. The **resolution** command has options for **filter** or **any**. When **any** is selected, the system does a lookup in the tunnel-table and selects the tunnelling mechanism that is the most preferred (where most preferred is the lowest numeric value). When **filter** is selected, the **resolution-filter** context is used to filter the tunnelling protocols to only those that are deemed suitable for this service. In the example of *Output 6-1* only **sr-isis** is configured, and therefore only SR-ISIS can be used as a tunnelling mechanism for binding Next-Hops to MPLS LSPs.
- b. When **vxlan** is selected, the VXLAN header uses the VXLAN Network Identifier (VNI) as a demultiplexer between EVIs. The command **vxlan vni <number>** must therefore be configured at service level to provide that demultiplexing information. An example of using EVPN with VXLAN data-plane is provided later in this chapter.

As the intention is to use an SR-ISIS-based MPLS data-plane, the appropriate configuration using **mpls auto-bind-tunnel** is shown. In addition, **ecmp** is configured with a value of 2. This is to allow load-balancing of traffic from R1 destined towards CE3 via R3 and R6 that both support the all-active Ethernet segment (using aliasing). In other words, aliasing is enabled by configuring **ecmp**.

Output 6-1: EVPN-VPLS configuration at R1

```
vpls "three" {
  admin-state enable
  service-id 3
  customer "1"
  bgp-evpn {
    evi 3
    mac-duplication {
      retry 10
      detect {
        num-moves 4
        window 3
      }
    }
  }
  mpls 1 {
    admin-state enable
    ecmp 2
    auto-bind-tunnel {
      resolution filter
      resolution-filter {
        sr-isis true
      }
    }
  }
}
sap 1/1/c3/1:3 {
}
```

Once the **bgp-evpn mpls 1** context is administratively enabled the system will generate an Inclusive Multicast Ethernet Tag route (Route Type 3). This allows PE routers to auto-discover other members of the EVI, and through the use of the PMSI Tunnel attribute allows PE routers to advertise the P-tunnel mechanism that will be used for sending of broadcast unknown unicast, and multicast traffic into the EVI. *Debug 6-1* shows the Inclusive Multicast Ethernet Tag route generated by R1 (as received at R3). The NLRI shows the type `EVPN-INCL-MCAST` with the system generated Route-Distinguisher value and originator address set to the system address. The extended community attributes include a conventional Route-Target value (auto-generated through the **bgp-evpn evi** configuration), and the BGP Encapsulation extended community indicates MPLS as the tunnelling mechanism for the service. The PMSI Tunnel attribute indicates Ingress Replication and signals a downstream MPLS label that other PE routers should use when forwarding broadcast, unknown unicast or multicast traffic towards R1.

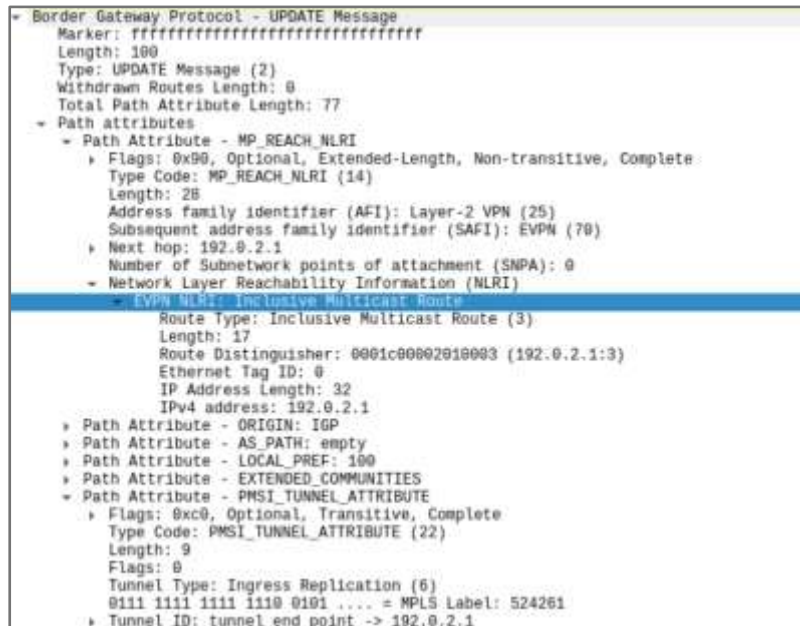
Debug 6-1: Inclusive Multicast Ethernet Tag Route received at R3

```
2 2025/08/20 15:04:36.568 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 91
  Flag: 0x90 Type: 14 Len: 28 Multiprotocol Reachable NLRI:
    Address Family EVPN
    NextHop len 4 NextHop 192.0.2.1
    Type: EVPN-INCL-MCAST Len: 17 RD: 192.0.2.1:3, tag: 0, orig_addr len: 32,
orig_addr: 192.0.2.1
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 0 AS Path:
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
  Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.1
  Flag: 0x80 Type: 10 Len: 4 Cluster ID:
    192.0.2.10
  Flag: 0xc0 Type: 16 Len: 16 Extended Community:
    target:64496:3
    bgp-tunnel-encap:MPLS
  Flag: 0xc0 Type: 22 Len: 9 PMSI:
    Tunnel-type Ingress Replication (6)
    Flags: (0x0)[Type: None BM: 0 U: 0 Leaf: not required]
    MPLS Label 8388176
    Tunnel-Endpoint 192.0.2.1
```

Figure 6-7 shows a packet capture of the same Update message from R1. The path attribute `MP_REACH_NLRI` shows the AFI of 25 (Layer-2 VPN) and SAFI of 70 (EVPN) together with the Route Type of 3 (Inclusive Multicast Route). The path attribute `PMSI_TUNNEL_ATTRIBUTE` is also expanded to show a tunnel type of ingress replication and an MPLS label of 524261 that should be used by other PEs when forwarding BUM traffic to R1. Observant readers may have noticed that the MPLS label shown in the debug output of *Debug 6-1* shows a value of 8388176 (which is actually outside of the MPLS label range used by SR-OS), whereas the label value associated with the same Inclusive Multicast Ethernet Tag route in *Figure 6-7* shows 524261. The reason for this is because the debug output can be considered as “real time”, and when the system is outputting to debug it hasn’t yet determined whether the MPLS label field contains a VXLAN VNI or MPLS label as determined by the BGP Tunnel Encapsulation

attribute. Therefore, the debug output shows the entire 24-bit value as received at the router.

Figure 6-7: Packet capture of Inclusive Ethernet Multicast Tag Route from R1



Output 6-2 shows the status of the EVPN at R3 after R1, R4, and R6 have advertised their Inclusive Multicast Ethernet Tag routes. Each of the PEs are listed as Tunnel Endpoints (TEPs) and each endpoint has an egress MPLS label that is used for forwarding broadcast, unknown unicast, and multicast traffic towards that PE (or endpoint). The output also contains a `Mcast` field. This field is set to `Yes` if a valid Inclusive Multicast Ethernet Tag route has been received from the remote Tunnel Endpoint. If the field is set to `No`, the MPLS (SR-ISIS) binding cannot be used to forward broadcast, unknown unicast and multicast traffic.

Output 6-2: EVPN-MPLS status at R3

```
A:admin@R3# show service id 3 evpn-mpls
```

```
=====
BGP EVPN-MPLS Dest (Instance 1)
=====
TEP Address                Transport:Tnl      Egr Label  Oper  Mcast  Num
                               State              MACs
-----
192.0.2.1                   isis:524315       524261     Up    bum    1
192.0.2.4                   isis:524316       524257     Up    bum    1
192.0.2.6                   isis:524335       524284     Up    bum    1
-----
Number of entries: 3
=====
```

Recall from Figure 6-6 that R3 and R6 support a multi-homed site (CE3), and as a result there are additional configuration requirements over that of R1 and R4. The significant difference is the definition of the Ethernet Segment (ES) and this is shown in Output 6-3. It's important to remember that the Ethernet Segment is not specific to any service but rather is a system-level entity. Its operating parameters are defined in the **service system bgp evpn** context,

and within this context the first line of configuration is the **route-distinguisher** which is a system-wide parameter used when sourcing Ethernet A-D per-ES routes to advertise ES capabilities. The Route-Distinguisher takes the format of IP address followed by number, and in this example is set to the system IP address followed by a zero. The **ethernet-segment** must then be created and given a name; in this case “*esi-1*”. This provides a new sub-context for defining the ES capabilities and operating mode.

The first parameter within the **ethernet-segment** sub-context is the **type** field. Originally Ethernet Segments were considered physical entities, such as a LAG or a port, but [24] extended the concept of Ethernet Segment to include pseudowires, VLANs, or other entities, all of which could be considered virtual. In this case the ES is configured as **type virtual** as it uses a VLAN range. This is followed by the **esi**, which is entered as 10 numbers separated either by a hyphen, a colon, or a space (00-11-22-33-44-55-66-77-88-99). Six ESI types are defined (Type 0-5), where the first (high-order) octet defines the format of the remaining 9-octets. A Type 5 (0x05) as shown in the example uses a 4-byte AS number followed by a 4-byte Local Discriminator value, with the low-order octet of the ESI value set to 0x00. The configured esi value (05:00:06:44:96:00:00:01:00:00) represents a Type 5 ESI (05), with the next 4-octets encoding an AS number of 64496 (00:06:44:96), and the following 4-bytes encoding a Local Discriminator value of 100 (00:00:01:00). The low-order octet is set to 0x00. Other possible ESI types include encoding of LACP System MAC and LACP Port Key (Type 1), Root Bridge MAC and priority (Type 2), System MAC and Local Discriminator value (Type 3), or Router ID and Local Discriminator value (Type 4). A Type 0 ESI type allows for an arbitrary 9-octet ESI value. SR-OS does however enforce that octets 2-7 are not all set to zero. The **multi-homing** command provides options for **single-active** or **all-active** multi-homing, and in this example uses **all-active**.

The **df-election** command provides the context for defining how the PE signals its DF and service-carving capabilities. The **es-activation-timer** permits a delay in the transition from non-DF to DF in order to avoid transient loops during switchover and can be configured with a value between 0 and 100 seconds. This is followed by the **service-carving-mode** command. Service-carving is the process of electing multiple DFs per ES to enable load-balancing of multi-destination traffic onto a given segment. It is done by electing a DF on a <ES, VLAN> basis. The options for service-carving are **auto**, **manual**, or **off**. The **auto** option uses a *mod N* function of the VLAN value to derive a DF as defined in [21]. The **off** option simply uses the lowest system IP address as DF for all services. The **manual** option allows the operator to manually configure a range of EVI values for which a nominated PE router will become DF. In this example the configured **evi** range is 1 to 99, and for this range (and indeed this ES) a **preference** value is used in the DF election.

A DF Election extended community is attached to the Auto-Discovery per-ES route which indicates whether the DF election is a

- Type 0 modulus-based algorithm [21]
- Type 1 highest random weight (HRW)-based algorithm [25]
- Type 2 Highest preference-based algorithm [26].
- Type 3 Lowest preference-based algorithm [26]

When **preference** is configured the DF Election extended community encodes a DF type 3 and a preference, with priority going to the lowest **preference** value. The DF Election extended community also contains a '*do-not-preempt*' DP-bit which if set means that any ES failover is non-revertive. It is configured using the **mode non-revertive** option following the **preference** command. The default behaviour is that the DP-bit is not set, and the ES is revertive. The example in *Output 6-3* shows that R3 has a preference of 100, whilst router R6 has a preference of 200. As lowest preference wins, R3 will be the DF. The **association** context allows the operator to select the type of object that this ES is based on, with the options being LAG, port, pw-port, or SDP. In this case **lag** is chosen, and "lag-10" is used. The **virtual-ranges** context allows for selection of either **dot1q** or **qinq** VLAN tags which constitute this virtual ES. In this example, the LAG is **qinq**, and the selected VLAN tags use the **s-tag** (outer Q-tag) with the range 1 to 999.

Output 6-3: EVPN Ethernet Segment configuration at R3

```

service {
  system {
    bgp {
      evpn {
        route-distinguisher "192.0.2.3:0"
        ethernet-segment "esi-1" {
          admin-state enable
          type virtual
          esi 0x050000644960000030000
          multi-homing-mode all-active
          df-election {
            es-activation-timer 0
            service-carving-mode manual
            manual {
              evi 1 {
                end 99
              }
              preference {
                value 100
              }
            }
          }
        }
        association {
          lag "lag-10" {
            virtual-ranges {
              qinq {
                s-tag 1 {
                  end 999
                }
              }
            }
          }
        }
      }
    }
  }
}

```

For completeness, *Output 6-4* shows the EVPN-VPLS configuration at R3, with a similar configuration applied at R6. From the perspective of the VPLS service provisioning, the configuration requirements are identical to that of R1 and R4. It is included here merely to

illustrate that by including a SAP in the service belonging to the ES (`sap lag-10:3.3`) that SAP (and service) automatically inherit the **ethernet-segment** configuration parameters.

Output 6-4: EVPN-VPLS configuration at R3

```
vpls "three" {
    admin-state enable
    service-id 3
    customer "1"
    proxy-arp {
        admin-state enable
        dynamic-populate true
    }
    bgp 1 {
    }
    bgp-evpn {
        evi 3
        mac-duplication {
            detect {
                num-moves 4
            }
        }
        mpls 1 {
            admin-state enable
            ecmp 2
            auto-bind-tunnel {
                resolution filter
                resolution-filter {
                    sr-isis true
                }
            }
        }
    }
}
sap lag-10:3.3 {
}
```

When the **ethernet-segment** ‘esi-1’ is placed into an **admin-state** of **enable** at router R3 a number of Update messages (MP_REACH NLRI) are generated. *Debug 6-2* shows the first of those Update messages, an Ethernet Segment route (Route Type 4). The NLRI of the route contains the RD value configured in the **service system bgp evpn** context as the Update is not specific to any given EVI but rather the ES. In addition to the RD, the NLRI contains the entire 10-octet ESI, as well as the originators IP address. The Update also contains an ES-Import Route-Target extended community attribute that encodes the high-order 6-octet portion of the ESI value. The ES-Import Route-Target is used instead of the Route-Target extended community to ensure that only PE routers that have corresponding Ethernet segment IDs will import this route. Finally, the DF-election extended community shows that the DF-type is preference, the ES has a revertive failover mechanism (DP-bit is not set), and the preference is 100

Debug 6-2: Ethernet Segment Route (Route Type 4) advertised by R3

```
9 2025/08/20 19:08:55.585 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 71
    Flag: 0x90 Type: 14 Len: 34 Multiprotocol Reachable NLRI:
        Address Family EVPN
```

```

NextHop len 4 NextHop 192.0.2.3
Type: EVPN-ETH-SEG Len: 23 RD: 192.0.2.3:0 ESI: 05:00:06:44:96:00:00:03:00:00,
IP-Len: 4 Orig-IP-Addr: 192.0.2.3
Flag: 0x40 Type: 1 Len: 1 Origin: 0
Flag: 0x40 Type: 2 Len: 0 AS Path:
Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
Flag: 0xc0 Type: 16 Len: 16 Extended Community:
df-election::DF-Type:Preference/DP:0/DF-Preference:100/AC:0
target:00:06:44:96:00:00

```

The same Ethernet Segment route is shown in the packet capture of *Figure 6-8*. The NLRI field is expanded to show the Ethernet Segment route (Type 4) comprising the RD, ESI, and Originating IP address of R3 (192.0.2.3) as previously described in *Figure 6-5*. There are two Extended Communities carried with the Update, an ES Import Route-Target and a DF Election. The latter indicates that this is the DF election type is preference and that R3 has a preference of 100. It also indicates that the ES failover is revertive.

Figure 6-8: Ethernet Segment Route advertised by R3



The second Update from R3 is shown in *Debug 6-3* and is an Ethernet A-D per-ES route. Recall that the Ethernet A-D per-ES route is a system level route used to advertise ES capabilities. The NLRI contains the RD, ESI, an MPLS label field set to 0, and an Ethernet Tag field set to the reserved value MAX-ET. The Update has a conventional Route-Target extended community used to determine which PE routers are eligible to import the Update, together with an ESI-Label extended community with a non-zero label value (524239). As R6 is the non-DF for this Ethernet segment, this is the label that it will use when forwarding broadcast, unknown unicast and multicast traffic received from the attached Ethernet segment towards R3, the DF. If a frame arrives at R3 with this label, it will not be forwarded onto the attached Ethernet segment. This is the process of split-horizon. Finally, the Single-Active bit of the Flags field in the ESI-Label extended community is set to a value of 0 meaning that the multi-homed site is operating in All-Active mode.

Debug 6-3: Ethernet A-D per-ES (Route Type 1) advertised by R3

```

7 2025/08/20 19:08:55.585 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 81
    Flag: 0x90 Type: 14 Len: 36 Multiprotocol Reachable NLRI:
        Address Family EVPN
        NextHop len 4 NextHop 192.0.2.3
        Type: EVPN-AD Len: 25 RD: 192.0.2.3:3 ESI: 05:00:06:44:96:00:00:03:00:00, tag:
MAX-ET Label: 0 (Raw Label: 0x0) PathId:
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0xc0 Type: 16 Len: 24 Extended Community:
        target:64496:3
        esi-label:524239/All-Active
        bgp-tunnel-encap:MPLS

```

Once again, a packet capture shows the same Ethernet AD per-ES Update message advertised by R3 with the NLRI expanded to illustrate the NLRI of the Ethernet A-D route (Type 1). The NLRI contains the RD, ESI, Ethernet Tag ID set to the maximum value for four octets of 4294967295 (expressed as MAX-ET in SR-OS debugs and outputs), and no MPLS label. The Update carries three Extended Communities, Route-Target, ESI showing all-active redundancy and a label value of 524239, and a BGP Encapsulation community.

Figure 6-9: A-D per-ES route advertised by R3

```

- Border Gateway Protocol - UPDATE Message
  Marker: ffffffffffffffffffffffffff
  Length: 194
  Type: UPDATE Message (2)
  Withdrawn Routes Length: 0
  Total Path Attribute Length: 81
  - Path attributes
    - Path Attribute - MP_REACH_NLRI
      - Flags: 0x90, Optional, Extended-Length, Non-transitive, Complete
      - Type Code: MP_REACH_NLRI (14)
      - Length: 36
      - Address family identifier (AFI): Layer-2 VPN (25)
      - Subsequent address family identifier (SAFI): EVPN (70)
      - Next hop: 192.0.2.3
      - Number of Subnetwork points of attachment (SNPA): 0
    - Network Layer Reachability Information (NLRI)
      - EVPN NLRI: Ethernet AD Route
        - Route type: Ethernet AD Route (1)
        - Length: 25
        - Route Distinguisher: 0001c0002030003 (192.0.2.3:3)
        - ESI: 05:00:06:44:96:00:00:03:00:00
        - Ethernet Tag ID: 4294967295
        - 0000 0000 0000 0000 0000 .... = MPLS Label 1: 0
    - Path Attribute - ORIGIN: IGP
    - Path Attribute - AS_PATH: empty
    - Path Attribute - LOCAL_PREF: 100
    - Path Attribute - EXTENDED_COMMUNITIES
      - Flags: 0xc0, Optional, Transitive, Complete
      - Type Code: EXTENDED_COMMUNITIES (15)
      - Length: 24
      - Carried extended communities: (3 communities)
        - Route Target: 64496:3 [Transitive 2-Octet AS-Specific]
        - ESI MPLS Label: All-Active redundancy, Label: 524239 [Transitive EVPN]
        - Encapsulation: MPLS Encapsulation [Transitive Opaque]

```

The final Update message generated by R3 as a result of administratively enabling the **ethernet-segment** “esi-1” is shown in *Debug 6-4* and is an Ethernet A-D per-EVI route. The Ethernet A-D per-EVI route is used for aliasing and is advertised regardless of whether the ESI is in All-Active mode or Active-Standby mode. It has a couple of subtle differences from the Ethernet A-D per-ES route, the most notable of which is that the Label field in the NLRI has a value (unlike the A-D per-ES route). When a PE router is load-balancing towards multiple PEs supporting an Ethernet segment, it uses the label value advertised with the Ethernet A-

D per-EVI route when forwarding traffic to a PE router that has not advertised the MAC address of the destination frame. In addition, as the Ethernet A-D per-EVI route deals with data-plane encapsulation (aliasing), it also contains the BGP Encapsulation extended community indicating MPLS as the tunnelling mechanism.

Debug 6-4: Ethernet A-D per-EVI (Route Type 1) advertised by R3

```
8 2025/08/20 19:08:55.585 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 73
    Flag: 0x90 Type: 14 Len: 36 Multiprotocol Reachable NLRI:
        Address Family EVPN
        NextHop len 4 NextHop 192.0.2.3
        Type: EVPN-AD Len: 25 RD: 192.0.2.3:3 ESI: 05:00:06:44:96:00:00:03:00:00, tag: 0
Label: 8388544 (Raw Label: 0x7fffc0) PathId:
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0xc0 Type: 16 Len: 16 Extended Community:
        target:64496:3
        bgp-tunnel-encap:MPLS
```

The packet capture below shows the expanded NLRI encoding the RD, ESI, Ethernet Tag, and MPLS label as previously described in *Figure 6-2*. The Ethernet Tag is set to zero, and the MPLS label value is 524284.

Figure 6-10: A-D per-EVI advertised by R3

```
Border Gateway Protocol - UPDATE Message
Marker: ffffffffffffffffffffffffffffffff
Length: 96
Type: UPDATE Message (2)
Withdrawn Routes Length: 0
Total Path Attribute Length: 73
+ Path attributes
  + Path Attribute - MP_REACH_NLRI
    + Flags: 0x90, Optional, Extended-Length, Non-transitive, Complete
    + Type Code: MP_REACH_NLRI (14)
    + Length: 36
    + Address family identifier (AFI): Layer-2 VPN (25)
    + Subsequent address family identifier (SAFI): EVPN (70)
    + Next hop: 192.0.2.3
    + Number of Subnetwork points of attachment (SNPA): 0
  + Network Layer Reachability Information (NLRI)
    + EVPN NLRI: Ethernet AD Route
      + Route type: Ethernet AD Route (1)
      + Length: 25
      + Route Distinguisher: 0001c00002030003 (192.0.2.3:3)
      + ESI: 05:00:06:44:96:00:00:03:00:00
      + Ethernet Tag ID: 0
      + 0111 1111 1111 1111 1100 .... = MPLS Label 1: 524284
    + Path Attribute - ORIGIN: IGP
    + Path Attribute - AS_PATH: empty
    + Path Attribute - LOCAL_PREF: 100
    + Path Attribute - EXTENDED_COMMUNITIES
```

At R3 the DF status for the **ethernet-segment** 'esi-1' can be verified. *Output 6-5* shows the list of DF candidates (R3 and R6) and their advertised preference values with the `yes` indication in the `DF` column indicating that R3 is indeed the DF. The same command at R6 (not shown) verifies that it is the non-DF for EVI 3. The DF unblocks transmit and receive for all traffic to/from the Ethernet segment. The non-DF can send traffic onto the segment for known destinations, but cannot forward broadcast, unknown unicast and multicast frames onto the segment.

Output 6-5: Ethernet Segment status at R3

```
A:admin@R3# show service system bgp-evpn ethernet-segment name "esi-1" evi evi-1 3
```

```
=====
```

```
EVI DF and Candidate List
```

```
=====
```

EVI	SvcId	Actv	Timer	Rem	DF	DF Last	Change
3	3	0			yes	08/20/2025	19:42:18

```
=====
```

```
DF Candidates
```

	Time Added	Oper Pref Value	Do Not Preempt
192.0.2.3	08/20/2025 19:42:18	100	Disabl*
192.0.2.6	08/21/2025 08:49:57	200	Disabl*

```
=====
```

```
Number of entries: 2
```

```
=====
```

With the EVI configuration in place and the auto-discovery process complete, the process of MAC learning can commence. Each PE learns MAC addresses on its local SAPs (or SDPs) using a combination of the following:

- i. Conventional data-plane MAC address learning.
- ii. Through the process of snooping ARP, GARP, and/or Neighbour Discovery (ND) messages generated from connected CE routers. This requires **proxy-arp** and/or **proxy-nd** to be configured and placed into an administratively enabled state and is discussed in more detail later in this chapter.
- iii. Through configuration of static MAC addresses towards a SAP or SDP. This approach is used for MAC addresses that do not move and require protection.

The MAC address of CE3 (52:54:00:b0:00:03) is locally learnt at R3 and is advertised in a MAC Advertisement route as shown in *Debug 6-5*. The NLRI contains the RD, ESI, MAC, and an MPLS label used for data-plane forwarding by remote PEs forwarding frames towards the advertised MAC. (Note that SR-OS does not include an IP address in the NLRI for any MAC addresses learnt in the data-plane on a SAP or SDP.) The Update includes a conventional Route-Target extended community attribute such that it is imported by all PEs in the EVI, together with a BGP Encapsulation extended community indicating MPLS as the tunnelling mechanism in use. It's worth reiterating that this MAC address could have been advertised by either R3 or R6 depending on CE3's hash routine. However, only one MAC advertisement is necessary since the MAC is associated with the ESI, and remote PEs will forward traffic for this MAC address to any PE supporting the associated ESI (assuming ECMP is enabled). If the MAC address ages out at R3 or the local attachment circuit should fail, or if there is any other reason that causes R3 to remove the MAC address, it simply sources an MP_UNREACH NLRI for the same NLRI causing remote PEs to update/flush their MAC tables.

Debug 6-5: MAC Advertisement route from R3

```
97 2025/08/21 09:53:30.396 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
```

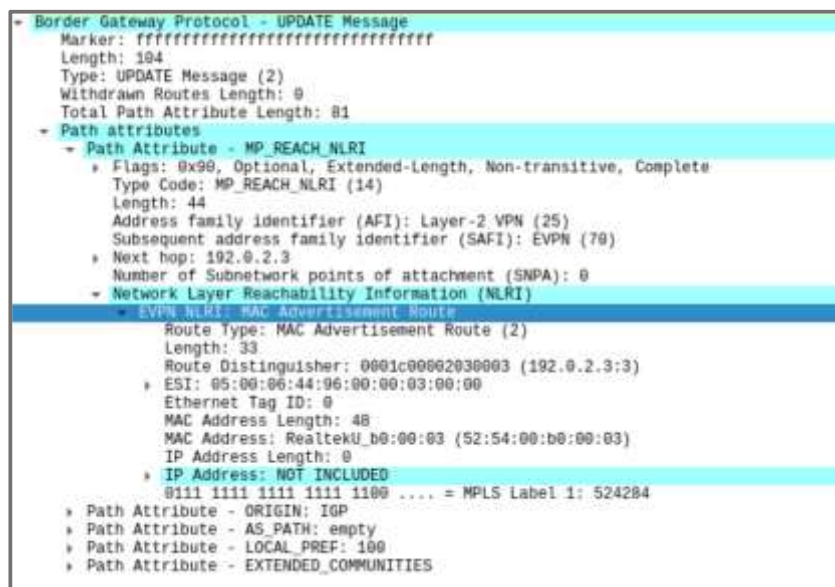
```

Withdrawn Length = 0
Total Path Attr Length = 81
Flag: 0x90 Type: 14 Len: 44 Multiprotocol Reachable NLRI:
  Address Family EVPN
  NextHop len 4 NextHop 192.0.2.3
  Type: EVPN-MAC Len: 33 RD: 192.0.2.3:3 ESI: 05:00:06:44:96:00:00:03:00:00, tag:
0, mac len: 48 mac: 52:54:00:b0:00:03, IP len: 0, IP: NULL, label1: 8388544 (Raw Label:
0x7fffc0)
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 0 AS Path:
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
  Flag: 0xc0 Type: 16 Len: 16 Extended Community:
    target:64496:3
    bgp-tunnel-encap:MPLS

```

The packet capture of the same Update message again shows the contents of the NLRI, indicating that it is a MAC Advertisement route (Type 2) encoding the RD, ESI, Ethernet Tag ID, MAC Address, IP Address, and MPLS label as previously described in *Figure 6-3*.

Figure 6-11: MAC Advertisement Route from R3



```

- Border Gateway Protocol - UPDATE Message
  Marker: ffffffff
  Length: 104
  Type: UPDATE Message (2)
  Withdrawn Routes Length: 0
  Total Path Attribute Length: 81
  - Path attributes
    - Path Attribute - MP_REACH_NLRI
      - Flags: 0x90, Optional, Extended-Length, Non-transitive, Complete
      - Type Code: MP_REACH_NLRI (14)
      - Length: 44
      - Address family identifier (AFI): Layer-2 VPN (25)
      - Subsequent address family identifier (SAFI): EVPN (70)
      - Next hop: 192.0.2.3
      - Number of Subnetwork points of attachment (SNPA): 0
    - Network Layer Reachability Information (NLRI)
      - EVPN NLRI: MAC Advertisement Route
        - Route type: MAC Advertisement Route (2)
        - Length: 33
        - Route Distinguisher: 0001c00002030003 (192.0.2.3:3)
        - ESI: 05:00:06:44:96:00:00:03:00:00
        - Ethernet Tag ID: 0
        - MAC Address Length: 48
        - MAC Address: RealtekU_b0:00:03 (52:54:00:b0:00:03)
        - IP Address Length: 0
        - IP Address: NOT INCLUDED
        - 0111 1111 1111 1111 1100 .... = MPLS Label 1: 524284
      - Path Attribute - ORIGIN: IGP
      - Path Attribute - AS_PATH: empty
      - Path Attribute - LOCAL_PREF: 100
      - Path Attribute - EXTENDED_COMMUNITIES

```

The MAC addresses of other CE routers (CE1, CE4) are locally learnt at their corresponding PE routers who also advertise those MAC addresses in MAC Advertisement routes to other PEs in the EVI. To verify this, *Output 6-6* shows the state of the VPLS Forwarding Database (FDB) at R1. In this output there are five entries, two of which are triggered by ESI label advertisements, and three of which are for known unicast MAC addresses:

- a. The first two entries (00:03:ff:00:03:3a and 00:06:ff:00:03:3a) are entries caused by A-D per-ES routes containing an ESI label from routers R3 and R6 hosting the Ethernet Segment 'esi-1'. The Source-Identifier column shows 192.0.2.3:524284 and 192.0.2.6:524282 against each of the MAC addresses respectively. The value 524284 is the ESI label advertised by R3 for the Ethernet Segment. Equally, 524282 is the ESI label advertised by R6 for the Ethernet Segment. These two entries are not required at R1 since it should have no reason to impose the

ESI label, but as the A-D per-ES route is advertised throughout the EVI, the entries are installed on all PE routers in the EVI.

- b. Entry three (52:54:00:b0:00:01) is CE1, and this MAC has been learned locally through SAP 1/1/c3/1:3.
- c. Entry four (52:54:00:b0:00:03) is CE3, and this has also been learned through an `Evpn` route. This MAC address was learnt in a MAC Advertisement route advertised by R3 (*Debug 6-5*), which also contained the ESI 05:00:06:44:96:00:00:03:00:00. As shown in *Output 6-7*, R1 has received an Ethernet A-D per-EVI route and an Ethernet A-D per-ES route from both R3 and R6 containing the same ESI value as that associated with the MAC Advertisement route. R1 also knows that the ESI is using All-Active multi-homing because the Single-Active bit in the ESI-label is set to zero. R1 can therefore load-balance traffic with known destinations to R3 and R6, and the `Source-Identifier` field therefore indicates the EVPN Ethernet segment ID (`eES`). When R1 forwards traffic destined for MAC address 52:54:00:b0:00:03 to R3 it uses the label associated with the MAC Advertisement route (524284). When R1 forwards traffic to R6 (that has not sourced a MAC Advertisement route), it uses the label advertised in the Ethernet A-D per-EVI route (524282).
- d. Entry four (52:54:00:b0:00:04) is CE4, and the `Type` column indicates that this has been learned through an `Evpn` route. The `Source-Identifier` column indicates that the forwarding path for this MAC is EVPN using an MPLS data-plane (`mpls-1`). It also indicates the Next-Hop of the advertising router, R4 (192.0.2.4), together with the MPLS label (524257) associated with the MAC Advertisement route.

Output 6-6: FDB table at R1

```
A:admin@R1# show service id 3 fdb detail
```

```
=====
Forwarding Database, Service 3
=====
```

ServId	MAC Transport:Tnl-Id	Source-Identifier	Type Age	Last Change
3	00:03:ff:00:03:3a	mpls-1: 192.0.2.3:524284	EvpnS:P	08/21/25 09:57:48
	isis:524301			
3	00:06:ff:00:03:3a	mpls-1: 192.0.2.6:524282	EvpnS:P	08/21/25 08:49:56
	isis:524328			
3	52:54:00:b0:00:01	sap:1/1/c3/1:3	LT/0	08/20/25 11:01:04
3	52:54:00:b0:00:03	eES: 05:00:06:44:96:00:00:03:00:00	Evpn	08/21/25 09:57:49
3	52:54:00:b0:00:04	mpls-1: 192.0.2.4:524257	Evpn	08/20/25 15:04:37
	isis:524294			

```
-----
No. of MAC Entries: 5
-----
Legend:L=Learned O=Oam P=Protected-MAC C=Conditional S=Static Lf=Leaf T=Trusted
=====
```

Output 6-7: Ethernet A-D Routes received at R1

```

A:admin@R1# show router bgp routes evpn auto-disc
=====
BGP Router ID:192.0.2.1          AS:64496          Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                  l - leaked, x - stale, > - best, b - backup, p - purge,
                  w - unused-weight-only
Origin codes   : i - IGP, e - EGP, ? - incomplete
=====
BGP EVPN Auto-Disc Routes
=====
Flag  Route Dist.      ESI                      NextHop
      Tag              Label
-----
u*>i  192.0.2.3:3      05:00:06:44:96:00:00:03:00:00  192.0.2.3
      0                                LABEL 524284

u*>i  192.0.2.3:3      05:00:06:44:96:00:00:03:00:00  192.0.2.3
      MAX-ET                             LABEL 0

u*>i  192.0.2.6:3      05:00:06:44:96:00:00:03:00:00  192.0.2.6
      0                                LABEL 524282

u*>i  192.0.2.6:3      05:00:06:44:96:00:00:03:00:00  192.0.2.6
      MAX-ET                             LABEL 0

-----
Routes : 4
=====

```

If one of the attachment circuits that form part of the Ethernet segment at R3 or R6 fails, the corresponding PE router generates a withdraw (MP_UNREACH NLRI) for the Ethernet A-D per-ES route, Ethernet A-D per-EVI route, and the Ethernet Segment route. *Debug 6-6* shows the MP_UNREACH generated by R3 as a result of the failure of the Ethernet segment (LAG-10). The following actions are triggered by other PE routers in the EVI:

- As a result of the withdrawal of the Ethernet Segment route, the non-DF will transition to DF.
- The withdrawal of the Ethernet A-D per-ES route causes receiving PE routers to update their MAC tables for all MAC addresses associated with this Ethernet segment, removing R3 from the list of Next-Hops. This is the process of mass withdraw which aids fast convergence.
- The withdrawal of the Ethernet A-D per-EVI route causes receiving PE routers that were formerly aliasing to update their MAC tables for this Ethernet segment, removing R3 from the list of viable Next-Hops.

Debug 6-6: A-D per-ES, per-EVI, and Ethernet Segment Route Withdraw from R3

```

106 2025/08/21 15:07:46.698 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
      Withdrawn Length = 0
      Total Path Attr Length = 86
      Flag: 0x90 Type: 15 Len: 82 Multiprotocol Unreachable NLRI:
        Address Family EVPN

```

```

Type: EVPN-ETH-SEG Len: 23 RD: 192.0.2.3:0 ESI: 05:00:06:44:96:00:00:03:00:00,
IP-Len: 4 Orig-IP-Addr: 192.0.2.3
Type: EVPN-AD Len: 25 RD: 192.0.2.3:3 ESI: 05:00:06:44:96:00:00:03:00:00, tag: 0
Label: 0 (Raw Label: 0x0) PathId:
Type: EVPN-AD Len: 25 RD: 192.0.2.3:3 ESI: 05:00:06:44:96:00:00:03:00:00, tag:
MAX-ET Label: 0 (Raw Label: 0x0) PathId:

```

To verify the data-plane of the EVPN a successful ping is issued from CE1 (172.16.100.1) to CE3 (172.16.100.3). *Figure 6-12* shows an Echo Request captured on the wire at the egress of R1 as it transits through the EVPN backbone. The frame contains a two-level label stack. The top label of {14003} represents R3's SR-MPLS Node-SID advertised through IS-IS. The bottom label of {524284} is the service label advertised by R3 as previously described in *Debug 6-5* and *Figure 6-11*.

Figure 6-12: Ping from CE1 to CE3 through EVPN

```

> Frame 11: 124 bytes on wire (992 bits), 124 bytes captured (992 bits) on interface bri2, id 1
> Ethernet II, Src: 52:54:00:f0:8f:f0 (52:54:00:f0:8f:f0), Dst: 52:54:00:80:ec:d2 (52:54:00:80:ec:d2)
> 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 100
> MultiProtocol Label Switching Header, Label: 14003, Exp: 0, S: 0, TTL: 255
> MultiProtocol Label Switching Header, Label: 524284, Exp: 0, S: 1, TTL: 255
> Ethernet II, Src: 52:54:00:b0:00:01 (52:54:00:b0:00:01), Dst: 52:54:00:b0:00:03 (52:54:00:b0:00:03)
✓ Internet Protocol Version 4, Src: 172.16.100.1, Dst: 172.16.100.3
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 84
  Identification: 0xe676 (58998)
  > 000. .... = Flags: 0x0
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: ICMP (1)
  Header Checksum: 0x740d [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 172.16.100.1
  Destination Address: 172.16.100.3
  [Stream index: 7]
> Internet Control Message Protocol

```

Proxy-ARP in EVPN

One of the main problems when providing large-scale Layer-2 multipoint services is controlling the amount of broadcast, unknown unicast, and multicast traffic that enters the service. It places a burden of replication on one or more routers and potentially consumes large amounts of bandwidth unnecessarily. The problem of unknown unicast traffic is largely negated in EVPN due to the use of the BGP EVPN control plane to advertise MACs, which avoids the data-plane flood-and-learn behaviour.

However, some Ethernet frames are inherently broadcast or multicast, and flooding of those frames is a requirement by default. One of these frame types is ARP (ff:ff:ff:ff:ff:ff) and its IPv6 counterpart Neighbour Discovery (ND) (33:33:00:00:00:02). To avoid the requirement to flood every ARP/ND frame throughout the EVI, the proxy-ARP and/or proxy-ND function can be used. The proxy-ARP/proxy-ND function allows PE routers to learn IP/MAC pair

associations, populate a proxy-ARP/ND cache with that information, and subsequently respond to ARP/ND requests from connected CE devices instead of flooding the frames. Using proxy-ARP IP/MAC pair associations can be learned from the following sources:

- IP/MAC pairs advertised through EVPN MAC Advertisement routes.
- User-configured static ARP/ND entries.
- Snooped dynamic IP/MAC pair associations learned from ARP/GARP/ND messages received on local SAPs/SDPs.

Output 6-8 provides an example of the basic configuration requirements for proxy-ARP within an EVPN. Within the VPLS service the **proxy-arp** container provides the context for configuring the operating parameters of the proxy-ARP cache. By default, the cache is populated only with EVPN routes once it is placed into an **admin-state** of **enable**. The **dynamic-populate true** command is used to allow dynamic entries from snooped ARP/GARP messages on SAPs/SDPs to also populate the cache. For the minimum configuration, the **proxy-arp** context is then administratively enabled. Other configuration options exist within the **proxy-arp** context including definition of static IP/MAC entries into the cache (**static**), setting of timers for aging out entries (**age-time**), and the ability to limit the size of the cache (**table-size**). A similar configuration also exists for proxy-ND using the **proxy-nd** context.

Output 6-8: Proxy ARP Configuration

```
service
  vpls "three" {
    proxy-arp {
      admin-state enable
      dynamic-populate true
    }
  }
}
```

The example topology in *Figure 6-6* is again used to illustrate how the proxy-ARP caches are populated. In this example CE1 has IP address 172.31.100.1 and MAC address 52:54:00:b0:00:01 and is connected to router R1. CE4 has IP address 172.31.100.4 and MAC address 52:54:00:b0:00:04 and is connected to router R4. The following sequence illustrates how the proxy-ARP caches are populated.

1. CE1 sends an ARP request for 172.31.100.4 (CE4).
2. R1 intercepts the ARP request and carries out the following:
 - a. It populates its FDB with CE1's source MAC 52:54:00:b0:00:01.
 - b. It advertises the MAC 52:54:00:b0:00:01 and IP address 172.31.100.1 in an EVPN MAC Advertisement route. If the MAC was previously advertised, the MAC Mobility extended community sequence number is not incremented.
 - c. It populates its proxy-ARP cache with IP/MAC address pair {172.31.100.1, 52:54:00:b0:00:01}.
 - d. It takes the IP/MAC pair learnt from the ARP request and sources a system-generated GARP message for that IP/MAC pair. The GARP message is sent to other SAPs/SDPs local to R1. If the **proxy-arp** option **evpn flood gratuitous-arp true** is enabled (default), the GARP message is also sent to the EVI. (Note that the **evpn flood gratuitous-arp true** command is equally applicable to GARP messages received from CEs.)

- e. The original ARP request can be flooded into the EVI or suppressed based on the **proxy-arp** option **evpn flood unknown-arp-req**. The default is enabled (**true**), and assuming this is not changed, R1 floods the ARP request into the EVI.
3. The ARP request is flooded to R4, who in turn floods it to CE4. CE4 then responds with an ARP reply.
4. R4 receives the EVPN MAC Advertisement route sent by R1 in step 2b. R4 populates its FDB with MAC address 52:54:00:b0:00:01 and populates its proxy-ARP cache with the IP/MAC address pair {172.31.100.1, 52:54:00:b0:00:01}.
5. When CE4 responds with an ARP reply, R4 carries out the following:
 - a. It populates its FDB with MAC 52:54:00:b0:00:04.
 - b. It advertises the MAC 52:54:00:b0:00:04 and IP address 172.31.100.4 in an EVPN MAC Advertisement route. Once again, if the MAC was previously advertised, the MAC Mobility extended community sequence number is not incremented.
 - c. It populates its proxy-ARP cache with IP/MAC address pair {172.31.100.4, 52:54:00:b0:00:04}.
6. Because R4 knows the destination of the Target IP and Target MAC¹ fields in the ARP reply from step 2b, the ARP-reply can be unicast to R1 instead of broadcast.
7. R1 receives the EVPN MAC Advertisement route sent by R4 in step 5b. It populates its FDB with MAC address 52:54:00:b0:00:04 and populates its proxy-ARP cache with the IP/MAC address pair {172.31.100.4, 52:54:00:b0:00:04}.

At this point, R1 and R4 can reply to any ARP request for IP address 172.31.100.1 or 172.31.100.4 without the requirement to flood an ARP request into the EVI. *Output 6-9* shows the contents of the proxy-ARP cache at R1 truncated to show only the ARP cache table. The `VPLS Proxy Arp Entries` table shows the IP/MAC pairs that been stored in the cache. The IP/MAC of CE1 (172.31.100.1, 52:54:00:b0:00:01) has a type of `dyn`, indicating that it was learnt dynamically by snooping on a local SAP. The IP/MAC of CE4 (172.31.100.4, 52:54:00:b0:00:04) has a type of `evpn`, indicating that the IP/MAC pair was learnt through an EVPN MAC Advertisement route.

Output 6-9: Proxy-ARP cache at R1

```
A:admin@R1# show service id 3 proxy-arp detail | match "Proxy Arp Entries" post-lines 10
VPLS Proxy Arp Entries
=====
IP Address           Mac Address          Type  Status  Flags  Last Update
-----
172.16.100.1         52:54:00:b0:00:01    dyn   active             09/01/2025 14:31:21
172.16.100.4         52:54:00:b0:00:04    evpn  active             09/01/2025 14:31:16
-----
Number of entries : 2
Legend : I=Immutable
=====
```

In order to maintain the liveliness of the proxy-ARP/ND cache, SR-OS may generate one of the following three types of ARP/ND messages for a given entry:

¹ When responding to an ARP request with an ARP reply, the target device (CE4) swaps the target and sender HA and IP pairs. As a result, the target HA and IP fields in the ARP reply reflect CE1's MAC and IP addresses.

Periodic refresh messages	Periodic refresh messages are ARP requests or Neighbour Solicitations for a given IP address. Their frequency is governed by the send-refresh command, and their objective is to keep existing FDB and proxy-ARP/ND cache entries alive. The default is never, which means they are never sent.
Unsolicited refresh messages	These are GARP or Neighbour Advertisement messages that are sent by the system when a new entry is learnt or updated. Their objective is to update the attached host/route caches.
Confirm messages	These are unicast ARP requests or unicast Neighbour Solicitation messages that are sent by the system when a new MAC is learnt for an existing IP address. The objective of the confirm message is to verify that a given IP address has actually moved to a different attachment circuit and is associated with the new MAC. If the IP address has not moved and this is actually a duplicate IP address, the response from the IP address will cause duplicate detection to activate.

When proxy-ARP is enabled, the system will advertise a “chassis” MAC in a MAC Advertisement Route with the static bit in the MAC Mobility extended community set to 1. This chassis MAC is installed in the FDB of remote PEs and is used as the source MAC address for confirm messages. By advertising the chassis MAC in a MAC Advertisement route, remote PEs responding to the confirm message will know where to send the confirm reply rather than flooding it in the EVPN network. *Output 6-10* shows the FDB at R1 with **proxy-arp** enabled. The first three entries in the FDB are the chassis MACs advertised by R3, R4, and R6 respectively and are known through type `EvpnS:P` with the last two flags used to indicate that these entries are static and protected.

Output 6-10: Chassis MACs in FDB table at R1

```
A:admin@R1# show service id 3 fdb detail
```

Forwarding Database, Service 3				
ServId	MAC Transport:Tnl-Id	Source-Identifier	Type Age	Last Change
3	00:03:ff:00:03:3a	mpls-1: 192.0.2.3:524284	EvpnS:P	08/26/25 13:22:33
	isis:524346			
3	00:04:ff:00:03:3a	mpls-1: 192.0.2.4:524283	EvpnS:P	09/01/25 14:30:43
	isis:524338			
3	00:06:ff:00:03:3a	mpls-1: 192.0.2.6:524281	EvpnS:P	08/26/25 13:27:19
	isis:524367			
3	52:54:00:b0:00:01	sap:1/1/c3/1:3	LT/0	08/28/25 17:09:55
3	52:54:00:b0:00:03	eES: 05:00:06:44:96:00:00:03:00:00	Evpn	08/26/25 14:42:09

```

3          52:54:00:b0:00:04 mpls-1:          Evpn      08/26/25 13:22:35
              192.0.2.4:524283
          isis:524338
-----
No. of MAC Entries: 6
-----
Legend:L=Learned O=Oam P=Protected-MAC C=Conditional S=Static Lf=Leaf T=Trusted
=====

```

Static Protected MACs

In certain scenarios there may be a requirement to configure some or all MAC addresses as static such that they are not permitted to move and are not subject to MAC move procedures. To permit this, these MAC addresses are advertised with a MAC Mobility extended community where the static/sticky flag is set to 1 and the sequence number is set to zero. These MAC addresses can be considered protected MACs and are referred to in SR-OS as *conditional* static MACs. The 'condition' being that they depend on the operational status of the SAP/SDP.

Output 6-11 shows the configuration of a static MAC for CE1's MAC address at R1. The **static-mac** context provides the container to configure the MAC-to-SAP binding. Following the **mac** address, the **monitor forward-status** argument specifies that the status of the SAP or SDP should be monitored when advertising the MAC and is used for dual-homing operations. If the SAP/SDP is up, the static MAC is advertised. If the SAP/SDP is operationally down, the static MAC is not advertised.

Output 6-11: Conditional Static MAC configuration at R1

```

service
  vpls "three" {
    fdb {
      static-mac {
        mac 52:54:00:b0:00:01 {
          sap 1/1/c3/1:3
          monitor forward-status
        }
      }
    }
  }
}

```

With the **static-mac** configuration in place on an operational SAP, *Output 6-12* shows the MAC Advertisement route advertised by R1, where the MAC Mobility extended community has the static (or sticky) bit set, and the sequence number is zero.

Output 6-12: Conditional Static MAC Advertisement route from R1

```

A:admin@R1# show router bgp routes evpn mac mac-address 52:54:00:b0:00:01 hunt | match
'Community|mac-mobility|Mac Address|Nexthop'
Nexthop      : 192.0.2.1
Res. Nexthop : n/a
Community    : target:64496:3 bgp-tunnel-encap:MPLS mac-mobility:Seq:0/Static
Mac Address   : 52:54:00:b0:00:01

```

In the FDB at R1, the conditional static MAC is shown as known through type CStatic.

Output 6-13: FDB entry with conditional static MAC at R1

```
A:admin@R1# show service id 3 fdb mac 52:54:00:b0:00:01

=====
Forwarding Database, Service 3
=====
ServId      MAC                Source-Identifier      Type      Last Change
      Transport:Tnl-Id      Age
-----
3           52:54:00:b0:00:01  sap:1/1/c3/1:3        CStatic:  09/01/25 17:21:40
                                   P
-----
Legend:L=Learned O=Oam P=Protected-MAC C=Conditional S=Static Lf=Leaf T=Trusted
=====
```

In the FDB of PEs receiving the MAC Advertisement route with the static bit set, the MAC address is shown as known through type EvpnS:P.

Output 6-14: FDB entry at remote PE for conditional static

```
A:admin@R4# show service id 3 fdb mac 52:54:00:b0:00:01

=====
Forwarding Database, Service 3
=====
ServId      MAC                Source-Identifier      Type      Last Change
      Transport:Tnl-Id      Age
-----
3           52:54:00:b0:00:01  mpls-1:                EvpnS:P   09/01/25 17:22:19
                                   192.0.2.1:524261
                                   isis:524300
-----
Legend:L=Learned O=Oam P=Protected-MAC C=Conditional S=Static Lf=Leaf T=Trusted
=====
```

As previously stated, static MACs are MACs that require protection. To protect configured static MAC addresses the following procedures are used:

- i. Once **bgp-evpn** is enabled in the VPLS service, all SAPs and SDPs within the service are internally configured for MAC protection with restricted learning. Protected MACs are contained in a MAC protect list, and the system will protect any MAC in this list from being learned or re-learned on any SAP or SDP with restricted learning.
- ii. Local static MACs or remote EVPN static MACs are considered as protected, and as such are contained in the MAC protect list.
- iii. If a frame with a source MAC address matching one of the protected MACs is received on a different SAP or SDP to that contained in the MAC protect list, the frame is discarded.

In the above example, R1 has configured a conditional static MAC for CE1's MAC address (52:54:00:b0:00:01) towards a local SAP and has advertised this MAC into EVPN. As shown in *Output 6-14* this is maintained in R4's FDB as EVPN static (EvpnS:P).

To test the protected status of this MAC, Ethernet frames are injected on the local SAP at R4 with the same protected source MAC (52:54:00:b0:00:01) and as illustrated in *Output 6-15*, these frames are locally discarded.

Output 6-15: Discarded frames at R4 due to Protected MAC

```

A:admin@R4# show service id 3 sap "1/1/c3/1:3" sap-stats | match 'Sap Statistics' post-
lines 11
Sap Statistics
-----
Last Cleared Time      : 09/01/2025 17:55:01
                        Packets              Octets
CPM Ingress           : 25                  1600
Forwarding Engine Stats
Dropped               : 25                  1700
Received Valid        : 0                   0
Off. HiPrio           : 0                   0
Off. LowPrio          : 0                   0
Off. Uncolor          : 0                   0
Off. Managed          : 0                   0

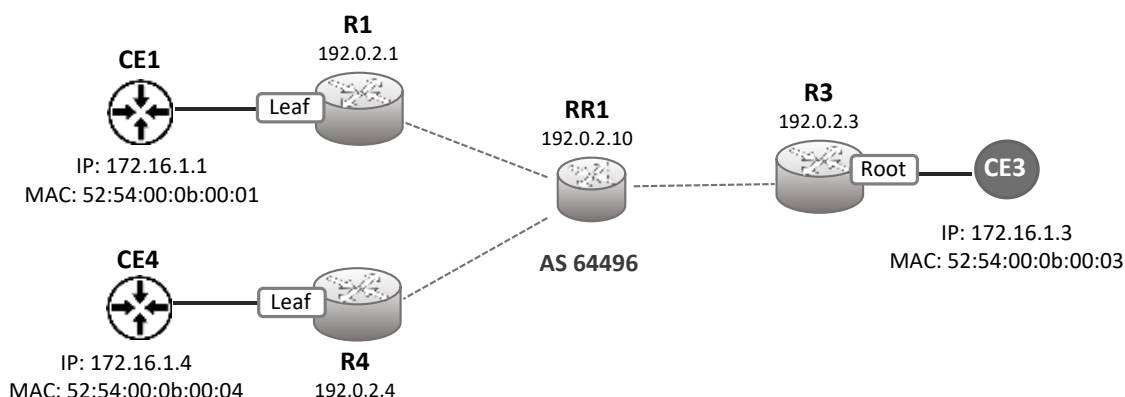
```

E-Tree

The Metro Ethernet Forum (MEF) has defined a rooted-multipoint Ethernet service known as an Ethernet-Tree (E-Tree) (MEF6.1). In an E-Tree service customer sites are typically represented as an Attachment Circuit (AC) which is labeled as either a Root or a Leaf site. Root sites can communicate with all other customer sites (both Root and Leaf sites). However, Leaf sites can communicate with Root sites but not with other Leaf sites. From a protocol standardisation perspective, E-Tree for EVPN is defined in [28], which introduces the EVPN E-Tree extended community to signal and identify Leaf ACs. The significant advantage that EVPN introduces over an E-Tree in a traditional VPLS service is that filtering of known unicast Leaf-to-Leaf traffic can be discarded at the ingress PE as opposed to the egress PE.

To demonstrate the use of E-Tree, the topology shown in *Figure 6-13* is used. R1, R3, and R4 all belong to AS 64496 and constitute the PE routers supporting EVPN. All of the PE routers are IBGP peered with a Route-Reflector, RR1, and are configured to support the EVPN Address Family. The IGP in use within AS 64496 is IS-IS, and all routers with the exception of RR1 are configured to run SR-ISIS. An EVPN-VPLS service is extended between R1, R3, and R4. CE1 and CE4 are attached to R1 and R4 respectively and represent leaf sites. CE3 is connected to R3 and represents a root site. That is, CE3 can communicate with both CE1 and CE4 and CE1 and CE4 can communicate with CE3 but not with each other.

Figure 6-13: Test Topology for EVPN E-Tree



Before configuring the E-Tree service the **etree-leaf-label true** command must be configured on all nodes with leaf ACs. The **etree-leaf-label true** command dynamically allocates an E-Tree leaf label on the system. Optionally the **etree-leaf-label-value** command may be used to statically configure a leaf label, which must come from the static label range, and must be the same on all nodes. (This is a pre-requisite for IXR Gen-1 platforms.)

Output 6-16: EVPN E-Tree Leaf label allocation

```

service {
  system {
    bgp {
      evpn {
        etree-leaf-label true
      }
    }
  }
}

```

Output 6-17 shows the configuration of the EVPN E-Tree service at router R1. The E-Tree service is a vanilla EVPN service except for the fact that the **etree true** command is present at service level to denote this as an E-Tree service, and the SAP towards CE1 is configured as a leaf AC using the **etree-leaf true** command. The same configuration is applied at R4, and the only notable exception at R3 is that the SAP is not configured with **etree-leaf**, which inherently makes it a root AC.

Output 6-17: EVPN E-Tree configuration at R1

```

vpls "eight" {
  admin-state enable
  service-id 8
  customer "1"
  etree true
  bgp-evpn {
    evi 8
    routes {
      incl-mcast {
        advertise-ingress-replication true
      }
    }
  }
  mpls 1 {
    admin-state enable
    auto-bind-tunnel {

```

```

        resolution filter
        resolution-filter {
            sr-isis true
        }
    }
}
sap 1/1/c3/1:8 {
    etree-leaf true
}
}

```

As shown in the above configuration, the command **incl-mcast advertise-ingress-replication true** dictates that all PEs in the EVI advertise a dedicated BUM label with their Inclusive Multicast Ethernet Tag (IMET) routes. *Output 6-18* taken at R1 shows that the SR label advertised for BUM traffic by R3 (192.0.2.3) is 524238 and the SR label advertised for BUM traffic by R4 (192.0.2.4) is 524240.

Output 6-18: EVPN-MPLS output at R1

```

A:admin@R1# show service id 8 evpn-mpls

=====
BGP EVPN-MPLS Dest (Instance 1)
=====
TEP Address                Transport:Tnl      Egr Label  Oper  Mcast  Num
                        State                        MACs
-----
192.0.2.3                   isis:524238       524238     Up    bum    0
192.0.2.4                   isis:524240       524240     Up    bum    0
-----
Number of entries: 2
=====
... [ snip ]

```

Filtering of BUM Traffic

To implement EVPN E-Tree each PE originates an A-D per-ES route to use for BUM egress filtering. The A-D per-ES route contains an EVPN E-Tree extended community containing a Flags field and a Leaf field. The Flags field defines an L-flag as the Leaf indication bit, and when set to 1 indicates a Leaf AC. The Leaf label is used for BUM egress filtering. The following output shows the A-D per-ES route advertised by R1 with Leaf label 524248 carried in the E-Tree extended community.

Output 6-19: A-D per-ES route advertised by R1

```

A:admin@R1# show router bgp routes evpn auto-disc rd 192.0.2.1:8 hunt

=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                l - leaked, x - stale, > - best, b - backup, p - purge,
                w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====

```

```

BGP EVPN Auto-Disc Routes
=====
-----
RIB In Entries
-----

RIB Out Entries
-----
Network          : n/a
Nexthop           : 192.0.2.1
Path Id           : None
To                : 192.0.2.10
Res. Nexthop      : n/a
Local Pref.       : 100
Interface Name    : NotAvailable
Aggregator AS     : None
Aggregator        : None
Atomic Aggr.      : Not Atomic
MED               : None
AIGP Metric       : None
IGP Cost          : n/a
Connector         : None
Community         : target:64496:8 etree::L:0/Leaf-Label:524248
                   bgp-tunnel-encap:MPLS
Cluster           : No Cluster Members
Originator Id     : None
Peer Router Id    : 192.0.2.10
Origin            : IGP
AS-Path           : No As-Path
EVPN type         : AUTO-DISC
ESI               : ESI-0
Tag               : MAX-ET
Route Dist.       : 192.0.2.1:8
MPLS Label        : LABEL 0
Route Tag         : 0
Neighbor-AS       : n/a
DB Orig Val       : N/A
Final Orig Val    : N/A
Source Class      : 0
Dest Class        : 0

-----
Routes : 1
=====

```

CE1 does not have an ARP entry for CE4 (172.16.1.4). Similarly, as R4 has not advertised a MAC route for CE4's MAC, R1 does not have an entry in its FDB for CE4. A ping is therefore originated from CE1 to CE4, which results in CE1 originating an ARP request for 172.16.1.4.

Output 6-20: Ping originated from CE1 to CE4

```

A:ce1# ping router 172.16.1.4
PING 172.16.1.2 56 data bytes
Request timed out. icmp_seq=1.
Request timed out. icmp_seq=2.
Request timed out. icmp_seq=3.
Request timed out. icmp_seq=4.
Request timed out. icmp_seq=5.

```

When this broadcast ARP frame arrives at R1 it is treated as BUM traffic. It is therefore flooded towards R3 and R4. A packet capture shows the packet forwarded towards R4 with label stack {14004, 524240, 524243}. The first label (14004) represents R4's SR-ISIS Node-SID. The second label (524240) represents the BUM label that R4 advertised with its IMET route for this service (see *Output 6-18*). The final label (524243) is the Leaf label that R4 advertised in its A-D per-ES route as shown in *Output 6-21*. In other words, the normal label stack is imposed (transport label, BUM label) and then the Leaf label is pushed on as the bottom-of-stack label with S=1.

Figure 6-14: Packet capture of CE1's ARP flooded towards R4

```

> Frame 17: 90 bytes on wire (720 bits), 90 bytes captured (720 bits) on interface bri2, id 1
> Ethernet II, Src: 52:54:00:f0:8f:f0 (52:54:00:f0:8f:f0), Dst: 52:54:00:80:ec:d2 (52:54:00:80:ec:d2)
> 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 100
> MultiProtocol Label Switching Header, Label: 14004, Exp: 0, S: 0, TTL: 255
> MultiProtocol Label Switching Header, Label: 524240, Exp: 0, S: 0, TTL: 255
> MultiProtocol Label Switching Header, Label: 524243, Exp: 0, S: 1, TTL: 0
> Ethernet II, Src: 52:54:00:0b:00:01 (52:54:00:0b:00:01), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
> Address Resolution Protocol (request)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: request (1)
  Sender MAC address: 52:54:00:0b:00:01 (52:54:00:0b:00:01)
  Sender IP address: 172.16.1.1
  Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
  Target IP address: 172.16.1.4

```

Output 6-21: A-D per-ES route advertised by R4

```

A:admin@R4# show router bgp routes evpn auto-disc rd 192.0.2.4:8 hunt | match Community
Community      : target:64496:8 etree::L:0/Leaf-Label:524243

```

A similar packet is flooded towards R3 with label stack {14003, 524238, 524240}. The first label (14003) represents R3s SR-ISIS Node-SID. The second label (524238) is the BUM label that R3 advertised with its IMET route for this service (*Output 6-18*). The final label (524240) is the Leaf label that R3 advertised in its A-D per-ES route as shown in *Output 6-22*.

Figure 6-15: Packet capture of CE1's ARP flooded towards R3

```

> Frame 18: 90 bytes on wire (720 bits), 90 bytes captured (720 bits) on interface bri2, id 1
> Ethernet II, Src: 52:54:00:f0:8f:f0 (52:54:00:f0:8f:f0), Dst: 52:54:00:80:ec:d2 (52:54:00:80:ec:d2)
> 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 100
> MultiProtocol Label Switching Header, Label: 14003, Exp: 0, S: 0, TTL: 255
> MultiProtocol Label Switching Header, Label: 524238, Exp: 0, S: 0, TTL: 255
> MultiProtocol Label Switching Header, Label: 524240, Exp: 0, S: 1, TTL: 0
> Ethernet II, Src: 52:54:00:0b:00:01 (52:54:00:0b:00:01), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
> Address Resolution Protocol (request)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: request (1)
  Sender MAC address: 52:54:00:0b:00:01 (52:54:00:0b:00:01)
  Sender IP address: 172.16.1.1
  Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
  Target IP address: 172.16.1.4

```

Output 6-22: A-D per-ES route advertised by R3

```

A:admin@R3# show router bgp routes evpn auto-disc rd 192.0.2.3:8 hunt | match Community
Community      : target:64496:8 etree::L:0/Leaf-Label:524240

```

When these packets reach the network ingress of R4 and R3 a normal ILM lookup is performed for each label (including the EVI label) in the stack. However, if a Leaf label follows then all the Leaf ACs are excluded when flooding to the domain.

Conversely, when R3 floods traffic into the EVI no Leaf label is present, which ensures that all Root to Leaf traffic is not dropped. With an empty ARP table at CE3 and an empty EVPN FDB table at R3, CE3 attempts a ping towards CE1 which generates an ARP request with a

destination MAC of ff:ff:ff:ff:ff:ff. When R3 receives this packet, it floods it into the EVI without a Leaf label as bottom-of-stack. *Figure 6-16* shows the ARP request flooded towards R1 with label stack {14001, 524247} representing R1s SR-ISIS Node-SID followed by the BUM label advertised in its IMET route (*Output 6-23*).

Figure 6-16: Packet capture of CE3's ARP flooded towards R1

```

> Frame 12: 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface bri5, id 0
> Ethernet II, Src: 52:54:00:ed:9c:1c (52:54:00:ed:9c:1c), Dst: 52:54:00:8a:b0:fb (52:54:00:8a:b0:fb)
> 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 100
> MultiProtocol Label Switching Header, Label: 14001, Exp: 0, S: 0, TTL: 255
> MultiProtocol Label Switching Header, Label: 524247, Exp: 0, S: 1, TTL: 255
> Ethernet II, Src: 52:54:00:0b:00:03 (52:54:00:0b:00:03), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
> Address Resolution Protocol (request)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: request (1)
  Sender MAC address: 52:54:00:0b:00:03 (52:54:00:0b:00:03)
  Sender IP address: 172.16.1.3
  Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
  Target IP address: 172.16.1.1

```

Figure 6-17 shows the same ARP request flooded towards R4 with label stack {14004, 524240} which represents R4's SR-ISIS Node-SID followed by the BUM label advertised in its IMET route (*Output 6-23*).

Figure 6-17: Packet capture of CE3's ARP flooded towards R4

```

> Frame 13: 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface bri5, id 0
> Ethernet II, Src: 52:54:00:ed:9c:1c (52:54:00:ed:9c:1c), Dst: 52:54:00:8a:b0:fb (52:54:00:8a:b0:fb)
> 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 100
> MultiProtocol Label Switching Header, Label: 14004, Exp: 0, S: 0, TTL: 255
> MultiProtocol Label Switching Header, Label: 524240, Exp: 0, S: 1, TTL: 255
> Ethernet II, Src: 52:54:00:0b:00:03 (52:54:00:0b:00:03), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
> Address Resolution Protocol (request)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: request (1)
  Sender MAC address: 52:54:00:0b:00:03 (52:54:00:0b:00:03)
  Sender IP address: 172.16.1.3
  Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
  Target IP address: 172.16.1.1

```

Output 6-23: EVPN-MPLS output at R3 showing advertised BUM labels

```

A:admin@R3# show service id 8 evpn-mpls

=====
BGP EVPN-MPLS Dest (Instance 1)
=====

```

TEP Address	Transport:Tnl	Egr Label	Oper State	Mcast	Num MACs
192.0.2.1	isis:524247	524247	Up	bum	0
192.0.2.4	isis:524240	524240	Up	bum	0

```

-----
Number of entries: 2
-----

```

It can therefore be concluded the BUM traffic is successfully precluded from leaf-to-leaf communication using filtering at the egress PE.

Filtering of Known Unicast Traffic

Unlike BUM traffic where filtering is carried out at egress, filtering of known unicast traffic is carried out at ingress. A PE participating in an E-Tree with a Leaf AC originates MAC/IP routes also containing the EVPN E-Tree extended community. The MPLS label value in the community is zero, but the L-flag is set to 1 if the AC is a Leaf. This allows other PEs in the EVI to associate MACs with Leaf or Root nodes.

In *Output 6-24* R1 has a Leaf AC towards CE1 hence when advertising a MAC route with CE1's MAC address, the L-flag is set to 1, the MPLS label in the E-Tree extended community is set to zero, and the MPLS1 label carries the label associated with the MAC/IP route as per normal EVPN signalling procedures.

Output 6-24: MAC Advertisement route from R1 for CE1's MAC address

```
A:admin@R1# show router bgp routes evpn mac rd 192.0.2.1:8 hunt
=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                  l - leaked, x - stale, > - best, b - backup, p - purge,
                  w - unused-weight-only
Origin codes   : i - IGP, e - EGP, ? - incomplete
=====
BGP EVPN MAC Routes
=====
-----
RIB In Entries
-----
-----
RIB Out Entries
-----
-----
Network       : n/a
Nextthop      : 192.0.2.1
Path Id       : None
To            : 192.0.2.10
Res. Nextthop : n/a
Local Pref.   : 100
Aggregator AS : None
Atomic Aggr.  : Not Atomic
AIGP Metric   : None
Connector     : None
Community     : target:64496:8 etree::L:1/Leaf-Label:0 bgp-tunnel-encap:MPLS
Cluster       : No Cluster Members
Originator Id : None
Origin        : IGP
AS-Path       : No As-Path
EVPN type     : MAC
ESI           : ESI-0
Tag           : 0
IP Address    : n/a
Interface Name : NotAvailable
Aggregator    : None
MED           : None
IGP Cost      : n/a
Peer Router Id : 192.0.2.10
```

```

Route Dist.      : 192.0.2.1:8
Mac Address      : 52:54:00:0b:00:01
MPLS Label1     : LABEL 524247      MPLS Label2      : n/a
Route Tag       : 0
Neighbor-AS     : n/a
DB Orig Val     : N/A                Final Orig Val    : N/A
Source Class    : 0                  Dest Class       : 0

```

```

-----
Routes : 1
=====

```

As R3 is a Root AC, MAC/IP routes that it advertises do not carry the E-Tree extended community with the Leaf indication bit set. The below output shows the MAC route originated for CE3 (52:54:00:0b:00:03) without an E-Tree extended community attached.

Output 6-25: MAC Advertisement route from R3 for CE3's MAC address

```

A:admin@R3# show router bgp routes evpn mac rd 192.0.2.3:8 hunt
=====
BGP Router ID:192.0.2.3      AS:64496      Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP EVPN MAC Routes
=====
-----
RIB In Entries
-----
-----
RIB Out Entries
-----
Network      : n/a
Nexthop      : 192.0.2.3
Path Id      : None
To           : 192.0.2.10
Res. Nexthop : n/a
Local Pref.  : 100
Interface Name : NotAvailable
Aggregator AS : None
Aggregator   : None
Atomic Aggr. : Not Atomic
MED          : None
AIGP Metric  : None
IGP Cost     : n/a
Connector    : None
Community    : target:64496:8 bgp-tunnel-encap:MPLS
Cluster      : No Cluster Members
Originator Id : None
Peer Router Id : 192.0.2.10
Origin       : IGP
AS-Path      : No As-Path
EVPN type    : MAC
ESI          : ESI-0
Tag          : 0
IP Address   : n/a
Route Dist.  : 192.0.2.3:8
Mac Address  : 52:54:00:0b:00:03
MPLS Label1  : LABEL 524238      MPLS Label2      : n/a
Route Tag    : 0
Neighbor-AS  : n/a
DB Orig Val  : N/A                Final Orig Val    : N/A
Source Class : 0                  Dest Class       : 0

```

```
-----
Routes : 1
=====
```

Advertising MAC routes in this manner allows each PE in the E-Tree service to learn of other MAC routes in the EVI and store them in the FDB together with an indication of whether the MAC is a Leaf. In *Output 6-26* taken at R1, CE3 (52:54:00:0b:00:03) is denoted as having been learned through EVPN (Evpn). CE4 (52:54:00:0b:00:04) has also been learned through EVPN but is stored in the FDB with an indication that it is a Leaf node (Evpn, Lf). If a unicast frame destined towards CE4 is received on the AC from CE1, it is dropped at ingress by R1. Note that these discarded frames are not shown in SAP statistics.

Output 6-26: EVPN-VPLS FDB at R1

```
A:admin@R1# show service id 8 fdb detail

=====
Forwarding Database, Service 8
=====
```

ServId	MAC	Source-Identifier	Type	Last Change
	Transport:Tnl-Id		Age	
8	52:54:00:0b:00:01	sap:1/1/c3/1:8	LT/0	08/27/25 14:25:40
8	52:54:00:0b:00:03	mpls-1: 192.0.2.3:524238	Evpn	08/27/25 14:25:41
	isis:524346			
8	52:54:00:0b:00:04	mpls-1: 192.0.2.4:524240	Evpn,Lf	08/27/25 14:30:29
	isis:524338			

```
-----
No. of MAC Entries: 3
-----
Legend:L=Learned O=Oam P=Protected-MAC C=Conditional S=Static Lf=Leaf T=Trusted
=====
```

As an example, CE1 can ping CE3 (172.16.1.3) at MAC 52:54:00:0b:00:03 because it is known through EVPN, but it is not able to ping CE4 (172.16.1.4) at MAC 52:54:00:0b:00:04 because the MAC route is associated with another Leaf AC at R1.

Output 6-27: Pings from CE1 to CE3 (successful) and CE4 (fail)

```
A:cel# ping router 4000 172.16.1.3
PING 172.16.1.3 56 data bytes
64 bytes from 172.16.1.3: icmp_seq=1 ttl=64 time=16.8ms.
64 bytes from 172.16.1.3: icmp_seq=2 ttl=64 time=14.4ms.
64 bytes from 172.16.1.3: icmp_seq=3 ttl=64 time=17.4ms.
64 bytes from 172.16.1.3: icmp_seq=4 ttl=64 time=14.2ms.
64 bytes from 172.16.1.3: icmp_seq=5 ttl=64 time=19.3ms.

---- 172.16.1.3 PING Statistics ----
5 packets transmitted, 5 packets received, 0.00% packet loss
round-trip min = 14.2ms, avg = 16.4ms, max = 19.3ms, stddev = 1.92ms

A:cel# ping router 4000 172.16.1.4
PING 172.16.1.4 56 data bytes
Request timed out. icmp_seq=1.
Request timed out. icmp_seq=2.
Request timed out. icmp_seq=3.
Request timed out. icmp_seq=4.
Request timed out. icmp_seq=5.
```

```

---- 172.16.1.4 PING Statistics ----
5 packets transmitted, 0 packets received, 100% packet loss

```

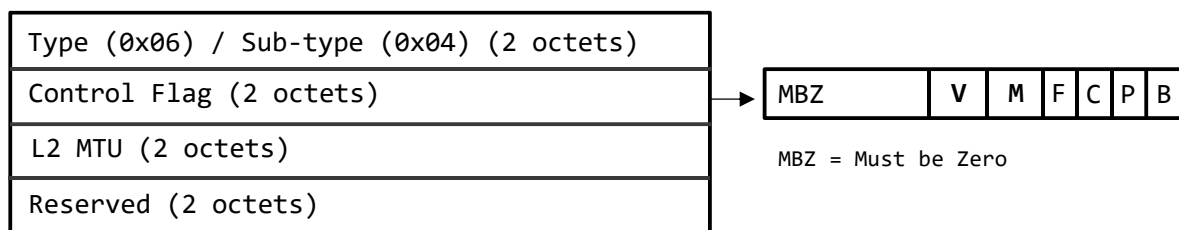
EVPN-VPWS

EVPN-VPWS uses a simplified version of the EVPN control plane used for EVPN-VPLS to deliver point-to-point services without a Layer 2 MAC lookup. Traffic is forwarded from attachment circuit to attachment circuit through the EVPN-VPWS as Ethernet frames and is frequently likened to the MEF E-Line service. By using the EVPN control plane, EVPN-VPWS also benefits from single-active redundancy as well as all-active redundancy with flow-based load-balancing.

The EVPN-VPWS control plane uses only the Ethernet Auto-Discovery Route (Type 1) to set up and maintain the end-to-end service. The MPLS label associated with the per-EVI Ethernet A-D route can be used in forwarding user traffic towards the destination attachment circuit since traffic from the originating Ethernet segment can only be forwarded to a single destination Ethernet segment, without requiring a MAC lookup. A given EVPN-VPWS instance is identified by a pair of per-EVI Ethernet A-D routes that together identify the VPWS service instance endpoints and the VPWS service instance. In the control plane the VPWS service instance is identified using Ethernet Tag advertised by each PE node in the NLRI (*Figure 6-2*). In the data plane, the value of the MPLS label advertised by one PE is used by the other PE to send traffic for that VPWS service instance.

The EVPN-VPWS control plane also makes use of the EVPN Layer 2 Attributes extended community, which is included with per-EVI Ethernet A-D routes. An L2 MTU field is used to indicate the MTU of the attachment circuit. A received L2 MTU of zero means that no MTU checking against the local MTU is needed. A received non-zero MTU must be checked against the local MTU, and if there is a mismatch the local PE must not add the remote as the EVPN destination for the corresponding VPWS service. The Control Flag field has a number of flags which are described below.

Figure 6-18: EVPN Layer 2 Attributes Extended Community



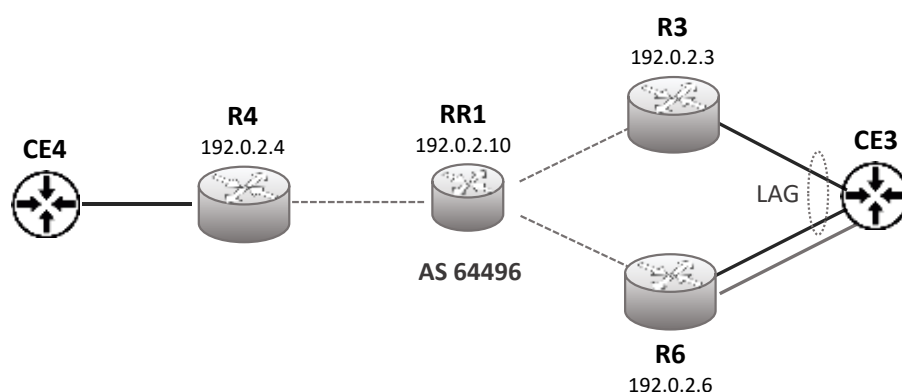
The Control Flags field currently defines six bits, which are described as follows:

- P If set to 1 in single-active multi-homing scenarios, this flag indicates that the advertising PE is the primary PE. It must be set to 1 for all-active multi-homing scenarios by all active PEs.
- B If set to 1 in single-active multi-homing scenarios, this flag indicates that the advertising PE is the backup PE.
- C If set to 1, a control word must be present when sending EVPN packets to this PE.
- F If set to 1, a flow label (hash label) must be present when sending EVPN packets to this PE.
- M 2-bits indicating the Flexible-Cross-Connect (FXC) [27] mode of operation. 00 is backwards compatible with devices not supporting FXC and signals that it is not in use. 01 indicates VLAN-signalled FXC, and 10 is the Default FXC.
- V 2-bits indicating the Flexible-Cross-Connect (FXC) [27] normalisation mode. 00 is backwards compatible with devices not supporting FXC and means it is not in use. 01 indicates single-VID normalisation, whilst 10 indicates double-VID normalisation.

Note that the M-bits and V-bits are used for Flexible-Cross-Connect (FXC) [27]. FXC provides a method of multiplexing multiple attachment circuits (VLANs) onto a single aggregate EVPN-VPWS service, saving labels and signaling resources. FXC and its various modes of operation is described later in this chapter.

To illustrate the use of EVPN-VPWS the topology shown in *Figure 6-19* is used. R3, R4, and R6 all belong to AS 64496 and constitute the PE routers supporting EVPN. All of the PE routers are IBGP peered with a Route-Reflector, RR1, and are configured to support the EVPN Address Family. The IGP in use within AS 64496 is IS-IS, and, as the intention is to run EVPN with an MPLS data-plane, all routers with the exception of RR1 are configured to run SR-ISIS. CE3 and CE4 constitute the CE routers that belong to the EVPN-VPWS instance. CE4 has a single link to R4. CE3 has a single-homed connection to R6 and also has a dual-homed LAG to both R3 and R6. This section will start by describing a straightforward EVPN-VPWS service with two single-homed CE routers. It will then move on to describe an EVPN-VPWS service that includes a dual-homed site.

Figure 6-19: Test Topology for EVPN-VPWS



Single-Homed EVPN-VPWS

The EVPN-VPWS service is initially constructed between R4 and R6, and unsurprisingly SR-OS models it as a **epipe** service in SR-OS. *Output 6-28* shows the configuration parameters for the service with an MPLS data-plane as configured at R4, with a similar configuration at R6. Within the **epipe** service, the **bgp-evpn** context provides the container to define the operating parameters of the EVI. The first line within this context is the **evi** command with an instance number that should be unique to this EVPN instance. For an EVPN-VPWS service, the **evi** does little but allow for auto-configuration of the Route-Distinguisher and Route-Target values. Like EVPN-VPLS, the Route-Distinguisher uses a Type 1 RD with the system address in the 4-byte Administrator subfield, followed by the EVI value in the 2-byte Assigned Number subfield. The Route-Target uses the router's Autonomous System in the 2-byte Global Administrator field followed by the EVI value in the 4-byte Local Administrator field. If the auto-configured Route-Distinguisher and Route-Target are not appropriate, or if the **evi** is omitted, the service-level **bgp** context can be used to manually provision both **route-distinguisher** and **route-target** values.

The **local-attachment-circuit** context allows for configuration of an **eth-tag** value, which is placed into the Ethernet Tag ID field of the advertised A-D per-EVI. At a receiving PE this Ethernet Tag ID must match the value that is locally configured under the **remote-attachment-circuit eth-tag** value in order for the EVPN-VPWS to become operational. For ease, the example uses **eth-tag 2** for both **local-** and **remote-attachment-circuit**.

The **bgp-evpn** node also provides the context to select the required data-plane tunnelling mechanism. This example uses **mpls**, in which case the PE router must resolve the BGP Next-Hop for EVPN routes to an MPLS tunnel before the prefix is considered valid. The **auto-bind-tunnel** context allows the user to select the preferred MPLS tunnel mechanism. The **resolution** command has options for **filter** or **any**. When **any** is selected, the system does a lookup in the tunnel-table and selects the tunnelling mechanism that is the most preferred (where most preferred is the lowest numeric value). When **filter** is selected, the **resolution-filter** context is used to filter the tunnelling protocols to only those that are deemed suitable for this service. In this example only **sr-isis** is configured, and therefore only SR-ISIS LSPs can be used as a tunnelling mechanism for binding Next-Hops to MPLS LSPs.

Output 6-28: Configuration of EVPN-VPWS service at R4

```
service {
  epipe "two" {
    admin-state enable
    service-id 2
    customer "1"
    sap 1/1/c3/1:2 {
    }
    bgp-evpn {
      evi 2
      local-attachment-circuit "local-AC" {
        eth-tag 2
      }
      remote-attachment-circuit "remote-AC" {
```

```

        eth-tag 2
    }
    mpls 1 {
        admin-state enable
        auto-bind-tunnel {
            resolution filter
            resolution-filter {
                sr-isis true
            }
        }
    }
}
}
}
}
}

```

When the **epipe** service is placed into an **admin-state** of **enable** at R4, the A-D per-EVI route is generated as shown in *Debug 6-7*. The NLRI contains an RD, an ESI field that is set to zero for a single-homed site, an Ethernet Tag ID value of 2, and an MPLS label. The EVPN L2 Attributes extended community shows an L2 MTU of 1514-bytes, with the F-bit, C-bit, P-bit, and B-bit all set to zero.

Debug 6-7: Ethernet A-D per-EVI Route from R4

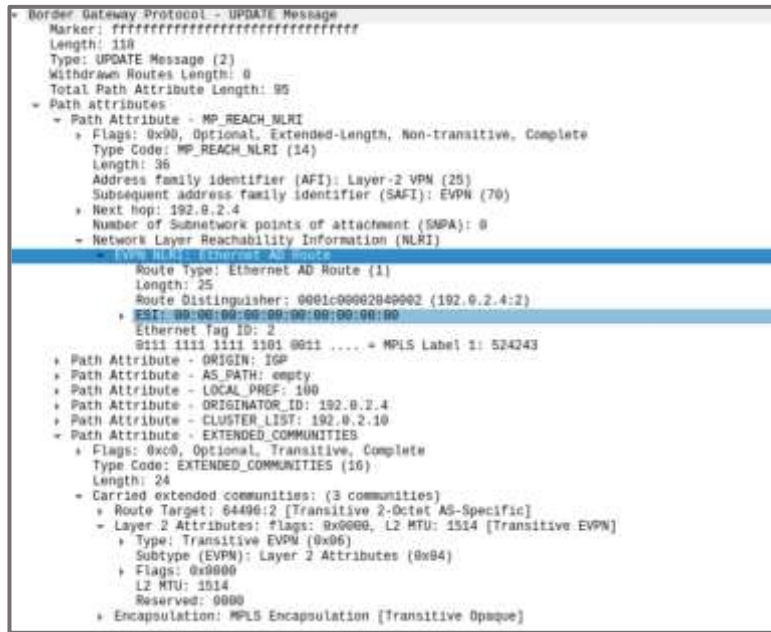
```

1 2025/08/21 16:27:22.039 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 81
    Flag: 0x90 Type: 14 Len: 36 Multiprotocol Reachable NLRI:
        Address Family EVPN
        NextHop len 4 NextHop 192.0.2.4
        Type: EVPN-AD Len: 25 RD: 192.0.2.4:2 ESI: ESI-0, tag: 2 Label: 8387888 (Raw
Label: 0x7ffd30) PathId:
        Flag: 0x40 Type: 1 Len: 1 Origin: 0
        Flag: 0x40 Type: 2 Len: 0 AS Path:
        Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
        Flag: 0xc0 Type: 16 Len: 24 Extended Community:
            target:64496:2
            l2-attribute:MTU: 1514 V: 0 M: 0 F: 0 C: 0 P: 0 B: 0
            bgp-tunnel-encap:MPLS

```

Figure 6-20 shows a packet capture of the Ethernet A-D per-EVI route with the NLRI attribute expanded to show the RD, ESI set to zero, Ethernet Tag ID of 2, and an MPLS label of 524243. There are three Extended Communities carried with the Update: Route-Target, Layer 2 Attributes, and BGP Encapsulation. The Layer 2 Attributes community shows the MTU of 1514-bytes and all Flags set to zero.

Figure 6-20: Ethernet A-D per-EVI Route advertised by R4



Output 6-29 shows verification at R6 that the **epipe** service is correctly signalled and operational. Note that the advertised MTU of 1514-bytes carried in the L2 attributes extended community differs to the actual MTU of the SAP. This is because the system uses the default **service-mtu** which is 1514-bytes. A mismatch in MTU will mean that the service will remain operationally down.

Output 6-29: Single-Homed EVPN-VPWS service verification

```
A:admin@R6# show service id 2 base
```

```
=====
Service Basic Information
=====
```

Service Id	: 2	Vpn Id	: 0
Service Type	: Epipe		
MACSec enabled	: no		
Name	: two		
Description	: (Not Specified)		
Customer Id	: 1	Creation Origin	: manual
Last Status Change:	08/21/2025 16:27:32		
Last Mgmt Change	: 08/21/2025 16:25:16		
Test Service	: No		
Admin State	: Up	Oper State	: Up
MTU	: 1514		
Vc Switching	: False		
SAP Count	: 1	SDP Bind Count	: 0
Per Svc Hashing	: Disabled	Lbl Eth/IP L4 TEID:	Disabled
Ignore MTU Mismatch	: Disabled		
Vxlan Src Tep Ip	: N/A		
Force QTag Fwd	: Disabled		
Lcl Switch Svc St	: sap		
Oper Group	: <none>		

```
-----
Service Access & Destination Points
-----
```

Identifier	Type	AdmMTU	OprMTU	Adm	Opr
------------	------	--------	--------	-----	-----

```

-----
sap:1/1/c3/1:2                                q-tag      9212      9212      Up      Up
=====
* indicates that the corresponding row element may have been truncated.

```

A failure of the SAP at R6 is simulated by placing it into an **admin-state of disable**, upon which R6 sends a Withdraw of the A-D per-EVI route and locally transitions the **epipe** service into an operationally down state.

Output 6-30: NLRI Withdrawal on SAP failure at R6

```

3 2025/08/25 13:55:57.809 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 34
    Flag: 0x90 Type: 15 Len: 30 Multiprotocol Unreachable NLRI:
        Address Family EVPN
        Type: EVPN-AD Len: 25 RD: 192.0.2.6:2 ESI: ESI-0, tag: 2 Label: 0 (Raw Label:
0x0) PathId:

```

Dual-Homed (single-active) EVPN-VPWS

For the example of the EVPN-VPWS service with dual-homing, the topology in *Figure 6-19* is again used, but in this instance CE3 is dual-homed to routers R3 and R6 using a LAG. EVPN-VPWS can support either all-active dual-homing or single-active dual-homing. The following example uses single-active dual-homing simply because the structure and configuration of the ESI is identical to that used with EVPN-VPLS, and an example of all-active ESI has already been described with EVPN-VPLS.

When using single-active dual-homing, the requirement is for the non-DF to notify its attached CE of its inactive state such that the CE does not forward traffic to it. This is possible at a service-level using ETH-CFM, but since that is not widely supported on CE routers, the example uses LACP standby signalling (sometimes referred to as Port-Active mode). This requires the use of an **oper-group**, which cannot be assigned to Ethernet Segments that are configured as virtual or contain EVI ranges; hence the Ethernet segment is also reconfigured to represent an association with a physical LAG.

Prior to detailing the service-specific configuration, the configuration required on R3 and R6 to support the single-active dual-homed Ethernet Segment is provided in *Output 6-31*. The configuration output is taken from R3, but a similar configuration is applied at R6. The advertised preference at R3 remains 100, whilst R6 advertises preference 200. When EVI ranges are used the lowest preference is preferred, however when a physical LAG/port is used the highest preference is preferred. Hence, R6 will be the DF and R3 will be the non-DF. Note that the **oper-group** is applied at the ESI level and its operational status is driven by the ES DF status. The **oper-group** is then monitored at LAG level using the **monitor-oper-group** command; hence when the **oper-group** status goes down, the LAG signals standby state using LACP.

When using an **oper-group** under a LAG (Port-Active mode) as in this example, it is recommended to use the **ac-df-capability exclude** command under the **ethernet-segment** configuration. When **ac-df-capability** is set to **include** (SR-OS default) the PEs connected to the ES build a candidate list for each EVI based on the presence of Ethernet Segment routes (Type 4) and Ethernet A-D per-ES/per-EVI (Type 1) routes. The framework for Ethernet VPN ESI DF election [79] specifies that when PEs build their candidate DF election list they do not include PEs when no A-D per-ES or A-D per-EVI routes for those PEs are present. With this behaviour, PE_m can be DF for one set of EVIs whilst PE_n can be DF for a different set of EVIs. When using a LAG with an **oper-group** (Port-Active mode), the LAG on the standby PE is operationally down and it is not possible therefore for that PE to be DF for any EVI. When the **ac-df-capability** is set to **exclude** the DF candidate list is built based only on the presence of Ethernet Segment (Type 4) routes. That is, the presence or absence of an A-D per-ES or A-D per-EVI does not influence the candidate DF election list. With this approach, even if one EVI fails on the active PE, it does not trigger a DF election on the backup. Should that happen, the ES state may become operationally up, which would cause the **oper-group** to become operationally up, which in turn would also cause the standby LAG to become operationally up. This is not the desired behaviour for Port-Active mode, hence the command **ac-df-capability** is set to **exclude** and must be configured consistently on all PEs attached to the same ES.

Output 6-31: Configuration at R3 and R6 for single-active Ethernet Segment

```
lag "lag-10" {
    admin-state disable
    encap-type qinq
    mode access
    lacp-xmit-interval slow
    monitor-oper-group "DF-LACP"
    lacp {
        mode active
        system-id 01:01:2e:00:00:4f
        administrative-key 32769
    }
    port 1/1/c6/1 {
    }
}
service {
    oper-group "DF-LACP" {
        hold-time {
            down 1
            up 0
        }
    }
}
system {
    bgp {
        evpn {
            route-distinguisher "192.0.2.3:0"
            ethernet-segment "esi-1" {
                admin-state enable
                esi 0x05000644960000030000
                multi-homing-mode single-active
                ac-df-capability exclude
                oper-group "DF-LACP"
                df-election {
                    es-activation-timer 0
                    service-carving-mode manual
                }
            }
        }
    }
}
```


name						
10	up	down	No	0	0	N/A
lag-10						

The same sequence of show outputs is taken at router R6 beginning with the status of the Ethernet segment. As can be seen there is a single service ID of 2 and R3 is the DF for this segment indicated by the *yes* in the DF column.

Output 6-35: Status of Ethernet Segment at R6

```
A:admin@R6# show service system bgp-evpn ethernet-segment name "esi-1" evi
```

```
=====
```

```
EVI Information
```

```
=====
```

EVI	SvcId	Actv Timer Rem	DF
2	2	0	yes

```
=====
```

```
Number of entries: 1
```

```
=====
```

As the **oper-group** status is driven by the DF election for the Ethernet segment, it follows that it is also operationally up.

Output 6-36: Status of Oper-Group at R6

```
A:admin@R6# show service oper-group "DF-LACP"
```

```
=====
```

```
Service Oper Group Information
```

```
=====
```

```
Oper Group      : DF-LACP
```

```
Creation Origin  : manual
```

```
Hold DownTime   : 1 secs
```

```
Members         : 1
```

```
Oper Status: up
```

```
Hold UpTime: 0 secs
```

```
Monitoring : 1
```

```
=====
```

As the LAG at R3 monitors the status of the **oper-group** and matches its operational status, it also follows that it is operationally up.

Output 6-37: Status of LAG at R6

```
A:admin@R6# show lag lag-10
```

```
=====
```

```
Lag Data
```

```
=====
```

Lag-id	Adm	Opr	Weighted	Threshold	Up-Count	MC	Act/Stdby
name							

```
=====
```

10	up	up	No	0	1		N/A
lag-10							

```
=====
```

Finally, the status of the LAG at CE3 is verified. Here it can be seen that port 1/1/c5/1 towards R6 (DF) is up, but port 1/1/c4/1 to R3 (non-DF) is down as a result of R3 signalling *out_of_sync* through LACP.

Output 6-38: LAG status at CE3

```
*A:ce3# show lag 10 port
```

```
=====
```

```
Lag Port States
```

```
Gen. Flags: p - Primary
```

```
LACP Flags: d - Disabled, e - Enabled, f - Fallback active
```

```
=====
```

Name								
Id	Port-id	Adm	Act/ Stdby	Opr	Flags	Sub-group	Forced	Prio

lag-10								
10	1/1/c4/1	up	active	down	pe	1	-	32768
	1/1/c5/1	up	active	up	e	1	-	32768

```
=====
```

The configuration at router R4 is identical to the configuration previously used for single-homed EVPN-VPWS and shown in *Output 6-28*. The configuration of the dual-homed EVPN-VPWS service at R3 is shown in *Output 6-39* with the same configuration applied at R6. The previously described **bgp-evpn** parameters do not change for a dual-homed site and therefore they are not repeated here. The configuration is shown here primarily to reiterate that by including a SAP in the service belonging to the Ethernet segment (`sap lag-10:2.2`) that SAP (and service) automatically inherits the behaviour of the **ethernet-segment** parameters.

Output 6-39: Dual-homed EVPN-VPWS service at R3 (and R6)

```

epipe "two" {
    admin-state enable
    service-id 2
    customer "1"
    sap lag-10:2.2 {
    }
    bgp-evpn {
        evi 2
        local-attachment-circuit "dual-homed-site" {
            eth-tag 2
        }
        remote-attachment-circuit "single-homed-site" {
            eth-tag 2
        }
    }
    mpls 1 {
        admin-state enable
        auto-bind-tunnel {
            resolution filter
            resolution-filter {
                sr-isis true
            }
        }
    }
}
}

```

When the Ethernet segment is placed into an administratively enabled state, both R3 and R6 advertise Auto-Discovery (A-D) (type 1) and Ethernet Segment (ES) (type 4) routes. There are two A-D routes: an A-D route per-ES and an A-D route per-EVI and *Output 6-40* shows those routes as advertised by R6. The first route is the A-D per-EVI (for EVI 2) route, which is

advertised on a per-service basis. It contains the value of the ESI for the Ethernet segment, the Ethernet tag (Tag) of 2, and an MPLS label. For EVPN-VPWS, the value of the advertised Ethernet tag must match the value of the locally configured **remote-attachment-circuit eth-tag** for the service to become operational. For a single-homed site the ESI value in the A-D per-EVI route is always zero (as shown in *Debug 6-7* and *Figure 6-20*) but as R6 is part of a dual-homed site the ESI value configured in the associated **ethernet-segment** is advertised. A receiving PE will therefore associate this A-D per-EVI route with that ESI and forward traffic to the appropriate egress PE based on receiving a valid A-D per-ES for that ESI. The A-D per-EVI also contains the EVPN L2 Attributes extended community (l2-attribute) which shows an L2 MTU of 1514-bytes, with the F-bit, C-bit, B-bit, all set to zero and the P-bit all set to one to indicate that it is primary (or the DF). The corresponding route advertised by R3 will have the F-bit, C-bit, and P-bit all set to zero with the B-bit set to one to indicate that it is backup.

The second route is the A-D per-ES route which is a system-level route advertising the ES capabilities such as the multi-homing mode shown as *Single-Active*, and is used for fast convergence, and split-horizon filtering with the ESI extended community label (esi-label). It is recognisable because the Ethernet Tag is set to MAX-ET and has an MPLS Label value of 0.

Output 6-40: A-D per-ES and per-EVI advertised by R3

```
A:admin@R6# show router bgp routes evpn auto-disc rd 192.0.2.6:2 hunt
=====
BGP Router ID:192.0.2.6          AS:64496          Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                  l - leaked, x - stale, > - best, b - backup, p - purge,
                  w - unused-weight-only
Origin codes   : i - IGP, e - EGP, ? - incomplete
=====
BGP EVPN Auto-Disc Routes
=====
-----
RIB In Entries
-----
-----
RIB Out Entries
-----
-----
Network       : n/a
Nexthop       : 192.0.2.6
Path Id       : None
To            : 192.0.2.10
Res. Nexthop  : n/a
Local Pref.   : 100
Aggregator AS : None
Atomic Aggr.  : Not Atomic
AIGP Metric   : None
Connector     : None
Community     : target:64496:2 l2-attribute:MTU: 1514 V: 0 M: 0 F: 0 C: 0 P:
                  1 B: 0 bgp-tunnel-encap:MPLS
Cluster       : No Cluster Members
Originator Id : None
Origin        : IGP
AS-Path       : No As-Path
Interface Name : NotAvailable
Aggregator    : None
MED           : None
IGP Cost      : n/a
Peer Router Id : 192.0.2.10
```

```

EVPN type      : AUTO-DISC
ESI            : 05:00:06:44:96:00:00:03:00:00
Tag            : 2
Route Dist.    : 192.0.2.6:2
MPLS Label     : LABEL 524238
Route Tag      : 0
Neighbor-AS    : n/a
DB Orig Val    : N/A
Source Class   : 0
Final Orig Val : N/A
Dest Class     : 0

-----

Network        : n/a
NextHop        : 192.0.2.6
Path Id        : None
To             : 192.0.2.10
Res. NextHop   : n/a
Local Pref.    : 100
Aggregator AS  : None
Atomic Aggr.   : Not Atomic
AIGP Metric    : None
Connector      : None
Community      : target:64496:2 esi-label:524284/Single-Active
                  bgp-tunnel-encap:MPLS
Cluster        : No Cluster Members
Originator Id  : None
Origin         : IGP
AS-Path        : No As-Path
EVPN type      : AUTO-DISC
ESI            : 05:00:06:44:96:00:00:03:00:00
Tag            : MAX-ET
Route Dist.    : 192.0.2.6:2
MPLS Label     : LABEL 0
Route Tag      : 0
Neighbor-AS    : n/a
DB Orig Val    : N/A
Source Class   : 0
Final Orig Val : N/A
Dest Class     : 0

-----

Routes : 2
=====

```

The A-D routes are accompanied by an Ethernet Segment (type 4) route, used for DF election purposes. The following output shows the Ethernet Segment route advertised by R6. The preference is advertised as 200, while the DP (Do-Not-Pre-empt) bit is set to 0 indicating the ES has a revertive failover restoral. The route also has an ES-Import Route Target extended community attribute (target:00:06:44:96:00:00), which is used instead of a conventional Route Target extended community. It encodes the high-order 6-octet portion of the 9-octet ESI value. This ensures that the route is imported only by PEs that are multi-homed to the same Ethernet segment (and therefore form the DF candidate list) and not by other PEs in the EVI.

Output 6-41: Ethernet Segment route advertised by R6

```

A:admin@R6# show router bgp routes evpn eth-seg rd 192.0.2.6:0 hunt
=====
BGP Router ID:192.0.2.6      AS:64496      Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only

```



```

Origin codes : i - IGP, e - EGP, ? - incomplete
=====
BGP EVPN Eth-Seg Routes
=====
-----
RIB In Entries
-----
-----
RIB Out Entries
-----
Network      : n/a
Nexthop      : 192.0.2.6
Path Id      : None
To           : 192.0.2.10
Res. Nexthop : n/a
Local Pref.  : 100
Aggregator AS : None
Atomic Aggr. : Not Atomic
AIGP Metric  : None
Connector    : None
Community    : df-election::DF-Type:Preference/DP:0/DF-Preference:200/AC:1
               target:00:06:44:96:00:00
Cluster      : No Cluster Members
Originator Id : None
Origin       : IGP
AS-Path      : No As-Path
EVPN type    : ETH-SEG
ESI          : 05:00:06:44:96:00:00:03:00:00
Originator IP : 192.0.2.6
Route Dist.  : 192.0.2.6:0
Route Tag    : 0
Neighbor-AS  : n/a
DB Orig Val  : N/A
Source Class : 0
Interface Name : NotAvailable
Aggregator    : None
MED           : None
IGP Cost      : n/a
Peer Router Id : 192.0.2.10
Final Orig Val : N/A
Dest Class    : 0
-----
Routes : 1
=====

```

To verify the data plane connectivity of the dual-homed EVPN-VPWS service, a ping is successfully passed from CE4 to CE3.

Output 6-42: Ping through EVPN-VPWS from CE4 to CE3

```

A:ce4# ping router 2 192.168.40.1
PING 192.168.40.1 56 data bytes
64 bytes from 192.168.40.1: icmp_seq=1 ttl=64 time=7.22ms.
64 bytes from 192.168.40.1: icmp_seq=2 ttl=64 time=6.57ms.
64 bytes from 192.168.40.1: icmp_seq=3 ttl=64 time=6.51ms.
64 bytes from 192.168.40.1: icmp_seq=4 ttl=64 time=6.26ms.
64 bytes from 192.168.40.1: icmp_seq=5 ttl=64 time=7.84ms.

```

Flexible-Cross-Connect (FXC)



Flexible-Cross-Connect is only supported on FP-based platforms. It is not supported on the 7250 IXR Gen 1 or Gen2/2c platforms.

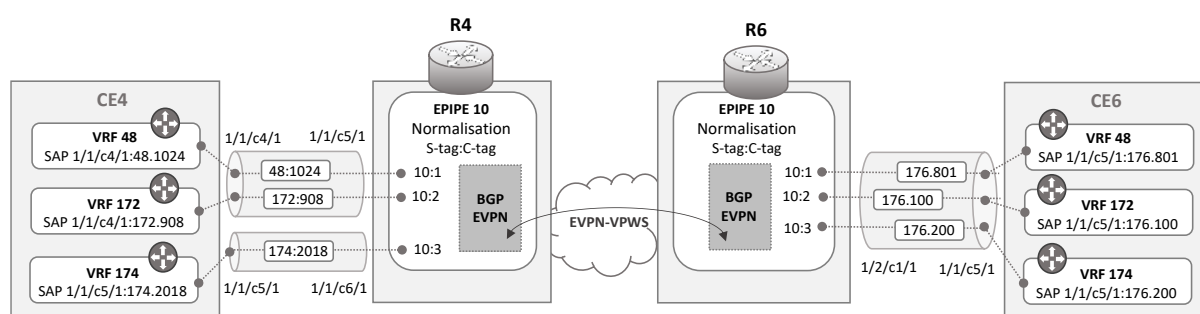
In some operator environments there may be a large number of attachment circuits (ACs) that need to be backhauled across an IP/MPLS network. A VPWS service for every pair of ACs can soon become a scaling problem, therefore there is a requirement to aggregate these ACs and multiplex many of them onto a single VPWS service tunnel in order to a) reduce the number of EVPN service labels associated with EVPN-VPWS services, and b) reduce the EVPN BGP signaling required to maintain each AC pair. In addition, these ACs (VLANs) may or may not require tag manipulation (or VLAN translation).

The specification in [29] describes a new EVPN-VPWS service that multiplexes multiple attachment circuits into a single EVPN VPWS service tunnel whilst still providing single-active and all-active multi-homing. Since multiplexing of ACs onto a single EVPN-VPWS service can be done across several physical interfaces, the possibility exists that there can be overlapping VLAN IDs across these interfaces. Therefore, these VLAN IDs must be translated into unique VLAN IDs to avoid collision. This VLAN ID translation is referred to as “VID normalisation”.

There are two options regarding the way in which multiple ACs are carried and signalled in a single EVPN-VPWS service tunnel. Both options are intended for single-homed or multi-homed endpoints. The first option does not offer a mechanism to signal the state of each AC to the far end and is the default mode operation for FXC. The second option is called VLAN-signalled flexible cross-connect and allows the state of each AC to be signalled via BGP so that restorative action can be taken.

The following topology is used to validate the FXC solution. CE4 and CE6 are attached to R4 and R6 respectively. CE4 is connected to R4 with two separate physical links such that attachment circuits can be presented to R4 on different ports (1/1/c5/1 and 1/1/c6/1). There is a single link between R6 and CE6 which represents a hand-off to a third-party provider where VLAN IDs are normalised to the values expected by that provider. The VLAN IDs presented to R4 on the left side of the schematic are intentionally different to those that R6 uses to connect to CE6 in each VPRN instance. To verify end-to-end connectivity three VRFs (VRF 48, 172, and 174) are configured on CE2 and CE6 and point-to-point interfaces are configured between each to test IP connectivity.

Figure 6-21: Network Topology for FXC testing



FXC in Default Mode

The default mode of FXC can be used for single-homed or multi-homed endpoints but does not provide a solution to signal the state of each AC to the far end. The following example uses single-homed endpoints, and the configuration shown in *Output 6-43* is used at R4 with a similar configuration applied at R6. When both endpoints are participating in FXC the same normalisation VLAN IDs must be configured at each endpoint. However, it is possible to have only a single endpoint using FXC and a regular EVPN VPWS as the other endpoint, which may be useful where VLAN manipulation is only required at one end of the service. The **flexible-cross-connect true** command allows the user to configure multiple SAPs as well as **bgp-evpn mpls** in the same **epipe** service. In this example there are three SAPs across two ports, each with individual Q-in-Q service-delimiting tags. The **qtag-normalization** context is used to translate, or normalise, the service-delimiting VLAN tags into different tags to be transported through the FXC service tunnel. Within this context there are options for **single-tag** or **double-tag** normalisation, with the latter being used in this example. Within the **bgp-evpn** context, a local and remote Ethernet tag must be configured as per normal EVPN-VPWS procedures. As a result, a single EVPN label is allocated per FXC service and there is no signalling of the individual (normalised) VLANs in BGP EVPN.

Output 6-43: Configuration for FXC Default Mode at R4

```

epipe "ten" {
    admin-state enable
    service-id 10
    customer "1"
    flexible-cross-connect true
    sap 1/1/c5/1:48.1024 {
        qtag-normalization {
            double-tag {
                c-tag 1
                s-tag 10
            }
        }
    }
    sap 1/1/c5/1:172.908 {
        qtag-normalization {
            double-tag {
                c-tag 2
                s-tag 10
            }
        }
    }
    sap 1/1/c6/1:174.2018 {
        qtag-normalization {
            double-tag {
                c-tag 3
                s-tag 10
            }
        }
    }
    bgp-evpn {
        evi 10
        local-attachment-circuit "R4-FXC-Default" {
            eth-tag 10
        }
        remote-attachment-circuit "R6-FXC-Default" {
            eth-tag 10
        }
    }
}

```

```

mpls 1 {
    admin-state enable
    auto-bind-tunnel {
        resolution filter
        resolution-filter {
            sr-isis true
        }
    }
}
}
}
}

```

In the data-plane, both imposition and disposition PEs must be aware of the VLAN as the imposition PE performs normalisation, and the disposition PE does a VLAN ID lookup and subsequent translation. As multiple VLANs may be carried in a single FXC service, the EVPN label alone is not sufficient to provide de-multiplexing information. More specifically, the EVPN label identifies the FXC service (VID-VRF), and then the normalised VLAN ID lookup identifies the egress interface. In summary, the normalisation and translation actions are as follows:

- At ingress, remove the service-delimiting VLAN tags present on the SAP and replace them with the normalised VLAN tags before inserting them into the EVPN-VPWS payload.
- At egress, once the MPLS label has identified the FXC service, normalised VLAN tags carried as part of the payload are stripped and service-delimiting VLAN tags are imposed before forwarding out onto the AC.

For clarity, *Table 6-3* lists the presented VLAN IDs/ACs for each of the three VPRN service instances instantiated on CE4 and CE6 together with the normalised VLAN IDs.

Table 6-3: Attachment Circuit and Normalised VIDs

CE VPRN	R4 ACs/VIDs	Normalised VIDs	R6 ACs/VIDs
VPRN 48	1/1/c5/1:48.1024	10:1	1/1/c5/1:176.801
VPRN 172	1/1/c5/1:172.908	10:2	1/1/c5/1:176.100
VPRN 174	1/1/c5/1:174.2018	10:3	1/1/c5/1:176.200

If both ends of the EVPN-VPWS are participating in the default FXC solution, each PE generates an Ethernet A-D per EVI route with ESI=0, the Ethernet Tag field set to the VPWS service instance ID, and the MPLS label field set to a dynamically generated EVPN service label representing the EVPN VPWS service tunnel. The route is also sent with the EVPN Layer-2 extended community extended to include the M-bits and the V-bits, both of which are 2-bits in length. They are used as follows:

- M 2-bits indicating the Flexible-Cross-Connect (FXC) [27] mode of operation.
00 is backwards compatible with devices not supporting FXC and signals that it is not in use. 01 indicates VLAN-signalled FXC, and 10 is the Default FXC.
- V 2-bits indicating the Flexible-Cross-Connect (FXC) [27] normalisation mode.
00 is backwards compatible with devices not supporting FXC and means it is

not in use. 01 indicates single-VID normalisation, whilst 10 indicates double-VID normalisation.

The following output shows the Ethernet A-D per EVI route originated by R4 after **epipe** “ten” is put into an **admin-state** of **enable**. The Layer 2 Attributes extended community can be seen with the P-bit, B-bit, and C-bit set to zero. The M-flag is set to 10 to indicate that the FXC mode of operation is default, and the V-flag is set to 10 to indicate that double-VID normalisation is taking place.

Output 6-44: A-D per-EVI route generated by R4 for FXC Default Mode

```
A:admin@R4# show router bgp routes evpn auto-disc rd 192.0.2.4:10 hunt
=====
BGP Router ID:192.0.2.4      AS:64496      Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                l - leaked, x - stale, > - best, b - backup, p - purge,
                w - unused-weight-only
Origin codes   : i - IGP, e - EGP, ? - incomplete
=====
BGP EVPN Auto-Disc Routes
=====
-----
RIB In Entries
-----
-----
RIB Out Entries
-----
Network       : n/a
Nexthop       : 192.0.2.4
Path Id       : None
To            : 192.0.2.10
Res. Nexthop  : n/a
Local Pref.   : 100
Aggregator AS : None
Atomic Aggr.  : Not Atomic
AIGP Metric   : None
Connector     : None
Community     : target:64496:10 l2-attribute:MTU: 1514 V: Double-VID M:
                Default F: 0 C: 0 P: 0 B: 0 bgp-tunnel-encap:MPLS
Cluster       : No Cluster Members
Originator Id : None
Origin        : IGP
AS-Path       : No As-Path
EVPN type     : AUTO-DISC
ESI          : ESI-0
Tag           : 10
Route Dist.   : 192.0.2.4:10
MPLS Label    : LABEL 524243
Route Tag     : 0
Neighbor-AS   : n/a
DB Orig Val   : N/A
Source Class  : 0
Interface Name : NotAvailable
Aggregator    : None
MED           : None
IGP Cost      : n/a
Peer Router Id : 192.0.2.10
Final Orig Val : N/A
Dest Class    : 0
-----
Routes : 1
=====
```

With the configuration applied and the exchange of the Ethernet A-D per-EVI routes the **epipe** in default FXC mode is administratively and operationally up as are all of the constituent SAPs.

Output 6-45: Epipe "ten" status with Default FXC Mode

```
A:admin@R4# show service id 10 base
```

```
=====
Service Basic Information
=====
```

```
Service Id       : 10                Vpn Id           : 0
Service Type     : Epipe
MACSec enabled   : no
Name            : ten
Description      : (Not Specified)
Customer Id      : 1                Creation Origin    : manual
Last Status Change: 08/26/2025 13:36:46
Last Mgmt Change : 08/26/2025 13:34:38
Test Service     : No
Admin State      : Up                Oper State        : Up
MTU              : 1514
Vc Switching     : False
SAP Count        : 3                SDP Bind Count     : 0
Per Svc Hashing  : Disabled          Lbl Eth/IP L4 TEID: Disabled
Ignore MTU Mismatch*: Disabled
Vxlan Src Tep Ip : N/A
Force QTag Fwd   : Disabled
Lcl Switch Svc St : sap
Oper Group       : <none>
```

```
-----
Service Access & Destination Points
-----
```

Identifier	Type	AdmMTU	OprMTU	Adm	Opr
sap:1/1/c5/1:172.908	qinq	9212	9212	Up	Up
sap:1/1/c5/1:48.1024	qinq	9212	9212	Up	Up
sap:1/1/c6/1:174.2018	qinq	9212	9212	Up	Up

```
=====
```

The configured per-SAP normalised VLAN IDs must be carried in the EVPN-VPWS service to allow the disposition PE to normalise the VLANs. A ping is initiated in VPRN 48 at CE4 (192.168.126.1) towards CE6 (192.168.126.2) to verify IP connectivity through the EVPN-VPWS FXC service and this is successful as shown.

Output 6-46: Ping from CE4 to CE6 for VPRN 48

```
A:ce4# ping router-instance "FXC-AC1" 192.168.126.2
PING 192.168.126.2 56 data bytes
64 bytes from 192.168.126.2: icmp_seq=1 ttl=64 time=8.41ms.
64 bytes from 192.168.126.2: icmp_seq=2 ttl=64 time=7.31ms.
64 bytes from 192.168.126.2: icmp_seq=3 ttl=64 time=7.02ms.
64 bytes from 192.168.126.2: icmp_seq=4 ttl=64 time=6.79ms.
64 bytes from 192.168.126.2: icmp_seq=5 ttl=64 time=7.18ms.
```

A packet capture is taken of the ping after it has been encapsulated by R4, which shows that the packet is encapsulated with a label stack of {14006, 524238} representing R6's SR-ISIS Node-SID of 14006 followed by the service label 524238 advertised by R6 in its Ethernet A-

D per-EVI route. The encapsulated Ethernet packet carries the configured normalised VLAN IDs of S-tag = 10 and C-tag = 1 as listed in *Table 6-3*.

Figure 6-22: Packet capture of FXC Default Mode with encapsulated normalised VLANs

```

> Frame 12: 132 bytes on wire (1056 bits), 132 bytes captured (1056 bits) on interface bri3, id 0
> Ethernet II, Src: 52:54:00:a8:02:0f (52:54:00:a8:02:0f), Dst: 52:54:00:a9:55:3e (52:54:00:a9:55:3e)
> 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 100
> MultiProtocol Label Switching Header, Label: 14006, Exp: 0, S: 0, TTL: 255
> MultiProtocol Label Switching Header, Label: 524238, Exp: 0, S: 1, TTL: 255
> Ethernet II, Src: 52:54:00:a1:1a:67 (52:54:00:a1:1a:67), Dst: 52:54:00:0c:16:68 (52:54:00:0c:16:68)
v 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 10
    000. .... = Priority: Best Effort (default) (0)
    ...0 .... = DEI: Ineligible
    .... 0000 0000 1010 = ID: 10
    Type: 802.1Q Virtual LAN (0x8100)
v 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 1
    000. .... = Priority: Best Effort (default) (0)
    ...0 .... = DEI: Ineligible
    .... 0000 0000 0001 = ID: 1
    Type: IPv4 (0x0800)
> Internet Protocol Version 4, Src: 192.168.126.1, Dst: 192.168.126.2
> Internet Control Message Protocol

```

While the default FXC mode uses a single EVPN label and single EVPN A-D per-EVI route, it has the disadvantage that a failure of a single SAP in the FXC service cannot be signalled in isolation. The remote PE will continue sending traffic towards the failed SAP, but it will be discarded at the destination PE. Multi-homing can be supported but all SAPs in the FXC must belong to the same Ethernet Segment (ES). While individual SAP failures still go unnoticed, port failures mean the entire ES fails and the A-D per-EVI and A-D per-ES routes are withdrawn.

Also note when using the default FXC mode that a physical port failure will only be signalled if it is the only port in the service. For example, in the test topology R6 has a single connection to CE6 and if this were to fail the A-D per-EVI route would be withdrawn by R6.

Debug 6-8: MP_UNREACH_NLRI for A-D per-EVI route from R6

```

185 2025/08/26 17:00:16.152 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 34
    Flag: 0x90 Type: 15 Len: 30 Multiprotocol Unreachable NLRI:
        Address Family EVPN
        Type: EVPN-AD Len: 25 RD: 192.0.2.6:10 ESI: ESI-0, tag: 10 Label: 0 (Raw Label:
0x0) PathId:

```

However, R4 has two physical links to CE4, and if one of these were to fail it would not be signalled and R4 would not withdraw the A-D per-EVI route because at least one VLAN in the FXC service is active.

FXC in VLAN-Signalled Mode

In the VLAN-signalled mode many normalised VLANs representing multiple ACs from multiple ESs and interfaces can be multiplexed into a single EVPN-VPWS service tunnel. However,

unlike the default FXC mode these normalised VIDs are signalled using the EVPN Address Family. For each normalised VID the PE generates an Ethernet A-D per-EVI route where the ESI field is set to zero for a non-ES interface and to the ESI value for an attachment circuit that is part of an ES. The Ethernet Tag field is set to the normalised VID, and the MPLS label field is set to a label taken from the dynamic label range. This label is the same for all attachment circuits multiplexed onto a single EVPN-VPWS service tunnel. When a single normalised VID is used, the lower 12 bits of the Ethernet Tag ID field is set to that VID. When a double normalised VID is used, the lower 12 bits of the Ethernet Tag ID field are set to the inner VID, while the higher 12 bits are set to the outer VID.

The configuration shown in *Output 6-47* is applied at R4 for the VLAN-signalled FXC service. The **flexible-cross-connect** command is still set to **true** at service level and the **qtag-normalization** remains the same, but within the **bgp-evpn** context there are subtle differences to the default FXC mode. For a generic EVPN-VPWS service, the Ethernet Tag ID within an Ethernet A-D route is encoded with the configured local **eth-tag**. To allow for per-AC state to be signalled across the FXC service, the VLAN-signalled mode encodes the normalised VLAN IDs within the Ethernet Tag ID field. The command **vlan-signalled-flexible-cross-connect true** enables the VLAN-signalled FXC mode and when is present the configuration of an **eth-tag** within the **local-attachment-circuit** and **remote-attachment-circuit** is blocked.

Output 6-47: Configuration at R4 for FXC VLAN-Signalled Mode

```

epipe "ten" {
  admin-state enable
  service-id 10
  customer "1"
  flexible-cross-connect true
  sap 1/1/c5/1:48.1024 {
    qtag-normalization {
      double-tag {
        c-tag 1
        s-tag 10
      }
    }
  }
  sap 1/1/c5/1:172.908 {
    qtag-normalization {
      double-tag {
        c-tag 2
        s-tag 10
      }
    }
  }
  sap 1/1/c6/1:174.2018 {
    qtag-normalization {
      double-tag {
        c-tag 3
        s-tag 10
      }
    }
  }
}
bgp-evpn {
  evi 10
  local-attachment-circuit "R4-FXC-VLAN-Signalled" {
    vlan-signaled-flexible-cross-connect true
  }
}

```



```

mpls 1 {
    admin-state enable
    auto-bind-tunnel {
        resolution filter
        resolution-filter {
            sr-isis true
        }
    }
}
}
}
}

```

When the VLAN-signalled EVPN-VPWS service is put into an **admin-state** of **enable** at R4, an A-D per-EVI route is advertised for each normalised VID. The example has three normalised VIDs; hence three A-D per-EVI routes are shown in *Debug 6-9* packed into a single Update message. The NLRI contains an RD, an ESI value of zero, and different Ethernet Tag IDs (*tag*).

In the debug output and the following wireshark capture of the same BGP Update message the Ethernet Tag ID is shown as 104845762, 10485761, and 10485763 respectively in each of the three NLRI representing the three normalised VLANs. The Ethernet Tag ID is part of the Ethernet A-D route NLRI. It is a 4-byte field used to identify a broadcast domain within an EVPN instance (which is often a VLAN), but for VLAN-aware FXC mode it encodes the normalised VLANs as two 12-bit entities. The C-tag is encoded in the right-most 12-bits of the right-most 2-bytes. The S-tag is encoded in the right-most 12-bits of the left-most two bytes.

- As an example, 10485762 is 0x00 0a 00 02. The C-tag is encoded in the right-most 12-bits of the right-most two bytes (00 02), which represents C-tag = 2 (binary 000000000010). The S-tag is encoded in the right-most 12-bits of the left-most two bytes (00 0a), which represents S-tag = 10 (binary 000000001010). The concatenated values are interpreted as 10485761/10485762/10485763 for VLANs 10:1/10:2/10:3 respectively.

Because the Ethernet Tag ID is part of the Ethernet A-D route, the encoding of the normalised VLANs into the Ethernet Tag ID field allows for per-AC signalling of state. In other words, each AC represents its own NLRI. Note also that in the Layer-2 Attributes extended community the M-flag is now set to 1 to indicate VLAN-signalled FXC mode is in use.

Debug 6-9: A-D per-EVI routes advertised by R4 for VLAN-signalled FXC mode

```

21 2025/08/26 17:38:39.353 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 135
    Flag: 0x90 Type: 14 Len: 90 Multiprotocol Reachable NLRI:
        Address Family EVPN
        NextHop len 4 NextHop 192.0.2.4
        Type: EVPN-AD Len: 25 RD: 192.0.2.4:10 ESI: ESI-0, tag: 10485762 Label: 8387856
(Raw Label: 0x7fffd10) PathId:
        Type: EVPN-AD Len: 25 RD: 192.0.2.4:10 ESI: ESI-0, tag: 10485761 Label: 8387856
(Raw Label: 0x7fffd10) PathId:
        Type: EVPN-AD Len: 25 RD: 192.0.2.4:10 ESI: ESI-0, tag: 10485763 Label: 8387856
(Raw Label: 0x7fffd10) PathId:
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:

```

```

Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
Flag: 0xc0 Type: 16 Len: 24 Extended Community:
    target:64496:10
    l2-attribute:MTU: 1514 V: Double-VID M: VLAN-signaled F: 0 C: 0 P: 0 B: 0
    bgp-tunnel-encap:MPLS

```

A packet capture also shows the three NLRI packed into a single BGP Update message. The interpretation of the Ethernet Tag ID field has already been explained and represent the three normalised VIDs. Although there are three A-D per-EVI routes, it is a single EVPN-VPWS service tunnel and as such a single MPLS label value is used for all three routes.

Figure 6-23: Packet capture of A-D per-EVI in FXC VLAN-signalled mode

```

- Border Gateway Protocol - UPDATE Message
  Marker: ffffffffffffffffffffffffffffffff
  Length: 158
  Type: UPDATE Message (2)
  Withdrawn Routes Length: 0
  Total Path Attribute Length: 135
  - Path attributes
    - Path Attribute - MP_REACH_NLRI
      - Flags: 0x90, Optional, Extended-Length, Non-transitive, Complete
      - Type Code: MP_REACH_NLRI (14)
      - Length: 90
      - Address family identifier (AFI): Layer-2 VPN (25)
      - Subsequent address family identifier (SAFI): EVPN (70)
      - Next hop: 192.0.2.4
      - Number of Subnetwork points of attachment (SNPA): 0
    - Network Layer Reachability Information (NLRI)
      - EVPN NLRI: Ethernet AD Route
        - Route Type: Ethernet AD Route (1)
        - Length: 25
        - Route Distinguisher: 0001c0000204000a (192.0.2.4:10)
        - ESI: 00:00:00:00:00:00:00:00:00
        - Ethernet Tag ID: 10485761
        - 0111 1111 1111 1101 0001 .... = MPLS Label 1: 524241
      - EVPN NLRI: Ethernet AD Route
        - Route Type: Ethernet AD Route (1)
        - Length: 25
        - Route Distinguisher: 0001c0000204000a (192.0.2.4:10)
        - ESI: 00:00:00:00:00:00:00:00:00
        - Ethernet Tag ID: 10485761
        - 0111 1111 1111 1101 0001 .... = MPLS Label 1: 524241
      - EVPN NLRI: Ethernet AD Route
        - Route Type: Ethernet AD Route (1)
        - Length: 25
        - Route Distinguisher: 0001c0000204000a (192.0.2.4:10)
        - ESI: 00:00:00:00:00:00:00:00:00
        - Ethernet Tag ID: 10485763
        - 0111 1111 1111 1101 0001 .... = MPLS Label 1: 524241
    - Path Attribute - ORIGIN: IGP
    - Path Attribute - AS_PATH: empty
    - Path Attribute - LOCAL_PREF: 100
    - Path Attribute - EXTENDED_COMMUNITIES

```

Prior to simulating a failure, a snapshot of the health of the EVPN-VPWS service tunnel in VLAN-signalled FXC mode at both R4 and R6 is captured. As shown below, the service is admin/operationally up/up and all SAPs are up.

Output 6-48: Healthcheck snapshot of EVPN-VPWS service tunnel at R4 and R6

```

A:admin@R4# show service id 10 base | match 'Admin State|Identifier|sap:'
Admin State      : Up          Oper State      : Up
Identifier       Type          AdmMTU  OprMTU  Adm  Opr
sap:1/1/c5/1:172.908    qinq          9212    9212    Up   Up
sap:1/1/c5/1:148.1024   qinq          9212    9212    Up   Up
sap:1/1/c6/1:174.2018   qinq          9212    9212    Up   Up

A:admin@R6# show service id 10 base | match 'Admin State|Identifier|sap:'
Admin State      : Up          Oper State      : Up
Identifier       Type          AdmMTU  OprMTU  Adm  Opr
sap:1/2/c1/1:176.100    qinq          9212    9212    Up   Up
sap:1/2/c1/1:176.200    qinq          9212    9212    Up   Up
sap:1/2/c1/1:176.801    qinq          9212    9212    Up   Up

```

The following command at R6 displays the active FXC SAP connections. As can be seen there are three active SAPs with their service delimiting and normalised tags displayed. The destination identifier is R4 (192.0.2.4) and the EVPN-VPWS is resolved over SR-ISIS.

Output 6-49: Active FXC SAP connections for the EVPN-VPWS service tunnel at R6

```
A:admin@R6# show service id 10 evpn-mpls fxc

=====
FXC SAP Connections (Instance 1)
=====
Dest Identifier          Sap (Normalized Tags)          Last Change
  Transport:Tnl-id
-----
mpls-1:192.0.2.4:524241    1/2/c1/1:176.100 (10.2)      08/26/2025
                               17:38:39
      isis:524316          1/2/c1/1:176.200 (10.3)      08/26/2025
                               17:38:39
                               1/2/c1/1:176.801 (10.1)      08/26/2025
                               17:38:39
-----
Number of Entries : 1
=====
```

To simulate a failure of a single attachment circuit, SAP 1/1/c5/1:172:908 (normalised to 10:2) is put into an **admin-state** of **disable** at R4. Thereafter, R4 generates a MP_UNREACH_NLRI Ethernet A-D per-EVI route for the attachment circuit with Ethernet Tag ID 10485762 (VID 10:2). This can be seen in *Debug 6-10* which shows the Update as received at router R6.

Debug 6-10: MP_UNREACH_NLRI for

```
206 2025/08/26 18:28:43.234 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 34
  Flag: 0x90 Type: 15 Len: 30 Multiprotocol Unreachable NLRI:
    Address Family EVPN
      Type: EVPN-AD Len: 25 RD: 192.0.2.4:10 ESI: ESI-0, tag: 10485762 Label: 0 (Raw
Label: 0x0) PathId:
```

Traffic is discarded at the ingress PE when there is no A-D per-EVI route for tags 10:2. The SAP stays operationally up, but traffic is dropped because there no remote peer matching tags are found. Notwithstanding its operational state, SAP statistics are available to show dropped packets.

Output 6-50: Health-check status of FXC SAPs at R6

```
A:admin@R6# show service id 10 base | match 'Admin State|Identifier|sap:'
Admin State      : Up          Oper State      : Up
Identifier       Type          AdmMTU  OprMTU  Adm  Opr
sap:1/2/c1/1:176.100  qinq         9212    9212    Up   Up
sap:1/2/c1/1:176.200  qinq         9212    9212    Up   Up
sap:1/2/c1/1:176.801  qinq         9212    9212    Up   Up
```

Although the SAP stays operationally up, a repeat of the command showing the active FXC SAP connections in *Output 6-49* prior to the simulated failure shows that the SAP with normalised tags 10:2 has been removed.

Output 6-51: Active FXC SAP connections for the EVPN-VPWS service tunnel at R6

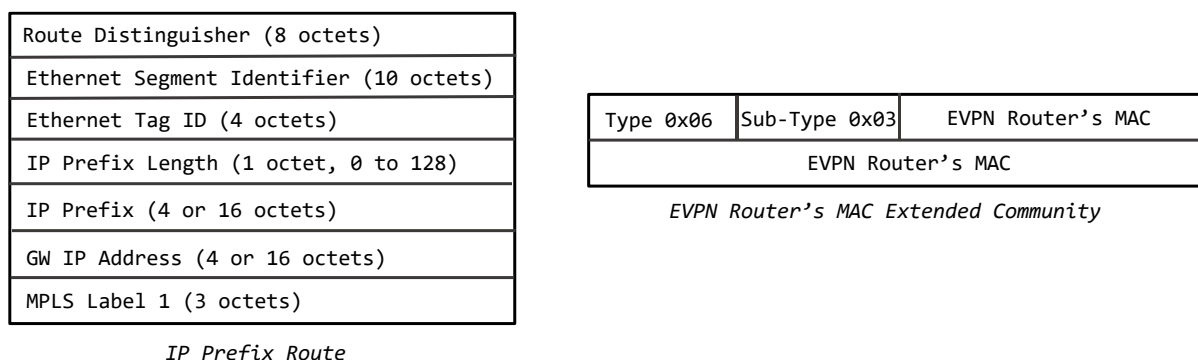
```
A:admin@R6# show service id 10 evpn-mpls fxc

=====
FXC SAP Connections (Instance 1)
=====
Dest Identifier          Sap (Normalized Tags)          Last Change
  Transport:Tnl-id
-----
mpls-1:192.0.2.4:524241  1/2/c1/1:176.200 (10.3)      08/26/2025
                               17:38:39
      isis:524316         1/2/c1/1:176.801 (10.1)      08/26/2025
                               17:38:39
-----
Number of Entries : 1
=====
```

IP Prefix Advertisement (Route-Type 5)

EVPN provides an extensible and flexible multihoming VPN solution for intra-subnet connectivity among tenant systems and end devices, where an IP subnet is represented by an EVPN instance (EVI). However, there are scenarios for which there is a need for dynamic and efficient inter-subnet connectivity among these tenant systems and end devices while maintaining the generic properties of EVPN. Inter-subnet communication is often performed by some centralised Layer 3 gateway, which can often result in backhauling and/or hairpinning of traffic. In order to overcome these issues, Integrated Routing and Bridging (IRB) [30] functionality can be enabled on the PEs and/or Network Virtualisation Edges (NVEs), which provides Layer 2 and/or Layer 3 services to attached tenant systems and end devices. When an NVE with IRB capability receives tenant traffic over an attachment circuit, it can either locally bridge the tenant intra-subnet traffic, or it can route the tenant inter-subnet traffic on a packet-by-packet basis. This allows for more optimal forwarding and avoids the sub-optimal centralised gateway approach. To facilitate inter-subnet connectivity, the IP Prefix Route (route Type 5) is introduced, and is specified in [29]. The encoding is illustrated in *Figure 6-24* where an IPv4 prefix has the length field of the BGP EVPN NLRI set to 34 and an IPv6 prefix has the length field of the BGP EVPN NLRI set to 58. *Figure 6-24* also shows the EVPN Router's MAC extended community attribute. This attribute is carried with a MAC Advertisement route (Type 2) and is advertised when a BGP Encapsulation extended community attribute specifies an encapsulation that requires an Ethernet frame. The EVPN Router's MAC extended community specifies the MAC address to be used as the inner destination MAC address in an Ethernet frame sent to the advertising PE. It is used for Overlay Indexing, and its use will be demonstrated throughout this section.

Figure 6-24: EVPN IP Prefix route encoding



The Route-Distinguisher, Ethernet Segment Identifier (ESI), and Ethernet Tag ID have been previously discussed earlier in this chapter and are therefore not repeated here. The IP Prefix Length can be set to a value between 0 and 32 for IPv4 and between 0 and 128 for IPv6 and it specifies the number of bits in the prefix. The MPLS label is encoded as 3 octets, where the high-order 20 bits contain the label value.

In a Data Centre environment, physical network functions (PNFs) Virtual Machines (VMs) and Containerised Machines (CMs) can connect to the EVPN and generate traffic to/from their own IP and MAC addresses. Other Virtual Appliances (VAs) such as firewalls, load-balancers, or NAT devices can forward traffic to/from IP addresses of different end devices sitting behind them with the same MAC address. These PNFs/VMs/CMs or VAs do not run dynamic routing protocols but instead rely on the NVE to advertise connected subnets on their behalf using the EVPN control plane. In addition, these PNFs/VMs/CMs and VAs can be moved between or within the same or different Data Centres by Cloud Management Systems, so a level of mobility must be supported.

If the intent of the IP Prefix route is that the NVEs advertise IP addresses together with a Next-Hop address, why not simply re-use the BGP/MPLS IP-VPN control plane? There are a number of reasons why the BGP/MPLS IP-VPN control plane cannot be re-used, not least that prefixes in a Data Centre must be advertised with flexibility that does not exist in IP-VPNs. For example, the advertised Next-Hop can be an IRB IP address used to connect an EVPN to a VRF for routing between subnets, or a floating IP address (such as a VRRP address, or even a MAC address). In addition, the requirement in a Data Centre is to simplify and run a single unified control plane.

If the BGP/MPLS IP-VPN control plane can't be used, then why not use the IP Address and IP Address Length fields of the MAC Advertisement route for IP prefix advertisement? Again, there are a number of reasons for this, and not least is the fact that some use-cases make the use of a MAC Advertisement route inefficient for IP prefix advertisement. For example, assume NVE1 and NVE2 are respectively connected to VA1 and VA2, which are running VRRP for redundancy. In normal operation VA1 (connected to NVE1) is the VRRP Master, and there is a requirement for NVE1 to advertise 1,000 IP prefixes. If a VRRP transition takes place such that VA2 transitions from Standby to Master and the VRRP logical IP address owner changes, the following takes place:

- a. If the MAC Advertisement Route is used for IP prefix advertisement, during the VRRP transition there is a requirement for NVE1 to send 1,000 Withdraws for the IP prefixes it originally advertised, and a further requirement for NVE2 to send 1,000 Updates for the 1,000 IP prefixes.
- b. If an IP Prefix Route is used for IP prefix advertisement, it is possible to advertise the 1,000 IP prefixes associated to the VRRP logical IP address, and only a single MAC Advertisement route for advertising ownership of the VRRP logical IP address associated to the VRRP logical MAC address. During the VRRP transition where the logical IP address and MAC changes, there is only a requirement for a single withdraw from NVE1 (withdrawing the MAC Advertisement Route) and a single Update from NVE2 (advertising the MAC Advertisement Route). So, there is a significant reduction in control plane activity using this approach. This technique is called *Overlay Indexing* and other examples of its use are described later in this section. It's worth mentioning that frequently passive VRRP is used in EVPN services. Passive VRRP avoids transitions of the logical IP address previously described by not sharing a Master/Standby relationship, however, the scenario with a change of ownership of a logical IP address still serves to illustrate a use-case for overlay indexing.

In addition to the above, by using a separate Route Type for IP prefix advertisement, there is a clean separation of functions between Route Types; Route Type 2 (MAC Advertisement Route) is used for MAC and ARP resolution advertisement, and Route Type 5 (IP Prefix route) is used for the advertisement of prefixes. The advertisement of IP prefixes is completely decoupled from the advertisement of any MAC addresses that may be related to those IP prefixes, and this has the potential to optimise the EVPN control plane significantly.

Type 5 IP Prefix routes support the concept of recursive lookup resolution through Overlay Indexes. The Overlay Index is a value that is used to direct the network to find the appropriate next hop for an IP prefix that is not directly reachable from the forwarding device. The Overlay Index is indicated along with the IP Prefix by the presence of a non-zero value in the ESI field, Gateway IP field, or EVPN Router's MAC extended community, depending on whether the IP Prefix next hop is an ESI, an IP address, or a MAC address. If an NVE/PE receives an IP Prefix route that specifies an Overlay Index, the NVE cannot use the IP Prefix in its VRF routing table until it has recursively resolved the Overlay Index:

- If the IP Prefix route specifies an ESI as the Overlay Index, a recursive resolution is only possible if the NVE/PE has received and installed a Route Type 1 (Auto-Discovery per-EVI) route specifying that ESI.
- If the IP Prefix route specifies a Gateway IP Address as the Overlay Index, a recursive resolution is only possible if the NVE/PE has received and installed a Route Type 2 (MAC/IP Advertisement route) specifying that IP address in the IP Address field of its NLRI.
- If the IP Prefix route specifies a MAC address as the Overlay Index, a recursive resolution can only be done if the NVE/PE has received and installed a Route Type 2 (MAC/IP Advertisement route) specifying that MAC address in the MAC Address field of its NLRI.

When using the IP Prefix route to advertise inter-subnet connectivity in the IP-VRF to IP-VRF model, the terms Broadcast Domain (BD) and Supplementary Broadcast Domain (SBD) are commonly used. A BD is exactly what it says and covers the extent of single-subnet connectivity, defining the boundary to which broadcast frames can be forwarded. In the simplest case of VLAN-based services an EVI consists of a single BD, but in the case of VLAN-aware bundle services it may contain multiple BD's. An SBD is a BD that does not have any attachment circuits, only IRB interfaces, and is used to provide connectivity among all the IP-VRFs of the tenant.

Section 4.4 of [29] discusses three potential ways to deliver EVPN IP Prefix distribution for the IP-VRF-to-IP-VRF model, depending on the existence and characteristics of the SBD and IRB interfaces for the IP-VRFs:

- 1) Interface-less model: No SBD and no Overlay Indexes required.
- 2) Interface-ful with SBD IRB model: This model requires an SBD, as well as Gateway IP address as Overlay Indexes.
- 3) Interface-ful with unnumbered SBD IRB model: This model requires SBD, as well as MAC addresses as Overlay Indexes.

The remainder of this section discusses the three models and provides worked examples of each.

Interface-ful with SBD IRB model

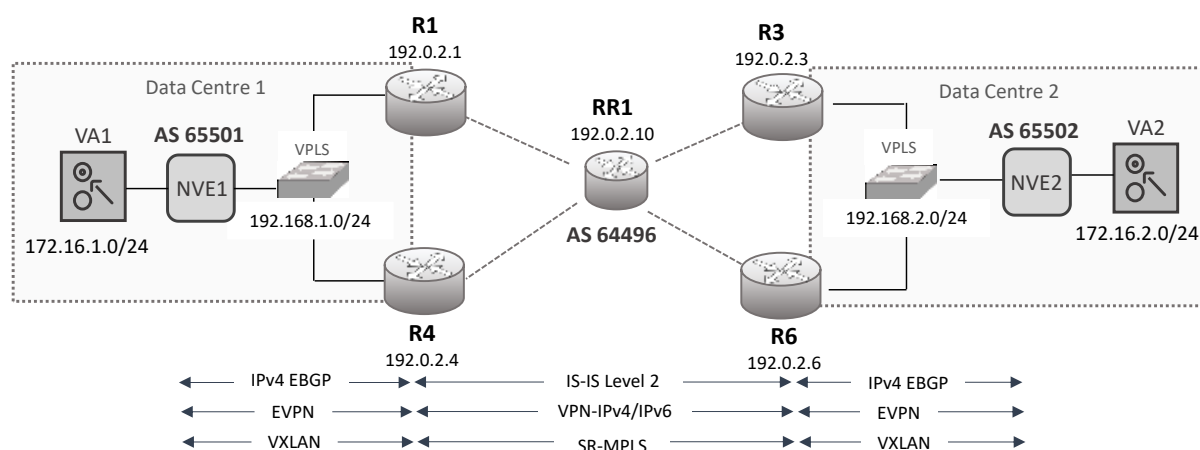
The Interface-ful (IFF) IP-VRF-to-IP-VRF model uses an SBD implemented as an R-VPLS bound to a full IP interface. That is to say, the IP interface has an IP address and MAC address specific to the SBD.

Figure 6-25 is used to demonstrate inter-subnet connectivity using the Interface-ful with SBD IRB model. R1, R3, R4, and R6 all belong to AS 64496 and make up the PE routers forming the IP/MPLS core. All of the PE routers are IBGP peered with a Route-Reflector, RR1, and are configured to support the VPN-IPv4/IPv6 Address Families. The IGP in use within AS 64496 is IS-IS, and, as the intention is to run an MPLS data-plane, all routers with the exception of RR1 are configured to run SR-ISIS. NVE1 within Data Centre 1 is dual-homed, with direct links to R1 and R4 and peers in EBGP with both PE routers with each peer advertising its system address. This can be considered the NVO underlay (or Data Centre fabric), which runs native IP. The data-plane within the Data Centre is VXLAN, which is used as a tunnelling mechanism to create the overlay networks. There is one or more overlay network for each tenant within the Data Centre, and each uses the VXLAN Network Identifier (VNI) within the VXLAN header as a demultiplexer. BGP EVPN is used as the control plane for overlay creation, and NVE1 therefore peers in EBGP with R1 and R4 for the EVPN Address Family. In the example overlay model within Data Centre 1, R1, R4, and NVE1 are connected to a multi-access segment 192.168.1.0/24 using a VPLS, which represents the SBD. NVE1 then connects to a Virtual Appliance (VA) with subnet 172.16.1.0/24. NVE2 within Data Centre 2 essentially replicates the setup of NVE1 in Data Centre 1. In the example overlay in Data Centre 2, R3, R6, and

NVE2 are connected to a multi-access segment 192.168.2.0/24 using a VPLS. NVE2 is connected to a VA with subnet 172.16.2.0/24.

In this topology the PE routers act as Data Centre Gateways. Towards the Data centres they run an IPv4 EBGp underlay, BGP EVpn overlay, and VXLAN data-plane. Towards the core they run BGP/MPLS IP-VPN. The objective is to facilitate inter-subnet connectivity between VA1 subnet 172.16.1.0/24 in Data Centre 1 and VA2 subnet 172.16.2.0/24 in Data Centre 2 and will focus on the configuration requirements of the PE routers functioning as Data Centre Gateways.

Figure 6-25: Test Topology for Interface-ful with SBD IRB model



The necessary service configuration for router R1, functioning as a data centre gateway, is shown in *Output 6-52*. The process of switching intra-subnet traffic at Layer-2 and concurrently allowing for routing of traffic between subnets at Layer-3 is commonly referred to as IRB, and within SR-OS an IRB is implemented using a Routed-VPLS, or R-VPLS. An R-VPLS is essentially a routing instance (IES or VPRN) that is logically linked to a VPLS instance. Hence, *Output 6-52* contains both a VPRN service instance (eleven) and a VPLS service instance (twelve). The logical link between the two instances is created when the VPLS service instance uses the **routed-vpls** command, which then allows the VPRN service to refer to the service name of the VPLS under an IP interface (in place of a SAP or SDP). Hence **interface “evi-twelve”** refers to **vpls “twelve”**.

The VPRN contains the typical **route-distinguisher** and **vrf-target** parameters previously discussed in chapter 3. It also contains the **auto-bind-tunnel** context, which is configured with a **resolution-filter** of **sr-isis** for VPN-IPv4 prefix Next-Hop resolution. The **domain-id** represents the Domain Path, or D-PATH BGP prefix attribute [32]. When interworking between IP-VPN and EVPN and passing prefixes between the two Sub-Address Families, the D-PATH attribute identifies the domain(s) through which the routes have passed. This information can then be used to prevent control plane loops, or to provide full path visibility of all the EVPN/IP-VPN interconnect gateways through which a route has passed and modify the best path selection based on it. The **domain-id “64496:11”** is configured within the VPRN **bgp-ipvpn mpls** instance, and it will be attached to EVPN IP Prefix routes advertised into Data

Centre 1. Subsequently, if a route is received by R1 or R4 with this **domain-id** it will be dropped. The **domain-id** can be attached to both EVPN IP Prefix routes and IP-VPN routes. In other words, the D-PATH value of “64496:11” represents the IP-VPN side of the interconnect gateway, and it is attached to routes that have passed through the IP-VPN domain and are being advertised into Data Centre 1 as EVPN IP Prefix routes.

Within the VPLS service the **bgp 1** context allows for manual configuration of the Route-Target values used for the service. When they auto-generated they use the system AS number together with the **evi**, however, since NVE1 and R1/R4 are in different Autonomous Systems this would result in different Route-Target values resulting in routes not being imposed. They are therefore manually configured to use the AS number of Data Centre 1. Within the **bgp-evpn** context the **evi** command is used to provision the unique EVPN instance number, and to provide uniqueness to the auto-generated Route-Distinguisher and Route-Target values used in EVPN Updates. The **bgp-evpn** context allows for selection of the data-plane tunnelling mechanism, which in this case is **vxlan** (towards the Data Centre). Before the **vxlan** option can be put into an **admin-state** of **enable**, a **vni** must be configured that will form part of the VXLAN encapsulation and serves as a demultiplexer between different VXLAN overlay networks. This is implemented using the **vxlan instance 1 vni 12** configuration line at service level and is a 24-bit identifier with values in the range 1-16777215. Within the **bgp-evpn** context the command **ip-prefix advertise true** is also configured. When configured, this command enables the following:

- a. The advertisement of IP prefixes into EVPN using IP Prefix routes (Route Type 5). Prefixes held in the attached routing instance (VPRN 11) will be advertised into EVPN within the VPLS 12 context.
- b. The installation of IP prefixes learnt through EVPN in VPLS 12 into the routing table of VPRN 11. The Next-Hop of these prefixes is set to the value of the Gateway field in the IP Prefix route, and the preference is set to 169.

Finally, the **domain-id** is “65501:12” is configured. The purpose of the **domain-id** has already been explained above, and this is the D-PATH attribute that will be attached to EVPN IP Prefix routes when they are readvertised in VPN-IPv4.

Similar configurations to that of *Output 6-52* is put in place at R3, R4, and R6.

Output 6-52: Configuration for IFF with SBD IRB model at R1

```
vprn "eleven" {
  admin-state enable
  service-id 11
  customer "1"
  bgp-ipvpn {
    mpls {
      admin-state enable
      route-distinguisher "64496:11"
      domain-id "64496:11"
      vrf-target {
        import-community "target:64496:11"
        export-community "target:64496:11"
      }
      auto-bind-tunnel {
        resolution filter
      }
    }
  }
}
```

```

        resolution-filter {
            sr-isis true
        }
    }
}
interface "evi-twelve" {
    ipv4 {
        primary {
            address 192.168.1.1
            prefix-length 24
        }
    }
    vpls "twelve" {
    }
}
vpls "twelve" {
    admin-state enable
    service-id 12
    customer "1"
    vxlan {
        instance 1 {
            vni 12
        }
    }
    routed-vpls {
    }
    bgp 1 {
        route-target {
            export "target:65501:12"
            import "target:65501:12"
        }
    }
    bgp 1 {
        route-target {
            export "target:64496:12"
            import "target:64496:12"
        }
    }
    bgp-evpn {
        evi 12
        routes {
            ip-prefix {
                advertise true
                domain-id "65501:12"
            }
        }
        vxlan 1 {
            admin-state enable
            vxlan-instance 1
        }
    }
}
}

```

Once the **bgp-evpn vxlan** context for vpls “twelve” is put into an **admin-state** of **enable** at R1, the system will generate an Inclusive Multicast Ethernet Tag route (Route Type 3) as shown in *Debug 6-11*. This is similar to the Inclusive Multicast Ethernet Tag route shown previously in this chapter (*Debug 6-1*), with the exception that VXLAN is now the tunnelling mechanism, and as a result the BGP Encapsulation extended community is set to indicate this. Also note that because VXLAN encapsulation is used, the MPLS Label field within the PMSI Tunnel attribute indicates the VNI.

Debug 6-11: Inclusive Multicast Ethernet Tag route advertised by R1 towards NVE1

```

30 2025/08/28 18:45:21.105 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.101
Peer 1: 192.0.2.101: UPDATE
Peer 1: 192.0.2.101 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 76
    Flag: 0x90 Type: 14 Len: 28 Multiprotocol Reachable NLRI:
        Address Family EVPN
        NextHop len 4 NextHop 192.0.2.1
        Type: EVPN-INCL-MCAST Len: 17 RD: 192.0.2.1:12, tag: 0, orig_addr len: 32,
orig_addr: 192.0.2.1
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 6 AS Path:
        Type: 2 Len: 1 < 64496 >
    Flag: 0xc0 Type: 16 Len: 16 Extended Community:
        target:65501:12
        bgp-tunnel-encap:VXLAN
    Flag: 0xc0 Type: 22 Len: 9 PMSI:
        Tunnel-type Ingress Replication (6)
        Flags: (0x0) [Type: None BM: 0 U: 0 Leaf: not required]
        MPLS Label 12
        Tunnel-Endpoint 192.0.2.1

```

The reception of an Inclusive Multicast Ethernet Tag route from NVE1 triggers R1 and R2 to automatically establish VXLAN bindings to it for the EVI. A VXLAN binding consists of an {egress VTEP, egress VNI} pair as shown in *Output 6-53*. In this output, the egress VNI is 12, which is inserted in the VXLAN header. The `Mcast` field is set to `BUM` if a valid Inclusive Multicast Ethernet Tag Route is received from the remote VTEP. It should also be noted that VXLAN bindings automatically inherit the split-horizon behaviour of a VPLS. That is, when traffic is received from one VXLAN tunnel, it cannot be forwarded into another VXLAN tunnel.

Output 6-53: VXLAN VTEP binding to NVE1 at R1

```

A:admin@R1# show service id 12 vxlan destinations instance 1

=====
Egress VTEP, VNI (Instance 1)
=====
VTEP Address                               Egress VNI Oper   Mcast Num
                                           State      MACs
-----
192.0.2.101                               12         Up    BUM    1
-----
Number of Egress VTEP, VNI : 1
=====

BGP EVPN-VXLAN Ethernet Segment Dest (Instance 1)
=====
Eth SegId                               Num. Macs      Last Update
-----
No Matching Entries
=====

```

Together with the Inclusive Multicast Ethernet Tag route, router R1 will also generate MAC Advertisement Routes for known MACs. With the **ip-prefix advertisement true** command enabled, IP Prefix routes for prefixes held in the attached routing instance (VPRN 10) will also be generated. Similarly, NVE1 will generate MAC Advertisement Routes for known MACs, and

will also advertise IP Prefix routes for any of its known subnets. In this example, NVE1 advertises an IP Prefix route for prefix 172.16.1.0/24 as shown in *Debug 6-12*. The BGP Encapsulation extended community is set to VXLAN, and the Route-Target extended community uses the appropriate values for the service. The Gateway field (`gw_ip`) is set to 192.168.1.2, which is the IP address of the SBD R-VPLS (IRB) interface at NVE1. From the perspective of R1 and R4 receiving this IP Prefix route, this represents the Next-Hop. As the BGP Encapsulation extended community indicates VXLAN, the Label field in the NLRI reflects the VXLAN VNI. Finally, the ESI and Ethernet Tag fields are both set to 0.

Debug 6-12: IP Prefix route advertised by NVE1

```
86 2025/08/29 08:26:26.485 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.101
Peer 1: 192.0.2.101: UPDATE
Peer 1: 192.0.2.101 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 81
    Flag: 0x90 Type: 14 Len: 45 Multiprotocol Reachable NLRI:
        Address Family EVPN
        NextHop len 4 NextHop 192.0.2.101
        Type: EVPN-IP-PREFIX Len: 34 RD: 192.0.2.101:12, ESI: ESI-0, tag: 0, ip_prefix:
172.16.1.0/24 gw_ip 192.168.1.2 Label: 12 (Raw Label: 0xc)
        Flag: 0x40 Type: 1 Len: 1 Origin: 0
        Flag: 0x40 Type: 2 Len: 6 AS Path:
            Type: 2 Len: 1 < 65501 >
        Flag: 0xc0 Type: 16 Len: 16 Extended Community:
            target:65501:12
            bgp-tunnel-encap:VXLAN
```

Immediately following the IP Prefix route, *Debug 6-13* shows a MAC Advertisement route sourced by NVE1 containing the MAC address for the Gateway IP address (`gw_ip`) previously advertised in the IP Prefix route (*Debug 6-12*). This MAC Advertisement route will populate the ARP cache of the R-VPLS instance at R1 and R4, thereby negating the requirement for those PE routers to ARP for the advertised Gateway IP address.

Debug 6-13: MAC Advertisement route advertised by NVE1

```
85 2025/08/29 08:26:26.485 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.101
Peer 1: 192.0.2.101: UPDATE
Peer 1: 192.0.2.101 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 92
    Flag: 0x90 Type: 14 Len: 48 Multiprotocol Reachable NLRI:
        Address Family EVPN
        NextHop len 4 NextHop 192.0.2.101
        Type: EVPN-MAC Len: 37 RD: 192.0.2.101:12 ESI: ESI-0, tag: 0, mac len: 48 mac:
00:29:ff:ff:ff:49, IP len: 4, IP: 192.168.1.2, label1: 12 (Raw Label: 0xc)
        Flag: 0x40 Type: 1 Len: 1 Origin: 0
        Flag: 0x40 Type: 2 Len: 6 AS Path:
            Type: 2 Len: 1 < 65501 >
        Flag: 0xc0 Type: 16 Len: 24 Extended Community:
            target:65501:12
            bgp-tunnel-encap:VXLAN
            mac-mobility:Seq:0/Static
```

This process of advertising a Gateway IP for a prefix and then following it with a MAC Advertisement route for that Gateway IP and MAC allows for the Gateway IP to be recursively resolved and is an example of Overlay Indexing. This is shown in R1's ARP cache for VPRN 11 shown in *Output 6-54*, where the MAC address 00:29:ff:ff:ff:49 advertised by NVE1 above is

indicated as being learnt through EVPN (Evp). Also note that since the MAC address was learnt through BGP, it has an infinite expiry time. If the MAC address should ever be removed, a Withdraw (MP_UNREACH_NLRI) message will be sent by the advertising PE causing the ARP entry to be removed.

Output 6-54: ARP Cache for VPRN 11 at R1

```
A:admin@R1# show router 11 arp
```

```
=====
ARP Table (Service: 11)
=====
IP Address      MAC Address      Expiry    Type    Interface
-----
192.168.1.1     00:01:ff:ff:ff:49 00h00m00s Oth[I]   evi-twelve
192.168.1.2     00:29:ff:ff:ff:49 00h00m00s Evp[I]   evi-twelve
-----
No. of ARP Entries: 2
=====
```

Once the EVPN and VPN-IPv4 routes have been exchanged the contents of the VPRN routing-table at R1 can be validated as shown in *Output 6-55*. There are four entries in the route-table which have been populated as follows:

- Prefix 172.16.1.0/24 is VA1's subnet in Data Centre 1. It is known through BGP EVPN Interface-ful (EVPN-IFF) and has a Next-Hop of 192.168.1.2, which is the address of the SBD IRB ad NVE1. As it is known through BGP EVPN, it is installed with a route-preference of 169. R1 will re-advertise this route in VPN-IPv4 (subject to route policy) using the Route-Target value configured in VPRN 11.
- Prefix 172.16.2.0/24 is VA2's subnet in Data Centre 2, whilst prefix 192.168.2.0/24 is the R-VPLS interface linking VPRN 11 with VPLS 12 (EVI 12) at R3 and R6. Both entries are known through BGP VPN and have a Next-Hop of 192.0.2.3 (R3), which is resolved to SR-ISIS (tunneled:sr-isis:524346). As both entries are known through BGP VPN, they are installed with a route-preference of 170. R1 will re-advertise this route in EVPN (subject to route-policy) using the Route-Target value configured in VPLS 12.
- Prefix 192.168.1.0/24 is the R-VPLS interface linking VPRN 11 with VPLS 12 (EVI 12) at R1 and R4, and as such is a 'Local' route. R1 will re-advertise this route in BGP VPN (subject to route-policy) with the Route-Target value configured in VPRN 11.

Output 6-55: VPRN 10 route-table at R1

```
A:admin@R1# show router 11 route-table
```

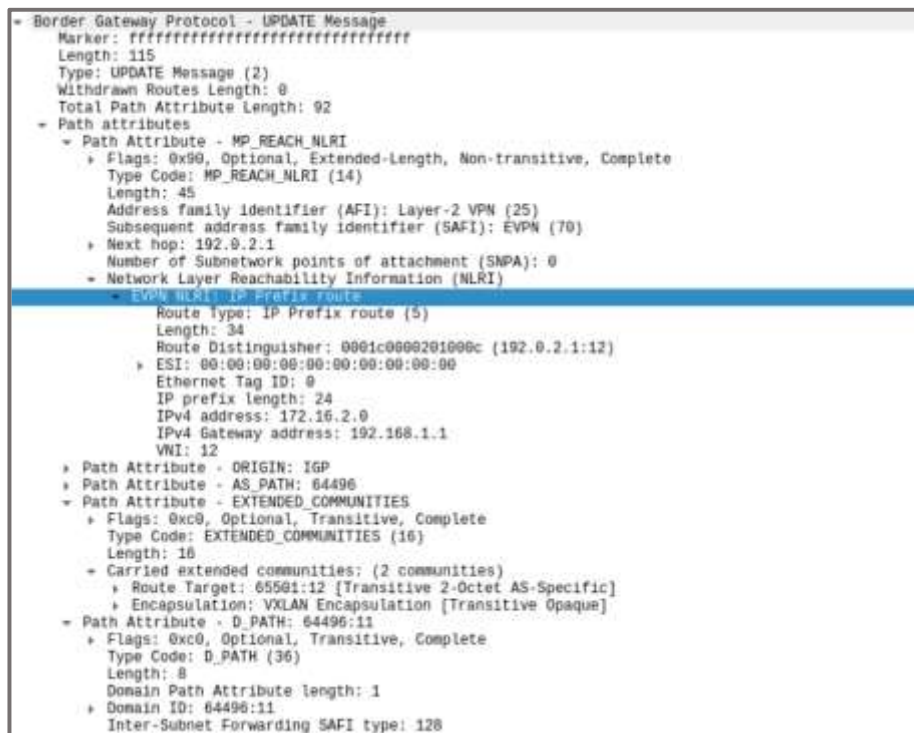
```
=====
Route Table (Service: 11)
=====
Dest Prefix[Flags]      Type    Proto    Age          Pref
  Next Hop[Interface Name]      Metric
-----
172.16.1.0/24           Remote  EVPN-IFF  00h48m08s    169
    192.168.1.2                0
172.16.2.0/24           Remote  BGP VPN   01h10m45s    170
    192.0.2.3 (tunneled:SR-ISIS:524346) 400
=====
```

192.168.1.0/24	Local	Local	20h04m02s	0
evi-twelve			0	
192.168.2.0/24	Remote	BGP VPN	14h58m15s	170
192.0.2.3 (tunneled:SR-ISIS:524346)			400	

No. of Routes: 4				

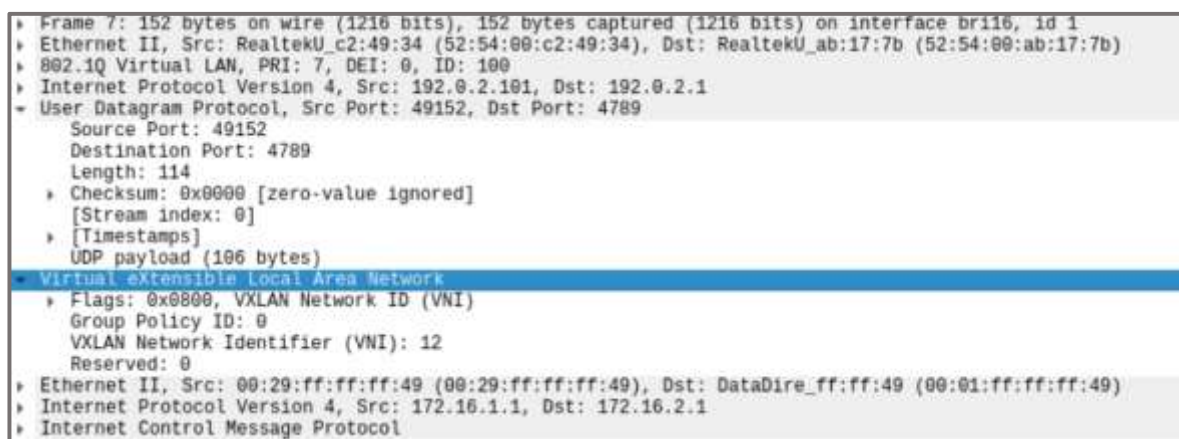
Figure 6-26 shows a packet capture of the VA2 prefix 172.16.2.0/24 learned by R1 in VPN-IPv4 as it is interworked and readvertised as an EVPN IP Prefix route. The NLRI contains the RD, an ESI and Ethernet Tag field both set to zero, and the prefix 172.16.2.0/24 together with its Gateway address of 192.168.1.1, which is R1's SBD IRB address. There are two Extended Communities, Route-Target and BGP Encapsulation, and as the latter indicates VXLAN encapsulation, the MPLS label field of the NLRI has the VNI of 12. The D-PATH attribute is also present. This contains the value of 64496:11 to indicate that the prefix came from the IP-VPN domain.

Figure 6-26: Packet capture of IP Prefix route advertised by R1



With all the routes and FDB entries populated, a successful ping is executed from VA1 (172.16.1.1) to VA2 (172.16.2.1). A packet capture of the ping is shown in Figure 6-27, taken on the link between NVE1 and R1. The ICMP packet is encapsulated in VXLAN (IP/UDP) and has a VNI of 12. The source address of the encapsulating header is NVE1 (192.0.2.101) and the destination is R1 (192.0.2.1). When the packet arrives at R1 the VXLAN header will be removed and replaced with a label stack before forwarding across the BGP/MPLS IP-VPN core.

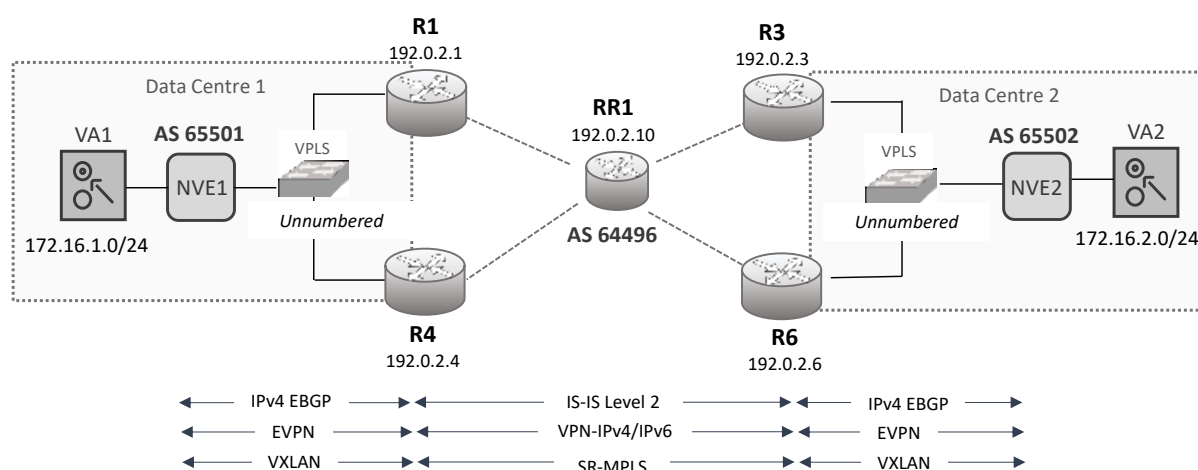
Figure 6-27: Packet capture of ping between VA1 and VA2



Interface-ful with Unnumbered SBD IRB model

Interface-ful refers to the fact that there exists a full IRB interface between the VPRN and the SBD, but in this instance the interface is unnumbered (i.e. no IP address is assigned to it). Figure 6-28 shows the topology that will be used to demonstrate this model, and as can be seen it is very similar to the previous model with the exception that the overlay VPLS between the NVE and PEs/data centre gateways is unnumbered.

Figure 6-28: Test Topology for Interface-ful with Unnumbered SBD IRB model



The Interface-ful IP-VRF to IP-VRF with unnumbered SBD model is implemented in SR-OS using a VPRN logically linked to an R-VPLS, but as the SBD IRB is unnumbered, this logical link uses the concept of an EVPN tunnel. The following output shows the configuration on R1, with similar configuration applied at the other PE routers. The configuration of VPLS instance “**vpls eleven**” has not changed from the previous Interface-ful with SBD IRB model and is therefore not explained again here. However, the IP interface “**evi-twelve**” within “**vprn eleven**” which forms the unnumbered SBD has changed. There is no IP address configured

(hence it is unnumbered) and the **evpn-tunnel** context indicates **supplementary-broadcast-domain** is **true**. Hence there is an unnumbered SBD.

Output 6-56: Configuration for IFF with unnumbered SBD IRB model at R1

```
vpls "twelve" {
    admin-state enable
    service-id 12
    customer "1"
    vxlan {
        instance 1 {
            vni 12
        }
    }
    routed-vpls {
    }
    bgp 1 {
        route-target {
            export "target:65501:12"
            import "target:65501:12"
        }
    }
    bgp-evpn {
        evi 12
        routes {
            ip-prefix {
                advertise true
                domain-id "65501:12"
            }
        }
        vxlan 1 {
            admin-state enable
            vxlan-instance 1
        }
    }
}
vprn "eleven" {
    admin-state enable
    service-id 11
    customer "1"
    bgp-ipvpn {
        mpls {
            admin-state enable
            route-distinguisher "64496:11"
            domain-id "64496:11"
            vrf-target {
                import-community "target:64496:11"
                export-community "target:64496:11"
            }
            auto-bind-tunnel {
                resolution filter
                resolution-filter {
                    sr-isis true
                }
            }
        }
    }
}
interface "evi-twelve" {
    vpls "twelve" {
        evpn-tunnel {
            supplementary-broadcast-domain true
        }
    }
}
}
```


Using the Interface-ful with unnumbered SBD IRB model, *Debug 6-14* shows the EVPN IP Prefix route advertised by NVE1 towards R1. Note that the prefix 172.16.1.0/24 is no longer associated with a Gateway Address (gw_ip) as it is encoded as 0.0.0.0, and a MAC address is carried within the EVPN Router's MAC extended community (mac-nh).

Debug 6-14: EVPN IP Prefix route advertised by NVE1

```
261 2025/08/29 15:28:34.485 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.101
Peer 1: 192.0.2.101: UPDATE
Peer 1: 192.0.2.101 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 89
  Flag: 0x90 Type: 14 Len: 45 Multiprotocol Reachable NLRI:
    Address Family EVPN
    NextHop len 4 NextHop 192.0.2.101
    Type: EVPN-IP-PREFIX Len: 34 RD: 192.0.2.101:12, ESI: ESI-0, tag: 0, ip_prefix:
172.16.1.0/24 gw_ip 0.0.0.0 Label: 12 (Raw Label: 0xc)
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 6 AS Path:
      Type: 2 Len: 1 < 65501 >
    Flag: 0xc0 Type: 16 Len: 24 Extended Community:
      target:65501:12
      mac-nh:00:29:ff:ff:ff:49
      bgp-tunnel-encap:VXLAN
```

Figure 6-29 shows a packet capture of the same EVPN IP Prefix route with the IPv4 Gateway address encoded as 0.0.0.0 and the EVPN Router MAC encoding the MAC address to be used for recursive resolution.

Figure 6-29: Packet capture of IP Prefix route from NVE1

```
- Border Gateway Protocol - UPDATE Message
  Marker: ffffffffffffffffffffffffffffffff
  Length: 112
  Type: UPDATE Message (2)
  Withdrawn Routes Length: 0
  Total Path Attribute Length: 89
  - Path attributes
    - Path Attribute - MP_REACH_NLRI
      - Flags: 0x90, Optional, Extended-Length, Non-transitive, Complete
      - Type Code: MP_REACH_NLRI (14)
      - Length: 45
      - Address family identifier (AFI): Layer-2 VPN (25)
      - Subsequent address family identifier (SAFI): EVPN (70)
      - Next hop: 192.0.2.101
      - Number of Subnetwork points of attachment (SNPA): 0
    - Network layer Reachability Information (NLRI)
      - EVPN NLRI: IP Prefix route
        - Route type: IP Prefix route (5)
        - Length: 34
        - Route Distinguisher: 0001c0000265000c (192.0.2.101:12)
        - ESI: 00:00:00:00:00:00:00:00
        - Ethernet Tag ID: 0
        - IP prefix length: 24
        - IPv4 address: 172.16.1.0
        - IPv4 Gateway address: 0.0.0.0
        - VNI: 12
      - Path Attribute - ORIGIN: IGP
      - Path Attribute - AS_PATH: 65501
    - Path Attribute - EXTENDED_COMMUNITIES
      - Flags: 0xc0, Optional, Transitive, Complete
      - Type Code: EXTENDED_COMMUNITIES (16)
      - Length: 24
      - Carried extended communities: (3 communities)
        - Route Target: 65501:12 [Transitive 2-Octet AS-Specific]
        - EVPN Router MAC: Router MAC: 00:29:ff:ff:ff:49 [Transitive EVPN]
          - Type: Transitive EVPN (0x06)
          - Subtype (EVPN): EVPN Router MAC (0x03)
          - Router MAC: 00:29:ff:ff:ff:49 (00:29:ff:ff:ff:49)
        - Encapsulation: VXLAN Encapsulation [Transitive Opaque]
```

The IP Prefix route from NVE1 is accompanied by a MAC Advertisement route containing the MAC Address previously encoded in the EVPN Router's MAC of the IP Prefix route. This MAC

Advertisement route allows for recursive resolution of the IP Prefix route and is another example of Overlay Indexing.

Debug 6-15: MAC Advertisement route advertised by NVE1

```
260 2025/08/29 15:28:34.484 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.101
Peer 1: 192.0.2.101: UPDATE
Peer 1: 192.0.2.101 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 88
  Flag: 0x90 Type: 14 Len: 44 Multiprotocol Reachable NLRI:
    Address Family EVPN
    NextHop len 4 NextHop 192.0.2.101
    Type: EVPN-MAC Len: 33 RD: 192.0.2.101:12 ESI: ESI-0, tag: 0, mac len: 48 mac:
00:29:ff:ff:ff:49, IP len: 0, IP: NULL, label: 12 (Raw Label: 0xc)
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 6 AS Path:
    Type: 2 Len: 1 < 65501 >
  Flag: 0xc0 Type: 16 Len: 24 Extended Community:
    target:65501:12
    bgp-tunnel-encap:VXLAN
    mac-mobility:Seq:0/Static
```

The advertised MAC within the MAC Advertisement route is installed in the FDB table for the SBD VPLS instance, but as no adjacent IP Next Hop exists does not populate the VPRN ARP cache.

Output 6-57: VPLS 12 FDB table at R1

```
A:admin@R1# show service id 12 fdb detail

=====
Forwarding Database, Service 12
=====
```

ServId	MAC	Source-Identifier	Type	Last Change
	Transport:Tnl-Id		Age	
12	00:01:ff:ff:ff:49	cpm	Intf	08/28/25 09:08:58
12	00:04:ff:ff:ff:49	vxlan-1:	EvpnS:P	08/29/25 15:15:16
		192.0.2.4:12		
12	00:29:ff:ff:ff:49	vxlan-1:	EvpnS:P	08/29/25 15:28:34
		192.0.2.101:12		

```
-----
No. of MAC Entries: 3
```

When the IP Prefix route for prefix 172.16.1.0/24 is received at R1 it creates a route-table entry in the VPRN with a preference of 169, which is one less than the conventional 170 preference for BGP-VPN routes. There are only two route entries compared to the four with the numbered IRB SBD model (*Output 6-55*), simply because the IRB SBD is unnumbered. The prefix 172.16.1.0/24 from NVE1 has a Next Hop of `evi-twelve` (ET-00:29:ff:ff:ff:49) to indicate that forwarding interface and that the Next Hop was learned through an EVPN Router's MAC extended community as opposed to a Gateway IP address and associated MAC Advertisement route.

Output 6-58: VPRN 11 route-table at R1 with IFF and SBD IRB unnumbered

```
A:admin@R1# show router 11 route-table

=====
Route Table (Service: 11)
```

Dest Prefix[Flags] Next Hop[Interface Name]	Type	Proto	Age Metric	Pref
172.16.1.0/24 evi-twelve (ET-00:29:ff:ff:ff:49)	Remote	EVPN-IFF	00h13m26s 0	169
172.16.2.0/24 192.0.2.3 (tunneled:SR-ISIS:524346)	Remote	BGP VPN	02h55m28s 400	170

No. of Routes: 2

Flags: n = Number of times nexthop is repeated
 B = BGP backup route available
 L = LFA nexthop available
 S = Sticky ECMP requested

=====

With the control plane in place, it only remains to test the data plane. *Figure 6-30* shows a packet capture of a successful ping from VA1 to VA2 taken on the interface between NVE1 and R1. The Echo Request is encapsulated in VXLAN with an IP source of NVE1 (192.0.2.101) and a destination of R1 (192.0.2.1). When the packet reaches R1 a lookup will be carried out in the VPRN route-table, after which it will impose the relevant MPLS label stack and forward to R3. Note that within the encapsulated packet the source MAC address (00:29:ff:ff:ff:49) is the MAC address advertised in the EVPN Router's MAC extended community by NVE1. (The destination MAC address in the encapsulated packet will be the MAC address carried in an EVPN Router's MAC extended community by R1.)

Figure 6-30: Packet capture of ping from VA1 to VA2

```

Frame 29: 152 bytes on wire (1216 bits), 152 bytes captured (1216 bits) on interface br116, id 0
Ethernet II, Src: RealtekU_c2:49:34 (52:54:00:c2:49:34), Dst: RealtekU_ab:17:7b (52:54:00:ab:17:7b)
802.1Q Virtual LAN, PRI: 7, DEI: 0, ID: 100
Internet Protocol Version 4, Src: 192.0.2.101, Dst: 192.0.2.1
User Datagram Protocol, Src Port: 49152, Dst Port: 4789
  Source Port: 49152
  Destination Port: 4789
  Length: 114
  Checksum: 0x0000 [zero-value ignored]
  [Stream index: 0]
  [Timestamps]
  UDP payload (106 bytes)
    Virtual extensible Local Area Network
      Flags: 0x0800, VXLAN Network ID (VNI)
      Group Policy ID: 0
      VXLAN Network Identifier (VNI): 12
      Reserved: 0
      Ethernet II, Src: 00:29:ff:ff:ff:49 (00:29:ff:ff:ff:49), Dst: DataDire_ff:ff:49 (00:01:ff:ff:ff:49)
      Internet Protocol Version 4, Src: 172.16.1.1, Dst: 172.16.2.1
      Internet Control Message Protocol

```

Interface-less IP-VRF-to-IP-VRF model

In an interface-less IP-VRF-to-IP-VRF model no SBD is required and there is no requirement for recursive resolution when receiving an IP prefix route. Instead, the Next-Hop of the IP Prefix route is directly resolved to an EVPN tunnel without the need for any other route. Two variants of this model are supported:

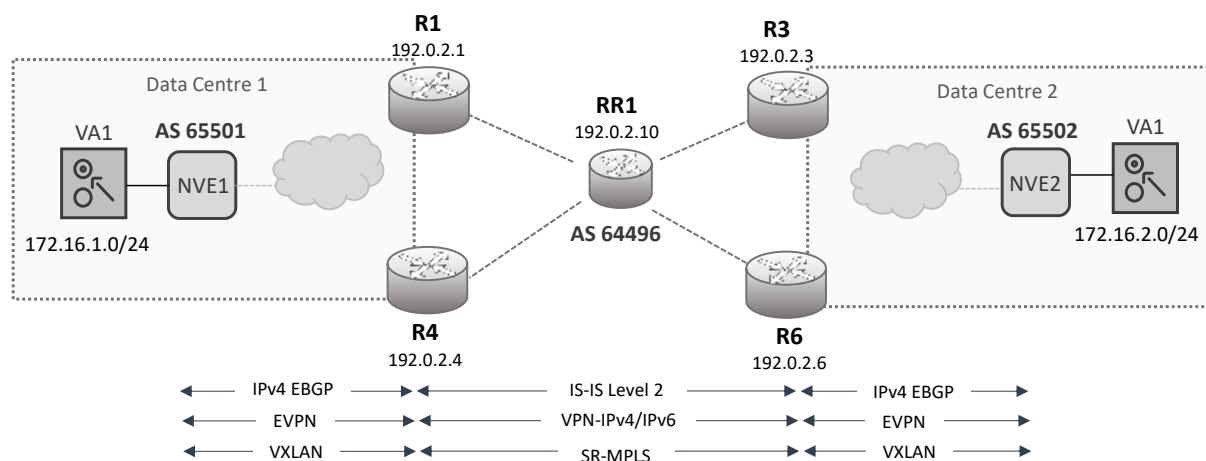
- a) **Interface-less EVPN for Ethernet:** This sub-model carries packets through the EVPN in an Ethernet payload. It is identical to the Interface-ful unnumbered SBD model in the data-path but does not require a recursive resolution to a MAC Advertisement (Type-2) route. Instead, it uses the EVPN Router's MAC extended community

advertised with the IP Prefix route as the destination address in the encapsulated Ethernet payload. It is enabled using the service **vpn interface vpls evpn-tunnel** and **bgp-evpn ip-prefix advertisement true** commands. Additionally, because no MAC Advertisement route is needed, **bgp-evpn mac-advertisement false** may be configured to prevent the advertisement of the MAC route.

- b) **Interface-less EVPN for IP:** This sub-model carries packets through the EVPN as pure IP in the same way that IP-VPN does. The interface-less EVPN for IP sub-model does not encapsulate packets in Ethernet, but rather the IP packet is directly encapsulated with the EVPN service label and transport label. The data-path is identical to that used for IP-VPN services. It is implemented using a **bgp-evpn mpls** context within a VPRN service.

Figure 6-31 is used to illustrate the use of the Interface-Less IP-VRF-to-IP-VRF model. It is the same topology used to demonstrate the Interface-ful with SBD IRB and Interface-ful with unnumbered SBD IRB models, but there is no longer an interface between the Data Centre Gateways and their connected NVEs. To be more precise, there are still physical links running EBGp to advertise the underlay topology, but there is no overlay interface present. The Data Centres continue to use BGP EVPN as a control plane and VXLAN for the data-plane, whilst the core continues to use BGP/MPLS IP-VPN. As in the previous demonstrations, the objective is to facilitate inter-subnet connectivity between VA subnet 172.16.1.0/24 in Data Centre 1 and VA subnet 172.16.2.0/24 in Data Centre 2

Figure 6-31: Test Topology for Interface-Less IP-VRF-to-IP-VRF model



Output 6-59 shows the configuration for the IP sub-model of the Interface-less IP-VRF-to-IP-VRF model at R1, with similar configuration applied at the other Data Centre Gateways. The most notable change from the Interface-ful models previously described in this chapter is the absence of a VPLS instance. The entire configuration is contained in a VPRN instance that contains both a **bgp-ipvpn** context and a **bgp-evpn** context. The **bgp-ipvpn** context contains the parameters intended for use within the MPLS core. Similarly, the **bgp-evpn** context contains the parameters intended for use within the connected Data Centre. Note that **vxlان** cannot be administratively enabled within the **bgp-evpn** context until the **vxlان**

instance has been allocated a **vni** at service level. It is worth pointing out that when interworking as a gateway in this manner, the use of different Route-Distinguishers in each of the **evpn/ipvpn** instances is recommended. The use of different Route-Targets is intended to allow for control of route distribution between the IP-VPN core and the connected Data Centre.

Output 6-59: Configuration for Interface-Less IP-VRF-to-IP-VRF model at R1

```

vprn "eleven" {
  admin-state enable
  service-id 11
  customer "1"
  bgp-evpn {
    vxlan 1 {
      admin-state enable
      route-distinguisher "65501:12"
      domain-id "65501:12"
      vxlan-instance 1
      vrf-target {
        import-community "target:65501:12"
        export-community "target:65501:12"
      }
    }
  }
  bgp-ipvpn {
    mpls {
      admin-state enable
      route-distinguisher "64496:11"
      domain-id "64496:11"
      vrf-target {
        import-community "target:64496:11"
        export-community "target:64496:11"
      }
      auto-bind-tunnel {
        resolution filter
        resolution-filter {
          sr-isis true
        }
      }
    }
  }
  vxlan {
    instance 1 {
      vni 12
    }
  }
}

```

With the above configuration in place *Debug 6-16* shows the IP Prefix route advertised by NVE1 for the VA subnet 172.16.1.0/24. The NLRI contains the RD, zero-value ESI and Ethernet Tag fields, the IP Prefix 172.16.1.0/24 and a Gateway address of 0.0.0.0. The MPLS label field is also set to zero because the encapsulation defined by the BGP Encapsulation extended community is VXLAN. The route also carries the EVPN Router's MAC extended community which provides the MAC address that should be used as the inner destination address for packets sent to the prefix.

Debug 6-16: IP Prefix Route advertised by NVE1

```

Peer 1: 192.0.2.101: UPDATE
Peer 1: 192.0.2.101 - Received BGP UPDATE:

```

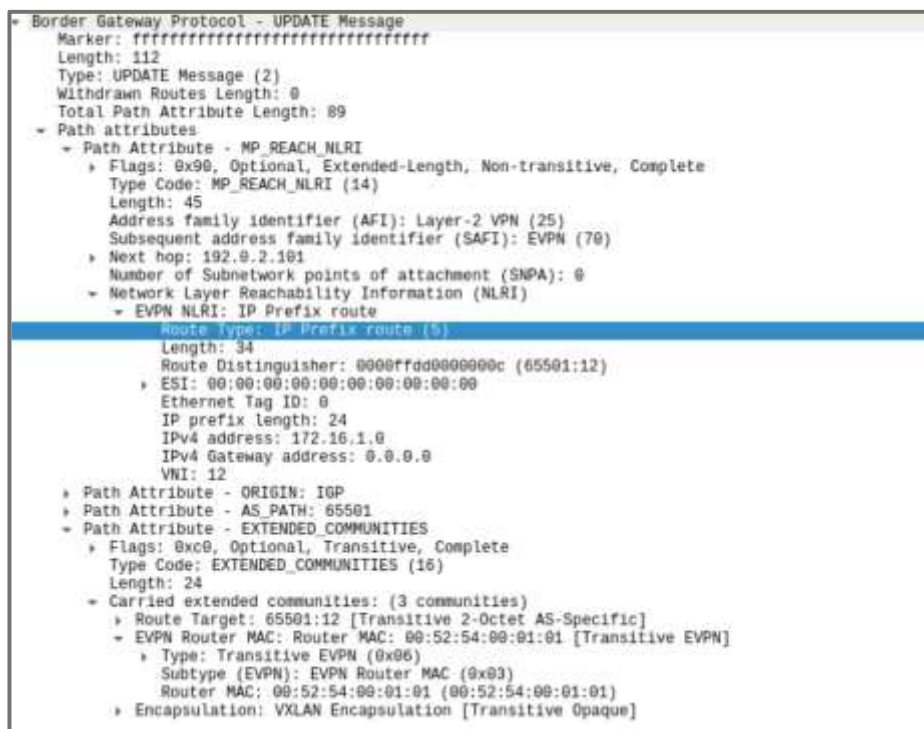
```

Withdrawn Length = 0
Total Path Attr Length = 89
Flag: 0x90 Type: 14 Len: 45 Multiprotocol Reachable NLRI:
  Address Family EVPN
  NextHop len 4 NextHop 192.0.2.101
  Type: EVPN-IP-PREFIX Len: 34 RD: 65501:12, ESI: ESI-0, tag: 0, ip_prefix:
172.16.1.0/24 gw_ip 0.0.0.0 Label: 12 (Raw Label: 0xc)
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 6 AS Path:
  Type: 2 Len: 1 < 65501 >
  Flag: 0xc0 Type: 16 Len: 24 Extended Community:
  target:65501:12
  mac-nh:00:52:54:00:01:01
  bgp-tunnel-encap:VXLAN

```

Figure 6-32 shows a packet capture of the same IP Prefix route advertise by NVE1. Note that Wireshark is able to represent the MPLS Label field in the NLRI as VNI because the BGP Encapsulation extended community specifies VXLAN encapsulation.

Figure 6-32: Packet capture of IP Prefix route advertised by NVE1



The route-table at R1 shows two entries as expected. The first entry is for the VA1 prefix 172.16.1.0/24 and has a Next-Hop of 192.0.2.101 (NVE1), which is the VTEP endpoint for the VXLAN tunnel with a VNI of 12 (tunneled:VXLAN:12). It is learnt through protocol EVPN-IFL and has a preference of 170. The second entry is for the VA2 prefix 172.16.2.0/24 with a Next-Hop of 192.0.2.3 (R3), which is the endpoint of the SR-ISIS LSP with the tunnel-table identifier of 524346 (tunneled:SR-ISIS:524346). It is learned through protocol BGP VPN and also has a preference of 170.

Output 6-60: Route-Table at R1

```
A:admin@R1# show router 11 route-table
```

```

=====
Route Table (Service: 11)
=====
Dest Prefix[Flags]                                Type    Proto    Age          Pref
  Next Hop[Interface Name]                        Metric
-----
172.16.1.0/24                                     Remote  EVPN-IFL  00h07m12s    170
      192.0.2.101 (tunneled:VXLAN:12)              0
172.16.2.0/24                                     Remote  BGP VPN    15h30m06s    170
      192.0.2.3 (tunneled:SR-ISIS:524346)           400
-----
No. of Routes: 2

```

A successful ping is carried out from VA1 to VA2 and is captured on the wire between NVE1 and R1. The outer encapsulation is VXLAN which uses a VNI of 12 as a demultiplexer value. The inner packet is Ethernet and IP, where the source and destination MAC addresses of the Ethernet header are populated with the contents of the EVPN Router's MAC extended community carried with the IP Prefix routes advertised by NVE1 and R1 respectively.

Figure 6-33: Capture of packet from VA1 to VA2

```

Frame 17: 152 bytes on wire (1216 bits), 152 bytes captured (1216 bits) on interface br116, id 0
Ethernet II, Src: RealtekU_c2:49:34 (52:54:00:c2:49:34), Dst: RealtekU_ab:17:7b (52:54:00:ab:17:7b)
802.1Q Virtual LAN, PRI: 7, DEI: 0, ID: 100
Internet Protocol Version 4, Src: 192.0.2.101, Dst: 192.0.2.1
User Datagram Protocol, Src Port: 49152, Dst Port: 4789
Virtual extensible Local Area Network
  Flags: 0x0800, VXLAN Network ID (VNI)
  Group Policy ID: 0
  VXLAN Network Identifier (VNI): 12
  Reserved: 0
Ethernet II, Src: 00:52:54:00:01:01 (00:52:54:00:01:01), Dst: 00:54:52:00:00:01 (00:54:52:00:00:01)
  Destination: 00:54:52:00:00:01 (00:54:52:00:00:01)
  Source: 00:52:54:00:01:01 (00:52:54:00:01:01)
  Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 172.16.1.1, Dst: 172.16.2.1
Internet Control Message Protocol

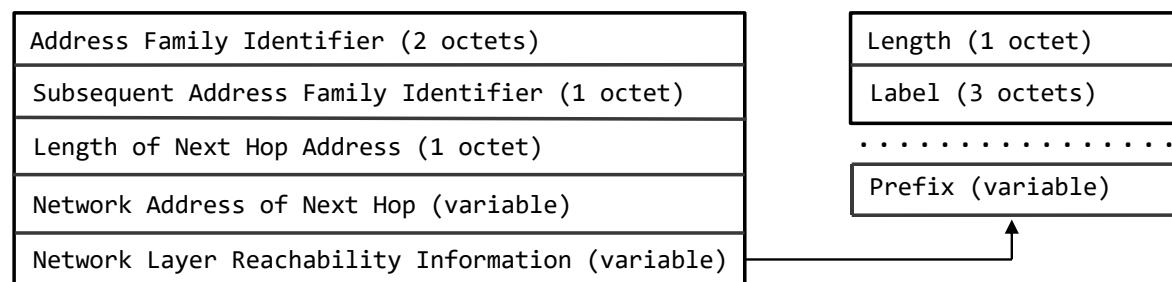
```

The reasonably obvious advantage of using the Interface-less model is that it does not require SBD/backhaul R-VPLS configuration. It also scales better than the Interface-ful models because of the reduced number of services and MAC/ARP entries on the SBD. Be aware however that the lack of recursive resolution (IP Prefix route to MAC/IP route) means that convergence may increase with scale.

7 Labeled Unicast IPv4

When an Update message carries information about a particular prefix, it is also possible to ‘piggy-back’ MPLS label mapping information for that prefix. The ability to carry label information in BGP [9] builds on the Multi-Protocol BGP extension MP_REACH_NLRI attribute using AFI 1 (IPv4) with SAFI 4 (NLRI with MPLS labels) to indicate the presence of a label. The NLRI of the MP_REACH_NLRI attribute is encoded as one or more triples in the form <length, label, prefix>.

Figure 7-1: Labeled BGP NLRI Encoding



The label field of the NLRI carries one or more labels with each label being encoded as three octets, where the high-order 20-bits contain the label value, and the low-order bit contains the Bottom of Stack indicator.

When using BGP to advertise labels that will subsequently be used in the data-plane for MPLS forwarding, it is important to make a distinction between adjacent and non-adjacent peers. For example, assume that routers R1 and R2 are directly adjacent and that router R2 advertises label L for prefix P to router R1. Router R1 can thereafter push label L onto an MPLS packet forwarded to R2 without issue. However, now consider the case where there are four routers in the topology R1—R2—R3—R4. Routers R4 and R1 are BGP peers and router R4 advertises label L for prefix P to router R1. When R1 subsequently imposes label L onto a packet or label stack, it cannot forward it as top of stack. Rather, it must push on another label so that router R2 sees a label at the top of the stack that it understands and that R2 advertised. Similarly, R2 must swap the top label so that R3 sees a label at the top of the stack that it advertised. Router R4 can be the only router that processes label L. Although this seems fairly obvious, it is worth remembering. The following sub-sections will discuss some use-cases for using BGP to advertise MPLS labels and the adjacent peer versus non-adjacent peer scenarios are evident.

The output shown in *Debug 7-1* gives an example of a MP_REACH_NLRI for a labeled BGP Address Family with prefix 192.0.2.1/32 and label 524267 using the encoding shown above. A BGP speaker can withdraw a previously advertised prefix and label simply by listing the NLRI of a previously advertised route in the withdrawn routes field of an MP_UNREACH_NLRI attribute. In this case, the label information carried as part of the NLRI should be set to 0.

Debug 7-1: Labeled IPv4 Unicast prefix

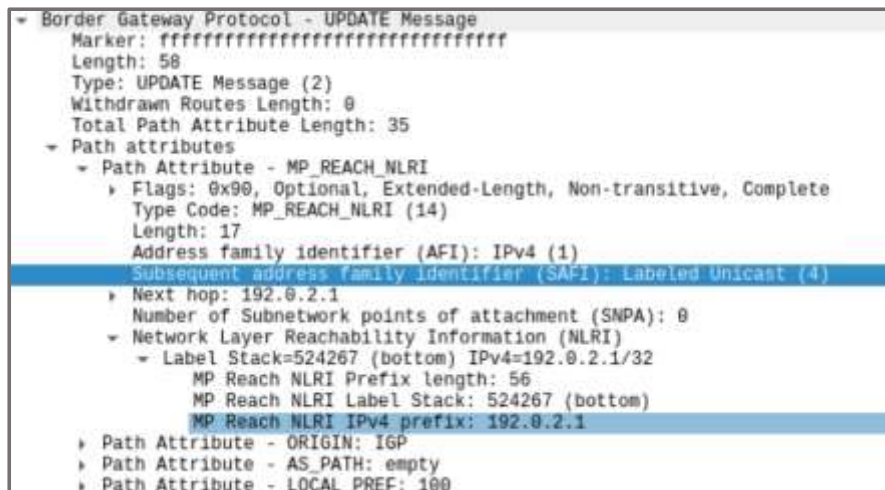
```

3 2025/09/03 08:10:06.077 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.101
Peer 1: 192.0.2.101: UPDATE
Peer 1: 192.0.2.101 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 35
    Flag: 0x90 Type: 14 Len: 17 Multiprotocol Reachable NLRI:
        Address Family LBL-IPV4
        NextHop len 4 NextHop 192.0.2.1
        192.0.2.1/32 Label 524267
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100

```

Figure 7-2 shows a packet capture of the same Update message with the MP_REACH_NLRI attribute encoding AFI 1 (IPv4) and SAFI 4 (Labeled Unicast). The NLRI field is expanded to show the prefix (192.0.2.1) together with the advertised label value of 524267 (with Bottom of Stack indicator set).

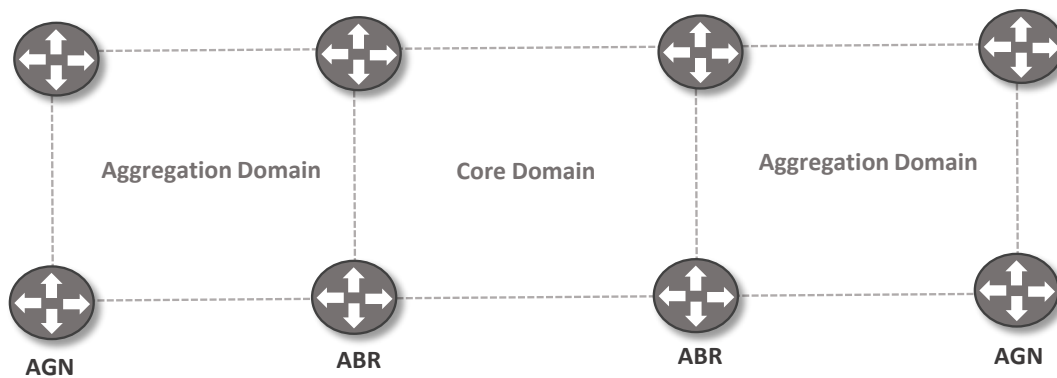
Figure 7-2: Packet capture of Labeled BGP Update



Seamless MPLS

Seamless MPLS [33] is a framework that can be used to extend MPLS networks across aggregation and access domains such that the entire network can be considered as a single MPLS domain spanning potentially hundreds of thousands of nodes. Although the legitimacy of [33] has long since expired, there are many networks that utilise the approach defined by this document. It does not define any new protocols or technologies but rather relies on existing ones, whilst accounting for the limited feature-set and scale of smaller aggregation nodes (AGNs) and access nodes (ANs). The architecture uses a ‘divide and conquer’ approach, where a large problem is divided into many smaller problems. It builds upon ‘typical’ Service Provider topologies and hierarchy is a key element. Aggregation domains are never connected to each other, only to the core, whilst the gateway between the core and aggregation layers is implemented by an Area Border Router (ABR).

Figure 7-3: Seamless MPLS Architecture



The objective of Seamless MPLS is to tackle a number of key issues including scaling to hundreds of thousands of nodes, redundancy, sub-second re-convergence, and providing better network manageability through endpoint-only provisioning across MPLS domains. A key component to all of these is that the transport layer is completely de-coupled from the service layer.

Transport Layer

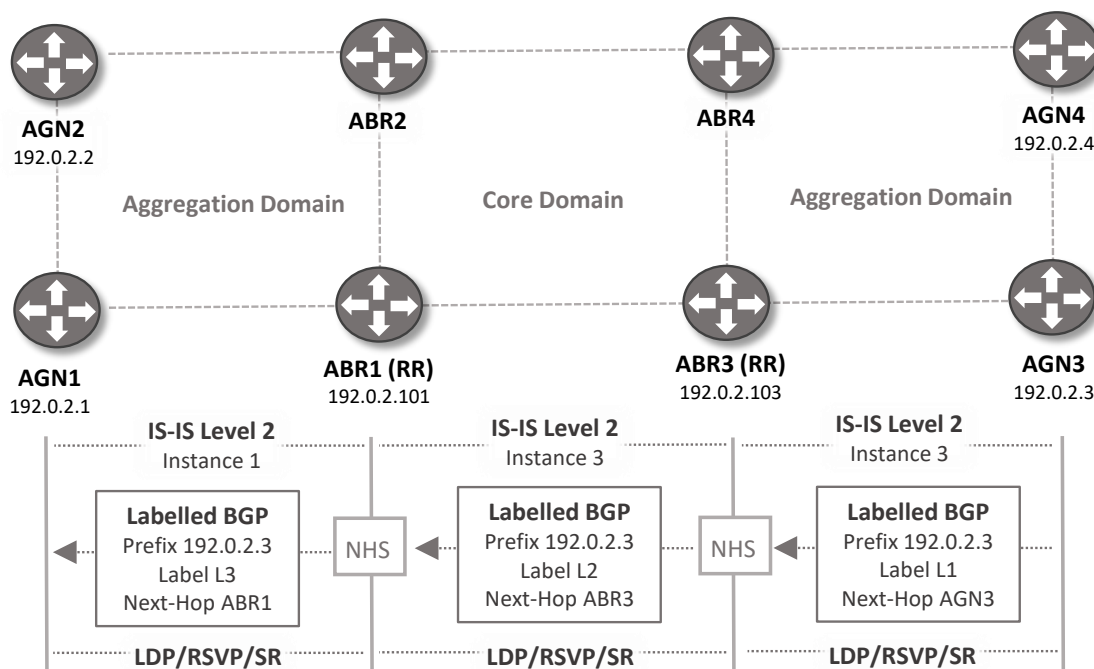
Intra-domain routing can be implemented using OSPF or IS-IS, although for the purpose of this overview only IS-IS is discussed. Depending on the size of the network and the provider requirements, IS-IS can be either a single instance with multiple levels or multiple instances with single or multiple levels. Where multiple instances are used, the ABR typically implements instance 0 (the default IS-IS instance) towards the core domain, and one or more non-zero instances towards the aggregation domain(s). Either way, the objective is to create smaller and more manageable IGP islands.

Inter-domain routing is achieved using Labeled Unicast BGP to distribute loopback/system addresses between domains. To allow for separation of core/aggregation IGP domains or limited route-leaking between core/aggregation IGP domains, the ABR performs the role of an IBGP Route-Reflector. The notable difference for the ABR is that while advertising the labeled BGP prefix upstream it imposes Next-Hop-Self and allocates its own downstream label. This effectively puts the ABR into the data-path. The ABR can implement Next-Hop-Self in the direction of the core only, or in both directions (towards the core and towards the aggregation domain). Both approaches are valid. Implementing Next-Hop-Self towards the core only is used when a single instance of IS-IS is adopted across all domains. In this scenario the core domain represents Level-2, and the aggregation domains represent Level-1. Loopbacks of ABRs and PEs situated in the core are leaked from Level-2 to Level-1 so that Next-Hops of advertised labeled unicast IPv4 prefixes can be correctly resolved. Implementing Next-Hop-Self in both directions is used when non-zero instances of IS-IS are used in the aggregation domains. This allows for complete IGP separation between core and aggregation domains with no route-leaking between instances. In situations where multiple aggregation domains connect to the same ABR, multiple non-zero instances can be used to

control the size of the link-state database in the AGNs. If a single instance of IS-IS was used in this situation, the aggregation domains connecting to the same ABR would form a contiguous Level-1 link state topology. There is a cost associated with imposing Next-Hop-Self however. When an ABR sets Next-Hop-Self on a labeled unicast IPv4 prefix it must allocate a new local label for that prefix and advertise this label to its upstream peers. At the same time, it must programme a label-swap entry in the LFIB for the received and advertised labels. Imposing Next-Hop-Self therefore consumes LFIB resources. It should be reasonably clear that when imposing Next-Hop-Self in both directions the rate of consumption is doubled when compared to imposing Next-Hop-Self towards the core only.

An example of a seamless MPLS topology imposing Next-Hop-Self in both directions is illustrated in *Figure 7-4*. The figure shows two aggregation domains connected to a core domain, each with its own IGP and each running its MPLS data-plane of choice (LDP, RSVP, SR-MPLS, etc). At the BGP layer, AGN3 advertises its system address to its peer ABR3 with Next-Hop AGN3 and label L1. ABR3 performs the role of an IBGP Route-Reflector, and as such it reflects the IBGP-learned prefix from AGN3 to its peer ABR1 setting Next-Hop-Self on the prefix and allocating a new label L2 so that it can perform a label-swap action between the domains. When the prefix arrives at ABR1, it performs the same function, reflecting the prefix to AGN1, setting Next-Hop-Self, and allocating a new label L3.

Figure 7-4: Seamless MPLS Inter-Domain Routing



There are a number of ways that Seamless MPLS can be implemented and *Figure 7-4* is a relatively simple example of one of them. The example shows only the architecture, including aggregation domains, but another layer of hierarchy could exist from the aggregation domain towards the access domain. The objective is not to list all of the possible design

choices with their advantages and disadvantages, but simply to illustrate how labeled BGP is used in this environment to stitch together discrete and manageable ‘islands’.

While the IGP in *Figure 7-4* are completely isolated with no route-leaking between them, the potential exists (subject to policy) that a given AGN will receive a labeled BGP prefix for system addresses of all nodes in the network (AN, AGN, ABR, core PE) except pure P nodes. By default, as long as the Next-Hop can be resolved, these prefixes are placed in the FIB consuming MPLS resources. Whether this is acceptable or not depends on the capabilities of the AGN and the size of the network. However, potential optimisations are possible so that these prefixes are only held in RIB-In (which is typically larger than the FIB by an order of magnitude) and only downloaded to the FIB when there is active forwarding state. In SR-OS this is implemented on the 7250 IXR family using the **selective-label-ipv4-install** command, which can be applied at **bgp**, **group**, or **neighbor** level. When set to **true** any received labeled IPv4 routes are marked as invalid if they are not required to resolve a Next-Hop for any locally configured services. This includes the far-end address of any SDPs and the BGP Next-Hop of any VPN-IPv4/v6 and EVPN routes.

Output 7-1 and *Output 7-2* show an example BGP and associated route-policy configuration for ABR1 using *Figure 7-4* as an example topology. The example is broken into BGP configuration (*Output 7-1*) and policy configuration (*Output 7-2*) for readability, and only the pertinent parts of the configuration are described. Although the outputs describe the BGP configuration for ABR1, similar configuration should exist at the AGNs. The outputs assume that IS-IS and MPLS are configured and operational. In this example, SR-MPLS is used for the MPLS data-plane in both the aggregation and core domains, but the choice of data-plane can be the same or different between domains.

The **advertise-inactive true** command is used to overcome an issue when using labeled BGP to advertise prefixes that are also known by some other protocol such as the IGP. Consider the case in *Figure 7-4* where ABR3 receives the labeled BGP prefix for AGN3’s system address and needs to advertise this upstream to ABR1. ABR3 learns AGN3’s system address in labeled BGP and also in IS-IS (Level-2). Because of default route preferences in SR-OS only the IS-IS learnt prefix is installed in the route-table, and the BGP learnt prefix remains in RIB-In, which basically means that ABR3 does not reflect the route upstream. The **advertise-inactive true** command causes the best BGP route (and only the best route) to be advertised even if it is not the most preferred route within the system for a given destination (an IGP route also exists). When the labeled BGP prefix is advertised, a label swap entry is programmed even though the BGP prefix is inactive. The **backup-path** command followed by the argument **label-ipv4 true** enables Edge PIC for labeled IPv4 routes and is used to provide for fast reconvergence in the event of ABR failure. (The use of Edge PIC is described in further detail in Chapter 9.)

The **next-hop-resolution labeled-routes transport-tunnel** context is used to instruct the system what tunnelling mechanism to use to resolve the BGP Next-Hop of labeled BGP routes when the peers are non-adjacent. By default, SR-OS uses LDP as a transport protocol to attempt to resolve these BGP Next-Hops, but this behaviour is configurable. Within the **labeled-routes transport-tunnel** context the **family** command is used to enable different

tunnel selection criteria for labeled IPv4/IPv6 and VPN routes, and when entered opens a new context for the selected family. In this case the selected **family** is **label-ipv4**. Within the **family** context, the **resolution** command has options for **filter** or **any**. When **any** is selected, the system does a lookup in the tunnel-table and selects the tunnelling mechanism that is the most preferred (lowest numeric value). When **filter** is selected, the **resolution-filter** context is used to filter tunnelling protocols to only those that are permitted. If the requirement is to use only RSVP, then the filter list should contain only RSVP. If for example, both LDP and RSVP are configured, the system will use the preferred tunnel from that filtered list. As RSVP has a lower preference than LDP by default, it would be used in preference to LDP for binding Next-Hops to LSPs. As this example uses SR-MPLS with IS-IS and LDP is the default configuration, the **resolution-filter** is configured as **ldp false** and **sr-isis true**.

Labeled IPv4 and IPv6 routes are retained in separate RIBs from unlabeled IPv4 and IPv6 BGP routes, and to allow more control over the import of IP routes from the routing table into BGP RIBs, the **rib-management** context is used. Within this context the **route-table-import** command provides the ability to use route-policy to control the import of routes into one of the IPv4/IPv6 or Labeled IPv4/IPv6 RIBs. In this example the **label-ipv4** context is selected, under which the **route-table-import** command is used to control which routes are imported into the relevant RIB. This example references a policy with the name “Labeled-Unicast”. This policy is shown in *Output 7-2* and allows the system address of ABR1 (192.0.2.101) to be imported into the **label-ipv4** RIB and thereafter advertised in labeled IPv4. After this prefix has been imported into the relevant RIB, an export policy is applied at **group** or **neighbor** level to advertise the labeled unicast BGP route (and any other routes subject to policy). The **export** policy applied at **group** or **neighbor** level can be the same policy as the one used in the **rib-management** context, or it can be a different policy since the label has already been attached to the prefix during the process of import to the **label-ipv4** RIB. This example uses the same policy as used under **rib-management** for simplicity and applies it to each of its AGN neighbours in its aggregation domain (in *Figure 7-4* this is AGN1 and AGN2.) Since this policy only references the system IP address of ABR1, you may wonder how other routes will be advertised to AGN1 and AGN2. The example **policy-statement** does not have a configured **default-action**. The default behaviour of the **default-action** is to allow routes learnt from BGP to be advertised in BGP. Hence, when ABR1 learns labeled IPv4 BGP routes for other AGNs (notably AGN3 and AGN4) it will propagate them to AGN1 and AGN2 in labeled BGP.

There are two peer **group** contexts with similar configuration, one towards the core domain and the other towards the aggregation domain. Within each **group** context the **peer-as** is configured appropriately to represent a single AS number where all sessions are IBGP. The **family** context has arguments to allow for advertisement of **label-ipv4** and **label-ipv6** Address Families only (in addition to the default IPv4). This is sufficient to build the transport layer, but clearly more Address Families will be required to construct the overlay service layer. Both **group** contexts have **next-hop-self** configured, which means that ABR1 will impose itself as the Next-Hop for labeled IPv4 and IPv6 advertised both to the core domain and to the aggregation domain. The group “Aggregation Domain” contains a **cluster** context with a **cluster-id** entered in dotted decimal format containing its system address 192.0.2.101. This command effectively means that ABR1 is a Route-Reflector and that IBGP peers within this

context are Route-Reflector clients. It can be enabled at BGP level, group level, or neighbour level. In this instance, all the peers in ABR1's connected aggregation domain are Route-Reflector clients. The configured **cluster-id** is populated into the Cluster ID attribute when reflecting Updates to Route-Reflector clients and is used to prevent cluster loops. A Route-Reflector receiving an Update containing its' own Cluster ID will drop the route. Within this aggregation domain it is reasonable that ABR2 would use the same **cluster-id** as ABR1, although it may use its own. Although not shown in *Figure 7-4*, towards the core domain ABR1 is itself a Route-Reflector client.

The last part of the configuration simply configures the **neighbor** parameters. There are two neighbours (192.0.2.1 and 192.0.2.2) belonging to the "Aggregation Domain" and a single neighbour (192.0.2.10) belonging to the "Core Domain". All three **neighbor** contexts contain parameters such as peer **group**, an MD5 **authentication-key**, and where appropriate, an **export policy** previously described.

Output 7-1: Seamless MPLS ABR Configuration example

```

bgp {
  advertise-inactive true
  backup-path {
    label-ipv4 true
  }
  next-hop-resolution {
    labeled-routes {
      transport-tunnel {
        family label-ipv4 {
          resolution filter
          resolution-filter {
            ldp false
            sr-isis true
          }
        }
      }
    }
  }
  rib-management {
    label-ipv4 {
      route-table-import {
        policy-name "Labeled-Unicast"
      }
    }
  }
  group "Core-Domain" {
    next-hop-self true
    peer-as 64496
    family {
      label-ipv4 true
      label-ipv6 true
    }
  }
  group "Aggregation-Domain" {
    next-hop-self true
    peer-as 64496
    family {
      label-ipv4 true
      label-ipv6 true
    }
    cluster {
      cluster-id 192.0.2.101
    }
  }
}

```

```

    }
    neighbor "192.0.2.1" {
        group "Aggregation-Domain"
        authentication-key "C3i5c6IP3VMhQFGff3RCM6O4qzEOtDo= hash2"
        export {
            policy ["Labeled-Unicast"]
        }
    }
    neighbor "192.0.2.2" {
        group "Aggregation-Domain"
        authentication-key "C3i5c6IP3VMhQFGff3RCM3Kf+/MN6G8= hash2"
        export {
            policy ["Labeled-Unicast"]
        }
    }
    neighbor "192.0.2.10" {
        group "Core-Domain"
        authentication-key "C3i5c6IP3VMhQFGff3RCM8pKOP5nZxU= hash2"
    }
}

```

Output 7-2 illustrates the policy referenced at *neighbor* level in *Output 7-1*. In reality, the policy is likely to be a little more restrictive about which prefixes are advertised into each domain, but nonetheless the output illustrates the minimum requirement.

Output 7-2: ABR Route-Policy example

```

policy-options {
    prefix-list "System-Address" {
        prefix 192.0.2.101/32 type exact {
        }
    }
}
policy-statement "Labeled-Unicast" {
    entry 10 {
        from {
            prefix-list ["System-Address"]
        }
        to {
            protocol {
                name [bgp-label]
            }
        }
        action {
            action-type accept
            origin igp
        }
    }
}
}

```

There are a number of ways to view label values that are advertised for a particular prefix. As one would expect, displaying the RIB-In as shown in *Output 7-3* gives details on advertised prefix and label, together with the Next-Hop and resolved Next-Hop information. However, where the router is performing a label-swap operation based on received/advertised labeled BGP routes, the command illustrated in *Output 7-4* is also useful.

Output 7-3: Labeled IPv4 prefix from AGN3

```

A:admin@ABR1# show router bgp routes 192.0.2.3/32 label-ipv4 detail | match
'Network|Nexthop|IPv4 Label'
Network      : 192.0.2.3/32
Nexthop      : 192.0.2.103

```



```
Res. Nexthop   : 192.0.2.103 (ISIS Tunnel)
IPv4 Label     : 524232
```

The Received Label and Advertised Label columns represent the labels that will be programmed into the LFIB as label swap entries. The Internal value in the Label Origin column indicates that the routes were learned in IBGP. It follows that an External value will indicate that the routes were learned in EBGP.

Output 7-4: Labeled BGP received and advertised routes

```
A:admin@ABR1# show router bgp inter-as-label

=====
BGP Inter-AS labels
Flags: B - entry has backup, P - entry is promoted
=====
NextHop                Received      Advertised      Label
                        Label          Label          Origin
-----
0.0.0.0                 0            524252         Edge
192.0.2.1               524267       524258         Internal
192.0.2.2               14002        524255         Internal
192.0.2.103             524231       524244         Internal
192.0.2.103             524232       524245         Internal
-----
Total Labels allocated: 5
=====
```

Service Layer

With the transport layer in place as described in the previous section, services can now be implemented and for the purpose of illustration the following examples will configure an IP-VPN service and a legacy point-to-point pseudowire using SDPs between AGN1 and AGN2. Of course, a pre-requisite for configuration of services is that the relevant signalling mechanisms are in place, and these are not encompassed by the transport layer.

For support of BGP/MPLS IP-VPN services there is clearly a requirement for BGP peering between PEs (AGNs) in support of the VPN-IPv4 and/or VPN-IPv6 Address Families. Not shown in *Figure 7-4* for clarity is a Route-Reflector in the core domain, and for this example the AGNs will peer directly to that Route-Reflector for service-related Address Families. In reality, large aggregation domains will in all likelihood have their own Route-Reflectors for service routes, although most operators tend to keep the advertisement of labeled unicast routes and service routes peered to separate dedicated Route-Reflectors. For delivery of legacy pseudowire services the example will use a targeted LDP session between AGN1 and AGN3 to deliver a single-segment pseudowire.

Creation of a VPRN at AGN1 is straightforward and is no different from any other VPRN configuration. However, it is important that the **auto-bind-tunnel** context caters for resolving BGP Next-Hops with multiple tunnelling protocols. In the example below at AGN1 the **resolution-filter** is configured for **sr-isis true**, but **bgp** is also set to **true** by default and is required in this multi-domain scenario.

Output 7-5: Configuration of VPRN service at AGN1

```

vprn "one" {
    admin-state enable
    service-id 1
    customer "1"
    autonomous-system 64496
    bgp-ipvpn {
        mpls {
            admin-state enable
            route-distinguisher "64496:1"
            vrf-target {
                import-community "target:64496:1"
                export-community "target:64496:1"
            }
            auto-bind-tunnel {
                resolution filter
                resolution-filter {
                    sr-isis true
                }
            }
        }
    }
}
bgp {
    min-route-advertisement 1
    group "CE-EBGP" {
        as-override true
        split-horizon true
        send-communities {
            extended false
        }
    }
    neighbor "192.168.100.10" {
        group "CE-EBGP"
        authentication-key "JK/6NxCqPrX7NU7cD3w1PseqMUXNFik= hash2"
        peer-as 64512
        family {
            ipv4 true
        }
        import {
            policy ["RFC8212-allow-ebgp"]
        }
        export {
            policy ["VPN-IPv4/v6-to-EBGP"]
        }
    }
}
interface "link-to-CE2" {
    ipv4 {
        primary {
            address 192.168.100.9
            prefix-length 30
        }
    }
    sap 1/1/c3/1:200 {
    }
}
}

```

Output 7-6 shows the VPRN route-table at AGN1 and there are three entries.

- Prefix 172.31.1.0/24 is learned locally through BGP, and its associated Next-Hop is a directly connected interface.

- Prefix 172.31.2.0/24 is learned from AGN2 (192.0.2.2) and is resolved to an SR-ISIS LSP (tunneled:SR-ISIS:524294) because the local aggregation domain is using SR-MPLS with IS-IS as the preferred MPLS data-plane, and because the **resolution-filter** included **sr-isis**.
- Prefix 172.31.3.0/24 is learned from AGN3, and this has a Next-Hop of AGN3's system address 192.0.2.3. This Next-Hop cannot be resolved within the aggregation domain and requires recursive resolution using the Next-Hop of the labeled IPv4 prefix to this destination, and as a result it is shown as (tunneled:BGP).

Output 7-6: VPRN route-table at AGN1

```
A:admin@AGN1# show router 1 route-table 172.31.0.0/16 longer
```

```
=====
```

```
Route Table (Service: 1)
```

```
=====
```

Dest Prefix[Flags] Next Hop[Interface Name]	Type	Proto	Age	Pref
172.31.2.0/24 192.0.2.2 (tunneled:SR-ISIS:524294)	Remote	BGP VPN	00h07m58s 100	170
172.31.3.0/24 192.0.2.3 (tunneled:BGP)	Remote	BGP VPN	00h07m58s 1000	170
172.31.1.0/24 192.168.100.10	Remote	BGP	00h16m58s 0	170

```
-----
```

```
No. of Routes: 3
```

```
Flags: n = Number of times nexthop is repeated
```

```
      B = BGP backup route available
```

```
      L = LFA nexthop available
```

```
      S = Sticky ECMP requested
```

```
=====
```

A look at the tunnel-table entries for AGN3 (192.0.2.3) shows that there are two entries. The first entry containing an SDP is not useful here and is configured in readiness to support Layer-2 services. The second entry has an Owner of `bgp` and has a Nexthop of 192.0.2.101 (ABR1). In short, AGN1 cannot resolve the Next-Hop 192.0.2.3 (AGN3) directly, hence it requires recursion to a labeled unicast BGP route to that same destination – in this case to a Next-Hop of 192.0.2.101 (ABR1).

Output 7-7: MPLS tunnel-table at AGN1 for 192.0.2.3/32 (AGN3)

```
A:admin@AGN1# show router tunnel-table 192.0.2.3/32
```

```
=====
```

```
IPv4 Tunnel Table (Router: Base)
```

```
=====
```

Destination Color	Owner	Encap	TunnelId	Pref	Nexthop	Metric
192.0.2.3/32	sdg	MPLS	2003	5	192.0.2.3	0
192.0.2.3/32	bgp	MPLS	262148	12	192.0.2.101	1000

```
-----
```

```
Flags: B = BGP or MPLS backup hop available
```

```
      L = Loop-Free Alternate (LFA) hop available
```

```
      E = Inactive best-external BGP route
```

```
      k = RIB-API or Forwarding Policy backup hop
```

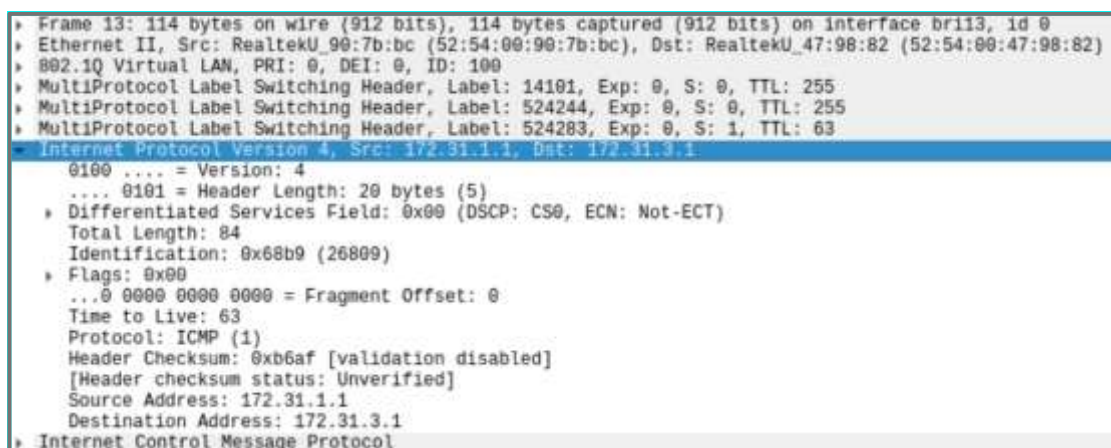
```
=====
```

AGN1 will thereafter resolve the Next-Hop of 192.0.2.101 (ABR1) to an SR-ISIS LSP and will impose a three-level label stack of {140101, 524244, 524283} when forwarding traffic towards prefix 172.31.3.0/24 at AGN3 as shown in *Figure 7-5*:

- The top label 14101 is the SR-ISIS Node-SID label for ABR1.
- The middle label of 524244 is the label advertised with the labeled IPv4 unicast route for AGN3's prefix, 192.0.2.3. This prefix was received by AGN1 with a Next-Hop of ABR1. Hence, AGN1's only way to resolve the Next-Hop of 192.0.2.3 (AGN3) is through ABR1.
- The bottom label of 524283 is the label that AGN3 advertised with the VPN-IPv4 prefix 172.31.3.0/24. This VPN-IPv4 prefix was received by AGN1 with a Next-Hop of AGN3 which it cannot directly resolve without recursion to BGP.

Upon receiving this traffic, ABR1 will pop the top (active) label and label-swap the middle label to a label value advertised by the downstream Next-Hop (ABR3).

Figure 7-5: Packet capture of VPRN traffic from AGN1 to AGN3



Note: When traffic is forwarded over a BGP labeled route that is subsequently resolved to an RSVP, LDP, or SR-MPLS LSP, the system selects a single LSP for resolving that BGP labeled route. There is no support for load-balancing of traffic forwarded over a BGP labeled route over multiple LSPs.

The deployment of Layer-3 IP-VPN services down to the aggregation and access domains represents another interesting challenge in the form of BGP hierarchy for the VPN-IPv4/IPv6 Address Families, and for Next-Hop resolution of VPN-IP prefixes. For example, assume that there are 10,000 AGNs and a large number of them participate in Layer-3 VPNs, there will be a requirement for an AGN performing the role of PE to resolve a large number of Next-Hops (one for every PE from which it learns routes). The Next-Hop resolution of a high number of remote PEs will very likely cause scaling problems for smaller devices situated in aggregation and/or access domains because each Next-Hop consumes FIB space. To overcome this problem the ABRs shown in *Figure 7-3* can optionally perform the role of Route-Reflector for

VPN-IPv4 prefixes (in the same manner as they do for labeled IPv4 prefixes) and implement Next-Hop-Self on VPN-IPv4/VPN-IPv6 and EVPN prefixes advertised upstream. The impact of this is to significantly reduce the number of Next-Hops that have to be stored at the AGNs because the number of Next-Hops is bounded by the number of VPN Route-Reflectors in the same domain rather than all remote PEs. Imposing Next-Hop-Self for VPN prefixes does of course mean that the imposing router is placed in the forwarding path. This would not be a typical configuration for propagation of VPN routes, where Route-Reflectors are normally control-plane-only devices, but is not untypical in Seamless MPLS architectures. The function is enabled at the global BGP level using the command **rr-vpn-forwarding true** and essentially means that AGNs resolve all VPN prefixes to the VPN Route-Reflector(s) within their own domain.

Output 7-8: Next-Hop-Self for VPN prefixes

```

bgp {
  rr-vpn-forwarding true
  group "Core-Domain" {
    next-hop-self true
    peer-as 64496
    family {
      ...
    }
  }
  group "Aggregation-Domain" {
    next-hop-self true
    peer-as 64496
    family {
      ...
    }
  }
  neighbor...
}

```

For point-to-point inter-domain Layer-2 services a targeted LDP session is extended from end to end once the SDP has been configured. The associated SDP must resolve to a labeled BGP tunnel, and to do this, the command **bgp-tunnel true** is used in the SDP configuration together with a **delivery-type** of **mpls**. The **far-end** of the SDP is the destination of the Epipe service and once the SDP is administratively enabled a targeted LDP session will be established. *Output 7-9* shows the SDP and associated Epipe configuration at AGN1 for a pseudowire extended to AGN3. *Output 7-10* that follows it shows the targeted LDP session that is created when the SDP is placed into an **admin-state** of **enable**. In the usual convention, the SDP is subsequently referenced in the Epipe service configuration together with a SAP to create a single-segment pseudowire.

Output 7-9: Seamless-MPLS Epipe configuration

```

service {
  epipe "two" {
    admin-state enable
    service-id 2
    customer "1"
    spoke-sdp 2003:2 {
      admin-state enable
    }
    sap 1/1/c3/1:2 {

```

```

        admin-state enable
    }
}
sdp 2003 {
    admin-state enable
    delivery-type mpls
    bgp-tunnel true
    far-end {
        ip-address 192.0.2.3
    }
}
}

```

Output 7-10: Targeted LDP session from AGN1 to AGN3

```
A:admin@AGN1# show router ldp session targeted ipv4
```

```
=====
LDP IPv4 Sessions
=====
```

Peer LDP Id	Adj Type	State	Msg Sent	Msg Recv	Up Time
192.0.2.3:0	Targeted	Established	2085	2084	0d 03:05:49

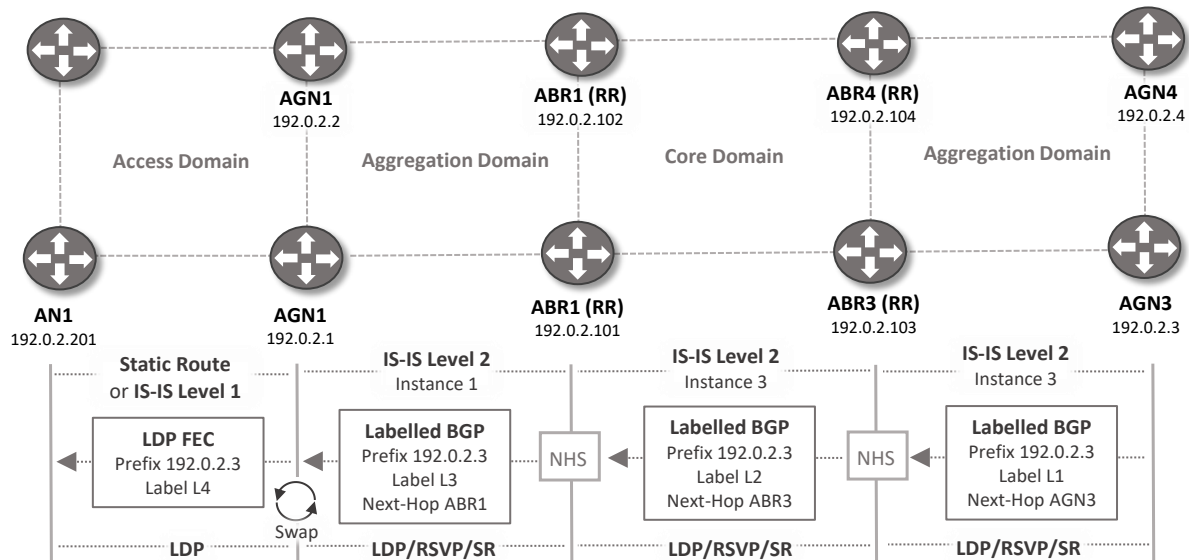
```
-----
No. of IPv4 Sessions: 1
=====
```

If the requirements exist to create an Epipe service to an AGN/ABR/PE device within the same aggregation/access domain (for example, from AGN1 to ABR1) the associated SDP resolves to an LDP/RSVP/SR-MPLS LSP and therefore must not have the command **bgp-tunnel true** configured, otherwise the SDP will not become operational. An option exists however, to configure the SDP as a **mixed-lsp-mode**, which allows for an RSVP LSP to be backed up by an LDP LSP or an LDP LSP to be backed up by a BGP LSP. Because there is a definitive requirement to use BGP in a Seamless MPLS environment **mixed-lsp-mode** is useful only if LDP is used as the transport-level MPLS protocol. If it is used, the SDP should be configured with **bgp-tunnel** and **ldp** to allow the SDP to select the LSP tunnel type as appropriate.

BGP-LDP FEC Stitching

The transport layer defined in *Figure 7-4* works providing that Labeled IPv4 BGP is supported from end-to-end (across Core, Aggregation, and potentially Access domains). However, the devices in the Aggregation or Access domains are typically low(er)-end devices, which may not support labeled IPv4 BGP, and may support only RSVP and/or LDP at the MPLS transport layer. At the same time, there may be a desire or requirement to run labeled IPv4 BGP where possible in the remainder of the network. This is the purpose of BGP-LDP Forwarding Equivalence Class (FEC) stitching. The example topology shown in *Figure 7-4* is therefore extended to encompass an access domain with an access node, AN1. This access node can run IS-IS, static routing, and LDP, but not labeled IPv4 BGP. As a result, and in order to run labeled IPv4 BGP in the remainder of the network, AGN1 performs an interworking function between labeled IPv4 BGP and LDP.

Figure 7-6: BGP-LDP FEC Stitching



When AGN1 receives AN1's system address 192.0.2.201 in an LDP FEC, it translates that LDP FEC and advertises that prefix in a BGP Update message (subject to RIB-Out policy) containing a label. At the same time, it programmes a label swap entry for the received LDP/advertised BGP labels. In addition, AGN1 will advertise a new FEC-label binding for the FEC to all of its LDP neighbours. Conversely, when AGN1 receives a labeled IPv4 BGP route that is permitted to be advertised into LDP (subject to policy), it first verifies that there is an LDP/RSVP/SR-MPLS LSP to the BGP Next-Hop of the prefix, and if yes, advertises a new LDP FEC-label to all of its LDP neighbours. At the same time, it programmes a label swap entry for the received BGP/advertised LDP labels. Using this methodology, AN1 has no requirement to run labeled IPv4 BGP and operates a conventional two-level label stack {service label, transport label}, where the transport label ensures that packets reach the BGP-LDP FEC stitching point at AGN1.

In the data-plane for the direction AN1 to AGN3 (left to right), AGN1 swaps the top LDP label for a BGP label and then pushes on an LDP/RSVP/SR-MPLS label to reach the Next-Hop ABR/ASBR that advertised the labeled BGP route. In the data-plane for the direction AGN3 to AN1 (right to left), AGN1 pops the top (LDP/RSVP/SR-MPLS) label, and swaps the BGP label for an LDP label to reach the next-hop for the prefix. It should therefore be clear that in the schematic of *Figure 7-6* the access domain uses a two-level label stack whilst the aggregation/core domains use a three-level label stack.

Output 7-11 and *Output 7-12* show the BGP/policy configuration requirement for AGN1. The example is broken separately into BGP configuration (*Output 7-11*) and policy configuration (*Output 7-12*) for readability. The majority of the BGP configuration is the same for AGN1 as that previously described for ABR1 (*Output 7-1*), however, there are a couple of notable exceptions. Within the **neighbor** context, the command **advertise-ldp-prefix true** instructs the system to implement the FEC stitching in the direction of LDP to BGP and therefore

advertise LDP FECs into labeled IPv4 BGP. The two neighbours therefore represent AGN1's aggregation domain neighbours, ABR1 and ABR2.

Output 7-11: BGP configuration at AGN1

```

bgp {
    group "Labeled-Unicast" {
        peer-as 64496
        family {
            label-ipv4 true
            label-ipv6 true
        }
    }
    neighbor "192.0.2.101" {
        advertise-ldp-prefix true
        group "Labeled-Unicast"
        export {
            policy ["Labeled-Unicast"]
        }
    }
    neighbor "192.0.2.102" {
        advertise-ldp-prefix true
        group "Labeled-Unicast"
        export {
            policy ["Labeled-Unicast"]
        }
    }
}

```

Output 7-12 illustrates the policy referenced at group level in *Output 7-11*. Entry 10 is required in order to advertise AGN1's system address into labeled BGP and has been previously described (*Output 7-2*), whilst entry 20 is required to take the FEC for AN1's address (learned through LDP) and advertise this to (labeled) BGP.

Output 7-12: Route-Policy at AGN1

```

policy-options {
    prefix-list "AN1" {
        prefix 192.0.2.201/32 type exact {
        }
    }
    prefix-list "System-IP" {
        prefix 192.0.2.1/32 type exact {
        }
    }
}
policy-statement "Labeled-Unicast" {
    entry 10 {
        from {
            prefix-list ["System-IP"]
        }
        to {
            protocol {
                name [bgp-label]
            }
        }
        action {
            action-type accept
            origin igp
        }
    }
    entry 20 {
        from {
            prefix-list ["AN1"]
        }
    }
}

```



```

        protocol {
            name [ldp]
        }
    }
    action {
        action-type accept
    }
}
}
}

```

Output 7-13 and *Output 7-14* show an example LDP/policy configuration for AGN1, and once again the outputs are broken into LDP configuration (*Output 7-13*) and policy configuration (*Output 7-14*) for readability. Within the LDP configuration, the only notable configuration is the command **export-tunnel-table**, which references the policy defined in *Output 7-14*. This implements the FEC stitching for the selected prefixes in the BGP to LDP direction. In this example, the policy specifies that only the prefix for AGN3 may be redistributed from BGP to LDP. The interface towards AN1 is also added for the desired Address-Family within the **interface-parameters** context.

Output 7-13: LDP configuration at AGN1

```

router "Base" {
    ldp {
        export-tunnel-table ["Export-LDP"]
        interface-parameters {
            interface "link-to-AN1" {
                ipv4 {
                    admin-state enable
                }
            }
        }
    }
}

```

Output 7-14: Route-Policy for LDP

```

policy-options {
    prefix-list "AGN3" {
        prefix 192.0.2.3/32 type exact {
        }
    }
    policy-statement "Export-LDP" {
        entry 10 {
            from {
                prefix-list ["AGN3"]
                protocol {
                    name [bgp-label]
                }
            }
            action {
                action-type accept
            }
        }
    }
}

```

To verify that the BGP-LDP FEC stitching is in place and functional at AGN1, *Output 7-15* shows the LDP to BGP control-plane direction for prefix 192.0.2.201 (AN1), where the

received (LDP) label is 524276 and the advertised (BGP) label is 524263. The label origin is also identified as being learnt through LDP (InternalLDP). In the data-plane, AGN1 pops the top level (LDP/RSVP/SR-MPLS) label, and swaps the BGP label 524263 for label 524276.

Output 7-15: LDP to BGP FEC Stitch

```
A:admin@AGN1# show router bgp inter-as-label

=====
BGP Inter-AS labels
Flags: B - entry has backup, P - entry is promoted
=====
NextHop                Received      Advertised    Label
                        Label          Label          Origin
-----
0.0.0.0                 0             524267         Edge
192.0.2.201            524276        524263         InternalLdp
-----
Total Labels allocated: 2
=====
```

Conversely, *Output 7-16* shows the BGP to LDP control-plane direction for prefix 192.0.2.3/32 (AGN3), where the received (BGP) label is 524245, and the advertised (LDP) label is 524262. In the data-plane, AGN1 swaps label 524262 for label 524245 (the operation is shown in the output as *Swap*), and then pushes an LDP/RSVP/SR-MPLS label to get to the BGP Next-Hop that advertised the prefix (in this case, ABR1 at 192.0.2.101). Also note in the output that prefix 192.0.2.3/32 has a (B) flag to denote that it has a BGP Next-Hop.

Output 7-16: BGP to LDP FEC stitch

```
A:admin@AGN1# show router ldp bindings active prefixes prefix 192.0.2.3/32

=====
LDP Bindings (IPv4 LSR ID 192.0.2.1)
              (IPv6 LSR ID 2001:db8:4496::1)
=====
Label Status:
  U - Label In Use, N - Label Not In Use, W - Label Withdrawn
  WP - Label Withdraw Pending, BU - Alternate For Fast Re-Route
  e - Label ELC
FEC Flags:
  LF - Lower FEC, UF - Upper FEC, M - Community Mismatch,
  BA - ASBR Backup FEC
  (S) - Static          (M) - Multi-homed Secondary Support
  (B) - BGP Next Hop    (BU) - Alternate Next-hop for Fast Re-Route
  (I) - SR-ISIS Next Hop (O) - SR-OSPF Next Hop
  (C) - FEC resolved with class-based-forwarding
=====
LDP IPv4 Prefix Bindings (Active)
=====
Prefix                Op
IngLbl                EgrLbl
EgrNextHop            EgrIf/LspId
-----
192.0.2.3/32 (B)      Swap
524262                524245
192.0.2.101          LspId 65542
-----
No. of IPv4 Prefix Active Bindings: 1
=====
```

Inter-AS Option C

Inter-AS Type C defines a model for interconnecting Autonomous Systems using EBGP between ASBRs of neighbouring ASs to redistribute labeled IPv4 prefixes containing PE system addresses between domains. The PE system addresses learnt from the neighbouring AS in labeled EBGP are then redistributed into the ASBR's own AS using IBGP² whilst performing a label swap. This subsequently allows an ingress PE router in one Autonomous System to establish a label switched path to an egress PE router in the neighbouring Autonomous System. When the PE prefixes from the neighbouring AS are known, multihop VPN-IPv4 BGP sessions can be set up between PE routers within each AS to PE routers in the neighbouring AS; or the multihop sessions can be established between Route-Reflectors (although the Route-Reflectors should not modify the Next-Hop attribute of BGP Updates across the EBGP session).

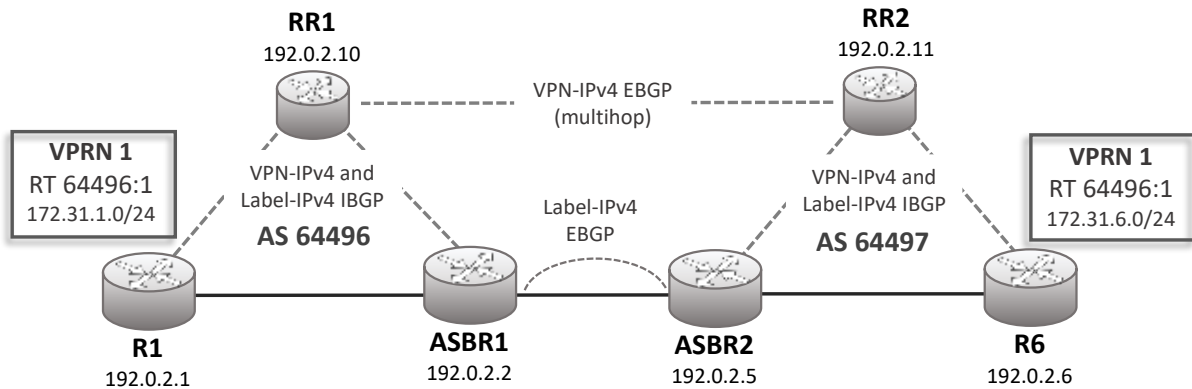
It should be clear in this model that VPN-IPv4 prefixes are not held on the ASBRs. The Type C model does not have such definitive demarcation points as a Type B model (where the only prefixes exchanged between AS's are VPN-IPv4/IPv6) but does provide a more scalable approach in that MPLS data-plane resources are consumed only for infrastructure addresses (PEs, and RRs) rather than VPN prefixes.

The topology depicted in *Figure 7-7* is used to illustrate the use of Inter-AS Type C. Routers R1, ASBR1, and RR1 belong to AS 64496, whilst Routers R6, ASBR2, and RR2 belong to AS 64497. R1 and ASBR1 in AS 64496 are IBGP peered with RR1 for the labeled IPv4 and VPN-IPv4/IPv6 Address Families and the three routers advertise their system addresses into labeled IPv4 BGP. Similarly, R6 and ASBR2 in AS 64497 are IBGP peered with RR2 for the labeled IPv4 and VPN-IPv4/IPv6 Address Families and again all three advertise their system addresses into labeled IPv4 BGP. Routers ASBR1 and ASBR2 peer in labeled IPv4 EBGP to advertise system addresses between AS 64496 and AS 64497. The Route-Reflectors RR1 and RR2 peer in multihop EBGP for the VPN-IPv4/IPv6 Address Families, but do not modify the Next-Hop attribute of any advertised VPN prefixes.

A VPRN service is provisioned between R1 and R6 to check for route propagation and verification of the data-plane.

² The specification also allows for redistribution of labeled IPv4 prefixes from EBGP into some other label distribution protocol (i.e. LDP).

Figure 7-7: Inter-AS option C topology



Output 7-17 shows the configuration implemented on ASBR1. ASBR1 is responsible for advertising labeled IPv4 system addresses from AS 64496 towards the adjacent ASBR (ASBR2), setting Next-Hop to self, and programming a label swap action for the received/advertised BGP labels. ASBR1 is also responsible for redistributing labeled BGP prefixes learnt from ASBR2 into IBGP towards RR1, again setting Next-Hop to self and again programming a label swap action for received/advertised BGP labels. The configuration for external peering includes the **advertise-inactive true** command, which as described in the previous section, causes the best BGP route (and only the best route) to be advertised even if it is not the most preferred route in the route-table for a given destination (in this case the system addresses from AS 64496). The configuration also only permits the **label-ipv4** Address Family in EBGP.

Output 7-17: ASBR1 BGP Inter-AS option C configuration

```

bgp {
  group "EBGP-AS64497" {
    peer-as 64497
    advertise-inactive true
    family {
      label-ipv4 true
    }
  }
  group "IBGP-AS64496" {
    peer-as 64496
    split-horizon true
    family {
      vpn-ipv4 true
      vpn-ipv6 true
      label-ipv4 true
    }
  }
  neighbor "192.0.2.10" {
    group "IBGP-AS64496"
    authentication-key "C3i5c6IP3VMhQFGff3RCM8pKOP5nZxU= hash2"
  }
  neighbor "192.168.0.14" {
    group "EBGP-AS64497"
    authentication-key "JK/6NxQcPrX7NU7cD3w1PkmpZKMQDY8= hash2"
  }
}

```

```
}
}
```

As described previously, in its role as ASBR router, ASBR1 performs a label swap of BGP labels received in IBGP and advertised in EBGP, and also for labels received in EBGP and advertised in IBGP. In the latter case, it is also necessary to push on a transport level label (for example, LDP, RSVP, or SR-MPLS) to reach a non-adjacent next hop. The labels are programmed on a {Next-Hop, Received Label} basis and can be verified using command shown in *Output 7-18*. The first two entries are fairly self-explanatory and are the routers R1 and RR1 in AS 64496 respectively. The second two entries are the routes 192.0.2.6 and 192.0.2.11 received from ASBR2 (192.168.0.14). By looking at ASBR1's BGP RIB-In and RIB-Out for these two prefixes it's possible to determine that the third entry is for prefix 192.0.2.11 and the final entry is for prefix 192.0.2.6.

Output 7-18: ASBR1 Inter-AS BGP Label

```
A:admin@ASBR1# show router bgp inter-as-label
```

```
=====
BGP Inter-AS labels
Flags: B - entry has backup, P - entry is promoted
=====
NextHop                Received      Advertised    Label
                        Label         Label         Origin
-----
192.0.2.1              524260       524272        Internal
192.0.2.10             524283       524271        Internal
192.168.0.14           524282       524275        External
192.168.0.14           524283       524274        External
-----
Total Labels allocated: 4
=====
```

Output 7-19 illustrates the configuration requirements for the Route-Reflector RR1 in AS 64496. For the internal BGP peers, note again the use of the **advertise-inactive** and the presence of the **label-ipv4 true** commands to allow both propagation of labeled IPv4 routes as well as advertising its own system address in labeled BGP. The internal peer group also has a **cluster-id** to define any configured peers as Route-Reflector clients. For the external peer, the session is configured for **multihop** with a TTL indicating up to 16 hops. The **vpn-apply-export** and **vpn-apply-import** commands are both configured to allow global instance route-policies to be applied to VPN routes (VPN-IPv4/IPv6, EVPN, and RT Constraint). There are no import or export policies shown in the example, but it's very likely that these would exist in a production network to control the routes that are both advertised and received.

Output 7-19: RR1 Inter-AS option C configuration

```
bgp {
  advertise-inactive true
  group "IBGP-AS64496" {
    peer-as 64496
    family {
      vpn-ipv4 true
      vpn-ipv6 true
      label-ipv4 true
    }
    cluster {
```

```

        cluster-id 192.0.2.10
    }
    export {
        policy ["Labeled-Unicast"]
    }
}
group "EBGP-AS64497" {
    multihop 16
    vpn-apply-export true
    vpn-apply-import true
    peer-as 64497
    family {
        vpn-ipv4 true
        vpn-ipv6 true
    }
}
neighbor "192.0.2.1" {
    group "IBGP-AS64496"
    authentication-key "C3i5c6IP3VMhQFGff3RCM6O4qzE0tDo= hash2"
}
neighbor "192.0.2.2" {
    group "IBGP-AS64496"
    authentication-key "C3i5c6IP3VMhQFGff3RCM3Kf+/MN6G8= hash2"
}
neighbor "192.0.2.11" {
    group "EBGP-AS64497"
    authentication-key "JK/6NxCqPrX7NU7cD3w1PkTjHave9wo= hash2"
}
}

```



Note: The scenario where an SR-OS device is deployed as a control plane only Route-Reflector for labeled IPv4 or IPv6 routes presents an interesting problem. When the Route-Reflector receives a labeled prefix, it attempts to resolve the BGP Next-Hop (which will be an IPv4 address for a labeled IPv4 prefix or an IPv4-mapped IPv6 address for a 6PE prefix) to an LSP as part of its decision process. In the event that the Route-Reflector is not running MPLS (which is a typical configuration for a control plane only Route-Reflector), any labeled prefixes in the RIB-In are marked as 'invalid'. In this case, the prefix is still reflected, but in the event that there are a number of paths in the RIB-In, only the first 'invalid' path received is advertised. To allow for the Route-Reflector to exercise the full BGP path selection algorithm before advertising its preferred path, it is necessary to instruct the Route-Reflector to resolve BGP Next-Hops for labeled routes to an IGP route using the route-table instead of an LSP using the tunnel-table. This is enabled in the **bgp next-hop-resolution labeled-routes** context using the command **rr-use-route-table true**. However, if the Route-Reflector does not resolve the BGP Next-Hop to an LSP in the tunnel-table, it cannot install a valid label swap operation into the FIB. As a result, the Route-Reflector cannot be in the data-plane: only the control plane. For that reason, when using the command **rr-use-route-table true**, it is a pre-requisite to disable downloading of prefixes to the FIB using the command **route-table-install false** in the global BGP context.

Using the previously described configurations, the BGP labels are propagated across the Autonomous Systems and allow for Inter-AS LSPs to be established. These can be verified at R1, where LSPs to R6 and RR2 are established through BGP. As previously discussed, because the BGP-learned labels are from a non-adjacent peer R1 must impose a transport level label onto the BGP label to reach the next hop. Consequently, an inter-domain VPN service will result in a three-level label stack consisting of {service label, BGP label, transport label}.

Output 7-20: Tunnel-table at R1

```
A:admin@R1# show router tunnel-table protocol bgp
```

```
=====
IPv4 Tunnel Table (Router: Base)
=====
Destination      Owner      Encap TunnelId  Pref  Nexthop      Metric
  Color
-----
192.0.2.6/32      bgp        MPLS   262146      12    192.0.2.2     1000
192.0.2.11/32     bgp        MPLS   262147      12    192.0.2.2     1000
-----
Flags: B = BGP or MPLS backup hop available
      L = Loop-Free Alternate (LFA) hop available
      E = Inactive best-external BGP route
      k = RIB-API or Forwarding Policy backup hop
=====
```

At a service-level, no specific configuration is required to establish a Layer-3 VPN between the Autonomous Systems beyond that already described in Chapter 3, noting that **auto-bind-tunnel** implicitly permits the use of BGP to resolve Next-Hops for VPN prefixes.

VPN “one” is configured on Routers R1 and R6 to provide Inter-AS connectivity and R6 advertises prefix 172.31.6.0/24 with Route-Target 64496:1 which is imported by R1 as shown in *Output 7-21*. Note that the VPN-IPv4 prefix advertised by R6 is received at R1 with the Next-Hop still reflected as R6 (192.0.2.6). To forward traffic in the data-path, R1 must resolve that Next-Hop address to the BGP LSP for R6 shown in *Output 7-20*, which in turn must be resolved to an LSP to the Next-Hop of ASBR1 (192.0.2.2).

Output 7-21: VPRN route-table at router R1

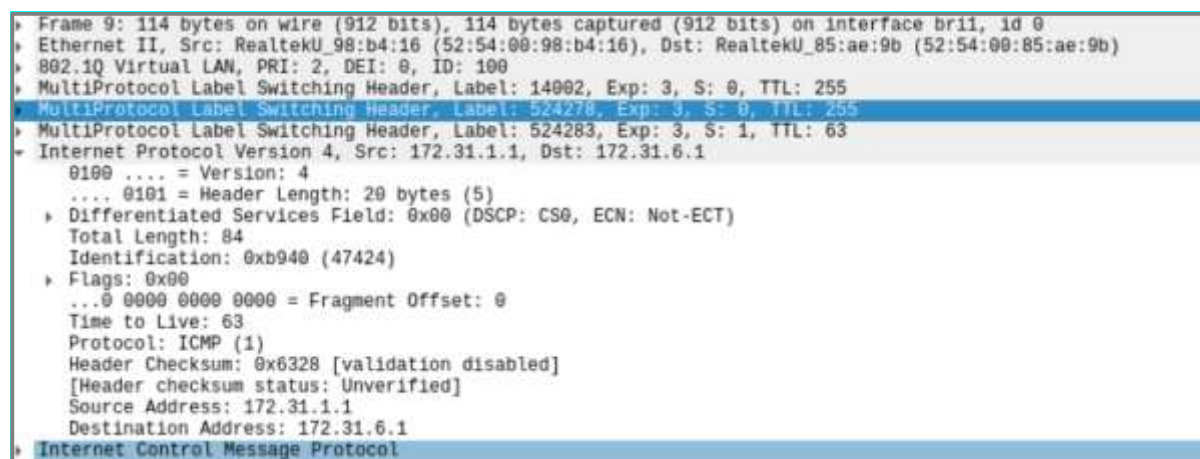
```
A:admin@R1# show router 1 route-table 172.31.0.0/16 longer
```

```
=====
Route Table (Service: 1)
=====
Dest Prefix[Flags]      Type   Proto   Age           Pref
  Next Hop[Interface Name]      Metric
-----
172.31.1.0/24           Remote BGP      04h59m47s    170
    192.168.100.2              0
172.31.6.0/24           Remote BGP VPN 00h04m16s    170
    192.0.2.6 (tunneled:BGP)    1000
-----
No. of Routes: 2
Flags: n = Number of times nexthop is repeated
      B = BGP backup route available
      L = LFA nexthop available
      S = Sticky ECMP requested
```

To verify the data-plane through the inter-AS option C setup, a successful ping is executed from R1 (172.31.1.1) to R6 (172.31.6.1). *Figure 7-8* shows a packet capture of an Echo Request packet as it leaves router R1 towards ASBR1. The captured packet consists of a three-level MPLS label stack:

- Segment routing with IS-IS is in use within AS 64496, and the top label {14002} is the Node-SID for ASBR1.
- The middle label {524278} is the label associated with the labeled unicast BGP route for R6 (192.0.2.6).
- The bottom-of-stack label {524283} is the label that R6 associated with the VPN-IPv4 prefix 172.31.6.0/24.

Figure 7-8: Packet capture of ping from R1 to R6



Note: PE-CE IPv6 for BGP/MPLS IP-VPNs (VPN-IPv6) in an Inter-AS environment is supported in SR-OS. The mechanics are largely the same as for VPN-IPv4, but in this case the system must resolve the IPv4-mapped IPv6 Next-Hop address to a BGP LSP.

Carriers Carrier



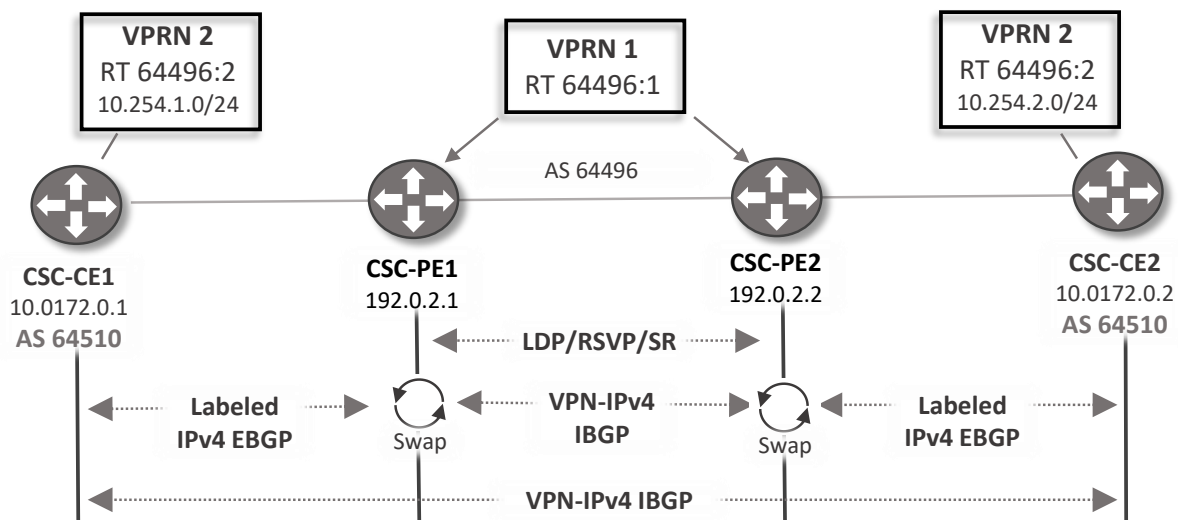
IP-VPN Carriers Supporting Carrier (CSC) is not supported on the 7250 IXR Gen 1 or Gen2/2c platforms.

The BGP/MPLS IP-VPN framework [7] defines a model where an IP-VPN consumer may be a Service Provider that is in turn providing services to its own customer base. This provider may be an ISP, or it may be a provider that is also offering IP-VPN services. In either case, the provider uses the backbone of another Service Provider to extend its connectivity, and the

totally transparent to the Carriers' Carrier CSC-PE routers. In the case of an ISP, IPv4 prefixes are advertised between CSC-CEs, but again they are totally transparent to the CSC-PEs.

The schematic in *Figure 7-10* is used to illustrate the use of the Carriers' Carrier model to provide hierarchical IP-VPNs. In this simple topology CSC-PE1 and CSC-PE2 form the Carriers' Carrier backbone AS 64496 and use VPRN 1 to provide a Carriers' Carrier VPN. CSC-CE1 and CSC-CE2 are simulating provider PE routers, in turn providing a VPN service to their own customer base using VPRN 2. Note that whilst this simple topology shows provider VPNs being delivered directly from the CSC-CEs, it is entirely possible that provider PEs can be attached "behind" the CSC-CEs to deliver services. In this case, however, it would be necessary for the CSC-CEs to redistribute infrastructure prefixes and labels learnt in IPv4 EBGP into IBGP (with label) or an IGP with LDP.

Figure 7-10: Carriers' Carrier Test Topology



Output 7-22 illustrates the configuration requirements for CSC-PE1 in order to support Carriers' Carrier VPN. The difference between a conventional VPN and a Carriers' Carrier VPN is the addition of the **carrier-carrier-vpn true** command, which essentially enables the use of MPLS within the context of a VPRN. In addition, unlike a conventional PE-CE interface defined as an access interface, the interface towards CSC-CE1 is defined as a **network-interface**, again to support MPLS. The BGP peering towards CSC-CE1 includes support for the **label-ipv4** Address Family, enabling the use of labeled BGP for the IPv4 Address Family. The associated import and export policies allow all prefixes to be imported to the RIB-In and allow for the export of prefixes from VPN-IPv4 to labeled unicast BGP and are illustrated in *Output 7-23* for completeness. Lastly, because both CSC-CEs are in a common AS, the CSC-PE routers use the **as-override** command to replace AS 64510 in the AS_PATH attribute with the backbone AS number 64496 when advertising IPv4 prefixes to the CSC-CEs. This allows for the use of a common AS number across CSC-CEs whilst ensuring that IPv4 infrastructure addresses advertised by CSC-PEs are not rejected because of their own AS number appearing in the AS_PATH attribute.

Output 7-22: CSC-PE1 Configuration

```

service {
  vprn "four" {
    admin-state enable
    service-id 4
    customer "1"
    autonomous-system 64496
    carrier-carrier-vpn true
    bgp-ipvpn {
      mpls {
        admin-state enable
        route-distinguisher "64496:4"
        vrf-target {
          import-community "target:64496:4"
          export-community "target:64496:4"
        }
        auto-bind-tunnel {
          resolution filter
          resolution-filter {
            sr-isis true
          }
        }
      }
    }
  }
  bgp {
    min-route-advertisement 1
    group "EBGP-to-CSC-CE2" {
      as-override true
      peer-as 64510
      family {
        label-ipv4 true
      }
    }
    neighbor "192.168.46.5" {
      group "EBGP-to-CSC-CE2"
      authentication-key "JK/6NxCqPrX7NU7cD3w1P17bAuxtYus= hash2"
      import {
        policy ["RFC8212-allow-ebgp"]
      }
      export {
        policy ["VPN-IPv4-to-Labeled-EBGP"]
      }
    }
  }
  network-interface "link-to-CSC-CE2" {
    port 1/1/c3/1:4
    ipv4 {
      primary {
        address 192.168.46.5
        prefix-length 30
      }
    }
  }
}

```

Output 7-23: Route-Policy for CSC-PE1

```

policy-options {
  policy-statement "RFC8212-allow-ebgp" {
    default-action {
      action-type accept
    }
  }
}

```

```

    }
    policy-statement "VPN-IPv4/v6-to-EBGP" {
        entry 10 {
            from {
                protocol {
                    name [bgp-vpn]
                }
            }
            to {
                protocol {
                    name [bgp-label]
                }
            }
            action {
                action-type accept
            }
        }
    }
}

```

As previously described, CSC-PE1 implements a label-swap between received/advertised labeled IPv4/VPN-IPv4 prefixes. This can be seen in *Output 7-24* which shows entries on a {next-hop, received-label} basis. The output shows the received label 524284 for IPv4 prefix 192.0.2.101/32 (system address of CSC-CE1) and the associated label 524240 advertised for the same prefix into VPN-IPv4. The label origin is External (BGP) using Carriers Carrier (ExtCarCarVpn).

Output 7-24: CSC-PE1 LFIB entries for CSC interface

```
A:admin@CSC-PE1# show router bgp inter-as-label
```

```
=====
BGP Inter-AS labels
```

```
Flags: B - entry has backup, P - entry is promoted
```

```
=====
NextHop                Received      Advertised      Label
                        Label          Label          Origin
-----
192.168.46.2           524284        524240         ExtCarCarVpn
-----
```

```
Total Labels allocated: 1
```

```
=====
```

While *Output 7-24* shows the label swap entry for received labeled IPv4 unicast Update and the advertised VPN-IPv4 Update it does not show the corresponding label swap entry for CSC-CE2s system address which was learnt through VPN-IPv4 from CSC-PE2 and advertised in labeled IPv4 BGP towards CSC-CE1. This is shown at VPRN level as shown in *Output 7-25* where the received label of 524236 is the label value CSC-PE2 advertised with the VPN-IPv4 prefix 192.0.2.102/32, and the advertised label of 524237 is the label included with the labeled IPv4 unicast for the same prefix advertised to CSC-CE1.

Output 7-25: CSC-PE1 BGP LFIB entries for VPRN CSC interface

```
A:admin@CSC-PE1# show router 4 bgp inter-as-label
```

```
=====
BGP Inter-AS labels
```

```
Flags: B - entry has backup, P - entry is promoted
```

```
=====
```

NextHop	Received Label	Advertised Label	Label Origin
192.0.2.2	524235	524238	Internal
192.0.2.2	524236	524237	Internal

Total Labels allocated: 2			
=====			



Note: In SR-OS the default mode of label allocation is label-per-VRF. For a Carrier Supporting Carrier VPRN, labels are allocated on a {next-hop, received label} basis, which is essentially what BGP always does when implementing label swaps. A per-VRF label does not provide sufficient context simply because different prefixes may be forwarded over the same or different PE-CE links with the same or different MPLS labels. In essence, a per-VRF label only works when the VRF does an IP lookup.

At CSC-CE1 the presence of a BGP-signalled LSP to CSC-CE2 (192.0.2.102) can be verified with a Next-Hop of CSC-PE1 (192.168.0.1). This allows for a BGP session to be established between the system addresses of CSC-CE1 and CSC-CE2 and for the advertisement with VPN prefixes.

Output 7-26: Tunnel-table at CSC-CE1

```
*A:CSC-CE1# show router tunnel-table
```

```
=====
IPv4 Tunnel Table (Router: Base)
=====
```

Destination Color	Owner	Encap	TunnelId	Pref	Nexthop	Metric
192.0.2.102/32	bgp	MPLS	262145	12	192.168.46.1	1000

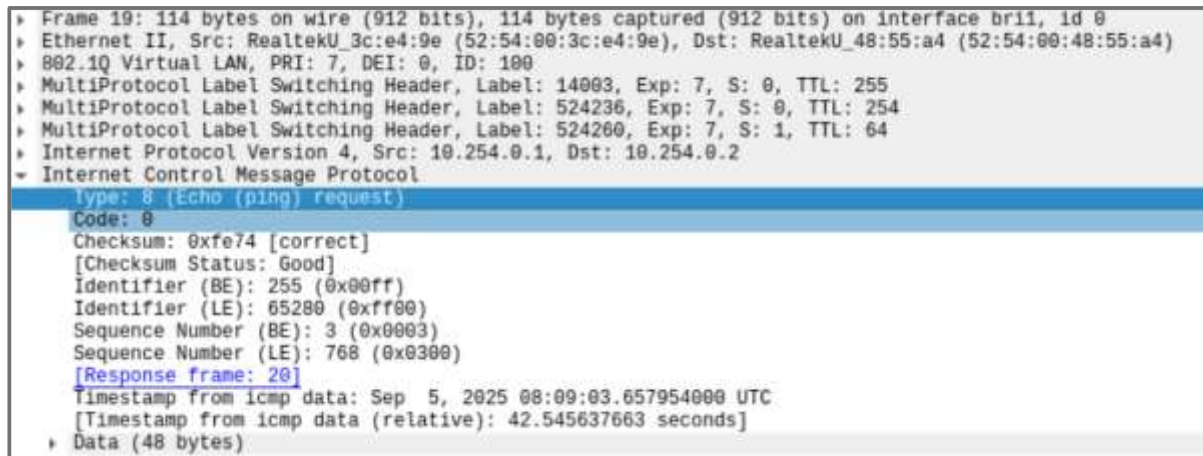
```
-----
Flags: B = BGP or MPLS backup hop available
       L = Loop-Free Alternate (LFA) hop available
       E = Inactive best-external BGP route
       k = RIB-API or Forwarding Policy backup hop
=====
```

To verify the data-path, a VPRN service instance is configured on CSC-CE1 and CSC-CE2 and within that VPRN instance a loopback interface with address 10.254.0.1/32 is configured at CSC-CE1 and a loopback interface with address 10.254.0.2/32 configured at CSC-CE2. A ping is then successfully carried out within that VPRN service instance from CSC-CE1 (10.254.0.1) to CSC-CE2 (10.254.0.2) as shown in *Figure 7-11*, which is a packet capture taken as the packet leaves CSC-PE1. There are three labels imposed on the packet:

- SR-MPLS is in use between CSC-PE1 and CSC-PE2 and the CSC VPRN at both CSC-PEs uses **auto-bind-tunnel** with a **resolution-filter** of **sr-isis**. The top label of {14003} is the Node-SID label of CSC-PE2.

- The second label of 524236 is the VPN-IPv4 prefix that CSC-PE2 advertised for CSC-CE2s system address 192.0.2.102/32.
- The bottom label with Bottom of Stack set to 1 is the service label that CSC-CE2 advertised in VPN-IPv4 to CSC-CE1.

Figure 7-11: Packet capture of Echo Request from CSC-CE1 to CSC-CE2



IPv6 PE Router (6PE)

6PE provides the ability to interconnect IPv6 islands over an IPv4 MPLS backbone using IPv6-enabled PE routers and is described in [34]. The PE routers operate as dual-stack with native IPv6 towards the IPv6 CE and IPv4 towards the core. IPv6 routes are exchanged between the PE and CE using an IGP or BGP and advertised between PE routers using Multi-Protocol BGP AFI 2 (IPv6) SAFI 4 (NLRI with MPLS labels) to indicate the presence of a label. In SR-OS this label is always the Explicit-Null label (value 2). Like its IPv4 counterpart, the NLRI of the MP_REACH_NLRI attribute is encoded as a triple in the form <length, label, prefix> as previously illustrated in *Figure 7-1*. Note that, unlike the extensions to BGP/MPLS IP-VPN for IPv6 described in Chapter 3 (referred to as 6VPE) 6PE does not provide any VRF-awareness.

In the data-plane, an IPv4 MPLS LSP must exist between ingress PE and egress PE. Any native IPv6 traffic received from the CE must be forwarded into the IPv4 LSP together with the Explicit-Null label advertised by that egress PE. Hence a two-level label stack is used, consisting of the Explicit Null label signalled with the IPv6 NLRI, and the transport label. In SR-OS, RSVP is not supported in conjunction with 6PE so LDP or SR-MPLS must be used as the transport layer label distribution mechanism. The use of the Explicit-Null label means that the egress PE must always make a Layer-3 forwarding decision after the label stack has been popped. The operation of 6PE is not considered IPv6-over-IPv4 because when tunnelling the IPv6 packets over the IPv4 MPLS backbone there is no IPv4 header imposed, but rather the ingress PE router performs label imposition directly onto the IPv6 packet.

To illustrate the use of 6PE assume a simple scenario consisting of two PE routers, R1 and R3, providing IPv6 connectivity over an IPv4 MPLS backbone. R1 has IPv6 interface 2001:db8:33ac::1/126 that peers externally in IPv6 with AS 64511, whilst R3 has IPv6

interface 2001:db8:33ac::5/126 that peers externally in IPv6 with AS 64512. AS 64511 peered with R1 advertises IPv6 prefix 2001:db8:2f2:100::/56 whilst AS 64512 peered with R3 advertises IPv6 prefix 2001:db8:2f2:300::/56. The MPLS control plane is SR-ISIS, and the PE routers are peered in IBGP with support for the Labeled IPv6 Address Family. An example of the required BGP configuration for support of 6PE at router R1 is shown in *Output 7-27* and is followed in *Output 7-28* by the route policies that are referenced in it. The **next-hop-resolution** is included to indicate that under **labeled-routes transport-tunnel** the **family label-ipv6** context must be configured for the relevant tunnelling protocol, which in this case is **sr-isis**. As **ldp** is enabled by default, this is also set to **false**. The use of **rib-management** has been discussed earlier in this chapter, and as previously mentioned, labeled IPv4 and IPv6 routes are retained in separate RIBs from unlabeled IPv4 and IPv6 BGP routes. To allow more control over the import of IP routes from the routing table into BGP RIBs, the **rib-management** context is used. In this example it must be used to import routes from the IPv6 RIB to labeled IPv6 BGP at ingress, and from labeled IPv6 BGP to the IPv6 RIB at egress. The **ipv6 route-table-import** context therefore references a policy that accepts the prefix 2001:db8:2f2:300::/56 learned in labeled IPv6 BGP from R3 and allows it to be advertised into BGP. (That is, a prefix is imported from the labeled IPv6 RIB into the IPv6 RIB.) This policy is also applied to the external peer within the **neighbor export** context. Similarly, the **label-ipv6 route-table-import** context references a policy that accepts the prefix 2001:db8:2f2:100::/56 advertised by the external peer AS 64511 in IPv6 and allows it to be advertised into labeled BGP. (That is, a prefix is imported from the IPv6 RIB into the labeled IPv6 RIB.) The remainder of the BGP configuration consists of the (peer) **group** parameters and the external **neighbor** which references the **group**, and for which only the **family** context allows only the **ipv6** Address Family.

Output 7-27: 6PE BGP configuration at R1

```

router "Base" {
  bgp {
    next-hop-resolution {
      labeled-routes {
        transport-tunnel {
          family label-ipv6 {
            resolution filter
            resolution-filter {
              ldp false
              sr-isis true
            }
          }
        }
      }
    }
  }
  rib-management {
    ipv6 {
      route-table-import {
        policy-name "6PE-LabelIPv6-to-IPv6"
      }
    }
    label-ipv6 {
      route-table-import {
        policy-name "6PE-IPv6-to-LabelIPv6"
      }
    }
  }
}

```

```

    group "EBGP-to-AS64511" {
        peer-as 64511
        split-horizon true
    }
    neighbor "2001:db8:33ac::2" {
        group "EBGP-to-AS64511"
        authentication-key "JK/6NxCqPrX7NU7cD3w1Pi7VK0rQZhe= hash2"
        family {
            ipv6 true
        }
        export {
            policy ["6PE-LabelIPv6-to-IPv6"]
        }
    }
}

```

Output 7-28: Route-Policies at R1 for 6PE

```

policy-options {
    prefix-list "IPv6-6PE-Prefixes" {
        prefix 2001:db8:2f2:100::/56 type exact {
        }
    }
    prefix-list "LabelIPv6-6PE-Prefixes" {
        prefix 2001:db8:2f2:300::/56 type exact {
        }
    }
    policy-statement "6PE-IPv6-to-LabelIPv6" {
        entry 10 {
            from {
                prefix-list ["IPv6-6PE-Prefixes"]
                protocol {
                    name [bgp]
                }
            }
            action {
                action-type accept
            }
        }
    }
    policy-statement "6PE-LabelIPv6-to-IPv6" {
        entry 10 {
            from {
                prefix-list ["LabelIPv6-6PE-Prefixes"]
                protocol {
                    name [bgp-label]
                }
            }
            to {
                protocol {
                    name [bgp]
                }
            }
            action {
                action-type accept
            }
        }
    }
}

```

Debug 7-2 shows the prefix labeled IPv6 prefix 2001:db8:2f2:300::/56 advertised by R3 as received at R1. The Next-Hop attribute is encoded as ::fff:192.0.2.3 which is an IPv4-mapped IPv6 address encoding R3's IPv4 system address (192.0.2.3) and allows for the

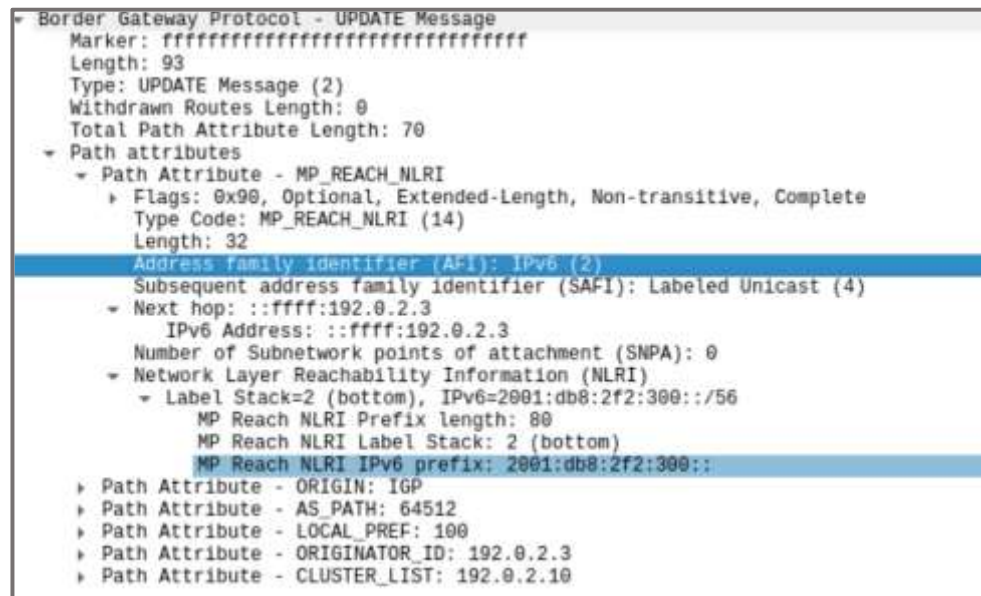
advertisement of IPv6 NLRI with a Next-Hop of the same Address Family. It also allows for the receiving PE to extract the IPv4 address and map it to an IPv4 LSP for Next-Hop resolution. The label field is also set to 2 to indicate the well-known label value of Explicit-Null.

Debug 7-2: MP_REACH_NLRI for IPv6 advertised by R3

```
8 2025/09/05 15:54:14.582 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 70
  Flag: 0x90 Type: 14 Len: 32 Multiprotocol Reachable NLRI:
    Address Family LBL-IPV6
    NextHop len 16 Global NextHop ::ffff:192.0.2.3
    2001:db8:2f2:300::/56 Label 2
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 6 AS Path:
    Type: 2 Len: 1 < 64512 >
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
  Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.3
  Flag: 0x80 Type: 10 Len: 4 Cluster ID:
    192.0.2.10
```

The same Update message is captured on the wire as shown in *Figure 7-12*. The MP_REACH_NLRI attribute indicates that the Update is AFI 2 for IPv6 and SAFI 4 for labeled unicast. The NLRI shows the IPv6 prefix together with the label value of 2 with Bottom of Stack indicator set to 1. The remainder of the path attributes are self-explanatory.

Figure 7-12: Packet capture of IPv6 MP-BGP Update from R3



As previously described 6PE provides no VRF-awareness, therefore any advertised and valid prefixes are held in the IPv6 global routing table.

Output 7-29: Global routing table for IPv6 at R1

```
A:admin@R1# show router route-table ipv6 2001:db8:2f2::/48 longer
```

```
=====
```

IPv6 Route Table (Router: Base)

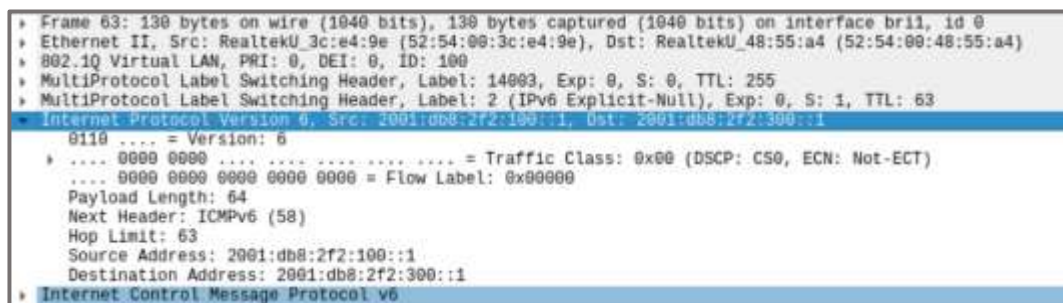
Dest Prefix[Flags] Next Hop[Interface Name]	Type	Proto	Age Metric	Pref
2001:db8:2f2:100::/56	Remote	BGP	00h28m03s 170	
2001:db8:33ac::2			0	
2001:db8:2f2:300::/56	Remote	BGP_LABEL	00h11m23s 170	
192.0.2.3 (tunneled:SR-ISIS:524301)			200	

No. of Routes: 2

Flags: n = Number of times nexthop is repeated
 B = BGP backup route available
 L = LFA nexthop available
 S = Sticky ECMP requested

To verify the data-plane between AS 64511 connected to R1 and AS 64512 connected to R3 through the 6PE backbone a successful ping is carried out from 2001:db8:2f2:100::1 (AS 64511 connected to R1) to 2001:db8:2f2:300::1 (AS 64512 connected to R3). *Figure 7-13* shows a packet capture of an Echo Request as it leaves router R1. The two-level label stack is composed of {14003} which represents R3's SR-MPLS Node-SID label followed by the Explicit-Null label.

Figure 7-13: Echo Request from AS 64511 to AS 64512 through 6PE backbone



The specification in [34] outlines the use of 6PE in Inter-AS scenarios and makes comparisons with the Type A, B, and C described in Section 10 of [7]. A Type A interconnect (EBGP redistribution of IPv6 routes from AS to neighbouring AS) will clearly work because it essentially represents a PE-CE interface. However, the use of the Explicit-Null label imposes some restrictions on the use of Inter-AS Type B and C. The Explicit-Null label can be used only at the bottom of the label stack and indicates that the stack must be popped and forwarding based on the IPv6 header. For a Type B interconnect, there is a requirement to implement a swap of the service-level label, so Explicit-Null cannot be used across this form of interconnect. For a Type C interconnect, SR-OS cannot resolve a labeled IPv6 prefix to a labeled IPv4 prefix, so 6PE across this type of interconnect is also not possible.



Note: Some consideration may be needed where an SR-OS device is deployed as a control plane only Route-Reflector for labeled IPv4 or IPv6 routes. When the Route-Reflector receives a labeled prefix, it attempts to resolve the BGP Next-Hop (which will be an IPv4 address for a labeled-IPv4 prefix or an IPv4-mapped IPv6 address for a 6PE prefix) to an LSP as part

of its decision process. In the event that the Route-Reflector is not running MPLS (which is a typical configuration for a control plane only Route-Reflector), any labeled prefixes in the RIB-In are marked as 'invalid'. In this case, the prefix is still reflected, but in the event that there are a number of paths in the RIB-In, only the first 'invalid' path received is advertised. To allow for the Route-Reflector to exercise the full BGP path selection algorithm before advertising its preferred path, it is necessary to instruct the Route-Reflector to resolve BGP Next-Hops for labeled routes to an IGP route using the route-table instead of an LSP using the tunnel-table. This is enabled in the **bgp next-hop-resolution** context using the command **labeled-routes rr-use-route-table true**. However, if the Route-Reflector does not resolve the BGP Next-Hop to an LSP in the tunnel-table, it cannot install a valid label swap operation into the FIB. As a result, the Route-Reflector cannot be in the data-plane: only the control plane. For that reason, when using the command **rr-use-route-table**, it is a pre-requisite to disable downloading of prefixes to the FIB using the command **route-table-install false** in the global BGP context.

8 Segment Routing and BGP

BGP is used in Segment Routing (SR) in both the creation of SR-MPLS and SRv6 tunnels and in the signaling of overlay services to consume those tunnels.

BGP can be used to distribute SR-MPLS Node-SID labels using the BGP Prefix Segment, or Prefix-SID, where those labels can be used for shortest-path routing or can be part of a traffic-engineered path. To a lesser extent, BGP can be used to advertise SRv6 Locator prefixes, but BGP currently has no concept of the SRv6 Locator TLV used by IS-IS and OSPF, and therefore Locator prefixes are advertised as native IPv6 prefixes in Multi-Protocol BGP. SR-OS refers to these as non-Locator prefixes, but nonetheless they are still used for Inter-AS scenarios, and in environments where no IGP is present. BGP can also be used by an egress router or northbound controller to distribute SR-MPLS or SRv6 Policies to instantiate SR paths at ingress routers that can be either shortest-path or, traffic-engineered.

Overlay BGP services such as IP-VPN, EVPN and base router prefixes that consume SR-MPLS tunnels require no modifications because they continue to advertise a label with VPN NLRI just like any other MPLS client. The introduction of SRv6 did however require modifications in the way that VPN NLRI are advertised in order to consume SRv6 tunnels. This came in the form of a new Service SID TLV used to signal an SRv6 SID together with information on how the SID is constructed.

This chapter will start by describing how SR-MPLS Node-SIDs are advertised in BGP. It will then look at how SR-MPLS and SRv6 SR Policies are advertised in BGP, before finally detailing how BGP is used to signal overlay VPN services to consume the tunnels.

Node-SID Distribution using the BGP Prefix-SID

A BGP Prefix Segment is a BGP prefix with a Prefix-SID attached. It is a globally unique SID within the SR domain and carries an instruction to forward SR-encapsulated packets over the ECMP-aware best path computed by BGP to the prefix. BGP Prefix-SID allows Prefix-SIDs to be advertised across domains, or in environments where there is no IGP present, such as large data centres.

To carry the Prefix-SID in BGP, a Prefix-SID attribute is defined [35] that can be attached to prefixes in (Multiprotocol BGP) IPv4 and IPv6 Labeled Unicast NLRI [36]. In SR-OS, the Prefix-SID attribute is currently supported for the Labeled IPv4 Unicast address family only. The BGP Prefix-SID attribute is an optional, transitive path attribute that currently defines two TLVs, the Label-Index TLV and the Originator SRGB TLV.

Figure 8-1: BGP Prefix-SID Attribute and the Label-Index and Originator SRGB TLVs

Path Attribute:	MP_REACH_NLRI <Flags, Next-Hop, AFI/SAFI, Label, Prefix>
Path Attribute:	Origin, AS_PATH, Next-Hop, etc
Path Attribute:	BGP Prefix-SID

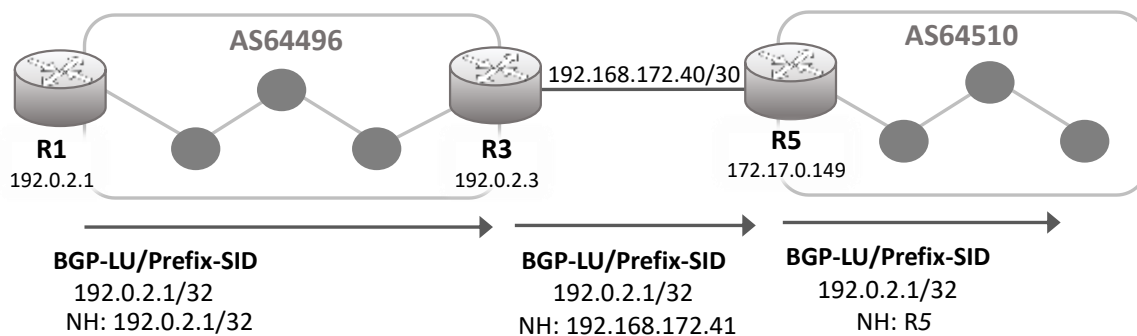
Label-Index TLV		
Type	Length	Reserved
Flags		Label Index
Label Index		

Originator SRGB TLV		
Type	Length	Flags
Flags	SRGB Block n (6 octets)	
SRGB Block n (cont.)		

The Label-Index TLV encodes a 32-bit label from the SRGB and must be present in the BGP Prefix-SID attribute attached to IPv4/IPv6 Labeled Unicast prefixes. The Label Index field defines the assigned label as an offset from the SRGB start label, and although it contains a Flags field there are currently no flags defined. A BGP Prefix-SID attribute received without the Label-Index TLV is considered invalid. The Originator SRGB TLV is an optional TLV and contains the SRGB of the router originating the prefix to which the BGP Prefix-SID is attached. The SRGB field is 6 octets in length and contains 3 octets specifying the first label in the range followed by 3 octets specifying the number of labels in the range. The SRGB field may appear multiple times in an Originator SRGB TLV if the SRGB consists of multiple ranges that are concatenated. Like the Label-Index TLV there are currently no flags defined.

The simple Inter-Domain topology represented in *Figure 8-2* is used to demonstrate the use of BGP Prefix-SID. The intent is to exchange BGP Prefix-SIDs between AS 64496 and AS 64497 and will focus on the advertisement of R1's system address 192.0.2.1/32 from AS 64496 to AS 64510, and R5's system address 172.17.0.149/32 from AS64510 to AS 64496 using labeled unicast BGP with associated BGP Prefix-SIDs.

Figure 8-2: Inter-Domain SR-MPLS with BGP Prefix-SID



By default, SR-OS enforces the uniqueness of a Node-SID between BGP and any existing IGP instance. That is, if a SID is assigned to an interface, that interface and SID cannot be advertised concurrently in an IGP and BGP as BGP will consider this a conflict and will not advertise the SID. Notwithstanding that, it is possible to advertise either a Prefix-SID that is common both to BGP and the IGP or using a dedicated Prefix-SID that is unique to BGP. Both approaches have different configuration requirements:

- If there is a requirement to use the same Prefix-SID in an IGP and BGP, a shared SID must be used. When a shared SID is used, BGP will automatically add the Prefix-SID attribute whenever a local interface with Prefix-SID information is redistributed as a labeled unicast BGP route and BGP **segment-routing** is enabled. In this scenario the **prefix-sid-range** command must be set to **global** in both BGP and the IGP.
- If the intent is that BGP Prefix-SIDs are not shared with the IGP and are unique to BGP, **segment-routing** first needs to be enabled in the base BGP context, and some or all of the SRGB allocated to BGP.

This example follows the latter approach and uses Prefix-SIDs that are unique to BGP. This approach requires that some or all of the SRGB is allocated to BGP Prefix-SIDs as shown in *Output 8-1*. Setting the **prefix-sid-range** to **global** allocates all of the SRGB to BGP Prefix-SIDs, while using the **start-label** and **max-index** parameters allows for allocation of BGP Prefix-SIDs that are a subset of the SRGB. The range defined by the **start-label** and **max-index** parameters must reside within the global SRGB for the command to be successful. The range also needs to be unique to avoid conflict or overlap with other SID allocations such as IGP-based Node-SIDs. In this example the SRGB within the test setup is 12000-19999, of which the range 12000-19499 is allocated to IS-IS, hence the remaining subset 19500-19999 of the SRGB is allocated to BGP. The **prefix-sid-range** command also defines the label block that is advertised in the Originator SRGB TLV of the BGP Prefix-SID attribute. Once the label block is assigned, **segment-routing** can be put into an **admin-state** of **enable**.

Output 8-1: SRGB Prefix-SID allocation to BGP

```
router "Base" {
  bgp {
    segment-routing {
      admin-state enable
      prefix-sid-range {
        start-label 19500
        max-index 499
      }
    }
  }
}
```

As previously described in Chapter 7, labeled unicast BGP routes are retained in a separate RIB from unlabeled BGP routes, and a separate RIB exists for Label-IPv4 and Label-IPv6. Non-labeled unicast BGP routes are imported into the labeled unicast BGP RIB using the route-policy framework and the BGP **rib-management** context. *Output 8-2* shows the configuration at router R1 to import its system IP address 192.0.2.1/32 to the Label-IPv4 RIB with a Prefix-SID attached. The **policy-statement** is fairly generic but notably an **sr-label-index** is used as an **action** output in the policy and can have the value 1-524287. The value defined by the **sr-**

label-index is advertised in the Label-Index TLV of the BGP Prefix-SID attribute. The **sr-label-index** command has an optional **prefer-igp** argument. If this argument is selected and the BGP **segment-routing prefix-sid-range** is **global**, the system will initially try to use the IGP SR label index for the IGP route matched by the policy entry as long as that route matches a received IGP SR route (it will not impact local routes). If the IGP route does not have a SID index, or **prefer-igp** argument is not selected, or the **prefix-sid-range** is not **global**, the value specified by the **sr-label-index** is used.

Output 8-2: Assigning a Prefix-SID to Labeled Unicast BGP routes

```

policy-options {
  prefix-list "system-IP" {
    prefix 192.0.2.1/32 type exact {
    }
  }
  policy-statement "SystemIP-to-LabelIPv4" {
    entry 10 {
      from {
        prefix-list ["system-IP"]
      }
      to {
        protocol {
          name [bgp-label]
        }
      }
      action {
        action-type accept
        sr-label-index {
          value 1
        }
      }
    }
  }
}

```

The configured **policy-statement** is then applied as a **route-table-import** within the **rib-management** context of BGP and the Prefix-SID attribute is attached to the imported routes as a result of the **sr-label-index** action output within that policy.

Output 8-3: Importing routes into the Label-IPv4 RIB

```

router "Base" {
  bgp {
    rib-management {
      label-ipv4 {
        route-table-import {
          policy-name "SystemIP-to-LabelIPv4"
        }
      }
    }
  }
}

```

An export policy is then applied at **group** or **neighbor** level to advertise the labeled unicast BGP route. The **export** policy applied at **group** or **neighbor** level can be the same policy as the one used in the **rib-management** context, or it can be a different policy since the Prefix-SID attribute has already been attached to the prefix during the process of import to the Label-IPv4 RIB. The example below uses the same policy for simplicity and applies it to a

Route-Reflector peer within AS 64496. The peer has the **label-ipv4** address-family enabled to allow the prefix to be advertised.

Although not shown in the output, BGP requires the **advertise-inactive** command to be set to **true** to allow BGP speakers to advertise system IP addresses, or indeed any other addresses known through the IGP. Because of default route preferences, any local or IGP-learned prefix is installed in the route-table and as the BGP prefix is not preferred it is not advertised to other peers. When set to **true**, the **advertise-inactive** command causes the best BGP route (and only the best route) to be advertised even if it is not the most preferred route within the system for a given destination (an IGP route also exists).

Output 8-4: Exporting Labeled Unicast IPv4 routes

```

    bgp {
      group "IBGP-Core" {
        family {
          label-ipv4 true
        }
      }
      neighbor "192.0.2.10" {
        group "IBGP-Core"
        export {
          policy ["SystemIP-to-LabelIPv4"]
        }
      }
    }

```

The prefix is subsequently advertised by R1. The Prefix-SID attribute is shown on the last line of the below output with the Label-Index TLV having a value of 1, and the Originator SRGB TLV showing the first label in the range as 19500 and the number of labels as 499 as configured. The resultant (start_label + index) value of 19501 is shown in the IPv4 Label field with a Label Type field showing the action associated with that label, which is a pop action.

Output 8-5: Label-IPv4 RIB-Out at R1

```

A:admin@R1# show router bgp routes 192.0.2.1/32 label-ipv4 hunt
=====
--- [snip] ---
-----
RIB Out Entries
-----
Network       : 192.0.2.1/32
Nextthop      : 192.0.2.1
Path Id       : None
To            : 192.0.2.10
--- [snip] ---
Originator Id : None                      Peer Router Id : 192.0.2.10
IPv4 Label    : 19501                     Label Type     : SR_POP
Lbl Allocation : PER-PREFIX
Origin        : IGP
AS-Path       : No As-Path
Route Tag     : 0
Neighbor-AS   : n/a
DB Orig Val   : N/A                      Final Orig Val : N/A
Source Class  : 0                        Dest Class     : 0
Prefix SID    : index 1, originator-srgb [19500/500]
-----

```



```
Routes : 1
```

The BGP Prefix-SID advertised by R1 is also shown in *Figure 8-3* captured on-the-wire as it egresses router R1. The AFI/SAFI is IPv4 labeled unicast and the NLRI shows the label value of 19501 together with the IPv4 prefix of R1 (192.0.2.1). The BGP Prefix-SID is carried as an optional transitive attribute and is expanded to show the Label-Index TLV with an Index Value of 1, together with the Originator SRGB TLV showing the SRGB base of 19500 with a range of 499.

Figure 8-3: BGP Prefix-SID advertised by R1

```

Border Gateway Protocol - UPDATE Message
Marker: ffffffffffffffffffffffffffffffff
Length: 82
Type: UPDATE Message (2)
Withdrawn Routes Length: 0
Total Path Attribute Length: 59
  Path attributes
    Path Attribute - MP_REACH_NLRI
      Flags: 0x90, Optional, Extended-Length, Non-transitive, Complete
      Type Code: MP_REACH_NLRI (14)
      Length: 17
      Address family identifier (AFI): IPv4 (1)
      Subsequent address family identifier (SAFI): Labeled Unicast (4)
      Next hop: 192.0.2.1
      Number of Subnetwork points of attachment (SNPA): 0
      Network Layer Reachability Information (NLRI)
        Label Stack=19501 (bottom) IPv4=192.0.2.1/32
        MP Reach NLRI Prefix length: 56
        MP Reach NLRI Label Stack: 19501 (bottom)
        MP Reach NLRI IPv4 prefix: 192.0.2.1
    Path Attribute - ORIGIN: IGP
    Path Attribute - AS_PATH: empty
    Path Attribute - LOCAL_PREF: 100
    Path Attribute - BGP Prefix-SID
      Flags: 0xc0, Optional, Transitive, Complete
      Type Code: BGP Prefix-SID (40)
      Length: 21
      Label-Index: 1
        Type: Label-Index (1)
        Length: 7
        Reserved: 00
        Label-Index Flags: 0x0000
        Label-Index Value: 1
      Originator SRGB
        Type: Originator SRGB (3)
        Length: 8
        Originator SRGB Flags: 0x0000
        SRGB Blocks
          SRGB Block(19500:500)
            SRGB Base: 19500
            SRGB Range: 500

```

A received IPv4 labeled unicast BGP route containing a Prefix-SID is installed in the route-table and the tunnel-table and can have ECMP next-hops or a fast reroute backup next-hop. It can be used for any service that supports labeled BGP transport. Prior to being used however, a labeled unicast IPv4 route with a Prefix-SID attribute must be resolved in the same way as a labeled unicast IPv4 route without a Prefix-SID attribute. Broadly, there are two cases: non-adjacent next-hops, and adjacent next-hops. Non-adjacent next-hops need to resolve the next-hop of the labeled unicast IPv4 route to a tunnel with an endpoint that corresponds to the same next-hop. This is simply because the route contains a label that would not be understood by the receiving routers adjacent downstream neighbours. Adjacent next-hops that set next-hop-self have no requirement to resolve next-hops to a tunnel. Essentially the BGP Prefix-SID becomes the active/top-of-stack label. This is because the labeled unicast route contains a label (BGP Prefix-SID) that was advertised by the receiving routers downstream neighbour. This type of environment would be typical for inter-domain route exchange where ASBRs peer in EBGp using interface addresses, or an IGP-free Data Centre where spines and leaves peer in EBGp using interface addresses. As AS 64496

also has SR for IS-IS enabled it is selected to resolve the labeled BGP routes, and this is enabled in the BGP **next-hop-resolution** context with a **resolution-filter** as shown in *Output 8-6*.

Output 8-6: Next-Hop Resolution for Labeled IPv4 routes

```
router "Base" {
  bgp {
    next-hop-resolution {
      labeled-routes {
        transport-tunnel {
          family label-ipv4 {
            resolution filter
            resolution-filter {
              sr-isis true
            }
          }
        }
      }
    }
  }
}
```



Note: A device like a control-plane-only Route-Reflector that is not in the data-path can set **route-table-install** to **false** and **next-hop-resolution** for **labeled-routes** to **rr-use-route-table true** to resolve BGP next-hops.

Router R3 is configured to peer in IBGP with the AS 64496 Route-Reflector, and in EBGp with router R5 in AS 64510. When a prefix is learned through labeled unicast IPv4 BGP, it is possible to add a Prefix-SID or to modify the received Prefix-SID value using an **import** policy before re-advertising the same route to other peers. This might be useful to avoid label conflicts where an external domain advertises Prefix-SID values that are already in-use within the internal domain. It is again accomplished using the same **sr-label-index** command as an **action** output, but as the route is already populated in the Label-IPv4 RIB no **rib-management** configuration is required. To illustrate how this is achieved R3 modifies the Prefix-SID value received from R1 before advertising the same prefix to R5 in AS 64510. Recall that R1 uses index 1, which with an SRGB start-label of 19500 yields a label value of 19501. Using the policy shown in *Output 8-7* R3 modifies the label of the prefix that it advertises externally to an index value of 401.

Output 8-7: Modifying a received Prefix-SID value

```
policy-options {
  prefix-list "System-IP-R1" {
    prefix 192.0.2.1/32 type exact {
    }
  }
  policy-statement "Prefix-SID-Rewrite" {
    entry 10 {
      from {
        prefix-list ["System-IP-R1"]
      }
      to {
        protocol {
          name [bgp-label]
        }
      }
    }
  }
}
```

```

        action {
            action-type accept
            sr-label-index {
                value 401
            }
        }
    }
}

```

Adding or modifying a Prefix-SID to a received prefix must be implemented as an **import** policy. The above policy is therefore applied on R3's IBGP session towards the AS 64496 Route-Reflector.

Output 8-8: Applying IBGP Import Policy at R3

```

bgp {
    neighbor "192.0.2.10" {
        group "IBGP-Core"
        authentication-key "C3i5c6IP3VMhQFGff3RCM8pKOP5nZxU= hash2"
        import {
            policy ["Prefix-SID-Rewrite"]
        }
    }
}

```

The pertinent parts of the prefix advertised to R5 are shown in *Output 8-9*. The Next-Hop is the interface address of the EBGp peering session between R3 and R5 and as the peers are adjacent next-hops there is no requirement to resolve next-hops to a tunnel. The Prefix-SID contains the modified Prefix-SID label-index of 401, which with an SRGB start-label of 19500 gives a label value of 19901. The `Label Type` field shows the programmed action at R3, which is an SR label swap.

If required, the BGP Prefix-SID can be prevented from propagating outside of an SR domain using the **block-prefix-sid** command at BGP, group, or neighbour level. This command will remove the Prefix-SID attribute from all routes sent and received to and from the peers included in the scope of the command. Without it, the attribute will be propagated unconstrained.

Output 8-9: BGP Prefix-SID advertised to R5 in AS 64510

```

A:admin@R3# show router bgp routes 192.0.2.1/32 label-ipv4 hunt | match "RIB Out" post-
lines 24
RIB Out Entries
-----
Network       : 192.0.2.1/32
Nexthop       : 192.168.172.41
Path Id       : None
To            : 192.168.172.42
--- [snip] ---
IPv4 Label    : 19901                      Label Type      : SR_SWAP
Lbl Allocation : PER-PREFIX
Origin        : IGP
AS-Path       : 64496
--- [snip] ---
Prefix SID    : index 401, originator-srgb [19500/500]

```

To allow for bidirectional connectivity router R5 advertises a labeled unicast BGP route for its prefix 172.17.0.149/32 to R3, which is advertised internally in AS 64496. The Prefix-SID

attribute contains a label-index of 149 and an originator SRGB of 95000, which equates to an IPv4 label value of 19649. Router R1 resolves the BGP next-hop to an SR IS-IS tunnel.

Output 8-10: R3's advertised BGP Prefix-SID received at R1

```
A:admin@R1# show router bgp routes 172.17.0.149/32 label-ipv4 detail

=====
BGP LABEL-IPV4 Routes
=====
Original Attributes

Network       : 172.17.0.149/32
Nexthop       : 192.0.2.3
Path Id       : None
From          : 192.0.2.10
Res. Nexthop  : 192.0.2.3 (ISIS Tunnel)
Local Pref.   : 100                               Interface Name : NotAvailable
--- [snip] ---
IPv4 Label    : 19649
Origin        : IGP
Flags         : Used Valid Best Group-Best In-TTM In-RTM
Route Source  : Internal
AS-Path       : 64510
Route Tag     : 0
Neighbor-AS   : 64510
DB Orig Val   : NotFound                          Final Orig Val : N/A
Source Class  : 0                                Dest Class    : 0
Add Paths Send : Default
RIB Priority   : Normal
Last Modified  : 00h01m28s
Prefix SID    : index 149, originator-srgb [95000/500]
--- [snip] ---
```

As the labeled unicast BGP prefix for R5 is resolved and valid, R1 installs a route-table entry and a tunnel-table entry for it. This may now be used for BGP and services as required.

Output 8-11: Tunnel-Table entry for R5 at router R1

```
A:admin@R1# show router tunnel-table 172.17.0.149/32

=====
IPv4 Tunnel Table (Router: Base)
=====
Destination      Owner      Encap TunnelId  Pref  Nexthop      Metric
  Color
-----
172.17.0.149/32  bgp        MPLS  262145      12    192.0.2.3    1000
-----
Flags: B = BGP or MPLS backup hop available
      L = Loop-Free Alternate (LFA) hop available
      E = Inactive best-external BGP route
      k = RIB-API or Forwarding Policy backup hop
=====
```



Note: A labeled unicast route carrying a BGP Prefix-SID cannot appear as the top segment in an SR Policy otherwise the policy will not be correctly resolved.

When using BGP Prefix-SIDs there is a need to be mindful of SID conflicts. When the label-index of an advertised BGP Prefix-SID is combined with the SRGB start-label to derive a local label value that label may have already been allocated to another application, hence a conflict

exists. Any existing conflicts will be shown in the CLI output of the labeled BGP route attribute flags field as a `ConflictSID`. SR-OS will address conflicts such as these in the following manner:

- If the conflict is with another BGP route for another prefix, all of the conflicting BGP routes are advertised with a normal labeled unicast BGP label taken from the dynamic label range.
- If the conflict is with an IGP route and BGP is not attempting to redistribute that IGP prefix as a labeled unicast IPv4 or IPv6 prefix containing an **sr-label-index** action set to **prefer-igp true**, the IGP route takes priority and the BGP route is advertised with a normal labeled unicast BGP label from the dynamic range.
- If the conflict is with an IGP route, and BGP is attempting to redistribute that IGP route as a labeled unicast IPv4 or IPv6 prefix with a route-policy that has the **sr-label-index** action set to **prefer-igp true**, this is not considered a conflict and BGP uses the IGP-advertised label to derive the BGP label-index value. This effectively stitches the BGP SR tunnel to the IGP SR tunnel.

Advertisement of SR Policies in BGP



The 7250 IXR generation one and generation 2/2c platforms provide support for SR Policy but only support a single segment list per policy.

The SR Policy Architecture [37] is a generic framework that describes the procedures and processes that a headend node should carry out when instantiating such a path. An SR Policy consists of an ordered list of segments on a node, sufficient to implement a traffic engineered path through the network. The segments can contain any type of Segment Identifier (SID), including Adjacency-SIDs, Node-SIDs, Anycast-SIDs, and Binding SIDs. The headend can then steer traffic into the SR Policy as appropriate, and SR Policy has inherent mechanisms allowing for traffic to be dynamically associated with a given policy.

An SR Policy can have one or multiple candidate paths, and SR-OS provides support for both. When explicit candidate paths are used, each path will contain one or more Segment-Lists, where each Segment-List contains the ordered set of SIDs required to provide the source-routed path from headend to destination. When a candidate path contains multiple Segment-Lists, each is assigned a weight for the purpose of weighted load-balancing. Candidate paths can be instantiated through a variety of mechanisms, including PCEP, BGP, or static local configuration, and SR-OS provides support for instantiating SR Policies using static local configuration and through BGP. This section is focused on the advertisement of SR Policies in BGP, but as SR-OS requires static configuration of an SR Policy before it can be advertised into BGP, it will describe both. To aid understanding, the section will start with a brief overview of SR Policy. The term BGP SR Policy is interchangeably used with BGP SR TE Policy.

SR Policy Overview

An SR Policy is identified through the tuple {headend, color, endpoint}. The headend is the node where the SR policy is instantiated, and the node that will be responsible for steering traffic into the policy with the relevant SID stack. From the perspective of the headend, the SR Policy can be identified using the {color, endpoint} tuple.

Color

Each SR Policy is associated with a color, and it is a fundamental part of the policy. It is a 32-bit numerical value that the headend uses to associate the SR Policy with a characteristic, such as low-latency or high-throughput, for the purpose of traffic steering. Color is also a 32-bit transitive extended community [31] that can be attached to a given BGP Update message in order to associate itself with a corresponding SR Policy. For example, if headend H learns a BGP route R with {next-hop N, color extended community C, and VPN label V} and headend H has a valid SR Policy P to {endpoint N, color C}, it can associate BGP route R with the SR Policy P. When H receives packets with a destination matching R, it forwards them using the instructions contained within SR Policy P.

Endpoint

The endpoint is the destination of the policy specified as an IPv4 or IPv6 address, although ‘wildcard’ destinations can be used and are discussed later in this chapter.

Discriminator

Each candidate path of an SR Policy is associated with a 32-bit discriminator value to uniquely identify it within the context of that SR Policy. When static SR Policies are used the discriminator has local significance only. When BGP is used to advertise SR Policies [38] a *distinguisher* field forms part of the NLRI, and this serves as the discriminator value. Because it forms part of the NLRI its purpose is to make the candidate path unique from an NLRI perspective. BGP SR Policy uses standard BGP propagation and best-path selection, hence a unique distinguisher ensures that best-path selection does not unnecessarily suppress SR Policy advertisements, for example, at a Route-Reflector. Multiple candidate paths can exist for a given SR Policy, although only one path can be selected as the best path of the policy and become the active path. If several candidate paths of the same SR Policy {endpoint, color} are advertised via BGP to a headend, unique distinguishers for each NLRI are recommended.

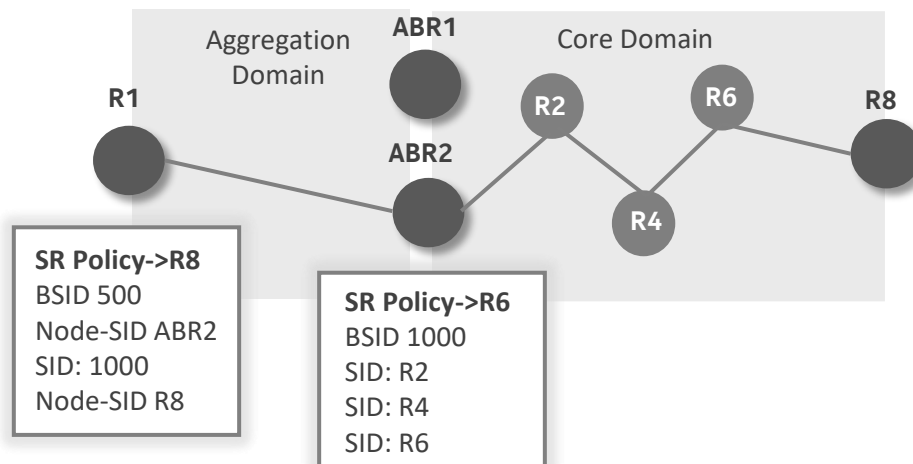
Binding SID (BSID)

The Segment Routing architecture defines the use of a Binding Segment, or BSID. A BSID is bound to an SR Policy, and packets arriving at a node with an active label equal to the BSID are steered into that SR Policy. This action may well mean swapping the incoming active label with a number of outgoing labels representing the SR Policy path. When used in this manner, the Binding SID serves as an anchor point, sometimes referred to as a ‘*BSID anchor*’ that allows one domain to be isolated from another domain. This is illustrated in *Figure 8-4*, where ABR2 acts as a BSID anchor between the aggregation domain and the core domain. ABR2 has

an SR Policy to R8 with the path R2-R4-R6 and with a BSID of 1000. R1's resulting SR Policy to R8 consists of the path {Node-SID ABR2, 1000, Node-SID R8}. When a packet is forwarded into this policy and arrives at ABR2, it will pop the Node-SID ABR2, and swap (BSID) label 1000 for the label stack {R2, R4, R6}.

Note that if ABR2 had an SR policy that included the path R2-R4-R6 and the Node-SID of R8, the label stack in R1's resulting SR policy could be even further reduced to {Node-SID ABR2, 1000}.

Figure 8-4: Binding SID (BSID) Anchor



The BSID serves as an anchor point which allows one domain to be isolated from the churn of another domain. If something changes in the path R2-R4-R6, ABR2 can repair the path locally without needing to change the BSID value known at R1. R1 is therefore protected from the churn in the core domain. The BSID also serves to reduce the number of segments/labels that the headend needs to impose to create an end-to-end traffic engineered path.

Segment List

The Segment List encodes a single path towards the endpoint. Multiple Segment Lists may be included in each SR Policy candidate path. Each Segment List may contain multiple Segments and may carry a Weight sub-TLV. Multiple Segments can be concatenated together within a Segment List to constitute an end-to-end path of the SR Policy. There are several types of the Segment that allow it to be encoded as variants of IPv4/IPv6 node address or local/remote address, and with a SID in the form of an MPLS label or IPv6 address. The SID contained within each Segment can be any form of SID, including Node-SID, Adjacency-SID, Anycast-SID, or Binding SID.

The optional Weight field within a Segment List is used to implement (weighted) load-balancing in the presence of multiple Segment Lists. By default, SR-OS assigns a weight value of 1 to each Segment List and will consequently treat multiple Segment Lists as equal-cost by default.

Preference

The preference of a candidate path is used to select the best candidate path for an SR Policy in relation to other candidate paths. Multiple candidate paths can exist in an SR Policy, but only one candidate path can be selected as the best and active path. When multiple candidate paths exist that are considered valid, the candidate path with the highest preference will be selected. The default value of the preference is 100. If multiple candidate paths have the same preference, the protocol origin (PCEP, BGP SR Policy, static local configuration) may be considered, followed by the lower value of originator, followed by the higher value of discriminator. The default values for protocol origin are 30 for static SR policies, 20 for BGP SR policies, and 10 for PCEP signalled SR policies, with the highest value being preferred.

SR TE Policy NLRI

To allow a candidate path for an SR Policy to be advertised in BGP the ‘SR TE Policy’ (SAFI 73) is defined [35] and is carried in an update message using BGP multiprotocol extensions [38]. The AFI must be IPv4 or IPv6. An SR Policy candidate path may be advertised from a centralised controller, or it may simply be advertised by a router; for example, an egress router advertising one or more paths towards itself. The structure of the SR TE Policy NLRI is shown in *Figure 8-5* and encodes the parameters of the SR Policy that have been previously described.

Figure 8-5: SR TE Policy NLRI

Path Attribute:	MP_REACH_NLRI <Distinguisher, Policy Color, Endpoint>		
Path Attribute:	Origin, AS_PATH, Local_PREF, etc		
Path Attribute:	Tunnel_Encapsulation_Attribute: Tunnel Type SR Policy		
	Sub-TLV:	Binding SID	
	Sub-TLV:	Preference	
	Sub-TLV:	Segment List	
		Sub-TLV:	Segment
		Sub-TLV:	Segment

The SR TE Policy NLRI is used to identify an SR Policy candidate path, and as it uses Multi-Protocol BGP it is carried in an MP_REACH/UNREACH_NLRI path attribute. The NLRI contains the color, endpoint, and distinguisher values described above.

The other parameters of the SR Policy candidate path such as BSID, Segment List, and Preference are carried as sub-TLVs of the Tunnel Encapsulation Attribute [31] using a new Tunnel-Type known as ‘SR Policy’ (value 15).

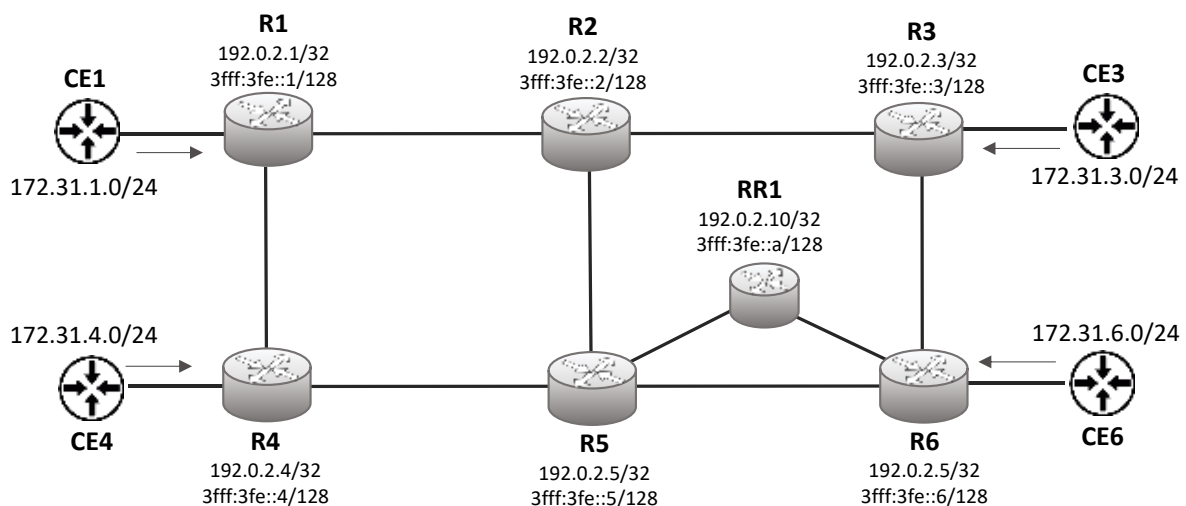
Test Topology

The generic network topology in *Figure 8-6* is used to illustrate the use of both BGP SR policy and static SR policy. Routers R1 to R6 are in a flat IS-IS Level 2 domain and the relevant system addresses are shown. All routers peer in IBGP with a Route-Reflector RR1 for multiple Address Families including IPv4 and IPv6 SR Policy. IBGP peering will use IPv4 transport whilst illustrating the use of IPv4 SR Policy but will use IPv6 transport for the illustration on the use of IPv6 SR Policies. IPv6 SR Policies could in fact be advertised over IPv4 transport, however, the requirement at the ingress router is to correlate between the IPv6 endpoint of the SRv6 Policy, and the Next-Hop of advertised VPN prefixes.

SR is enabled with an SRGB of 12000-19999, and Node-SIDs are taken from the range 1400<router-id>, for example router R1 has Node-SID 14001 and router R5 has Node-SID 14005. Two /32 locator blocks are used for SRv6, both of which have node lengths of 16-bits to yield a total 48-bit locator length. Locator block 3fff:3ea::/32 is assigned to the base algorithm (algorithm 0) while Locator block 3fff:3eb::/32 is assigned to Algorithm 128. Node numbers are 1 to 6 for routers R1-R6 respectively. For example, R1's advertised locators are 3fff:3ea:1::/48 for algorithm 0 and 3fff:3eb:1::/48 for algorithm 128, while R5's locators are 3fff:3ea:5::/48 for algorithm 0 and 3fff:3eb:5::/48 for algorithm 128.

Finally, a number of CE routers are used and each of them advertises a single IPv4 prefix to its connected (PE) router. These will be used for the purpose of illustrating traffic steering with SR Policy.

Figure 8-6: Test Topology for SR Policy



SR Policy MPLS Local Block

For SR Policies that use MPLS, a Binding SID may be allocated either a local or global SID. In SR-OS they are always allocated a local SID, in which case the BSID needs to be within the range of a locally-configured Segment Routing Local Block (SRLB). In SR-MPLS SRLBs are reserved label blocks used for specific local purposes such as SR Policy/BSID, Adjacency Set

SIDs, and assignment of static Adjacency SIDs. A dedicated SRLB is required per-application and has local significance only. Since they have local significance, the same values could be used on all SR routers in the domain for operational ease. Ranges for each SRLB are taken from the dynamic label range. As the SRGB in the topology is {12000,19999} the SR Local Block {20000, 23999} is allocated to the use of SR Policy.

Initially the SR Local Block is created using the **reserved-label-block** command within the **mpls-labels** context. Within the context of that local block, the **start-label** and **end-label** define the range of the block. Once the SRLB is defined it is dedicated to the specific application, which in this case is SR Policies. After the **reserved-label-block** has been assigned, **sr-policies** must then put into an **admin-state** of **enable**.

Output 8-12: SR Policy MPLS Label Block

```
router "Base" {
  mpls-labels {
    reserved-label-block "sr-policy" {
      start-label 20000
      end-label 23999
    }
  }
  segment-routing {
    sr-policies {
      admin-state enable
      reserved-label-block "sr-policy"
    }
  }
}
```

Static SR Policy

As previously indicated, SR Policies can be applied through static configuration, or they can be learned through BGP. In this section the necessary steps are shown for instantiation of an SR policy for MPLS with static configuration using router R1 as a headend.

Output 8-13 shows the configuration of a static SR policy at R1 (192.0.2.1) with an endpoint of R3 (192.0.2.3). The static SR policy is initially created within the **sr-policies** context and begins by selecting the **type** of SR Policy this is intended to be, with the options being **mpls** or **srv6**. The default is **mpls** and as a result it would not normally be shown in a configuration output. It is included here however to emphasise the importance of this option. This is followed by assigning a **binding-sid** of 20001. In this example the SR Policy is local to R1, and the BSID value is therefore within the range of R1's SRLB allocated to SR Policies. If this static policy were to be advertised into BGP, the advertised BSID value must be in the range of the SRLB configured on the target headend. The policy **color** is 100 and is a 4-octet value that can be configured in the range 1-4294967295. The **distinguisher** represents the discriminator value and is also a 4-octet value with the same range and is configured simply as 12345. As previously described, the distinguisher is part of the NLRI, and its purpose is to uniquely identify a candidate path within the context of an SR Policy. For example, two candidate paths of the same policy would require different discriminator values to avoid BGP best-path route suppression. When BGP is used to advertise SR Policies the NLRI uses the

distinguisher value as the discriminator. SR-OS therefore uses the command **distinguisher** as a generic term to mean both discriminator and distinguisher.

The **endpoint** is the IPv4 or IPv6 address of the destination for the policy and is configured as R3's address 192.0.2.3. There are special circumstances where the value 0.0.0.0 or 0::0 is allowed as an endpoint. This is referred to as *color-only steering* and can be used for forwarding of traffic into an SR Policy with no specific endpoint address. The **head-end** is the target node where the SR Policy is to be instantiated. If the SR Policy is statically configured on the headend itself for forwarding of traffic locally into that policy, the value **local** should be used as shown in this example. If the SR Policy is configured somewhere other than the headend for the purpose of advertising into BGP towards the headend, the value of the **head-end** parameter should be the IPv4 address of that headend. When the SR Policy is advertised in BGP, the headend address is also encoded as an IPv4 address specific Route-Target extended community, which allows for potential constraining of route propagation to only the node that needs it.

The final parameter is the **segment-list**. The configuration output shows the segment list consisting of four segments which represent R1's Adj-SID towards R2, R2's Adj-SID towards R5, R5's Adj-SID towards R6, followed by R3's Node-SID. There is no particular rationale for using this SID stack, but it serves to illustrate that both Adj-SIDs and Node-SIDs with strict or loose hops can be used, where an Adj-SID represents a strict-hop and a Node-SID represents a loose-hop. The segment list has an optional weight parameter used for load-balancing across multiple segment lists. In this example only a single segment list exists, hence the default weight value of 1 is retained.

Output 8-13: Configuration of static SR Policy to R3 at R1

```
router "Base" {
  segment-routing {
    sr-policies {
      static-policy "R1-to-R3-Color100" {
        admin-state enable
        color 100
        endpoint 192.0.2.3
        head-end local
        binding-sid 20001
        distinguisher 12345
        type sr-mpls
        segment-list 1 {
          admin-state enable
          segment 1 {
            mpls-label 524272
          }
          segment 2 {
            mpls-label 524278
          }
          segment 3 {
            mpls-label 524267
          }
          segment 4 {
            mpls-label 14003
          }
        }
      }
    }
  }
}
```

}

Once the segment list and the static-policy are put into an **admin-state** of **enable** the operational state of the static SR Policy can be validate as shown in *Output 8-14*. The **Active** field shows whether this candidate path is the selected path in the presence of multiple candidate paths. The SR Policy segment list is considered valid if the headend is able to perform path resolution for the first SID in the segment list into one or more outgoing interfaces and next-hops. As the **Active** state shows **Yes**, the parameters of the SR Policy have been validated, and the first SID in the segment list (segment 1) has been resolved. The segment 1 label is an MPLS label 524272, and the state is shown as **resolved-up**, indicating that this is a valid segment list.

Output 8-14: Operational state of static SR Policy to R3

```
A:admin@R1# show router segment-routing sr-policies static color 100 end-point 192.0.2.3

=====
SR-Policies Path
=====
-----
Type                : srMpls
Active              : Yes                      Owner                : static
Operational         : Yes
Color               : 100
Head                : 0.0.0.0                  Endpoint Addr       : 192.0.2.3
RD                  : 12345                     Preference         : 100
BSID                : 20001
TunnelId            : 917515                    Age                 : 54463
Origin ASN          : 0                        Origin              : 0.0.0.0
NumReEval           : 0                       LastReEvalReason    : none
NumActPathChange    : 0                       Last Change         : 09/09/2025 09:30:22
Maintenance Plcy:
Ret Path BFD Lbl:

Path Segment Lists:
Segment-List        : 1                        Weight               : 1
Num Segments        : 4                       Last Change          : 09/04/2025 17:46:10
  1 MPLS Label      : 524272                   State                : resolved-up
  2 MPLS Label      : 524278                   State                : N/A
  3 MPLS Label      : 524267                   State                : N/A
  4 MPLS Label      : 14003                    State                : N/A
=====
```

If the policy is considered valid, it is populated in the tunnel-table with an owner of **sr-policy**. The entry indicates the destination and color and always has a metric value of 0 regardless of how the SR Policy is instantiated. The 0 metric is used simply because there is no effective way for the headend to determine a more reflective value for an SR Policy when learnt through BGP or statically configured. The preference value assigned to SR Policy is 14 and is currently non-configurable.

Output 8-15: Static SR Policy tunnel-table entry at R1

```
A:admin@R1# show router tunnel-table 192.0.2.3/32 protocol sr-policy

=====
IPv4 Tunnel Table (Router: Base)
=====
Destination          Owner      Encap TunnelId  Pref  Nexthop      Metric
-----
```

Color					

192.0.2.3/32	sr-policy	MPLS	917515	14	192.168.0.2
100					0

Flags: B = BGP or MPLS backup hop available					
L = Loop-Free Alternate (LFA) hop available					
E = Inactive best-external BGP route					
k = RIB-API or Forwarding Policy backup hop					
=====					

To test the data-path of the newly created static SR Policy an LSP Ping [39] is issued from R1 referencing the previously configured **endpoint** and **color**. This is essentially an MPLS Echo Request using the MPLS label stack defined in *Output 8-13*. The MPLS Echo Request contains a base header followed by information about the Forwarding Equivalence Class (FEC) whose MPLS path is being verified. In ping mode, the packet should reach the end of the path, at which point it is sent to the control plane of the egress LSR to verify whether it is indeed an egress for the FEC. In traceroute mode the packet is sent to the control plane of each transit LSR, which performs various checks to confirm that it is indeed a transit LSR for this path. The MPLS Echo Request is a UDP packet with a destination IP address in the 127/8 range for IPv4, and that same range embedded in an IPv4-mapped IPv6 address for IPv6. Routing of the MPLS Echo Request is based solely in the imposed label stack, hence this destination IP address is never used in a forwarding decision. The MPLS Echo Request packet contains Return Code and Return Subcode fields that allow the destination to indicate how it processed the Echo Request, and a set of pre-defined Return Code values are contained in section 3.1 of [39]. The Return Subcode uses the notation <RSC> and contains the point in the label stack where processing was terminated. If the RSC is 0, no labels were processed, otherwise the packet was label switched at depth <RSC>.

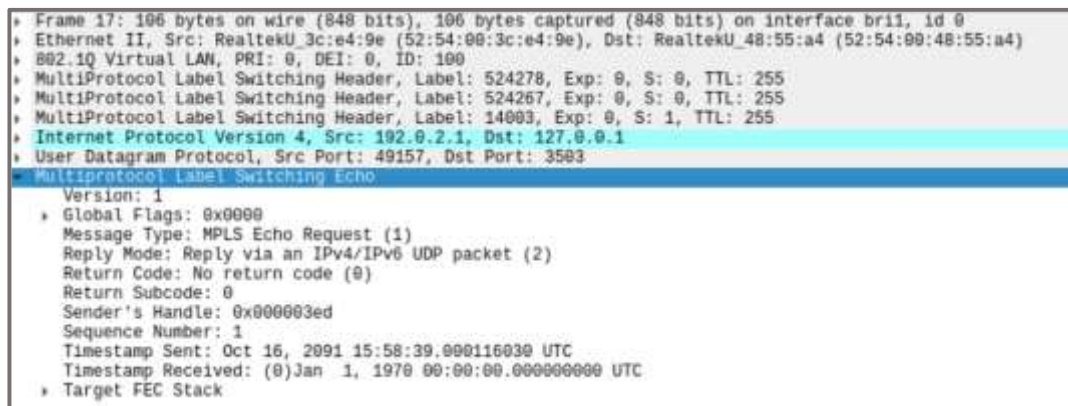
Output 8-16: LSP-Ping from R1 to R3 through static SR Policy

```
A:admin@R1# oam lsp-ping sr-policy endpoint 192.0.2.3 color 100
LSP-PING color 100 endpoint 192.0.2.3: 76 bytes MPLS payload
Seq=1, send from intf link-to-R2, reply from 192.0.2.3
      udp-data-len=32 ttl=255 rtt=5.15ms rc=3 (EgressRtr)

---- LSP color 100 endpoint 192.0.2.3 PING Statistics ----
1 packet sent, 1 packet received, 0.00% packet loss
round-trip min = 5.15ms, avg = 5.15ms, max = 5.15ms, stddev = 0.000ms
```

Figure 8-7 shows a packet capture of the MPLS Echo Request packet as it egresses router R1 towards R2. The label stack represents segments two, three, and four of the configured static SR Policy. Segment one is the Adj-SID of the interface over which R1 should forward the packet and is therefore not imposed on the label stack.

Figure 8-7: Packet capture of SR Policy LSP-Ping



Using BGP to Advertise SR Policies

BGP SR Policies can be advertised either from a centralised controller or from an SR-OS router; perhaps an egress router advertising a preferred downstream path to one or more ingress routers. This section will source the BGP SR Policy from a router to illustrate how this is achieved.

When an SR-OS router advertises SR Policies into BGP the policy must first be statically configured in order to provide the relevant information to populate the BGP path attributes. To that end, a static policy is configured at router R2 creating the same SR Policy from R1 to R3 as previously illustrated with a static SR Policy. The policy has a **head-end** of R1 (192.0.2.1), an **endpoint** of R3 (192.0.2.3), and a **color** of 100. The **binding-sid** is 20001 to fit within the range of the SRLB configured at R1, and a **distinguisher** value of 12345. If a second candidate path were to be advertised for this SR Policy a different distinguisher value would be required to avoid route suppression at the Route-Reflector. The **segment-list** contains four segments representing a path that routes R1-R2-R5-R3 and contains both strict hops with Adj-SIDs and a loose hop with a Node-SID.

Output 8-17: Configuration at R2 for R1-to-R3 SR Policy

```
router "Base" {
    segment-routing {
        sr-policies {
            static-policy "R1-to-R3-Color100" {
                admin-state enable
                color 100
                endpoint 192.0.2.3
                head-end 192.0.2.1
                binding-sid 20001
                distinguisher 12345
                segment-list 1 {
                    admin-state enable
                    segment 1 {
                        mpls-label 524272
                    }
                    segment 2 {
                        mpls-label 524278
                    }
                    segment 3 {
```

```

        mpls-label 524267
    }
    segment 4 {
        mpls-label 14003
    }
}
}
}
}
```

To advertise the static policy into BGP, two steps are required at R2. Firstly, the command **sr-policy-import true** must be configured within the global BGP context. This command instructs BGP to import all statically configured non-local SR Policies from the SR database into the BGP RIB such that they can be advertised towards BGP peers supporting the SR TE Policy Address Family. Secondly, the BGP peering with the Route-Reflector RR1 (192.0.2.10) must have the **sr-policy-ipv4 true** command enabled within the **family** context to be able to support the SR TE Address Family. (Although not shown, the **sr-policy-ipv4** address family is enabled on all routers for the RR to peer to all of its client and reflect SR TE Policies.)

Output 8-18: BGP Configuration at R2 for advertising IPv4 SR Policies

```

    bgp {
      sr-policy-import true
      group "IBGP-Core" {
        family {
          sr-policy-ipv4 true
        }
      }
      neighbor "192.0.2.10" {
        group "IBGP-Core"
      }
    }
  }
}

```

Debug 8-1 shows the Update message containing the SR Policy as received at R1. The AFI is 1 (IPv4) and the SAFI is 73 (SR Policy) which SR-OS displays as `SR_POLICY_IPV4`. The Next-Hop is R2's system address (192.0.2.2), and the NLRI contains the discriminator or `RD`, `Color`, and `Endpoint`. The Route-Target extended community is encoded as an IPv4 address specific Route-Target and has a value of `target:192.0.2.1:0`, representing R1's system address. The Tunnel-Encapsulation path attribute contains a Tunnel-Type of SR Policy and lists the Preference, Binding SID, and a Segment List.

Debug 8-1: SR Policy advertised by router R2

```
51 2025/09/09 16:50:36.582 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 132
    Flag: 0x90 Type: 14 Len: 22 Multiprotocol Reachable NLRI:
        Address Family SR_POLICY_IPV4
        NextHop len 4 NextHop 192.0.2.2
        [Len 96] RD: 12345 Color: 100 Endpoint: 192.0.2.3
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.2
    Flag: 0x80 Type: 10 Len: 4 Cluster ID:
        192.0.2.10
```



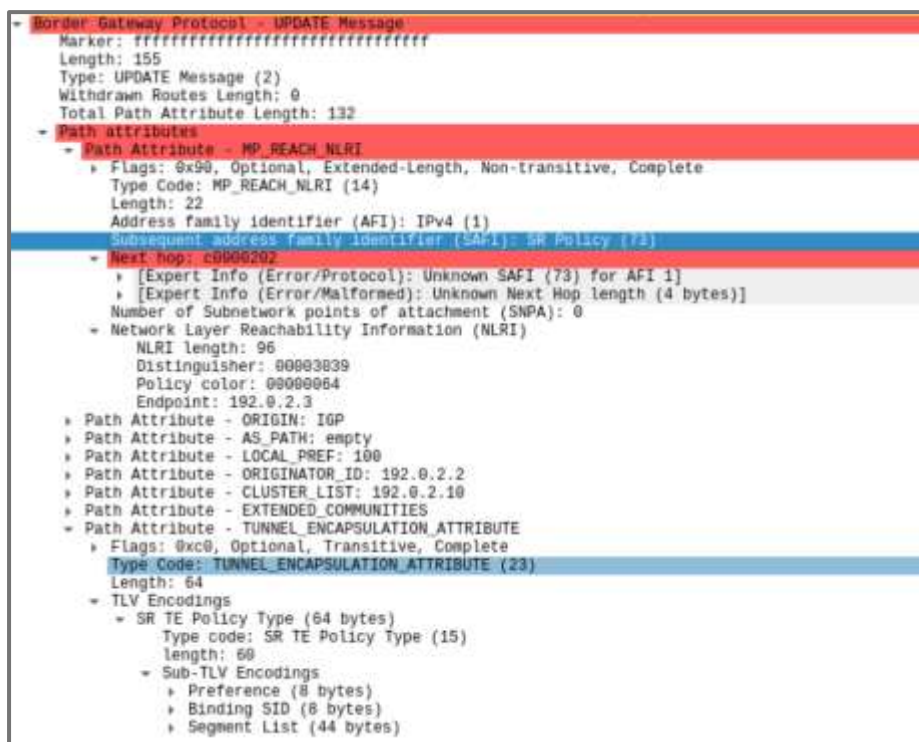
```

Flag: 0xc0 Type: 16 Len: 8 Extended Community:
  target:192.0.2.1:0
Flag: 0xc0 Type: 23 Len: 64 Tunnel-Encapsulation-attr:
  Tunnel-Encap-Type:- sr-policy
  Preference SubTlv:-
    flags: 0x0 value: 100
  Binding Sid SubTlv:-
    len: 6 flags: 0x0 Label: 20001
  Segment List 0 SubTlv:- len: 41 Weight: 1
    Segment 0 -Type 1 Flags 0x0 Label: 524272 TC:0 S:0 ttl:255
    Segment 1 -Type 1 Flags 0x0 Label: 524278 TC:0 S:0 ttl:255
    Segment 2 -Type 1 Flags 0x0 Label: 524267 TC:0 S:0 ttl:255
    Segment 3 -Type 1 Flags 0x0 Label: 14003 TC:0 S:0 ttl:255

```

Figure 8-8 shows a packet capture of the same Update message to help understand how the SR Policy update is structured, notwithstanding the fact that Wireshark has incorrectly decoded some of the NLRI. The Next Hop is displayed as c0000202, which is hex for R2's system address of 192.0.2.2, and the length of that Next Hop is 4-bytes as one would expect for an IPv4 address in dotted decimal format. The NLRI length is set to 96-bytes which is correct for AFI 1 (IPv4). When the AFI is 2 (IPv6), the NLRI length value must be 192-bytes. The Distinguisher field is displayed as 00003039 which is hex for 12345, and the Color field is displayed as 00000064 which again is hex for 100. The Tunnel Encapsulation attribute contains the preference, binding SID, and the Segment List.

Figure 8-8: Packet capture of SR Policy Update advertised by R2



When a BGP SR Policy Update is received by an SR-OS router, it is believed to be targeted at a given router as the headend if either:

- The Update has no Route-Target Extended Community and a NO-ADVERTISE standard community.

- The Update has an IPv4 address-specific Route-Target Extended Community with an IPv4 address matching the system address of this router.

When R1 receives the BGP SR Policy Update, the second bullet is applicable and as such it interprets the route as targeted towards it as the headend. The following output is taken at R1 and shows that the SR Policy is active and as the first SID in both segment lists have been correctly resolved. Note also that the owner is BGP.

Output 8-19: R1 to R3 SR Policy at R1

```
A:admin@R1# show router segment-routing sr-policies bgp end-point 192.0.2.3 color 100

=====
SR-Policies Path
=====
-----
Type           : srMpls
Active          : Yes
Operational     : Yes
Color          : 100
Head           : 0.0.0.0
RD             : 12345
BSID           : 20001
TunnelId       : 917520
Origin ASN     : 64496
NumReEval      : 0
NumActPathChange: 0
Maintenance Plcy:
Ret Path BFD Lbl:

Endpoint Addr   : 192.0.2.3
Preference      : 100
Age             : 424
Origin          : 192.0.2.2
LastReEvalReason: none
Last Change    : 09/09/2025 16:53:54

Path Segment Lists:
Segment-List    : 1
Num Segments    : 4
  1 MPLS Label  : 524272
  2 MPLS Label  : 524278
  3 MPLS Label  : 524267
  4 MPLS Label  : 14003
Weight          : 1
Last Change     : 09/04/2025 17:46:10
State           : resolved-up
State           : N/A
State           : N/A
State           : N/A
=====
```

As with the static SR policy, if the policy is considered valid, it is populated in the tunnel-table with an owner of `sr-policy`. The entry indicates the destination and color and always has a metric value of 0 regardless of how the SR Policy is instantiated. The 0 metric is used simply because there is no effective way for the headend to determine a more reflective value for an SR Policy when learnt through BGP or statically configured. The preference value assigned to SR Policy is 14 and is currently non-configurable.

Output 8-20: SR Policy tunnel-table entry at R1

```
A:admin@R1# show router tunnel-table 192.0.2.3/32 protocol sr-policy

=====
IPv4 Tunnel Table (Router: Base)
=====
Destination      Owner      Encap TunnelId  Pref  Nexthop      Metric
Color
-----
192.0.2.3/32     sr-policy MPLS  917520   14    192.168.0.2   0
100
-----
```

```

Flags: B = BGP or MPLS backup hop available
      L = Loop-Free Alternate (LFA) hop available
      E = Inactive best-external BGP route
      k = RIB-API or Forwarding Policy backup hop
=====

```

Redundant SR Policies and Seamless-BFD

As described in the last section, it is possible to have multiple candidate paths of the same SR Policy for redundancy. When candidate path redundancy is provided in this manner there can only be one active candidate path, with the other path(s) held in an inactive state. In this scenario there needs to be a mechanism to detect failure of the active path and failover to the inactive path, and that mechanism is Seamless BFD (S-BFD). S-BFD can provide a continuity check for the integrity of one or more segment lists of an MPLS or SRv6 SR Policy and provide a trigger for taking restorative action when a session fails. When using S-BFD with SR Policies there are two potential modes of operation, ECMP protected mode and linear mode. In ECMP protected mode all segment lists of the top two candidate paths of an SR Policy are programmed into the data-path. Up to 32 segment lists can be supported per candidate path, and traffic is load-balanced across all of the active segment lists. A segment list is only included if its S-BFD session is up. ECMP protected mode is best suited to networks where there is a high level of ECMP coverage. Conversely, linear mode functions as active/standby with multiple backup paths to protect a primary path. It programs one segment list from each of the top three candidate paths into the data-path, with only one as active.

BFD requires two nodes to exchange locally allocated discriminators. These discriminators enable the identification of the sender and the receiver of BFD packets over the particular session and act as a demultiplexer in the event of multiple BFD sessions running between two systems. Any SR-OS router that needs to reflect S-BFD packets requires a reflector to be enabled, and within this **reflector** context a static **discriminator** value is configured in the range 524288-526335 and should be unique throughout the domain. The initiator uses this together with a randomly generated local discriminator to create uniqueness for the purpose of demultiplexing. As router R3 will be the endpoint of the redundant SR Policy candidate paths, it will need to reflect the S-BFD packets encapsulated inside those candidate paths. The relevant reflector configuration is therefore shown in *Output 8-21*.

Output 8-21: R3 S-BFD reflector configuration

```

bfd {
    seamless-bfd {
        reflector "sbfd-reflector" {
            admin-state enable
            discriminator 524288
        }
    }
}

```



Note: If the Reflector is an FP-based platform it should use FP3 or later on all linecards used for network ports as received packets are turned around on the linecard.



Note: The 7250 IXR Gen 2/2c platforms only support hardware-based sBFD sessions, and these sessions are identified by the use of discriminators in the range [80000..82047]. This range is used exclusively to enable FP-based S-BFD sessions that support 10ms timers.

The configured S-BFD discriminator is implicitly propagated into IS-IS or OSPF as long as the **advertise-router-capability** command is set to **true** within the IS-IS or OSPF context. The S-BFD discriminator value is carried in the Router Capability TLV and has area flooding scope. Within a given area an S-BFD initiator can use the flooded information to create a dynamic peer-to-discriminator mapping. In a multi-area/multi-domain network this is not possible because the Router Capability TLV has area flooding scope; and in this case a static peer-to-discriminator mapping table must be configured. Although it is not required in the test topology (that has a flat IS-IS Level 2 area), *Output 8-22* shows the configuration of a static mapping table at R1 for peer R3 (192.0.2.3). Within the **seamless-bfd** context, this simply requires the definition of a **peer**, which can be either an IPv4 or IPv6 address, and the relevant **discriminator** value configured within that **peer** context. In an SR Policy configured to use S-BFD, the endpoint address is used to determine the relevant discriminator value from this table.

Output 8-22: Initiator Peer-to-Discriminator mapping table

```
router "Base" {
    bfd {
        seamless-bfd {
            peer 192.0.2.3 {
                discriminator 524288
            }
        }
    }
}
```



Note: If the Initiator is an FP-based platform it requires FP2 hardware or higher and CPM5 or higher.

S-BFD is applied to an SR Policy using a **maintenance-policy** configured within the **segment-routing** context and *Output 8-23* shows the **maintenance-policy** configured at the headend R1. The **maintenance-policy** defines the characteristics of the S-BFD protection applied to a given SR policy. Within that policy the **mode** command allows the user to select the desired protection mode and can be either **ecmp-protected** or **linear** as previously described. The example here uses **linear** mode. The **bfd-liveness** command when set to **true** enables BFD on all of the segment lists of the relevant candidate path(s), and when set to **false** disables BFD. The **revert-timer** is used when the primary path recovers after a failure and specifies the time period in minutes that the system waits before reverting back to that path. If no **revert-timer** is configured the system will immediately revert after primary path recovery.

Finally, the **bfd-template** command references an active/configured **bfd-template**, and in this case the template configured below in *Output 8-24* is used.

Output 8-23: SR Maintenance Policy at R1

```
router "Base" {
    segment-routing {
        maintenance-policy "sr-maint-policy" {
            admin-state enable
            bfd-liveness true
            bfd-template "sbfd-template"
            mode linear
            revert-timer 1
        }
    }
}
```

The **bfd-template** allows for configuration of the **receive-interval** and **transmit-interval** of BFD packets, specified in milliseconds. The minimum **transmit-interval** is 10 milliseconds, and an SR-OS Reflector will respond with a Required Minimum RX Interval of 3 milliseconds to indicate that this is the minimum interval of BFD Control packets that it supports. The **type** command specifies the whereabouts in the system that the BFD packets will be processed. For maximum scale it is recommended that BFD sessions are run in hardware, and this creates platform dependencies:

- On FP-based platforms this is on the P-chip of the CPM, and this is selected using the **cpm-np** argument. The **bfd-template** has no default setting for the **type** command, but the system will automatically associate multi-hop BFD sessions with the CPM P-chip. The configuration of **type cpm-np** shown in the output is simply for clarity about where the BFD packets are processed.
- For 7250-IXR platforms, the **type** command should be set to **fp** to enable the use of hardware-based BFD sessions.

Output 8-24: Initiator S-BFD template

```
bfd {
    bfd-template "sbfd-template" {
        receive-interval 5000
        transmit-interval 5000
        type cpm-np
    }
}
```

The method for associating the **maintenance-policy** with an SR Policy depends on whether the SR Policy is statically configured or learned through BGP. For a static SR policy, the **maintenance-policy** is simply configured within the **static-policy** context. For an SR policy learned through BGP the route-policy framework is used. The match, or **from** criteria within a **policy-statement** entry allows the user to select any or all of family, color, distinguisher, and endpoint to identify a given SR policy, and in *Output 8-25* only the **family** and **color** are used. The action accepts the route(s) and associates the previously configured **sr-maintenance-policy**.

Output 8-25: Associating the SR Maintenance Policy with BGP SR Policy

```

policy-options {
  policy-statement "sr-policy-sbfd" {
    entry 10 {
      from {
        family [sr-policy-ipv4]
        color 100
      }
      action {
        action-type accept
        sr-maintenance-policy "sr-maint-policy"
      }
    }
  }
}

```

It is then necessary to apply the policy as appropriate. As SR-Policies are used to build network infrastructure LSPs it is equally necessary to apply the policy at global BGP level rather than at any particular service level. In the test topology the policy is applied to the IBGP peering to RR1 over which SR Policy routes are advertised.

Output 8-26: Applying Import Route-Policy

```

router "Base" {
  bgp {
    neighbor "192.0.2.10" {
      import {
        policy ["sr-policy-sbfd"]
      }
    }
  }
}

```

To provide redundant candidate paths for the same SR Policy, routers R2 and R5 each advertise an SR Policy candidate path towards the headend router R1. Recall that an SR Policy NLRI carries the distinguisher, color and endpoint fields, with the distinguisher used to uniquely identify a path within the context of that SR Policy. BGP SR Policy uses standard BGP propagation and best-path selection, and in the test topology where a Route-Reflector is present, different discriminator values at R2 and R5 are required to avoid BGP best-path route selection and subsequent route suppression of one path. When advertising redundant candidate paths, it's worth noting that the BSID of the candidate paths should be the same in order for them to be considered part of the same SR Policy. As shown in *Output 8-27* the candidate paths for the SR Policy advertised by R2 (192.0.2.2) and R5 (192.0.2.5) have discriminator (RD) values of 12345 and 23456 respectively. Because the NLRI is formed of the tuple {color, distinguisher, endpoint} each path represents distinctly different NLRI. Consequently, both are considered valid, used, and best BGP.

Output 8-27: R1's RIB-In showing SR Policy with Multiple Candidate paths

```

A:admin@R1# show router bgp routes sr-policy-ipv4 color 100
=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,

```

```

w - unused-weight-only
Origin codes : i - IGP, e - EGP, ? - incomplete

```

BGP SR-POLICY-v4 Routes

Flag	RD/Color/End Point Nexthop (Router) As-Path	LocalPref Path-Id	MED IGP Cost Label
u*>i	12345/100/192.0.2.3 192.0.2.2 No As-Path	100 None	None N/A
u*>i	23456/100/192.0.2.3 192.0.2.5 No As-Path	100 None	None N/A

Routes : 2

Output 8-28 shows R1's BGP SR Policy candidate paths for color 100 and endpoint 192.0.2.3 (R3) and there are two paths present. The first path is the one advertised by R5 (Origin: 192.0.2.5) with path R1-R4-R5-R2-R3, and this is not active. The second path is the one advertised by R2 (Origin: 192.0.2.2) with path R1-R2-R5-R6-R3, and this path is currently active. SR Policy has a preference attribute that is used to select the best candidate path, and the higher preference wins. The route from R2 has a preference of 200, while the route from R5 has a preference of 100 therefore R2's candidate path is selected as active. It can also be seen from *Output 8-28* that the S-BFD state for both paths is up (Active S-BFD Sess. : 1). Both paths have the same BSID value.

Output 8-28: BGP SR Policy with linear protection

```
A:admin@R1# show router segment-routing sr-policies bgp end-point 192.0.2.3 color 100
```

SR-Policies Path

```

Type           : srMpls
Active         : No
Operational    : Yes
Color          : 100
Head           : 0.0.0.0
RD             : 23456
BSID           : 20001
TunnelId       : 917520
Origin ASN     : 64496
NumReEval      : 0
NumActPathChange : 0
Act S-BFD Sess. : 1
Maintenance Plcy : sr-maint-policy
Ret Path BFD Lbl :

Owner          : bgp
Endpoint Addr   : 192.0.2.3
Preference      : 100
Age             : 6328
Origin          : 192.0.2.5
LastReEvalReason : none
Last Change     : 09/09/2025 18:38:11
S-BFD Threshold : 1

```

Path Segment Lists:

Segment-List	: 1	Weight	: 1
Num Segments	: 4	Last Change	: 09/04/2025 17:46:10
1 MPLS Label	: 524254	State	: resolved-up
2 MPLS Label	: 524263	State	: N/A
3 MPLS Label	: 524280	State	: N/A
4 MPLS Label	: 14003	State	: N/A

```

Type           : srMpls
Active          : Yes
Operational     : Yes
Color          : 100
Head           : 0.0.0.0
RD             : 12345
BSID           : 20001
TunnelId       : 917520
Origin ASN     : 64496
NumReEval      : 0
NumActPathChange : 0
Act S-BFD Sess. : 1
Maintenance Plcy: sr-maint-policy
Ret Path BFD Lbl:

Path Segment Lists:
Segment-List   : 1
Num Segments   : 4
  1 MPLS Label : 524280
  2 MPLS Label : 524278
  3 MPLS Label : 524267
  4 MPLS Label : 14003
Weight         : 1
Last Change    : 09/04/2025 17:46:10
State          : resolved-up
               : N/A
               : N/A
               : N/A
Endpoint Addr  : 192.0.2.3
Preference     : 200
Age            : 6328
Origin         : 192.0.2.2
LastReEvalReason: none
Last Change    : 09/09/2025 18:33:31
S-BFD Threshold : 1

```

Output 8-29 shows the operational state of the S-BFD sessions for the SR Policy from R1 to R3 and as can be seen two S-BFD sessions are present, one for each path of the policy. The additional 'detail' argument of this command gives further information on session state, such as up time, number of transitions, as well as local and remote discriminator values.

Output 8-29: Seamless BFD spawned from the SR Policy at R1

```

A:admin@R1# show router bfd seamless-bfd session sr-policy

=====
Legend:
  Session Id = Interface Name | LSP Name | Prefix | RSVP Sess Name | Service Id
  wp = Working path  pp = Protecting path
=====
BFD Session
=====
Session Id           State      Tx Pkts  Rx Pkts
Rem Addr/Info/SdpId:VcId  Multipl  Tx Intvl Rx Intvl
Protocols             Type      LAG Port  LAG ID
Loc Addr
-----
192.0.2.3/32          Up        N/A      N/A
  192.0.2.3            3        5000    5000
    srPolicy           cpm-np    N/A      N/A
    192.0.2.1
192.0.2.3/32          Up        N/A      N/A
  192.0.2.3            3        5000    5000
    srPolicy           cpm-np    N/A      N/A
    192.0.2.1
-----
No. of BFD sessions: 2
=====

```

In steady state the active (Active: Yes) candidate path of the SR Policy advertised by R2 with distinguisher (RD) 12345 routes via R1-R2-R5-R6-R3, while the inactive path (Active: No) advertised by R5 with distinguisher 23456 routes via R1-R4-R5-R2-R3. S-BFD is active on both paths (Act S-BFD Sess. : 1).

Output 8-30: SR Policy in steady state

```

A:admin@R1# show router segment-routing sr-policies bgp end-point 192.0.2.3 color 100 |
match 'Active|Operational|Origin|RD|S-BFD Sess'
Active      : No                               Owner      : bgp
Operational : Yes
RD          : 23456                           Preference  : 100
Origin ASN  : 64496                           Origin      : 192.0.2.5
Act S-BFD Sess. : 1                          S-BFD Threshold : 1

Active      : Yes                               Owner      : bgp
Operational : Yes
RD          : 12345                           Preference  : 200
Origin ASN  : 64496                           Origin      : 192.0.2.2
Act S-BFD Sess. : 1                          S-BFD Threshold : 1

```

The link R1-R2 is disabled causing the S-BFD session on the active candidate path to fail. The operational state of the SR Policy is updated to reflect the failure immediately following detection of the S-BFD failure. The previously active candidate path from R2 (Origin: 192.0.2.2) is now inactive and there are zero active S-BFD sessions active on this path. Conversely, the previously inactive candidate path from R5 (Origin: 192.0.2.5) is now active.

Output 8-31: SR Policy after link failure

```

A:admin@R1# show router segment-routing sr-policies bgp end-point 192.0.2.3 color 100 |
match 'Active|Operational|Origin|RD|S-BFD Sess'
Active      : Yes                               Owner      : bgp
Operational : Yes
RD          : 23456                           Preference  : 100
Origin ASN  : 64496                           Origin      : 192.0.2.5
Act S-BFD Sess. : 1                          S-BFD Threshold : 1

Active      : No                               Owner      : bgp
Operational : No
RD          : 12345                           Preference  : 200
Origin ASN  : 64496                           Origin      : 192.0.2.2
Act S-BFD Sess. : 0                          S-BFD Threshold : 1

```

When the link R1-R2 is restored, the S-BFD session is re-established, and the active candidate path is reverted to the one advertised by R2 with RD 12345 and preference 200. Both S-BFD sessions are now re-established.

Output 8-32: SR Policy after link restoral

```

A:admin@R1# show router segment-routing sr-policies bgp end-point 192.0.2.3 color 100 |
match 'Active|Operational|Origin|RD|S-BFD Sess'
Active      : No                               Owner      : bgp
Operational : Yes
RD          : 23456                           Preference  : 100
Origin ASN  : 64496                           Origin      : 192.0.2.5
Act S-BFD Sess. : 1                          S-BFD Threshold : 1

Active      : Yes                               Owner      : bgp
Operational : Yes
RD          : 12345                           Preference  : 200
Origin ASN  : 64496                           Origin      : 192.0.2.2
Act S-BFD Sess. : 1                          S-BFD Threshold : 1

```


One final point of note with regard to the use of S-BFD with SR Policies. If the maximum supported number of paths has already been programmed (two for ECMP protected mode and three for linear mode) and a new path is received with a higher (better) preference value than the existing active path, this new path is not immediately installed. It is only considered for programming if/when one of the currently programmed paths is removed or withdrawn.

SRv6 Policies

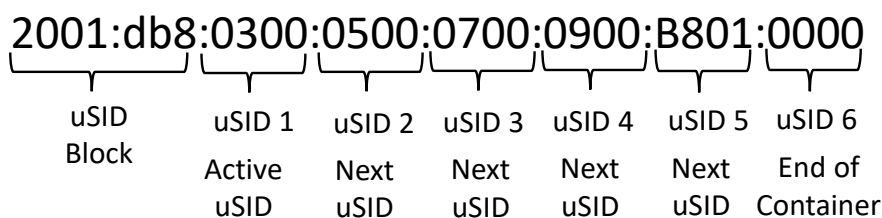


Note: SR Policies consisting of SRv6 SIDs (generic 128-bit SID or uSID) are not supported on 7250 IXR Gen 1 platforms.

So far, this section has covered the advertisement of SR Policies with SIDs consisting of MPLS labels, but it is equally possible to use SRv6 SIDs. Often these are termed SRv6 Policies, but they are in fact just SR Policies with Segment Lists made up of 128-bit SRv6 addresses instead of MPLS labels. Like their SR-MPLS counterparts, these SRv6 Policies can be configured statically, or they can be learned through BGP.

SRv6 Policies can contain paths that consist solely of micro-SIDs, or uSIDs, or they may contain a combination of uSIDs and regular 128-bit SIDs. The manner in which segment-lists containing uSIDs are instantiated, either statically configured or advertised through BGP, and the surrounding application and service support are exactly the same as SRv6 Policies with generic SRv6 SIDs. The significant difference with segment-lists containing uSIDs is that SID compression is possible, where a number of 128-bit uSIDs from the same uSID Locator block are compressed into one or more uSID containers. The uSID container is a fully-formed 128-bit SRv6 SID containing a Locator block followed by a series of uSIDs. The first uSID container is encoded in the destination address of the IPv6 header, with subsequent uSID containers being encoded in segment-lists of the SRH.

Figure 8-9: uSID Container

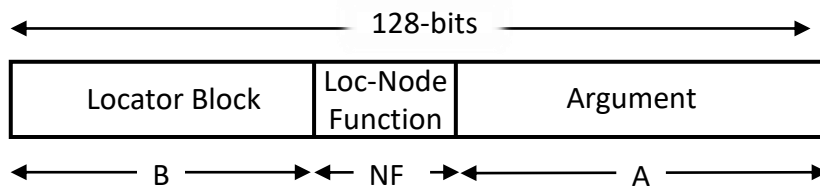


A given uSID block is sub-divided to allocate blocks to both global uSIDs and local uSIDs. The set of uSIDs available for global and local uSID allocation are referred to as the global information base (GIB) and local information base (LIB) respectively. A global uSID identifies the shortest path route to a node in the SR domain. An IP route (for example /48) is advertised by the parent node for each of its global uSIDs (analogous to an SRv6 locator), and the parent node executes a variant of the END behaviour. A node may have multiple global uSIDs allocated to it, for example to support Flex-Algorithm. Equally, multiple nodes may share the same global uSID to support anycast. A local uSID identifies an instruction that

is specific to its parent, such as a cross-connect to a specific interface and is used to identify local instructions such as adjacency-SIDs and services. If R1 and R2 are different nodes of the uSID domain and L is a local uSID value, then R1 and R2 may bind different behaviours to L. No IP route is advertised by a parent node for its local uSID.

Native SRv6 SIDs consist of a Locator, a Function, and an Argument, that allows an SRv6 router to determine what behaviour, if any other than native IPv6 forwarding, it needs to execute for a given SID. That structure does not exist for uSIDs because the Locator-Node and Function are combined in a single field as shown in *Figure 8-10*, while the Argument is used to carry the remaining C-SIDs in the current C-SID container.

Figure 8-10: uSID (NEXT-C-SID) structure



As a result of this combined Locator-Node and Function field, a uSID needs to be identifiable as belonging to either the GIB or the LIB to allow a given node to execute the correct behaviour. This is achieved by splitting the available hex range of a uSID into non-overlapping GIB and LIB spaces such that either of them are identifiable as belonging to an allocated range. As an example, assume the following:

- 48-bit uSID block length of 2001:db8:0::/48.
- 16-bit uSID length.
- A uSID 2001:db8:0:ABCD::/64 is said to be allocated from its parent block 2001:db8:0::/48. The uSID ABCD is referred to as the Locator-Node.

The Locator-Node is a 16-bit uSID containing a hex range of 0x0001-FFFF, hence there are 65,534 possible non-zero decimal values. If hex range 0x0001-DFFF is allocated to GIB, and the remaining hex range E000-FFFF to LIB, then in such a scheme the uSID block 2001:db8:0::/48 would support up to 57,343 global uSIDs (routers) and each router would support up to 8,191 local uSIDs ($57,343 + 8,191 = 65,534$). In this scheme, the uSID D123 belongs to GIB, whereas the uSID E123 belongs to LIB. The hex range can be split to suit operator requirements. It might be desirable to have a larger LIB than 8k, but it follows that less hex values are subsequently made available to GIB. For example, if hex range 0x0001-4FFF is allocated to GIB and the remaining hex range 5000-FFFF is allocated to LIB then this scheme would support up to 20,479 global uSIDs and each router would support up to 45,055 local uSIDs ($20,479 + 45,055 = 65,534$).

Although the Locator-Node and Function are combined in a single field, this does not mean that GIB and LIB share the same 16-bit uSID hextet. They do not; a uSID is either GIB or LIB and cannot be both. The splitting of the uSID range into GIB and LIB is merely to allow an SRv6 uSID router to determine how to correctly process an incoming packet. More explicitly,

when an uSID SRv6 router receives a packet with a uSID container containing two or more uSIDs, or a single uSID followed by the '0000' End-of-Container' marker and an SRH header containing a second uSID, it needs to be able to lookup not only the first uSID in the header, but also the second uSID (regardless of whether it is in the uSID container or in an SRH). If the router is a segment endpoint the first uSID will very likely belong to GIB and will be the router's uN SID. The second uSID may be a globally routable value (from GIB) or it may be a locally instantiated value (from LIB). The router determines which of those it is from the value of the **global-sid-entries** command, which determines the size of the GIB and LIB and is the same value network-wide. If the uSID comes from GIB, then the uSID container is shifted left and forwarded to the next router based on a longest prefix match in the route-table. If it comes from LIB, then it is processed in the appropriate service or BGP context.

The remainder of this section will describe advertisement of SRv6 Policies into BGP using uSIDs. As previously described, the significant advantage of using uSID for SRv6 Policies is that the uSIDs can be compressed. However, there is a set of criteria that SRv6 micro-segments of a given segment-list must meet before they can be considered eligible for compression. SR-OS uses a compression algorithm that parses through the SRv6 SIDs of a segment-list and compresses SIDs only if they meet all conditions of any of the following three algorithms:

Set 1: $B \subset [43, 50]$ AND $(BL, NL, FL, AL) == (\{8, 16, 24, 32, 40, 48, 56, 64\}, 16, 0, 128-BL-16)$ AND $ipv6[0, BL-1] \neq 0$ AND $ipv6[BL, BL+15] \neq 0$ and $ipv6[BL+16, 127] == 0$

Set 2: $B \subset [52, 59]$ OR $B \subset [85, 92]$ OR $B \subset [93, 94]$ AND $(BL, NL, FL, AL) == (\{8, 16, 24, 32, 40, 48, 56, 64\}, 16, 16, 128-BL-32)$ AND $ipv6[0, BL-1] \neq 0$ AND $ipv6[BL, BL+15] \neq 0$ AND $ipv6[BL+16, BL+31] \neq 0$ AND $ipv6[BL+32, 127] == 0$

Set 3: $B \subset [52, 59]$ OR $B \subset [85, 92]$ OR $B \subset [93, 94]$ AND $(BL, NL, FL, AL) == (\{8, 16, 24, 32, 40, 48, 56, 64\}, 0, 16, 128-BL-15)$ AND $ipv6[0, BL-1] \neq 0$ AND $ipv6[BL, BL+15] \neq 0$ AND $ipv6[BL+16, BL+127] == 0$

At first glance the compression algorithm might appear somewhat daunting, but it is easier to understand when broken into sub-sections. Using set 2 as an example, there are three sub-sections shown below. Sub-section a) covers the allowable endpoint behaviour, with a logical AND for b) which covers the allowable components of the SID structure, again with a logical AND for c) which covers the acceptable IPv6 address formats:

a) $B \subset [52, 59]$ OR $B \subset [85, 92]$ OR $B \subset [93, 94]$

- The endpoint Behaviour B must be a subset of one of the flavours listed between square brackets. The full list of supported flavours that can potentially be compressed are listed in *Table 8-1* and the ones covered above allow the various flavours of End.X with NEXT-CSID.

b) $AND (BL, NL, FL, AL) == (\{8, 16, 24, 32, 40, 48, 56, 64\}, 16, 16, 128-BL-32)$

- The Block Length (BL) must be one of the values in curly brackets, whilst the Node-Length (NL) and the Function-Length (FL) must each be 16-bits. Finally, the Argument-Length (AL) must be the length of 128-bits minus the Block Length minus 32-bits (the 32-bits representing the combined NL and FL).

- c) $AND\ ipv6[0, BL-1] \neq 0\ AND\ ipv6[BL, BL+15] \neq 0\ AND\ ipv6[BL+16, BL+31] \neq 0\ AND\ ipv6[BL+32, 127] == 0$
- The IPv6 address (SID) must have a non-zero value between 0-bits and BL-1, between bits BL and BL+15, and between bits BL+16 and BL+31. The IPv6 address must have a zero value between bits BL+32 and 127-bits.

Table 8-1: Supported SRv6 Endpoint Behaviours

Value	Description
43	End with NEXT-CSID
44	End with NEXT-CSID and PSP
45	End with NEXT-CSID and USP
46	End with NEXT-CSID, PSP and USP
47	End with NEXT-CSID and USD
48	End with NEXT-CSID, PSP and USD
49	End with NEXT-CSID, USP and USD
50	End with NEXT-CSID, PSP, USP and USD
52	End.X with NEXT-CSID
53	End.X with NEXT-CSID and PSP
54	End.X with NEXT-CSID and USP
55	End.X with NEXT-CSID, PSP and USP
56	End.X with NEXT-CSID and USD
57	End.X with NEXT-CSID, PSP and USD
58	End.X with NEXT-CSID, USP and USD
59	End.X with NEXT-CSID, PSP, USP and USD
85-92	End.T with NEXT-CSID and decap flavours
93	End.B6.Encaps with NEXT-CSID
94	End.B6.Encaps.Red with NEXT-CSID

A static SRv6 Policy is configured at router R2 defining a path from R1 to R3 along the path R1-R4-R5-R2-R3 and the output of the configured SR Policy is shown in *Output 8-33*. The intention is to advertise the policy to router R1, and as previously described, when advertising IPv6 SR Policy Address Family the test topology uses IPv6 IBGP peering. This is simply because when attempting to resolve a VPN or IPv4/IPv6 prefix to an SRv6 Policy it must correlate between the IPv6 endpoint of the SRv6 Policy, and the Next-Hop of advertised VPN and/or IPv4/IPv6 prefixes. The configuration of the policy begins with the attributes such as **color**, **distinguisher**, and **type**, which have been previously described so are not repeated here. The **endpoint** of the policy is R3's system address 3fff:ffe::3 and this is the

same address that R3 uses as a Next-Hop when advertising VPN and IPv4/IPv6 prefixes, hence the SR Policy will be able to resolve to that Next-Hop address. The **segment-routing-v6** context provides the context to configure the **binding-sid**. The **binding-sid** parameter is followed by an **index**, which is set to 1 as SR-OS currently only supports a single binding SID per SRv6 Policy. The **binding-sid** can be entered in one of two formats, using either **locator** or **ip-address**, depending on the value of the **headend** command:

- If the headend is **local**, the **binding-sid** is entered as a **locator** that is configured on the system, and requires a **locator-name** and a **function**, which can currently only be **end-b6-encaps-red**. The **locator** context has an optional **function-value** parameter where the function value can be statically defined. If no **function-value** is configured, the system automatically allocates one.
- If the headend is a non-local IP address, such as a policy that is being advertised to another router, the **binding-sid** must be entered as an **ip-address**, where that address is a 128-bit IPv6 address. The configured **ip-address** must belong to a valid locator on the remote system and must have a function value that is lower than the **max-entries** value for that locator and that has not been previously allocated. When using uSIDs, it's important to remember that the **max-entries** command starts at the beginning of the LIB. The size of the GIB and LIB is determined by the value of the **global-sid-entries** command, which in this example is set to 56 (not shown). As a result, the beginning of the LIB is 0xe000.

The **binding-sid ip-address** of 3fff:3ea:1:e009:: in *Output 8-33* can be broken down as follows:

- 3fff:3ea: is the 32-bit locator block for uSID Algorithm 0
- 1: is the Locator-node for router R1
- e009:: is a value taken from LIB, where LIB starts at 0xe000 and **static-function** has **max-entries** set to 16. Hence, the value e009 sits between e000 and e000+16.

The **segment-list** context provides the ability to construct a path consisting of micro-SIDs within each **segment**. FP-based platforms limit the SID depth of the SRH to a maximum of 24 uSID segments in the SRv6 Policy candidate path (not accounting for any additional LFA SIDs which are encoded in a separate SRH using H.Insert.Red encapsulation). Hence up to 24 **segments** consisting of micro-SIDs can be configured within each **segment-list**. The 7250 IXR platforms can push up to three containers, hence the number of segments that can be supported is determined by the number of uSIDs that can fit into each container. If a container can support five uSIDs for example, then the limit of segments that can be supported is 15. Both the FP-based limit and the 7250-IXR limit are not inclusive of the service SID. Within the **segment** context the **srv6-sid** is entered as a conventional IPv6 address, after which the **behavior-and-structure** context is used to configure the endpoint behaviour and structure of the uSID. When compressed SIDs are used, the first uSID following the Locator block encodes the Locator-Node/Function while the Argument carries the subsequent uSID's. This common Locator-Node/Function field dictates that only uSIDs of the same behaviour and structure can be compressed, and as a result the behaviour and structure of each uSID in the **segment-list** needs to be made known to the system such that

they can be evaluated for potential compression. This example uses a path consisting of uN (End with NEXT-CSID) SIDs, and as such the **behavior** command is configured as **end-next-csid**. The **block-length** is set to **32** to reflect the Locator block length. As the C-SID structure has a Locator block followed by the combined Locator-Node/Function field, the **node-length** is set to **16** and the **function-length** is set to **0** as the Locator-Node/Function field is a single 16-bit field that comes from GIB. The Argument length is non-configurable and is always set to 128-BL-NL-FL. The example **segment-list** consists of three segments, each containing an SRv6 uSID with an End with NEXT-CSID behaviour:

- The first **segment** has **srv6-sid** 3fff:3ea:4:: representing R4's uN SID for Algorithm 0
- The second **segment** has **srv6-sid** 3fff:3ea:5:: representing R5's uN SID for Algorithm 0
- The third and final segment has **srv6-sid** 3fff:3ea:2:: representing R2's uN SID for Algorithm 0

Output 8-33: Static SRv6 Policy configuration at R2

```
router "Base" {
  segment-routing {
    sr-policies {
      static-policy "R1-to-R3-uSID-uN" {
        admin-state enable
        color 200
        endpoint 3fff:ffe::3
        head-end 192.0.2.1
        distinguisher 34567
        type srv6
        segment-routing-v6 {
          binding-sid 1 {
            ip-address 3fff:3ea:1:e009::
          }
        }
      }
      segment-list 1 {
        admin-state enable
        segment 1 {
          srv6-sid 3fff:3ea:4::
          behavior-and-structure {
            behavior end-next-csid
            block-length 32
            node-length 16
            function-length 0
          }
        }
        segment 2 {
          srv6-sid 3fff:3ea:5::
          behavior-and-structure {
            behavior end-next-csid
            block-length 32
            node-length 16
            function-length 0
          }
        }
        segment 3 {
          srv6-sid 3fff:3ea:2::
          behavior-and-structure {
            behavior end-next-csid
            block-length 32
            node-length 16
            function-length 0
          }
        }
      }
    }
  }
}
```

```

    }
  }
}

```

To advertise the static policy into BGP, two steps are required at R2. Firstly, the command **sr-policy-import true** must be configured within the global BGP context (which was previously configured for advertising the IPv4 SR Policy). Secondly, the BGP peering with the Route-Reflector RR1 (3fff:ffe:a::) must have the **sr-policy-ipv6 true** command enabled within the **family** context to be able to support the SR TE Address Family. (Although not shown, the **sr-policy-ipv6** address family is enabled on all routers for the RR to peer to all of its client and reflect SR TE Policies.)

Output 8-34: R2's BGP configuration for advertising IPv6 SR Policies

```

bgp {
  sr-policy-import true
  group "IPv6-IBGP-Core" {
    family {
      sr-policy-ipv6 true
    }
  }
  neighbor "3fff:ffe::a" {
    group "IPv6-IBGP-Core"
  }
}

```

Debug 8-2 shows the Update message containing the IPv6 SR Policy as received at R1. The Address Family is displayed as SR_POLICY_IPV6, while the NLRI shows a Next-Hop of R2 (3fff:ffe::2), an RD of 34567, a Color, and an Endpoint of R3's system address 3fff:ffe::3. As with the IPv4 SR Policy advertisement, the Route-Target extended community is encoded as an IPv4 address specific Route-Target and has a value of target:192.0.2.1:0, representing R1's system address. The Tunnel-Encapsulation path attribute contains a Tunnel-Type of SR Policy and lists the Preference, Binding SID encoding an IPv6 address, and a Segment List containing three segments.

Debug 8-2: SR Policy IPv6 route from R2 received at R1

```

114 2025/09/10 15:35:35.582 BST minor: DEBUG #2001 Base Peer 1: 3fff:ffe::a
Peer 1: 3fff:ffe::a: UPDATE
Peer 1: 3fff:ffe::a - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 228
  Flag: 0x90 Type: 14 Len: 46 Multiprotocol Reachable NLRI:
    Address Family SR_POLICY_IPV6
    NextHop len 16 Global NextHop 3fff:ffe::2
    [Len 192] RD: 34567 Color: 200 Endpoint: 3fff:ffe::3
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 0 AS Path:
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
  Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.2
  Flag: 0x80 Type: 10 Len: 4 Cluster ID:
    192.0.2.10
  Flag: 0xc0 Type: 16 Len: 8 Extended Community:
    target:192.0.2.1:0
  Flag: 0xc0 Type: 23 Len: 136 Tunnel-Encapsulation-attr:
    Tunnel-Encap-Type:- sr-policy

```



```

Preference SubTlv:-
  flags: 0x0  value: 100
Binding Sid SubTlv:-
  len: 6  flags: 0x0  Label: 0
SRv6 Binding Sid SubTlv:-
  len: 18  flags: 0x0  SRv6 Sid: 3fff:3ea:1:e009::
Segment List 0 SubTlv:- len: 93 Weight: 1
  Segment 0 -Type 13 Flags 0x10 SID: 3fff:3ea:4::
  Segment 1 -Type 13 Flags 0x10 SID: 3fff:3ea:5::
  Segment 2 -Type 13 Flags 0x10 SID: 3fff:3ea:2::

```

Output 8-35 shows the operational state of the static SRv6 Policy. The `active` field shows whether this path is the selected path when multiple candidate paths are in use. The SRv6 Policy is considered valid if the first SRv6 SID in the segment list can be resolved, and as the first SID `3fff:3ea:4::` is `resolved-up` the SRv6 Policy is considered valid.

Output 8-35: Operational status of SRv6 Policy at R1

```

A:admin@R1# show router segment-routing sr-policies bgp end-point 3fff:ffe::3 color 200

=====
SR-Policies Path
=====
-----
Type           : srv6
Active          : Yes                               Owner          : bgp
Operational     : Yes
Color           : 200
Head            : 0.0.0.0                           Endpoint Addr   : 3fff:ffe::3
RD              : 34567                             Preference     : 100
SRv6 BSID 1     : 3fff:3ea:e009::
TunnelId        : 917526                             Age             : 532
Origin ASN      : 64496                             Origin          : 3fff:ffe::2
NumReEval       : 0                                LastReEvalReason: none
NumActPathChange: 0                                Last Change     : 09/10/2025 15:35:36
Maintenance Plcy:
Ret Path BFD SID:

Path Segment Lists:
Segment-List    : 1                                Weight          : 1
Num Segments    : 3                                Last Change     : 09/04/2025 17:46:10
  1 SRv6 SID    : 3fff:3ea:4::                      State : resolved-up
    Behavior & Structure
      Behavior   : 43                                Block Length    : 32
      Node Length: 16                                Function Length  : 0
  2 SRv6 SID    : 3fff:3ea:5::                      State : N/A
    Behavior & Structure
      Behavior   : 43                                Block Length    : 32
      Node Length: 16                                Function Length  : 0
  3 SRv6 SID    : 3fff:3ea:2::                      State : N/A
    Behavior & Structure
      Behavior   : 43                                Block Length    : 32
      Node Length: 16                                Function Length  : 0
=====

```

The following output provides the user with a view of how the destination address of the IPv6 header (and if necessary SRH) is constructed, and therefore a view of whether compression took place or not. The `Destination` field is not to be confused with the destination address of the IPv6 header, and displays the endpoint of the SRv6 Policy, which is R3's IPv6 system address, `3fff:ffe::3/128`. The `Lbl/SID` field contains the IPv6 destination address the system will insert in the IPv6 header, and consequently it is possible to determine whether

compression took place. In this case the SID is 3fff:3ea:4:5:2:: which encodes the Locator block 3fff:3ea::/32, followed by the uN SIDs for R4, R5, and R2 respectively. Although not shown, the remaining 32-bits of the destination address are padded with zeros to indicate the reserved '0000' for End-of-Container. The SRv6 Policy containing segments with uN behaviour matched all of the criteria of Set 1 of the compression algorithm, and as a result the uN SIDs for R4, R5, and R2 are compressed into a single uSID container that is placed in the destination address of the IPv6 header. As an SRv6 Policy is always subject to FPE processing, the `NextHop` shows an internal IP address with a forwarding interface (`Intf`) of pxc-1.b.1.

Output 8-36: FP-Tunnel-Table output for SRv6 Policy with uN SIDs

```
A:admin@R1# show router fp-tunnel-table 1 ipv6 protocol srv6-policy

=====
IPv6 Tunnel Table Display

Legend:
label stack is ordered from bottom-most to top-most
B - FRR Backup
=====
Destination                                Protocol      Tunnel-ID
  Lbl/SID
  NextHop                                Intf/Tunnel
  Lbl/SID (backup)
  NextHop      (backup)
-----
3fff:ffe::3/128                            SRV6-Policy   -
  3fff:3ea:4:5:2::
  0.140.1.1                                pxc-1.b:1
-----
Total Entries : 1
-----
```

Using SR Policies for Services and IPv4/IPv6 BGP

The Color [40] attribute is a fundamental part of steering traffic into an SR policy. Color is a transitive extended community that can be attached to a given BGP Update message in order to associate itself with a corresponding policy. A headend can potentially steer traffic into an SR policy as a midpoint (or BSID anchor) or as an ingress router using color-based traffic steering:

- At a midpoint or BSID anchor, if an incoming packet has an active label that matches the BSID of a valid policy, the incoming label is swapped for the label(s) contained in the active path of that policy and traffic is forwarded along that path.
- At an ingress router, if a BGP or service route is received containing a Color extended community with a value corresponding to a valid local SR policy, and the endpoint of that policy matches the Next-Hop of the BGP/service route, traffic is forwarded into the associated policy.

The Color extended community has two flags known as the Color-Only (CO) bits that allow for a headend to optionally steer traffic into an SR policy without the need to explicitly define a policy endpoint that matches the next-hop of a BGP or service route. In this case the

endpoint can be the null address (0.0.0.0 for IPv4 and ::0 for IPv6) and traffic is steered into a policy based purely on correlation of color. Color-only traffic steering using a null endpoint SR Policy is limited only to unlabeled BGP address families. The reason for this is simply that when an egress router advertises a downstream label in a labeled BGP Update (i.e., VPN-IPv4/IPv6, EVPN, BGP Labeled Unicast etc) that egress router needs to see that same label in received packets in order to be able to demultiplex into the relevant service/Next-Hop and forward traffic towards the destination. If a headend is forwarding traffic into an SR Policy with a null endpoint, that headend is unaware of the egress router and therefore cannot impose the relevant downstream-advertised BGP/service label into the label stack. For that reason, Color-Only traffic steering can only be applied to unlabelled BGP Updates. If the CO bits are set to 00 it indicates that if there is a valid SR Policy (N, C) where N is the IPv4 or IPv6 endpoint address and C is a color, steer into SR policy (N, C). This is referred to as per-destination traffic steering and is used throughout this section.

The topology in *Figure 8-6* is used to illustrate traffic steering using SR and SRv6 Policies. When the former is used, IBGP peering uses IPv4 transport. When the latter is used, IBGP peering uses IPv6 transport.

Traffic Steering with SR Policy

When incoming packets match a BGP/service route with a Next-Hop that resolves to an SR Policy, it is referred to as per-destination traffic steering. The static SR Policy from R1 to R3 configured in *Output 8-13* with Color 100 is used to illustrate how it is applied.

VPRN 'one' is configured on routers R1 and R3 with import/export Route-Target 64496:1 to connect the CE routers. Since there is currently only a single SR Policy to R3 (endpoint 192.0.2.3) this example shows how it is used by R1 to resolve the prefix 172.31.3.0/24 advertised by CE3 and consequently R3.

Output 8-37 shows the **auto-bind-tunnel** at R1 with the **resolution-filter** set to **sr-policy true**. The remainder of the VPRN parameters are not shown for conciseness.

Output 8-37: Auto-Bind-Tunnel Resolution-Filter at R1

```

service {
  vprn "one" {
    bgp-ipvpn {
      mpls {
        auto-bind-tunnel {
          resolution filter
          resolution-filter {
            sr-policy true
          }
        }
      }
    }
  }
}

```

CE3 is locally connected to R3 and advertises prefix 172.31.3.0/24 into IPv4 BGP, which R3 subsequently advertises as a VPN-IPv4 route. In addition to attaching the Route-Target

extended community to the VPN-IPv4 route, R3 also attaches a Color extended community with color value 100 to match that of the SR Policy at R1. R3's VRF export policy is shown in *Output 8-38*. Note that when configuring the Color extended community the syntax "*color:co.value*" is used. Therefore, in the example configuration the CO bits are 00 and the color value is 100.

Output 8-38: VRF Export Policy at R3

```

policy-options {
  community "color-100" {
    member "color:00:100" { }
  }
  community "vrf-one" {
    member "target:64496:1" { }
  }
  policy-statement "vrf-one-export" {
    entry 10 {
      from {
        protocol {
          name [bgp]
        }
      }
      to {
        protocol {
          name [bgp-vpn]
        }
      }
      action {
        action-type accept
        community {
          add ["vrf-one" "color-100"]
        }
      }
    }
  }
}

```

At R1 the VPN-IPv4 route with {next-hop R3, color extended community 100} resolves to R1's static SR policy with {endpoint R3, color 100}. The VPN-IPv4 route is therefore imported into the VPRN route-table with an indication that it is resolved to the SR Policy with tunnel ID 917527, which is the tunnel ID of the static SR policy. Traffic from R1 to VPN-IPv4 prefix 172.31.3.0/24 will therefore be forwarded into the SR Policy using the label stack defined in the segment list.

Output 8-39: VPRN "one" route-table for prefix 172.31.3.0/24 at R1

```
A:admin@R1# show router 1 route-table 172.31.3.0/24
```

```

=====
Route Table (Service: 1)
=====
Dest Prefix[Flags]                                Type    Proto    Age           Pref
Next Hop[Interface Name]                          Metric
-----
172.31.3.0/24                                     Remote  BGP VPN    00h13m19s    170
192.0.2.3 (tunneled:SR-Policy:917527)             0
-----
No. of Routes: 1
Flags: n = Number of times nexthop is repeated
      B = BGP backup route available
      L = LFA nexthop available

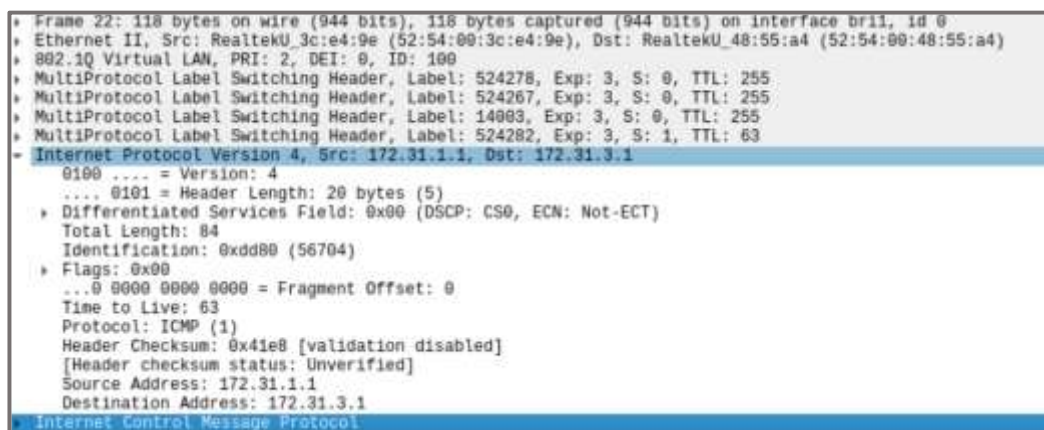
```

S = Sticky ECMP requested

To verify the connectivity a ping is issued from CE1 (172.31.1.1) to CE3 (172.31.3.1) that is successful. *Figure 8-11* shows a packet capture of the Echo Request from CE1 as it egresses router R1 on the interface towards R4. From top to bottom the label stack is as follows:

- Label 524278 represents R4's Adj-SID for the link R4-R5
- Label 524267 represents R5's Adj-SID for the link R5-R2
- Label 14003 represents R3's SR-MPLS Node-SID
- Label 524282 is the service label advertised with the VPN-IPv4 prefix 172.31.3.0/24 by router R3.

Figure 8-11: Packet capture of Echo Request from R1 to R3 using SR Policy



Traffic Steering with SRv6 Policy

The SRv6 Policy from R1 to R3 configured at router R2 (see *Output 8-33*) and advertised in BGP to R1 with Color 200 and Endpoint 3fff:ffe::3 is used to illustrate how SRv6 Policies are applied to services.

Regardless of whether SRv6 Policies are statically configured or learned in BGP, or whether they use generic SRv6 SIDs or uSID's, once they are programmed in the IPv6 tunnel-table they can be used by VPRN, EVPN-VPLS, EVPN-VPWS, and EVPN-IFL services, as well as BGP shortcuts. The example in this section uses a VPRN service to illustrate how SRv6 Policies are used for services, but the procedure is fundamentally the same or similar for the other listed services.

To enable the VPRN service between R1 and R3 to use SRv6 Policies as transport, the VPRN requires service level configuration within the **segment-routing-v6** <index> context specifying the generic SRv6 or uSID SRv6 Locators that can be referenced by the **default-locator** contained in the **bgp-ipvpn segment-routing-v6** context. This example uses a **micro-segment-locator** so refers to the use of uSIDs, with **function udt4** and **udt6** to specify the Endpoint behaviour of Layer 3 lookup in a VRF. More than one **locator** or **micro-segment-locator** can be configured at service level, but only one of those configured can be referenced by the **default-locator**. A **resolution** command is used to select whether the service should

use SRv6 or SRv6 Policy to resolve BGP service routes. The **tunnel-table** option means that the system will try to resolve BGP routes containing an SRv6 service SID to an SRv6 Policy with a matching color and an endpoint that corresponds to the received BGP Next-Hop. The **route-table** option means that the system will try to resolve the received SRv6 service SID in the BGP route to an SRv6 Locator in the route-table. The **fallback-tunnel-to-route-table** option does what it says and means that the system will first try resolving the route to an SRv6 Policy in the tunnel-table, and if none is found it will fall back to resolving the route to an SRv6 Locator in the route-table. It's worth emphasising that when selecting one of the **tunnel-table/route-table** options that the system is not just switching between SRv6 or SRv6 Policy transport, it is also switching the way in which a received route is resolved. SRv6 uses the received SRv6 service SID and looks in the route-table for a matching Locator. SRv6 Policy uses the BGP Next-Hop and tries to find a policy with a matching endpoint and color.

Output 8-40: VPRN configuration for SRv6 Policy at R1

```

service {
    vprn "one" {
        segment-routing-v6 1 {
            micro-segment-locator "uSID-Alg0-Locator" {
                function {
                    udt4 {
                    }
                    udt6 {
                    }
                }
            }
        }
    }
    bgp-ipvprn {
        segment-routing-v6 1 {
            resolution tunnel-table
            srv6 {
                instance 1
                default-locator "uSID-Alg0-Locator"
            }
        }
    }
}

```

As the example configuration at R1 uses a **resolution of tunnel-table** the system will look in the tunnel-table for a matching SRv6 Policy and attempt to resolve routes with a matching color and an endpoint that corresponds to the received BGP Next-Hop. VPN-IPv4 and VPN-IPv6 routes advertised between R1 and R3 are appended with the extended community Color of 00:200. *Output 8-41* shows R1's VRF route-table containing the prefix 172.31.3.0/24 advertised by CE3 (and subsequently R3). The prefix is known through BGP VPN and has a Next-Hop of R3's system address 3fff:ffe::3. As the prefix has a color of 200, it matches the SRv6 Policy to R3 and is shown as `tunneled:SRv6-Policy:917531`, where 917531 correlates with the tunnel-ID for the SRv6 Policy.

Output 8-41: VPRN route-table at R1 with SRv6 Policy

```
A:admin@R1# show router 1 route-table 172.31.3.0/24
```

```

=====
Route Table (Service: 1)
=====

```

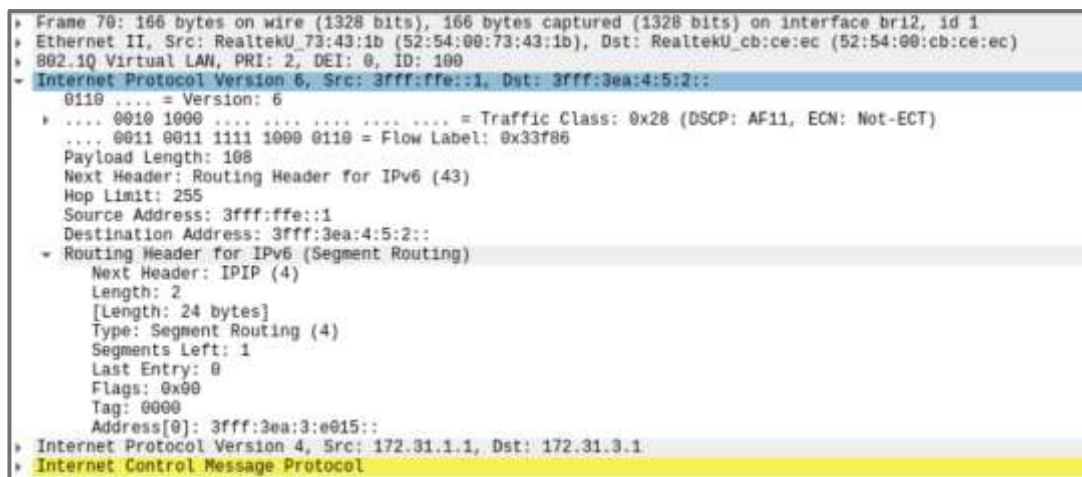
Dest Prefix[Flags]	Type	Proto	Age	Pref
Next Hop[Interface Name]	Metric			

172.31.3.0/24	Remote	BGP VPN	00h01m08s	170
3fff:ffe::3 (tunneled:SRV6-Policy:917531)			0	

No. of Routes: 1				
Flags: n = Number of times nexthop is repeated				
B = BGP backup route available				
L = LFA nexthop available				
S = Sticky ECMP requested				
=====				

Once again, to verify the connectivity a ping is issued from CE1 (172.31.1.1) to CE3 (172.31.3.1) that is successful. *Figure 8-12* shows a packet capture of the Echo Request from CE1 as it egresses router R1 on the interface towards R4 using the SRv6 Policy. The destination address has the compressed uSID of 3ffe:3ea:4:5:2:: representing the uSID Algorithm 0 block 3fff:3ea::/32 followed by the uSID uN values for routers R4, R5, and R2 respectively. The packet contains a Routing Header of type Segment Routing that contains a single segment containing the SRv6 Service SID that R3 advertised with the VPN-IPv4 prefix 172.31.3.0/24.

Figure 8-12: Packet capture of Echo Request from R1 to R3 using SRv6 Policy



SRv6 in BGP Overlay Services

When using an MPLS (or SR-MPLS) core, BGP-based services such as global IPv4/IPv6, IPv4/IPv6-VPN, and EVPN advertise their relevant reachability and associate an MPLS label with those NLRI. This enables Layer 2 and Layer 3 overlay services with BGP as a control plane and MPLS as a data plane. When using an SRv6 domain, BGP-based services are extended to advertise SRv6 Service SIDs [41] in order to provide Layer 2 and Layer 3 overlay services with a BGP control plane and an SRv6 data plane. SRv6 Service SIDs refer to an SRv6 SID associated with one of the service-specific endpoint behaviours on the advertising PE router, such as End.DT4 (IPv4-L3VPN) or End.DT2U (EVPN Bridging Unicast).

A BGP Prefix Segment is a BGP prefix with a Prefix-SID [35] attached and is carried in Multi-Protocol BGP. An optional transitive BGP attribute known as the BGP Prefix-SID attribute can be associated with a BGP Prefix-SID, and it is this attribute that is extended to carry SRv6 Service TLVs. There are two new TLVs of the BGP Prefix-SID attribute used to signal SRv6 SIDs for Layer 2 and Layer 3 overlay services:

- The SRv6 L3 Service TLV. This TLV carries Service SID information for SRv6-based Layer 3 services and corresponds to the equivalent of an MPLS label carried in VPN-IPv4/VPN-IPv6 service routes. Example endpoint behaviours include but are not limited to End.DX4, End.DT4, End.DX6, and End.DT6.
- The SRv6 L2 Service TLV. This TLV carries Service SID information for SRv6-based Layer 2 services and corresponds to the equivalent of MPLS Label1 for EVPN service routes. Example endpoint behaviours include but are not limited to End.DX2, End.DX2V, and End.DT2U.

SRv6 service-related information is structured in a hierarchical way consisting of SRv6 Service TLVs, which carry SRv6 Service sub-TLVs, which in turn carry SRv6 Service Data sub-sub-TLVs. Starting from the top of the hierarchy, the structure of the SRv6 Service TLV is shown in *Figure 8-13* and is encoded in the BGP Prefix-SID attribute. It is a simple TLV format that acts as a holding or umbrella structure, with the option to carry SRv6 Service sub-TLVs with variable length.

Figure 8-13: SRv6 Service TLV

SRv6 Service TLV Type	SRv6 Service TLV Length	SRv6 Service TLV Value
// SRv6 Service Sub-TLVs //		

The format of the SRv6 Service sub-TLVs carried in the SRv6 Service TLV is again a simple TLV structure, and SRv6 Service Sub-TLV Type 1 is assigned to the SRv6 SID Information sub-TLV. Its encoding is shown in *Figure 8-14* and it is used to signal the SRv6 SID value assigned to the NLRI together with the endpoint behaviour associated with that SID value. It also has a variable field for encoding the next level of SRv6 service information, the SRv6 Service Data sub-sub-TLVs.

Figure 8-14: SRv6 SID Information Sub-TLV

SRv6 Service Sub-TLV Type	SRv6 Service Sub-TLV Length	Reserved1
// SRv6 SID Value (16 bytes) //		
SID Flags	Endpoint Behavior	Reserved2
// SRv6 Service Data Sub-Sub TLVs //		

The SRv6 Service sub-TLV above carries SRv6 Service Data sub-sub-TLVs, and sub-sub-TLV Type 1 is assigned to the SRv6 SID Structure sub-sub TLV. Its encoding is shown in *Figure 8-15* and as the name suggests it is used to provide information about the SID encoding. The Locator Block Length, Locator Node Length, Function Length, and Argument Length fields together provide information as to exactly how the 128-bit SID is structured. The Transposition Length and Transposition Offset fields require some explanation. Some types of BGP services such as IP-VPN or EVPN use a per-SID allocation (analogous to label-per-VRF) where a single SID is shared across multiple NLRIs, and this provides efficient packing of NLRI into BGP updates. Other types of BGP services like EVPN-VPWS use a per-pseudowire allocation where each NLRI has its own unique SID, resulting in inefficient NLRI packing. To allow for efficient NLRI packing, the SRv6 SID value contained in the SRv6 SID Information Sub-TLV may contain just the common part of the SRv6 SID (the Locator) while the variable parts of the SRv6 SID (Function and Argument) are encoded into the existing MPLS label fields. Because the rest of the SID is common for routes of the same type in the service, multiple routes can be packed into the same BGP Update. This type of encoding is referred to as *Transposition*, and the SRv6 SID Structure sub-sub-TLV describes the sizes of the Locator block, Locator Node, Function, and Argument, as well as indicating the offset and length of the transposition.

Figure 8-15: SRv6 SID Structure Sub-Sub-TLV

SRv6 Service Data Sub-Sub-TLV Type		SRv6 Service Sub-Sub-TLV Length		Locator Block Length	
Locator Node Length		Function Length	Argument Length	Transposition Length	
Transposition Offset					

For service routes, SR-OS transposes up to 20-bits of the SRv6 SID Function value into the Label field of the NLRI, which is bounded by the size of the MPLS label field. As the function length is configurable as either 16-bits or between 20-96 bits, it follows that the Transposition Length is therefore either 16-bits or 20-bits, and the Transposition Offset identifies the point in the SID where the transposed value starts. For example, if the Locator {Block + Node} length is 64-bits, and the Function length is 20-bits, then the transposition offset is 64-bits. If the Locator {Block + Node} length is 64-bits and the Function Length is 32-bits, only 20-bits of the Function are transposed as the right-most bits of the Label field, and the transposition offset is 76-bits. This offset represents the 64-bit Locator and 12-bits of the Function field that are not transposed. If there is no transposition, SR-OS expects an Implicit Null label value in the label field.

To help clarify this transposition behaviour, *Output 8-42* shows a VPN-IPv4 service route that is part of a VPRN overlay service on an SRv6 core . The output is truncated to contain only

the relevant parts, but the SRv6 Service TLV, SRv6 SID Information sub-TLV, and SRv6 SID Structure sub-sub-TLV are all shown. The `Sid` field in the SRv6 SID Information shows the advertised Locator prefix `3fff:3ea:1::`, which according to the SRv6 SID Structure sub-sub-TLV is formed of a 32-bit Locator Block (`Loc-Block-Len`) and 16-bit Locator Node (`Loc-Node-Len`). The Function follows the Locator, and as shown in the `Func-Len` field it is 20-bits in length. This 20-bits is shown in the `Full Sid` field as being hex `7fff:d`. This is the value that is transposed into the NLRI MPLS label and is shown in the `VPN Label` as the decimal equivalent 524285. The resulting transposition offset (the point that marks the beginning of the transposed value) is shown in the `Tpose-offset` field as 64-bits.

Output 8-42: VPN-IPv4 Prefix with SRv6 Service TLV

```
A:admin@R1# show router bgp routes 172.31.1.0/24 vpn-ipv4 hunt

--- [snip] ---
-----
RIB Out Entries
-----
Network       : 172.31.1.0/24
NextHop       : 192.0.2.1
Route Dist.    : 64496:1          VPN Label       : 524285
Path Id       : None
To            : 192.0.2.10
--- [snip] ---
Neighbor-AS   : 64512
DB Orig Val   : N/A              Final Orig Val : N/A
Source Class  : 0                Dest Class    : 0
SRv6 TLV Type : SRv6 L3 Service TLV (5)
SRv6 SubTLV   : SRv6 SID Information (1)
Sid           : 3fff:3ea:1::
Full Sid      : 3fff:3ea:1:7fff:d000::
Behavior      : End.DT46 (20)
SRv6 SubSubTLV : SRv6 SID Structure (1)
Loc-Block-Len : 32                Loc-Node-Len  : 16
Func-Len      : 20                Arg-Len       : 0
Tpose-Len     : 20                Tpose-offset  : 48
-----
Routes : 1
```

The same VPN-IPv4 prefix is shown in the packet capture of *Figure 8-16*. The `MP_REACH_NLRI` is AFI 1 (IPv4) SAFI 128 (Labeled VPN Unicast) and contains the IP prefix, Route-Distinguisher, and MPLS label value of 524285 with the Bottom of Stack flag set to one. The BGP Prefix-SID attribute contains the SRv6 L3 Service TLV with the SRv6 SID-Information sub-TLV and the SRv6 SID Structure sub-sub-TLV. It's worth emphasising that the `Full Sid` field shown in *Output 8-42* is not an advertised value. The SRv6 SID value contains the Locator-Block and Locator-Node fields, and the Function part is transposed. It is provided in the CLI output purely for operator ease to show a concatenated view of the entire SID constructed of Locator, Function, and Argument.

One final point to mention is that the advertised SRv6 Locator-Block and Locator-Node must be routable within the IPv6 FIB in order for the advertised prefix to be correctly resolved.

Figure 8-16: VPN-IPv4 Prefix with SRv6 Service TLV

```

- Border Gateway Protocol - UPDATE Message
  Marker: ffffffffffffffffffffffffffffffff
  Length: 138
  Type: UPDATE Message (2)
  Withdrawn Routes Length: 0
  Total Path Attribute Length: 187
  + Path attributes
    + Path Attribute - MP_REACH_NLRI
      * Flags: 0x90, Optional, Extended-Length, Non-transitive, Complete
      Type Code: MP_REACH_NLRI (14)
      Length: 32
      Address family identifier (AFI): IPv4 (1)
      Subsequent address family identifier (SAFI): Labeled VPN Unicast (128)
      * Next hop: R0n0:0 IPv4=192.0.2.1
      Number of Subnetwork points of attachment (SAPA): 0
    + Network Layer Reachability Information (NLRI)
      + BGP Prefix
        Prefix Length: 112
        Label Stack: 524285 (bottom)
        Route Distinguisher: 64496:1
        No Reach NLRI IPv4 prefix: 172.21.1.0
    + Path Attribute - ORIGIN: IGP
    + Path Attribute - AS_PATH: 64512
    + Path Attribute - LOCAL_PREF: 100
    + Path Attribute - EXTENDED_COMMUNITIES
    + Path Attribute - BGP Prefix-SID
      * Flags: 0xc8, Optional, Transitive, Complete
      Type Code: BGP Prefix-SID (40)
      Length: 37
    + SRv6 L3 Service
      Type: SRv6 L3 Service (5)
      Length: 34
      Reserved: 00
      + SRv6 Service Sub-TLVs
        + SRv6 Service Sub-TLV - SRv6 SID Information
          Type: SRv6 SID Information (1)
          Length: 30
          Reserved: 00
          SRv6 SID Value: 3fff-3e8:1::
          SRv6 SID Flags: 0x00
          SRv6 Endpoint Behavior: End.DT46 (0x0014)
          Reserved: 00
        + SRv6 Service Data Sub-Sub-TLVs
          + SRv6 Service Data Sub-Sub-TLV - SRv6 SID Structure
            Type: SRv6 SID Structure (1)
            Length: 6
            Locator Block Length: 32
            Locator Node Length: 16
            Function Length: 20
            Argument Length: 0
            Transposition Length: 28
            Transposition Offset: 48

```

SR-OS currently provides SRv6 support for VPRN (End.DT4, End.DT6), VPRN services with EVPN-IFL (again End.DT4, End.DT6), EVPN-VPWS (End.DT2U, End.DT2M), EVPN-VPWS (End.DX2), and IPv4/IPv6 BGP routes (End.DT4, End.DT6) in the base routing instance. *Output 8-43* provides an example of enabling SRv6 for a VPRN service. Within the base **vprn** context a **segment-routing-v6** context is configured followed by an integer that can either be 1 or 2. SR-OS allows for up to two instances to allow for a scenario where an SRv6 router is connected to two SRv6 domains and is interworking between them. Within the **segment-routing-v6** context a **locator** or **micro-segment-locator** is configured, which must reference a previously configured **locator** or **micro-segment-locator** instance. Within the **locator** or **micro-segment-locator** context the relevant Endpoint behaviours are enabled under the **function** context. In this example, both **end-dt4** and **end-dt6** endpoint behaviours are enabled to support a VPRN that is running both IPv4 and IPv6. For a VPRN running both IPv4 and IPv6 this could equally be configured as **end.dt46**, with the advantage of this approach being that it would only consume only a single service SID rather than two service SIDs consumed by the **end-dt4** and **end-dt6** approach. This may be important for smaller platforms where service SID resource is scarce. It's also worth noting that PEs within the same VPRN service can be configured differently if required (the behaviour of a service SID is not validated by a PE receiving a service route NLRI).

Within the **bgp-ipvprn** context the same **segment-routing-v6** instance is referenced, under which the conventional VPRN service parameters such as **route-distinguisher** and **vrf-import/export** policies are configured. An **srv6** context then allows for the assignment of a

default-locator to each configured SRv6 instance. For EVPN-VPLS and EVPN-VPWS services the configuration would be similar to *Output 8-43* with the exception that **segment-routing-v6** is enabled under the **bgp-evpn** context rather than the **bgp-ipvpn** context used for VPRN services.

Output 8-43: Enabling SRv6 in services

```

vprn "one" {
    segment-routing-v6 1 {
        locator "Alg0-Locator" {
            function {
                end-dt46 {
                }
            }
        }
    }
}
bgp-ipvpn {
    segment-routing-v6 1 {
        admin-state enable
        route-distinguisher "64496:1"
        vrf-target {
            import-community "target:64496:1"
            export-community "target:64496:1"
        }
        srv6 {
            instance 1
            default-locator "Alg0-Locator"
        }
    }
}

```

SRv6 can also be used for IPv4/IPv6 BGP prefixes in the base routing instance and the necessary configuration is shown in *Output 8-44*. Within the base router **bgp** context a **segment-routing-v6** context is created, under which the **family** command allows for selection of either **ipv4** or **ipv6** address families. Under each **family** the **add-srv6-tlvs** command instructs the router to add the SRv6 Service TLVs for IPv4 prefixes advertised in BGP and references an existing generic or uSID Locator using the **locator-name** or **micro-segment-locator-name** command. If the **ignore-received-srv6-tlvs** is set to **false** the system will process received SRv6 Service TLVs and use the Locator for resolving prefixes in addition to performing BGP Next-Hop reachability checks. If set to **true**, the system will ignore any received SRv6 Service TLVs and use the received BGP Next-Hop to resolve the prefix. The referenced **locator** 'Alg0-Locator' within the **base-routing-instance** context is also included in the output, mainly to highlight the need to configure the **end-dt4** function in order to enable that Endpoint behaviour for this locator. When enabling the **end-dt4** function a sub-context is created under which a static Function **value** can be assigned in the range 1-1048575. If no explicit **value** is specified, a dynamic value is assigned.

Output 8-44: SRv6 configuration for Base router BGP

```

router "Base" {
    bgp {
        segment-routing-v6 {
            family ipv4 {
                ignore-received-srv6-tlvs false
                add-srv6-tlvs {
                    locator-name "Alg0-Locator"
                }
            }
        }
    }
}

```

```

    }
  }
}
segment-routing {
  segment-routing-v6 {
    base-routing-instance {
      locator "Alg0-Locator" {
        function {
          end-dt4 {
            }
          }
        }
      }
    }
  }
}
}
}
}

```

Although not shown in *Output 8-44* the option exists under the **segment-routing-v6** context at both BGP **group** and **neighbour** level to advertise routes to certain peers with the SRv6 TLVs removed using the command **route-advertisement family strip-srv6-tlvs true**. Equally, if required, it is possible to drop routes from certain peers that contain SRv6 TLVs using the command **route-advertisement drop-routes-with-srv6-tlvs true**. The intent of these commands is to allow for routers in the network that may not be able to properly process BGP updates containing SRv6 information, or to constrain the advertisement of routes with SRv6 TLVs for security reasons.

With the above configuration in place, *Output 8-45* shows an IPv4 prefix learnt from an external peer being advertised into IBGP. The SRv6 Service TLV contains the SID-Information sub-TLV and SID Structure sub-sub-TLV but as there is no MPLS label associated with the prefix transposition cannot be used, hence the transposition length (*Tpose-Len*) and the transposition offset (*Tpose-offset*) fields are both set to zero.

Output 8-45: IPv4 prefix with SRv6 L3 Service TLV

```

A:admin@R1# show router bgp routes 172.16.0.0/20 hunt
=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
--- [snip] ---
-----
RIB Out Entries
-----
Network       : 172.16.0.0/20
Nexthop       : 192.0.2.1
Path Id       : 7
To            : 192.0.2.10
Res. Protocol  : INVALID                      Res. Metric    : 0
Res. Nexthop   : n/a
Local Pref.    : 100                          Interface Name : NotAvailable
Aggregator AS  : None                        Aggregator     : None
Atomic Aggr.   : Not Atomic                  MED            : None
AIGP Metric    : None                        IGP Cost       : 0
Connector      : None
Community      : No Community Members
Cluster        : No Cluster Members
Originator Id  : None                        Peer Router Id  : 192.0.2.10
Origin         : IGP
AS-Path        : 64511
Route Tag      : 0
Neighbor-AS    : 64511

```

DB Orig Val	: NotFound	Final Orig Val	: N/A
Source Class	: 0	Dest Class	: 0
SRv6 TLV Type	: SRv6 L3 Service TLV (5)		
SRv6 SubTLV	: SRv6 SID Information (1)		
Sid	: 3fff:3ea:1:7fff::		
Behavior	: End.DT4 (19)		
SRv6 SubSubTLV	: SRv6 SID Structure (1)		
Loc-Block-Len	: 32	Loc-Node-Len	: 16
Func-Len	: 20	Arg-Len	: 0
Tpose-Len	: 0	Tpose-offset	: 0

9 Reconvergence

The time to reconverge upon change and/or network failure has become an important factor in delivery of business services. Historically BGP was intentionally slow to reconverge in order to avoid route oscillation between Autonomous Systems. However, the requirement is now frequently to deliver sub-second reconvergence times because of the real-time nature of traffic being carried through Layer-3 and Layer-2 VPNs. This has meant changes not only in the speed with which BGP reacts to failure but also changes in the protocol to allow for increased path visibility between BGP speakers. To avoid lengthy reconvergence times caused by BGP Update propagation, an ingress router must be made aware of not just the primary Next-Hop, but also a viable backup Next-Hop. These can frequently be suppressed by best-path selection at an intermediate BGP speaker. This chapter will therefore describe some of the mechanisms that can be used for providing a faster reaction time to failure as well as increased path visibility to an ingress router.

Advertisement of Multiple Paths (ADD-PATH)

The Advertisement of Multiple Paths (ADD-PATH) [42] is an extension to BGP that allows for advertisement of multiple paths for the same prefix/NLRI. This allows for upstream route propagation without subsequent advertisements replacing earlier ones by an intervening BGP speaker's path-selection process. The purpose is to reduce route oscillations, enable load-balancing, and improve routing convergence by making an alternative path immediately available to a re-converging router.

To achieve this, each advertised path is identified by a 4-octet Path Identifier. To carry the Path Identifier in the Update message the NLRI encodings are extended by prepending the Path Identifier field. The combination of Path Identifier and prefix thereafter identifies a given path. Assignment of Path Identifier values is local, and where a BGP speaker re-advertises a route with Path Identifiers, it must generate its own Path Identifier.

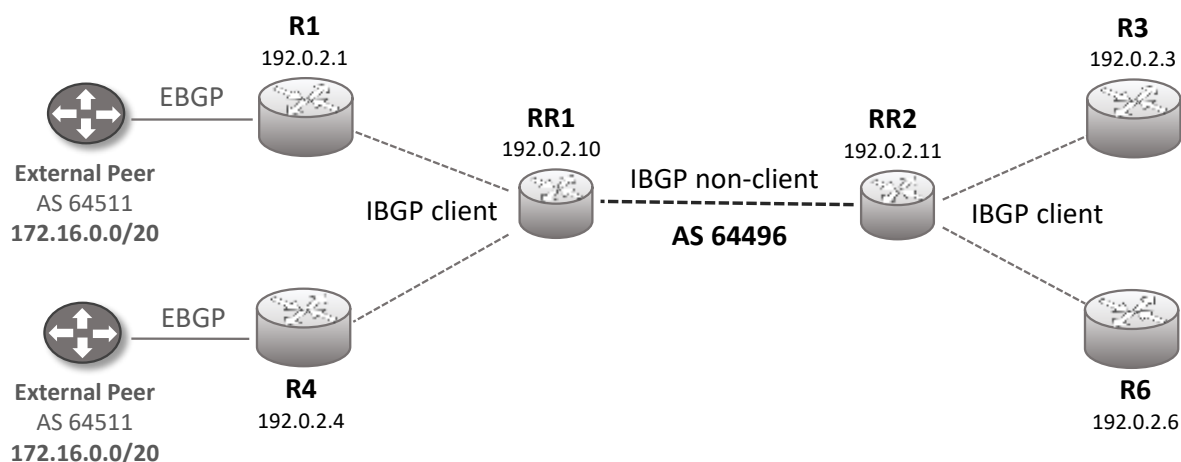
Figure 9-1: ADD-PATH NLRI

Path Identifier (4 octets)
Length (1 octet)
Prefix (variable)

ADD-PATH is a capability negotiated in the Open exchange (capability code 69). During the exchange the peers negotiate Send/Receive values indicating a) whether they are willing to receive multiple paths from the peer for a given AFI/SAFI, b) would like to send multiple paths to the peer for a given AFI/SAFI, or c) both. *Figure 9-2* is used to illustrate the concept of the ADD-PATH capability. In this topology routers R1, R3, R4, and R6 are in AS 64496; R1 and R4

are clients of RR1 and R3 and R6 are clients of RR2. Both R1 and R4 peer externally with AS 64511 and learn prefix 172.16.0.0/20. The physical topology is not shown, but the objective is for R3 to receive multiple copies of the 172.16.0.0/20 prefix with redundant Next-Hops to provide for faster reconvergence under failure. Of course, without ADD-PATH, RR1 would receive the prefix 172.16.0.0/20 from both R1 and R4 but would execute the best-path selection algorithm and only propagate that best-path Update upstream to RR2.

Figure 9-2: ADD-PATH Test Topology



The ADD-PATH capability is negotiated for each AFI and SAFI that is desired by BGP speakers. The ADD-PATH capability also contains a Send/Receive field that indicates whether the sender is able to receive multiple paths from its peer (value 1), able to send multiple paths to its peer (value 2) or both (value 3) for the given AFI/SAFI. SR-OS provides ADD-PATH support for the EVPN, IPv4/IPv6, Label-IPv4/IPv6, VPN-IPv4/IPv6, MVPN-IPv4/IPv6, and MCAST-VPN-IPv4/IPv6 Address Families and the configuration reflects that it is negotiated on a per AFI/SAFI basis. The **add-path** context is available at BGP, group, or neighbour level. Within the **add-path** context the required **family** is selected after which the user must indicate the maximum number of paths that the router should **send** for each prefix, followed by an optional **receive** keyword. (If the **receive** keyword is not included it is enabled by default.) If a given BGP speaker has two paths to a given prefix and the ADD-PATH **send** keyword is configured with a value of two, both prefixes will be propagated to its peers, and each propagated path will have a different Path Identifier. For the purpose of illustration all BGP speakers in AS 64496 of Figure 9-2 are configured with a **send** value of 2. Output 9-1 shows an example of the required configuration at RR1 with different peer groups for client IBGP sessions (R1 and R4) and non-client IBGP sessions (RR2).

Output 9-1: Configuration of ADD-PATH for IPv4 Address Family at RR1

```
router "Base" {
  bgp {
    group "CLIENTS" {
      peer-as 64496
      family {
        ipv4 true
      }
    }
  }
}
```

```

        cluster {
            cluster-id 192.0.2.10
        }
        add-paths {
            ipv4 {
                send 2
                receive true
            }
        }
    }
    group "NON-CLIENTS" {
        peer-as 64496
        family {
            ipv4 true
        }
        add-paths {
            ipv4 {
                send 2
                receive true
            }
        }
    }
    neighbor "192.0.2.1" {
        group "CLIENTS"
    }
    neighbor "192.0.2.4" {
        group "CLIENTS"
    }
    neighbor "192.0.2.11" {
        group "NON-CLIENTS"
    }
}

```

Debug 9-1 shows the prefix 172.16.0.0/20 advertised by router R1 towards RR1. In the debug, the Path-ID is actually shown as a suffix of the prefix 172.16.0.0/20 in the NLRI, but this is simply to make the output more readable - the Path Identifier is actually prepended to the prefix.

Debug 9-1: Prefix 172.16.0.0/20 advertised by R1 with Path-ID

```

694 2025/09/15 17:52:53.465 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 27
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 6 AS Path:
        Type: 2 Len: 1 < 64511 >
    Flag: 0x40 Type: 3 Len: 4 Nexthop: 192.0.2.1
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    NLRI: Length = 8
        172.16.0.0/20 Path-ID 11

```

The Path Identifier generated by router R1 when advertising the 172.16.0.0/20 prefix to a neighbour can also be displayed using the command shown in *Output 9-2*.

Output 9-2: Path-ID displayed in per-neighbour advertised routes

```

A:admin@R1# show router bgp neighbor 192.0.2.10 advertised-routes ipv4
=====
BGP Router ID:192.0.2.1          AS:64496          Local AS:64496
=====

```


Legend -
 Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
 l - leaked, x - stale, > - best, b - backup, p - purge,
 w - unused-weight-only
 Origin codes : i - IGP, e - EGP, ? - incomplete

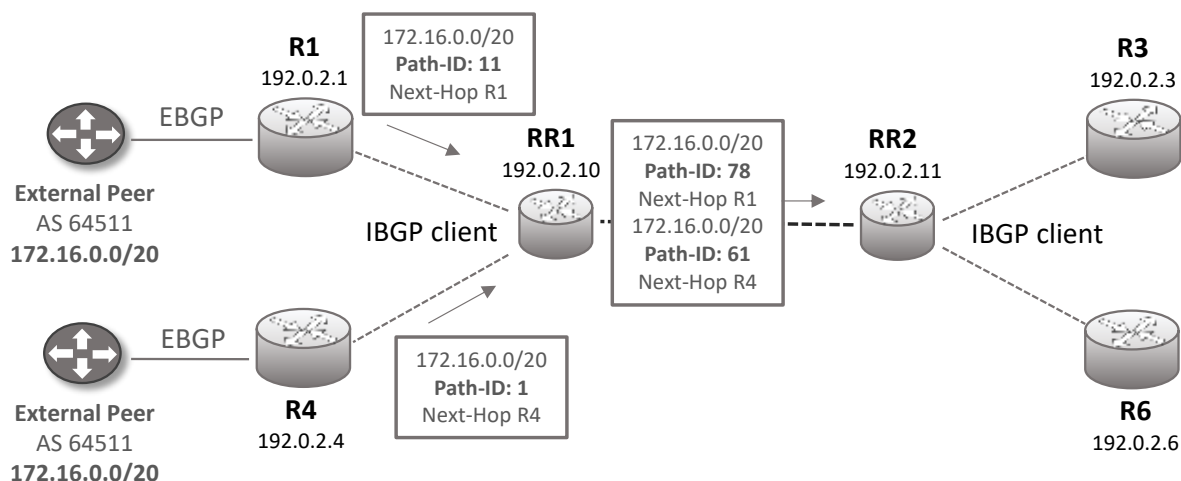
BGP IPv4 Routes

Flag	Network Nexthop (Router) As-Path	LocalPref Path-Id	MED IGP Cost Label
i	172.16.0.0/20 192.0.2.1 64511	100 11	None 0 -

Routes : 1

When RR1 receives two paths for the same prefix but with different Path Identifiers, it advertises both paths upstream because of the **add-path send 2** configuration; noting that it generates its own Path Identifiers when re-advertising the prefix. The same action is taken at RR2, which results in two paths being advertised to R3 as illustrated in *Figure 9-3*.

Figure 9-3: ADD-PATH Prefix Propagation



If a BGP speaker is configured with **add-paths send *n*** and has more than *n* paths available in the RIB-In, it selects the best *n* overall paths for each prefix as candidates for upstream propagation whilst attempting to meet split-horizon and/or Next-Hop diversity objectives. This selection of the paths to advertise uses a modified path selection algorithm as follows:

- If the best path is a non-BGP route exported to the neighbour, advertise only that path (unless advertise-inactive is set).
- If the best path is a BGP route from the neighbour and split-horizon applies, start with the next-best path and advertise the single best path from each set of paths with the same BGP Next-Hop until *n* paths have been advertised or there are no more valid paths.
- If the best path is a BGP route from the neighbour and split-horizon does not apply, start with the best path and advertise the single best path from each set of paths

with the same BGP Next-Hop until n paths have been advertised or there are no more valid paths.

At R3 verification is obtained that both paths have been successfully advertised with a Next-Hop of R1 (Path-ID 9) and a Next-Hop of R4 (Path-ID 5). These paths can be used for backup (PIC) or load-balancing purposes (Multipath) depending on the user requirement.

Output 9-3: BGP routes with Path-ID at R3

```
A:admin@R3# show router bgp routes 172.16.0.0/20
=====
BGP Router ID:192.0.2.3          AS:64496          Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP IPv4 Routes
=====
Flag  Network                               LocalPref  MED
      Nexthop (Router)                     Path-Id    IGP Cost
      As-Path                               Label
-----
u*>i  172.16.0.0/20                         100        None
      192.0.2.1                             9          200
      64511                                  -
*i    172.16.0.0/20                         100        None
      192.0.2.4                             5          300
      64511                                  -
-----
Routes : 2
```

ADD-PATH is a useful mechanism for increasing BGP path visibility. For the VPN-IPv4/IPv6 Address Families, a well-known and widely deployed mechanism to achieve the same objective is to use different Route-Distinguishers at dual-homed sites to make a given set of IPv4/IPv6 prefixes unique to each site. Because ADD-PATH can be enabled on an Address Family basis the choice is with the operator as to which mechanism is most suitable to their environment.

The use of ADD-PATH at an Address Family level does however mean that more BGP paths are made available to routers, which in turn consumes memory. It may therefore be desirable to limit the number of paths that are sent for a particular set of prefixes. In other words, it may be desirable to send multiple paths only for prefixes where increased visibility and fast reconvergence is required but limit the number of paths to one for other prefixes. SR-OS allows for limiting the number of paths that are advertised for a given prefix or set of prefixes using the route-policy framework. *Output 9-4* provides an example of how the number of paths can be limited for a certain prefix or number of prefixes as configured at RR1. In this example, RR1 has a non-client IBGP session to RR2 which allows for the propagation of two paths (**add-paths ipv4 send 2 receive**). For the purpose of this example however, the objective is to restrict the advertisement of two paths to the prefix 172.16.0.0/20 only. All other prefixes should only have a single path advertised.

Entry 10 of the **policy-statement** “Selective-ADD-PATH” references the 172.16.0.0/20 prefix using the **from** criteria and has an **action** of **accept**. This prefix will therefore exit the policy at this point. Following this, the policy has a **default-action** of **accept**, which then limits the number of paths that can be advertised to one using the **add-paths-send-limit** command. The **policy-statement** is then applied as an **import** policy towards the RR clients, R1 and R4. When the prefix 172.16.0.0/20 is subsequently received from R1 and R4 it exits the import policy at entry 10. The prefix is then advertised towards RR2 and is subject to the **add-paths** configuration applied to this IBGP session. When any other prefix from R1 or R4 is received at RR1, it exits the import policy using the **default-action**, which is to limit the number of advertised paths to 1.

Output 9-4: ADD-PATH for selected prefixes at RR1

```

policy-options {
  prefix-list "AS64511-Prefixes" {
    prefix 172.16.0.0/20 type exact {
    }
  }
  policy-statement "Selective-ADD-PATH" {
    entry 10 {
      from {
        prefix-list ["AS64511-Prefixes"]
      }
      action {
        action-type accept
      }
    }
    default-action {
      action-type accept
      add-paths-send-limit 1
    }
  }
}
router "Base" {
  bgp {
    group "CLIENTS" {
      import {
        policy ["Selective-ADD-PATH"]
      }
    }
    neighbor "192.0.2.1" {
      group "CLIENTS"
    }
    neighbor "192.0.2.4" {
      group "CLIENTS"
    }
  }
}

```



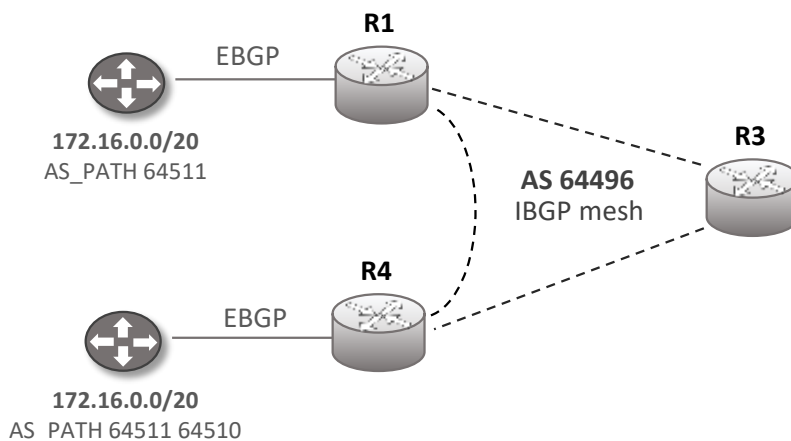
Note: Policies implementing ADD-PATH are only supported when applied as import policies. They are not supported when applied as export policies. The reason is that the processing of an export policy is effectively “too late” in the BGP route processing pipeline to reach a decision about the number of ADD-PATHs to maintain per prefix.

Best External

Best External is a mechanism that allows a BGP speaker to advertise its best external path to IBGP peers even if its own selected best path is received from an internal peer. By advertising the best external route when different from the best route, additional path visibility can be provided to the IBGP mesh. When two paths are available to reach a given destination and one is preferred, the availability of an alternate path in the RIB means that only a FIB update is required should the preferred Next-Hop fail. In addition, the presence of two paths can reduce route oscillation.

Best External does not require any protocol extensions in order to function but instead modifies the route advertisement criteria of the base BGP specification. Take the example of *Figure 9-4* where routers R1, R3, and R4 form part of AS 64496 and are fully IBGP meshed. Router R1 learns the prefix 172.16.0.0/20 externally with AS_PATH 64511, whilst router R4 learns the prefix 172.16.0.0/20 with AS_PATH 64511 64510. In this scenario, router R4 does not advertise its externally learnt prefix in IBGP because it prefers the internally learnt prefix from R1 due to the shorter AS_PATH. The result is that router R3 only has one path to 172.16.0.0/20 in its RIB-In.

Figure 9-4: Route propagation without Best External



If, however Best External is enabled at R4, it advertises the prefix 172.16.0.0/20 learnt through EBGP even though its best path is learnt from an internal peer. The result is that router R3 now has two paths to prefix 172.16.0.0/20 and can therefore reconverge more quickly in the event of failure of the preferred Next-Hop. Best External is enabled through configuration of the **advertise-external** command followed by keywords for each applicable Address Family. It can only be applied at the global BGP level and is supported for the IPv4, IPv6, Label-IPv4, and Label-IPv4 Address Families.

Output 9-5: Configuration of Best External at R4

```

router "Base" {
    bgp {
        advertise-external {
            ipv4 true
        }
    }
}
  
```

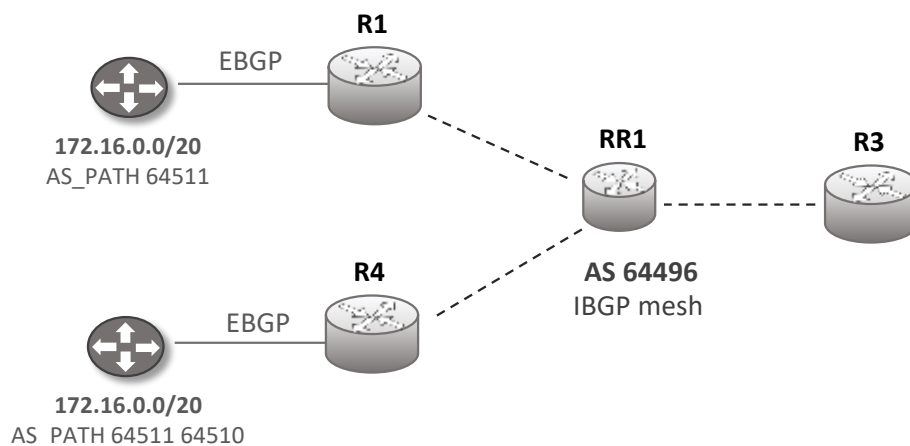
```

    group "IBGP-Mesh" {
        peer-as 64496
        family {
            ipv4 true
        }
        ...etc
    }
}

```

In a simple topology like *Figure 9-4* Best External provides a good solution to increase path visibility. This solution is arguably better than ADD-PATH because it doesn't require any signalled/advertised extensions (only a modified route advertisement criterion) and therefore doesn't require that the whole network is upgraded/re-configured to support it. However, if the logical topology is modified a little to insert a Route-Reflector as shown in *Figure 9-5*, the Best External mechanism doesn't achieve the objective of increasing path visibility to R3. This is because the Route-Reflector implements the standard best path algorithm across the internally learnt paths from R1 and R4 (there is no external path) and only advertises the best path upstream to R3.

Figure 9-5: Best External with Route-Reflection



The Best External draft suggests some modifications to Route-Reflector route advertisement procedures in an effort to increase path visibility of paths advertised to and from a Route-Reflector cluster. To increase path visibility to a cluster, the draft suggests that if client-to-client reflection is disabled and the cluster operates as a mesh, a Route-Reflector may advertise to the cluster the preferred path from the set of paths not received from within the cluster. To increase path visibility from a cluster, the draft suggests that when advertising a route to a non-client IBGP peer, a BGP speaker may advertise an alternative best route from a cluster if the preferred path is learnt from outside the cluster. Given the unlikely scenario of a Service Provider disabling client-to-client reflection on an existing Route-Reflector, these advertisement rules are not implemented in SR-OS. Where Route Reflection is used, the only solution is to enable ADD-PATH in conjunction with Best External (or in an IP-VPN environment, optionally use different Route-Distinguishers in conjunction with Best External).

The route advertisement rules suggested by the draft are however, supported in SR-OS between members of a Confederated AS. That is, if a BGP speaker has **advertise-external** enabled and its preferred path is a route from a confed-EBGP peer in AS *m*, then this preferred path should be advertised to all other confed-EBGP peers AND the best internal route should be advertised to confed-EBGP peers in AS *m*. The best internal route is the one found by running the BGP path selection algorithm across the paths in the RIB-IN excluding those learnt from member AS *m*.



Note: Within a VPRN service the **export-inactive-bgp** command provides a behaviour very similar to best external. It allows the preferred BGP path to be exported as a BGP-VPN route even if it is inactive in the route-table because a BGP-VPN route from another PE is present.

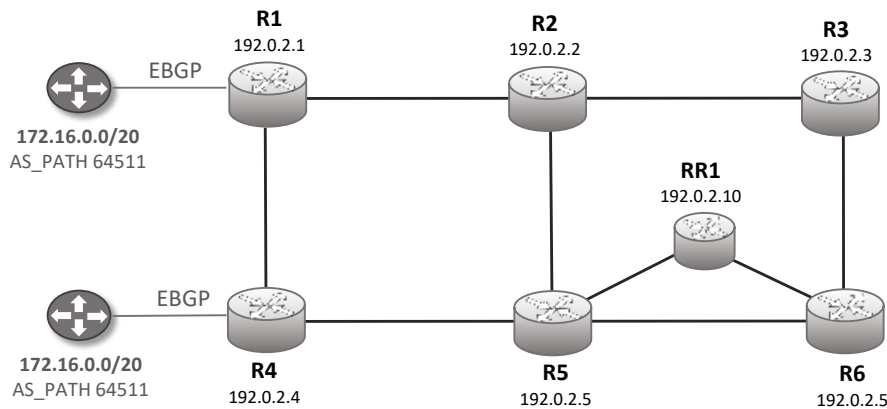
Next-Hop Tracking

Next-Hop Tracking is a mechanism that actively monitors all route-table and MPLS tunnel-table modifications and immediately triggers BGP Next-Hop Resolution add/delete/modify messages to the FIB when a change is detected. Even when alternate paths are already programmed into the FIB (for example when PIC is enabled or ECMP/IBGP-Multipath is in use), the CPM still must notify the IOM/IMMs of reachability failure/changes to allow the datapath to be re-programmed accordingly. Next-Hop Tracking is enabled by default (and cannot be disabled) and ensures that this update process is entirely event-driven based upon the current network state.

The active BGP Next-Hop for a unicast IPv4 NLRI is resolved by the longest prefix match of the IPv4 Next-Hop address that is installed and active in the forwarding table (and similar logic applies to an IPv6 BGP Next-Hop address associated with a unicast IPv6 NLRI). If there is no active and eligible longest prefix match for the Next-Hop address, associated BGP prefixes are flagged as invalid in the RIB-In.

Assume a scenario as shown in *Figure 9-6*. Routers R1 to R6 form part of AS 64496 and each is peered in IBGP with Route-Reflector RR1. IS-IS is used as the IGP and the Autonomous System is entirely Level-2. Routers R1 and R4 are peered externally with AS 64511 and both learn prefix 172.16.0.0/20 from their external peers. ADD-PATH is configured on all AS 64496 routers, and as a result router R3 receives two paths for 172.16.0.0/20 via R1 and R4. Because there is no BGP multipath in use both are held in RIB-In as valid but only one is installed in the route-table as best and used which is the route via R1 (192.0.2.1) due to shorter IGP cost to the Next-Hop.

Figure 9-6: Next-Hop Tracking Use-Case Topology



If router R1 should fail, router R3 must reconverge the BGP Next-Hop for the prefix 172.16.0.0/20. In *Debug 9-2*, the Route-Table Manager (RTM) is responsible for maintaining the route-table and the removal of R1's system and peering address (192.0.2.1) from the route-table (RTM DELETE) following the failure is seen in as the first event in the output. This is followed within milliseconds by the immediate modification (MOD_ROUTE) of the active Next-Hop to R4's system address (192.0.2.4) for prefix 172.16.0.0/20. This is a function of Next-Hop Tracking.

Debug 9-2: Next-Hop Tracking at R3

```

258 2025/09/16 16:18:15.160 BST minor: DEBUG #2001 Base PIP
PIP: ROUTE
instance 1 (Base), RTM DELETE event
  New Route Info
    prefix: 192.0.2.1/32 (0x1194ca300) preference: 18 metric: 200 backup metric: 0
owner: ISIS ownerId: 0
  1 ecmp hops 0 backup hops:
    hop 0: 192.168.0.5 @ if 6, weight 0

266 2025/09/16 16:18:15.167 BST minor: DEBUG #2001 Base BGP
BGP: RTM
VR 1: Group IBGP-Core: Peer 192.0.2.10 - MOD_ROUTE 172.16.0.0/20 with RTM Prefix:
172.16.0.0/20

```

As shown in *Output 9-6* the Next-Hop information for router R1 is removed and the Next-Hop becomes unresolved.

Output 9-6: Next-Hop resolution for R1

```

A:admin@R3# show router bgp next-hop 192.0.2.1
=====
BGP Router ID:192.0.2.3          AS:64496          Local AS:64496
=====

BGP Next Hop
=====
Next Hop                                Pref      Owner
Resolving Prefix                       FibProg   Metric
Resolved Next Hop                     Colored   Ref. Count
Admin-tag-policy                       FlexAlgo  Last Mod.
=====

```

```

192.0.2.1          -          -
  Unresolved        -
  --               -
  --               00h01m33s
-----
Next Hops : 1

```

The test topology of *Figure 9-6* is now slightly modified so that routers R3 and R6 are part of IS-IS Level-1 and routers R2 and R5 are Level-1/Level-2 routers. R2 and R5 redistribute 32-bit system addresses from Level-2 into Level-1 and set the Attach bit in LSPs that are sent into the Level-1 area so that router R3 has a default-route from both Level-1/Level-2 routers. This is a common scenario, but one that can impact how Next-Hop Tracking operates. If a failure of router R1 is simulated once again, the sequence of events is shown in *Debug 9-3* and as can be seen the first event is a straightforward removal of R1's system address (RTM DELETE). However, R3 still has two default routes towards its Level-1/Level-2 routers and as a result is still able to resolve the current BGP Next-Hop (192.0.2.1) for prefix 172.16.0.0/20 until the Withdraw is received (frame 41) some three minutes later (after the BGP session has timed out). At this point R3 modifies the Next-Hop for 172.16.0.0/20 to router R4.

Debug 9-3: RTM debug for R1's system address

```

37 2025/09/16 13:21:23.230 BST minor: DEBUG #2001 Base PIP
PIP: ROUTE
instance 1 (Base), RTM DELETE event
  New Route Info
    prefix: 192.0.2.1/32 (0x1194d3ff0) preference: 15 metric: 126 backup metric: 0
owner: ISIS ownerId: 0
  1 ecmp hops 0 backup hops:
    hop 0: 192.168.0.5 @ if 6, weight 0

41 2025/09/16 13:24:23.882 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
  Withdrawn Length = 8
    172.16.0.0/20 Path-ID 122
  Total Path Attr Length = 0

```

As long as the default-route to R2 and/or R5 exists router R3 will be able to resolve router R1's BGP Next-Hop address and this can be seen in the following output which shows that the Next-Hop is being resolved with prefix 0.0.0.0/0 (Resolving Prefix).

Output 9-7: BGP Next-Hop Resolution for R1's system address at R3

```

A:admin@R3# show router bgp next-hop 192.0.2.1
=====
BGP Router ID:192.0.2.3      AS:64496      Local AS:64496
=====

BGP Next Hop
=====
Next Hop                                Pref      Owner
  Resolving Prefix                     FibProg   Metric
  Resolved Next Hop                     Colored   Ref. Count
  Admin-tag-policy                      FlexAlgo  Last Mod.
-----
192.0.2.1                                15        ISIS

```


0.0.0.0/0	Y	63
192.168.0.5	N	1
--	--	00h00m33s

Next Hops : 1

In this scenario it would be beneficial to exclude the default routes from the set of routes that are considered eligible for BGP Next-Hop resolution, and this is the purpose of Next-Hop Tracking policies. *Output 9-8* shows an example of a Next-Hop Tracking Policy used to overcome the scenario described previously. First, the route-policy framework is used to create a **policy-statement** that excludes the default route. The same **policy-statement** is then referenced within the global or VPRN BGP context using the **policy** keyword within the **next-hop-resolution** context.



Note: The route-policy used with Next-Hop Tracking should only attempt to match a prefix-list and/or a protocol name/instance. Other match criteria are not supported.

Output 9-8: Configuration for Next-Hop Tracking policy

```

policy-options {
  prefix-list "Default-Route" {
    prefix 0.0.0.0/0 type exact {
    }
  }
  policy-statement "NEXT-HOP-TRACKING" {
    entry 10 {
      from {
        prefix-list ["Default-Route"]
      }
      action {
        action-type reject
      }
    }
  }
}
router "Base" {
  bgp {
    next-hop-resolution {
      policy "NEXT-HOP-TRACKING"
    }
  }
}

```

Once again a failure of router R1 is simulated with the preceding configuration in place. As shown *Debug 9-4* R1's system address is again removed from the route-table (RTM DELETE) and immediately following this prefix 172.16.0.0/20 is modified (MOD_ROUTE) such that R4's system address is the active Next-Hop.

Debug 9-4: Removal of R1's system address from the RTM

```

379 2025/09/16 16:41:17.130 BST minor: DEBUG #2001 Base PIP
PIP: ROUTE
instance 1 (Base), RTM DELETE event
  New Route Info
    prefix: 192.0.2.1/32 (0x1194c14d8)  preference: 15  metric: 126  backup metric: 0
owner: ISIS ownerId: 0

```

```

1 ecmp hops 0 backup hops:
hop 0: 192.168.0.5 @ if 6, weight 0

386 2025/09/16 16:41:17.139 BST minor: DEBUG #2001 Base BGP
BGP: RTM
VR 1: Group IBGP-Core: Peer 192.0.2.10 - MOD_ROUTE 172.16.0.0/20 with RTM Prefix:
172.16.0.0/20

```

Output 9-9 once again shows that R1's system/peering address (192.0.2.1) is no longer resolvable (Unresolved). As a result of this R3 will treat any Updates received from R1 as invalid. This includes the prefix 172.16.0.0/20 which will now be updated to reflect a Next-Hop of router R4.

Output 9-9: BGP Next-Hop Resolution for router R1

```

A:admin@R3# show router bgp next-hop 192.0.2.1
=====
BGP Router ID:192.0.2.3      AS:64496      Local AS:64496
=====

BGP Next Hop
=====
Next Hop      Resolving Prefix      Pref      Owner
      Resolved Next Hop      FibProg      Metric
      Admin-tag-policy      Colored      Ref. Count
                        FlexAlgo      Last Mod.
-----
192.0.2.1      -                      -
      Unresolved                      -
      --                      -
      --                      00h00m05s
-----
Next Hops : 1
=====

```

The policy referenced in the **next-hop-resolution** node only affects which routes in the route-table are eligible to resolve a BGP Next-Hop address. The policy does not affect the way BGP Next-Hops are resolved to MPLS tunnels. If the network shown in *Figure 9-6* was an MPLS network and the service at R3 was a VPRN, the policy would not be required when R3 was configured as an IS-IS Level-1 router. This is because during the simulated failure of R1, the system IP address of R1 would have been removed from the route-table of R3 and replaced by the default route or routes for BGP Next-Hop resolution. Importantly however, the LSP to R1 would have been removed from the tunnel-table. As a result, the BGP route with Next-Hop of R1 would have been held in RIB-In and flagged as invalid.

Prefix Independent Convergence

In many networks large numbers of prefixes are reachable via more than one path. BGP Prefix Independent Convergence (PIC) is the term given to describe techniques that can reconverge upon failure in a time period that does not depend on the number of prefixes being restored. This is done by organising BGP prefixes into Path-Lists consisting of primary paths together with pre-computed backup paths, and through implementation of FIB hierarchy.

PIC can be categorised into core PIC or edge PIC, although in reality they are just elements of the hierarchical forwarding chain. Core PIC describes a scenario where a link or node on the path to the BGP Next-Hop fails, but the BGP Next-Hop remains reachable. Edge PIC describes a scenario where an edge node or edge link fails which results in a change of the BGP Next-Hop.

Core PIC

In SR-OS Core PIC is implemented by programming each BGP route to an IP prefix with an indirect next-hop that is actually a pointer to a set of one or more IGP next-hops. Many BGP routes can share the same indirect Next-Hop and are known as a Next-Hop set. This indirect Next-Hop may resolve to an adjacent IGP peer, which in turn may have multiple equal cost paths to reach it. This form of hierarchy is often called *indirection*. In the event that the IP interface and/or MPLS LSP used to reach a BGP Next-Hop transitions or there is a topology change, only the Next-Hop set is modified and only a small number of FIB objects are modified without the requirement to modify the possibly large number of BGP prefixes.

The process of triggering updates to the FIB is managed by an event-based mechanism similar to Next-Hop Tracking that actively monitors all IGP and/or MPLS route-table and tunnel-table modifications and immediately triggers Next-Hop Resolution add/delete/modify messages to the FIB as appropriate.

Core PIC is enabled by default and cannot be disabled.

Edge PIC

As previously described, edge PIC describes the scenario where an edge node or edge link fails resulting in a change of BGP Next-Hop for a given number of prefixes. Edge PIC is applicable to a router only when more than one path is known. In most cases it will be necessary to run a mechanism such as ADD-PATH for IP and/or VPN-IP prefixes, or unique Route-Distinguishers for VPN-IP prefixes.

When edge PIC is enabled, the BGP decision process is modified so the output (best path) becomes a Path-List consisting of {primary, backup}. There may be one primary path or there may be more than one primary path, if, for example BGP multipath is enabled and multiple equal-cost paths exist. The backup path is computed by executing the BGP decision process (down to the lowest IP address as the final tie-breaker) on all the available paths except those already selected for primary paths, or all those that have a Next-Hop attribute in common with selected primary paths. There may only be one backup path, and there may be none if the only available backup path fails to meet the previously described criteria.

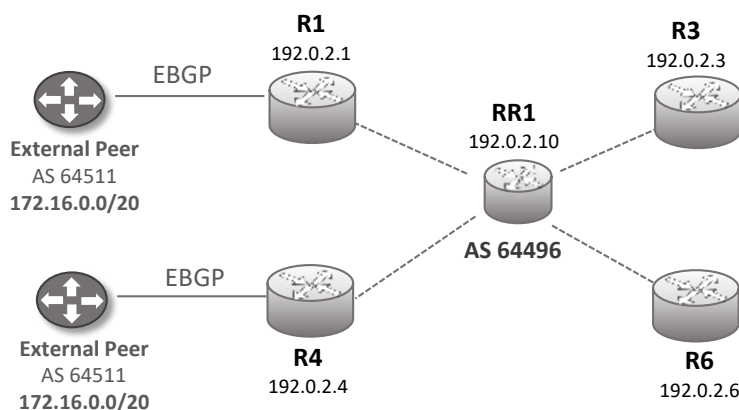
When a route is programmed into the data-path the associated {primary, backup} Path-List is also considered, and all routes with a common Path-List are grouped together retaining their primary/backup paths. This means that when there is a change to the way a BGP Next-Hop is resolved, only one forwarding plane update is needed as opposed to one update for

every route in the group. If a BGP Next-Hop becomes unreachable (detected by Next-Hop Tracking) and no other valid primary paths are available, the data-path re-programmes the common Path-Lists to use the backup path. This results in a rapid reconvergence of traffic that is independent of the number of prefixes. The following events can trigger failure of the primary path and switchover to the backup path:

- The peer IP address is unreachable, and peer-tracking is enabled.
- The BFD session associated with BGP peer goes down.
- The BGP session terminated with peer (for example, send/receive Notification).
- There is no longer any route (allowed by the **next-hop-resolution** policy) that can resolve the BGP Next-Hop address.
- The MPLS tunnel that resolves the Next-Hop goes down (for example, for LDP the peer withdraws its label mapping, or for RSVP an ResvTear is received, or for SR the OSPF LSA or IS-IS LSP carrying the Prefix-SID is aged out).

Because the ADD-PATH feature is complimentary to the use of edge PIC, the topology used for edge PIC is similar to that used for illustrating the use of ADD-PATH (*Figure 9-2*), although only a single Route-Reflector is present. Routers R1, R3, R4, and R6 are in AS 64496 and are IBGP clients of a Route-Reflector, RR1. Routers R1 and R4 peer externally with AS 64511 and learn prefix 172.16.0.0/20. All routers are configured to use ADD-PATH for the IPv4 Address Family, and as a result router R3 receives two paths for prefix 172.16.0.0/20: one with a Next-Hop of R1 and one with a Next-Hop of R4.

Figure 9-7: Edge PIC Test Topology



Edge PIC is supported in the global routing context and within the VPRN context for IPv4, IPv6, Labeled-IPv4, and Labeled-IPv6 routes. Edge PIC is also supported for VPN-IPv4 and VPN-IPv6 routes on a per-VPRN basis. Edge PIC can be configured generically on a per-Address Family basis, or it can be configured in a more granular way for a subset of prefixes using route-policy. When edge PIC is enabled on a per-Address Family basis it is configured using the **backup-path** command, which can only be provisioned at the BGP level and not the group or neighbour level. *Output 9-10* illustrates the configuration of edge PIC at router R3. Here the example is provided at global BGP level, but the configuration is identical within a VPRN context for the IPv4 and IPv6 Address Families. When enabled within the context of a

VRPN, edge PIC is applicable only to routes learnt in IPv4/IPv6 BGP from PE-CE peers. (Edge PIC for VPN-IPv4/VPN-IPv6 routes is discussed later in this section.)

Output 9-10: Edge PIC configuration at R3

```
router "Base" {
  bgp {
    backup-path {
      ipv4 true
    }
    add-paths {
      ipv4 {
        send 2
        receive true
      }
      label-ipv4 {
        send 2
        receive true
      }
    }
  }
}
```

There are a number of CLI commands to verify that a route with multiple paths has been successfully installed with a primary and backup route. One method is to validate against the RIB-In as shown in *Output 9-11*. Here, the first entry for 172.16.0.0/20 with Next-Hop 192.0.2.1 (R1) is the primary path, and the second entry with Next-Hop 192.0.2.4 (R4) is the backup path denoted by the backup (b) flag.

Output 9-11: RIB-In with backup route at R3

```
A:admin@R3# show router bgp routes 172.16.0.0/20
=====
BGP Router ID:192.0.2.3      AS:64496      Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP IPv4 Routes
=====
```

Flag	Network Nexthop (Router) As-Path	LocalPref Path-Id	MED IGP Cost Label
u*>i	172.16.0.0/20 192.0.2.1 64511	100 196	None 200 -
ub*i	172.16.0.0/20 192.0.2.4 64511	100 101	None 300 -

```
-----
Routes : 2
```

Another equally simple way is to validate against the route-table as shown in *Output 9-12*. In this output the presence of a backup route is denoted by the [B] flag. A third method (not shown) is to use the command “*show router route-table <prefix> extensive*”.

Output 9-12: Route-table with backup at R3

```
A:admin@R3# show router route-table 172.16.0.0/20
```

```
=====
```

Route Table (Router: Base)				
Dest Prefix[Flags]	Type	Proto	Age	Pref
Next Hop[Interface Name]			Metric	
172.16.0.0/20 [2] [B]	Remote	BGP	00h09m34s	170
192.168.0.5			200	

```
-----
```

No. of Routes: 1
 Flags: n = Number of times nexthop is repeated
 B = BGP backup route available
 L = LFA nexthop available
 S = Sticky ECMP requested

In the same way that edge PIC can be configured within the base routing instance on a per-Address Family basis, it can be enabled for VPN-IPv4 and VPN-IPv6 Address Families on a per-VRPN basis. Enabling edge PIC for VPN-IPv4 and/or VPN-IPv6 Address Families on a per-VRPN basis is done using the **bgp-vpn-backup** command with **ipv4** and/or **ipv6** set to **true**.

Output 9-13: Edge PIC for VPN-IPv4/IPv6

```
vrpn "one" {
    bgp-ipvpn {
        mpls {
            admin-state enable
            route-distinguisher "64496:1"
            vrf-target {
                import-community "target:64496:1"
                export-community "target:64496:1"
            }
            auto-bind-tunnel {
                resolution any
            }
        }
    }
    bgp-vpn-backup {
        ipv4 true
        ipv6 true
    }
}
```

When enabled, the IP-VRF route-table can be used to verify the presence of a backup path. In *Output 9-14* the 'extensive' argument is added to the route-table output which provides detailed information on both the primary and backup paths. The primary path is listed first with the relevant indirect Next-Hop, resolving Next-Hop (tunnel), and metric. The backup path follows with the same information denoted by an indication that it is backup (Backup-Path : Yes). If the 'extensive' argument is not used, the prefix will appear in the route-table with a [B] flag to indicate that it has a backup path.

Output 9-14: IP-VRF route-table with backup path

```
A:admin@R1# show router 1 route-table 172.31.56.0/24 extensive
```

```
=====
```

```
Route Table (Service: 1)
```

```
=====
Dest Prefix      : 172.31.56.0/24
Protocol        : BGP_VPN
Age             : 00h00m29s
Preference      : 170
Indirect Next-Hop : 192.0.2.3
Label           : 524237
VPN Next-Hop Index : 893
QoS             : Priority=n/c, FC=n/c
Source-Class    : 0
Dest-Class      : 0
ECMP-Weight     : N/A
Resolving Next-Hop : 192.0.2.3 (SR-ISIS tunnel:524832)
Metric          : 200
ECMP-Weight     : N/A
Indirect Next-Hop : 192.0.2.6
Label           : 524283
VPN Next-Hop Index : 909
QoS             : Priority=n/c, FC=n/c
Source-Class    : 0
Dest-Class      : 0
Backup-Path     : Yes
ECMP-Weight     : N/A
Resolving Next-Hop : 192.0.2.6 (SR-ISIS tunnel:524833)
Metric          : 300
ECMP-Weight     : N/A
-----
```

```
No. of Destinations: 1
```

Implementation of edge PIC can have an impact on FIB resources in the presence of labeled BGP routes. If the primary Next-Hop corresponds to an unlabeled BGP route resolved by an IP route, the linecard CPU programs the data-plane with the backup path immediately after the failure has occurred. If the primary Next-Hop corresponds to a labeled BGP route resolved by an MPLS tunnel, the data-plane is programmed with the primary and backup Next-Hop Label Forwarding Entry (NHLFE) information prior to the failure. The difference between failure times is fairly insignificant; for the unlabeled BGP route the linecard CPU has to instruct the data-path to replace one indirection object with another, and for the labeled BGP route the linecard CPU has to tell the data-path that a failure of the primary path occurred.

If the consumption of hardware resources is a concern, it may be preferable to enable edge PIC at a more granular level for only for a subset of prefixes, rather than all prefixes of a given Address Family. This can be implemented through the route-policy framework using the command **install-backup-path**. For example, assume that router R3 in *Figure 9-7* now receives prefix 172.16.0.0/20 plus an additional prefix 172.16.32.0/20. The operator would like to implement edge PIC for prefix 172.16.0.0/20, but not for any other prefixes received by R3, including prefix 172.16.32.0/20. (Clearly implementing edge PIC for two prefixes wouldn't be a major concern, but two prefixes are used simply for ease of illustration and in order to be able to highlight the differences between them.) *Output 9-15* provides an example of the required configuration at R3. Entry 10 of the **policy-statement** "*Selective-Edge-PIC*" references the **prefix-list** "*Backup-Prefix*" containing the prefix 172.16.0.0/20 as **from** criteria. Entry 10 then has an **action-type** of **accept**, together with **install-backup-path true**. This essentially enables Edge PIC for this matched prefix only. The **default-action** has

an **action-type** of **accept**, which accepts all other prefixes but does not install backup paths for those prefixes.

Output 9-15: Selective-Edge-PIC configuration at R3

```

policy-options {
  prefix-list "Backup-Prefix" {
    prefix 172.16.0.0/20 type exact {
    }
  }
  policy-statement "Selective-Edge-PIC" {
    entry 10 {
      from {
        prefix-list ["Backup-Prefix"]
      }
      action {
        action-type accept
        install-backup-path true
      }
    }
    default-action {
      action-type accept
    }
  }
}
router "Base" {
  bgp {
    group "IBGP-Core" {
      peer-as 64496
      ...etc
    }
    neighbor "192.0.2.10" {
      group "IBGP-Core"
      import {
        policy ["Selective-Edge-PIC"]
      }
    }
  }
}

```

By checking the route-table for prefixes 172.16.0.0/20 and 172.16.32.0/20 it can be seen that the prefix 172.16.0.0/20 has been programmed with a primary and backup (denoted by the [B] flag), whereas the prefix 172.16.32.0/20 only has a primary path. As a result, it can be derived that the selective installation of backup paths has been successful.

Output 9-16: Route-Table at R3 with selective edge PIC

```

A:admin@R3# show router bgp routes 172.16.0.0/16 longer
=====
BGP Router ID:192.0.2.3      AS:64496      Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                l - leaked, x - stale, > - best, b - backup, p - purge,
                w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP IPv4 Routes
=====
Flag  Network                               LocalPref  MED
      Nexthop (Router)                     Path-Id     IGP Cost
      As-Path                               Label

```



```

-----
u*>i 172.16.0.0/20          100      None
      192.0.2.1            196      200
      64511                -
ub*i 172.16.0.0/20          100      None
      192.0.2.4            101      300
      64511                -
u*>i 172.16.32.0/20         100      None
      192.0.2.1            207      200
      64511                -
-----
Routes : 3

```

The **install-backup-path** command can be used in route-policy associated with BGP import policies and VRF import policies. When a BGP import policy is applied to a BGP peer within a VPRN, the **install-backup-path** command applies purely to unlabeled IPv4 and IPv6 routes. When a BGP import policy is applied to a base router BGP peer the **install-backup-path** command applies to labeled and unlabeled IPv4 and IPv6 prefixes, and to VPN-IPv4 and VPN-IPv6 prefixes. In the case of the latter, in order for the base BGP instance to apply policy to VPN-IPv4/VPN-IPv6 prefixes, the command **vpn-apply-import** must also be configured.

Minimum Route Advertisement Interval

The base BGP `MinRouteAdvertisementIntervalTimer` (MRAI) determines the minimum amount of time that must elapse between an advertisement and/or withdrawal of routes to a particular destination by a BGP speaker to a peer. The timer is intended to reduce the amount of control plane churn and to allow for efficient packing of NLRI into BGP Update messages where multiple NLRI are carried in a single Update message (up to the TCP MSS). It operates on a per-peer basis and is implemented in SR-OS with the **min-route-advertisement** command entered in seconds, with a default of 30 seconds. The BGP specification [1] states that two Update messages sent by a BGP speaker to a peer that advertise feasible routes and/or withdrawal of unfeasible routes must be separated by at least the **min-route-advertisement** timer for that peer. This is not strictly enforced by SR-OS, which empties its TCP socket buffer at every **min-route-advertisement** interval. This may result in multiple Update messages being sent, but once the TCP socket buffer is emptied, any further Updates are enqueued until the next **min-route-advertisement** timer expires.

Because an Update message is subject to the MRAI, it clearly has an impact on convergence times. This can be seen in *Debug 9-5* where ten prefixes are advertised into the router from an external peer but are not re-advertised internally (frame 1997) until the MRAI timer expires (frame 1996). In addition, the ten prefixes are efficiently packed into a single Update message.

Debug 9-5: MRAI and Update packing

```

1996 2025/09/17 12:02:16.464 BST minor: DEBUG #2001 Base BGP
BGP: TIMER
Peer VR 1: Group IBGP-Core: Peer 192.0.2.10 - MinRouteAdvertisementInterval Timer Expired

```

```

1997 2025/09/17 12:02:16.465 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 27
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 6 AS Path:
    Type: 2 Len: 1 < 64511 >
  Flag: 0x40 Type: 3 Len: 4 Nexthop: 192.0.2.1
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
  NLRI: Length = 80
    172.16.10.0/24 Path-ID 21
    172.16.9.0/24 Path-ID 22
    172.16.8.0/24 Path-ID 23
    172.16.7.0/24 Path-ID 24
    172.16.6.0/24 Path-ID 25
    172.16.5.0/24 Path-ID 26
    172.16.4.0/24 Path-ID 27
    172.16.3.0/24 Path-ID 28
    172.16.2.0/24 Path-ID 29
    172.16.1.0/24 Path-ID 30

```

SR-OS allows for configuration of the MRAI at the BGP, group, and neighbour levels for both global and IP-VRF BGP instances. Setting the correct value for the MRAI essentially represents a trade-off. The default 30-second configuration allows batching of multiple NLRIs into fewer Update messages but obviously implies a delay in propagation of those messages of up to 30 seconds at each BGP speaker that processes them. A more aggressive MRAI value of, for example, 1 or 2 seconds implies faster convergence but at the cost of a loss of NLRI packing and increased BGP control plane activity.

Output 9-17: MRAI configuration

```

router "Base" {
  bgp {
    group "EBGP" {
      min-route-advertisement 10
    }
    group "IBGP" {
      min-route-advertisement 5
    }
  }
}

```

The MRAI runs as a ‘time-window’ and each configured MRAI runs independently. In other words, assume router R1 receives a BGP Update for prefix ‘P’ from an external peer and has an MRAI of five seconds configured for its IBGP peers. The prefix ‘P’ is propagated into the Autonomous System somewhere between 0 and 5 seconds.

Rapid Withdrawal

By default, the configured MRAI applies to advertisement of both feasible routes and unfeasible routes (withdraws). Frequently it is desirable that withdraws are propagated more quickly than Update messages containing feasible routes in order to avoid black-holes. The command **rapid-withdrawal** disables the **min-route-advertisement** interval when a BGP speaker has one or more Update messages to send containing the withdrawal of unfeasible routes. It can only be enabled at global BGP level within the base routing context and within a VPRN and applies to all Address Families. The intention of this command is to speed up

reconvergence during a failure condition at the cost of increased control plane activity and potential route oscillation.

Output 9-18: Rapid-Withdrawal configuration

```
router "Base" {
    bgp {
        rapid-withdrawal true
    }
}
```

Rapid-Update

The **rapid-update** command is again intended to speed up reconvergence. The command disables the **min-route-advertisement** interval when a BGP speaker has one or more Update messages to send containing feasible routes. Unlike its counterpart the **rapid-withdrawal**, the **rapid-update** command can be configured to be selective about which Address Families it applies to and has support for the VPN-IPv4/IPv6, MVPN-IPv4/IPv6, MDT-SAFI, Label-IPv4/IPv6, L2VPN, MCAST-VPN-IPv4/IPv6, and EVPN Address Families. It is however only supported at global level in the base routing context. The **rapid-update** command is only applicable to the configured Address Families. All other Address Families continue to have their Update messages containing feasible routes sent at the interval determined by the **min-route-advertisement** command.

Output 9-19: Rapid-Update configuration

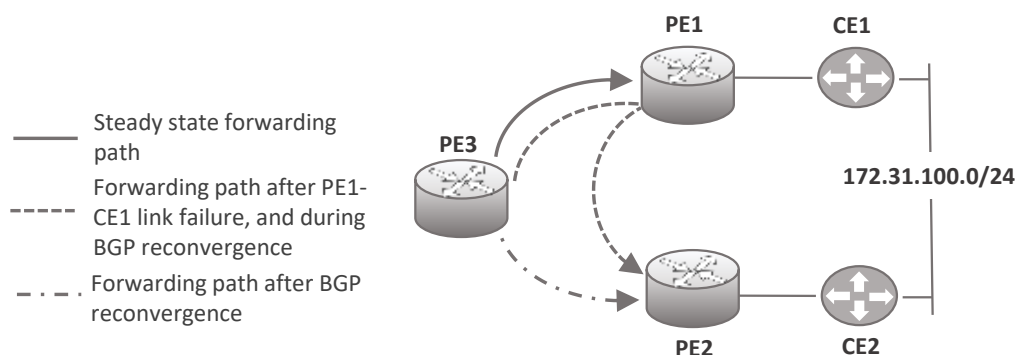
```
router "Base" {
    bgp {
        rapid-update {
            label-ipv4 true
            label-ipv6 true
        }
    }
}
```

Update Propagation

When considering the correct MRAI timers to use and whether it should be accompanied with **rapid-update** or **rapid-withdrawal**, consideration needs to be given to other mechanisms that may negate the requirement for an aggressive MRAI, as this will almost certainly increase the control plane load. These mechanisms may already be in place. Mechanisms such as ADD-PATH, or multiple Route-Distinguishers for VPN-IPv4/IPv6 in conjunction with edge PIC ensure that an alternate path is available for fast convergence. If the failure notification is provided by Next-Hop Tracking there is no dependency on rapid BGP Update propagation. The cost of implementing mechanisms such as ADD-PATH or edge PIC may come elsewhere (for example, increased memory consumption or FIB space), but there will be a compromise that meets the networks requirements.

Even where the failure notification is advertised by BGP, an aggressive MRAI is not always needed. A service that frequently calls for fast reconvergence is IP-VPN, for which, by default, SR-OS uses a label-per-VRF approach when signalling VPN-IPv4/VPN-IPv6 prefixes. Because there may be more than one site of a VPN connected to a PE router, this label-per-VRF approach dictates a requirement for an IP lookup on egress before forwarding packets to the preferred CE device (it is not possible to simply label-switch towards the CE as there is a single label but potentially multiple CE devices). This egress IP lookup provides the capability to protect traffic during a PE-CE link failure. Consider the scenario in *Figure 9-8* where CE1 and CE2 are dual-homed to PE1 and PE2 respectively and both advertise prefix 172.31.100.0/24. PE1 and PE2 have different Route-Distinguishers, and as a result PE3 has two paths for prefix 172.31.100.0/24. In steady state PE3 prefers PE1 as the Next-Hop to reach prefix 172.31.100.0/24. If the PE1-CE1 link fails, PE3 will continue to forward traffic towards PE1 until BGP has reconverged and PE3 selects PE2 as the new preferred Next-Hop. When this pre-convergence traffic arrives at PE1, it pops the label stack and performs a VRF table IP lookup for prefix 172.31.100.0/24. The Next-Hop via CE1 has gone, so the active preferred Next-Hop is now PE2. Consequently, PE1 re-imposes a two-level label stack and forwards the traffic towards PE2, where it is subsequently forwarded to CE2. This ‘U-turn switch’ will maintain traffic continuity of in-flight packets until PE3 receives the BGP withdraw (MP_UNREACH_NLRI) from PE1 for VPN prefix 172.31.100.0/24 and updates its Next-Hop towards PE2.

Figure 9-8: Egress IP Lookup and PE-CE Link Failure



The egress IP lookup (or ‘U-turn switch’) minimizes service disruption during PE-CE link failure and also means that any outage experienced is not dependent on the configured MRAI. However, it relies on an egress IP lookup to function. If, for example, the failure was an ASBR-to-ASBR link where traffic is label-switched, it would be ineffective, and the outage length would be determined by other factors.

Delayed Route Advertisement

Whenever BGP in the base routing instance or an IP-VRF is initialised a number of steps are required to resynchronise its BGP RIB with the BGP RIB of its BGP peers. When this process

is complete the router's BGP instance is said to have converged. The sequence is typically as follows:

- The restarting router must re-establish all of its peering sessions (both configured and auto-discovered).
- The restarting router must relearn all BGP routes advertised by its direct BGP neighbours, which includes not just their best paths but potentially some additional paths.
- The restarting router must advertise to its direct neighbours its own locally originated BGP routes plus the received routes that are considered the best paths following the best-path selection process.

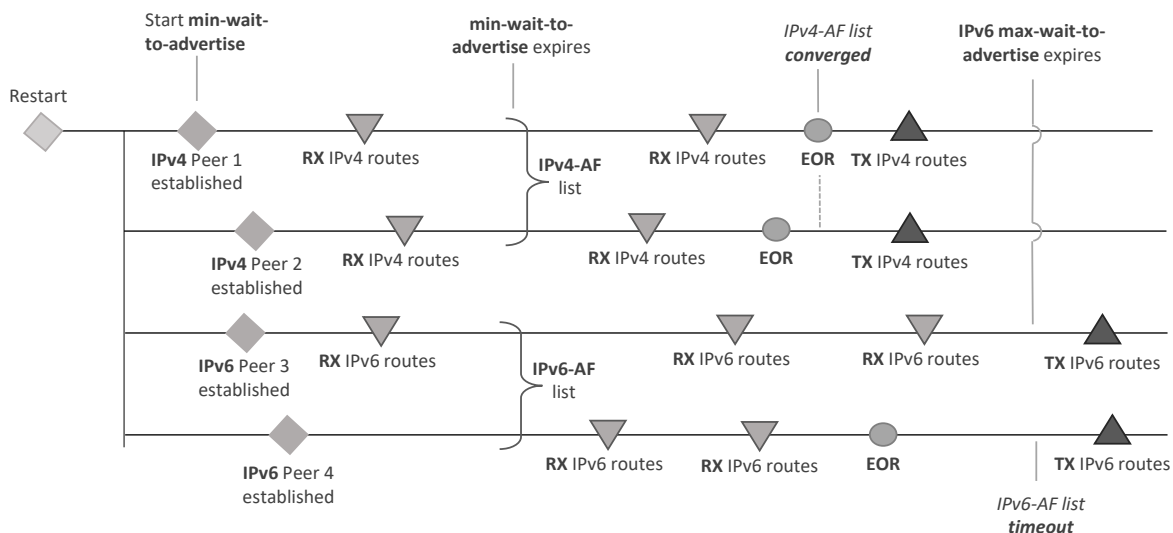
The default behaviour of a router is to execute all of the above points in parallel. That is, as soon as the first BGP session has been re-established, the restarting router starts to advertise its own best paths to the BGP neighbour, even though it is still in the early stages of rebuilding its own RIB-In. As more and more BGP sessions are re-established and more routes are learned it's very likely that routes previously considered best are no longer best, leading to multiple route advertisements for the same prefix: each one of these re-advertisements incrementally better than the previous until the very last advertisement. This continual re-advertisement of routes can substantially increase the processing load not only on the restarting router, but also on its peers. It also lengthens the overall convergence time and can cause short-term suboptimalities in traffic forwarding.

When a router's BGP instance is restarting, a more considered approach might be to delay advertisement of all BGP routes in a given Address Family until such time that convergence has been reached for that family, or until a configurable time limit has expired. This is the purpose of the delayed route advertisement feature in SR-OS. Whenever BGP in the base routing instance or in an IP-VRF is restarted either following a system restart, or being administratively enabled, or simply a 'clear router bgp protocol' command, the convergence state for each Address Family enters a *waiting* state. When the first BGP peer is re-established, a timer known as the **min-wait-to-advertise** is started and the convergence state moves from waiting to *partial*. The **min-wait-to-advertise** timer has a configurable range of [5..3600] seconds with a default of 30 seconds, and while the timer is running no routes are advertised to any peer. When the timer expires, the BGP instance forms a number of lists, one for each of the supported Address Families, which includes IPv4, IPv6, Label-IPv4, Label-IPv6, VPN-IPv4, and VPN-IPv6. (Any BGP session that is re-established after the **min-wait-to-advertise** timer has expired cannot be added to the list.) For each Address Family list, the BGP instance expects to learn all of the routes from all of the peers in that list, followed by an End of RIB (EOR) [44] marker from the peer for that Address Family to indicate that it has completed advertising all of its routes. When this has occurred, the local BGP instance enters a state of *converged* for that Address Family. Once that converged state has been reached, the router begins to advertise its own RIB-Out routes for that Address Family to all of its established peers (regardless of whether they are in the current Address Family list or not). This process is repeated for each Address Family list.

At the time when the first peer for a given Address Family is re-established, a second timer known as the **max-wait-to-advertise** is also started, which again can be configured in the range [5..3600] seconds with a default equal to three times the **min-wait-to-advertise** timer. When the **max-wait-to-advertise** timer expires for a given Address Family list, the router starts advertising all of its own RIB-Out routes for all of its established peers even if it has not yet received an EOR marker from all of the peers in the current Address Family list.

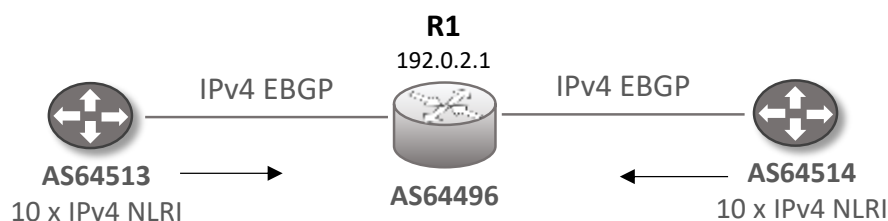
Figure 9-9 depicts the timelines described above. When peer 1 is established following the restart the **min-wait-to-advertise** timer is started. While this timer is running four peers are established, two with IPv4 Address Family and two with IPv6 Address Family. Some may start to advertise routes whilst the **min-wait-to-advertise** timer is running, but when it the timer expires the peers are organised into two Address Family lists, the `ipv4-address-family` list and the `ipv6-address-family` list. For each Address Family list BGP expects to receive routes from its peers followed by an EOR marker for that Address Family. Peer 1 and peer 2 both advertise their IPv4 routes and follow it with an EOR marker. As a result, the `ipv4-address-family` list moves into a state of converged and the system starts to advertise its RIB-Out routes to peer 1 and peer 2. In the `ipv6-address-family` list peer 4 advertises its IPv6 routes followed by an EOR marker, but peer 3 has not sent an EOR marker by the time that the `ipv6-address-family` list **max-wait-to-advertise** expires. As a result, the convergence state moves to *timeout* and the system advertises its IPv6 routes to all established peers including those that are not in the `ipv6-address-family` list.

Figure 9-9: Delayed route advertisement example timeline



To illustrate how delayed route advertisement works in SR-OS the simplified schematic shown in *Figure 9-10* is used. Router R1 is the device under test and has two external BGP peers, one in AS 64513 and one in AS 64514. Both external peers have 10 x IPv4 NLRI that are advertised to router R1 who re-advertises the routes as per standard BGP procedures.

Figure 9-10: Test Topology for delayed route advertisement



The necessary configuration for delayed route advertisement applied at router R1 is shown in *Output 9-20* and is enabled within the **convergence** context. Within that context the **min-wait-to-advertise** is configured and is a system-wide parameter (that is, it is not Address Family specific). Each Address Family is then configured with the **max-wait-to-advertise** parameter under the relevant **family** context. As the test topology only uses IPv4 NLRI only the **family ipv4** is configured. The timer values used here are not determined by any particular formula or platform constraint and serve mainly to allow the relevant outputs to be gathered showing the various convergence states of the router.

Output 9-20: Delayed route advertisement configuration

```

router "Base" {
  bgp {
    convergence {
      min-wait-to-advertise 60
      family ipv4 {
        max-wait-to-advertise 180
      }
    }
  }
}

```

Output 9-21 shows the initialisation of BGP with the '*clear router bgp*' command and follows it by displaying the BGP convergence output for the IPv4 Address Family. At this point the convergence is in a **waiting** state following the initialisation and there are currently no peers established. The display also shows the operational system-wide **min-wait-to-advertise** and Address Family specific **max-wait-to-advertise** values.

Output 9-21: BGP initialisation at R1

```

A:admin@R1# clear router bgp protocol

[/]
A:admin@R1# show router bgp convergence | match 'BGP IPv4 Convergence' post-lines 11
BGP IPv4 Convergence
=====
Min wait advertise timer                : 60
Established peers at min wait timer expiry : 0
Current established peers                : 0
First session established time           : 00h00m00s
Last session established time            : 00h00m00s
Max Wait advertise timer                 : 180
Converged peers                          : 0
Converged state                          : waiting
Converged time                           : N/A
=====

```

The external peer at AS 64513 is started and establishes its EBGP session with router R1. As soon as the first peer is established the **min-wait-to-advertise** timer is started and the convergence state moves from waiting to partial.

Output 9-22: Convergence state after EBGP session with AS 64513 established

```
A:admin@R1# show router bgp convergence | match 'BGP IPv4 Convergence' post-lines 11
BGP IPv4 Convergence
=====
Min wait advertise timer           : 60
Established peers at min wait timer expiry : 0
Current established peers          : 1
First session established time      : 00h00m17s
Last session established time       : 00h00m17s
Max Wait advertise timer           : 180
Converged peers                    : 1
Converged state                    : partial
Converged time                     : N/A
=====
```

The external peer at AS 64514 is now started and also establishes its EBGP session with router R1. There are now currently two established peers. The converged state remains partial, and the **min-route-advertisement** timer continues to run.

Output 9-23: Convergence state after EBGP peering with AS 64514 established

```
A:admin@R1# show router bgp convergence | match 'BGP IPv4 Convergence' post-lines 11
BGP IPv4 Convergence
=====
Min wait advertise timer           : 60
Established peers at min wait timer expiry : 0
Current established peers          : 2
First session established time      : 00h00m17s
Last session established time       : 00h00m35s
Max Wait advertise timer           : 180
Converged peers                    : 2
Converged state                    : partial
Converged time                     : N/A
=====
```

At this point, it's worth taking a look at the summary view of the BGP peering status. As shown in *Output 9-24* there are two peers established, and both have negotiated the IPv4 Address Family. Router R1 has received 10 routes from each peer, and 10 routes are active as both show 10/10/0 in the Rcv/Act/Sent column. However, no routes have been re-advertised by router R1 and will not be until the **min-route-to-advertise** timer has expired, and both external peers have advertised all the routes in their RIB-Out followed by an EOR marker.

Output 9-24: BGP peering summary at the partial converged state

```
A:admin@R1# show router bgp summary | match 'BGP Summary' post-lines 13
BGP Summary
=====
Legend : D - Dynamic Neighbor
=====
Neighbor
Description
AS PktRcvd InQ Up/Down State|Rcv/Act/Sent (Addr Family)
PktSent OutQ
-----
```



```

192.168.1.2
        64513      6    0 00h00m38s 10/10/0 (IPv4)
                    5    0
192.168.1.6
        64514      5    0 00h00m21s 10/10/0 (IPv4)
                    3    0

```

When the **min-route-to-advertise** timer has expired and all peers in the ipv4-address-family list (AS 64513 and AS 64514) have sent their RIB-Out contents followed by an EOR marker for the IPv4 Address Family, the convergence state transitions to **converged**.

[/]

Output 9-25: BGP convergence state after EOR marker received from all IPv4 peers

```

A:admin@R1# show router bgp convergence | match 'BGP IPv4 Convergence' post-lines 11
BGP IPv4 Convergence
=====
Min wait advertise timer           : 60
Established peers at min wait timer expiry : 2
Current established peers          : 2
First session established time      : 00h00m17s
Last session established time       : 00h00m35s
Max Wait advertise timer           : 180
Converged peers                    : 2
Converged state                     : converged
Converged time                      : 00h01m00s
=====

```

Once the router has reached the **converged** state a repeat of the BGP peering summary shows that both routers show 10/10/10 in the Rcv/Act/Sent column. R1 waited until all the peers in the ipv4-address-family list advertised the contents of their respective RIB-Out entries and sent an EOR marker before reaching the converged state. Following this, R1 was able to start advertising its best paths towards its peers.

Output 9-26: Summary of BGP peers after R1 reaches the converged state

```

A:admin@R1# show router bgp summary | match 'BGP Summary' post-lines 13
BGP Summary
=====
Legend : D - Dynamic Neighbor
=====
Neighbor
Description
          AS PktRcvd InQ  Up/Down  State|Rcv/Act/Sent (Addr Family)
          PktSent OutQ
-----
192.168.1.2
        64513      7    0 00h01m07s 10/10/10 (IPv4)
                    7    0
192.168.1.6
        64514      6    0 00h00m49s 10/10/10 (IPv4)
                    6    0

```

Finally, it's worth mentioning that the '*show router bgp neighbor*' command has been extended to include a few lines on convergence. The **Exceeded min-wait** field supports Yes, No, and Unknown values. Yes means that the neighbour session peering session occurred after the last **min-wait-to-advertise** timer expired (and as a result was not placed into the relevant convergence Address Family list(s)). Conversely, No means that the neighbour session was established before the **min-wait-to-advertise** timer expired. If When

Unknown is displayed it means that the **min-wait-to-advertise** timer has not yet been reached since the last BGP initialisation. The `Sent-EOR` and `Recv-EOR` fields indicate the list of Address Families for which an EOR was either received from a neighbour or sent to a neighbour.

Output 9-27: BGP Neighbour output with convergence status

```
A:admin@R1# show router bgp neighbor 192.168.1.2 | match Convergence post-lines 1
Convergence Sent-EOR : ipv4
Convergence Recv-EOR : ipv4
Exceeded min-wait    : No
```

Useful log events are included to indicate when a given Address Family transitions from partial to the converged state, as well as when the max-wait-to-advertise timer expires with the Address Family is still in the partial state. These events may be useful to help derive the correct values to use to optimise the convergence process.

Output 9-28: Error logs showing move to converged state

```
18254 2025/09/18 07:57:55.094 BST MINOR: BGP #2042 Base VR 1
"Convergence state for family ipv4 transitioned from partial to converged"
```

TCP Maximum Segment Size (MSS)

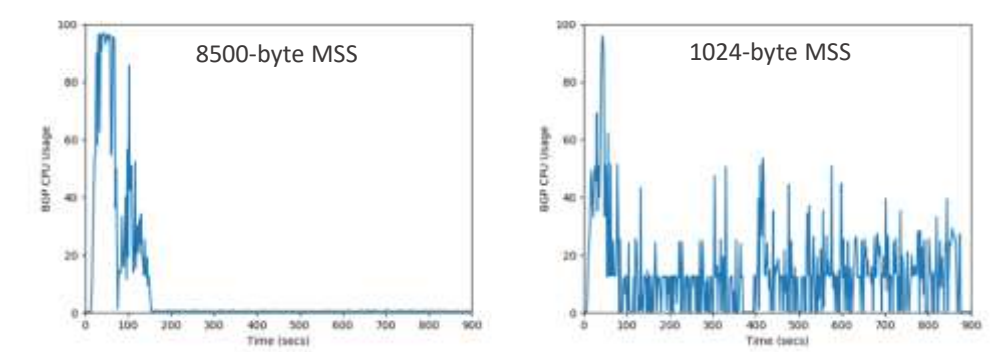
The TCP Maximum Segment Size (MSS) is a 16-bit TCP Kind (Kind 4) used in the three-way initial TCP handshake to communicate the receive segment size at the TCP stack at the sending size of the TCP connection. In other words, the MSS represents the largest segment size that the advertising router is willing to receive. In a TCP connection, larger TCP segment sizes generally deliver greater throughput on the understanding that the underlying network path supports the corresponding Maximum Transmission Unit (MTU) without requiring fragmentation. This is because a larger segment size requires less TCP/IP overhead and yield, requires less packets to process, and results in the most efficient use of the data payload within each transmitted segment.

As BGP is carried over TCP, the MSS value is of course applicable and by default, SR-OS uses an MSS value of 1024-bytes for IPv4 peering sessions and 1220-bytes for IPv6 peering sessions. In some (not all) circumstances, the MSS can have a significant impact on BGP convergence in scaled setups. As the size of the supported TCP segment increases, so does the size of Update messages contained within that segment, which in turn means that more NLRI can be packed into one or more of those Update messages. The original BGP standard [1] specified the maximum segment size for BGP should be 4096-bytes. This was subsequently updated by the BGP Extended Message [86] that increased the maximum message size from 4096-bytes to 65,535-octets for all messages except for Open and Keepalive messages. To support this increased message size, [86] also specified that peers should include the BGP Extended Message capability (Code 6) in the Open exchange,

however, this is not a mandatory requirement and is therefore not currently advertised by SR-OS. SR-OS does however support the extended message length.

Figure 9-11 shows the results of an indicative reasonably scaled convergence test (the details of which are largely irrelevant to the discussion) using two notably different MSS values. The left graph shows the convergence results using an MSS value of 8500-bytes while the right graph shows the convergence results using the default 1024-bytes. The time to converge (the time when the last BGP route is learned) is shown on the 'X' axis while the 'Y' axis shows the CPU consumption during the test. In this particular test case the results are overwhelmingly in favour of a larger MSS. This may not always be the case, and the intention is certainly not to present MSS as the “silver bullet” of convergence. It is not.

Figure 9-11: TCP MSS impact on convergence



The procedure for determining MSS during connection setup is as follows:

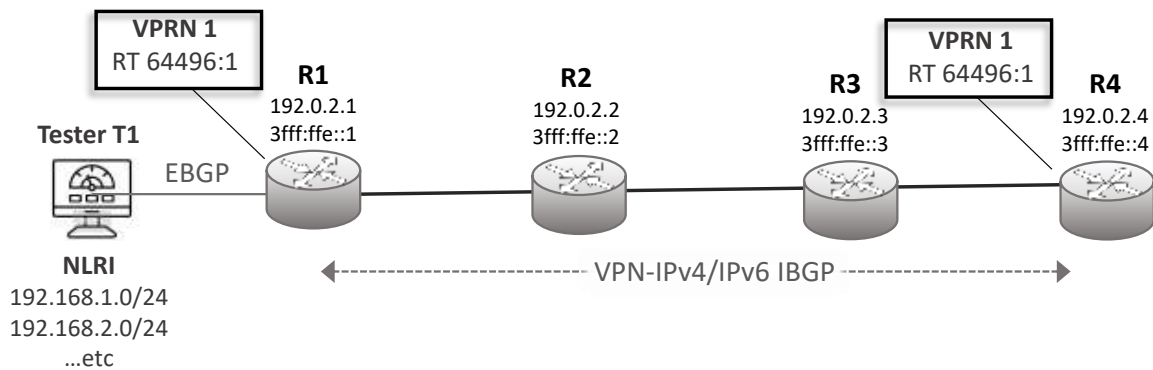
- The initiating host sends a SYN packet containing the TCP MSS kind with a value that equals the largest segment size it is willing to receive. This is typically the MTU of the outgoing interface minus 40-bytes TCP/IP overhead for IPv4 connections and minus 60-bytes TCP/IP overhead for IPv6 connections.
- The receiving host performs a similar MSS calculation and returns the SYN/ACK packet with an MSS value indicating the largest segment size it is willing to receive.
- Although the MSS can be considered a unidirectional value, typically each peer uses the smallest of the advertised MSS values to determine its maximum segment size that it can send for that connection.

Test Topology

To test and validate the use of extended message support and manipulation of TCP MSS the simple topology shown in *Figure 9-12* is used. Routers R1 to R4 belong to a single IS-IS level 2 domain with an internal BGP peering session extended between routers R1 and R4 supporting the VPN-IPv4 and VPN-IPv6 Address Families. This BGP session uses IPv4 and IPv6 transport throughout this section to illustrate the difference when path MTU is used. An IP-VRF is provisioned on routers R1 and R4, with a tester T1 connected to R1 to advertise large numbers of IPv4 and IPv6 BGP routes to validate that NLRI are efficiently packed into VPN-

IPv4/IPv6 Update messages according to the operational MSS of the peering session. The operational IP-MTU of all links is 9194-bytes unless otherwise specified.

Figure 9-12: Test topology for TCP MSS



SR-OS allows users to modify the default TCP MSS either statically or dynamically through the process of path MTU discovery. Both approaches and their usage are discussed in the following sub-sections.

Static TCP MSS

SR-OS supports the configuration of a static TCP MSS using the **tcp-mss** command at both group and neighbour level within the range 384-bytes to 9746-bytes. It is expected to be the same value at either end of the TCP connection and should allow for 40-bytes TCP/IP overhead for IPv4 sessions and 60-bytes TCP/IP overhead for IPv6 sessions. Static configuration of TCP MSS should be used in environments where the MTU is controlled and administered end-to-end and is not likely to change for any reason. If **tcp-mss** is configured on a BGP session that is in the Established state, it will not take effect until the BGP session is re-established and the TCP three-way handshake takes place again.

Output 9-29: Configuration for static TCP-MSS

```
router "Base" {
  bgp {
    neighbor "192.0.2.4" {
      admin-state enable
      group "PMTU"
      tcp-mss 4096
      authentication-key "C3i5c6IP3VMhQFGff3RCM5QVKE0RvXM= hash2"
    }
  }
}
```

Figure 9-13 shows the three-way TCP handshake using the static 4096-byte MSS configured for the IPv4 peering session above. The first packet shows the SYN from R1 with an MSS value of 4096, which is followed in the second packet with the SYN/ACK from R4 also containing an MSS of 4096. After the ACK in packet three, the TCP connection is passed to the application and moves to the BGP Open exchange.

Figure 9-13: Three-way TCP handshake with 4096-byte MSS

TCP	94 49698 → 179 [SYN] Seq=0 Win=32768 Len=0 MSS=4096 WS=1 TSval=2844096925 TSecr=0 MD5
TCP	94 179 → 49698 [SYN, ACK] Seq=0 Ack=1 Win=32768 Len=0 MSS=4096 WS=1 TSval=3892702714 TSecr=2844096925 MD5
TCP	90 49698 → 179 [ACK] Seq=1 Ack=1 Win=32768 Len=0 TSval=2844096925 TSecr=3892702714 MD5
BGP	153 OPEN Message
BGP	153 OPEN Message
TCP	90 179 → 49698 [ACK] Seq=64 Ack=64 Win=32768 Len=0 TSval=3892702714 TSecr=2844096925 MD5
TCP	90 49698 → 179 [ACK] Seq=64 Ack=64 Win=32768 Len=0 TSval=2844096925 TSecr=3892702714 MD5
BGP	109 KEEPALIVE Message

Unlike path MTU discovery, when a static MSS is configured IPv4 packets are not set with 'Do not Fragment' in the IP header, hence any BGP packets that exceed an MTU along the path between source and destination will be subject to fragmentation. *Output 9-30* shows the operational MSS of R1's BGP peering session to R4 with the same **tcp-mss** value of 4096 applied at both nodes.

Output 9-30: Operational MSS of R1-R4 BGP session

```
A:admin@R1# show router bgp neighbor 192.0.2.4 | match Mss
Tcp Mss           : 4096
Tcp Rcvd Mss      : 4096
```

The operational MSS can also be seen within the system connections output, which also shows the received MSS (RcvdMss) and the operational MSS (OperMss).

Output 9-31: System connections view of R1-R4 BGP session

```
A:admin@R1# show system connections

=====
Connections
=====
Prot   RecvQ   TxmtQ Local Address                      State
      RcvdMss OperMss Remote Address                      vRtrID
-----
--- [snip] ---
TCP    0        0 192.0.2.1.49698                    ESTABLISH
      4096    4096 192.0.2.4.179                      1
--- [snip] ---
```

Tester T1 connected to router R1 is then used to advertise a large number of IPv4 and IPv6 NLRI that R1 subsequently advertises into VPN-IPv4/IPv6 routes. *Debug 9-6* shows one of the Update messages sourced by R1 containing an MP_REACH NLRI with a number of VPN-IPv4 prefixes. Of note however is the total path attribute length which shows a value of 4058 bytes indicating that the operational MSS is being utilised by BGP.

Debug 9-6: VPN-IPv4 Update message from R1

```
109 2026/01/11 11:42:41.477 GMT minor: DEBUG #2001 Base Peer 1: 192.0.2.4
Peer 1: 192.0.2.4: UPDATE
  Withdrawn Length = 0
  Total Path Attr Length = 4058
  Flag: 0x90 Type: 14 Len: 4007 Multiprotocol Reachable NLRI:
    Address Family VPN_IPV4
    NextHop len 12 NextHop 192.0.2.1
    192.169.245.0/24 RD 64496:1 Label 524284 (Raw label 0x7fffc1)
    192.169.244.0/24 RD 64496:1 Label 524284 (Raw label 0x7fffc1)
    192.169.243.0/24 RD 64496:1 Label 524284 (Raw label 0x7fffc1)
  --- {etc} ---
```

Figure 9-14 shows a packet capture of the BGP Update messages sent from router R1 in a single BGP packet. The packet in the figure is 3633-bytes in length (3615-bytes at the IP layer) but is the second part of a reassembled TCP segment totalling 7697-bytes. The first segment previously sent in frame 48 was 4064-bytes while this frame, frame 50, is 3633-bytes. In short, there is reassembly done at the TCP layer, but the MSS is being utilised to the maximum.

Figure 9-14: Packet capture of MP_REACH NLRI with static MSS of 4096-bytes

```

* Frame 50: 3723 bytes on wire (29784 bits), 3723 bytes captured (29784 bits) on interface br11, id 0
* Ethernet II, Src: RealtekU_d3:64:b6 (52:54:00:d3:64:b6), Dst: RealtekU_10:ca:a4 (52:54:00:10:ca:a4)
* 802.1Q Virtual LAN, PRI: 7, DEI: 0, ID: 100
* Internet Protocol Version 4, Src: 192.0.2.1, Dst: 192.0.2.4
* Transmission Control Protocol, Src Port: 49597, Dst Port: 179, Seq: 4065, Ack: 1, Len: 3633
  Source Port: 49597
  Destination Port: 179
  [Stream index: 3]
  [Conversation completeness: Incomplete (12)]
  [TCP segment len: 3633]
  Sequence Number: 4065 (relative sequence number)
  Sequence Number (raw): 1689986640
  [Next Sequence Number: 7698 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 3984595314
  1101 ... = Header Length: 52 bytes (13)
  * Flags: 0x018 (PSH, ACK)
  Window: 32768
  [Calculated window size: 32768]
  [Window size scaling factor: -1 (unknown)]
  Checksum: 0x6383 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  * Options: (32 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps, TCP MD5 signature, No-Operation (NOP), No-Operation (NOP)
  * [Timestamps]
  * [SEQ/ACK analysis]
    [Bytes in flight: 3633]
    [Bytes sent since last PSH flag: 7697]
    TCP payload (3633 bytes)
    TCP segment data (3633 bytes)
  * [2 Reassembled TCP Segments (7697 bytes): #48(4064), #50(3633)]
* Border Gateway Protocol - UPDATE Message
* Border Gateway Protocol - UPDATE Message

```

Path MTU Discovery

When an IP host has a large amount of data to send to another IP host, the most optimum method of transferring that data is to use the largest possible IP datagram that avoids fragmentation anywhere along the path. Inherently IP has no way to discover what that datagram size is, and this is the purpose of Path MTU Discovery [85]. When using path MTU discovery, the process differs slightly for IPv4 and IPv6:

- For IPv4 the host initially sends IP packets using the MTU of its first hop with all packets all having the Do-not-Fragment (DF) bit set. If any of the packets are too large to be forwarded without fragmentation by a router along the path to the destination, it will discard the packet and generate an ICMP message back to the source with the type “Destination Unreachable” (Type 3) and the Code “Fragmentation Needed and DF-bit Set” (Code 4). This ICMP message also contains the MTU of the outgoing interface so that the sending host can adjust its MTU accordingly.
- IPv6 has no concept of fragmentation or a Do-Not-Fragment bit, hence the process is slightly different. The host simply sends packets using the MTU of its first hop and if any of the packets are too large to be forwarded by a router on a given link, it will simply generate an ICMPv6 message back to the source with the type “Packet Too Big” (Type 2). Again, this ICMPv6 message contains the MTU of the outgoing interface so that the sending host can adjust its MTU accordingly.

When TCP is used between sender and receiver and path MTU discovery is in use, the TCP Maximum Segment Size (MSS) advertised in the SYN and SYN-ACK exchange must allow for the additional IP overhead. That is, the TCP MSS is defined to be the relevant IP packet size minus 40-bytes for IPv4 and 60-bytes for IPv6, representing the IP and TCP headers. For both IPv4 and IPv6 a liberal or optimistic position is taken with the TCP header length and [84] suggests that a value of 20-bytes is used, which is the minimum TCP header length and does not allow for any TCP options. Very often the TCP header will be longer, and when TCP authentication is used between peers (which accounts for the majority if not the entirety of peers in production networks), the TCP header length will certainly be longer. For example, TCP MD5 authentication uses the TCP Signature option (Kind 19) and outputs a 16-byte digest giving a total option length of 18-bytes. TCP-AO uses the TCP-AO option (Kind 29) and [56] specifies the use of either the AES-128-CMAC-96 MAC algorithm, which produces a 128-bit MAC, or the HMAC SHA-1 algorithm which produces a 160-bit output. Both algorithms are truncated to 12-bytes to yield a total option of 14-bytes. When either of these algorithms are in use and/or the TCP header length is greater than 20-bytes, the TCP stack must reduce the size of any TCP segments accordingly.

As the process for path MTU discovery differs slightly, this section provides two short examples: one with an IPv4 peering session, and one with an IPv6 peering session.

IPv4 Path MTU Discovery

The configuration to enable path MTU discovery for IPv4 is shown in *Output 9-32* and is applied to both R1 and R4. It consists solely of the **path-mtu-discovery** command which can be applied at group or neighbor level.

Output 9-32: IPv4 BGP peering with path MTU discovery

```
router "Base" {
  bgp {
    neighbor "192.0.2.4" {
      admin-state enable
      group "PMTU"
      authentication-key "C3i5c6IP3VMhQFGff3RCM5QVKE0RvXM= hash2"
      path-mtu-discovery true
    }
  }
}
```

Like the static TCP MSS configuration, the TCP MSS determined by path MTU discovery will not take effect until the TCP three-way handshake takes place again, which for a BGP peer in the Established state means a down/up transition to pick up the new MSS value. *Figure 9-15* shows the TCP three-way handshake with path MTU enabled. The first packet shows the SYN packet from R1 with an MSS value of 9154, which is followed in the second packet with the SYN/ACK also containing an MSS value of 9154. The handshake is completed in the third packet with the ACK followed by the BGP Open message. Recall that all interfaces in the test topology have an IP MTU of 9194-bytes, hence the advertised MSS value of 9154-bytes allows for the 40-byte TCP/IP overhead of this IPv4 session.

Figure 9-15: Three-way TCP handshake with path MTU discovery

TCP	84 49923 → 179 [SYN] Seq=0 Win=32768 Len=0 MSS=8154 WS=1 TSval=1979362788 TSecr=0 MD5
TCP	84 179 → 49923 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0 MSS=8154 WS=1 TSval=609671557 TSecr=1979362788 MD5
TCP	90 49923 → 179 [ACK] Seq=1 Ack=1 Win=32768 Len=0 TSval=1979362788 TSecr=609671557 MD5
BGP	153 OPEN Message
BGP	153 OPEN Message
BGP	109 KEEPALIVE Message
BGP	109 KEEPALIVE Message

Output 9-33 shows the resulting operational MSS values for the BGP neighbor R4 as seen at router R1.

Output 9-33: TCP MSS with path MTU discovery enabled

```
A:admin@R1# show router bgp neighbor 192.0.2.4 | match Mss
Tcp Mss           : 9154
Tcp Rcvd Mss      : 9154
```

Tester T1 connected to router R1 is then used to advertise a large number of IPv4 and IPv6 NLRI that R1 subsequently advertises into VPN-IPv4/IPv6 routes. *Figure 9-16* shows a BGP packet sourced by R1 containing a number of Update messages. The total frame size is 9212-bytes at the Ethernet layer, or 9194-bytes at the IP layer. Note that the TCP options field is 32-bytes in length, containing a TCP MD5 signature option, a Timestamps option, and some No-Operation options used to pad to a 32-bit (4-byte) boundary. Also note that the TCP Segment Length is 9102-bytes, meaning that although *Output 9-36* shows the operational TCP MSS as 9134-bytes, reflecting a 40-byte IP overhead and 20-byte TCP overhead, the TCP segment length has been reduced by a further 32-bytes to reflect the additional 32-bytes in the TCP header. It's therefore reasonable to assume that a small amount of the TCP MSS shown in *Output 9-33* is not made available to BGP.

Figure 9-16: BGP Update message with 9154-byte MSS

- Frame 27: 9212 bytes on wire (73696 bits), 9212 bytes captured (73696 bits) on interface Br11, id 0 - Ethernet II, Src: RealtekU_01:0b:e7 (52:54:00:01:0b:e7), Dst: RealtekU_08:d3:9c (52:54:00:08:d3:9c) 802.1Q Virtual LAN, PRI: 7, DEI: 0, ID: 100 - Internet Protocol Version 4, Src: 192.0.2.1, Dst: 192.0.2.4 - Transmission Control Protocol, Src Port: 49923, Dst Port: 179, Seq: 102, Ack: 824, Len: 9122 - Source Port: 49923 - Destination Port: 179 [Stream index: 1] [Conversation completeness: Incomplete, DATA (15)] [TCP Segment Len: 9122] - Sequence Number: 102 (relative sequence number) - Sequence Number (raw): 652358807 [Next Sequence Number: 9224 (relative sequence number)] - Acknowledgment Number: 824 (relative ack number) - Acknowledgment number (raw): 366486829 1101 = Header Length: 52 bytes (13) - Flags: 0x010 (ACK) - Window: 32768 [Calculated window size: 32768] [Window size scaling factor: 1] - Checksum: 0x73ea [unverified] [Checksum Status: Unverified] - Urgent Pointer: 0 - Options: (32 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps, TCP MD5 signature, No-Operation (NOP), No-Operation (NOP) [Timestamps] [SEQ/ACK analysis] 9224 = TCP segment data (9122 bytes) - TCP segment data (9122 bytes) - Border Gateway Protocol - UPDATE Message - Border Gateway Protocol - UPDATE Message

The BGP prefixes advertised by tester T1 are subsequently withdrawn after which the R2-R3 link is configured to have an IP-MTU of 8000-bytes. Tester T1 then readvertises the BGP prefixes with the intention being to advertise Update messages that are too large to be transmitted across this interface and validate the outcome. The result is shown in *Figure 9-17*:

- Frame 26 is a BGP packet containing multiple Update messages and has a total length of 9212-bytes at the Ethernet layer, or 9194-bytes at the IP layer. Frame 27 is a second BGP packet again containing multiple Update messages and this time totalling 8106-bytes at the IP layer.
- This is followed in Frame 28 with an ICMP Destination Unreachable/Fragmentation Needed message from 192.168.0.5 (router R2). An expanded view of this ICMP packet is shown in *Figure 9-18* and shows the Type 3 (Destination Unreachable) Code 4 (Fragmentation Needed) with a Next-Hop MTU field showing the 8000-byte IP MTU of the R2-R3 link. This is what enables the source to adjust its MSS accordingly. This is followed by the IP header and first 64-bytes of the offending packet requiring fragmentation.
- Referring back to the packet sequence in *Figure 9-17*, frame 29 is a TCP retransmission but this time the packet size is 8000-bytes at the IP layer. This packet size was of course determined by the ICMP packet returned by R2.
- The retransmissions continue in frames 30, 32, and 33, but the packet size never goes beyond the IP MTU of 8000-bytes.

Figure 9-17: BGP Updates with lower MTU on R2-R3 link

192.0.2.1	192.0.2.4	BGP	9212 UPDATE Message, UPDATE Message, UPDATE Message
192.0.2.1	192.0.2.4	BGP	8124 UPDATE Message, UPDATE Message, UPDATE Message, UPDATE Message, UPDATE Message, UPDATE Message, UPDATE Message, UPDATE Message
192.168.0.5	192.0.2.1	ICMP	174 Destination unreachable (Fragmentation needed)
192.0.2.1	192.0.2.4	TCP	6018 [TCP Retransmission] 50426 → 179 [ACK] Seq=802 Ack=824 Win=32768 Len=7960 TSval=1377198196 TSecr=2907090385
192.0.2.1	192.0.2.4	TCP	62 [TCP Retransmission] 50426 → 179 [ACK] Seq=8096 Ack=824 Win=32768 Len=12 TSval=1377198196 TSecr=2907090385
192.0.2.4	192.0.2.1	TCP	70 179 → 50426 [ACK] Seq=824 Ack=8062 Win=28995 Len=0 TSval=2907090385 TSecr=1377198196
192.0.2.1	192.0.2.4	TCP	6018 [TCP Retransmission] 50426 → 179 [ACK] Seq=8062 Ack=824 Win=32768 Len=7960 TSval=1377198196 TSecr=2907090385
192.0.2.1	192.0.2.4	TCP	1158 [TCP Retransmission] 50426 → 179 [FIN, ACK] Seq=16010 Ack=824 Win=32768 Len=1200 TSval=1377198196 TSecr=2907090385
192.0.2.4	192.0.2.1	TCP	70 179 → 50426 [ACK] Seq=824 Ack=17298 Win=27921 Len=0 TSval=2907090385 TSecr=1377198196

Figure 9-18: ICMP message with sourced by R2

▶ Frame 28: 174 bytes on wire (1392 bits), 174 bytes captured (1392 bits) on interface br11, id 0 ▶ Ethernet II, Src: RealtekU_08:d3:9c (52:54:00:08:d3:9c), Dst: RealtekU_91:0b:e7 (52:54:00:91:0b:e7) ▶ 802.1Q Virtual LAN, PRI: 7, DEI: 0, ID: 100 ▶ Internet Protocol Version 4, Src: 192.168.0.5, Dst: 192.0.2.1 ▶ Internet Control Message Protocol Type: 3 (Destination unreachable) Code: 4 (Fragmentation needed) Checksum: 0x0dd0 [correct] [Checksum Status: Good] Unused: 0000 MTU of next hop: 8000 ▶ Internet Protocol Version 4, Src: 192.0.2.1, Dst: 192.0.2.4 ▶ Transmission Control Protocol, Src Port: 50426, Dst Port: 179, Seq: 3732089427, Ack: 3710018290 ▶ Border Gateway Protocol

In short, the path MTU discovery process has operated as desired, and the correct MSS adjusted to meet the reduced IP MTU while still maximising the highest MSS possible. This is shown in *Output 9-34* which shows the transmitted MSS as 7960-bytes, which is 8000-bytes after allowing for TCP/IP overhead.

Output 9-34: Operational MSS after R2-R3 link MTU decrease

```
A:admin@R1# show router bgp neighbor 192.0.2.4 | match Mss
Tcp Mss          : 7960
Tcp Rcvd Mss     : 9154
```

IPv6 Path MTU Discovery

When IPv6 transport is used for BGP peering, the default MSS is 1220-bytes, which represents the minimum IPv6 MTU of 1280-bytes minus 60-bytes for TCP/IP overhead. Enabling path MTU discovery for IPv6 is exactly the same as for IPv4 but is shown in *Output 9-35* for completeness and again consists solely of the **path-mtu-discovery** command. Note that TCP MD5 authentication is again used to ensure a TCP header length larger than the minimum 20-bytes.

Output 9-35: IPv6 BGP peering with path MTU discovery

```
router "Base" {
    bgp {
        neighbor "3fff:ffe::4" {
            admin-state enable
            group "PMTU"
            authentication-key "C3i5c6IP3VMhQFGff3RCM5QVKE0RvXM= hash2"
            path-mtu-discovery true
        }
    }
}
```

Figure 9-19 shows the three-way TCP handshake between R1 and R4 using the IPv6 peering. The first packet shows the SYN from R1 with an MSS value of 9134, which is followed in the second packet with the SYN/ACK from R4 also containing an MSS value of 9134. The handshake is completed in the third packet with the ACK followed by the BGP open exchange. As the outgoing interface of both routers is 9194-bytes, the advertised MSS value of 9134-bytes represents the subtraction of a 40-byte IP header and 20-byte TCP header.

Figure 9-19: Three-way TCP handshake with path MTU discovery

16 4.0000000000	3fff:ffe::1	3fff:ffe::4	TCP	80 49565 - 179 [SYN] Seq=0 Win=32768 Len=0 MSS=9134 W=0 T=0
17 4.0000000000	3fff:ffe::4	3fff:ffe::1	TCP	80 179 - 49565 [SYN, ACK] Seq=0 Ack=1 Win=32768 Len=0 MSS=9134 W=0 T=0
18 4.0100000000	3fff:ffe::1	3fff:ffe::4	TCP	80 49565 - 179 [ACK] Seq=1 Ack=1 Win=32768 Len=0 T=0
19 4.0100000000	3fff:ffe::1	3fff:ffe::4	BGP	161 OPEN Message
20 4.0100000000	3fff:ffe::4	3fff:ffe::1	BGP	161 OPEN Message
21 4.0100000000	3fff:ffe::1	3fff:ffe::4	BGP	169 KEEPALIVE Message
22 4.0200000000	3fff:ffe::4	3fff:ffe::1	BGP	169 KEEPALIVE Message

The operational MSS of the IPv6 session can be seen in *Output 9-36*, where the operational MSS of 9134-bytes represents the outgoing interface MTU of 9194-bytes minus 60-bytes TCP/IP overhead.

Output 9-36: TCP MSS with path MTU discovery enabled

```
A:admin@R1# show router bgp neighbor "3fff:ffe::4" | match Mss
Tcp Mss          : 9134
Tcp Rcvd Mss     : 9134
```

Tester T1 connected to router R1 is then used to advertise a large number of IPv4 and IPv6 NLRI that R1 subsequently advertises into VPN-IPv4/IPv6 routes. *Figure 9-20* shows one of the BGP messages advertised to R4 by R1, where the BGP packet contains two Update messages each containing a large number of VPN-IPv4 and VPN-IPv6 NLRI. The frame has a size of 9212-bytes at the Ethernet layer, or 9194-bytes at the IP layer. Note once again that the TCP options field is 32-bytes in length, containing a TCP MD5 signature option, a Timestamps option, and some No-Operation options used to pad to a 32-bit (4-byte) boundary. Also note that the TCP Segment Length is 9102-bytes. What this means is that

although *Output 9-36* shows the operational MSS as 9134-bytes, reflecting a 40-byte IP overhead and 20-byte TCP overhead, the TCP segment length has actually been reduced by a further 32-bytes to reflect the 32-byte increase in the TCP header length to 52-bytes. As in the IPv4 example, although the TCP MSS is shown as 9134-bytes, it's reasonable to assume that a small percentage of this MSS will not be made available to BGP.

Figure 9-20: BGP Update message with 9134-byte MSS

```

Frame 129: 9212 bytes on wire (73696 bits), 9212 bytes captured (73696 bits) on interface br11, id 0
Ethernet II, Src: RealtekU_da:8a:66 (52:54:00:da:8a:66), Dst: RealtekU_0a:47:8d (52:54:00:0a:47:8d)
802.1Q Virtual LAN, PRI: 7, DEI: 0, ID: 100
Internet Protocol Version 6, Src: 3fff:ffe::1, Dst: 3fff:ffe::4
Transmission Control Protocol, Src Port: 179, Dst Port: 50588, Seq: 20, Ack: 20, Len: 9102
  Source Port: 179
  Destination Port: 50588
  [Stream index: 3]
  [Conversation completeness: Incomplete (12)]
  [TCP Segment Len: 9102]
  Sequence Number: 20 (relative sequence number)
  Sequence Number (raw): 3059063267
  [Next Sequence Number: 9122 (relative sequence number)]
  Acknowledgment Number: 20 (relative ack number)
  Acknowledgment number (raw): 2242250418
  1101 .... = Header Length: 52 bytes (13)
  Flags: 0x010 (ACK)
  Window: 32768
  [Calculated window size: 32768]
  [Window size scaling factor: -1 (unknown)]
  Checksum: 0xe9ec [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  Options: (32 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps, TCP MSS signature, No-Operation (NOP), No-Operation (NOP)
  [Timestamps]
  [SEQ/ACK analysis]
  TCP payload (9102 bytes)
  TCP segment data (910 bytes)
Border Gateway Protocol - UPDATE Message
Border Gateway Protocol - UPDATE Message

```

As before, the BGP prefixes advertised by tester T1 are subsequently withdrawn after which the R2-R3 link is configured to have an IP-MTU of 8000-bytes. Tester T1 then readvertises the BGP prefixes with the intention being to advertise Update messages that are too large to be transmitted across this interface and validate the outcome. The result is shown in *Figure 9-21*:

- Frame 133 shows a BGP message containing two Updates that has a total frame size of 9212-bytes at the Ethernet layer or 9194-bytes at the IP layer.
- This is followed in frame 134 by an ICMPv6 packet from 2001:db8:33ac:2200::4:5 (router R2's IPv6 address for the R2-R3 link) with type 'Packet Too Big'. The ICMPv6 Packet Too Big response is shown in *Figure 9-22* and has an Next-Hop MTU field which contains the 8000-byte MTU of the R2-R3 link. It also contains the IP header of the offending packet and the following 64-bytes of data so that the source can reconcile it with the original outgoing packet.
- Frame 136 shows a retransmission of frame 133, but this time the packet size is 8018-bytes at the Ethernet layer or 8000-bytes at the IP layer. This packet size is the result of R1 adjusting its operational MSS to reflect the value contained in the ICMPv6 Packet Too Big message.
- The retransmissions continue in frames 137, 140, and 141, but the packet size never goes beyond the IP MTU of 8000-bytes.

Figure 9-21: BGP Updates with lower MTU on R2-R3 link

No.	Time	Source	Destination	Protocol	Length	Info
133	0.04509361	3fff:ffe::1	3fff:ffe::4	BGP	4212	UPDATE Message, UPDATE Message
134	0.04532710	2001:db8::1	3fff:ffe::1	ICMPv6	54	Packet Too Big
135	0.04542540	192.168.0.1	192.168.0.2	TCP	80	82154 → 8000 [ACK] Seq=28 Win=22786 Len=0 TSval=2755475346 TSecr=2443178188 RST
136	0.04543210	3fff:ffe::1	3fff:ffe::4	TCP	60	8000 → 82154 [ACK] Seq=28 Win=0 Len=0 TSval=2755475346 TSecr=2443178188 RST
137	0.04543330	3fff:ffe::1	3fff:ffe::4	TCP	60	8000 → 82154 [ACK] Seq=28 Win=0 Len=0 TSval=2755475346 TSecr=2443178188 RST
138	0.050430230	192.168.0.1	192.168.0.2	TCP	80	8000 → 82154 [ACK] Seq=28 Win=0 Len=0 TSval=2755475346 TSecr=2443178188 RST
139	0.119026350	3fff:ffe::4	3fff:ffe::1	TCP	60	82154 → 8000 [ACK] Seq=28 Win=0 Len=0 TSval=2755475346 TSecr=2443178188 RST
140	0.119026350	3fff:ffe::4	3fff:ffe::1	TCP	60	82154 → 8000 [ACK] Seq=28 Win=0 Len=0 TSval=2755475346 TSecr=2443178188 RST
141	0.120725400	3fff:ffe::1	3fff:ffe::4	BGP	418	
142	0.120725400	3fff:ffe::1	3fff:ffe::4	TCP	60	8000 → 82154 [ACK] Seq=28 Win=0 Len=0 TSval=2755475346 TSecr=2443178188 RST
143	0.259079230	192.168.0.1	192.168.0.2	TCP	80	8000 → 82154 [ACK] Seq=28 Win=0 Len=0 TSval=2755475346 TSecr=2443178188 RST
144	0.259079230	192.168.0.1	192.168.0.2	TCP	80	8000 → 82154 [ACK] Seq=28 Win=0 Len=0 TSval=2755475346 TSecr=2443178188 RST

Figure 9-22: ICMPv6 'Packet Too Big' from router R2

```

* Frame 134: 194 bytes on wire (1552 bits), 194 bytes captured (1552 bits) on interface br11, id 0
* Ethernet II, Src: RealtekU_da:47:8d (52:54:00:0a:47:8d), Dst: RealtekU_da:0a:66 (52:54:00:0a:0a:66)
* 802.1Q Virtual LAN, PRI: 7, DEI: 0, ID: 100
* Internet Protocol Version 6, Src: 2001:db8:33ac:2200::4:5, Dst: 3fff:ffe::1
* Internet Control Message Protocol v6
  Type: Packet Too Big (2)
  Code: 0
  Checksum: 0x1a17 [correct]
  [Checksum Status: Good]
  MTU: 8000
* Internet Protocol Version 6, Src: 3fff:ffe::1, Dst: 3fff:ffe::4
* Transmission Control Protocol, Src Port: 179, Dst Port: 50508, Seq: 3859037811, Ack: 2242251273
* Border Gateway Protocol - UPDATE Message
  Marker: ffffffffffffffffffffffffffffffff
  Length: 4093
  Type: UPDATE Message (2)
  Withdrawn Routes Length: 0
  Total Path Attribute Length: 4070
  Path attributes
  
```

Once again the path MTU discovery process has operated as desired, and the correct MSS adjusted to meet the reduced IP MTU while still maximising the highest possible MSS. This is shown in *Output 9-34* which shows the transmitted MSS as 7940-bytes, which is 8000-bytes minus 60-bytes for TCP/IP overhead.

Output 9-37: Operational MSS after R2-R3 link MTU decrease

```

A:admin@R1# show router bgp neighbor "3fff:ffe::4" | match Mss
Tcp Mss : 7940
Tcp Rcvd Mss : 9134

```

An increased MSS can be a useful mechanism to help speed up convergence, and the benefits cannot be disputed for Update packing efficiency. The default TCP MSS for BGP is fairly conservative, and many networks today operate with much larger MTUs that could tolerate a higher MSS. In addition, path MTU discovery offers a mechanism that can offer an increased TCP MSS while still adapting to any lower MTUs that it may encounter on its path.

10 Multicast

Delivery of multicast traffic from one or more sources to potentially many receivers requires that the correct multicast forwarding state is established on all routers in the path from source to receiver. This includes performing packet replication where necessary (for example, receive traffic for group (S^1, G^1) on interface i , and forward that traffic on interface i^1 and i^2). Primarily, Protocol Independent Multicast (PIM) has been used to create this multicast state, which, unlike some of its less widely deployed predecessors, is abstract from the underlying protocols used to exchange unicast reachability. However, PIM needs that unicast reachability information to determine the Reverse Path Forwarding (RPF) interface towards the source or Rendezvous Point (RP), and this is where BGP first became important in multicast environments. In Intra-AS environments this information could be obtained from the IGP, but in Inter-AS environments this information is exchanged using BGP, and consequently the IPv4 Multicast Address Family was introduced.

As extensions for multicast in BGP-MPLS IP-VPNs were defined, originally using the ‘draft-rosen’ architecture, BGP was again used as an Auto-Discovery mechanism for PIM neighbours of the same multicast domain. Later developments in Multicast VPN technology subsumed the draft-rosen architecture as a subset of its capabilities and extended the role of BGP so that it can be used to actually create multicast forwarding state, thereby effectively replacing PIM in the core of the network.

This chapter starts by looking at the role of BGP in Inter-Domain Multicast and then goes on to describe how BGP is used both for auto-discovery of MVPN members and creation/maintenance of Multicast forwarding state.

Inter-Domain IPv4-IPv6 PIM

The use of PIM Sparse Mode (PIM-SM) and Any Source Multicast (ASM) represents some challenges when the sources and receivers are situated in different Autonomous Systems. Within a common AS an active source registers to the Rendezvous Point (RP) and when any interested receivers join to the same RP, PIM (S,G) state is created between the source and the RP and traffic flows down the $(*,G)$ shared tree from the RP towards the receiver. However, consider the case when the source for group G is situated in AS 64510 and an interested receiver for group G is situated in AS 64511. The source registers to the RP in AS 64510, and the interested receiver joins to the RP in AS 64511, but because the RP in AS 64511 isn’t aware that the source in AS 64510 is sending to group G the Any Source Multicast model is broken.

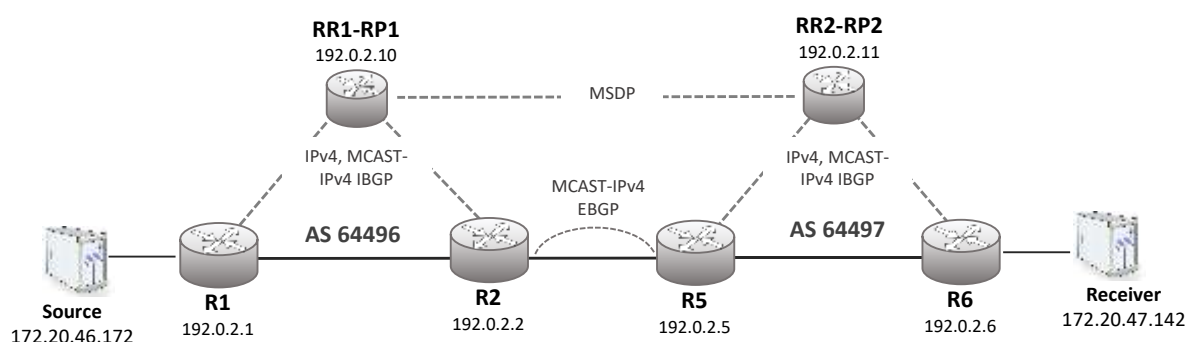
The Multicast Source Discovery Protocol (MSDP) was conceived to allow RPs to notify other RPs in different domains when a multicast source is active, using Source-Active (SA) messages. When an RP first learns of the presence of a source in its own AS, it encapsulates the first PIM Register packet in an SA message and advertises it to each of its MSDP peers. The SA message identifies the source, the group to which the source is sending, and the RP’s

own address. The SA messages are flooded to other MSDP peers using peer-RPF Flooding, which uses the originating RP's address (in the incoming SA message) to determine whether an incoming SA message was received on the correct interface towards the originator. If this is the case, the SA message is then re-advertised to each of the receiving RP's MSDP peers. When a receiving RP has interested receivers for the active source, it creates (S,G) state and joins directly towards the source. When multicast packets subsequently arrive from the source they are flooded down the shared tree (RPT) by the receiving RP, and when the last-hop router receives the first packet, it can optionally create (S,G) state towards the source. The originating RP thereafter periodically sends SA messages with a list of all the sources that are currently registered to the originating RP.

The use of MSDP solves one of the problems of Inter-Domain multicast in that it notifies other domains of an active source. However, another problem remains. Using the previous example, assume that the RP in AS 64511 receives an SA message from the RP in AS 64510 indicating source S is active for group G. Next, assume that the RP in AS 64511 has an interested receiver for group G, so it needs to send an (S,G) join towards the source S. The first thing the RP needs to do therefore is to determine the RPF interface towards source S. However, source S is not in the IGP of AS 64511 and so the RPF check fails. This is the purpose of Multicast IPv4/IPv6 BGP. It is used to advertise IPv4 prefixes across AS boundaries for RPF resolution and is carried in Multi-Protocol BGP using AFI 1/SAFI 2 for IPv4 or AFI2/SAFI 2 for IPv6.

To illustrate the use of MSDP and Multicast IPv4 the topology shown in *Figure 10-1* is used. Routers R1 and R2 belong to AS 64496 and are peered with RR1 for the IPv4 and Multicast IPv4 Address Families only. Router R1 has a single source connected in the base routing instance at address 172.20.46.172 sending to group 239.1.77.49. Routers R5 and R6 belong to AS 64510 and are peered with RR2, again for the IPv4 and Multicast IPv4 Address Families only. Both AS 64496 and AS 64510 run PIM-SM and each has an RP that is peered in MSDP with the RP of the neighbouring AS. Routers R2 and R5 are the peering routers between the AS's and peer with a single EBGp session supporting the Multicast-IPv4 Address Family only, through which the source (172.20.46.0/24) and RP (192.0.2.10/32) prefixes are advertised from R2 to R5 for RPF determination in AS 64497. The source (172.20.46.0/24) prefix is used for RPF lookup in the event of a PIM join towards the source, whilst the RP (192.0.2.10/32) prefix is used for RPF lookup of incoming MSDP Source-Active messages. The interconnect link is also configured for PIM in order to be able to pass multicast traffic between the domains.

Figure 10-1: Inter-Domain IPv4 Multicast test topology



Output 10-2 shows the configuration of RR1-RP1 within AS 64496. From a BGP perspective, there is very little to configure with the exception of the **mcast-ipv4** Address Family. The **msdp** context is used to provide remote and local peering addresses, together with some optional parameters such as the **active-source-limit** to control the number of Source-Active messages the peer will accept from the remote peer.

Output 10-1: Configuration for Inter-Domain Multicast at RP1-RR1

```
router "Base" {
  bgp {
    group "CLIENTS" {
      peer-as 64496
      family {
        ipv4 true
        mvpn-ipv4 true
      }
      cluster {
        cluster-id 192.0.2.10
      }
    }
    neighbor "192.0.2.1" {
      group "CLIENTS"
      authentication-key "C3i5c6IP3VMhQFGff3RCM6O4qzEOtDo= hash2"
    }
    neighbor "192.0.2.2" {
      group "CLIENTS"
      authentication-key "C3i5c6IP3VMhQFGff3RCM3Kf+/MN6G8= hash2"
    }
  }
  msdp {
    admin-state enable
    peer 192.0.2.11 {
      admin-state enable
      active-source-limit 512
      local-address 192.0.2.10
    }
  }
}
```

Recall that router R2 advertises the source subnet 172.20.46.0/24 and the RP address 192.0.2.10/32 to router R5 for RPF determination within AS 64497. To do that, R2 uses a conventional route policy such as the one shown in *Output 10-2*, but those prefixes must exist in the Multicast RIB. SR-OS separates the RIB and FIB for unicast and multicast to allow

for incongruent routing of unicast and multicast traffic, and by default IGP routes populate the unicast routing table and not the multicast table. Because R2 knows the source subnet and the RP address in the IGP, the multicast routing table can be populated with those routes using the **multicast-import ipv4** command under the IGP node (in this case IS-IS).

Output 10-2: Routing policy and EBGP peering at R2

```

policy-options {
  prefix-list "PIM-RPF-Prefixes" {
    prefix 172.20.46.0/24 type exact {
    }
    prefix 192.0.2.10/32 type exact {
    }
  }
  policy-statement "PIM-RPF" {
    entry 10 {
      from {
        prefix-list ["PIM-RPF-Prefixes"]
      }
      to {
        protocol {
          name [bgp]
        }
      }
      action {
        action-type accept
      }
    }
  }
}
router "Base" {
  bgp {
    group "EBGP" {
      peer-as 64497
      family {
        ipv4 true
        mcast-ipv4 true
      }
    }
    neighbor "192.168.0.14" {
      group "EBGP"
      authentication-key "JK/6NxQqPrX7NU7cD3w1PkmpZKMQDY8= hash2"
      export {
        policy ["PIM-RPF"]
      }
    }
  }
  isis 0 {
    multicast-import {
      ipv4 true
    }
  }
}

```

When the advertised prefixes are learnt by router R5 in AS 64497 in Multicast-IPv4 Address Family, they are automatically populated into the multicast routing table and propagated in IBGP to other BGP speakers in AS 64497 as shown in *Output 10-3*. When routers in AS 64497 execute a multicast RPF check by default they look only in the unicast routing table. Because the AS 64496 source subnet and RP address are learnt in Multicast-IPv4 they are not present in the unicast routing table. Therefore, it is necessary to configure the routers in AS 64497 to execute multicast RPF checks using both the multicast and unicast routing tables with the

rpf-table both command under the **pim** node. When the **both** keyword is used, SR-OS looks up the route in the multicast route table first, and if the route is not present there it looks up the route in the unicast route table. For completeness the configuration is shown in *Output 10-4*.

Output 10-3: Multicast routing table at R6

```
A:admin@R6# show router route-table mcast-ipv4 protocol bgp

=====
Multicast IPv4 Route Table (Router: Base)
=====
Dest Prefix[Flags]                                Type    Proto    Age          Pref
  Next Hop[Interface Name]                        Metric
-----
172.20.46.0/24                                     Remote  BGP       04h04m05s    170
      192.168.0.25                                100
192.0.2.10/32                                     Remote  BGP       04h04m05s    170
      192.168.0.25                                100
-----
No. of Routes: 2
Flags: n = Number of times nexthop is repeated
      B = BGP backup route available
      L = LFA nexthop available
      S = Sticky ECMP requested
=====
```

Output 10-4: Configuration for PIM RPF check

```
router "Base" {
    pim {
        ipv4 {
            rpf-table both
        }
    }
}
```

With the required configurations in place, the receiver connected to R6 joins to group 239.1.77.49 and establishes (*,G) state at RP2. The source at 172.20.46.172 connected to router R1 begins to send traffic to the group, which causes the first hop router R1 to register to RP1 in AS 64496. In turn, RP1 in AS 64496 sends an MSDP Source-Active message to RP2 in AS 64497.

Output 10-5: Source-Active message sourced by RP1

```
A:admin@rr1# show router msdp source-active

=====
MSDP Source Active Info
=====
Grp Address      Src Address      Origin RP      Peer Address    State Timer
-----
239.1.77.49      172.20.46.172   192.0.2.10     0.0.0.0         87
-----
MSDP Source Active : 1
=====
```

When RP2 receives the Source-Active message it recognises that it has an interested receiver and therefore sends a (S,G) PIM Join to create a shortest path tree between itself and the source. The RPF check for this Join message uses the multicast routing table because the source 172.20.46.0/24 is not present in the unicast routing table. When the (S,G) tree is established between RP2 in AS 64497 and R1 in AS 64496, traffic from source 172.20.46.172 flows along the (S,G) state from router R1 to RP2, and then down the (*,G) tree from RP2 to R6. The result is that RP2 has two PIM states for group 239.1.77.49 as shown in *Output 10-6*.

Output 10-6: PIM state for group 239.1.77.49 at RP2

```
A:admin@rr2# show router pim group 239.1.77.49
```

```
=====
```

```
Legend:  A = Active    S = Standby
```

```
=====
```

```
PIM Groups ipv4
```

```
=====
```

Group Address Source Address	Type RP	Spt Bit State	Inc Intf Inc Intf (S)	No.Oifs
239.1.77.49	(*,G)			1
*	192.0.2.11			
239.1.77.49	(S,G)		link-to-R6	0
172.20.46.172	192.0.2.11			

```
-----
```

```
Groups : 2
```

```
=====
```

When the last-hop router R6 receives the first packet down the shared tree from RP2 it can optionally create (S,G) state directly between itself and the source (thus moving off the shared tree between itself and RP2). By default, SR-OS moves to the shortest path tree when the first packet is received on the shared tree, and as *Output 10-7* shows this is indeed what R6 does. Again, the RPF check for this (S,G) join uses the multicast routing table populated by the Multicast-IPv4 routes advertised by AS 64496.

Output 10-7: PIM state for group 239.1.77.49 at router R6

```
A:admin@R6# show router pim group 239.1.77.49
```

```
=====
```

```
Legend:  A = Active    S = Standby
```

```
=====
```

```
PIM Groups ipv4
```

```
=====
```

Group Address Source Address	Type RP	Spt Bit State	Inc Intf Inc Intf (S)	No.Oifs
239.1.77.49	(*,G)		link-to-R5	1
*	192.0.2.11			
239.1.77.49	(S,G)	spt	link-to-R5	1
172.20.46.172	192.0.2.11			

```
-----
```

```
Groups : 2
```

```
=====
```

Router R6 now receives the traffic from the source along the (S,G) tree directly between itself as last-hop router and the first-hop router R1.

Output 10-8: Traffic flowing through (S,G) tree at R6

```

A:admin@R6# show router pim group 239.1.77.49 source 172.20.46.172 detail | match
'Incoming Intf' post-lines 5
Incoming Intf      : link-to-R5
Outgoing Intf List : MCAST-RECEIVER

Curr Fwding Rate   : 801.600 kbps
Forwarded Packets  : 181028          Discarded Packets : 0
Forwarded Octets   : 181390056      RPF Mismatches    : 0

```

Multicast in MPLS/BGP IP-VPNs

The original proposal for extending BGP/MPLS IP-VPNs to support IP multicast came from the draft-rosen-vpn-mcast framework. This became a subset of the wider-reaching Multicast-VPN specification (frequently referred to as Next-Generation Multicast-VPN) [45], retiring the draft-rosen architecture to historic status [47]. Although draft-rosen is a subset of the Multicast-VPN specification, it is useful to be able to distinguish between them. Hence this chapter refers to the original draft-rosen specification simply as draft-rosen, and the remainder of Multicast in BGP/MPLS IP-VPNs work simply as Multicast-VPN or MVPN.

Draft-Rosen

The draft-rosen framework uses PIM to extend an (S,G) or (*,G) multicast distribution tree from a customer site, through the Service Provider network, to one or more customer sites within the same VPN. Each PE router runs an instance of PIM for each multicast-enabled VRF (or MVPN). In each of these MVPN instances, the PE router maintains a PIM adjacency with connected CE routers and maintains separate MVPN multicast routing tables. Entities contained within these MVPNs are generically referred to 'C-instance'. For example, multicast state is referred to as C-multicast and represented as C-(S,G). The PE router also runs a global instance of PIM known as the Service Provider instance or 'P-instance', with which it forms adjacencies with each of its IGP neighbours such as P routers and/or other PE routers.

Each MVPN is assigned to a multicast domain that defines a set of PE routers supporting a VPN that are able to send multicast traffic to each other. Each multicast domain is configured with a multicast group address belonging to the Service Provider P-instance, which is used to create multicast tunnels between PE routers forming part of the MVPN. The encapsulation for these multicast tunnels is GRE; where the source address is the PE system address and the destination address is the P-instance multicast domain group address. Customer control plane (PIM) and data-plane traffic is subsequently encapsulated within these multicast tunnels so that it remains transparent to the Service Provider core.

With PIM in the C-instance and PIM in the P-instance where did BGP become useful in the draft-rosen architecture? The answer is for Auto-Discovery of PE routers forming part of the same multicast domain (MVPN). The PE routers belonging to the same multicast domain discover each other using the Multicast Distribution Tree SAFI (MDT SAFI) [47] NLRI, which

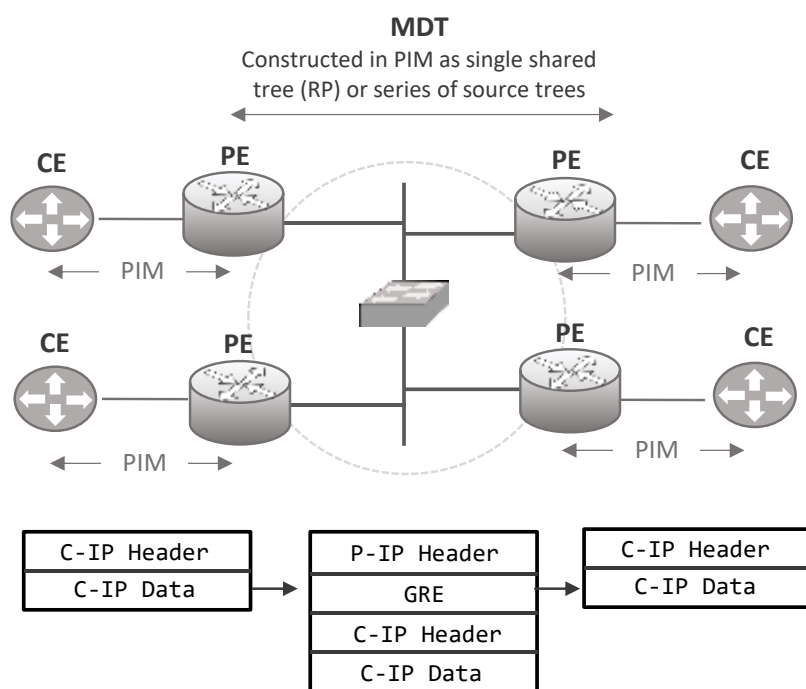
uses AFI 1 SAFI 66. The RD:IPv4 Address field contains the system address of the advertising PE router, whilst the Group Address field contains the P-instance group address assigned to this multicast domain. The MDT-SAFI NLRI is sent in a BGP Update message together with the relevant Route-Target extended community attribute used to define the members of the MVPN.

Output 10-9: MDT-SAFI NLRI

RD:IPv4 Address (12 bytes)
Group Address (4 octets)

Upon receiving a BGP Update with the MDT-SAFI NLRI and relevant Route Target values, PE routers in the multicast domain establish multicast tunnels to create a Multicast Distribution Tree (MDT) by sending PIM joins to the multicast domain P-instance address. In the case of using PIM-SM in the P-instance the PIM joins are sent towards the Rendezvous Point (RP) to create a (*,G) shared tree. In the case of Source-Specific PIM (PIM-SSM) in the P-instance, the MDT-SAFI NLRI provides the receiving PE with the source address to which it should join, and as such the PIM join is directed towards the source to create (S,G) multicast state in the core where each advertising PE router is the root of the tree and all of the receiving PE routers are leaves of the tree.

Output 10-10: Draft-Rosen MVPN Architecture

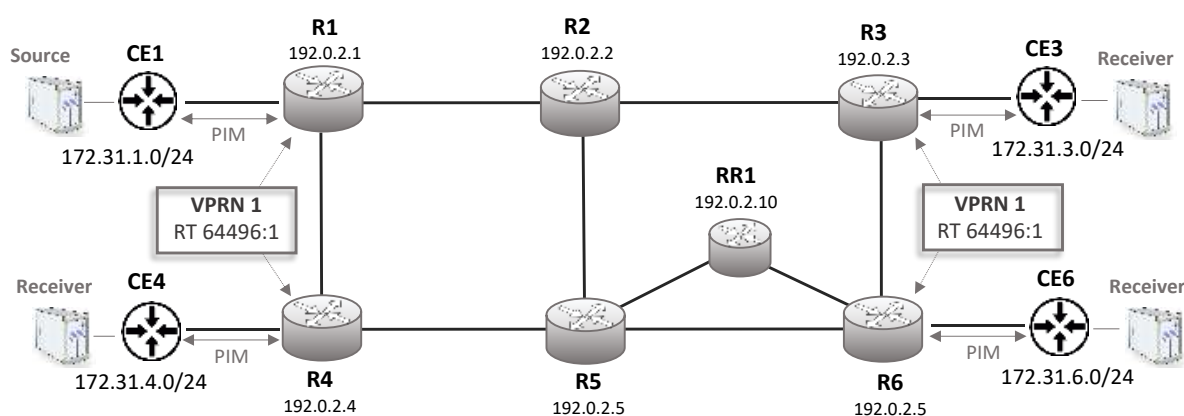


The MDT effectively creates a broadcast domain where all traffic forwarded onto the MDT by a given PE router is seen by every other PE router in the domain and is referred to as the

Default-MDT. The Default-MDT is useful for deployments of multicast such as broadcast/linear TV where the intention is that all PE routers receive all content but can lead to sub-optimality in cases where PE routers do not have interested receivers and the traffic is just silently discarded by the receiving PE. Clearly this has the potential to be sub-optimal and an inefficient use of bandwidth in the core of the Service Provider network. The draft-rosen framework and subsequent Multicast VPN specifications both detail how 'data' distribution trees can be created to increase optimality. Whilst PIM C-Join/Prune messages are always passed over the Default-MDT, separate Data-MDTs can be created to pass traffic only to PE routers that have interested receivers. Therefore, every PE in the multicast domain joins the Default-MDT, but a PE does not join a non-default distribution tree unless it is connected to an MVPN site that explicitly needs to receive traffic from a group that has been assigned to that tree. Within the draft-rosen framework, the method for signalling these Data-MDTs is through extensions to PIM.

To illustrate the use of BGP and the MDT-SAFI for draft-rosen Multicast-VPN Auto-Discovery the topology shown in *Figure 10-2* is used. Routers R1 to R6 belong to AS 64496 and run IS-IS level 2. All are IBGP peered with a Route-Reflector RR1 and are configured to support the MDT-SAFI Address Family. PIM is enabled in the base routing instance on network links to other adjacent routers and also on the system address. Routers R1, R3, R4, and R6 all form part of the MVPN multicast domain, and all have connected CE routers running PIM. The P-instance uses PIM-SSM using group address 239.255.1.1, whilst the C-instance runs both PIM-SSM and PIM-SM with an RP (Bootstrap Router) running on CE1. There is a single source connected to CE1 that is sending to group 232.2.78.48 with interested receivers connected to CE3, CE4, and CE6.

Figure 10-2: Draft-Rosen Test Topology



Output 10-11 shows the configuration requirements for the addition of draft-rosen Multicast VPN to a unicast VPN, although no explanation is given of the unicast parameters as these have been previously described in Chapter 3. The Multicast VPN parameters are all configured under the **mvpn** context, but before configuring any parameters within this context **pim** must be administratively enabled. In this example, PIM is running on the PE to CE interface towards CE1, but even if PIM isn't actually required towards a multicast receiver

(that is, the receiver is directly connected and only IGMP is required), **pim** still must be administratively enabled. The configuration syntax within the **mvpn** node is exactly the same for draft-rosen and next-generation Multicast-VPNs but uses the terminology from the latter. The first parameter is the **auto-discovery** command which, for draft-rosen, is set to **mdt-safi**. An alternative (default) option is to use the MVPN-IPv4 Address Family defined in Multicast VPN for Auto-Discovery, but MDT-SAFI pre-dated this NLRI and is historically used for draft-rosen. The **vrf-target** context allows for manual configuration of the Route-Target extended community that will be attached to the MDT-SAFI route used for auto-discovery. The default is the same as the unicast Route-Target and is only shown here to show that unicast and multicast topologies do not necessarily have to be congruent.

The **provider-tunnel** command provides the context for configuring the Default-MDT using the **inclusive** keyword. Within the **inclusive** context, **pim** is in use and the **mode** is **ssm** using a Default-MDT P-instance group address of 239.255.1.1. The **provider-tunnel** also provides the context for configuring one or more Data-MDTs using the **selective** keyword. Data-MDTs provide the capability to optimise bandwidth utilisation by sending traffic only to PE routers that have interested receivers but have the disadvantage that they create more state in the provider backbone. The **data-threshold** command therefore provides the means to restrict the creation of Data-MDTs only to certain C-groups and only once they have exceeded a certain bandwidth threshold. In the example, the C-group range is configured to be unconstrained (224.0.0.0/4), with the bandwidth **threshold** set at 100Kb/s. The system will monitor C-groups with (S,G) state and once a connected source has crossed this threshold, the PE router will signal all other PEs through the Default-MDT to indicate that a Data-MDT is being created for this C-group. This Data-MDT consumes another P-instance group, so to constrain how much state a given Multicast VPN can consume the **selective** context allows for a prefix/mask to be defined. In this case the **group-prefix 239.255.16.0/24** command allows creation of up to 256 Data-MDTs.

Output 10-11: Configuration for Draft-Rosen MVPN at R1

```

vprn "one" {
  pim {
    interface "link-to-CE1" {
    }
  }
  mvpn {
    auto-discovery {
      type mdt-safi
    }
    vrf-target {
      community "target:64496:1"
    }
    provider-tunnel {
      inclusive {
        pim {
          admin-state enable
          mode ssm
          group-address 239.255.1.1
        }
      }
      selective {
        data-threshold {
          group-prefix 224.0.0.0/4 {
            threshold 100
          }
        }
      }
    }
  }
}

```

```

    }
    }
    pim {
        mode ssm
        group-prefix 239.255.16.0/24
    }
}
}
}
bgp-ipvpn {
    mpls {
        admin-state enable
        route-distinguisher "64496:1"
        vrf-target {
            import-community "target:64496:1"
            export-community "target:64496:1"
        }
        auto-bind-tunnel {
            resolution filter
            resolution-filter {
                sr-isis true
            }
        }
    }
}
}
interface "link-to-CE1" {
    ipv4 {
        primary {
            address 192.168.100.1
            prefix-length 30
        }
    }
    sap 1/1/c3/1:200 {
    }
}
}
}

```

Once the MVPN configuration is complete and the service has been enabled, the process of MVPN Auto-Discovery starts in addition to conventional unicast prefix distribution. *Debug 10-1* shows the MDT-SAFI NLRI advertised by router R1. Note that the NLRI [MDT-SAFI] shows the configured RD of the VPRN and the IPv4 system address together with the Default-MDT configured within the **mvpn inclusive** context.

Debug 10-1: MDT-SAFI sourced from router R1

```

2 2025/09/22 11:16:14.741 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 51
    Flag: 0x90 Type: 14 Len: 26 Multiprotocol Reachable NLRI:
        Address Family MDT-SAFI
        NextHop len 4 NextHop 192.0.2.1
        [MDT-SAFI] Addr 192.0.2.1, Group 239.255.1.1, RD 64496:1
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100

```

When other routers in the MVPN receive the MDT-SAFI from R1 they subsequently know both the source (192.0.2.1) and group address (239.255.1.1) contained within it. Other PE routers in the MVPN therefore send PIM Joins directly towards the source-specific group so that R1

is the root and R3, R4, and R6 are leaves of the multicast tree. Equally, R3, R4, and R6 become the root of their own source-specific trees. Once the multicast state is created, each PE router sends PIM Hellos into the Default-MDT to discover and maintain PIM adjacencies. In the case of R1 in *Output 10-12*, there are four PIM neighbours; one to CE1 and three through the Default-MDT (1-mt-239.255.1.1) to R3 (192.0.2.3), R4 (192.0.2.4) and R6 (192.0.2.6).

Output 10-12: MVPN PIM adjacencies at R1

```
A:admin@R1# show router 1 pim neighbor

=====
PIM Neighbor ipv4
=====
Interface          Nbr DR Prty    Up Time      Expiry Time   Hold Time
Nbr Address
-----
link-to-CE1        1              0d 22:32:21   0d 00:01:36   105
  192.168.100.2
1-mt-239.255.1.1    1              0d 00:17:04   0d 00:01:42   105
  192.0.2.3
1-mt-239.255.1.1    1              0d 00:17:04   0d 00:01:16   105
  192.0.2.4
1-mt-239.255.1.1    1              0d 00:17:04   0d 00:01:40   105
  192.0.2.6
-----
Neighbors : 4
=====
```

Router R1 sends to the C-group 232.2.78.48 and the interested receivers behind CE3, CE4, and CE6 join to the group using IGMP. The respective CE routers send PIM Joins upstream to their connected PE routers which are propagated through the Default-MDT towards the source, where the upstream hop towards the source is calculated using the unicast routing table. *Output 10-13* shows the state of the C-instance group 232.2.78.48 at router R3. The output shows that the RPF neighbour towards the source (Rpf Neighbor) is R1 (192.0.2.1) as this is the router that advertised reachability to the unicast prefix 172.31.1.0/24 containing the source host. The incoming interface (IIF) for the multicast data is the Default-MDT (1-mt-239.255.1.1) and the outgoing interface list (OIL) contains a single entry towards CE3 (link-to-CE3). The current forwarding rate (Curr Fwding Rate) is ~801Kb/s which is calculated on the ingress linecard and does not account for any egress QoS and/or policies that might restrict that rate.

Output 10-13: Status of C-group 232.2.78.48 at router R3

```
A:admin@R3# show router 1 pim group 232.2.78.48 detail

=====
PIM Source Group ipv4
=====
Group Address       : 232.2.78.48
Source Address      : 172.31.1.3
RP Address          : 0
Advt Router         : 192.0.2.1
Flags               :                               Type           : (S,G)
Mode                : sparse
MRIB Next Hop       : 192.0.2.1
MRIB Src Flags      : remote
Keepalive Timer     : Not Running
Up Time             : 0d 01:36:48      Resolved By           : rtable-u
```



```

Up JP State      : Joined           Up JP Expiry      : 0d 00:00:04
Up JP Rpt       : Not Joined StarG  Up JP Rpt Override : 0d 00:00:00

Register State   : No Info
Reg From Anycast RP: No

Rpf Neighbor     : 192.0.2.1
Incoming Intf    : 1-mt-239.255.1.1
Incoming SPMSI Intf: 1-mt-239.255.1.1*
Outgoing Intf List : link-to-CE3

Curr Fwding Rate : 801.600 kbps
Forwarded Packets : 28459           Discarded Packets : 0
Forwarded Octets  : 28515918       RPF Mismatches    : 0
Spt threshold     : 0 kbps         ECMP opt threshold : 7
Admin bandwidth   : 1 kbps

-----
Groups : 1
=====

```

Figure 10-3 shows a packet capture of multicast traffic being forwarded towards C-group 232.2.78.48 as it leaves router R1. The outer IP header has a source address of R1's system address (192.0.2.1) and a destination address of the P-instance Default-MDT group 239.255.1.1 allocated to this MVPN service. The encapsulated protocol is GRE, and the GRE header indicates an encapsulated protocol of IP. The C-instance packet with source address 172.31.1.3 and destination 232.2.78.48 follows.

Figure 10-3: Capture of C-instance packet encapsulated into the Default-MDT

```

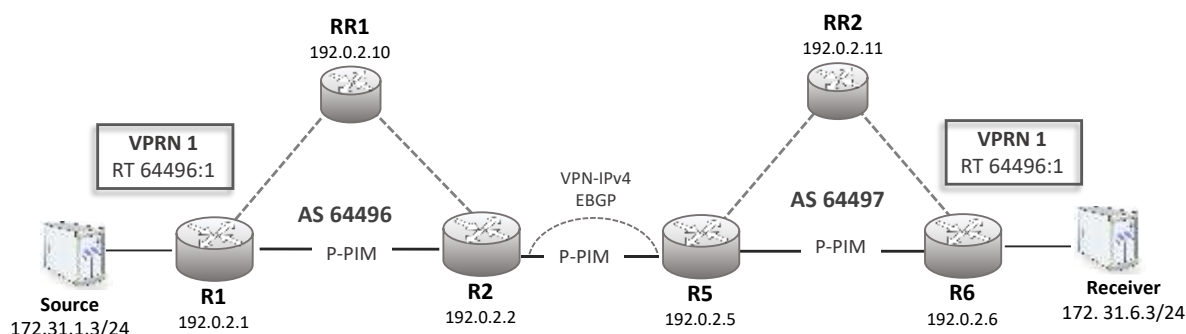
* Frame 13: 1044 bytes on wire (8352 bits), 1044 bytes captured (8352 bits) on interface br12, id 1
* Ethernet II, Src: RealtekU_05:f0:ad (52:54:00:05:f0:ad), Dst: IPv4mcast_7f:01:01 (01:00:5e:7f:01:01)
* 802.1Q Virtual LAN, PRI: 2, DEI: 0, ID: 100
* Internet Protocol Version 4, Src: 192.0.2.1, Dst: 239.255.1.1
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  * Differentiated Services Field: 0x30 (DSCP: AF12, ECN: Not-ECT)
  Total Length: 1026
  Identification: 0xf46a (62570)
  * Flags: 0x40, Don't fragment
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 255
  Protocol: Generic Routing Encapsulation (47)
  Header Checksum: 0xd02f [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.0.2.1
  Destination Address: 239.255.1.1
* Generic Routing Encapsulation (IP)
  * Flags and Version: 0x0000
  Protocol Type: IP (0x0800)
* Internet Protocol Version 4, Src: 172.31.1.3, Dst: 232.2.78.48
* User Datagram Protocol, Src Port: 1024, Dst Port: 1024
* Data (974 bytes)

```

Inter-AS Draft Rosen

Before moving on from draft-rosen multicast VPN, consider the use case of extending a multicast VPN across an Autonomous System boundary. To extend the multicast VPN through this interconnect, two extensions are required. Figure 10-4 is used as a reference consisting of two Autonomous Systems interconnected through the use of a Type B Interconnect ([7], Section 10).

Figure 10-4: Inter-AS Draft-Rosen



Connector Attribute

As previously described, the Default-MDT is a PIM-enabled interface. When a PE needs to send a C-multicast PIM join through the MDT to an Upstream Multicast Hop (UMH) it must determine the Reverse Path Forwarding (RPF) interface towards the particular C-address. The PE looks up the C-address in the VRF and if the C-address is learnt through MP-BGP its Next-Hop address is one of the PEs that is a PIM adjacency over the Default-MDT. That given multicast tunnel is the RPF interface. However, when VPN-IPv4 prefixes are advertised through a Type B interconnect, both ASBRs impose themselves as Next-Hops on the Update message but do not actually participate in the VPN or Multicast VPN. As a result, the Next-Hop attribute cannot be used to correlate to a PIM adjacency on the MDT, and the RPF interface towards the C-source cannot be determined.

The BGP *Connector* attribute solves this problem. The Connector attribute is an optional transitive attribute that is carried with VPN-IPv4/IPv6 prefixes in a Multicast VPN and contains the originating PEs system address (the same address that is used to forward multicast packets onto the MDT). Therefore, when a PE looks up a C-address in the VRF for RPF determination, it uses the Connector attribute instead of the Next-Hop attribute. In SR-OS, VPN-IPv4/IPv6 prefixes are advertised with the Connector attribute present when **auto-discovery** has been configured within the **mvpn** context of a VPRN service.

PIM RPF Vector

In Figure 10-4 router R1 advertises an Update message with an MDT-SAFI NLRI containing its system address towards R6 as part of the Auto-Discovery process. As a result, R6 sends a P-instance PIM join towards that system address to create hop-by-hop PIM state for the multicast distribution tree. Assume that R6's IGP next-hop is a P router, that router receives a PIM join destined towards R1, but R1's IP address is not known to this router because it is not known within the IGP of AS 64497. Therefore, the P router cannot forward the PIM join and it is dropped.

The RPF Vector TLV is an extension to PIM that specifies the IP address of the ASBR on the path to the root of the multicast distribution tree. When a PE router sends a PIM Join message into the core, it includes the PIM RPF Vector within the PIM Join attribute. Each core router thereafter does its RPF check on the address contained in the RPF Vector TLV and propagates the join towards the specified ASBR (Vector) to create the multicast distribution tree. PIM RPF Vector must be enabled on all PE routers, P routers, and ASBRs that are part of the multicast distribution tree. It is configured using the **rpfv mvpn** command under the global PIM context.

Output 10-14: PIM RPF Vector configuration

```
router "Base" {
    pim {
        rpfv {
            mvpn true
        }
    }
}
```

Again using *Figure 10-4* as reference, R6 distributes the VPN prefix 172.31.6.0/24 into VPN-IPv4, and because **auto-discovery** is configured under **mvpn**, the Update message contains the Connector attribute. *Output 10-15* shows the VPN-IPv4 prefix 172.31.6.0/4 sourced by R6 as seen at router R1. Note that the Next-Hop attribute is set to ASBR1 (192.0.2.2) and the presence of the Connector attribute consisting of the Route-Distinguisher and system address of the originating PE (192.0.2.6). R1 subsequently uses this attribute for RPF interface determination.

Output 10-15: BGP Connector attribute

```
A:admin@R1# show router bgp routes 172.31.6.0/24 vpn-ipv4 detail
=====
BGP Router ID:192.0.2.1          AS:64496          Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                l - leaked, x - stale, > - best, b - backup, p - purge,
                w - unused-weight-only
Origin codes   : i - IGP, e - EGP, ? - incomplete
=====
BGP VPN-IPv4 Routes
=====
Original Attributes

Network       : 172.31.6.0/24
Nexthop       : 192.0.2.2
Route Dist.   : 64496:1          VPN Label      : 524254
Path Id       : None
From          : 192.0.2.10
Res. Nexthop  : 192.168.0.2
Local Pref.   : 100
Aggregator AS : None            Interface Name : link-to-R2
Atomic Aggr.  : Not Atomic      Aggregator    : None
AIGP Metric   : None            MED           : None
Connector     : RD 64496:101, Originator 192.0.2.6
Community     : target:64496:1
Cluster       : 192.0.2.10 192.0.2.11
Originator Id : 192.0.2.6       Peer Router Id : 192.0.2.10
Fwd Class     : None           Priority       : None
```

```

Origin       : IGP
Flags        : Used Valid Best Group-Best
Route Source : Internal
AS-Path      : 64512
Route Tag    : 0
Neighbor-AS  : 64512
DB Orig Val  : N/A                Final Orig Val : N/A
Source Class : 0                  Dest Class    : 0
Add Paths Send : Default
Last Modified : 00h04m04s
VPRN Imported : 1

```

```

--- [snip] ---

```

```

Routes : 1

```

The P-instance group address used to form the Default-MDT in *Figure 10-4* is 239.255.1.1, which is an SSM group. As a result, when R1 receives an MDT-SAFI NLRI from R6 it sends a PIM Join directly towards the source (192.0.2.6), which includes the RPF Vector TLV. This is shown in *Output 10-16*, where the source address for the group is R6 (192.0.2.6), and the Advertising Router or RPF Vector (RPFV Proxy) is ASBR1 (192.0.2.2). The resulting RPF Vector neighbor (Upstream RPFV Nbr) is also set for the adjacent link to ASBR1 (192.168.0.2) together with the MRIB Next-Hop (MRIB Next Hop).

Output 10-16: PIM RPF Vector

```

A:admin@R1# show router pim group 239.255.1.1 source 192.0.2.6 detail

```

```

=====
PIM Source Group ipv4
=====

```

```

Group Address      : 239.255.1.1
Source Address     : 192.0.2.6
RP Address         : 0
Advt Router        : 192.0.2.6

Upstream RPFV Nbr  : 192.168.0.2
RPFV Type          : Mvpn 64496:1      RPFV Proxy       : 192.0.2.2

Flags              : spt                Type             : (S,G)
Mode               : sparse
MRIB Next Hop      : 192.168.0.2
MRIB Src Flags     : remote
Keepalive Timer Exp: 0d 00:03:03
Up Time            : 0d 00:46:47      Resolved By       : rtable-u

Up JP State        : Joined             Up JP Expiry      : 0d 00:00:12
Up JP Rpt          : Not Joined StarG   Up JP Rpt Override: 0d 00:00:00

Register State     : No Info
Reg From Anycast RP: No

Rpf Neighbor       : 192.168.0.2
Incoming Intf      : link-to-R2
Outgoing Intf List : system

Curr Fwding Rate   : 0.000 kbps
Forwarded Packets  : 93                 Discarded Packets : 0
Forwarded Octets   : 7254               RPF Mismatches    : 0
Spt threshold      : 0 kbps             ECMP opt threshold: 7
Admin bandwidth    : 1 kbps

```

 Groups : 1
 =====



Note: The preceding example of a Type B Interconnect has a requirement for both the BGP Connector Attribute and the PIM RPF Vector TLV in order to build the multicast distribution tree. In a Type C Interconnect, the Next-Hop attribute of the MDT-SAFI remains intact as the originating PE and not the ASBR, and so the use of the Connector attribute is superfluous. The PIM RPF Vector TLV must still be included in the PIM Join however and should be set to the Next-Hop of the labeled BGP route for the originating PE.

Multicast VPN

The architecture defined in [45] introduces the notion of a P-Multicast Service Interface (PMSI) to define the entity that connects a set of PE routers forming a Multicast VPN. The PMSI is considered an overlay on the P-network used for sending to all or some PEs in the MVPN. The PMSI can be constructed using a number of transport mechanisms such as PIM, mLDP, RSVP-TE, or P2MP-SR trees. There is, however, a very clear distinction between the multicast service defined by the PMSI, and the transport mechanism used to instantiate that PMSI, referred to as P-tunnels. This allows the MVPN architecture to easily facilitate the use of various transport protocols.

An Inclusive PMSI (I-PMSI) is one that enables a packet sent onto a particular MVPN to be received by all other PEs attached to the same MVPN. The I-PMSI is analogous to the Default-MDT of the draft-rosen Multicast VPN. A notable difference between draft-rosen and MVPN, however, is that the draft-rosen establishes MVPN-specific PIM adjacencies between PE routers forming that MVPN and uses those adjacencies to propagate C-multicast routing information from CE to CE using PIM. Because MVPN I-PMSIs can be constructed using transport protocols other than PIM, an option is provided to maintain a PIM-free provider core by implementing a PIM-BGP interworking function at the PE and using BGP to propagate C-multicast routing information between PE routers within the MVPN.

A Selective PMSI (S-PMSI) provides a mechanism where a packet sent onto the MVPN is received by a subset of the other PEs of that MVPN, and there may be multiple S-PMSIs per MVPN. Again, it is analogous to a Data-MDT of the draft-rosen multicast VPN.

The MVPN specification uses the MCAST-VPN NLRI, which is carried in Multi-Protocol BGP using AFI 1 SAFI 5. The information carried in the MCAST-VPN NLRI can be broken into two categories: information used for Auto-Discovery (A-D), and information used for distribution of C-multicast routing information. The format of the MCAST-VPN NLRI is shown in *Figure 10-5*. *Table 10-1* below it lists the seven possible Route-Types. Their use will be described throughout this section.

Figure 10-5: MCAST-VPN NLRI format

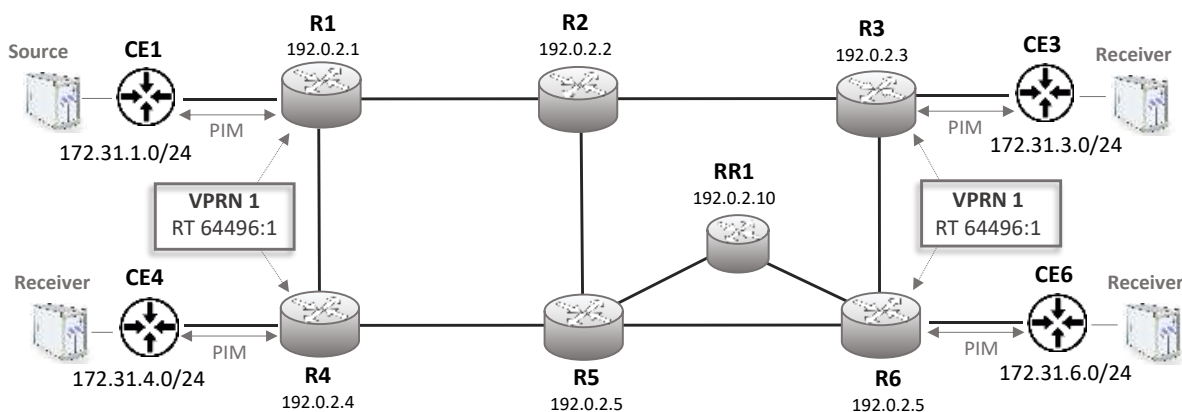
Route Type (1 octet)
Length (1 octet)
Route Type Specific (variable)

Table 10-1: MCAST-VPN NLRI Route Types

Route-Type	Category	Purpose
1	A-D Route	Intra-AS I-PMSI A-D route
2		Inter-AS I-PMSI A-D route
3		S-PMSI A-D route
4		Leaf A-D route
5		Source-Active A-D route
6	C-Multicast	Shared Tree Join route
7		Source Tree Join route

The test topology used for illustrating MVPN is largely the same as that used to demonstrate the use of Draft-Rosen but is repeated in *Figure 10-6* for ease of reference. Routers R1 to R6 belong to AS 64496 and run IS-IS level 2. All are IBGP peered with a Route-Reflector RR1 and are configured to support the MVPN-IPv4 Address Family. This section will describe a number of techniques for creating a point-to-multipoint multicast distribution tree including mLDP and RSVP-TE, and as a pre-requisite these are also enabled on all adjacency links. Routers R1, R3, R4, and R6 all form part of the MVPN multicast domain, and all have connected CE routers running PIM. The C-multicast domain operates both PIM-SM and PIM-SSM and there is an RP on CE1 advertising its' candidacy for group 224.0.0.0/4. There is a single source connected to CE1 that is sending to group 239.2.78.48 with interested receivers connected to CE3, CE4, and CE6.

Figure 10-6: MVPN test topology



Output 10-17 shows an example configuration for MVPN using mLDP to construct the multicast distribution tree. Once again unicast parameters are not covered within this chapter having been previously described in Chapter 3. The first notable parameter is the **auto-discovery** command, which for Multicast-VPN is set to **bgp**, meaning the MCAST-VPN A-D route (Route Type 1 or Route Type 2) is used for auto-discovery. Because PIM is not used within the PMSI in this example, the command **c-multicast signaling bgp** command is configured to enable the use of MCAST-VPN C-Multicast routes (Route Type 6 or 7) for propagation of C-multicast signalling. Within SR-OS, propagation of C-multicast signalling using BGP is the only option when mLDP or RSVP-TE is used for the I-PMSI transport tunnel. However, this does not completely eradicate the use of PIM in the provider core. For example, if PIM-SM is used in the C-multicast instance and PIM Bootstrap messages are used to propagate Group-to-RP mappings, these are propagated natively through the I-PMSI tunnel.

The **provider-tunnel** provides the context for configuring the I-PMSI using the **inclusive** keyword. Within the **inclusive** context, the example shows that mLDP is used for the transport protocol for the P-tunnel and all that is required here is that **mldp** is placed in an **admin-state** of **enable**. The **provider-tunnel** also provides the context for configuring one or more S-PMSIs using the **selective** keyword but note that SR-OS enforces the use of the same tunnel technology for both S-PMSI and I-PMSI, and consequently, mLDP is also configured here. The **data-threshold** context allows for configuration of a **threshold** in Kilobits/seconds that must be exceeded before an S-PMSI is instantiated. In addition to the bandwidth threshold, the **group-prefix** command allows for the range of C-multicast addresses to be constrained if required. In this example any C-multicast group is allowed to instantiate an S-PMSI after the threshold of 100Kb/s has been exceeded. The **maximum-p2mp-spmsi** command places a limit on the number of S-PMSI multicast distribution trees that can be instantiated by the MVPN service. Finally, the **vrf-target** command allows configuration of a Route-Target extended community value that may or may not be the same value as that used for unicast VPN membership. The Route-Target value is signalled as part of the Auto-Discovery process (detailed later in this section) and its function is exactly the same as for a unicast VPN; it allows each PE to discover the PEs that belong to a given MVPN. If any other PE has the advertised Route-Target value configured for import into a VRF, it treats the advertising PE as a member of the MVPN.

Output 10-17: MVPN configuration using P2MP mLDP

```

vprn "one" {
  pim {
    interface "link-to-CE1" {
    }
  }
  mvpn {
    c-mcast-signaling bgp
    auto-discovery {
      type bgp
    }
    vrf-target {
      community "target:64496:1"
    }
    provider-tunnel {

```

```

        inclusive {
            mldp {
                admin-state enable
            }
        }
        selective {
            data-threshold {
                group-prefix 224.0.0.0/4 {
                    threshold 100
                }
            }
            mldp {
                admin-state enable
                maximum-p2mp-spmsi 32
            }
        }
    }
}
bgp-ipvpn {
    mpls {
        admin-state enable
        route-distinguisher "64496:1"
        vrf-target {
            import-community "target:64496:1"
            export-community "target:64496:1"
        }
        auto-bind-tunnel {
            resolution filter
            resolution-filter {
                sr-isis true
            }
        }
    }
}
interface "link-to-CE1" {
    ipv4 {
        primary {
            address 192.168.100.1
            prefix-length 30
        }
    }
    sap 1/1/c3/1:200 {
    }
}
}

```

If an RSVP-TE P2MP LSP is used to construct the I-PMSI (and potentially one or more S-PMSIs) the configuration requirement is largely the same as mLDP with a couple of notable differences. Although the example continues to use mLDP as a transport tunnel, *Output 10-18* shows the differences between mLDP and RSVP-TE. Within the **inclusive** and **selective** PMSI contexts, the **rsvp** keyword is used to indicate the use of RSVP-TE P2MP LSPs. Within the same context, an **lsp-template** is also referenced. The **lsp-template** is configured under the **mpls** context and simply defines the characteristics that should be used to signal the P2MP Source-to-Leaf (S2L) sub-LSPs. For P2MP LSPs the **lsp-template** must use the **type p2mp** and must reference a **default-path** that defines the use of strict and/or loose hops or a dynamically computed path for the source-to-leaf (S2L) sub-LSP. In this example the **lsp-template** references a **default-path** called “*dynamic*” which has no strict/loose hops defined and therefore uses a dynamically computed path for each S2L sub-LSP.

Output 10-18: MVPN configuration using P2MP RSVP-TE

```

mpls {
  path "dynamic" {
    admin-state enable
  }
  lsp-template "vrf-one-p2mp-tree" {
    admin-state enable
    type p2mp-rsvp
    default-path "dynamic"
  }
}
vprn "one" {
  mvpn {
    c-mcast-signaling bgp
    auto-discovery {
      type bgp
    }
    vrf-target {
      community "target:64496:1"
    }
    provider-tunnel {
      inclusive {
        rsvp {
          admin-state enable
          lsp-template "vrf-one-p2mp-tree"
        }
      }
      selective {
        rsvp {
          admin-state enable
          lsp-template "vrf-one-p2mp-tree"
        }
      }
    }
  }
}
}

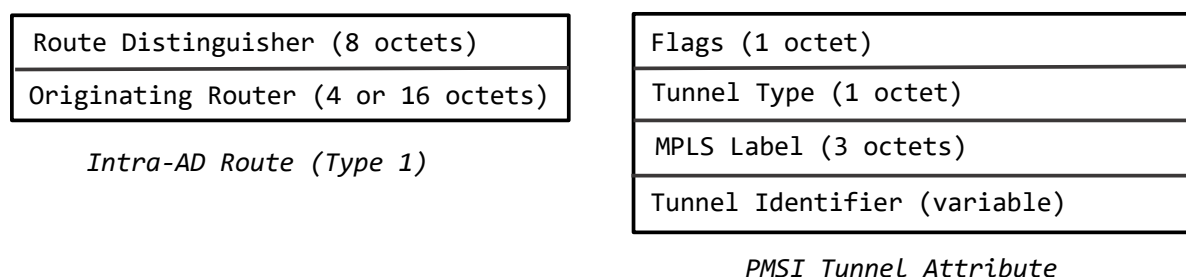
```

To commence Auto-Discovery/tunnel binding within a common AS, PE routers within an MVPN each source an Intra-AS I-PMSI A-D route (Route Type 1). The NLRI of the Intra-AS I-PMSI A-D route is straightforward and consists of an 8-byte Route-Distinguisher followed by the originating router's IP address. The Inter-AS A-D route (Route Type 2) differs only that the 8-byte Route-Distinguisher carries a 4-byte AS number instead of the originating router's IP address. Each of the A-D routes carries a P-Multicast Service Interface Tunnel (PMSI) Tunnel attribute to identify the type of Provider-Tunnel (P-tunnel) that will be used for this MVPN instance. The PMSI Tunnel attribute is an optional transitive attribute that includes the tunnel type (RSVP-TE, mLDP, PIM-SSM, PIM-SM, Ingress replication, etc), tunnel identifier, MPLS label, and a Flags field.

The L flag is currently the only flag specified in the Flags field and is used to indicate 'Leaf Information Required'. When this flag is set, the receiving PE must respond by generating a Leaf A-D Route (Route Type 4) indicating that it needs to join, or be joined to, the signalled PMSI Tunnel. This allows the originating PE to elicit the leaves of the tree and is useful when the PE wants to know who the receivers are before selecting a P-tunnel, referred to as Explicit-Tracking. A potential use-case will be described later in this section.

The MPLS Label field offers the ability to carry a label in the high-order 20 bits of this 3-octet field. When multiple MVPNs are aggregated onto a single P-multicast tree, some de-multiplexing information is required to allow the egress PE to determine to which MVPN the packet belongs. This is the function of the upstream assigned MPLS label signalled in the PMSI Tunnel attribute. When the de-multiplexing label is used, it is placed immediately beneath the P-multicast tree header. Aggregate trees are not currently supported in SR-OS. Whilst the intention of aggregate trees is to reduce P-multicast tree state, they are useful only when there is a high congruency of customer sites between different MVPN instances.

Figure 10-7: Intra-AS A-D Route (Type 1) with PMSI Tunnel Attribute



The Tunnel Identifier field identifies the type of tunnelling technology that will be used to establish the PMSI tunnel. *Table 10-2* lists the possible tunnel types together with the tunnel identifier for each tunnel type. For application with MVPN, SR-OS supports the RSVP-TE P2MP LSP (0x01), mLDP P2MP LSP (0x02), PIM-SSM tree (0x03), PIM-SM tree (0x04), BIER (0x0B), SR-MPLS P2MP tree (0x0C), and SRv6 P2MP tree (0x0D) tunnel types.

Table 10-2: PMSI Tunnel Identifiers

Value	Tunnel Type	Tunnel Identifier
0x00	No Tunnel Information	Not present (used for Explicit Tracking)
0x01	RSVP-TE P2MP LSP	<Tunnel ID, Extended Tunnel ID, P2MP ID>
0x02	mLDP P2MP LSP	Opaque P2MP FEC element
0x03	PIM-SSM Tree	<Sender Address, P-Multicast Group>
0x04	PIM-SM Tree	<P-root Node Address, P-Multicast Group>
0x05	BIDIR-PIM Tree	<Sender Address, P-Multicast Group>
0x06	Ingress Replication	Unicast tunnel endpoint IP address
0x07	mLDP MP2MP LSP	Opaque MP2MP FEC element
0x08	Transport Tunnel	
0x09	Unassigned	
0x0A	Assisted Replication Tunnel	Dependent on AR-REPLICATOR or AR-LEAF
0x0B	BIER	Sub-domain ID, BFR-ID, BFR-Prefix
0x0C	SR-MPLS P2MP tree	Root Address, Tree-ID
0x0D	SRv6 P2MP tree	Root Address, Tree-ID

Debug 10-2 shows the Intra-AS I-PMSI A-D route sourced by R3 as received at R1. The route carries a single NLRI with the RD set to the RD of the VRF and the Originator field set to the system address (in this case R3, 192.0.2.3). The extended community Route Target value determines whether the received route is eligible for import to a given MVPN and the semantics of the use of Route Target are exactly the same as used for BGP-MPLS IP-VPN. The PMSI Tunnel Attribute shows a Tunnel-type of LDP P2MP LSP (type 2), and when mLDP is the tunnel type the Tunnel Identifier is a P2MP Forwarding Equivalence Class (FEC) which is shown as the Root-Node 192.0.2.3 together with an opaque value of 0x2001 (8193).

Debug 10-2: MVPN-IPv4 Intra-AS A-D route advertised by R3

```
118 2025/09/22 17:22:05.306 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 105
  Flag: 0x90 Type: 14 Len: 23 Multiprotocol Reachable NLRI:
    Address Family MVPN_IPV4
    NextHop len 4 NextHop 192.0.2.3
    Type: Intra-AD Len: 12 RD: 64496:1 Orig: 192.0.2.3
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 0 AS Path:
  Flag: 0x80 Type: 4 Len: 4 MED: 0
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
  Flag: 0xc0 Type: 8 Len: 4 Community:
    no-export
  Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.3
  Flag: 0x80 Type: 10 Len: 4 Cluster ID:
    192.0.2.10
  Flag: 0xc0 Type: 16 Len: 8 Extended Community:
    target:64496:1
  Flag: 0xc0 Type: 22 Len: 22 PMSI:
    Tunnel-type LDP P2MP LSP (2)
    Flags: (0x0) [Type: None BM: 0 U: 0 Leaf: not required]
    MPLS Label 0
    Root-Node 192.0.2.3, LSP-ID 0x2001
```

The same Intra-AS A-D route advertised by R3 is shown in *Figure 10-8* where the MP_REACH_NLRI and the PMSI Tunnel attributes are both expanded to show their contents. The NLRI is minimal consisting of the Route-Distinguisher 64496:1 and an Originating Router field of R3 (192.0.2.3). The PMSI Tunnel Attribute shows the mLDP P2MP FEC for R3's system address 192.0.2.3 with an LSP-ID of 0x2001 or 8193. When other PE routers in the MVPN instance receive the Intra-AS A-D route with this PMSI Tunnel Attribute they will send an mLDP Label Mapping message to join the multicast distribution tree rooted at R3. Similarly, all of the other PE routers in the MVPN instance will also instantiate mLDP distribution trees rooted at their own system addresses to create an any-to-any mLDP mesh.

Figure 10-8: Intra-AS A-D route (Type 1) with PMSI Tunnel Attribute

```

- Border Gateway Protocol - UPDATE Message
Marker: ffffffffffffffffffffffffff
Length: 128
Type: UPDATE Message (2)
Withdrawn Routes Length: 0
Total Path Attribute Length: 195
+ Path attributes
+ Path Attribute - MP_REACH_NLRI
+   Flags: 0x00, Optional, Extended-Length, Non-transitive, Complete
+   Type Code: MP_REACH_NLRI (14)
+   Length: 23
+   Address family identifier (AFI): IPv4 (1)
+   Subsequent address family identifier (SAFI): MCAST-VPN (5)
+   Next hop: 192.0.2.3
+   Number of Subnetwork points of attachment (SNPA): 0
+ Network Layer Reachability Information (NLRI)
+   Route Type: Intra-AS I-PMSI A-D route (1)
+   Length: 12
+   Intra-AS I-PMSI A-D route (12 bytes)
+   Route Distinguisher: 64496:1
+   Originating Router: 192.0.2.3
+ Path Attribute - ORIGIN: IGP
+ Path Attribute - AS_PATH: empty
+ Path Attribute - MULTI_EXIT_DISC: 0
+ Path Attribute - LOCAL_PREF: 100
+ Path Attribute - COMMUNITIES: NO_EXPORT
+ Path Attribute - ORIGINATOR_ID: 192.0.2.3
+ Path Attribute - CLUSTER_LIST: 192.0.2.10
+ Path Attribute - EXTENDED_COMMUNITIES
+ Path Attribute - PMSI_TUNNEL_ATTRIBUTE
+   Flags: 0xc0, Optional, Transitive, Complete
+   Type Code: PMSI_TUNNEL_ATTRIBUTE (22)
+   Length: 22
+   Flags: 0
+   Tunnel Type: mLDP P2MP LSP (2)
+   0000 0000 0000 0000 0000 .... = MPLS Label: 0
+   Tunnel ID: Type: P2MP root node: 192.0.2.3 Id: 8193
+   mLDP P2MP FEC element type: P2MP (6)
+   mLDP P2MP FEC element address family: IPv4 (1)
+   mLDP P2MP FEC element address length: 4
+   mLDP P2MP FEC element root node address: 192.0.2.3
+   mLDP P2MP FEC element opaque length: 7
+   mLDP P2MP FEC element opaque value type: Generic LSP Identifier (1)
+   mLDP P2MP FEC element opaque value length: 4
+   mLDP P2MP FEC element opaque value unique Id: 8193

```

Output 10-19 shows the three Intra-AS A-D routes received at router R1 from the other PE routers participating in the MVPN instance. The contents of the NLRI are contained in the RD and OriginatorIP fields from routers R3 (192.0.2.3), R4, (192.0.2.4), and R6 (192.0.2.6) respectively. All three routes are valid, used, and best.

Output 10-19: MVPN-IPv4 Intra-AD routes received at R1

```

A:admin@R1# show router bgp routes mvpn-ipv4 type intra-ad
=====
BGP Router ID:192.0.2.1          AS:64496          Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP MVPN-IPv4 Routes
=====

```

Flag	RouteType	OriginatorIP	LocalPref	MED
	RD	SourceAS	Path-Id	IGP Cost
	Nexthop	SourceIP		Label
	As-Path	GroupIP		
u*>i	Intra-Ad	192.0.2.3	100	0
	64496:1	-	None	-
	192.0.2.3	-		
	No As-Path	-		
u*>i	Intra-Ad	192.0.2.4	100	0
	64496:1	-	None	-
	192.0.2.4	-		
	No As-Path	-		

```

u*>i  Intra-Ad          192.0.2.6          100          0
      64496:1          -              None          -
      192.0.2.6        -
      No As-Path        -
-----
Routes : 3
=====

```

When a PE router receives an Intra-AS I-PMSI A-D route (or Inter-AS I-PMSI A-D route) it must join to the multicast distribution tree rooted at the sender of the route. Depending on the tunnelling technique used to create the multicast distribution tree, there are numerous ways this can be accomplished which can be broadly categorised into three categories: receiver-driven, source-driven, and static/headend local.

- The receiver-driven approach is applicable to PIM-SM, PIM-SSM, and mLDP. For example, when a PE router receives an Intra-AS I-PMSI A-D route with a PMSI Tunnel attribute and the tunnel type is set to mLDP, the receiving PE advertises an LDP Label Mapping message upstream that includes a P2MP FEC for the {Root Node, LSP-ID} advertised in the PMSI Tunnel attribute together with a Label value that will be used in the downstream forwarding path. This is propagated upstream towards the root and may or may not be grafted onto the P2MP LSP of another leaf node. Because the receiver initiates the P2MP LSP it is said to be receiver-driven. In a similar vein, if PIM-SM or PIM-SSM were signalled as the tunnel type in the PMSI Tunnel attribute, the PE receiving the A-D route would send a PIM Join upstream towards the RP (PIM-SM) or Source (PIM-SSM).
- If the tunnel type of the PMSI Tunnel attribute is set to P2MP RSVP-TE, each PE participating in the MVPN learns of all the leaf PEs through I-PMSI A-D routes and then sends one or more RSVP Joins downstream to create P2MP S2L sub-LSP to all of the leaves of the tree. Each PE subsequently adds or removes S2L sub-LSPs as PEs are added to the MVPN or removed from it. In this case the tree is said to be a ‘source-driven’ tree.
- Other tunnel types can be categorised as static or behaviour that is local to the headend. When using BIER, the root of a multicast distribution tree learns the leaves of the tree using Intra- or Inter-AS A-D routes and can subsequently derive a BIER header bit string to represent each of the BFERs. This is purely a local action requiring no MVPN-specific signaling. Similarly, P2MP SR-MPLS and SRv6 trees can only be statically provisioned in SR-OS which again requires no MVPN-specific signaling.

Regardless of the approach that is used, assuming a successful exchange of Intra- or Inter-AS A-D routes takes place and LSPs are successfully established, each PE forming the MVPN becomes the root of a P-multicast tree with all other participating PEs forming leaves of the tree. There are exceptions. The default as described above assumes that a PE is both a multicast sender and receiver. It is possible to restrict a PE to either sender or receiver using the **mdt-type** command, which can be set to **sender-only** or **receiver-only**. When set to **receiver-only**, a PE advertises its Intra-/Inter-AS A-D I-PMSI route without a PMSI Tunnel attribute. This means of course that it will not become the root of a tree, but it will join as a leaf node to multicast distribution trees rooted at other nodes. When set to **sender-only**, the PE node will not join as a leaf node to any multicast distribution tree advertised in a PMSI

Tunnel attribute by another MVPN member PE. The purpose of both of these options is to avoid creating a full mesh of MDT/I-PMSI across all PE nodes participating in the MVPN instance. The default setting is **sender-receiver**.

Because P2MP LDP is used in the test topology the presence of P2MP LSPs can be verified as shown in *Output 10-20*. In this output there are two entries for LSP ID 8193 and R1 (192.0.2.1) as root. The first is with peer R2 (192.0.2.2:0) with a next hop (EgrNH) of 192.168.0.2 and egress interface of 1/1/c1/1:100. The egress label (EgrLbl) is 524267. The second entry is with peer R4 (192.0.2.4:0) with a next hop of 192.168.0.10 and egress interface of 1/1/c2/1:100. The egress label for this entry is 524241. These are both P2MP LDP bindings that will extend to all the leaf PEs within the MVPN instance.

The interface field in the output shows an integer of 73729 for the P2MP FEC with LSP-ID 8193 and root address of 192.0.2.1 (R1). This value will differ for P2MP FECs advertised in LDP Mapping messages towards other root PEs. SR-OS prepends this value to the term *mpls-if* when naming I-PMSI or S-PMSI tunnels. For example, the name mpls-if-73729 would refer to an I-PMSI rooted at R1 with LSP-ID 8193.

Output 10-20: P2MP LDP FECs at R1

```
A:admin@R1# show router ldp bindings p2mp p2mp-id 8193 root 192.0.2.1

=====
LDP Bindings (IPv4 LSR ID 192.0.2.1)
              (IPv6 LSR ID 3fff:ffe::1)
=====
Label Status:
    U - Label In Use, N - Label Not In Use, W - Label Withdrawn
    WP - Label Withdraw Pending, BU - Alternate For Fast Re-Route
    e - Label ELC
FEC Flags:
    LF - Lower FEC, UF - Upper FEC, M - Community Mismatch,
    BA - ASBR Backup FEC
=====
LDP Generic IPv4 P2MP Bindings
=====
P2MP-Id
RootAddr          Interface
Peer
IngLbl            EgrLbl
EgrNH             EgrIf/LspId
-----
8193
192.0.2.1          73729
192.0.2.2:0        524267
--
192.168.0.2        1/1/c1/1:100

8193
192.0.2.1          73729
192.0.2.4:0        524241
--
192.168.0.10       1/1/c2/1:100

-----
No. of Generic IPv4 P2MP Bindings: 2
=====
```

Recall that in the draft-rosen implementation where PIM is used to construct the PMSI (MDT), VRF-specific PIM adjacencies are established across the P-multicast instance (see *Output 10-12*) to discover MVPN neighbours and maintain adjacencies. In the MVPN specification when the PMSI is instantiated using non-PIM transport protocols such as mLDP, RSVP-TE, or BIER, PIM is clearly not present. Moreover, even where PIM is used to instantiate the PMSI, the Auto-Discovery process is achieved purely using BGP, and as a result there is no requirement to send PIM Hellos to discover neighbours and maintain adjacencies. *Output 10-21* shows this from R1's perspective. The adjacency to CE1 (192.168.100.2) is PIM and has an expiration time suitable to that protocol and will expire in 1 minute 26 seconds unless a PIM Hello is received, but the adjacencies to R3 (192.0.2.3), R4 (192.0.2.4) and R6 (192.0.2.6) have expiration times of *never*. This is because Auto-Discovery was facilitated by an MP_REACH_NLRI containing an Intra-AS I-PMSI A-D route, and if a PE is no longer available it simply withdraws that Update by sending a corresponding MP_UNREACH_NLRI. There is no requirement for PIM to maintain the adjacency.

Output 10-21: MVPN-specific PIM adjacencies at R1

```
A:admin@R1# show router 1 pim neighbor
```

```
=====
```

```
PIM Neighbor ipv4
```

```
=====
```

Interface Nbr Address	Nbr DR Prty	Up Time	Expiry Time	Hold Time
link-to-CE1 192.168.100.2	1	0d 16:13:23	0d 00:01:26	105
mpls-if-73730 192.0.2.6	1	0d 16:05:12	never	65535
mpls-if-73732 192.0.2.4	1	0d 16:04:21	never	65535
mpls-if-73738 192.0.2.3	1	0d 00:15:23	never	65535

```
-----
```

```
Neighbors : 4
```

```
=====
```

In the test topology PIM is used on the PE-CE links, but the MVPN is configured to use BGP for C-Multicast signalling. Referring back to *Figure 10-6*, CE1 has a source connected at address 172.31.1.3 that is sending traffic to group address 239.2.78.48. CE3 has an interested receiver for this group so as the last-hop router CE3 sends a PIM (S,G) Join towards the source. When this (S,G) Join is received at R3 it is not propagated any further towards the source using PIM but rather is mapped into a C-Multicast Source-Tree Join route (Route Type 7) and forwarded in an MP_REACH_NLRI Update. The format of the NLRI is shown in *Figure 10-9* and is the same for both a Shared-Tree Join, and a Source-Tree Join. For a Shared-Tree Join route the Multicast source field contains the C-RP address. For a Source-Tree Join route, the Multicast source field contains the C-S address.

Figure 10-9: C-Multicast Join route NLRI

Route Distinguisher (8 octets)
Source AS (4 octets)
Multicast Source Length (1 octet)
Multicast Source (variable)
Multicast Group Length (1 octet)
Multicast Group (variable)

The corresponding debug output taken at R3 as it advertises the Source-Tree Join for (172.31.1.3, 239.2.78.48) is shown in *Debug 10-3*. As this C-Multicast route is a Source-Tree Join route, the Multicast Source field contains the C-source address of 172.31.1.3. The C-Multicast Route also contains the RD of the Multicast VRF, Source AS, and C-group address that is being joined.

The ‘Target’ extended community present in the Source-Tree Join the debug output is not the same as the Route Target used for unicast routing and/or MVPN Auto-Discovery (which in this case is target:64496:1). So how does the receiving PE know to which MVPN this C-Multicast Update belongs? The MVPN specification introduces a BGP extended community called the “VRF Route Import”. It is constructed as an IP address (system address) followed by a 2-octet integer and allows the community to uniquely identify a VRF on a given PE. The VRF Route Import extended community is appended to every unicast VPN-IP prefix sourced by PEs belonging to the VPN unless it is known that this VPN-IP address could never be a multicast source and/or RP. For example, the Source-Tree Join advertised by R3 is destined towards a source 172.31.1.3 connected to CE1, which in turn is connected to R1. In *Output 10-22* the VPN-IPv4 prefix advertised by R1 for the prefix 172.31.1.0/24 carries the VRF Route Import community value 192.0.2.1:2, which uniquely identifies R1 and the specific VRF expressed as an integer at R1. This is the value that is used in the Route Target extended community by R3 when it sends the Source-Tree Join and ensures that the Update is imported only at R1. In this case, R1 is referred to as the Upstream Multicast Hop (UMH) from the perspective of R3 or any other receiver PEs.

The RD value used in the NLRI of the Source-Tree Join can also vary. In the test set-up of *Figure 10-6* the PEs in the MVPN share a common RD (64496:1). However, if different RD’s were in use within the MVPN, the NLRI in the Source-Tree Join advertised by R3 would contain the RD value of the VPN-IPv4 prefix advertised by R1 for the prefix 172.31.1.0/24. The reason for using the RD of the Upstream Multicast Hop is to try to alleviate load on the PE connected to the source. For example, if the source PE and receiver PE’s peer to a Route-Reflector, the Route-Reflector receives multiple Source-Tree Joins but only forwards the best path to the source. This use of Route Target and Route Distinguisher values derived from the VPN-IPv4 prefix sourced by the Upstream Multicast Hop is applicable to Source-Tree Joins, Shared-Tree Joins, and Leaf AD Routes, where there is only one intended recipient.

Debug 10-3: Source-Tree Join advertised by R3

```

82 2025/09/23 14:03:42.408 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 76
    Flag: 0x90 Type: 14 Len: 33 Multiprotocol Reachable NLRI:
        Address Family MVPN_IPV4
        NextHop len 4 NextHop 192.0.2.3
        Type: Source-Join Len:22 RD: 64496:1 SrcAS: 64496 Src: 172.31.1.3 Grp:
239.2.78.48
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0xc0 Type: 8 Len: 4 Community:
        no-export
    Flag: 0xc0 Type: 16 Len: 8 Extended Community:
        target:192.0.2.1:2

```

Output 10-22: VRF-Import Extended Community

```

A:admin@R1# show router bgp routes 172.31.1.0/24 vpn-ipv4 hunt | match Community
Community          : target:64496:1 vrf-import:192.0.2.1:2 source-as:64496:0

```

Figure 10-10 shows a packet capture of the same C-Multicast Source-Tree Join advertised by R3 with the MP_REACH_NLRI attribute and Extended Communities attribute expanded to clarify the encoding of the message.

Figure 10-10: C-Multicast Join route advertised by R3

```

+ Border Gateway Protocol - UPDATE Message
  Marker: #####
  Length: 99
  Type: UPDATE Message (2)
  Withdrawn Routes Length: 0
  Total Path Attribute Length: 76
  + Path attributes
    + Path Attribute - MP_REACH_NLRI
      + Flags: 0x90, Optional, Extended-Length, Non-transitive, Complete
      + Type Code: MP_REACH_NLRI (14)
      + Length: 33
      + Address family identifier (AFI): IPv4 (1)
      + Subsequent address family identifier (SAFI): MCAST-VPN (5)
      + Next hop: 192.0.2.3
      + Number of Subnetwork points of attachment (SNPA): 0
    + Network Layer Reachability Information (NLRI)
      + Route Type: Source Tree Join route (7)
      + Length: 22
      + Source-Tree Join route (22 bytes)
        + Route Distinguisher: 64496:1
        + Source AS: 64496
        + Multicast Source Length: 32
        + Multicast Source Address: 172.31.1.3
        + Multicast Group Length: 32
        + Multicast Group Address: 239.2.78.48
    + Path Attribute - ORIGIN: IGP
    + Path Attribute - AS PATH: empty
    + Path Attribute - MULTI_EXIT_DISC: 0
    + Path Attribute - LOCAL_PREF: 100
    + Path Attribute - COMMUNITIES: NO_EXPORT
    + Path Attribute - EXTENDED_COMMUNITIES
      + Flags: 0xc0, Optional, Transitive, Complete
      + Type Code: EXTENDED_COMMUNITIES (16)
      + Length: 8
      + Carried extended communities: (1 community)
        + Route Target: 192.0.2.1:2 [Transitive IPv4-Address-Specific]
          + Type: Transitive IPv4-Address-Specific (0x01)
          + Subtype (IPv4): Route Target (0x02)
          + IPv4 address: 192.0.2.1
          + 2-Octet AN: 2

```



Note: A few words about Upstream Multicast Hop (UMH) selection. In the example, the route towards the C-source 172.31.1.3 is known only via R1, so the process of selecting the UMH is straightforward. However, in the

event that a number of PEs have advertised the same VPN-IPv4 prefix for the C-source (or C-RP) subnet a mechanism is needed to ensure that Source-Tree/Shared-Tree Joins from all PEs are only sent to a single PE in order to avoid duplicate packets being sent onto the PMSI. This mechanism is known as the 'Single Forwarder Selection' and uses a process known as the UMH selection process to ensure that a single PE is selected. When determining the UMH for a particular C-source/C-RP, a PE groups all PEs that have advertised a unicast route for the C-source/C-RP subnet (not just the PE with the preferred unicast route) and place them into a 'UMH-route-candidate-set'. From this set, the default behaviour for UMH selection is to use the numerically highest IP address, which, of course ensures that all PEs within the multicast domain select the same UMH. This in turn avoids duplicate packets being sent onto the PMSI. This may not be guaranteed if, for example, UMH selection is based on the preferred unicast route towards the C-source/C-RP, particularly if the preferred unicast route selection is based on IGP distance to the BGP Next-Hop. However, SR-OS provides for alternative UMH selection criteria using one of the following:

- a) A hash-based selection algorithm. This allows for load-balancing of UMH per (C-root, C-G) state.
- b) Tunnel-status monitoring. This uses a unidirectional BFD-like mechanism between root and leaf that allows the leaf to monitor the tunnel health and switch to a backup UMH upon failure but is applicable only to RSVP I-PMSIs).
- c) Unicast route preference.

These options can be configured within the **mvpn** context using the command **umh-selection**.

When R1 receives the Source-Tree Join shown in *Debug 10-3* it creates the associated PIM state for C-group 239.2.78.48 within the MVPN and propagates the Join upstream towards the source (CE1) using PIM. As first-hop router, CE1 subsequently forwards multicast traffic downstream towards R1, at which point R1 recognises that a multicast source is active. If the MVPN instance supports both shared trees and source trees (or just shared trees), the PE detecting an active source advertises a Source Active A-D route (Route Type 5) to all other PE routers in the MVPN.



Note: It is assumed that the MVPN supports shared trees if group ranges outside of the IANA allocated SSM range of 232.0.0.0/8 are in use (even if they use SSM outside of this range). The required range of SSM groups is configurable under the base of VPRN **pim** context using the **ssm-groups group-range** command.

Figure 10-11: Source-Active A-D Route NLRI

Route Distinguisher (8 octets)
Multicast Source Length (1 octet)
Multicast Source (variable)
Multicast Group Length (1 octet)
Multicast Group (variable)

The purpose of the Source Active A-D route is to avoid packet duplication onto the PMSI in certain circumstances. Previously this section discussed the selection of a common and consistent Upstream Multicast Hop (UMH) to avoid packet duplication onto the PMSI. However, when an MVPN receiver switches from a shared tree (C-RP based) to a shortest path tree, this consistent UMH selection may not be enough. It is possible that this transition can cause packet duplication where both the RP and the source are forwarding traffic onto the PMSI.

For example, assume that PE1, PE2, PE3, PE4, and PE5 belong to the same MVPN, for which a multipoint-to-multipoint LSP is used to construct a Multi-Directional Inclusive PMSI (MI-PMSI). (Multipoint-to-multipoint LSPs are currently not supported in SR-OS, but the explanation is still valid.) PE1, PE2, and PE3 are on the (C-*, C-G) tree where PE4 is the selected UMH for this group (because the C-RP is connected to PE4). PE2 subsequently switches to the shortest path tree and generates a Source-Tree Join C-multicast route towards PE5, the selected UMH for the C-source. This results in PE1, PE2, and PE3 receiving duplicate traffic for the (C-S, C-G) - from PE4 on the C-RP shared tree and from PE5 on the source tree. The RPF check at the receiver PEs includes checking that the traffic is arriving on the correct interface but does not (and cannot) check if the traffic is arriving from the anticipated PE (the UMH). To avoid this situation, a PE detecting an active C-multicast source advertises the Source Active A-D route for the (C-S, C-G) entry. When other PEs in the MVPN receive this Source Active A-D route, they check if the C-group advertised in that Source Active A-D route matches an active (C-*, C-G) entry in the MVPN. If it does, the receiving PE sends a Source-Tree Join towards the PE originating the Source Active A-D route to receive the (C-S, C-G) traffic. If the receiving PE also has the PMSI as an outgoing interface in the MVPN (C-*, C-G) entry, it transitions that entry to represent (C-*, C-G, rpt) state. In other words, the PE acts as if it had received a PIM (C-*, C-G, rpt) prune from the PMSI, without actually having received one.

The advertising of Source Active A-D routes is not required when an MVPN supports only source-based trees (PIM-SSM) and can be administratively disabled using the **intersite-shared** command within the **mvpn** context. Note that although a PIM-SSM range may be defined within the VPRN PIM instance, this does not preclude PIM-SM from also operating within that same PIM instance - it only precludes PIM-SM from operating within that defined group range. Because router R1 has detected that C-source 172.31.1.3 is active, and this

MVPN instance has the capability to support both shared trees and source trees, it advertises a Source Active A-D route as shown in *Debug 10-4* containing the C-S address 172.31.1.3 and C-G address 239.2.78.48.

Debug 10-4: Source-Active A-D route advertised by R1

```
188 2025/09/23 14:11:38.815 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 72
  Flag: 0x90 Type: 14 Len: 29 Multiprotocol Reachable NLRI:
    Address Family MVPN_IPV4
    NextHop len 4 NextHop 192.0.2.1
    Type: Source-AD Len: 18 RD: 64496:1 Src: 172.31.1.3 Grp: 239.2.78.48
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 0 AS Path:
  Flag: 0x80 Type: 4 Len: 4 MED: 0
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
  Flag: 0xc0 Type: 8 Len: 4 Community:
    no-export
  Flag: 0xc0 Type: 16 Len: 8 Extended Community:
    target:64496:1
```

The Source-Active A-D route is also shown in the packet capture in *Figure 10-12*, where again the MP_REACH_NLRI and Extended Communities attributes are expanded to show the relevant encoding. Note that unlike the Source-Tree Join from R3 that used an IP address-specific Route-Target value to direct the Update towards the chosen UMH R1, the Source-Active A-D route uses the MVPN-wide Route-Target value with 2-octet AS number and 4-octet integer to ensure that all PEs are aware of the active source.

Figure 10-12: Source-Active A-D router advertised by R1

```
= Border Gateway Protocol - UPDATE Message
  Marker: ffffffffffffffffffffffffffffffff
  Length: 95
  Type: UPDATE Message (2)
  Withdrawn Routes Length: 0
  Total Path Attribute Length: 72
  = Path attributes
    = Path Attribute - MP_REACH_NLRI
      Flags: 0x90, Optional, Extended-Length, Non-transitive, Complete
      Type Code: MP_REACH_NLRI (14)
      Length: 29
      Address family identifier (AFI): IPv4 (1)
      Subsequent address family identifier (SAFI): MCAST_VPN (5)
      Next hop: 192.0.2.1
      Number of Subnetwork points of attachment (SNPA): 0
      = Network Layer Reachability Information (NLRI)
        Route Type: Source Active A-D route (5)
        Length: 18
        = Source Active A-D route (18 bytes)
          Route Distinguisher: 64496:1
          Multicast Source Length: 32
          Multicast Source Address: 172.31.1.3
          Multicast Group Length: 32
          Multicast Group Address: 239.2.78.48
      = Path Attribute - ORIGIN: IGP
      = Path Attribute - AS_PATH: empty
      = Path Attribute - MULTI_EXIT_DISC: 0
      = Path Attribute - LOCAL_PREF: 100
      = Path Attribute - COMMUNITIES: NO_EXPORT
      = Path Attribute - EXTENDED_COMMUNITIES
        Flags: 0xc0, Optional, Transitive, Complete
        Type Code: EXTENDED_COMMUNITIES (16)
        Length: 8
        = Carried extended communities: (1 community)
          = Route Target: 64496:1 [Transitive 2-Octet AS-Specific]
            Type: Transitive 2-Octet AS-Specific (0x00)
            Subtype (AS2): Route Target (0x02)
            2-Octet AS: 64496
            4-Octet AN: 1
```

Traffic is forwarded through the I-PMSI and is received by all PE routers that are members of the MVPN instance. The MVPN instance maintains PIM state for the joined group. In this

output example taken at R3 for the (172.31.1.3,239.2.78.48) source and group, the incoming interface is the I-PMSI shown as `mpls-if-73757` and the outgoing interface is the link towards CE3 (`link-to-CE3`). The RPF neighbour is R1 (192.0.2.1) and consequently R1 is the UMH. Traffic is being forwarded to group 239.2.78.48 from source 172.31.1.3 at a rate of ~40Kb/s, noting again that this measurement is taken at the ingress linecard before any egress QoS, rate-limiting, or policy is applied.

Output 10-23: PIM group state at router R3

```
A:admin@R3# show router 1 pim group 239.2.78.48 detail
```

```
=====
PIM Source Group ipv4
=====
```

```
Group Address      : 239.2.78.48
Source Address     : 172.31.1.3
RP Address         : 172.31.100.254
Advt Router        : 192.0.2.1
Flags              : spt                      Type           : (S,G)
Mode               : sparse
MRIB Next Hop      : 192.0.2.1
MRIB Src Flags     : remote
Keepalive Timer Exp: 0d 00:02:35
Up Time            : 0d 00:01:32              Resolved By       : rtable-u

Up JP State        : Joined                   Up JP Expiry       : 0d 00:00:28
Up JP Rpt          : Not Joined StarG         Up JP Rpt Override : 0d 00:00:00
```

```
Register State     : No Info
Reg From Anycast RP: No
```

```
Rpf Neighbor       : 192.0.2.1
Incoming Intf      : mpls-if-73757
Outgoing Intf List : link-to-CE3
```

```
Curr Fwding Rate   : 40.080 kbps
Forwarded Packets  : 274                      Discarded Packets : 0
Forwarded Octets   : 274548                    RPF Mismatches    : 0
Spt threshold      : 0 kbps                     ECMP opt threshold: 7
Admin bandwidth    : 1 kbps
```

```
-----
Groups : 1
=====
```

Figure 10-13 is a packet capture taken on the link between R1 and R2 and shows C-multicast traffic from source 172.31.1.3 to group; 239.2.78.48 encapsulated in the I-PMSI and forwarded by router R1. The encapsulation through the I-PMSI is lightweight and straightforward and consists of a single LDP label (524267). When router R3 received the Intra-AS I-PMSI A-D route from R1 it would have generated an LDP Label Mapping message towards router R1 for the FEC {root 192.0.2.1, LSP-ID 8193}. Upon receiving this Label Mapping message, R3's upstream neighbour on the path to R1, router R2, would also have advertised a Label Mapping message to R1 for the FEC {root 192.0.2.1, LSP-ID 8193} with the label value 524267. It is therefore the label that R1 will use in the downstream forwarding path as previously shown in *Output 10-20*.

Figure 10-13: Packet encapsulated in I-PMSI

```
* Frame 41: 1024 bytes on wire (8192 bits), 1024 bytes captured (8192 bits) on interface br11, id 0
* Ethernet II, Src: RealtekU_63:3a:2b (52:54:00:63:3a:2b), Dst: RealtekU_2a:9f:16 (52:54:00:2a:9f:16)
* 802.1Q Virtual LAN, PRI: 2, DEI: 0, ID: 100
* Multiprotocol Label Switching Header, Label: 524267, Exp: 2, S: 1, TTL: 253
  0111 1111 1111 1110 1011 ..... = MPLS Label: 524267 (0x7ffeb)
  ..... 010 ..... = MPLS Experimental Bits: 2
  ..... 1 ..... = MPLS Bottom Of Label Stack: 1
  ..... 1111 1101 = MPLS TTL: 253
* Internet Protocol Version 4, Src: 172.31.1.3, Dst: 239.2.78.48
* User Datagram Protocol, Src Port: 1024, Dst Port: 1024
* Data (974 bytes)
```

If a PE router that has joined to a C-Multicast group no longer has interested receivers, it sources a withdraw (MP_UNREACH_NLRI) for the previously advertised Source-Tree (or Shared-Tree) Join route. As the receiver behind CE3 no longer wishes to receive the content it sends an IGMP leave which causes CE3 to generate a PIM prune for group 239.2.78.48 towards R3. R3 consequently sources the MP_UNREACH_NLRI for the previously advertised Source-Tree Join route.

Debug 10-5: R3 Source-Tree Join Withdraw

```
88 2025/09/23 14:30:48.926 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 31
  Flag: 0x90 Type: 15 Len: 27 Multiprotocol Unreachable NLRI:
    Address Family MVPN_IPV4
    Type: Source-Join Len:22 RD: 64496:1 SrcAS: 64496 Src: 172.31.1.3 Grp:
    239.2.78.48
```

The A-D route signalling, subsequent P-tunnel instantiation, and C-Multicast routing information so far have been restricted to the I-PMSI, where every PE participating in the MVPN receives every packet forwarded onto the I-PMSI. As previously discussed, in the presence of C-multicast traffic where only a subset of the PEs in the MVPN have interested receivers, the option exists to optimise bandwidth utilisation and move to a Selective PMSI where only that subset of PEs receives the traffic.

Referring again to *Figure 10-6*, the host 172.31.1.3 behind CE1 sends to group 239.1.77.49 and an interested receiver behind CE4 has joined to the group. In this scenario, both R3 and R6 have no interested receivers but receive the C-multicast traffic from 239.1.77.49 over the I-PMSI (in a P2MP mLDP LSP) and simply discard the traffic. To optimise the traffic distribution, R1 monitors the data-threshold configured in the MVPN together with the permitted C-group range. When the threshold has been crossed, R1 signals the move from I-PMSI to S-PMSI using an S-PMSI A-D route (Route Type 3).

Figure 10-14: S-PMSI A-D Route NLRI

Route Distinguisher (8 octets)
Multicast Source Length (1 octet)
Multicast Source (variable)
Multicast Group Length (1 octet)
Multicast Group (variable)
Originating Router's IP Address

Debug 10-6 shows the S-PMSI A-D route sourced from R1 when host 172.31.1.3 exceeds the configured **data-threshold**, where the NLRI contains the RD and originating PE system address as well as the C-source and C-multicast group. PE routers in the MVPN that receive this Update message check to see if they have interested receivers, and if the answer is yes they establish the S-PMSI P-tunnel indicated in the PMSI Tunnel attribute. In this case, the P-tunnel transport protocol is mLDP, and the LSP ID is 8194 (0x2002). If the answer is no and PE routers do not have any interested receivers, they do not join to the S-PMSI tunnel indicated in the PMSI Tunnel attribute. However, they do maintain PIM state for the (S,G) entry and the S-PMSI tunnel in case any C-multicast joins are subsequently received.

After signalling the S-PMSI A-D route, the originating PE connected to the source waits the **data-delay-interval** (default three seconds) before switching the C-multicast traffic from the I-PMSI to the advertised S-PMSI.

Debug 10-6: S-PMSI A-D route advertised by R1

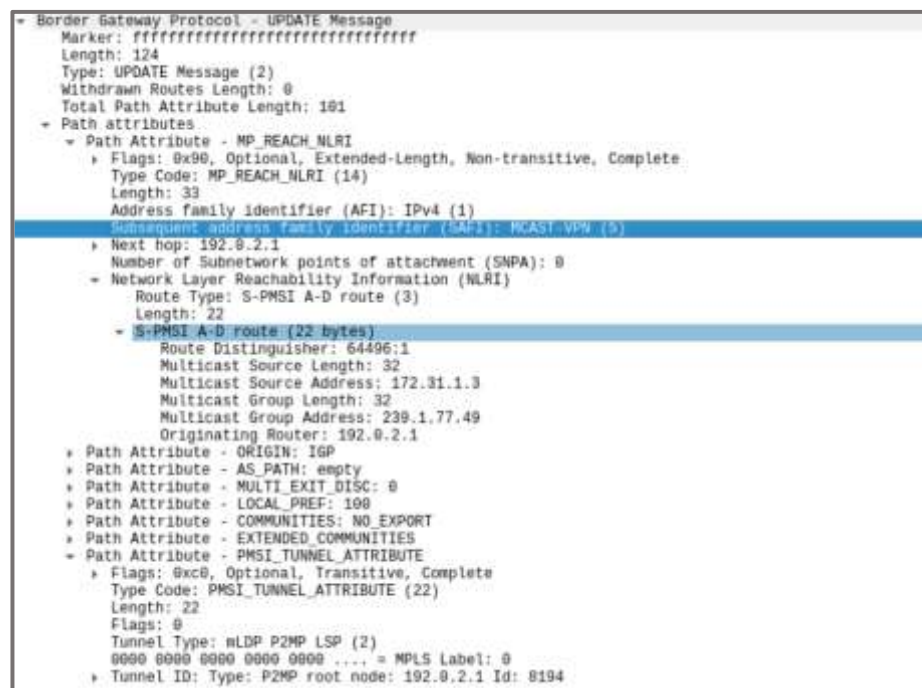
```

197 2025/09/23 17:02:20.742 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 101
    Flag: 0x90 Type: 14 Len: 33 Multiprotocol Reachable NLRI:
        Address Family MVPN_IPV4
        NextHop len 4 NextHop 192.0.2.1
        Type: SPMSI-AD Len: 22 RD: 64496:1 Orig: 192.0.2.1 Src: 172.31.1.3 Grp:
239.1.77.49
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0xc0 Type: 8 Len: 4 Community:
        no-export
    Flag: 0xc0 Type: 16 Len: 8 Extended Community:
        target:64496:1
    Flag: 0xc0 Type: 22 Len: 22 PMSI:
        Tunnel-type LDP P2MP LSP (2)
        Flags: (0x0)[Type: None BM: 0 U: 0 Leaf: not required]
        MPLS Label 0
        Root-Node 192.0.2.1, LSP-ID 0x2002

```


The same S-PMSI A-D route advertised by R1 is shown in *Figure 10-15*, once again with the MP_REACH_NLRI and PMSI Tunnel attributes expanded to illustrate the encoding. The NLRI shows the RD, source of 172.31.1.3, group of 239.1.77.49, and the Originating Router IP address of R1 (192.0.2.1). The PMSI Tunnel attribute again shows a tunnel type of mLDP, the root node of R1 and an LSP ID of 8194.

Figure 10-15: S-PMSI A-D route advertised by R1



Before proceeding, hypothetically consider the scenario where the preceding S-PMSI A-D Route was constructed using an RSVP-TE P2MP LSP instead of an mLDP P2MP LSP. Recall that P2MP LDP LSPs use receiver-driven trees, so upon receipt of an S-PMSI A-D route specifying P2MP LDP as the tunnel-type, the receiving PE has all the information it needs (Root-Node, LSP-ID) to join as a leaf of the tree rooted at the originating PE. In contrast, P2MP RSVP LSPs use source-driven trees, and so some information is needed at the source in order for it to know which PEs it should signal the P2MP S2L sub-LSPs to. In other words, the source needs to know who the leaves of the tree are before it can join them to the tree. In the case of the I-PMSI, that information is derived from the PMSI Tunnel attribute of the Intra-AS I-PMSI A-D Route because they are sourced by every PE in the Multicast VPN and contain each PE's originating IP address. In the case of the S-PMSI however, the PE connected to the source cannot know the subset of remote PEs in the Multicast VPN that have interested receivers and who therefore have a requirement to join to the S-PMSI tree. So, when using RSVP-TE P2MP LSPs for S-PMSI creation, the S-PMSI A-D Route has the L flag (Leaf Required) set in the PMSI tunnel attribute. Remote PEs that have interested receivers subsequently respond with a Leaf A-D Route (Route Type 4) indicating a requirement to join to the PMSI tunnel. This provides the PE connected to the source with the information about who the leaves of the tree should be and allows for RSVP-TE P2MP S2L sub-LSPs to be signalled to each of those leaves.

Figure 10-16: Leaf A-D Route NLRI

Route Key (variable)
Originating Route's IP address

In the example using mLDP, the PE connected to the source continues to monitor the data-threshold of the C-source even after the S-PMSI has been signalled. If the monitored rate falls below the configured threshold, the PE withdraws the advertised S-PMSI A-D route in an MP_UNREACH_NLRI and thereafter reverts the C-multicast traffic back to the I-PMSI.

Debug 10-7: S-PMSI A-D route withdraw from R1

```
198 2025/09/23 17:27:48.692 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 31
    Flag: 0x90 Type: 15 Len: 27 Multiprotocol Unreachable NLRI:
        Address Family MVPN_IPV4
        Type: SPMSI-AD Len: 22 RD: 64496:1 Orig: 192.0.2.1 Src: 172.31.1.3 Grp:
239.1.77.49
```

Using BGP to propagate C-Multicast routing information can have a significant impact on the PIM Join (and to a lesser extent Prune) latency times because, by default, the MCAST-VPN NLRI is subject to the Minimum Route Advertisement Interval (MRAI) at each BGP speaker. The problem even more acute where there are multiple BGP speakers in series, such as where Route-Reflectors are used. To help maintain join latency times, the **rapid-update** command under the global BGP context can be used for the **mvpn-ipv4** and **mvpn-ipv6** Address Families, which bypasses the configured **min-route-advertisement** interval and sends Update messages as soon as they are originated or received.

Output 10-24: Rapid-Update for MVPN-IPv4/IPv6 Address Families

```
router "Base" {
    bgp {
        rapid-update {
            mvpn-ipv4 true
            mvpn-ipv6 true
        }
    }
}
```

Wildcard S-PMSI

The use of the S-PMSI previously described restricts a given P-instance S-PMSI tunnel to carrying a single C-multicast flow. This is because the identifier of the specified C-multicast flow, the (C-S,C-G), is encoded into the NLRI of the S-PMSI route together with the identifier of the P-tunnel which is encoded into the PMSI Tunnel attribute. Each S-PMSI therefore specifies a single C-multicast flow.

In addition, the Auto-Discovery routes described in [45] and [46] and described so far do not specify a method to allow for all customer flows to be carried in a single P-instance tunnel except in the case of the I-PMSI where that tunnel is joined by all the PE routers in the MVPN instance.

Those constraints are tackled by the use of wildcards encoded in the NLRI of S-PMSI Auto-Discovery routes [48], hence the term wildcard S-PMSI. A wildcard is intended to replace the C-S, the C-G, or both, to encode three kinds of wildcards: (C-*,C-*), (C-S,C-*), and (C-*,C-G):

- The (C-*,C-*) is used to indicate that a selective P-tunnel that will be used as it's default P-tunnel to carry all C-multicast flows by default.
- A PE router may be connected to an MVPN site containing several multicast sources that are all sourcing traffic to a common multicast group. The use of the (C-*,C-G) wildcard allows for a single S-PMSI P-tunnel to be used to bind these C-flows to a single P-tunnel. It may also be used to bind all the C-multicast flows of a customer's (*,G) tree to a single P-tunnel.
- A PE router may be connected to an MVPN site containing a single multicast source that is sending traffic to multiple channels. The use of the (C-S,C-*) wildcard allows for a multiple C-multicast flows to be bound to a single S-PMSI P-tunnel (which may of course be diverse from a redundant source's P-tunnel).

When advertising a wildcard S-PMSI the format of the S-PMSI A-D route is identical to that previously depicted in *Figure 10-14*, however, the encoding changes such that a zero-length source or group represents the corresponding wildcard:

- A source wildcard is encoded as a zero-length source field. The 'Multicast Source Length' field contains the value 0x00 and the 'Multicast Source' field is omitted.
- A group wildcard is encoded as a zero-length group field. That is, the 'Multicast Group Length' field contains the value 0x00 and the 'Multicast Group' field is omitted.

A wildcard S-PMSI may be used in place of the I-PMSI, so that all traffic is carried on the S-PMSI by default. In this case, the Intra-/Inter-AS I-PMSI A-D route is either suppressed, or if advertised as part of the Auto-Discovery process, it does not carry a PMSI Tunnel attribute. An S-PMSI A-D route is advertised by all members of the MVPN instance carrying a PMSI Tunnel attribute with the relevant information on how to construct the P-instance tunnel. The S-PMSI A-D route does not however carry any source or group information (it is omitted). PE routers receiving the S-PMSI A-D route only instantiate the P-instance tunnel indicated in the PMSI Tunnel attribute if they have interested receivers.

The use of wildcard S-PMSI in place of the I-PMSI P-tunnel instance is the model supported in SR-OS. The intention of the wildcard S-PMSI as the default P-tunnel is to avoid a full mesh of LSPs between the member PEs of the MVPN instance, thereby reducing control plane signaling and associated state. It is also potentially more bandwidth efficient as only PEs with interested receivers receive the multicast content (instead of all PEs with an I-PMSI).

To demonstrate the use of wildcard S-PMSI the topology shown in *Figure 10-6* is reused, and mLDP is used to instantiate P-tunnels throughout. A key point to reemphasize is that the C-multicast domain operates both PIM-SM and PIM-SSM, and consequently an RP and

Bootstrap Router (BSR) are configured on CE1. The RP advertises its' candidacy for the group range 224.0.0.0/4 and as it is the only RP in the candidate-RP set, it becomes the elected RP. The group-to-RP mapping is thereafter disseminated throughout the C-multicast instance using PIM Bootstrap Messages (BSM). There is a single source connected to CE1 that is sending to group 239.2.77.51 with interested receivers connected to CE3, CE4, and CE6.

Output 10-25 shows the MVPN configuration for enabling the use of wildcard S-PMSI and is applied at the PEs supporting the MVPN instance. Within the **provider-tunnel inclusive** context the **wildcard-spmsi true** command enables the (C-*, C-*) S-PMSI instead of an I-PMSI for this MVPN. When this is enabled the Intra-Inter-AS A-D route is advertised by each member PE, but it does not contain a PMSI Tunnel attribute. An S-PMSI A-D route is also advertised that contains the PMSI Tunnel attribute containing the P-tunnel information with the group and source address fields omitted. A **suppress-ipmsi** command is available within the **inclusive** context that when set to **true** suppresses the advertisement of Intra-/Inter-AS A-D routes, with the Auto-discovery process thereafter using the advertised S-PMSI A-D routes. The use of **suppress-ipmsi** is however mutually exclusive with the use of **bsr**. As indicated in the test topology description, the C-instance runs both ASM and SSM and operates an RP and BSR. When an I-PMSI is in place the BSR messages used to disseminate group-to-RP mappings is encapsulated in the I-PMSI and flooded to all member PEs using multicast. When the I-PMSI is not present an alternate solution is required, and SR-OS offers two options:

- When **bsr unicast** is selected the system floods BSR messages to all neighbour PEs in the MVPN using unicast encapsulation. These BSR unicast messages use the advertising PEs system address as the source address, and the destination PEs system address as the destination address. To ensure these are BSR messages are correctly processed within the MVPN instance a 'system' interface must be configured within the VRF that uses the PEs system address. In *Output 10-25* this is shown as the "System" interface and is set to **loopback true** to ensure that it is operationally active and advertised to other member PEs of the MVPN.
- When **bsr spmsi** is selected, a full-mesh of LSPs is required between the member PEs to exchange the BSR messages. When this option is selected a P-tunnel must be enabled within the **provider-tunnel selective** context.

Since a full-mesh of LSPs somewhat defeats one of the benefits of using wildcard S-PMSI, and given the nominal message rate of BSR, the former option of **bsr unicast** is selected and used within this example.

Output 10-25: MVPN with Wildcard S-PMSI configuration

```
vprn "one" {
  mvpn {
    c-mcast-signaling bgp
    auto-discovery {
      type bgp
    }
    vrf-target {
      community "target:64496:1"
    }
    provider-tunnel {
```

```

        inclusive {
            bsr unicast
            wildcard-spmsi true
            mldp {
                admin-state enable
            }
        }
    }
}
interface "System" {
    loopback true
    ipv4 {
        primary {
            address 192.0.2.1
            prefix-length 32
        }
    }
}
}

```

Before looking further at the signaling aspects of wildcard S-PMSI, it's worth briefly verifying that PIM BSR operates as expected within the C-multicast instance. The **bsr unicast** option is used, and system addresses are configured within the MVPN VRF instances and advertised to other member PEs. Recall that CE1 is the RP and Bootstrap Router and as such bootstrap messages containing group-to-RP mappings are originated by this router. *Output 10-26* shows the group-to-RP mappings learnt through these BSR messages at router R1.

Output 10-26: Group-to-RP mappings learnt at R1

```
A:admin@R1# show router 1 pim rp
```

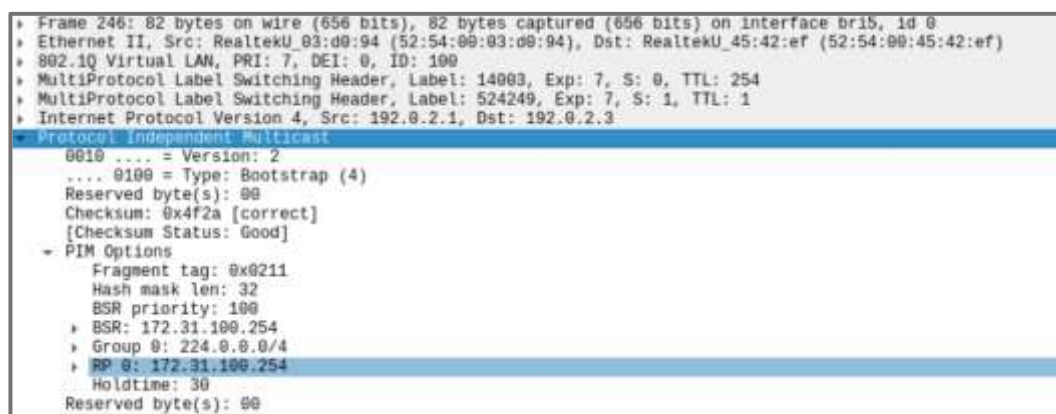
```
=====
PIM RP Set ipv4
=====
```

Group Address	RP Address	Type	Hold Prio	Expiry Time
224.0.0.0/4	172.31.100.254	Dynamic	100 30	0d 00:02:10

```
-----
Group Prefixes : 1
=====
```

Figure 10-17 shows a BSR message forwarded from R1 to R3 encapsulated as unicast MPLS (in this example the VRF uses SR-ISIS). The BSR message has a source address of R1's system address (192.0.2.1) and a destination address of R3 (192.0.2.3). Note that these addresses are routable within the MVPN instance as a result of configuring system addresses within the VRF. The encapsulated PIM packet is a Bootstrap message (type 4) and contains the group range 224.0.0.0/4 with an RP of 172.31.100.254, which is the RP configured at CE1. The MPLS encapsulation has two labels. The upper label of {14003} represents the Node-SID of router R3. The bottom label of {524249} is the per-VRF label advertised by R3 for this MVPN instance.

Figure 10-17: Unicast BSR message from R1 to R3



As a result of the unicast encapsulation, router R3 has learnt the C-multicast instance group-to-RP mappings advertised through BSR messages from CE1. The IP header will be replaced with a new header containing R3's PE-CE interface as the source address and a destination address of 224.0.0.13 used for PIM. The BSR message will then be advertised to CE3.

Output 10-27: Group-to-RP mappings learnt at R3

```
A:admin@R3# show router 1 pim rp
```

```
=====
PIM RP Set ipv4
=====
Group Address                                Hold Expiry
RP Address                                Type      Prio Time Time
-----
224.0.0.0/4
  172.31.100.254                          Dynamic   100   30   0d 00:01:30
-----
Group Prefixes : 1
=====
```

Within the MVPN instance, I-PMSI A-D routes are advertised by all member PEs but do not carry a PMSI Tunnel attribute and are therefore used entirely for the purpose of Auto-Discovery. S-PMSI A-D routes are also advertised by all member PEs, and these carry the PMSI Tunnel attribute together with the wildcard source/group encoding. *Debug 10-8* shows the S-PMSI A-D route advertised by router R3 as received at R1. The NLRI contains a source address of 0.0.0.0 and a group address of 0.0.0.0 representing the wildcards. The PMSI Tunnel attribute indicates a tunnel type of P2MP LDP with a root node of R3 (192.0.2.3) and an LSP ID of 0x2001 (8193).

Debug 10-8: S-PMSI A-D route from R3 received at R1

```
572 2025/09/26 09:42:34.306 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 107
  Flag: 0x90 Type: 14 Len: 25 Multiprotocol Reachable NLRI:
    Address Family MVPN_IPV4
    NextHop len 4 NextHop 192.0.2.3
    Type: SPMSI-AD Len: 14 RD: 64496:1 Orig: 192.0.2.3 Src: 0.0.0.0 Grp: 0.0.0.0
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
```

```

Flag: 0x40 Type: 2 Len: 0 AS Path:
Flag: 0x80 Type: 4 Len: 4 MED: 0
Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
Flag: 0xc0 Type: 8 Len: 4 Community:
    no-export
Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.3
Flag: 0x80 Type: 10 Len: 4 Cluster ID:
    192.0.2.10
Flag: 0xc0 Type: 16 Len: 8 Extended Community:
    target:64496:1
Flag: 0xc0 Type: 22 Len: 22 PMSI:
    Tunnel-type LDP P2MP LSP (2)
    Flags: (0x0)[Type: None BM: 0 U: 0 Leaf: not required]
    MPLS Label 0
    Root-Node 192.0.2.3, LSP-ID 0x2001

```

When a PE receives an S-PMSI A-D route containing the PMSI Tunnel attribute it does not immediately join to the tunnel indicated in the tunnel attribute unless it has interested receivers. As R1 does not have any interested receivers, no P2MP LDP Label Mapping message is sent upstream towards the root node. In essence, MVPN signaling is complete, but no LSPs of the specified tunnel type are instantiated as yet.

Output 10-28: P2MP LDP bindings at R1

```

A:admin@R1# show router ldp bindings p2mp p2mp-id 8193 root 192.0.2.3

=====
LDP Bindings (IPv4 LSR ID 192.0.2.1)
    (IPv6 LSR ID 3fff:ffe::1)
=====
Label Status:
    U - Label In Use, N - Label Not In Use, W - Label Withdrawn
    WP - Label Withdraw Pending, BU - Alternate For Fast Re-Route
    e - Label ELC
FEC Flags:
    LF - Lower FEC, UF - Upper FEC, M - Community Mismatch,
    BA - ASBR Backup FEC
=====
LDP Generic IPv4 P2MP Bindings
=====
P2MP-Id
RootAddr                                Interface
Peer
IngLbl                                  EgrLbl
EgrNH                                  EgrIf/LspId
-----
No Matching Entries Found
=====

```

Routers CE3 and CE4 send (*,G) PIM Joins towards routers R3 and R4 respectively creating the relevant PIM state within the MVPN instance at both PE routers. Both PEs also advertise a Shared-Tree Join towards the selected UMH, which in this case is R1. *Debug 10-9* shows the Shared-Tree Join advertised by router R3. The NLRI encodes the group address as 239.2.77.51 and since this is a (*,G) Join the source field is populated with the C-RP address of 172.31.100.254. Also note that the Route-Target extended community uses an IPv4-address specific value targeted at the system address of router R1. This value is the same as the value used in the Route-Import extended community that R1 attached to the C-RP address when it was advertised in VPN-IPv4 and ensures that the Update message is only imported by router R1.

Debug 10-9: Shared-Tree Join route advertised by R3

```

576 2025/09/26 10:04:22.306 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 90
    Flag: 0x90 Type: 14 Len: 33 Multiprotocol Reachable NLRI:
        Address Family MVPN_IPV4
        NextHop len 4 NextHop 192.0.2.3
        Type: Shared-Join Len:22 RD: 64496:1 SrcAS: 64496 Src: 172.31.100.254 Grp:
239.2.77.51
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0xc0 Type: 8 Len: 4 Community:
        no-export
    Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.3
    Flag: 0x80 Type: 10 Len: 4 Cluster ID:
        192.0.2.10
    Flag: 0xc0 Type: 16 Len: 8 Extended Community:
        target:192.0.2.1:2

```

As both R3 and R4 now have interested receivers, both send LDP Label Mapping messages towards the UMH (advertised in the PMSI Tunnel attribute) to create a root-to-leaf multicast distribution tree. In this case the root address is R1 (192.0.2.1) and the LSP-ID is 8193. This creates two P2MP LDP bindings for LSP-ID 8193 at router R1, both with a root address of 192.0.2.1 (R1). The first entry has a peer of 192.0.2.2:0 with egress label 524267 and egress interface 1/1/c1/1:100. This is the entry towards router R2 who is on the path of the multicast distribution tree between R1 and R3. The second entry has a peer of 192.0.2.4:0 with egress label 524245 and egress interface 1/1/c2/1:100. This is the entry towards router R4.

Output 10-29: P2MP LDP bindings at R1

```

A:admin@R1# show router ldp bindings p2mp

=====
LDP Bindings (IPv4 LSR ID 192.0.2.1)
              (IPv6 LSR ID 3fff:ffe::1)
=====
Label Status:
    U - Label In Use, N - Label Not In Use, W - Label Withdrawn
    WP - Label Withdraw Pending, BU - Alternate For Fast Re-Route
    e - Label ELC
FEC Flags:
    LF - Lower FEC, UF - Upper FEC, M - Community Mismatch,
    BA - ASBR Backup FEC
=====
LDP Generic IPv4 P2MP Bindings
=====
P2MP-Id
RootAddr          Interface
Peer
IngLbl            EgrLbl
EgrNH             EgrIf/LspId
-----
8193
192.0.2.1          73747
192.0.2.2:0

```

```

--
192.168.0.2                                524267
                                           1/1/c1/1:100

8193
192.0.2.1                                73747
192.0.2.4:0
--
192.168.0.10                             524245
                                           1/1/c2/1:100

```

```

-----
No. of Generic IPv4 P2MP Bindings: 2
=====

```

The multicast sender attached to CE1 sends traffic for group 239.2.77.51 which is forwarded by R1 into the wildcard S-PMSI. To avoid packet duplication onto the PMSI, router R1 also advertises a Source-Active A-D route to identify itself as a source and as a UMH for this C-source.

Debug 10-10: Source-Active A-D route advertised by R1

```

578 2025/09/26 10:32:27.308 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 72
    Flag: 0x90 Type: 14 Len: 29 Multiprotocol Reachable NLRI:
        Address Family MVPN_IPV4
        NextHop len 4 NextHop 192.0.2.1
        Type: Source-AD Len: 18 RD: 64496:1 Src: 172.31.1.3 Grp: 239.2.77.51
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 0 AS Path:
    Flag: 0x80 Type: 4 Len: 4 MED: 0
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0xc0 Type: 8 Len: 4 Community:
        no-export
    Flag: 0xc0 Type: 16 Len: 8 Extended Community:
        target:64496:1

```

When the first packet is received by CE3 and CE4 down the shared tree, as last-hop routers both will send PIM Joins directly towards the source, thereby creating (S,G) state.

At this point it's worth summarising the steps taken so far using wildcard S-PMSI:

- All the member PEs of the MVPN instance advertise an Intra-/Inter-AS I-PMSI A-D route for the purpose of Auto-Discovery. The routes do not carry a PMSI Tunnel attribute.
- All the member PEs of the MVPN instance advertise an S-PMSI A-D route with a PMSI Tunnel attribute containing all the information required to instantiate the P-instance tunnel.
- When PEs receive the S-PMSI A-D from other MVPN members they do not automatically join to the P-tunnel multicast distribution tree rooted at the PE advertising the S-PMSI A-D route. Instead, only interested receivers join to the P-instance tunnel rooted at the PE connected to the source or RP. In this case where mLDP is used, PEs with interested receivers send an LDP Label Mapping message towards the root of the tree.

- As a result of the above steps, a full mesh of LSPs is avoided and multicast content is only distributed to MVPN member PEs that requested it (unlike the I-PMSI where all PEs receive all content and discard it if it is not required).

In the example set-up, CE3 and CE4 have joined to the group 239.2.77.51 but CE6 has not. *Output 10-30* shows the incoming interface, outgoing interface list, and current forwarding rate for the group 239.2.77.51 at R3, R4, and R6. The first two entries show the information for R3 and R4 where the forwarding rate is ~800Kb/s, whereas the last entry for R6 shows a forwarding rate of 0Kb/s. The entries for R3 and R4 also show the incoming interface with the nomenclature `mpls-if-<lsp-id>`, but they are denoted with a (W) flag to indicate that they are using wildcard S-PMSIs.

Output 10-30: Forwarding rate for group 239.2.77.51 at R3, R4, and R6

```
A:admin@R3# show router 1 pim group 239.2.77.51 source 172.31.1.3 detail | match
'Incoming Intf|Outgoing Intf|Curr Fwding Rate'
Incoming Intf      : mpls-if-73768(W)
Outgoing Intf List : link-to-CE3
Curr Fwding Rate   : 801.600 kbps

A:admin@R4# show router 1 pim group 239.2.77.51 source 172.31.1.3 detail | match
'Incoming Intf|Outgoing Intf|Curr Fwding Rate'
Incoming Intf      : mpls-if-73732(W)
Outgoing Intf List : link-to-CE4
Curr Fwding Rate   : 801.600 kbps

A:admin@R6# show router 1 pim group 239.2.77.51 source 172.31.1.3 detail | match
'Incoming Intf|Outgoing Intf|Curr Fwding Rate'
Incoming Intf      :
Outgoing Intf List :
Curr Fwding Rate   : 0.000 kbps
```

When an MVPN member PE who has joined the multicast distribution tree no longer has any interested receivers it can remove itself as a leaf from the tree. As the example uses mLDP, this involves sending a Label Release message for the label that was used to graft itself onto the tree rooted at the source. CE3 and CE4 together with their respective PEs R3 and R4 are receiving the multicast content, but the receiver connected to CE3 decides to leave the group. When the PIM prune is received by R3 it sources a withdraw message for both the Shared-Tree Join (used to join to the RP) and the Source-Tree Join (to join directly to the source), packed into the same Update message. This is followed immediately by an LDP Label Release message for the P2MP FEC for {root=192.0.2.1, LSP-ID 8193} used for the multicast distribution tree. This is then acknowledged by its upstream peer R2 (192.0.2.2:0) with a Label Release message. In short, R3 has removed itself from the multicast distribution tree as it has no interested receivers.

Debug 10-11: Shared-Tree/Source-Tree Join withdraw and LDP Label Release message

```
454 2025/09/26 13:12:58.927 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Send BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 55
    Flag: 0x90 Type: 15 Len: 51 Multiprotocol Unreachable NLRI:
```

```

    Address Family MVPN_IPV4
    Type: Shared-Join Len:22 RD: 64496:1 SrcAS: 64496 Src: 172.31.100.254 Grp:
239.2.77.51
    Type: Source-Join Len:22 RD: 64496:1 SrcAS: 64496 Src: 172.31.1.3 Grp: 239.2.77.51

455 2025/09/26 13:12:58.932 BST minor: DEBUG #2001 Base LDP
LDP: LDP
Send Label Withdraw packet (msgId 66142) to 192.0.2.2:0
Protocol version = 1
Label 524240 withdrawn for the following FECs
P2MP: root = 192.0.2.1, T: 1, L: 4, TunnelId: 8193

456 2025/09/26 13:12:58.935 BST minor: DEBUG #2001 Base LDP
LDP: LDP
Recv Label Release packet (msgId 66122) from 192.0.2.2:0
Protocol version = 1
Label 524240 released for the following FECs
P2MP: root = 192.0.2.1, T: 1, L: 4, TunnelId: 8193

```

The use of wildcard S-PMSI can be useful to avoid the requirement for a full mesh of LSPs between member PEs of an MVPN, but the use of BSR in the C-multicast instance does require some additional consideration as to how it will be carried over the provider core. Wildcard S-PMSI can offer bandwidth optimisation, but it is also dynamic in nature – receiver PEs come and go as and when they have interested receivers. Some providers may not favour this dynamic behaviour and prefer the static nature of I-PMSI even at the cost of additional bandwidth.

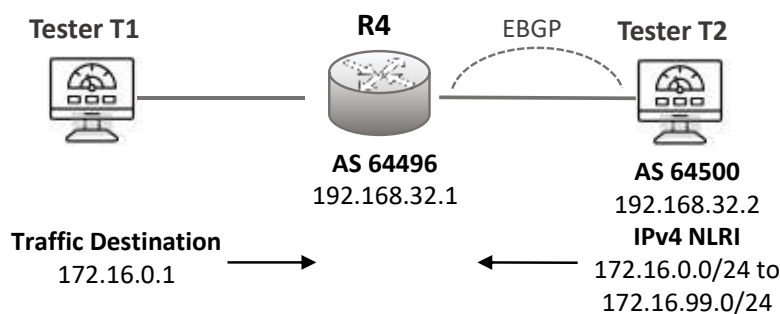
11 Graceful Restart and Error Handling

Graceful Restart is a mechanism applied to many protocols and relies on the fact that modern routers separate the control-plane (RIB) from the data-plane (FIB). Given this assumption, it is entirely possible that a router undergoing a control plane restart can maintain its forwarding state as intact during the restart, thereby significantly reducing the impact of the restart. Equally important, however, is that neighbours of the restarting router do not tear down protocol adjacencies and that they retain any routes learnt from the restarting router. When the restart has been completed, the restarting router also relies on its neighbours to readvertise all prefixes previously advertised to it so that it can re-populate its RIB-In. This is accomplished using the Graceful Restart mechanisms, which vary on a per-protocol basis, but the capability is generically referred to as Non-Stop Forwarding (NSF).

An alternative method is Non-Stop Routing (NSR), which involves redundant Route Processors (CPMs) within a router being fully synchronised so that if the active processor undergoes a control-plane restart the standby processor is able to assume the active state immediately, without interruption to, and fully transparent to adjacent neighbours. It follows therefore that if a router is NSR-capable that it does not rely on its Graceful Restart capable neighbours to re-advertise prefixes after a restart, simply because the synchronised Route Processors ensure that any processor restart is transparent to neighbours. However, implementing Graceful Restart is still useful even for NSR-capable routers. It may not need help from its neighbours during a processor restart, but it must understand Graceful Restart procedures so it can assist a neighbouring non-NSR-capable router undergoing a restart. This is essentially the implementation of SR-OS.

Throughout this chapter the following simple topology will be used to aid understanding of the Graceful Restart process. Router R4 in AS 64496 is externally peered with tester T2 in AS 64500 and learns prefixes 172.16.0.0/24 to 172.16.99.0/24 from that peer. Router R4 receives traffic from tester T1 with a destination address of 172.16.0.1, which R4 forwards to tester T2 based on the prefix learnt from T2. Tester T2 will be the restarting router, with router R4 used to describe the Graceful Restart process.

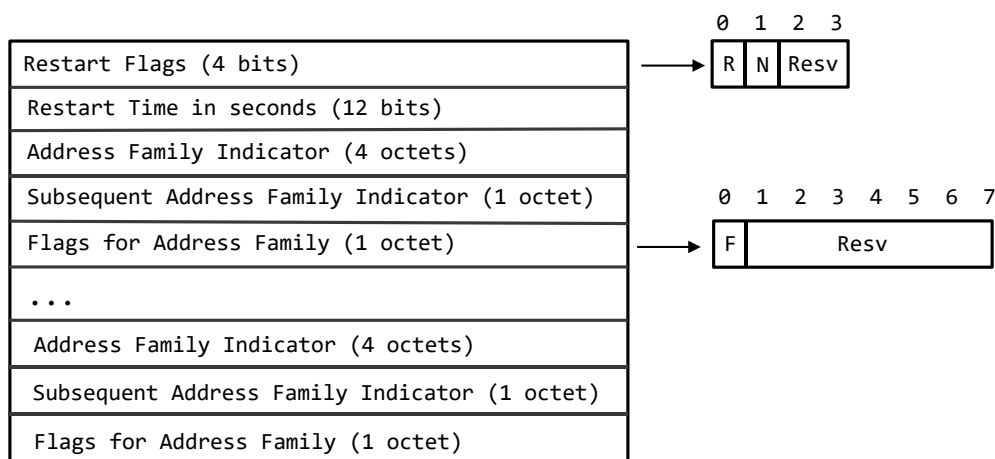
Figure 11-1: Test topology for Graceful Restart and Error Handling



Graceful Restart Mechanism

The Graceful Restart mechanism for BGP is defined in [49] and extended in [82]. In order for peers to agree on the use of Graceful Restart, it is negotiated as a capability during an Open exchange using a capability value encoded as shown in *Figure 11-2*. The most significant bit of the Restart Flags is the Restart State (R) bit, which when set to one it indicates that the BGP speaker has restarted. The second most significant bit is the Notification (N) bit, which indicates the support of Graceful Restart for Notification messages discussed later in this chapter. The Restart Time is the approximate time in seconds it takes for the restarting router to re-establish the BGP session. The AFI and SAFI fields indicate that Graceful Restart is supported for routes of that AFI/SAFI type. The most significant bit in the Flags for Address Family field contains the Forwarding State (F) bit, which when set to one indicates that the forwarding state for routes of this AFI/SAFI was preserved during the previous BGP restart. Only one instance of the capability should appear in the capability advertisement, but as shown in *Figure 11-2*, fields are repeated for support of multiple Address Families.

Figure 11-2: Graceful Restart Capability



The Graceful Restart mechanism also defines an End-of-RIB (EOR) marker, which indicates to a peer of a BGP speaker the completion of routing updates after the session is established. This is useful to allow a router that has just restarted to receive all routes from all of its peers before computing any best-path selection algorithms. The EOR marker is defined as an Update message with no NLRI and empty withdrawn NLRI for IPv4 Address-Family, or an Update message that contains the MP_UNREACH_NLRI attribute with no withdrawn routes for any other Address Family (including IPv4 that is encoded as Multi-Protocol BGP).

During the restart, the neighbour of the restarting router marks routes from the restarting router as stale. It also runs an internal timer (a 'stale-routes timer') and if the restarting router has not re-established the BGP session and re-advertised its routes before expiration of this timer, the neighbouring router deletes the routes marked as stale.

The configuration required to support Graceful Restart at router R4 is shown in *Output 11-1* and consists of the context **graceful-restart** followed by a **stale-routes-time** in seconds,

which defines the maximum time that routes are marked as stale during the restart, after which they are flushed. The output also shows a **restart-time** set to 300 seconds. This is the Restart Time advertised in the Graceful Restart capability and is configured with an explicit value for no other reason than to assist in correlating the capability exchange decoding below. With the inherent NSR capabilities in SR-OS, there should be no reason to require a restart.

Output 11-1: Graceful Restart configuration at R4

```
router "Base" {
  bgp {
    group "EBGP-to-AS64500" {
      min-route-advertisement 1
      split-horizon true
      family {
        ipv4 true
      }
      graceful-restart {
        restart-time 300
        stale-routes-time 300
      }
    }
    neighbor "192.168.32.2" {
      group "EBGP-to-AS64500"
      authentication-key "C3i5c6IP3VMhQFGff3RCM2u0NzsMh8w= hash2"
      peer-as 64500
    }
  }
}
```

During the session establishment between R4 in AS 64496 and tester T2 in AS 64500 the first packet is the Open message sent by router R4, shown in *Debug 11-1*. Of interest here is the Graceful Restart capability field which can be decoded as follows

0x01	Restart Flag (most significant four bits only) not set indicating router has not restarted
0x01 0x2c	Restart Time (twelve bits consisting of second byte and least significant four bits of first byte). Four bits of first byte = 0x01 (decimal 256), second byte = 0x2c (decimal 44) indicating Restart Time of 300 seconds.

The Graceful Restart capability field sent by router R4 contains Restart Flags and Restart Time fields but does not contain any AFI/SAFI fields or Flags for AFI/SAFI fields as shown in *Figure 11-2*. Sending AFI/SAFI fields in the Open message when a BGP session is initially established is not strictly required, so by default they are not present in SR-OS unless **gr-notification true** is configured.

Debug 11-1: Open message from router R4

```
Peer 1: 192.168.32.2 - Send (Passive) BGP OPEN: Version 4
AS Num 64496: Holdtime 90: BGP_ID 192.0.2.4: Opt Length 20 (ExtOpt F)
Opt Para: Type CAPABILITY: Length = 18: Data:
  Cap_Code GRACEFUL-RESTART: Length 2
  Bytes: 0x1 0x2c
  Cap_Code MP-BGP: Length 4
  Bytes: 0x0 0x1 0x0 0x1
  Cap_Code ROUTE-REFRESH: Length 0
  Cap_Code 4-OCTET-ASN: Length 4
```

```
Bytes: 0x0 0x0 0xfb 0xf0
```

Debug 11-2 shows the corresponding Open message from Tester T2. Again, the part of interest is the Graceful Restart capability field, which can be decoded as follows:

0x01	Restart Flag (most significant four bits only) not set indicating router has not restarted
0x01 0x2c	Restart Time (twelve bits consisting of second byte and least significant four bits of first byte). Four bits of first byte = 0x01 (decimal 256), second byte = 0x2c (decimal 44) indicating Restart Time of 300 seconds.
0x0 0x1	Address Family Indicator (2 octets) IP version 4
0x1	Subsequent Address Family Indicator (1 octet) Unicast Forwarding
0x0	Flags for AFI/SAFI indicating Forwarding state bit clear

Debug 11-2: Open Message received from tester T2

```
49 2025/09/24 18:11:46.383 BST minor: DEBUG #2001 Base BGP
BGP: OPEN
Peer 1: 192.168.32.2 - Received BGP OPEN: Version 4
  AS Num 64500: Holdtime 90: BGP_ID 192.0.0.117: Opt Length 24 (ExtOpt F)
  Opt Para: Type CAPABILITY: Length = 22: Data:
    Cap_Code MP-BGP: Length 4
    Bytes: 0x0 0x1 0x0 0x1
    Cap_Code ROUTE-REFRESH: Length 0
    Cap_Code GRACEFUL-RESTART: Length 6
    Bytes: 0x1 0x2c 0x0 0x1 0x1 0x0
    Cap_Code 4-OCTET-ASN: Length 4
    Bytes: 0x0 0x0 0xfb 0xf4
```

After an exchange of Keepalive messages the BGP session between R4 and T2 moves to the established state and T2 advertises 100 x IPv4 NLRI from 172.16.0.0/24 to 172.16.99.0/24 followed by the EOR marker shown in *Debug 11-3*. As previously described, for the IPv4 Address Family this is an Update message with no NLRI and empty withdrawn NLRI. Similarly, R4 sends an EOR marker.

Debug 11-3: EOR marker received from tester T2

```
55 2025/09/24 18:12:42.869 BST minor: DEBUG #2001 Base Peer 1: 192.168.32.2
Peer 1: 192.168.32.2: UPDATE
Peer 1: 192.168.32.2 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 0
  End-of-Rib marker (IPv4)
```

After exchanging EoR markers for the relevant Address Families, the status of the peer with regard to Graceful Restart can be displayed as shown in *Output 11-2*. Should the BGP peer undergo a restart that does not dictate the receipt of a Notification message, or an expiry of the Hold Time, the router will function as a Graceful Restart helper for the IPv4 Address Family. This includes preserving, and forwarding traffic to, any routes advertised by the peer (although they will be marked as stale) for the period of the restart.

Output 11-2: Graceful Restart status for external peer

```
*A:R4# show router bgp neighbor 192.168.32.2 graceful-restart
```

```

=====
BGP Neighbor 192.168.32.2 Graceful Restart
=====
Graceful Restart locally configured for peer: Enabled
GR Notification                               : Disabled
Peer's Graceful Restart feature               : Disabled
NLRI(s) that peer supports restart for       : ipv4
NLRI(s) that peer saved forwarding for        : None
NLRI(s) that restart is negotiated for       : None
NLRI(s) of received end-of-rib markers       : None
NLRI(s) of all end-of-rib markers sent       : ipv4
NLRI(s) peer supports NOTIFICATION GR for    : None
Restart time locally configured for peer     : 300 seconds
Restart time requested by the peer           : 300 seconds
Time until stale routes are deleted or
become long-lived stale                      : 360 seconds
Graceful restart status on the peer          : Not currently being helped
Long-Lived GR status on the peer             : Not currently being helped
Number of Restarts                           : 0
Last Restart at                              : Never
-----
LLGR Configuration                           : Disabled
Peer's LLGR feature                         : Disabled
NLRI(s) peer signaled LLGR for & stale time
& F-bit                                     : n/a
NLRI(s) LLGR negotiated for and stale time  : n/a
LLGR Restart time overridden for the peer    : n/a
NLRI(s) LLGR advertised & stale time & F-bit: n/a
=====

```

Notification support for Graceful Restart

The base Graceful Restart mechanism requires that normal BGP procedures are followed when a Notification message is sent or received, meaning that the BGP session is immediately closed. Notification support for Graceful Restart [82] defines an extension to Graceful Restart that permits the Graceful Restart procedures to be performed when the BGP speaker receives a Notification message, or the Hold Time expires. As previously described, support for Graceful Restart Notification is negotiated as a Graceful Restart capability by setting the 'N' bit of the Restart Flags field. When a BGP session is thereafter reset, both speakers operate as "Receiving Speakers" (or Graceful Restart helpers), and retain each other's routes.

When a BGP session enters the Graceful Restart process as a result of a critical error, there is a reasonable chance that the same error will occur when trying to re-establish the session during the Graceful Restart process. However, the process of re-establishing the BGP session and re-advertising the RIB-Out places some demand on CPU resource, so it cannot go unbounded. Therefore, if a critical error/Notification message has caused the router to enter the Graceful Restart process and another Update message with a critical error is received as the session is being re-established, the Graceful Restart process is immediately terminated without any further attempt to preserve forwarding state. To abort the process a Notification message is sent with the Cease Error Code and a Hard Reset Error subcode indicating that the BGP session must be terminated.

Output 11-3 shows the configuration for Notification triggered Graceful Restart using the **gr-notification** command within the **graceful-restart** context, which can be applied at group or neighbor level. In this example is applied at group level towards tester T2 in AS64500.

Output 11-3: Notification triggered Graceful Restart

```
router "Base" {
  bgp {
    group "EBGP-to-AS64500" {
      graceful-restart {
        gr-notification true
      }
    }
  }
}
```

With the above configuration in place, *Debug 11-4* shows the Open message generated by router R4 where the first four bits of the first byte (0x41) indicate that the 'N' bit is set, indicating support for Notification triggered Graceful Restart. It's also worth noting that when **gr-notification** is enabled, Address Families for which Graceful Restart is applicable are also advertised. There is no configuration that dictates the Address Families that are advertised. The system will simply advertise the applicable Address Families that are enabled for the peer from the list of supported Address Families:

0x41	Restart Flag (most significant four bits only) not set, but N-flag set indicating support for Notification triggered Graceful Restart
0x41 0x2c	Restart Time (twelve bits consisting of second byte and least significant four bits of first byte). Four bits of first byte = 0x01 (decimal 256), second byte = 0x2c (decimal 44) indicating Restart Time of 300 seconds.
0x0 0x1	AFI (two bytes) IP version 4
0x1	SAFI (one byte) Unicast Forwarding
0x0	Flags for AFI/SAFI indicating Forwarding state bit clear

Debug 11-4: Open message from R4 with Graceful Restart Notification support

```
296 2026/01/05 15:58:37.443 GMT minor: DEBUG #2001 Base BGP
BGP: OPEN
Peer 1: 192.168.32.2 - Send (Passive) BGP OPEN: Version 4
  AS Num 64496: Holdtime 90: BGP_ID 192.0.2.4: Opt Length 24 (ExtOpt F)
  Opt Para: Type CAPABILITY: Length = 22: Data:
    Cap_Code GRACEFUL-RESTART: Length 6
      Bytes: 0x41 0x2c 0x0 0x1 0x1 0x0
    Cap_Code MP-BGP: Length 4
      Bytes: 0x0 0x1 0x0 0x1
    Cap_Code ROUTE-REFRESH: Length 0
    Cap_Code 4-OCTET-ASN: Length 4
      Bytes: 0x0 0x0 0xfb 0xf0
```

This is followed in *Debug 11-5* with the Open message from tester T2 with the same bit set. The fact that the peer has actually signalled its support of Graceful Restart for Notification extension can be confirmed in *Output 11-4*.

Debug 11-5: Open message from T2 with Notification Graceful Restart enabled

```
297 2026/01/05 15:58:37.444 GMT minor: DEBUG #2001 Base BGP
```



```

BGP: OPEN
Peer 1: 192.168.32.2 - Received BGP OPEN: Version 4
  AS Num 64500: Holdtime 90: BGP_ID 192.0.0.124: Opt Length 16 (ExtOpt F)
  Opt Para: Type CAPABILITY: Length = 14: Data:
    Cap_Code MP-BGP: Length 4
    Bytes: 0x0 0x1 0x0 0x1
    Cap_Code GRACEFUL-RESTART: Length 6
    Bytes: 0x41 0x2c 0x0 0x1 0x1 0x0

```

Output 11-4: Graceful Restart Notification capability

```

A:admin@R4# show router bgp neighbor 192.168.32.2 graceful-restart

=====
BGP Neighbor 192.168.32.2 Graceful Restart
=====
Graceful Restart locally configured for peer: Enabled
GR Notification                               : Enabled
Peer's Graceful Restart feature               : Enabled
NLRI(s) that peer supports restart for       : ipv4
NLRI(s) that peer saved forwarding for       : None
NLRI(s) that restart is negotiated for       : ipv4
NLRI(s) of received end-of-rib markers       : None
NLRI(s) of all end-of-rib markers sent       : ipv4
NLRI(s) peer supports NOTIFICATION GR for    : None
Restart time locally configured for peer     : 300 seconds
Restart time requested by the peer           : 300 seconds
Time until stale routes are deleted or
become long-lived stale                      : 300 seconds
Graceful restart status on the peer          : Not currently being helped
Long-Lived GR status on the peer             : Not currently being helped
Number of Restarts                           : 16
Last Restart at                              : 01/05/2026 16:53:08
-----
LLGR Configuration                           : Disabled
Peer's LLGR feature                         : Disabled
NLRI(s) peer signaled LLGR for & stale time
& F-bit                                     : n/a
NLRI(s) LLGR negotiated for and stale time  : n/a
LLGR Restart time overridden for the peer    : n/a
NLRI(s) LLGR advertised & stale time & F-bit: n/a
=====

```

To verify the Graceful Restart process, traffic is forwarded from tester T1 towards the prefixes advertised by tester T2 in AS 64500, and this traffic transits through router R4 as illustrated in *Figure 11-1*. To simulate a restarting router BGP Keepalive messages are suspended at tester T2 causing router R4 to declare the BGP session down and enter Graceful Restart helper mode.

Debug 11-6: Router R4 entering Graceful Restart Helper mode

```

3 2026/01/05 16:25:07.970 GMT WARNING: BGP #2039 Base Peer 1: 192.168.32.2
"(ASN 64500) VR 1: Group EBGp-to-AS64500: Peer 192.168.32.2: moved from higher state
ESTABLISHED to lower state IDLE due to event HOLDDTIME"

307 2026/01/05 16:25:07.970 GMT minor: DEBUG #2001 Base BGP
BGP: RESTART
Peer VR 1: Group EBGp-to-AS64500: Peer 192.168.32.2: entering helper mode due to reason
hold_timer_expiry

```

The Graceful Restart status of the BGP neighbor can be verified in the field “Graceful restart status on the peer” using the command shown in *Output 11-5*. In this case,

the field shows “Rcvd restart request”, which could be considered somewhat misleading as no explicit restart request has been received by the router.

Output 11-5: Graceful Restart neighbor status

```
A:admin@R4# show router bgp neighbor 192.168.32.2 graceful-restart

=====
BGP Neighbor 192.168.32.2 Graceful Restart
=====
Graceful Restart locally configured for peer: Enabled
GR Notification                               : Enabled
Peer's Graceful Restart feature                : Enabled
NLRI(s) that peer supports restart for         : ipv4
NLRI(s) that peer saved forwarding for         : ipv4
NLRI(s) that restart is negotiated for         : None
NLRI(s) of received end-of-rib markers         : None
NLRI(s) of all end-of-rib markers sent         : None
NLRI(s) peer supports NOTIFICATION GR for     : None
Restart time locally configured for peer       : 300 seconds
Restart time requested by the peer             : 300 seconds
Time until stale routes are deleted or
become long-lived stale                       : 300 seconds
Graceful restart status on the peer            : Rcvd restart request
Long-Lived GR status on the peer              : Restart completed
Number of Restarts                            : 14
Last Restart at                               : 01/05/2026 16:39:05
-----
LLGR Configuration                           : Disabled
Peer's LLGR feature                          : Disabled
NLRI(s) peer signaled LLGR for & stale time
& F-bit                                       : n/a
NLRI(s) LLGR negotiated for and stale time    : n/a
LLGR Restart time overridden for the peer     : n/a
NLRI(s) LLGR advertised & stale time & F-bit: n/a
=====
```

It is important to note that while the router is functioning as a Graceful Restart helper the prefixes 172.16.0.0/24 to 172.16.99.0/24 learnt from tester T2 are still intact in the RIB and FIB but are internally marked as ‘stale’. This is denoted using the ‘x’ flag as shown in *Output 11-6*, which shows the prefixes as used, valid, best, and stale. As long as the routes remain intact in the RIB and FIB, it follows that router R4 is able to forward the traffic from tester T1 to tester T2.

Output 11-6: Routes marked as 'stale' during Graceful Restart

```
A:admin@R4# show router bgp routes 172.16.0.0/20 longer

=====
BGP Router ID:192.0.2.4      AS:64496      Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                l - leaked, x - stale, > - best, b - backup, p - purge,
                w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete

=====
BGP IPv4 Routes
=====
```

Flag	Network	LocalPref	MED
	Nextthop (Router)	Path-Id	IGP Cost
	As-Path		Label

```

-----
u*>xi 172.16.0.0/24          None      None
      192.168.32.2          None      0
      64500                  -
u*>xi 172.16.1.0/24          None      None
      192.168.32.2          None      0
      64500                  -
u*>xi 172.16.2.0/24          None      None
      192.168.32.2          None      0
      64500                  -
--- [ snip ] ---

```

The BGP session at tester T2 is then re-enabled and a restart simulated, after which a new BGP Open message is received at router R4 from tester T2 as shown in *Debug 11-7*. The Graceful-Restart capability code from tester T2 is decoded as follows:

0x81	Restart Flag (most significant four bits only) set indicating router has restarted
0x81 0x2c	Restart Time (twelve bits consisting of second byte and least significant four bits of first byte). Four bits of first byte = 0x01 (decimal 256), second byte = 0x2c (decimal 44) indicating Restart Time of 300 seconds.
0x0 0x1	AFI (two bytes) IP version 4
0x1	SAFI (one byte) Unicast Forwarding
0x80	Flags for AFI/SAFI indicating Forwarding state bit set

In summary, the restarting router is notifying the receiving router that it has undergone a restart, and that during that restart the forwarding plane was kept intact. As expected, the Open message from R4 is essentially the same as the original BGP session establishment.

Debug 11-7: Tester T2 Open message following restart

```

386 2026/01/05 16:42:38.655 GMT minor: DEBUG #2001 Base BGP
BGP: OPEN
Peer 1: 192.168.32.2 - Received BGP OPEN: Version 4
  AS Num 64500: Holdtime 90: BGP_ID 192.0.0.124: Opt Length 16 (ExtOpt F)
  Opt Para: Type CAPABILITY: Length = 14: Data:
    Cap_Code MP-BGP: Length 4
    Bytes: 0x0 0x1 0x0 0x1
    Cap_Code GRACEFUL-RESTART: Length 6
    Bytes: 0x81 0x2c 0x0 0x1 0x1 0x80

```

When Keepalive messages have been exchanged, the BGP session moves to the Established state and T2 sends a BGP Update message containing 100 x IPv4 NLRI from 172.16.0.0/24 to 172.16.99.0/24. As these routes have now been re-advertised (refreshed) by the restarting router, R4 removes the stale marker on these prefixes. However, router R4 cannot exit the Graceful Restart process until it has received the EOR marker from tester T2. When this has been received, R4 exits the Graceful Restart process.

Debug 11-8: Router R4 exists Graceful Restart helper mode

```

395 2026/01/05 16:42:40.043 GMT minor: DEBUG #2001 Base Peer 1: 192.168.32.2
Peer 1: 192.168.32.2: UPDATE
Peer 1: 192.168.32.2 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 0
  End-of-Rib marker (IPv4)

```

```

396 2026/01/05 16:42:40.043 GMT minor: DEBUG #2001 Base BGP
BGP: RESTART
Peer 1: 192.168.32.2: Received EOR marker for AFI/SAFI ipv4

399 2026/01/05 16:42:42.040 GMT minor: DEBUG #2001 Base BGP
BGP: RESTART
Peer 1: 192.168.32.2: exit helper mode due to reason end-of-rib

400 2026/01/05 16:42:42.040 GMT minor: DEBUG #2001 Base BGP
BGP: RESTART
BGP flushing stale routes for peer Peer 1: 192.168.32.2 AF All Address Families

```

Finally, verification is obtained that R4 has exited Graceful Restart helper mode.

Output 11-7: Router R4 Graceful Restart status

```

A:admin@R4# show router bgp neighbor 192.168.32.2 graceful-restart | match "Graceful
restart status"
Graceful restart status on the peer          : Restart completed

```

SR-OS supports Graceful Restart for the IPv4/IPv6, VPN-IPv4/IPv6, Label-IPv4/IPv6, Layer-2 VPN (BGP VPLS), Flow-IPv4/IPv6, and Route-Target Constrain (RTC) Address Families.

Error Handling

BGP has evolved from being simply an Exterior Gateway Protocol (EGP) to a protocol that is used by operators not just for inter-AS route exchange, but also for delivery of numerous key services such as IP-VPNs, VPLS, VPWS (EVPN or otherwise), as well as other value-added services. In addition, BGP can be used as a label distribution protocol infrastructure for building infrastructure LSPs, and the integrity of these LSPs are paramount to delivery of the afore-mentioned services. The services that are delivered using BGP are typically high-profile and high-revenue, but the consumers of these services have high expectations with regard to availability and reconvergence upon failure. Failures in BGP are commonly high-impact and can end up being reported in the media with the inevitable negative connotations.

The original BGP specification [1] mandated that if any errors are detected in an Update message the receiving BGP speaker should send a Notification message to the peer, which in turn meant that the session was torn down and prompted the removal of any routes learnt from that peer. This procedure is of course disruptive and was based on early assumptions that if a BGP session was taken down an alternate path through another AS would be available. However, with the increase in services delivered by BGP (which are primarily intra-AS), its survivability has become critical, and therefore some changes to the original specification were required with regard to error handling. These updates are specified in [50], which defines the requirements for increased protocol robustness and outlines the proposals for error handling whilst capturing the work of a number of previous proposals on increased error handling. Within this specification, four approaches are used for error

handling of Update messages. They are listed as follows, from the strongest action to the weakest action in order:

- Session Reset: This is the approach used in the base BGP specification, where a Notification is sent and the session is terminated.
- AFI/SAFI disable: Allows a BGP speaker that detects an error in an Update message for a given AFI/SAFI to subsequently ignore all the subsequent routes with the same AFI/SAFI received over that session.
- Treat-as-withdraw: The Update containing the errored path attribute must be treated as though all routes in the message had been withdrawn (just as if they had been listed in the WITHDRAWN ROUTES field or in the MP_UNREACH_NLRI field).
- Attribute discard: In this approach the malformed attribute is discarded, and the Update message continues to be processed. It can only be used where an attribute has an effect on route selection or installation.

Very broadly, errors can be placed into two categories – critical and non-critical.

Where the NLRI cannot be extracted from the Update message because of, for example, message length errors, the error is considered critical, and a Notification message must be sent to the advertising router. However, in an attempt to maintain forwarding during the session reset, Notification triggered Graceful Restart can be used as previously described. Non-Critical Errors are errors where the NLRI can be extracted from the Update message. Examples include invalid length errors in path attributes, missing mandatory attributes, attribute flag errors, and Update messages containing attributes that do not relate to the advertised NLRI. These errors use either the treat-as-withdraw approach or the attribute discard approach:

- Treat-as-withdraw is used for errors involving any of the ORIGIN, AS_PATH, NEXT_HOP, MED, or LOCAL_PREF attributes, or for any attribute that has an attribute length error. When using treat-as-withdraw the entire NLRI field or MP_REACH_NLRI field needs to be successfully passed.
- The attribute discard approach is used for errors involving the ATOMIC_AGGREGATE or AGGREGATOR attributes.

In order to increase the likelihood of being able to extract the NLRI, all Address Families should use MP_REACH and MP_UNREACH NLRI (including IPv4 unicast), and this attribute should always be the first in an Update message.

The error handling enhancements for non-critical errors are enabled in SR-OS at the BGP, group, and neighbour level using the **error-handling update-fault-tolerance** command, both within the base instance and VPRN instances of BGP. In newer SR-OS releases, the error handling behaviour specified in [50] is applicable to all BGP peers regardless of the configuration of the **update-fault-tolerance** command. If for some reason the error handling behaviour is not required or needed, the **error-handling legacy-mode** can be enabled, which effectively disables the enhanced error handling behaviour.

Output 11-8: Enhanced Error Handling configuration

```

router "Base" {
  bgp {
    error-handling {
      update-fault-tolerance true
    }
  }
}

```

To qualify the handling of non-critical errors, tester T2 sends an Update message to router R4 with a missing mandatory attribute; in this case the AS_PATH attribute.

Debug 11-9: Update message with missing AS_PATH attribute received at R4

```

242 2025/09/25 14:44:32.328 BST MINOR: DEBUG #2001 Base Peer 1: 192.168.32.2
"Peer 1: 192.168.32.2: UPDATE
Peer 1: 192.168.32.2 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 11
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 3 Len: 4 Nexthop: 192.168.32.2
  NLRI: Length = 4
    172.16.99.0/24
"

```

Without the updated error handling enhancements, such an Update message would have triggered a Notification message to be sent by R4 and the BGP session would be torn down, but with **update-fault-tolerance** enabled, the integrity of the BGP session is maintained and the Update is simply treated as a withdraw and not held in RIB-In. Note that if an Update is treated as a withdraw SR-OS generates an error message containing the affected peer and prefix as shown in *Output 11-9*.

Output 11-9: Log message for error in BGP Update message

```

792 2025/09/26 09:28:06.446 BST WARNING: BGP #2029 Base Peer 1: 192.168.32.2
"1: BGP Peer: 192.168.32.2, Route: 172.16.99.0/24 withdrawn because of error in BGP update
message."

```



Note: The following log event messages pertain to BGP Update errors:

tBgp4PathAttrDiscarded (TIMETRA-BGP-MIB.tBgpNotifications.24)

tBgp4PathAttrInvalid (TIMETRA-BGP-MIB.tBgpNotifications.16)

tBgp4RouteInvalid (TIMETRA-BGP-MIB.tBgpNotifications.15)

tBgp4UpdateInvalid (TIMETRA-BGP-MIB.tBgpNotifications.18)

tBgp4WithdrawnRtFromUpdateError (TIMETRA-BGP-MIB.tBgpNotifications.17)

In addition to the above error log messages, a counter is maintained on a per-neighbour basis to record received Update errors.

Output 11-10: Per-Neighbour Update error counter

```

A:admin@R4# show router bgp neighbor 192.168.32.2 | match "Update Errors"
Damp Peer Oscillatio*: Disabled          Update Errors          : 1

```

In addition to the error handling enhancements for critical and non-critical errors, SR-OS supports the capability to hold a BGP session in an idle state for exponentially increasing amounts of time to reduce the impact of continually trying to re-establish a BGP session. This exponential back-off logic is activated whenever a BGP session transitions from a non-idle state to an idle state and is enabled using the **damp-peer-oscillations** context at BGP, group, and neighbour levels (both within the base instance and within VPRN instances of BGP). The **damp-peer-oscillations** context has optional and configurable parameters to control the amount of time the session is held in idle state, known as **idle-hold-time**, after a transition from a non-idle state. The **initial-wait** defines the idle-hold-time (in seconds) after the first transition to idle state, whilst subsequent idle-hold-times are derived by increasing intervals of the **second-wait** value until the **max-wait** is reached. In addition to the parameters used to control the **idle-hold-time**, the **error-interval** defines the amount of time (in minutes) that must pass without any transitions before all back-off logic timers are reset.

Output 11-11: Damp-peer-oscillations configuration

```
router "Base" {  
  bgp {  
    damp-peer-oscillations {  
      error-interval 32  
      idle-hold-time {  
        initial-wait 0  
        second-wait 120  
        max-wait 1600  
      }  
    }  
  }  
}
```

The **damp-peer-oscillations** feature behaves completely independently of any error-handling functions described earlier in this section. However, if error-handling is enabled, it may be beneficial to also enable **damp-peer-oscillations** to control, for example, a BGP session that is trying to re-establish with a recurring critical error.

12 Security

Whilst the use of BGP has increased for delivering services within an Autonomous System it remains the de facto protocol for inter-AS route exchange, and as such it means that BGP must run on routers that are at the boundary of an Autonomous System or administrative domain. The point where the Autonomous System connects externally is frequently considered the security perimeter. Regardless of whether that external connection is a customer, a peering partner, a content provider, or something else, being able to secure that perimeter against any potential threats or attacks against internal network infrastructure or customers of that Autonomous System is of paramount importance. There are a number of mechanisms that are currently used today to implement security measures at the Autonomous System boundary, some simple and some perhaps a little more complex. There are also increasing demands to increase security and implement preventative security measures within the Autonomous System boundary. This chapter looks at how BGP can be used both proactively and reactively in helping the process of securing the perimeter and within the internal boundary of the Autonomous System.

FlowSpec



BGP FlowSpec is not supported on the 7250 IXR Gen 1 platform.

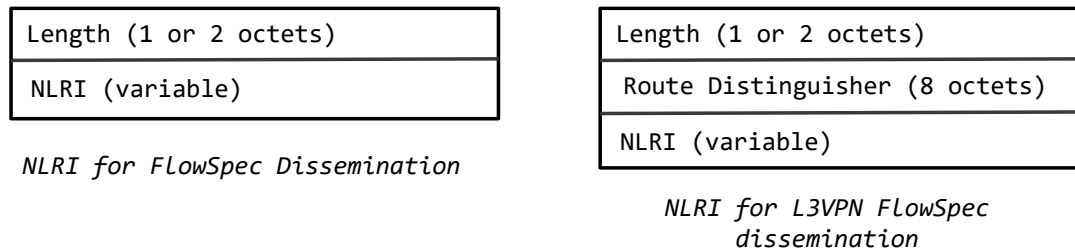
BGP Flow Specification (FlowSpec) [51] allows for encoding of flow specification information into Multi-Protocol BGP NLRI. A flow specification is an n -tuple consisting of several matching criteria such as source prefix, destination prefix, protocol, or ports that can be applied to IP traffic. Coupled with the flow specification information NLRI, extended community attributes provide the capability to define traffic filtering rules for the specified flow specification. The intention is to allow for automated creation of IP Filters to prevent intra-AS and inter-AS DDoS attacks, and to allow for redirection of traffic to other routing contexts (a 'dirty' VRF) for the purpose of off-ramping to scrubbing devices.

Flow specifications are intended to be more specific routing entries to a unicast aggregate prefix that exists in the routing table. An aggregate unicast prefix may be received by peer m and installed in the RIB/FIB, but a flow specification rule may then be received by a different peer n that defines more specific flow rules resulting in a different forwarding behaviour for this subset of traffic.

FlowSpec uses AFI 1 (IPv4) or AFI 2 (IPv6) with SAFI 133 (Dissemination of flow specification rules) or SAFI 134 (L3VPN dissemination of flow specification rules) and all of these Address Families are supported in SR-OS. Unlike other Address Families, there is no strict requirement for the Next-Hop attribute to be present in Update messages. If the application sourcing the Update does not require Next-Hop information (that is, it is not in the data path), the Length

of Next-Hop Address field in the MP_REACH_NLRI is set to zero and should be ignored on receipt.

Figure 12-1: FlowSpec NLRI



The FlowSpec NLRI consists of a list of optional components, which in general are encoded as <type, [operator, value]+>. The 1-byte type field specifies the type of component and primarily includes match criteria fields from the Layer 3/Layer 4 header but may also include packet length. It encodes information such as source/destination prefix, IP protocol, ICMP type/code, source/destination/port, or TCP fragment. The 1-byte operator field contains one of two types of operators, a numeric operator or a bit mask operator. The numeric operator is used to derive component values such as less-than, logical AND, greater-than, and equal-to. The bitmask operator is used to derive values such as logical NOT and match. With the exception of the TCP flags and Fragment which use a bitmask operator, the remaining type fields use a numeric operator.

Table 12-1 lists the possible flow specification ‘match’ criteria for IPv4 and VPN-IPv4, and a given packet matches the flow specification only when it matches all of the components present in the flow specification. These sub-components must follow strict type ordering if present. Because it is possible to encode multiple sub-components in the NLRI, the potential exists that more than one rule may match a particular flow. In this case, if the component types are different, the rule with the lowest numeric type value has higher precedence. If the component types are the same, a type-specific comparison is performed.

Table 12-1: FlowSpec subcomponents for IPv4 and VPN-IPv4 NLRI

Type	Component	SR-OS support
1	Destination IPv4 Prefix	Yes
2	Source IPv4 prefix	Yes
3	IP Protocol	Partial. No support for multiple values other than TCP or UDP
4	Port	Yes – single port or port-range
5	Destination Port	Yes – single port or port-range
6	Source port	Yes – single port or port-range
7	ICMP type	Partial – only a single value supported
8	ICMP code	Partial – only a single value supported
9	TCP flags	Yes, but limitations vary based on FP generation
10	Packet Length	Yes

11	DSCP	Yes
12	Fragment	Partial – no support for matching Last Fragment

Similarly, *Table 12-2* lists the possible flow specification ‘match’ criteria for IPv6 and VPN-IPv6. Like it’s IPv4/VPN-IPv4 counterparts, a given packet matches all of the components only when it matches all of the components present in the flow specification..

Table 12-2: FlowSpec subcomponents for IPv6 and VPN-IPv6

Type	Component	SR-OS support
1	Destination IPv6 Prefix	Partial – no support for prefix offset
2	Source IPv6 prefix	Partial – no support for prefix offset
3	Next Header	Partial – only a single value supported
4	Port	Yes – single port or port-range
5	Destination Port	Yes – single port or port-range
6	Source port	Yes – single port or port-range
7	ICMP type	Partial – only a single value supported
8	ICMP code	Partial – only a single value supported
9	TCP flags	Yes, but limitations vary based on FP generation
10	Packet Length	Yes
11	Traffic Class	Partial – only a single value is supported
12	Fragment	Partial – no support for matching Last Fragment
13	Flow Label	Partial – only a single value is supported

With the traffic identified using the flow specification NLRI, the next step is to execute an action upon that traffic. To achieve this ‘filtering’ action a number of extended community attributes are defined in the form 0xttss, where tt represents the type and ss indicates the sub-type of the extended community. *Table 12-3* lists the extended community filtering actions together with an indication of SR-OS support. The Traffic Rate extended community contains a two byte AS number followed by four bytes to specify the rate in bytes-per-second (in IEEE floating point format). The Traffic Action extended community consists of six bytes, of which only the least two significant bits of the last byte are currently defined. The ‘Terminal Action’ or ‘T’ bit, when set to 1 indicates that subsequent filtering rules should be applied and essentially represents a ‘next-entry’ action. When this bit is set to zero the evaluation of the traffic filter stops when this rule is applied. The ‘T’ bit is not currently supported in SR-OS. The ‘Sample’ bit or ‘S’ bit when set to 1, enables traffic sampling and logging for this flow specification. The Redirect-to-VRF option is intended to implement policy-based forwarding of identified traffic into a VRF to mitigate against a DDoS attack, with a Route-Target value being used to define the target redirect VRF. Similarly, there are a number of other redirect-to-IP and redirect-to-LSP actions that serve the same purpose for networks that do not have

an L3VPN infrastructure. Finally, the Mark Traffic Class extended community allows for remarking of DSCP/Traffic Class values at ingress.

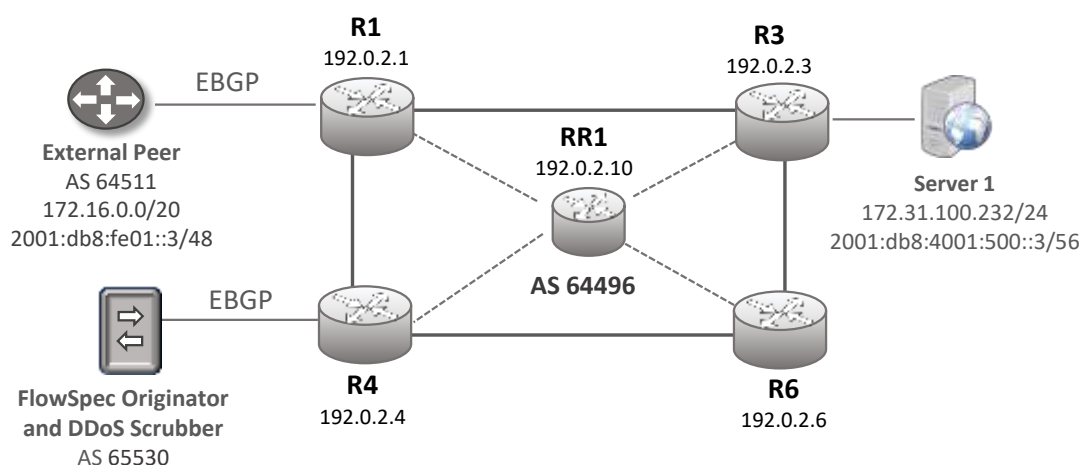
Table 12-3: FlowSpec filtering actions

Type	Extended Community Type	SR-OS support
0x8006	Traffic Rate	Yes
0x8007 S-bit	Traffic Action (Sample/Log)	Yes
0x8007 T-bit	Traffic Action (Next Entry)	No
0x8008	Redirect to VRF	Yes
0x8009	Mark Traffic Class	Yes
0x010c	Redirect to IPv4	Yes (IPv4 traffic only)
0x000c	Redirect to IPv6	Yes (IPv6 traffic only)
0x0900	Redirect to LSP	Partial

To demonstrate the use of FlowSpec the network topology illustrated in *Figure 12-2* is used. Routers R1, R3, R4, and R6 form part of AS 64496 and are IBGP peered with RR1 for IPv4/IPv6, Flow-IPv4/IPv6, and Flow-VPN-IPv4/IPv6 Address Families. Router R1 is externally peered with AS 64511 and learns IPv4 prefix 172.16.0.0/20 and IPv6 prefix 2001:db8:fe01::/48, both of which are advertised internally into AS 64496. Router R3 has a directly connected network with IPv4 subnet 172.31.100.0/24 and IPv6 subnet 2001:db8:4001:500::/56 hosting server 1. Both subnets are also advertised into IBGP which are in turn advertised externally to AS 64511. All IPv4 and IPv6 routes learnt through BGP within AS 64496 use BGP shortcuts for Next-Hop resolution. In summary, there is IPv4 and IPv6 connectivity between AS 64511 and server 1.

A FlowSpec route-server and simulated DDoS scrubber is connected to router R4 using EBGP and is also configured to support the IPv4, IPv6 Flow-IPv4/IPv6, and Flow-VPN-IPv4/IPv6 Address Families.

Figure 12-2: FlowSpec Test Topology



The relevant FlowSpec Address Families must be explicitly configured between the relevant peers to allow correct propagation of flow specification information. In the example of *Output 12-1*, FlowSpec IPv4/IPv6 and FlowSpec VPN-IPv4/IPv6 are all enabled. It's worth noting in the configuration example that the peering session uses IPv4, but it will accept IPv6 (FlowSpec IPv6 and FlowSpec VPN-IPv6) Address Families. Typically, the advertisement of IPv6 routes require an IPv6 Next-Hop, but in the case of FlowSpec routes the Next-Hop in the NLRI is set to zero and is not used, so a single IPv4 session for both IPv4 and IPv6 routes is acceptable. As with other Address Families, if this family is being added to an existing peer (or group of peers) it must be negotiated as a capability in an Open message and therefore triggers session capability renegotiation.

Output 12-1: Flowspec configuration at R4

```
router "Base" {
  bgp {
    group "FlowSpec-Originator" {
      family {
        flow-ipv4 true
        flow-ipv6 true
        flow-vpn-ipv4 true
        flow-vpn-ipv6 true
      }
    }
    group "IBGP-Core" {
      family {
        flow-ipv4 true
        flow-ipv6 true
        flow-vpn-ipv4 true
        flow-vpn-ipv6 true
      }
    }
    neighbor "192.0.2.10" {
      group "IBGP-Core"
      authentication-key "C3i5c6IP3VMhQFGff3RCM8pKOP5nZxU= hash2"
    }
    neighbor "192.168.254.18" {
      group "FlowSpec-Originator"
    }
  }
}
```

Attack Mitigation with Traffic Discard

As router R1 peers externally with AS 64511, it is the point where the FlowSpec filter will be applied at ingress, and *Output 12-2* shows the necessary configuration for installation of a FlowSpec filter for Flow-IPv4 Address Family. An IPv4 filter is created that allows for filter entries to be dynamically instantiated using the **embed** context. As the filter is an ingress filter towards an external peer, it would typically contain numerous static filter entries, hence the **flowspec offset** command indicates to the system that dynamically created FlowSpec filter entries should start after the indicated value. This allows both static and dynamic filter entries to co-exist. The filter is then applied as an ingress filter on the relevant SAP.

Output 12-2: FlowSpec Filter configuration at R1

```

filter {
  ip-filter "FlowSpec-Filter" {
    default-action accept
    filter-id 1
    embed {
      flowspec offset 100 {
        router-instance "Base"
      }
    }
  }
}
service {
  ies "EBGP-to-AS64511" {
    interface "EBGP-to-AS64511" {
      sap 1/1/c3/1:5 {
        ingress {
          filter {
            ip "FlowSpec-Filter"
          }
        }
      }
    }
  }
}

```

The FlowSpec originator then originates a Flow-IPv4 route which R4 advertises into AS 64496 and RR1 reflects to router R1. The Flow-IPv4 route uses the NLRI to identify traffic with a destination address of server 1 connected to router R3 at 172.31.100.232/32, using a TCP port range of greater than 4190 but less than 4199. The identified traffic is then subject to the filtering action extended community, which in this case is a rate-limit of 0Kb/s equivalent to a drop action. It's also worth noting the absence of a Next-Hop attribute in the Flow-IPv4 route.

Debug 12-1: FlowSpec IPv4 route received at R1

```

1020 2025/09/29 10:10:59.907 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 71
  Flag: 0x90 Type: 14 Len: 22 Multiprotocol Reachable NLRI:
    Address Family FLOW_IPV4
    NLRI len: 16
  dest_pref 172.31.100.232/32
  ip_proto  [ == 6 ]
  dest_port [ >4190 ] and [ <4199 ]
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 6 AS Path:
    Type: 2 Len: 1 < 65530 >
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
  Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.4
  Flag: 0x80 Type: 10 Len: 4 Cluster ID:
    192.0.2.10
  Flag: 0xc0 Type: 16 Len: 8 Extended Community:
    rate-limit: 0 kbps

```

The received FlowSpec route can also be verified in the RIB-In at R1, which provides a concise output of the flow attributes and traffic filtering function.

Output 12-3: Flow-IPv4 route in RIB-In

```

A:admin@R1# show router bgp routes flow-ipv4
=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                  l - leaked, x - stale, > - best, b - backup, p - purge,
                  w - unused-weight-only
Origin codes   : i - IGP, e - EGP, ? - incomplete
=====
BGP FLOW IPV4 Routes
=====
Flag  Network          Nexthop          LocalPref      MED
      As-Path
-----
u*>i  --              0.0.0.0         100            None
      65530

      Community Action:  rate-limit: 0 kbps
      Flowspec Components:
      Dest Pref  : 172.31.100.232/32
      Ip Proto   : [ == 6 ]
      Dest Port  : [ >4190 ] and [ <4199 ]
-----
Routes : 1
=====

```

Valid FlowSpec routes are dynamically instantiated in an IP filter known as “fSpec-0” which is created for the base routing instance. The entry within the filter is 256 and indicates that it has been inserted by BGP FlowSpec. If a static ingress IP filter is also in use on an interface, the FlowSpec generated filter entry/entries will be processed after all entries of the static provisioned filter entries, but prior to the default-action (whether configured or using the default default-action). The “fSpec-0” filter is spawned as appropriate to other IP filters that are configured to insert BGP FlowSpec entries, and as such the defined filter will not show traffic matches.

Output 12-4: IP filter "fSpec-0"

```

A:admin@R1# show filter ip "fSpec-0"
=====
IP Filter
=====
Filter Id       : fSpec-0
Scope          : Embedded
Type           : Normal
Shared Policer  : Off
Entries        : 1 (insert By Bgp)
Description     : IPv4 BGP FlowSpec filter for the Base router
-----
Filter Match Criteria : IP
-----
Entry          : 256
Origin         : Inserted by BGP FlowSpec
Description    : (Not Specified)
Log Id        : n/a
Src. IP       : 0.0.0.0/0
Src. Port     : n/a
Dest. IP      : 172.31.100.232/32

```

```

Dest. Port      : 4191..4198
Protocol        : 6
Dscp            : Undefined
ICMP Type       : Undefined      ICMP Code        : Undefined
Fragment        : Off            Src Route Opt   : Off
Sampling        : Off            Int. Sampling   : On
IP-Option       : 0/0            Multiple Option: Off
Tcp-flag        : (Not Specified)
Option-pres     : Off
Egress PBR      : Disabled
Primary Action   : Drop
Ing. Matches     : 0 pkts
Egr. Matches     : 0 pkts

```

The configured IP filter “FlowSpec-Filter” shown in *Output 12-2* and applied at ingress on the interface to AS 64511 shows that ingress traffic is being matched (Ing. Matches : 31021 pkts) and that it is being dropped (Primary Action : Drop). Recall that the IP filter has a **flowspec offset** of 100, and that the “fSpec-0” filter has an entry number of 256. The entry spawned in FlowSpec-Filter is therefore entry 356 and indicates that it has been inserted by fSpec-0 entry 256. This entry shows packet drops as a DDoS mitigation. As the **default-action** of the IPv6 filter is **accept**, all other traffic will be forwarded.

Output 12-5: IP Filter "FlowSpec-Filter"

```

A:admin@R1# show filter ip "FlowSpec-Filter"

=====
IP Filter
=====
Filter Id      : 1                      Applied       : Yes
Scope          : Template               Def. Action    : Forward
Type           : Normal
Shared Policer : Off
System filter  : Unchained
Radius Ins Pt  : n/a
CrCtl. Ins Pt  : n/a
RadSh. Ins Pt  : n/a
PccRl. Ins Pt  : n/a
Entries        : 0/0/0/1 (Fixed/Radius/Cc/Embedded)
Description     : (Not Specified)
Filter Name     : FlowSpec-Filter
-----
Filter Match Criteria : IP
-----
Entry          : 356
Origin         : Inserted by embedded filter fSpec-0 entry 256
Description    : (Not Specified)
Log Id         : n/a
Src. IP        : 0.0.0.0/0
Src. Port      : n/a
Dest. IP       : 172.31.100.232/32
Dest. Port     : 4191..4198
Protocol       : 6
Dscp           : Undefined
ICMP Type      : Undefined      ICMP Code      : Undefined
Fragment       : Off            Src Route Opt   : Off
Sampling       : Off            Int. Sampling   : On
IP-Option      : 0/0            Multiple Option: Off
Tcp-flag       : (Not Specified)
Option-pres    : Off
Egress PBR     : Disabled

```

```

Primary Action      : Drop
Ing. Matches       : 31021 pkts (15882752 bytes)
Egr. Matches       : 0 pkts

```

=====

Attack Mitigation with Redirect to VRF

Whilst discarding traffic at ingress is effective for some kinds of DDoS attacks, some types of attack mean that traffic can be treated by a scrubber and then forwarded to the destination address. This involves a redirect of traffic at ingress, and frequently the traffic is redirected to a VRF so that forwarding of the traffic can be isolated. Using this approach target traffic is off-ramped towards the scrubbing centre at the point of ingress to the network using a VRF that is often referred to as an 'off-ramp VRF' or a 'dirty VRF' and is only returned back to the network after it has been scrubbed. In this case, the 'off-ramp VRF' must exist on the router at which the filter is applied, and it must have Route-Target import values that match the Route Target values signalled in the FlowSpec route. In addition, there must be some other minimal routing information within the redirect VRF (a mitigation-route) to attract the off-ramp traffic towards the scrubbing centre.

For this example, router R1 has an 'off-ramp VRF' (VPRN "one") configured with Route-Target 64496:1. The 'off-ramp' VPRN contains a default-route with a Next-Hop of R4, where the traffic is forwarded to the DDoS scrubber for treatment. Once the scrubber has treated the traffic it will be returned to router R4 so that it can be onward forwarded to its destination. If a VRF is also used for this, it is frequently called an 'on-ramp' VRF or a 'clean VRF'. For this example, the DDoS scrubber has detected an anomaly in the traffic profile between multiple source addresses and the internal host 172.31.100.132 for TCP ports greater than 4109 but less than 4199 and determines that an attack is potentially underway. The scrubber therefore originates the redirect-to-VRF FlowSpec IPv4 route shown in *Debug 12-2* as received at router R1. The NLRI is the same as used for the traffic drop scenario previously described, but the extended community is now a Redirect-to-VRF with a Route-Target of 64496:1.

Debug 12-2: FlowSpec IPv4 route with Redirect-to-VRF Extended Community

```

1316 2025/09/29 13:20:58.700 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 71
    Flag: 0x90 Type: 14 Len: 22 Multiprotocol Reachable NLRI:
        Address Family FLOW_IPV4
        NLRI len: 16
    dest_pref 172.31.100.232/32
    ip_proto  [ == 6 ]
    dest_port  [ >4190 ] and [ <4199 ]
    Flag: 0x40 Type: 1 Len: 1 Origin: 0
    Flag: 0x40 Type: 2 Len: 6 AS Path:
        Type: 2 Len: 1 < 65530 >
    Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
    Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.0.2.4
    Flag: 0x80 Type: 10 Len: 4 Cluster ID:

```



```
192.0.2.10
Flag: 0xc0 Type: 16 Len: 8 Extended Community:
redirect-to-vrf:64496:1
```

The FlowSpec IPv4 route is installed in the “fSpec-0” filter and then cloned into the IP filter FlowSpec-Filter applied to the ingress of the interface towards AS 64511. *Output 12-6* shows the operational state of the filter with ingress packet matches. Notice the Primary Action of Forward (VRF) and an indication of the router-instance in which packets are forwarded – in this case Router: 1 (the “off-ramp” VRF).

Output 12-6: FlowSpec IP filter with Redirect-to-VRF entry

```
A:admin@R1# show filter ip "FlowSpec-Filter"

=====
IP Filter
=====
Filter Id           : 1                      Applied           : Yes
--- [snip] ---
Filter Name         : FlowSpec-Filter
-----
Filter Match Criteria : IP
-----
Entry               : 356
Origin              : Inserted by embedded filter fSpec-0 entry 256
Description         : (Not Specified)
Log Id              : n/a
Src. IP             : 0.0.0.0/0
Src. Port           : n/a
Dest. IP            : 172.31.100.232/32
Dest. Port          : 4191..4198
Protocol            : 6
Dscp                : Undefined
ICMP Type           : Undefined              ICMP Code       : Undefined
Fragment           : Off                    Src Route Opt    : Off
Sampling            : Off                    Int. Sampling    : On
IP-Option           : 0/0                    Multiple Option  : Off
Tcp-flag            : (Not Specified)
Option-pres         : Off
Egress PBR          : Disabled
Primary Action      : Forward (VRF)
  Router            : 1
  Extended Action    : None
PBR Down Action     : Drop (entry-default)
Ing. Matches        : 11265 pkts (5767680 bytes)
Egr. Matches        : 0 pkts

=====
```

To confirm that the redirect has been successful, *Output 12-7* shows a rate monitor on the ‘off-ramp’ VRF egress interface towards the DDoS scrubber at router R4. Here it can be seen that the forwarding rate is ~89pps towards the scrubber. Forwarding of return traffic once it has been treated is not covered here but is simply another interface into a routing context with sufficient routing information to forward the traffic towards its ultimate destination.

Output 12-7: Forwarding rate towards DDoS scrubber

```
A:admin@R4# monitor service id 1 sap "1/1/c3/1:200" rate

=====
Monitor statistics for Service 1 SAP 1/1/c3/1:200
```

```
=====
-----
At time t = 0 sec (Base Statistics)
-----
```

```
-----
Sap Aggregate Stats
-----
```

	Packets	Octets
Ingress		
Aggregate Offered	: 0	0
Aggregate Forwarded	: 0	0
Aggregate Dropped	: 0	0

Egress		
Aggregate Forwarded	: 214033	24764047
Aggregate Dropped	: 0	0

```
-----
Sap Statistics
-----
```

```
Last Cleared Time      : N/A
```

```
--- [snip] ---
```

```
Queueing Stats(Egress QoS Policy 1)
```

Dro. In/InplusProf	: 0	0	0.00
Dro. Out/ExcProf	: 0	0	0.00
For. In/InplusProf	: 0	7	~0.00
For. Out/ExcProf	: 89	45528	~0.00

Attack Mitigation using L3VPN

The final example of FlowSpec illustrates the use of VPN FlowSpec, and since the previous two examples have used IPv4, this example uses IPv6. As a consequence, the following changes are made to the test topology of *Figure 12-2*:

- The interface to the external peer in AS 64511 at router R1 and the interface to server 1 at router R3 are placed into an IP-VRF instance “one”. That is, an L3VPN is extended between router R1 and R3 to exchange IPv6 reachability between subnet 2001:db8:fe01::/48 in AS 64511 and subnet 2001:db8:4001:300::/56 connected to router R3 in AS 64496.
- As SR-OS only accepts Flow-VPN-IPv4 and FlowVPN-IPv6 routes in IBGP, the FlowSpec route originator connected to router R4 is modified to use AS 64496. In addition, to allow routes to be re-advertised internally into AS 64496, router R4 is configured as a Route-Reflector with Cluster-ID 192.0.2.4.

Output 12-8 shows the required configuration at router R1 to support FlowSpec VPN-IPv6. An IPv6 filter ‘FlowSpec-VPNIPv6-Filter’ is configured that allows for filter entries to be dynamically instantiated using the **embed** context. To allow for static entries to be created before dynamic entries, the **flowspec-offset** is set to a value of 200. The **router-instance** is set to “one” for the IP-VRF instance. As this is an IP-VRF, the filter could embed dynamic entries using FlowSpec-IPv6 routes received over a BGP peering session within the VRF, or FlowSpec-VPN-IPv6 routes received in the base routing instance. The IPv6 filter is then applied as an ingress filter on the relevant SAP within the IP-VRF.

Output 12-8: FlowSpec VPN-IPv6 configuration at R1

```

filter {
  ipv6-filter "FlowSpec-VPNIPv6-Filter" {
    default-action accept
    filter-id 6
    embed {
      flowspec offset 200 {
        router-instance "one"
      }
    }
  }
}
service {
  vprn "one" {
    interface "EBGP-to-AS64511" {
      sap 1/1/c3/1:200 {
        ingress {
          filter {
            ipv6 "FlowSpec-VPNIPv6-Filter"
          }
        }
      }
    }
  }
}

```

Debug 12-3 shows the FlowSpec-VPN-IPv6 route received at router R1 using AFI 2 (IPv6) SAFI 134 (L3VPN). The NLRI components identify the traffic as destination address 2001:db8:4001:500::3/128 (server 1) and source address of 2001:db8:fe01::3/128 using a UDP port range of greater than 4109 but less than 4199. Note in this L3VPN example that the NLRI carries a Route-Distinguisher field whose semantics are exactly the same as used with IP-VPN and EVPN. There are two Extended Communities. The first is a Route-Target target:64496:1 which identifies the IP-VRF to which this filter entry will be applied, and the second is the filtering action, which in this case is a drop (or rate-limit: 0 kbps).

Debug 12-3: Flow-VPN-IPv6 route received at R1

```

1640 2025/09/29 17:17:45.691 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
  Withdrawn Length = 0
  Total Path Attr Length = 117
  Flag: 0x90 Type: 14 Len: 62 Multiprotocol Reachable NLRI:
    Address Family FLOW_VPN_IPV6
    NLRI len: 56
    RD 64496:1
  dest_pref 2001:db8:4001:500::3/128 offset 0
  src_pref 2001:db8:fe01::3/128 offset 0
  ip_proto [ == 17 ]
  dest_port [ >4190 ] and [ <4199 ]
  Flag: 0x40 Type: 1 Len: 1 Origin: 0
  Flag: 0x40 Type: 2 Len: 0 AS Path:
  Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
  Flag: 0x80 Type: 9 Len: 4 Originator ID: 192.168.254.253
  Flag: 0x80 Type: 10 Len: 8 Cluster ID:
    192.0.2.10 192.0.2.4
  Flag: 0xc0 Type: 16 Len: 16 Extended Community:
    target:64496:1
    rate-limit: 0 kbps

```

Once again the received FlowSpec route can be viewed in the RIB-In at R1, providing a concise output of the flow attributes and traffic filtering function.

Output 12-9: Flow-VPN-IPv6 in RIB-In at router R1

```
A:admin@R1# show router bgp routes flow-vpn-ipv6
=====
BGP Router ID:192.0.2.1          AS:64496          Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP FLOW VPN IPV6 Routes
=====
Flag  RD          Nexthop          LocalPref    MED
      As-Path
-----
u*>i  64496:1       ::              100          None
      No As-Path

      Community Action: target:64496:1
      Community Action: rate-limit: 0 kbps
      Flowspec Components:
      Dest Pref : 2001:db8:4001:500::3/128 offset 0
      Src Pref  : 2001:db8:fe01::3/128 offset 0
      Ip Proto  : [ == 17 ]
      Dest Port : [ >4190 ] and [ <4199 ]
-----
Routes : 1
=====
```

A VRF-specific IPv6 filter is dynamically created by the system, which is appended with the VRF instance identifier. In this case the IP-VRF is instance 1, therefore the IPv6 filter 'fSpec-1' is created from which entries are cloned into other filter policies. The entry number in fSpec-1 is 256 and the Origin indicates that it was Inserted by BGP FlowSpec. As one would expect, the IPv6 match criteria is identical to that of the FlowSpec route. The IPv6 filter fSpec-1 currently shows zero matches (Ing. Matches : 0 pkts) and will always show zero matches. Matched traffic will only be evident in IPv6 filter entries cloned into other filter policies. The fSpec-1 filter can be considered a filter template whose contents will be cloned into other filter policy entries, and as there may be multiple of these showing matched traffic in this filter would be meaningless.

Output 12-10: System-generated fSpec-1 IPv6 filter

```
A:admin@R1# show filter ipv6 "fSpec-1"
=====
IPv6 Filter
=====
Filter Id       : fSpec-1
Scope          : Embedded
Type           : Normal
Shared Policer  : Off
Entries        : 1 (insert By Bgp)
Description     : IPv6 BGP FlowSpec filter for vprn 1
-----
```

```
Filter Match Criteria : IPv6
```

```
-----
Entry                : 256
Origin               : Inserted by BGP FlowSpec
Description          : (Not Specified)
Log Id              : n/a
Src. IP              : 2001:db8:fe01::3/128
Src. Port            : n/a
Dest. IP             : 2001:db8:4001:500::3/128
Dest. Port           : 4191..4198
Next Header          : 17
Dscp                 : Undefined
ICMP Type            : Undefined          ICMP Code       : Undefined
Sampling             : Off                Int. Sampling    : On
Tcp-flag             : (Not Specified)
Fragment             : Off                Routing Type0    : Off
HopByHop Opt         : Off                Routing Type4    : Off
Auth Hdr             : Off                ESP header       : Off
Flow-label           : n/a                Flow-label Mask : n/a
Egress PBR           : Disabled
Primary Action        : Drop
Ing. Matches          : 0 pkts
Egr. Matches          : 0 pkts
=====
```

The IPv6 filter 'FlowSpec-VPNIPv6-Filter' configured in *Output 12-8* and applied at ingress to the IP-VRF interface towards the external peer at AS 64511 shows that traffic with the match criteria identified by the FlowSpec-VPN-IPv6 NLRI components is being matched (Ing. Matches : 3638 pkts) and that is being dropped (Primary Action : Drop). As the **default-action** of the IPv6 filter is **accept**, all other traffic will be forwarded.

Output 12-11: IPv6 filter 'FlowSpec-VPNIPv6-Filter' applied at R1 towards AS 64511

```
A:admin@R1# show filter ipv6 "FlowSpec-VPNIPv6-Filter"
```

```
=====
IPv6 Filter
=====
Filter Id            : 6                      Applied          : Yes
--- [snip] ---
Filter Name          : FlowSpec-VPNIPv6-Filter
-----
Filter Match Criteria : IPv6
-----
Entry                : 456
Origin               : Inserted by embedded filter fSpec-1 entry 256
Description          : (Not Specified)
Log Id              : n/a
Src. IP              : 2001:db8:fe01::3/128
Src. Port            : n/a
Dest. IP             : 2001:db8:4001:500::3/128
Dest. Port           : 4191..4198
Next Header          : 17
Dscp                 : Undefined
ICMP Type            : Undefined          ICMP Code       : Undefined
Sampling             : Off                Int. Sampling    : On
Tcp-flag             : (Not Specified)
Fragment             : Off                Routing Type0    : Off
HopByHop Opt         : Off                Routing Type4    : Off
Auth Hdr             : Off                ESP header       : Off
Flow-label           : n/a                Flow-label Mask : n/a
Egress PBR           : Disabled
Primary Action        : Drop
```

```

Ing. Matches      : 3638 pkts (931328 bytes)
Egr. Matches      : 0 pkts

```

When the FlowSpec filter is no longer required it is withdrawn in an MP_UNREACH_NLRI Update message causing the dynamically instantiated filter entries to be removed and all associated TCAM resources to be released.

Debug 12-4: Withdraw of FlowSpec route

```

1643 2025/09/30 07:49:11.598 BST minor: DEBUG #2001 Base Peer 1: 192.0.2.10
Peer 1: 192.0.2.10: UPDATE
Peer 1: 192.0.2.10 - Received BGP UPDATE:
    Withdrawn Length = 0
    Total Path Attr Length = 24
    Flag: 0x90 Type: 15 Len: 20 Multiprotocol Unreachable NLRI:
        Address Family FLOW_IPV4
        NLRI len: 16
    dest_pref 172.31.100.232/32
    ip_proto  [ == 6 ]
    dest_port  [ >4190 ] and [ <4199 ]

```

FlowSpec Validation

FlowSpec routes are typically originated and contained within the administrative domain of an operator; particularly when used for DDoS mitigation purposes. This approach means applying ingress filters at the point where traffic enters the Autonomous System (AS), such as an external peering point. For these filters to be truly optimal however, they should be instantiated as close as possible to the source of the attack traffic, even if that means applying filters within another operator's domain. This means that FlowSpec routes must be exchanged between Autonomous Systems, which in turn requires that:

- A trust relationship exists between those Autonomous Systems.
- A method of validating FlowSpec routes advertised between Autonomous Systems exists. This is referred to as FlowSpec Validation.

Flow Specification rules received in Update messages are subject to the BGP path selection process in the same way as any other Update messages. In general, the first step of this process is to exclude routes that are considered unfeasible (for example the Next-Hop IP address is unreachable). In the case of FlowSpec Update messages this step is extended to validate a FlowSpec NLRI such that it is only considered feasible if the originator of the FlowSpec Update matches the originator of the best-match unicast route for the destination prefix embedded in the flow specification. The originator is defined as either the BGP Originator path attribute if Route-Reflection is being used, or the transport address of the BGP peer if this attribute is not present.

FlowSpec validation is enabled at the BGP level within the **flowspec** context. It can be useful when FlowSpec routes are advertised between Autonomous Systems, and a trust relationship exists between those Autonomous Systems allowing a FlowSpec route-server from outside the Autonomous System to install a dynamic filter on a router within the local

Autonomous System. Within the **flowspec** context, if the **validate-dest-prefix** command is set to **true**, the router will only consider a FlowSpec route valid if the originator of that FlowSpec route is also the originator of the destination prefix in the FlowSpec NLRI. If the originator of the FlowSpec route is not the originator of the destination prefix in the FlowSpec NLRI the route is considered invalid, retained in RIB-In, but not used for filtering purposes.

Output 12-12: FlowSpec validation configuration

```
router "Base" {
    bgp {
        flowspec {
            validate-dest-prefix true
        }
    }
}
```

As an example, using the output above the FlowSpec originator connected to R4 advertises a FlowSpec route for destination prefix 172.22.148.0/24. This FlowSpec filtering component contains a destination prefix that is not advertised in unicast IPv4 by the FlowSpec originator (or by any BGP speaker). As a result, R4 considers the FlowSpec route inactive. Note the 1/0/0 value in the Rcv/Act/Sent column against FlowIPv4 in *Output 12-13*. This route will be considered invalid and held in RIB-In but not used for filtering purposes.

Output 12-13: Invalid FlowSpec route at R4

```
A:admin@R4# show router bgp summary | match 'Neighbor|192.168.254.18' post-lines 5
Legend : D - Dynamic Neighbor
=====
Neighbor
Description
                AS PktRcvd InQ Up/Down State|Rcv/Act/Sent (Addr Family)
                PktSent OutQ
-----
192.0.2.10
192.168.254.18
                65530      10    0 00h01m14s 0/0/4 (IPv4)
                                0/0/0 (IPv6)
                                1/0/0 (FlowIPv4)
                                0/0/0 (FlowVpnIPv4)
                                0/0/0 (FlowIPv6)
```

The FlowSpec originator is then modified to source an IPv4 Update for prefix 172.22.148.0/24. Because this is the best path that R4 has towards that unicast prefix, and it is the same as the originator of the FlowSpec route. As a result, the FlowSpec route is considered valid and therefore shown as active (Act). The use of validation in this manner allows the dynamic filter to be installed at the point closest to the source of the attack.

Output 12-14: Valid FlowSpec route at R4

```
A:admin@R4# show router bgp summary | match 'Neighbor|192.168.254.18' post-lines 5
Legend : D - Dynamic Neighbor
=====
Neighbor
Description
                AS PktRcvd InQ Up/Down State|Rcv/Act/Sent (Addr Family)
                PktSent OutQ
```

```

-----
192.0.2.10
192.168.254.18
        65530          9      0 00h00m06s 1/1/4 (IPv4)
                        11     0           0/0/0 (IPv6)
                                1/1/0 (FlowIPv4)
                                0/0/0 (FlowVpnIPv4)
                                0/0/0 (FlowIPv6)

```

Within the **flowspec** context a second option, the **validate-redirect-ip**, can be used with Redirect-to-IP FlowSpec routes. When this option is set to **true** the router will only consider a FlowSpec route valid if the originator of the FlowSpec route is also the originator of the redirect IP address prefix.

FlowSpec provides a powerful and dynamic way to instantiate and remove ingress filters to mitigate against DDoS attacks and is widely used in production networks today. It allows for hardware resources to be consumed on a temporary basis, only when needed to mitigate a threat. It also allows for a single point to push configuration to a large number of devices dynamically without having to manually configure each filtering point.

Remote Triggered Black-Holing



Unicast RPF is not supported on the 7250 IXR Gen 1 or Gen2/2c platforms.

Unicast RPF (URPF) is an optional and configurable feature whereby the source address of an ingress IP packet is verified against the routing table to confirm that it was received on the routers best-path to that source address. URPF [54] helps to mitigate problems with spoofed IP addresses that lack a verifiable IP source address (or rapidly changing source IP addresses). It is an optional security mechanism that can be used to dynamically create filters on external interfaces, not using explicitly defined filter rules like FlowSpec, but instead using the routing table to create a blackhole for traffic with specific destination addresses.

SR-OS supports two modes of URPF, **strict** and **loose**:

- In the strict mode a lookup is made in the routing-table for the source IP address, and verification is made that the best route to the source is over the interface from which the packet arrived. If not, the packet is dropped. The strict option also has a sub-option in SR-OS, **strict-no-ecmp**, that negates the use of ECMP routes to verify the best route and interface towards the source.
- In the loose mode a lookup is made in the routing-table for the source IP address, and verification is made that a route exists in the routing-table, but not necessarily over the interface over which the packet arrived.

The loose mode of URPF helps to deal with problems in strict mode in the presence of asymmetry where redundant paths are available to a given source address. Note that for both modes, the option exists to ignore the presence of a default-route in the routing-table when looking up the path towards a source using the command **ignore-default**.

URPF is an integral part of providing an optional and rather simple security mechanism at the network perimeter facing external peers using Remote-Triggered Black-Hole (RTBH) filtering. The loose mode of URPF is sufficient for RTBH filtering and is applied at IP interface level using the command **urpf-check** under the **ipv4** or **ipv6** contexts as shown in *Output 12-15*.

Output 12-15: uRPF configuration

```
service {
  ies "External-Peering" {
    interface "EBGP-to-AS64511" {
      sap 1/1/c3/1:5 {
      }
      ipv4 {
        urpf-check {
          mode loose
        }
      }
    }
  }
}
```

The next step to enabling RTBH capability is to configure a static-route for a 32-bit IPv4 address with a destination of blackhole.

Output 12-16: Static-Route to Blackhole

```
router "Base" {
  static-routes {
    route 192.0.2.253/32 route-type unicast {
      blackhole {
        admin-state enable
      }
    }
  }
}
```

When the URPF check and static route to blackhole are in place, it is possible to announce bogons or other known bad source addresses into IPv4 IBGP (with the well-known attribute ‘no-export’) so that they are learnt by routers with external interfaces. When announcing these prefixes, the BGP Next-Hop attribute should correspond with the pre-configured static-route to black-hole.

To illustrate the use of RTBH the same topology is used for FlowSpec (*Figure 12-2*) where router R1 is externally peered with AS 64511. Router R1 learns prefix 172.16.0.0/20 from AS 64511 and advertises the internal prefix 172.31.100.0/24 towards it. URPF in loose mode is enabled at R1 on the interface towards the external peer, and a static route to black-hole is configured for prefix 192.0.2.253. In this example the objective is to ensure that no traffic enters the network at R1 from AS 64511 with a source-address of 10.0.0.0/8. To achieve this, router R4 (or any other internal router) sources an Update message for prefix 10.0.0.0/8 and sets the Next-Hop to 192.0.2.253 to reconcile with the static-route to blackhole at R1.

Output 12-17: RTBH dynamic filter at router R1

```
A:admin@R1# show router bgp routes 10.0.0.0/8
=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
```

```

Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete

```

```

=====
BGP IPv4 Routes
=====

```

Flag	Network NextHop (Router) As-Path	LocalPref Path-Id	MED IGP Cost Label
u*>i	10.0.0.0/8	100	None
	192.0.2.253	6472	1
	No As-Path		-

```

-----
Routes : 1
=====

```

When packets arrive at R1's external interface, the URPF check verifies that packets are received over the router's best path to that source address. In this case, packets arriving at the ingress of the interface towards with AS 64511 with a source address in the range of 10.0.0.0/8 are matched to a route-table entry with Next-Hop 192.0.2.253 and a preferred path to black-hole. Hence, ingress packets are discarded because of the URPF check as shown in *Output 12-18*.

Output 12-18: Unicast IPv4 RPF drops at R1

```

A:admin@R1# show port 1/1/c3/1 detail | match 'Forwarding Engine Drop Reason' post-lines
15
Ingress Port Forwarding Engine Drop Reason Statistics
=====
IPv4 Header Error                                0
IPv4 Invalid Address                             0
IPv6 Header Error                                0
IPv6 Invalid Address                             0
IP Route Blackholed                              0
ACL Filter Discards                              0
Unicast RPF Check Failed                          50
BFD Spoof Check Failed                           0
Unicast MAC Destination Address Mismatch          0
Multicast MAC With Unicast Dest IP                0
Unknown MAC Destination Address Discarded in VPLS 0
L2 Service MTU Exceeded                           0
Needs ICMP                                         0
Unknown Labeled Packet                            0

```

Bogons, other known 'bad' source prefixes can now be announced into IBGP from a centralised route-server with the same Next-Hop address to create a dynamic method of generating filters on external interfaces.

Generalised TTL Security Mechanism



BGP FlowSpec is not supported on the 7250 IXR Gen 1 or Gen2/2c platforms.

Generalised TTL Security Mechanism (GTSM) [55] is designed to protect a router from CPU-utilisation-based attacks. GTSM is based on the fact that most external peering sessions are established between directly adjacent routers and therefore uses the expected TTL value from the peer to provide a simple and robust form of defence against packets from other devices that are not directly connected. GTSM suggests that the TTL is set to 255 (the maximum possible for IP) and thereafter rejects any protocol packets that come in from configured peers that do not have an inbound TTL of 255.

GTSM is enabled at group or neighbour level using the command **ttl-security** followed by a minimum configurable TTL value. Although somewhat misleading, the **multihop** command is also needed with a corresponding TTL value in order to generate the correct TTL value, despite the fact that the BGP session is established between directly connected peers. If the TTL of received BGP packets does not match the configured **ttl-security** and **multihop** value, the manner in which BGP processes incoming packets is hardware dependent. Platforms that have a CPM-based NPU do not accept any packets that do not have the configured **ttl-security** and **multihop** values, therefore the BGP session will not reach the Established state. For platforms that do not have a CPM-based NPU, the BGP session will move to the Established state before Keepalive and other Update messages are dropped by the GTSM filter. The BGP session will therefore fail and consequently recycle through the same behaviour. It is important therefore that both peers have the same TTL configuration.

Output 12-19: GTSM configuration

```
router "Base" {
  bgp {
    group "EBGP-to-AS64511" {
      peer-as 64511
    }
    neighbor "192.168.1.2" {
      ttl-security 255
      multihop 255
      group "EBGP-to-AS64511"
    }
  }
}
```

If the session is an EBGP multihop session, the same intended TTL value should be configured with the **ttl-security** command and the **multihop** command. By default, SR-OS sets BGP packets sent to an IBGP peer with a TTL value of 64, while BGP packets sent a directly connected peer have a TTL value of 1.

Auto-Generation of Filters for BGP Peers

When implementing security policies, most operators have a requirement to constrain the source/destination IP address ranges and protocols/ports which can obtain access to a router's CPU resources. With SR-OS and Nokia routing platforms, this is typically implemented using Control and Processing Module (CPM) or linecard (IOM/IMM) filters. The former uses hardware-based policers and optionally queues on the actual CPM (Route Processor) to block/allow packets when they have crossed the switch fabric and reached the

CPM. The latter implements filtering on the line-card, which when applied on ingress, can block packets from leaving the ingress linecard.

When configuring and applying filters the address ranges used for internal control-plane and management-plane purposes are typically known and represent a one-time configuration. However, when a router is peering externally with a number of peers that are frequently being updated, the process of manually updating filters can become rather onerous (not to mention they are open to mis-configuration). Auto-generation of filters for BGP peers is a feature that can remove this operational overhead. It allows for configuration of an IPv4 or IPv6 prefix ‘match-list’ with specified criteria as part of a filter. When the router’s BGP peering configuration changes, and that change matches the specified criteria, the address prefixes are automatically updated. In turn all CPM and linecard filters that use the match-list are also updated. To allow for these automatic and ‘touch-less’ filter updates, the configured IPv4/IPv6 address prefix match-lists must be able to define acceptable ranges of allowable peer groups or neighbour addresses. To support this, regular expressions can be used to match against peer groups or specific BGP neighbours.

Output 12-20 provides a simple example of the match-list using regular expressions. The **ip-prefix-list** command creates a context where both static IPv4 address prefixes and auto-generated IPv4 address prefixes can be configured and may co-exist (a similar context **ipv6-prefix-list** exists for configuration of static and auto-generated IPv6 address prefixes). Within the **ip-prefix-list** context static address prefixes are configured using the command **prefix** followed by a prefix/length (not shown in this example), whereas auto-generated address prefixes are configured within the **apply-path** context. Within this context, the **bgp-peers** command defines matching criteria for allowable address prefixes using regular expressions if required. This example has three **bgp-peers** statements in the list. Each **bgp-peers** context defines a **group**, a **neighbor**, and a **router-instance**. The **group** command is used to reference a (peer) **group** configured in the base or IP-VRF BGP routing context. The **neighbor** command is used to define acceptable match criteria for one or more BGP peers within the afore-mentioned **group**. Lastly, the **router-instance** defines which base or IP-VRF routing instance the **group** and **neighbor** statements are applicable to. The three configured **bgp-peers** in this example are derived as follows:

- The **bgp-peers 1** context permits any BGP neighbour from the peer group IBGP, where “.” is used to match against any neighbour in the base routing instance.
- The **bgp-peers 2** group permits any BGP neighbours from the peer group EBGp providing the first three octets of the peer IP address are 192.168.0 and also refers to peers in the base routing instance.
- The **bgp-peers 3** group permits any BGP peers from the peer group CE-EBGP providing the first three octets of the peer IP address are 192.168.100. This group is applicable to BGP peers in IP-VRF “one”.

Output 12-20: IP-Prefix Match-List

```
filter {
  match-list {
    ip-prefix-list "BGP-PEERS" {
      description "Auto-Update of CPM filter for BGP peers"
```

```

        apply-path {
            bgp-peers 1 {
                group "IBGP"
                neighbor ".*"
                router-instance "Base"
            }
            bgp-peers 2 {
                group "EBGP"
                neighbor "192.168.1"
                router-instance "Base"
            }
            bgp-peers 3 {
                group "CE-EBGP"
                neighbor "192.168.100"
                router-instance "one"
            }
        }
    }
}

```

The match-list is then referenced in CPM-filter or generic IPv4/IPv6 filter entries as shown in *Output 12-21*, which gives an example of the former. Within each entry the **src-ip** command references the **ip-prefix-list** configured previously. Two entries are present to allow for either party remote or local router to initiate the BGP session.

Output 12-21: CPM-Filter with Prefix-Lists

```

system {
    security {
        cpm-filter {
            default-action accept
            ip-filter {
                entry 100 {
                    match {
                        protocol tcp
                        src-ip {
                            ip-prefix-list "BGP-PEERS"
                        }
                        src-port {
                            eq 179
                        }
                    }
                    action {
                        accept
                    }
                }
                entry 101 {
                    match {
                        protocol tcp
                        src-ip {
                            ip-prefix-list "BGP-PEERS"
                        }
                        dst-port {
                            eq 179
                        }
                    }
                    action {
                        accept
                    }
                }
            }
        }
    }
}

```

Output 12-22 shows the auto-generated IPv4 prefixes based on the a) the **bgp-peers** statements configured in the **match-list** and b) the configured BGP peers in both the base routing context and the IP-VRF “one”, and how they have been referenced (in this case the CPM filter entries 100 and 101). On this router there are four statically configured BGP peers. The first is peer 192.0.2.10/32 in group IBGP in the base routing context. The second and third are peers 192.168.1.2 and 192.168.2.2 which belong to the group EBG, also in the base routing context. The final peer is 192.168.100.2 in IP-VRF “one” (routing instance 1).

- An address prefix is auto-generated for the first peer (192.0.2.10/32) because the regular expression permitted all peers in group IBGP.
- An address prefix is also auto-generated for the second peer 192.168.1.2 in group EBG. An address prefix (and subsequent filter entry) is not however auto-generated for the third peer 192.168.2.2 in group EBG because the **neighbor** configuration in the **bgp-peers 2** context specified that the first three octets are 192.168.1. As a result, this peer does not have access to the CPM and the BGP session cannot be established.
- Finally, an address-prefix is auto-generated for the fourth and final peer 192.168.100.2 in group CE-EBGP within routing-instance 1.

Output 12-22: Auto-Generated BGP peering prefixes

```
A:admin@R1# show filter match-list ip-prefix-list "BGP-PEERS"
```

```
=====
Filter Match IP Prefix Lists
=====
ip-prefix-list "BGP-PEERS"
=====

  apply-path (bgp-peers) prefixes:
-----
    192.0.2.10/32      192.168.1.2/32      192.168.100.2/32
-----
NUM prefixes: 3

References:
-----
    Cpm-filter        entry 100      Src-Ip
    Cpm-filter        entry 101      Src-Ip
-----
NUM references: 2
=====
```

Output 12-23 shows the BGP neighbour summary for the base routing instance and shows that the neighbour 192.0.2.10 in group IBGP and the neighbour 192.168.1.2 in group EBG are both in the established state for their relevant Address Families. The neighbour 192.168.2.2 is in the connect state because an IP prefix for this peer was not auto-generated, and as a result there is no match in the CPM filter.

Output 12-23: BGP Neighbor summary for Base routing instance

```
A:admin@R1# show router bgp summary
```

```
=====
```

```

BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
BGP Admin State      : Up      BGP Oper State      : Up
--- [snip] ---

=====
BGP Summary
=====
Legend : D - Dynamic Neighbor
=====
Neighbor
Description
          AS PktRcvd InQ  Up/Down  State|Rcv/Act/Sent (Addr Family)
          PktSent OutQ
-----
192.0.2.10
          64496   3928   0 01d07h05m 3/2/2 (IPv4)
          4101   0      ...etc
192.168.1.2
          64511    243   0 01h59m00s 2/2/2 (IPv4)
          244    0
192.168.2.2
          64514     0    0 01h18m55s Connect
          0      0
-----

```

Output 12-24 shows the same BGP neighbour summary for IP-VRF “one” and routing-instance 1 showing that the neighbour 192.168.1.2 in group CE-EBGP is in the established state and can exchange routes for the IPv4 Address Family.

Output 12-24: BGP Neighbour summary for IP-VRF "one" (routing instance 1)

```

A:admin@R1# show router 1 bgp summary
=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
BGP Admin State      : Up      BGP Oper State      : Up
--- [snip] ---

=====
BGP Summary
=====
Legend : D - Dynamic Neighbor
=====
Neighbor
Description
          AS PktRcvd InQ  Up/Down  State|Rcv/Act/Sent (Addr Family)
          PktSent OutQ
-----
192.168.100.2
          64512  12380   0 04d07h07m 3/2/11 (IPv4)
          12410   0
-----

```

The TCP Authentication Option (TCP-AO)

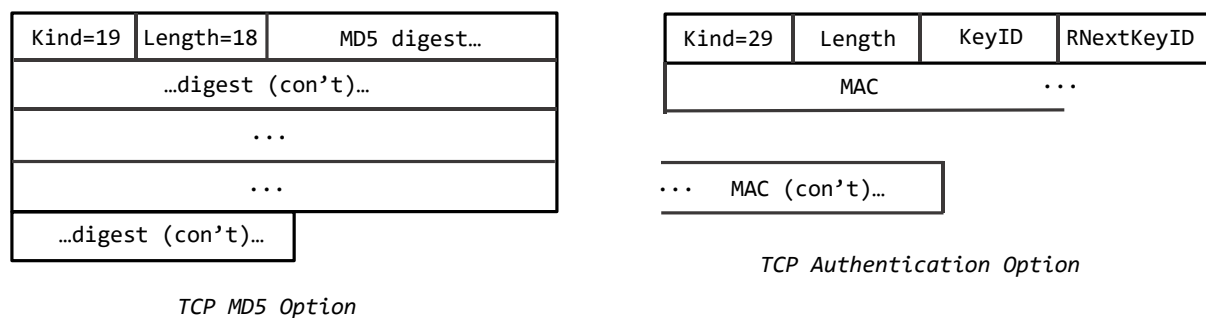
The TCP MD5 signature is a TCP option that authenticates TCP segments, including the TCP IPv4 pseudoheader, TCP header, and TCP data. Every segment sent on a TCP connection contains the 16-byte MD5 digest, which is generated using a key or password known to both ends of the TCP connection. It is widely deployed and used in production networks, but

security groups have raised concerns about the use of TCP MD5. It uses a simple keyed hash for authentication, which has proven to be problematic because of attacks on the algorithm itself. The TCP MD5 algorithm also lacks both key-management and algorithm agility.

The TCP Authentication Option (TCP-AO) [56] is intended to provide enhanced security protection for TCP-based applications such as BGP. It specifies the use of a larger set of stronger Message Authentication Codes (MACs), protects against replays (even for long-lived connections), and provides more details on the association of security with TCP connections than TCP MD5. It also provides a key coordination mechanism giving the ability to move from one key to another within the same TCP connection, which is perceived as a key asset of TCP-AO. It obsoletes TCP MD5, although its adoption has been relatively slow outside of greenfield deployments.

TCP-AO uses a configurable authentication key, which together with one of the numerous supported authentication algorithms form a Master Key Tuple (MKT) used to generate traffic keys to provide mutual authentication of TCP peers. The mandatory algorithms for TCP-AO are HMAC-SHA-1-96 and AES-128-CMAC-96. The TCP-AO option format is shown in *Figure 12-3* together with the MD5 option for comparison purposes. The TCP-AO Kind has an IANA-allocated value of 29, although some implementations are known to use have used and continue to use the experimental values 253 and 254. The KeyID indicates the MKT that was used to compute traffic keys for the TCP segment. The RNextKeyID is a 1-byte field indicating the MKT that is ready at the sender to be used to authenticate received segments. The signalling of the RNextKeyID in this manner provides a mechanism to indicate when a new MKT is ready, which allows the peer to commence use of that MKT. This permits the use of a new MKT use to be coordinated, avoiding any traffic loss that may be experienced by the sender using an MKT that is not yet ready at the receiver. The MAC (Message Authentication Code) field is typically 12-16 bytes in length but any length that fits in the header of the segment being authentication and contains the computed MAC for the TCP segment.

Figure 12-3: TCP MD5 Option and Authentication Option



Output 12-25 shows the configuration required to implement TCP-AO in SR-OS. The **keychain** 'BGP-KEYCHAIN' provides the context for the remainder of the TCP-AO parameters. The **tcp-option-number** provides the context to define the TCP option used. The configuration example uses **receive tcp-ao** and **send-tcp-ao** to use the standard option with Kind 29 but also provides the option to support the experimental values 253 and 254 in both transmit and receive directions (**option-253** and **option-254**). TCP-AO allows for the use of different MKTs in the send and receive direction, hence the **keychain** context allows for

configuration contexts in the **send**, **receive**, and **bidirectional** directions. This example uses the **bidirectional** option to enable use of the same MKT in both send and receive directions. Within the **bidirectional** context an **entry** must be configured with a number in the range [0..63,255]. This value is carried in the *KeyID* field of the TCP authentication option. Within the **entry** context, the **authentication-key** contains the user-selected key, whilst the **algorithm** allows for selection of one of the supported algorithms, which include the mandatory algorithms **aes-128-cmac-96** and **hmac-sha-1-96**, as well as **hmac-md5**, **hmac-sha-1**, **hmac-sha-256**, **aes-128-gcm-16**, **aes-128-cmac-128**, **password**, and **message-digest**. The **begin-time** command specifies the calendar date and time after which the **entry** is used to generate traffic keys and authenticate TCP segments. The **begin-time** syntax allows for various formats of day/month/year and hour/minute/second together with a time-zone, and the configured time-zone will automatically offset the hour from UTC accordingly when displaying the time in configuration. Lastly, the **tolerance** command specifies the amount of time that an eligible receive key should overlap with the active send key. The **infinite** option specifies that they can overlap infinitely and allows for peering devices that may not have synchronised wall-clock times (for example they do not run NTP or PTP).

Output 12-25: TCP Keychain configuration

```

system {
  security {
    keychains {
      keychain "BGP-KEYCHAIN" {
        tcp-option-number {
          receive tcp-ao
          send tcp-ao
        }
        bidirectional {
          entry 1 {
            authentication-key "C3i5c6IP3VMhQFGff3RCM5L3zT0hytM= hash2"
            begin-time 2025-09-30T00:01:00.0Z
            algorithm aes-128-cmac-96
            tolerance infinite
          }
        }
      }
    }
  }
}

```

The configuration example in *Output 12-25* contains a single entry for simplicity. To coordinate a key change, it would be necessary to have at least two entries, and for the purpose of explanation they are hypothetically called entry 1 and entry 2. These entries are configured on routers R1 and R2, and it is further assumed that entry 1 is in use. As a result, it is contained in the KeyID exchanged between the peers. Assuming that R1 initiates the key change, it sets the RNextKeyID to the value of 2 in the outgoing TCP segment. Upon receiving the TCP segment, R2 notes that the RNextKeyID is 2, and if it is configured and operational R2 sets the KeyID to 2. Once R2 has rolled over to the new MKT, R1 also sets the KeyID to 2.

The **keychain** is thereafter applied to the BGP peer using the **authentication-keychain** command either at BGP, group, or neighbour level as shown in Output 12-26.

Output 12-26: Applying the BGP keychain at neighbour level

```

router "Base" {
  bgp {
    neighbor "192.168.1.2" {
      authentication-keychain "BGP-KEYCHAIN"
      group "EBGP-to-AS64511"
      family {
        ipv4 true
      }
    }
  }
}

```

Lastly, *Output 12-27* shows the operational state of the peer with the keychain applied and TCP-AO active. The `Auth key chain` field shows the TCP-AO **keychain** applied to the session.

Output 12-27: Operational status of peer with TCP-AO

```

A:admin@R1# show router bgp neighbor 192.168.1.2

=====
BGP Neighbor
=====
-----
Peer          : 192.168.1.2
Description   : (Not Specified)
Group         : EBGP-to-AS64511
-----
Peer AS       : 64511           Peer Port      : 56577
Peer Address  : 192.168.1.2
Local AS      : 64496           Local Port    : 179
Local Address : 192.168.1.1
Peer Type     : External        Dynamic Peer   : No
State         : Established     Last State    : OpenConfirm
Last Event    : recvOpen
Last Error    : Cease (Connection Collision Resolution)
AS Override   : Disabled
Local Family  : IPv4
Remote Family : IPv4
--- [snip] ---
Auth key chain : BGP-KEYCHAIN
Disable Cap Nego : Disabled      Bfd Enabled    : Disabled
Default Route Tgt : Disabled
--- [snip] ---

```

Prefix Origin Validation

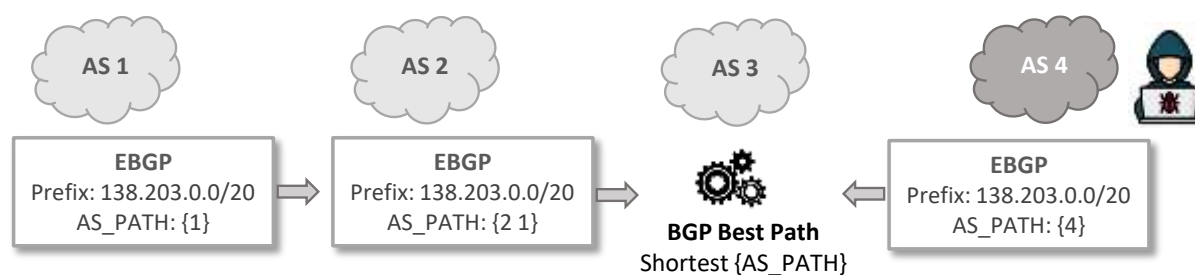


Prefix Origin Validation is not supported on the 7250 IXR Gen 1 platform.

Prefix Origin Validation is a mechanism that allows a router to validate the origination AS of BGP routes in order to prevent malicious or unintentional prefix mis-announcement and take the appropriate action based upon validation or non-validation of that prefix. It was conceived to attempt to tackle one of the most common attacks on the Internet, prefix

hijacking. BGP prefix hijacking is when an Autonomous System (attacker, or perhaps just a misconfiguration) illegitimately announces IP address prefixes that belong to a different Autonomous System, thus diverting traffic to the illegitimate AS. The result is that traffic intended for the legitimate network is rerouted through the illegitimate network causing either a denial of service, an outage for the affected prefix(es), or worse, traffic to be intercepted, inspected, or even impersonated. A simple example of prefix hijacking is shown in *Figure 12-4*. AS 1 has received prefix 138.203.0.0/20 from a Regional Internet Registry (RIR) and is therefore the only AS authorised to originate it onto the Internet. AS 4, either through malicious intent, configuration error, or unethical behaviour advertises the same prefix. A receiving AS, such as AS 3 would then receive two announcements of the prefix 138.203.0.0/20, and if the BGP best path selection process should choose the unauthorised announcement as the best path, traffic towards this prefix would be diverted into a black-hole (and potentially analysed). In the example shown, AS 3 would very likely consider AS 4 as the best path because of the shortest AS_PATH.

Figure 12-4: Example of prefix hijacking



In the example above, AS 4 announced the entire 138.203.0.0/20 prefix that is registered to AS 1. Another version of prefix hijacking is when an AS illegitimately originates a subnet of a prefix that an RIR assigned to another AS. Using the same example as above, if AS 4 were to originate the prefix 138.203.8.0/24 then this is a subnet of the prefix 138.203.0.0/20 originated by and registered to AS 1. Routers in AS 1, 2, and 3 would install both of these prefixes, and because of the longest match prefix rule, a portion of the traffic 138.203.0.0/20 (traffic for 138.203.8.0/24) would be diverted to AS 4, once again causing a black-hole.

Although there is unethical behaviour involved at AS 4, it also highlights an inherent vulnerability within BGP, which is the lack of a check on the identity of who advertises prefixes onto the Internet. In other words, there is no way to confirm that the entity from which a prefix was learned is actually authorised to advertise that prefix and has been assigned that prefix by an RIR. To solve this problem, a standard architecture [58] [59] was developed based on a public structure known as the Resource Public Key Infrastructure (RPKI). The overall architecture of RPKI consists of three main components:

- A public key infrastructure (PKI) with the necessary certificate objects.
- Digitally signed routing objects.

- A distributed repository system (RPKI repositories) to hold the objects that would also support periodic retrieval.

The RPKI system is based on resource certificates that define extensions to X.509 to represent IP addresses and AS identifiers. Separate digitally signed objects that define associations between AS's and IP address blocks are known as Route Origin Authorisation (ROAs). The ROA has four components:

- An IPv4 or IPv6 prefix with fixed prefix length.
- A maximum mask length that specifies which IP subnets from the originating prefix may be announced.
- The AS number of the AS which is authorised to originate the IP prefix or one of its subnets.
- A digital signature, based on X.509 certificates.

Although there is a single RPKI repository system that is accessed by relying parties, the authoritative data of the RPKI is published in a distributed set of servers (RPKI publication repositories) held at RIRs, NIRs, and ISPs. Local caches of that authoritative data are maintained and periodically refreshed directly or indirectly from the global RPKI. A BGP speaker establishes and keeps open a connection to one or more of these local caches with which it has a client/server relationship using a dedicated RPKI-Router protocol [60] that can be protected from man-in-the-middle attacks using SSHv2, TCP MD5, or IPSec. For each cache connection, the BGP speaker maintains the IP address/fully qualified domain name of the cache, any needed public key of the cache, any needed private key or certificate for itself, and a preference to indicate which cache is preferred in the event that the router is configured to peer with a number of caches. Periodically the router sends to the cache the serial number of the highest numbered data it has received from the cache, following which the cache responds with all data records that have serial numbers greater than that presented in the router's query. In addition, when the cache updates its own database, it sends a Notify message to all of its configured peers as a hint to poll for an update.

Output 12-28 shows the required configuration to establish a session to a local RPKI cache, noting that SR-OS supports RPKI sessions in the base routing instance, management instance, and in a VRF. In addition, there is no current support for protection of RPKI-Router connections. *Output 12-29* then shows the verification of the RPKI session state.

Output 12-28: RPKI session configuration

```
router "Base" {
    origin-validation {
        rpki-session 192.168.1.45 {
            admin-state enable
            local-address 192.168.1.1
            port 8282
        }
    }
}
```

Output 12-29: Verification of RPKI session state

```

A:admin@R1# show router origin-validation rpki-session "192.168.1.45" detail
=====
RPKI Session Information
=====
IP Address       : 192.168.1.45
Description      : (Not Specified)
-----
Port             : 8282                Oper State       : established
UpTime           : 0d 00:10:21         Flaps             : 2
Active IPv4 records: 14273             Active IPv6 records: 2192
Admin State      : Up                  Local Address      : 192.168.1.1
Hold Time        : 600                 Refresh Time       : 300
Stale Route Time : 3600                Connect Retry      : 120
Serial ID        : 0                   Session ID         : 6
=====
No. of Sessions  : 1
=====

```

The BGP speaker loads validated objects from the cache into local storage, where those objects have the content (IP address, prefix length, maximum length, origin AS number). A locally stored object of this nature is referred to as a Validated ROA Payload (VRP). *Output 12-30* shows a truncated version of the origin-validation database when it is loaded into the system from the RPKI cache server. The output shows the prefixes marked as [Dynamic], which indicates that the prefix was dynamically learnt from the RPKI cache. The Session IP Flags also show [B] for the two entries to indicate that they are known through a base routing instance session. An option exists to statically configure VRP entries using the **static-entry** command in the **router origin-validation** context, where the static entry can define a specific prefix, a maximum length, and an origin AS combination as either valid or invalid.

Output 12-30: Origin Validation database

```

A:admin@R1# show router origin-validation database
=====
Static and Dynamic VRP Database Entries
=====
Prefix Range [Flags]                                Origin AS
  Session IP [Flags]
-----
27.50.32.0/21-21 [Dynamic]                            6939
  192.168.1.45 [B]
27.50.32.0/24-24 [Dynamic]                            38186
  192.168.1.45 [B]
....

```

Each BGP speaker supporting prefix origin validation, whether in the base routing instance or in a VRF, determines the origin validation state for every route that it learns through BGP noting that the origin AS number is the rightmost AS of the AS_PATH attribute. The following definitions are used:

- Covered: A route prefix is said to be Covered by a VRP when the VRP prefix length is less than or equal to the route prefix length, and the VRP prefix address and the route prefix address are identical for all bits specified by the VRP prefix length. (In other words, the route prefix is either identical to the VRP prefix or more specific than the VRP prefix.)

- **Matched:** A route prefix is said to be Matched by a VRP when the route prefix is Covered by that VRP, the route prefix length is less than or equal to the VRP maximum length, and the route origin ASN is equal to the VRP ASN.

To use the contents of the origin-validation database to determine prefix-origin, it needs to be explicitly enabled using the command **origin-validation**, which can be enabled for **ipv4**, **ipv6**, **label-ipv4**, and **label-ipv6** Address Families. The command can be enabled at group or neighbour level.

Output 12-31: Origin-Validation configuration

```
router "Base" {
    bgp {
        group "EBGP-to-AS64511" {
            peer-as 64511
            origin-validation {
                ipv4 true
            }
        }
        neighbor "192.168.1.2" {
            group "EBGP-to-AS64511"
            family {
                ipv4 true
            }
        }
    }
}
```

When a BGP speaker subsequently receives an Update message from an external peer, the rightmost AS number in the AS_PATH attribute of the prefix is examined and one of the three validation states is derived:

- **NotFound:** No VRP Covers the route prefix.
- **Valid:** At least one VRP Matches the route prefix
- **Invalid:** At least one VRP Covers the route prefix, but no VRP Matches it.



Note: Depending on where the operator peers, there is every possibility that an operator taking an RPKI-cache may not receive any RPKI Invalid routes. Many Tier 1 operators filter them out, and although they may occasionally be seen in Tier 2 operators or at an IX, they are unlikely to be seen beyond that. The monitoring website here [Route Origin Validation Monitoring](#) allows operators to filter on current Invalid routes, and at the time of writing lists 2,374 RPKI Invalid routes in the Internet. Additionally, there are a number of RPKI Invalid 'beacons', which are prefixes made intentionally RPKI Invalid with an ROA only allowing AS 0 as an originating AS. These include the IPv4 prefixes 103.21.244.0/24, 185.49.142.0/24, 2606:4700:7000::/48, and 2a04:b907::/48.

By default, when a BGP speaker in SR-OS supports prefix origin validation the BGP decision process is not modified. However, two optional commands exist in the **best-path-selection** context that can change that behaviour. When **origin-invalid-unusable** is set to **true**, all routes that have an origin validation state of Invalid are considered unusable by the best-

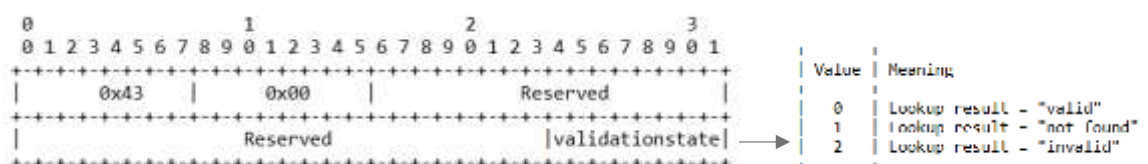
path selection algorithm and are not advertised to other BGP speakers. If it is not configured (default), prefixes that have invalid origin are considered usable and subject to the normal BGP best-path selection process. The **compare-origin-validation-state** modifies the BGP decision process so that an additional validation step is performed after removal of invalid routes (pre-qualification for best-path selection), and before the comparison of Local Preference (Rule 2). The additional step simply states that when comparing a pair of routes for a BGP destination, the route with the lowest “validation state” is preferred. That is, a Valid state is preferred to a NotFound state, and a NotFound state is preferred to an Invalid state.

Output 12-32: Best-path selection with Origin-Validation

```
router "Base" {
  bgp {
    best-path-selection {
      compare-origin-validation-state true
      origin-invalid-unusable true
    }
  }
}
```

To avoid every router in an Autonomous System peering with RPKI caches, the validation state identified by externally peering routers optionally may be propagated into the Autonomous System using an opaque Origin Validation State extended community attribute [61]. The Origin Validation State extended community has a Type value of 0x43 it is non-transitive, and a Sub-Type value of 0x00. The last two octets of the value field encode the validation state of a route, where 0x00=valid, 0x01=not found, and 0x02=invalid.

Figure 12-5: Origin Validation Extended Community

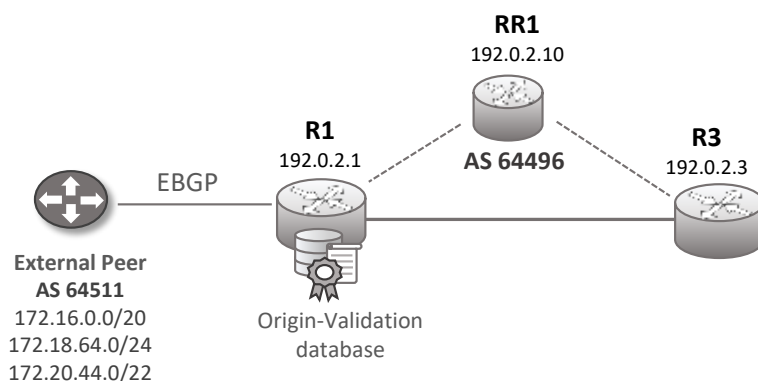


To illustrate the use of Origin Validation the topology shown in *Figure 12-6* is used. Routers R1 and R3 are in AS 64496 and peer with RR1 for the IPv4 Address Family. Router R1 peers externally with AS 64511 and has **origin-validation** enabled for the IPv4 address family. AS 64511 advertises prefixes 172.16.0.0/20, 172.18.64.0/24, and 172.20.44.0/22. The Origin Validation database at R1 would typically be populated with entries from an RPKI local cache, but for this illustration has two static entries:

- Prefix 172.16.0.0/20 with an Origin AS of 64511 and a maximum length mask of 22.
- Prefix 172.18.64.0/20 with an Origin AS of 64511 and a maximum length mask of 22

Of course, these private range prefixes would not be advertised onto the Internet or populated in an RPKI cache or Origin Validation database, but they will suffice for illustration purposes.

Figure 12-6: Origin-Validation Test Topology



Output 12-33 shows the Origin-Validation database at router R1 containing the two static entries described above.

Output 12-33: Origin-Validation database at R1

```
A:admin@R1# show router origin-validation database
```

```
=====
Static and Dynamic VRP Database Entries
=====
Prefix Range [Flags]                               Origin AS
  Session IP [Flags]
-----
172.16.0.0/20-22 [Static-V]                         64511
-
172.18.64.0/20-22 [Static-V]                       64511
-
-----
No. of VRP Database Entries: 2
-----
Flags: B = Base instance session
      M = Management instance session
      Static-V = Static-Valid; Static-I = Static-Invalid
=====
```

Given the two static entries in the Origin Validation database and the three prefixes advertised by AS 64511, Output 12-34 shows the derived Origin Validation state at R1 for the three prefixes:

- The first entry is for prefix 172.16.0.0/20 that shows an Origin Validation state of valid (DB Orig Val : Valid) because it is Matched by a VRP. That is, the route prefix is Covered by a VRP, its prefix length is less than or equal to the VRP maximum length, and the origin ASN is equal to the VRP ASN. The two lines in the prefix output represent the original attributes and the modified attributes (post-policy).
- The second entry is for prefix 172.18.64.0/24 that shows an Origin Validation state of invalid (DB Orig Val : Invalid). This is because the prefix is Covered by a VRP

but is not matched by a VRP as it is a more specific route of the 172.18.64.0/21 VRP with a greater prefix length than the specified maximum length.

- The last output is for prefix 172.20.44.0/22 that has an Origin Validation state of NotFound (DB Orig Val : NotFound). This is simply because no VRP Covers the prefix.

Output 12-34: Origin Validation state for AS 64511 routes at R1

```
A:admin@R1# show router bgp routes 172.16.0.0/20 detail | match "DB Orig Val"
DB Orig Val      : Valid                      Final Orig Val : N/A
DB Orig Val      : Valid                      Final Orig Val : Valid

[/]
A:admin@R1# show router bgp routes 172.18.64.0/24 detail | match "DB Orig Val"
DB Orig Val      : Invalid                    Final Orig Val : N/A
DB Orig Val      : Invalid                    Final Orig Val : Invalid

[/]
A:admin@R1# show router bgp routes 172.20.44.0/22 detail | match "DB Orig Val"
DB Orig Val      : NotFound                   Final Orig Val : N/A
DB Orig Val      : NotFound                   Final Orig Val : NotFound
```

To propagate the routes internally towards R3, router R1 uses route-policy to attach the Origin-Validation extended community with the relevant value to the prefixes. The policy in *Output 12-35* matches prefixes based on the Origin Validation state and sets the Origin-Validation extended community to a value of ext:4300:0, ext:4300:1, and ext:4300:2 for states Valid, NotFound, and Invalid respectively. Note in the community list the use of “members ext:43:0”. Historically when matching extended community values in policy the extended community had to be named; for example, target:64496:1, or origin:64496:2. To allow for a more flexible approach to matching extended community values, hex values can be used matching Type and Sub-Type. The use of extended community filtering in this manner is described further in Chapter 13.

Output 12-35: Route-Policy to propagate Origin-Validation state at R1

```
policy-options {
  community "Origin-Validation-state-INVALID" {
    member "ext:4300:2" { }
  }
  community "Origin-Validation-state-NOTFOUND" {
    member "ext:4300:1" { }
  }
  community "Origin-Validation-state-VALID" {
    member "ext:4300:0" { }
  }
  policy-statement "Propagate-Origin-Validation-State" {
    entry 10 {
      from {
        origin-validation-state valid
      }
      action {
        action-type accept
        community {
          add ["Origin-Validation-state-VALID"]
        }
      }
    }
    entry 20 {
      from {
```

```

        origin-validation-state invalid
    }
    action {
        action-type accept
        community {
            add ["Origin-Validation-state-INVALID"]
        }
    }
}
entry 30 {
    from {
        origin-validation-state not-found
    }
    action {
        action-type accept
        community {
            add ["Origin-Validation-state-NOTFOUND"]
        }
    }
}
}
}
}

```

The route-policy is then added as an import policy towards the external peer at AS 64511, such that the validation state is propagated to internal peers.

Output 12-36: Attach Route-Policy to peer at ingress

```

router "Base" {
    bgp {
        neighbor "192.168.1.2" {
            import {
                policy ["Propagate-Origin-Validation-State"]
            }
        }
    }
}

```

At router R3 the three prefixes, 172.20.0.0/20, 172.18.64.0/24, and 172.20.44.0/21 are received with the Origin Validation extended community values ext:4300:0, ext:4300:2, and ext:4300:1 respectively. SR-OS does not automatically set the origin-validation state based on the received extended community because it's not considered universally appropriate — IBGP and EBGP peers have different trust levels, and there are exceptions. Additionally, different operators may want different actions for routes with Origin Validation state Invalid, hence an import policy is required at R3 that does the inverse of the example in *Output 12-35*, matching the Origin Validation extended community value and setting the Origin Validation state accordingly. With such a policy in place the Origin Validation state in the original attributes indicates a state of NotFound (DB Orig Val : NotFound), but the state in the modified attributes after policy has been applied shows the correct Valid/Invalid/NotFound value (Final Orig Val : Valid/Invalid/NotFound).

Output 12-37: Origin Validation state at router R3

```

A:admin@R3# show router bgp routes 172.16.0.0/20 ipv4 detail | match "DB Orig Val"
DB Orig Val      : NotFound          Final Orig Val : N/A
DB Orig Val      : NotFound          Final Orig Val : Valid

[/]
A:admin@R3# show router bgp routes 172.18.64.0/24 ipv4 detail | match "DB Orig Val"

```

```
DB Orig Val      : NotFound          Final Orig Val : N/A
DB Orig Val      : NotFound          Final Orig Val : Invalid

[/]
A:admin@R3# show router bgp routes 172.20.44.0/22 ipv4 detail | match "DB Orig Val"
DB Orig Val      : NotFound          Final Orig Val : N/A
DB Orig Val      : NotFound          Final Orig Val : NotFound
```

13 General Topics

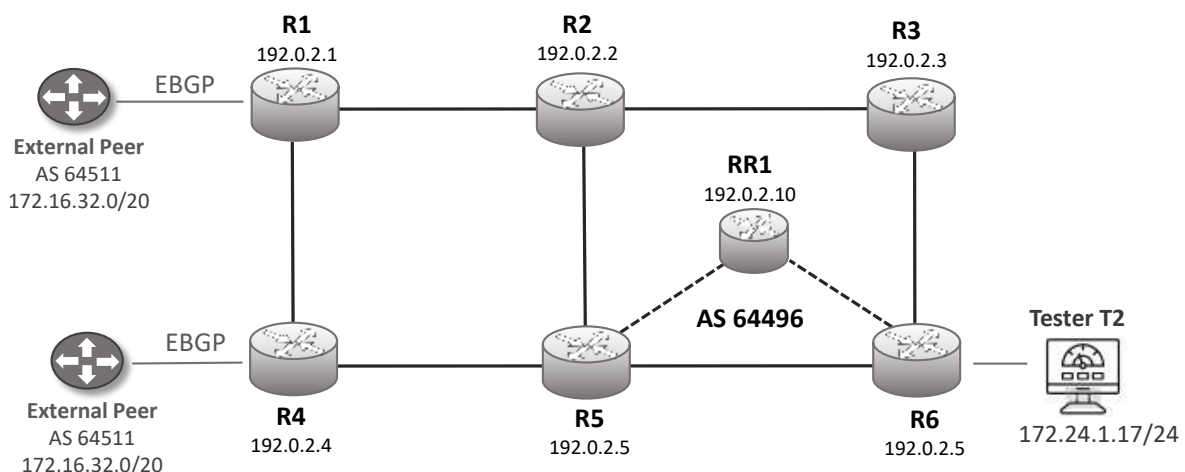
So far throughout this guide it has been relatively easy to categorise the applicability of BGP into chapter-length chunks. However, there are a number of applications for BGP that need to be described but do not warrant a dedicated chapter to describe their purpose and implementation. They are therefore covered within this general topics chapter.

Load-Balancing

Load-balancing provides the ability to maximise network capacity and aids fast reconvergence in a failure scenario because one or more backup path(s) are already installed in the FIB. To enable load-balancing there are three key features that require consideration: ECMP, IBGP-Multipath, and Multipath.

Throughout this section the topology shown in *Figure 13-1* will be used and where the topology is modified for a particular topic it will be described. Routers R1 to R6 and a Route-Reflector, RR1, are in a flat IS-IS Level 2 area. All routers belong to Autonomous System 64496 and Routers R1 to R6 peer in IBGP with RR1 as Route-Reflector clients to simplify BGP prefix advertisement. All IGP link metrics are 100 with the exception of the R2-R5 link which is metric 200. Segment Routing (SR-ISIS) is enabled within the domain, with IPv4 prefixes configured to use shortcut-tunnels with Next-Hops resolved to SR-ISIS. Routers R1 and R4 peer externally with AS 64511 and learn prefix 172.16.32.0/20 which is readvertised internally into AS 64496.

Figure 13-1: Load-Balancing and Multipath topology

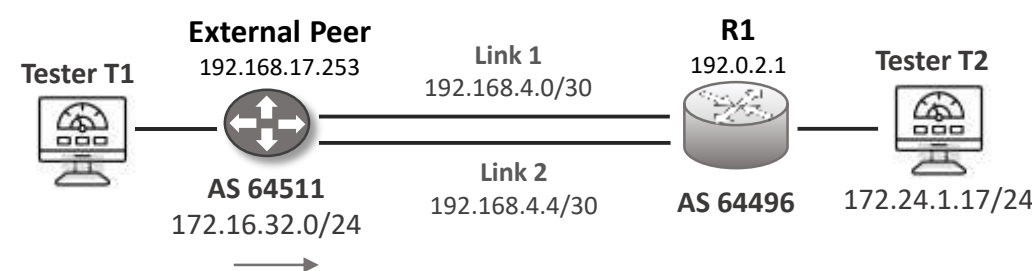


IBGP Multipath

In many topologies a BGP Next-Hop may be resolved by an IP route that has multiple ECMP next hops. In this scenario it is possible to use IBGP multipath to load-balance traffic across

those ECMP paths towards the Next-Hop. Consider the topology in *Figure 13-2*. Router R1 in AS 64496 is connected to an external peer at AS 64511 via two independent IP interfaces (i.e. they are not part of a LAG). R1 uses EBGP multihop to peer with AS 64511 and both peers use loopback addresses for that purpose, with R1 using 192.0.2.1/32 and the external peer using 192.168.17.253/32. To provide reachability to these loopback addresses, static route entries are configured via Link1 and Link2. Testers are connected to AS 64496 and AS 64511 to generate traffic for the purpose of validating load-balancing.

Figure 13-2: Test Topology for IBGP Multipath



With the default configuration router R1 has a FIB entry for the prefix 172.16.32.0/20 with a single next hop 192.168.4.2 via Link 1. Although two IGP with equivalent metrics are available, the router chooses the next hop with the lowest IP address because ECMP is not enabled.

Output 13-1: R1's FIB entry for prefix 172.16.32.0/20

```
A:admin@R1# show router fib 1 ip-prefix-prefix-length 172.16.32.0/20

=====
FIB Display
=====
Prefix [Flags]                                     Protocol
  NextHop
-----
172.16.32.0/20                                     BGP
  192.168.4.2 (Link1-to-AS64511)
-----
Total Entries : 1
=====
```

As the requirement is to load-balance traffic for the prefix 172.16.32.0/20 over both links, ECMP should initially be enabled at the global router level. Although the number of ECMP paths can be up to 64, the test topology dictates a requirement for only 2.

Output 13-2: ECMP configuration

```
router "Base" {
  ecmp 2
}
```

By enabling **ecmp 2** the loopback address (192.168.17.253/32) of the external peer in AS 64511 is programmed in the FIB with two entries for Link1 and Link2. Traffic for this address (which should be negligible other than BGP) will thereafter be load-balanced across both

links. However, as shown in *Output 13-4*, traffic for the prefix 172.16.32.0/20 advertised by AS 64511 still only resolves to Link1.

Output 13-3: R1's FIB entry for loopback address of external peer

```
A:admin@R1# show router fib 1 ip-prefix-prefix-length 192.168.17.253/32

=====
FIB Display
=====
Prefix [Flags]                                Protocol
  NextHop
-----
192.168.17.253/32                             STATIC
  192.168.4.2 (Link1-to-AS64511)
  192.168.4.6 (Link2-to-AS64511)
-----
Total Entries : 1
=====
```

Output 13-4: FIB entry for prefix 172.16.32.0/20 advertised by AS 64511

```
A:admin@R1# show router fib 1 ip-prefix-prefix-length 172.16.32.0/20

=====
FIB Display
=====
Prefix [Flags]                                Protocol
  NextHop
-----
172.16.32.0/20                                BGP
  192.168.4.2 (Link1-to-AS64511)
-----
Total Entries : 1
=====
```

Tester T2 in AS 64496 forwards traffic towards prefix 172.16.32.0/20 in AS 64511 and a monitor is placed on all ethernet ports at R1. The traffic uses 100 x source addresses from the 172.24.1.0/24 prefix, and 100 x destination addresses from the 172.16.32.0/20 prefix to generate sufficient entropy for load-balancing. *Output 13-5* shows a truncated version of the monitor output showing only the ports receiving or sending traffic. As the output shows, traffic is received on a single port (1/1/c1/1) and forwarded towards AS 64511 on a single port (1/1/c5/1).

Output 13-5: Monitor of relevant input/output ports at R1

```
A:admin@R1# monitor port all-ethernet-rates interval 3

=====
Monitor statistics for all Ethernet Port Rates
=====
Port-Id          D          Bits    Packets    Errors    Util
-----
1/1/c1/1         I          8228872    1003        0        0.08
                  O           2280        2         0        0.00
1/1/c5/1         I           552        1         0        0.00
```

O	8192552	1001	0	0.08
---	---------	------	---	------

To allow BGP to use all of the available links, IBGP multipath needs to be enabled. When enabled, the system attempts to use all of the available ECMP paths to the BGP Next-Hop. The term IBGP multipath is slightly misleading because it implies that it is only effective for IBGP. This is not the case, and it is equally applicable to EBGP as used in this illustration. IBGP multipath is enabled using the **ibgp-multipath true** command at the global BGP level either in the base routing instance or in an IP-VRF. (By default, **ibgp-multipath** is set to **true** – it was set to false above merely to illustrate the effect of having it enabled.)

Output 13-6: IBGP multipath configuration

```
router "Base" {
    bgp {
        ibgp-multipath true
    }
}
```

With **ibgp-multipath** set to **true** the FIB entry for prefix 172.16.32.0/20 advertised by AS 64511 now has Next Hops via Link1 and Link2.

Output 13-7: FIB entry for prefix 172.16.32.0/20 at R1

```
A:admin@R1# show router fib 1 ip-prefix-prefix-length 172.16.32.0/20
```

```
=====
FIB Display
=====
Prefix [Flags]                                Protocol
NextHop
-----
172.16.32.0/20                                BGP
  192.168.4.2 (Link1-to-AS64511)
  192.168.4.6 (Link2-to-AS64511)
-----
Total Entries : 1
=====
```

Traffic is again generated from Tester T2 in AS 64496 towards Tester T1 in AS 64511 using the same 100 x source/destination addresses to provide sufficient entropy. With IBGP multipath enabled, the traffic is reasonably well load-balanced. The ingress port (1/1/c1/1) receives 1000 packets per second and forwards 480 packets per second on port 1/1/c5/1 and 520 packets per second on port 1/1/c6/1.

Output 13-8: Input/Output ports at R1 with ECMP and IBGP Multipath enabled

```
A:admin@R1# monitor port all-ethernet-rates interval 3
```

```
=====
Monitor statistics for all Ethernet Port Rates
=====
Port-Id      D      Bits      Packets      Errors      Util
-----
1/1/c1/1     I      8222808     1002          0          0.08
              O      1752        2             0          0.00
1/1/c5/1     I          0          0             0          0.00
              O     3932160     480           0          0.04
```

1/1/c6/1	I	0	0	0	0.00
	O	4257112	520	0	0.04



Note: For load-balancing within the context of a VPRN, the IBGP-Multipath feature is applicable only to BGP peering across PE-CE interfaces. That is, if the BGP Next-Hop resolves to two or more equal-cost IGP Next-Hops, traffic can be load-balanced across those interfaces. Load-balancing of traffic between PE routers across the core is a function of ECMP at the global router level.

When using prefixes are advertised over an infrastructure that resolves BGP Next-Hops to LSPs/tunnels such as BGP shortcuts or labeled routes (VPN-IPv4/IPv6 or Label-IPv4/IPv6), **ibgp-multipath** does not have an impact on the load-balancing of traffic for that prefix. When a BGP Next-Hop is resolved to an LSP/tunnel that supports ECMP, the load-balancing of traffic through the tunnelled ECMP paths is automatic. For example, in *Figure 13-1* Tester T2 is connected to router R6 who learns prefix 172.16.32.0/20 from router R1 with a Next-Hop of 192.0.2.1. IPv4 BGP routes in this setup use shortcut-tunnels with Next-Hops resolved to SR-ISIS, and from R6's perspective there are two SR-ISIS ECMP paths towards router R1 (R6-R3-R2-R1 and R6-R5-R4-R1). Router R6 will therefore load-balance across those paths providing ECMP is enabled (greater than 1). This load-balancing is not a function of **ibgp-multipath**.

Multipath

Whereas IBGP multipath deals with the load-balancing of traffic to a prefix with a single BGP Next-Hop (with ECMP paths towards that Next-Hop), native multipath refers to the load-balancing of traffic towards the same prefix receive with multiple BGP Next-Hops. It's important to understand that paths considered eligible for multipath load-balancing must match the BGP best-path selection criteria up to and including step 13 (see *Table 0-1* in Appendix A for the list of best-path decision steps).

In *Figure 13-1* routers R1 and R4 learn prefix 172.16.32.0/20 from their respective external peers and reflect them internally into IBGP. ADD-PATH is used to ensure that the prefix from R1 and the prefix from R4 are both propagated by the Route-Reflector to other clients. Router R6 receives both of these paths to prefix 172.16.32.0/20 with different BGP Next-Hops, and the objective is to load-balance traffic between R1 and R4.

At the outset, router R6 receives the prefix 172.16.32.0/20 from R1 and R4 and prefers the route via R4 (Flags : Used Valid Best Group-Best In-RTM) because of the IGP cost to the BGP Next-Hop (TieBreakReason : NHCost). Only this route is installed in the route-table and FIB, with the route from R1 (192.0.2.1) held in RIB-In as valid. (The RIB-In modified attributes are not shown in *Output 13-9* for the sake of brevity because they are effectively the same as the original attributes.)

Output 13-9: BGP best-path selection at R6

```
A:admin@R6# show router bgp routes 172.16.32.0/20 detail | match
'Network|Nexthop|Flags|TieBreakReason'
Network       : 172.16.32.0/20
Nexthop       : 192.0.2.4
Res. Nexthop  : 192.0.2.4 (ISIS Tunnel)
Flags         : Used Valid Best Group-Best In-RTM
Network       : 172.16.32.0/20
Nexthop       : 192.0.2.1
Res. Nexthop  : 192.0.2.1 (ISIS Tunnel)
Flags         : Valid
TieBreakReason : NHCost
```

Equally, the FIB entry for prefix 172.16.32.0/20 shows a single entry with NextHop R4 (192.0.2.4). The BGP Next-Hop is resolved with SR-ISIS tunnel ID 524352.

Output 13-10: FIB entry for prefix 172.16.32.0/20 at R6

```
A:admin@R6# show router fib 1 ip-prefix-prefix-length 172.16.32.0/20
```

```
=====
FIB Display
=====
Prefix [Flags]                                Protocol
NextHop
-----
172.16.32.0/20                                BGP
  192.0.2.4 (Transport:SR-ISIS:524352)
-----
Total Entries : 1
=====
```

To enable load-balancing of traffic towards router R1 and router R4, the configuration shown in *Output 13-11* is applied at router R6 and begins in the **multipath** context by selecting the relevant Address Family, in this case **ipv4**. Other supported Address Families include **ipv6**, **label-ipv4**, and **label-ipv6** and allow for granular control of multipath at Address Family level. Within the **family** context there are options for configuring multipath for **ebgp** learnt routes and **ibgp** learnt routes, each keyword prefixed by an integer representing the maximum number of paths allowed. A **max-paths** command providing the same functionality of placing an upper boundary on the maximum number of allowed multipaths also exists at **multipath** level and at **family** level. Recall that paths that are eligible for multipath must have the same attributes up until step 13 of the path-selection process, including lowest IGP cost to the BGP Next-Hop. The **unequal-cost true** command is used to modify this behaviour and instructs the system to ignore the IGP metric for multipath eligible paths compared to the IGP metric to the best-path. In the example used, R6 learns the prefix 172.16.32.0/20 from R1 and R4 in IBGP and there are only 2 paths. As all link metrics are 100 (with the exception of the R2-R5 link which is 200), the IGP cost from R6 to R4 is 200, while the cost from R6 to R1 is 300. As a result, by default the path to R1 is not multipath eligible. The **unequal-cost true** changes that behaviour and ignores the IGP cost to the Next-Hop R1 such that the path it advertises becomes multipath eligible. When enabling **unequal-cost**, the use of tunnelling to the BGP Next-Hop is advised to avoid any possibility of a routing loop.

Output 13-11: Multipath configuration at R6

```

router "Base" {
    bgp {
        multipath {
            family ipv4 {
                ibgp 2
                unequal-cost true
            }
        }
    }
}

```

Output 13-12 shows the RIB-In at R6 for prefix 172.16.32.0/20 truncated to show only the relevant fields in the output. The first entry is from R4 (192.0.2.4) and shows as Used Valid Best Group-Best In-RTM and is resolved to SR-ISIS (ISIS Tunnel). The second entry is from router R1 (192.0.2.1) and shows as Used Valid Best In-RTM. As both routes are shown as best it indicates that both are used by multipath. The entry from R4 shows as Group-Best to indicate that it is the best path with the reason shown as shortest IGP cost to the Next-Hop (TieBreakReason : NHCost), but both paths are used and best.

Output 13-12: BGP paths for prefix 172.16.32.0/20 at R6 with Multipath

```

A:admin@R6# show router bgp routes 172.16.32.0/20 detail | match
'Network|NextHop|Flags|TieBreakReason'
Network      : 172.16.32.0/20
NextHop      : 192.0.2.4
Res. NextHop  : 192.0.2.4 (ISIS Tunnel)
Flags        : Used Valid Best Group-Best In-RTM

Network      : 172.16.32.0/20
NextHop      : 192.0.2.1
Res. NextHop  : 192.0.2.1 (ISIS Tunnel)
Flags        : Used Valid Best In-RTM
TieBreakReason : NHCost

```

Output 13-13 shows the FIB entry for prefix 172.16.32.0/20 at R6 and shows that there are two entries, R1 (192.0.2.1) and R4 (192.0.2.4) and both entries are resolved to SR-ISIS.

Output 13-13: FIB entry for prefix 172.16.32.0/20 at R6

```

A:admin@R6# show router fib 1 ip-prefix-prefix-length 172.16.32.0/20

=====
FIB Display
=====
Prefix [Flags]                                Protocol
NextHop
-----
172.16.32.0/20                                BGP
  192.0.2.1 (Transport:SR-ISIS:524391)
  192.0.2.4 (Transport:SR-ISIS:524352)
-----
Total Entries : 1
=====

```

Traffic is forwarded from Tester T2 (172.24.1.17/24) connected to R6 towards prefix 172.16.32.0/20 in AS 64511 at a rate of 1000 packets per second. To generate sufficient entropy for hashing of load-balanced traffic, 100 x source addresses and 100 x destination

addresses are used in the traffic pattern. To confirm that R6 load-balances the traffic, *Output 13-14* and *Output 13-15* show the results of monitoring the egress port(s) towards AS 64511 at routers R1 and R4 respectively. Router R1 has two external interfaces towards AS 64511, ports 1/1/c5/1 and 1/1/c6/1, which collectively forward at a rate of 500 packets per second. Router R4 has a single link that is also forwarding traffic towards AS 64511 at a rate of ~500 packets per second. Hence, the load-balancing is evenly distributed.

Output 13-14: Monitor port rate towards AS 64511 at router R1

```
A:admin@R1# monitor port 1/1/c5/1 port-string 1/1/c6/1 rate interval 3
```

```
=====
```

```
Monitor statistics for Ports
```

```
=====
```

	Input	Output

At time t = 3 sec (Mode: Rate)		

Port 1/1/c5/1		

Octets	0	245760
Packets	0	240
Errors	0	0
Bits	0	1966080
Utilization (% of port capacity)	0.00	0.02

Port 1/1/c6/1		

Octets	0	256000
Packets	0	250
Errors	0	0
Bits	0	2048000
Utilization (% of port capacity)	0.00	0.02

Output 13-15: Monitor port rate towards AS 64511 at router R4

```
A:admin@R4# monitor port 1/1/c3/1 rate interval 3
```

```
=====
```

```
Monitor statistics for Port 1/1/c3/1
```

```
=====
```

	Input	Output

At time t = 3 sec (Mode: Rate)		

Octets	120	522983
Packets	1	511
Errors	0	0
Bits	960	4183864
Utilization (% of port capacity)	~0.00	0.04

When using multipath the possibility exists to specify that a set of BGP neighbours in a peer group should be part of a selective multipath set that provides the user with finer granularity over the set of peers that can be installed as multipath paths. Selective multipaths are supported for the IPv4, IPv6, Label-IPv4, and Label-IPv6 Address Families and is enabled using the **multipath-eligible** command at either **neighbor** or **group** level. When enabled, if a

prefix is learned and the associated **max-paths** limit is N , then one of the following three options apply:

- If the best path for an IP prefix is received from a neighbour marked as **multipath-eligible**, then eliminate other RIB-In entries for the same prefix unless they also came from peers marked as **multipath-eligible**. From the remaining set of paths, choose the best N paths for installation as multipath routes.
- If the best route for an IP prefix was not received from a **multipath-eligible** peer, but there is at least one candidate multipath route for the same prefix from a **multipath-eligible** peer, multipath is not used.
- If no RIB-In exists from the set of all possible multipaths from a **multipath-eligible** peer, then choose the N best paths. This implements backward compatible behaviour.

The default configuration is that **multipath-eligible** is disabled.

Multipath with Weighted ECMP

When a BGP speaker receives a route from an external peer and advertises it to internal peers, it can be useful to attach an indication of the cost to reach that external peer to the Update. When used in conjunction with Multipath, this cost may be used by internal peers to influence the traffic load forwarded towards a given equal-cost destination prefix. For example, consider the topology in *Figure 13-1*. Router R1 and R4 are peered externally with router AS 64511. R4's external link is 10 Gigabit Ethernet, whereas R1 has 2 x 10 Gigabit Ethernet links. Router R1 and R4 both learn prefix 172.16.32.0/20 from AS 64511 which both advertise in IBGP. ADD-PATH is appended by R1 and R4 to ensure that routers internal to AS 64496 receive both available paths.

Router R6 is configured with ECMP and Multipath and will therefore load-balance traffic destined towards prefix 172.16.32.0/20 across both routers R1 and R4. With Multipath in use, **unequal-cost** is enabled at R6 due to the unequal cost IGP cost to the BGP Next-Hops. R6 also uses BGP shortcuts to resolve Next-Hops to SR-ISIS. Assume that R6 has 22.5Gb/s of traffic destined for prefix 172.16.32.0/20, then ~11.25Gb/s will be forwarded towards R1 and its external 2 x 10 Gigabit links towards AS 64511, and ~11.25Gb/s will be forwarded to R4 and its single 10 Gigabit Ethernet towards AS 64511. This means that ~1.25Gb/s of traffic will be dropped by R4 because the link is congested, despite the fact that there is spare capacity on the R1 external links. In this scenario, it would be useful if router R6 were made aware of the capacity of the external links towards AS 64511 and could use that knowledge to apply a weighting to its load-balancing mechanism. This is the purpose of the Link Bandwidth extended community.

The Link Bandwidth extended community [62] is an optional non-transitive attribute. It has a type of 0x40 and sub-type of 0x04, followed by a 2-byte Global Administrator field encoding the AS number of the router that attaches the extended community, followed by a 4-byte Local Administrator field that encodes the bandwidth in bytes (expressed in IEEE floating point format). There are two potential methods that can be used to attach a Link

Bandwidth extended community to a prefix. The first method is through use of the **link-bandwidth** context at **neighbor** or **group** level and is applied at router R4 as shown in *Output 13-16*. Once in this context, there are options for **accept-from-ebgp**, **add-to-received-ebgp**, **send-to-ebgp**, and **aggregate-used-paths**. The first option provides the ability to enable or disable the Bandwidth extended community applied to prefixes for a given family. The second and third options impact the direction on which the Bandwidth extended community is appended to prefixes. The fourth option, **aggregate-used-paths**, is used when multipath is enabled and aggregates the bandwidth of the used paths in the multipath set when setting Next-Hop-Self and re-advertising the prefix. In the example shown in *Output 13-16*, the Bandwidth extended community is added to received prefixes for the **ipv4** Address Family. Aside from IPv4, the supported address-families are IPv6, Label-IPv4/IPv6, and VPN-IPv4/IPv6. The AS number used in the extended community is the local AS number, and the bandwidth is the speed of the interface used for external peering. As router R4 uses a single 10 Gigabit Ethernet link and uses interface addresses for peering, it is straightforward for the system to determine the link bandwidth value to use. Where LAG is used the bandwidth is the speed of the combined member links but will not dynamically adjust if a link member fails.

Output 13-16: EBGp link bandwidth configuration at R4

```
router "Base" {
  bgp {
    group "EBGP-to-AS64511" {
      peer-as 64511
      family {
        ipv4 true
      }
    }
    neighbor "192.168.1.6" {
      group "EBGP-to-AS64511"
      link-bandwidth {
        add-to-received-ebgp {
          ipv4 true
        }
      }
    }
  }
}
```

When router R4 advertises the prefix 172.16.32.0/20 in IBGP towards RR1, it attaches the Link Bandwidth extended community containing its own AS number (64496) and the relevant bandwidth (shown in Mb/s).

Output 13-17: Link Bandwidth Extended Community appended by router R4

```
A:admin@R4# show router bgp routes 172.16.32.0/20 ipv4 hunt | match
'Network|Nexthop|Community'
Network      : 172.16.32.0/20
Nexthop      : 192.0.2.4
Res. Nexthop : n/a
Community    : bandwidth:64496:10000
```

Router R1 has two 10 Gigabit Ethernet links to AS 64511. The two links do not form part of a LAG, but rather R1 uses multihop EBGp for the peering session which terminates on

loopback addresses directly resolved to the two IP interfaces. As a result, the system is unable to determine the link bandwidth from a directly connected interface and therefore route-policy is used to attach the Link Bandwidth extended community. *Output 13-18* shows the relevant configuration applied at R1. A community list “AS64511-Link-Bandwidth” is configured containing the parameters of the Link Bandwidth extended community, where **bandwidth** is a keyword to represent the Link Bandwidth extended community, **64496** is the local AS number, and **20000** represents the two 10 Gigabit Ethernet links in Mb/s. The community is then added as a **default-action** in the **policy-statement** “prepend-bandwidth”. The **policy-statement** is then applied as an import policy to the AS 64511 neighbour configuration.

The route-policy method can be implemented in a base router BGP import policy or in a VRF import policy to add a Link Bandwidth extended community to prefixes received from IBGP or EBGp peers. In addition, a base router BGP export policy can be used to add a Link Bandwidth extended community to prefixes received from IBGP and EBGp peers.

Output 13-18: EBGp link bandwidth configuration at R1

```

policy-options {
  community "AS64511-Link-Bandwidth" {
    member "bandwidth:64496:20000" { }
  }
  policy-statement "prepend-bandwidth" {
    default-action {
      action-type accept
      community {
        add ["AS64511-Link-Bandwidth"]
      }
    }
  }
}
router "Base" {
  bgp {
    group "EBGP-to-AS64511" {
      peer-as 64511
      family {
        ipv4 true
      }
    }
    neighbor "192.168.1.6" {
      group "EBGP-to-AS64511"
      import {
        policy ["prepend-bandwidth"]
      }
    }
  }
}

```

When router R1 advertises the prefix 172.16.32.0/20 in IBGP towards RR1, it attaches the relevant Link Bandwidth extended community. A caveat of this approach is that should one of the links towards AS 64511 the Link Bandwidth value will remain unchanged.

Output 13-19: Link bandwidth Extended Community appended by R1

```

A:admin@R1# show router bgp routes 172.16.32.0/20 ipv4 hunt | match
Network      : 172.16.32.0/20
NextHop      : 192.0.2.1

```

```
Res. Nexthop      : n/a
Community         : bandwidth:64496:20000
```

At router R6, the prefix 172.16.32.0/20 is received from R4 (192.0.2.4) with a Link Bandwidth extended community of 64496:10000 and from router R1 (192.0.2.1) with a Link Bandwidth extended community of 64496:20000. When BGP paths are received with Link Bandwidth Extended Communities, SR-OS uses the following rules:

- If BGP Multipath and ECMP are configured and all of the eligible multipaths have a Link Bandwidth extended community, then weighted ECMP should be performed on the relative bandwidth of each path.
- If EIBGP Multipath and ECMP are enabled in a VPRN, and all of the eligible Next-Hops have a Link Bandwidth extended community then weighted ECMP should be performed based on the relative bandwidth of each path.
- The Link Bandwidth community is not used as a criteria for two or more paths to be considered equal for BGP/EIBGP Multipath purposes.

Output 13-20: Link Bandwidth Extended Communities received at router R6

```
A:admin@R6# show router bgp routes 172.16.32.0/20 detail | match
'Network|Nexthop|Community'
Network      : 172.16.32.0/20
Nexthop      : 192.0.2.4
Res. Nexthop : 192.0.2.4 (ISIS Tunnel)
Community    : bandwidth:64496:10000
Network      : 172.16.32.0/20
Nexthop      : 192.0.2.1
Res. Nexthop : 192.0.2.1 (ISIS Tunnel)
Community    : bandwidth:64496:20000
```

Router R6 is configured with **multipath** and **ecmp**, both set to a value of 2, and, as both paths from R1 and R4 have a Link Bandwidth extended community, weighted ECMP should be performed. *Output 13-21* shows the FIB entry for prefix 172.16.32.0/20 at R6. The Next-Hop entry of 192.0.2.1 (R1) has an ECMP-Weight of 2 associated with it representing the 20Gb/s of link capacity between R1 and AS 64511, whereas the Next-Hop entry of 192.0.2.4 (R4) has an ECMP-Weight of 1 associated with it representing the 10 Gb/s link between R4 and AS 64511. The result is that for traffic flows from R6 to destination prefix 172.16.32.0/20, 66% will be forwarded towards R1, and 33% will be forwarded towards R4.

Output 13-21: FIB entry for prefix 172.16.32.0/20 with Weighted ECMP at router R6

```
A:admin@R6# show router fib 1 ip-prefix-prefix-length 172.16.32.0/20 extensive
=====
FIB Display (Router: Base)
=====
Dest Prefix      : 172.16.32.0/20
Protocol         : BGP
Installed        : Y
Indirect Next-Hop : 192.0.2.1
  QoS            : Priority=n/c, FC=n/c
  Source-Class   : 0
  Dest-Class     : 0
  ECMP-Weight    : 2
  Resolving Next-Hop : 192.0.2.1 (SR-ISIS tunnel:524391)
  ECMP-Weight    : 1
Indirect Next-Hop : 192.0.2.4
```

```

QoS                : Priority=n/c, FC=n/c
Source-Class       : 0
Dest-Class         : 0
ECMP-Weight        : 1
Resolving Next-Hop : 192.0.2.4 (SR-ISIS tunnel:524352)
ECMP-Weight        : 1

```

```

=====
Total Entries : 1
=====

```

Traffic is forwarded from Tester T2 (172.24.1.17/24) connected to R6 towards prefix 172.16.32.0/20 in AS 64511 at a rate of 1000 packets per second. To generate sufficient entropy for hashing of load-balanced traffic, 100 x source addresses and 100 x destination addresses are used in the traffic profile. To confirm that R6 load-balances the traffic, *Output 13-22* and *Output 13-23* show the results of monitoring the egress port(s) towards AS 64511 at routers R1 and R4 respectively. Router R1 shown in *Output 13-22* has two external interfaces towards AS 64511 which collectively forward at a rate of ~640 packets per second. Router R4 has a single interface towards AS 64511 that forwards traffic at a rate of ~360 packets per second. It can therefore be concluded that traffic is being forwarded on a weighted 2:1 ratio.

Output 13-22: Monitor port rate towards AS 64511 at router R1

```
A:admin@R1# monitor port 1/1/c5/1 port-string 1/1/c6/1 rate interval 3
```

```
=====
Monitor statistics for Ports
=====
```

	Input	Output

At time t = 3 sec (Mode: Rate)		

Port 1/1/c5/1		

Octets	0	307200
Packets	0	300
Errors	0	0
Bits	0	2457600
Utilization (% of port capacity)	0.00	0.02

Port 1/1/c6/1		

Octets	0	347819
Packets	0	340
Errors	0	0
Bits	0	2782552
Utilization (% of port capacity)	0.00	0.02

Output 13-23: Monitor port rate towards AS 64511 at router R4

```
A:admin@R4# monitor port 1/1/c3/1 rate interval 3
```

```
=====
Monitor statistics for Port 1/1/c3/1
=====
```

	Input	Output

At time t = 3 sec (Mode: Rate)

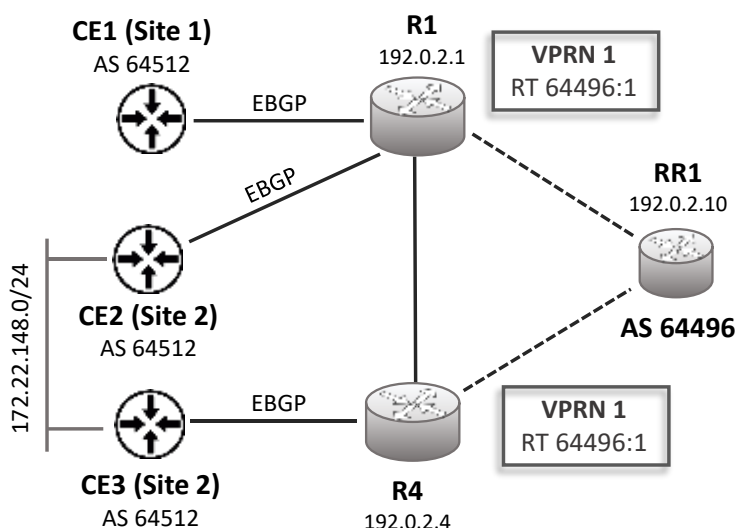
Octets	185	368663
Packets	2	360
Errors	0	0
Bits	1480	2949304
Utilization (% of port capacity)	~0.00	0.03

EIBGP Multipath

EIBGP multipath is an extension of multipath that allows for load-balancing amongst paths that a router learns from both EBGP and IBGP. It is intended for use within a VPRN context, and from the perspective of the PE router the use of EIBGP multipath dictates the ability to load-balance to the same prefix learnt through IPv4 BGP from a direct next-hop (CE router) and VPN-IPv4 BGP from a remote next-hop resolved through an MPLS LSP.

Figure 13-3 shows a use-case for EIBGP multipath. R1 and R4 are PE routers connected to a BGP/MPLS IP-VPN backbone and peer in IBGP with RR1 within AS 64496. CE2 and CE3 represent a dual-homed site (Site 2) where CE2 is connected to R1 and CE3 is connected to R4. Both CE2 and CE3 advertise the prefix 172.22.148.0/24 to their respective PEs in EBGP. CE1 is a single-homed site (Site 1) that is also connected to R1. The objective would be for R1 to load-balance traffic to 172.22.148.0/32 over the direct connection to CE2 and also via R4's direct connection to CE3 to maximise the use of the capacity on the PE to CE connections.

Figure 13-3: EIBGP Multipath



R1 learns the prefix 172.22.148.0/24 from CE2 in IPv4 EBGP and from R4 (via CE3) in VPN-IPv4 IBGP and by default installs the route learnt from CE2 as a result of rule 13 of the BGP path selection algorithm: Prefer routes learnt from an EBGP peer over routes learnt from an IBGP peer.

Output 13-24: VRF route-table entry for prefix 172.22.148.0/24 without EIBGP multipath

```
A:admin@R1# show router 1 route-table 172.22.148.0/24
```

```
=====
```

```
Route Table (Service: 1)
```

```
=====
```

Dest Prefix[Flags]	Type	Proto	Age	Pref
Next Hop[Interface Name]			Metric	
172.22.148.0/24	Remote	BGP	00h17m02s	170
192.168.100.18			0	

```
-----
```

```
No. of Routes: 1
```

```
Flags: n = Number of times nexthop is repeated
```

```
      B = BGP backup route available
```

```
      L = LFA nexthop available
```

```
      S = Sticky ECMP requested
```

```
=====
```

To enable load-balancing over EBGP and IBGP learned paths, the VPRN must be configured for **ecmp** >1 at the service level, **multipath** should be enabled, and the **bgp** instance must have **eibgp-loadbalance** set to **true**.

Output 13-25: EIBGP configuration at R1

```
service {
    vprn "one" {
        ecmp 4
        bgp {
            eibgp-loadbalance true
            multipath {
                max-paths 4
                ebgp 2
                ibgp 2
                unequal-cost true
                family ipv4 {
                    max-paths 4
                }
            }
        }
    }
}
```

After enabling **eibgp-multipath**, a check of the route-table for the VPRN at R1 shows that both the EBGP learnt route and the IBGP learnt route are installed in the route-table as shown in *Output 13-26*. As a result, traffic from CE1 toward the prefix 172.22.148.0/32 will be load-balanced over both PE-CE connections to Site 2.

Output 13-26: EIBGP load-balancing route-table entry for 172.22.148.0/24 at R1

```
A:admin@R1# show router 1 route-table 172.22.148.0/24
```

```
=====
```

```
Route Table (Service: 1)
```

```
=====
```

Dest Prefix[Flags]	Type	Proto	Age	Pref
Next Hop[Interface Name]			Metric	
172.22.148.0/24	Remote	BGP VPN	00h09m15s	170
192.0.2.4 (tunneled:SR-ISIS:524296)			100	
172.22.148.0/24	Remote	BGP	00h09m15s	170

192.168.100.18	0

No. of Routes: 1	
Flags: n = Number of times nexthop is repeated	
B = BGP backup route available	
L = LFA nexthop available	
S = Sticky ECMP requested	
=====	

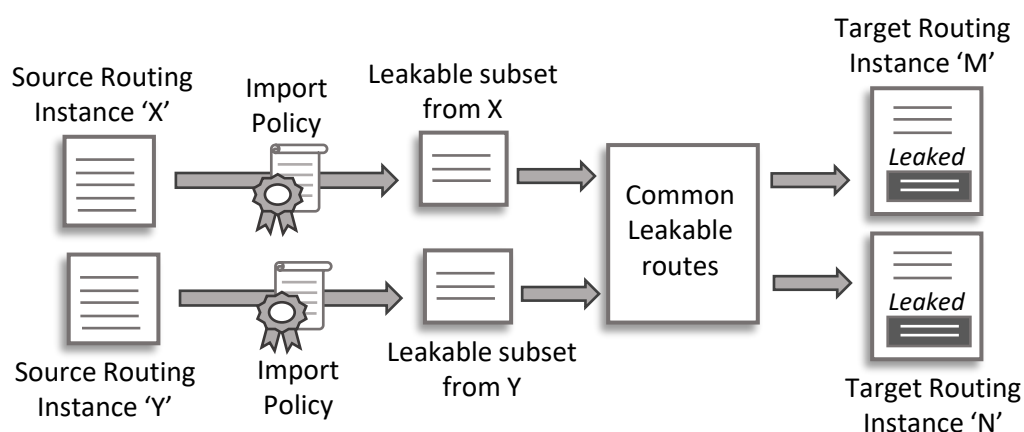
BGP RIB Leaking



The BGP RIB leaking feature is not support on the 7250 IXR Gen 1 and Gen 2 platforms.

Operators are familiar with the process of exporting BGP routes from one IP-VRF to another IP-VRF on the same system using VRF export policies, allowing for a form of ‘overlapping’ VRFs, or extranet functionality. While this approach provides visibility of exported routes between IP-VRF instances, it has drawbacks. Firstly, although the IP-VRF that imports the route shows the route as learned through BGP-VPN, any BGP attributes associated with exported route are lost. Secondly, and rather more obviously, this approach does not deal with the leaking of routes between the base routing instance and an IP-VRF. The BGP RIB leaking feature aims to resolve both of these issues by providing a more controlled and granular way to leak routes between routing instances. A BGP route (or routes) that is designated to be leaked into another routing instance is marked as *Leakable* upon import to the RIB-In. Once marked as Leakable, the prefix joins a common set of other Leakable prefixes that can be imported into other routing instances using RIB management import policies.

Figure 13-4: BGP RIB Leaking process



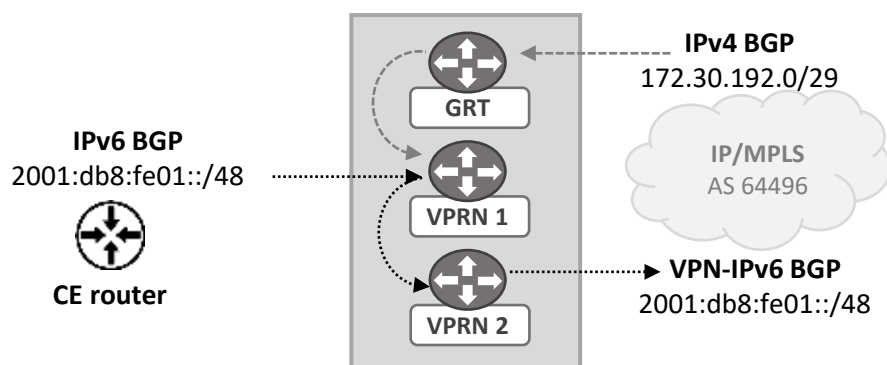
Whenever BGP routes are leaked from a source to a target routing instance, all BGP attributes remain intact with the leaked route. Once a route is leaked into the target routing instance, it can be retained locally in that routing instance, or it can be re-advertised to BGP peers of that instance or even exported as a VPN-IPv4/IPv6 route. When a leaked route is re-

advertised, the BGP Next-Hop is automatically reset to whatever Next-Hop is applicable. Route leaking can be implemented for IPv4, IPv6 BGP, and Label-IPv4/IPv6 routes as long as the route is considered valid. A BGP route does not have to be the best path or be used for forwarding in the source routing instance to be leaked. Route leaking can be implemented between routing instances as follows:

- GRT to VRF
- VRF to VRF
- VRF to GRT

To provide two examples of BGP RIB leaking, the schematic in *Figure 13-5* is used. In the first example, IPv4 prefix 172.30.192.0/29 is known by router R1 in the base routing instance, or global routing table (GRT), and will be leaked into IP-VRF “one” where it will be retained locally. In the second example, IPv6 prefix 2001:db8:fe01::/48 is learnt by IP-VRF “one” from an attached CE router and will be leaked into IP-VRF “two”. From IP-VRF “two” the prefix 2001:db8:fe01::/48 will be re-advertised into VPN-IPv6.

Figure 13-5: BGP RIB Leaking example



An IPv4 or IPv6 BGP route becomes a candidate for leaking when it is specifically marked by a BGP import policy. For the first example, IPv4 prefix 172.30.192.0/29 is being learned by router R1 from a Route-Reflector at 192.0.2.10. *Output 13-27* shows the necessary configuration to mark this prefix (and only this prefix) as a candidate for BGP RIB leaking. The **policy-statement** “GRT-to-VRF-LEAK” explicitly references a **prefix-list** and **protocol name** of **bgp** to identify the IPv4 route in question. The **action-type** is configured as **accept**, followed by setting the **bgp-leak** command to **true**. The policy is thereafter applied as an import policy on R1’s IBGP peering session to the Route-Reflector at address 192.0.2.10.

Output 13-27: Marking BGP paths as candidate for leaking

```

policy-options {
  prefix-list "GRT-LEAKABLE" {
    prefix 172.30.192.0/29 type exact {
    }
  }
  policy-statement "GRT-to-VRF-LEAK" {
    entry 10 {
      from {
        prefix-list ["GRT-LEAKABLE"]
        protocol {

```

```

        name [bgp]
    }
}
    action {
        action-type accept
        bgp-leak true
    }
}
    default-action {
        action-type accept
    }
}
}
router "Base" {
    bgp {
        group "IBGP-CORE" {
            peer-as 64496
            family {
                ipv4 true
                ...etc
            }
        }
    }
    neighbor "192.0.2.10" {
        group "IBGP-CORE"
        import {
            policy ["GRT-to-VRF-LEAK"]
        }
    }
}
}

```

Output 13-28 shows the RIB entry for prefix 172.30.192.0/29 after the policy is applied as an import policy to the RIB-In. Note that the `Flags` field shows the route marked as `Leakable`. Also note that in addition to the standard mandatory BGP attributes a community value of 64496:2249 is carried with the prefix. The intention is to ensure this community value remains with the prefix after being leaked.

Output 13-28: RIB entry for prefix 172.30.192.0/29 with Leakable Flag

```

A:admin@R1# show router bgp routes 172.30.192.0/29 detail
=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                l - leaked, x - stale, > - best, b - backup, p - purge,
                w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP IPv4 Routes
=====
Original Attributes

Network       : 172.30.192.0/29
Nextthop      : 192.0.2.4
Path Id       : 7077
From          : 192.0.2.10
Res. Protocol : SR-ISIS           Res. Metric      : 100
Res. Nextthop : 192.0.2.4 (ISIS Tunnel)
--- [snip] ---
Community     : 64496:2249
Cluster       : 192.0.2.10

```

```

Originator Id   : 192.0.2.4           Peer Router Id : 192.0.2.10
Fwd Class       : None                Priority         : None
Origin          : IGP
Flags           : Used Valid Best Group-Best Leakable In-RTM
Route Source    : Internal
AS-Path         : No As-Path
--- [snip] ---

```

Once the prefix is marked as Leakable, any other routing instance can import it as a Leaked route, and the target routing instance in this example is routing instance ‘1’, or VPRN “one”. *Output 13-29* shows the configuration applied at VPRN “one” to import the prefix 172.30.192.0/29. Once again, the **policy-statement** “VRF1-IMPORT-LEAKABLE-ROUTES” explicitly references a **prefix-list** containing the target prefix and the **protocol name** of **bgp**. Thereafter the **policy-statement** simply has an **action-type** of **accept**. Within the VPRN instance, the **rib-management** context is used to identify the Address Family as **ipv4**, after which the **leak-import** command is used to reference the previously configured route policy.

Output 13-29: RIB-Management import for VPRN “one”

```

policy-options {
  prefix-list "GRT-to-VRF1-LEAKABLE-PREFIXES" {
    prefix 172.30.192.0/29 type exact {
    }
  }
  policy-statement "VRF1-IMPORT-LEAKABLE-ROUTES" {
    entry 10 {
      from {
        prefix-list ["GRT-to-VRF1-LEAKABLE-PREFIXES"]
        protocol {
          name [bgp]
        }
      }
      action {
        action-type accept
      }
    }
  }
}
service {
  vprn "one" {
    bgp {
      rib-management {
        ipv4 {
          leak-import {
            policy ["VRF1-IMPORT-LEAKABLE-ROUTES"]
          }
        }
      }
    }
  }
}
}

```

With the **leak-import** policy applied to the **rib-management** context, the prefix is made visible in the route-table for IP-VRF “one”, or routing instance ‘1’. The protocol owner is still BGP, and the Next-Hop and resolving tunnel ID is retained with the leaked route.

Output 13-30: IP-VRF “one” route-table entry for 172.30.192.0/29

```

A:admin@R1# show router 1 route-table 172.30.192.0/29

```

```

=====

```

```

Route Table (Service: 1)
=====
Dest Prefix[Flags]                                Type   Proto   Age           Pref
  Next Hop[Interface Name]                        Metric
-----
172.30.192.0/29                                Remote BGP      00h10m46s  170
      192.0.2.4 (tunneled:SR-ISIS:524296)                100
-----
No. of Routes: 1
Flags: n = Number of times nexthop is repeated
      B = BGP backup route available
      L = LFA nexthop available
      S = Sticky ECMP requested
=====

```

Within the target routing instance, the leaked prefix can be displayed as a BGP route. The first thing to note is that the `Flags` field shows the route as `Leaked`. The `NextHop` field retains the Next-Hop attribute received by R1 as 192.0.2.4 (Base) with the `From` field value of `BGP Base` also indicating that this route was imported from the global routing table. Other attributes such as the `Community`, `Cluster ID`, and `Originator ID` are all retained with the `Leaked` route.

Output 13-31: BGP prefix 172.30.192.0/29 in the target routing instance

```

A:admin@R1# show router 1 bgp routes 172.30.192.0/29 detail
=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete

=====
BGP IPv4 Routes
=====
Original Attributes

Network       : 172.30.192.0/29
Nexthop       : 192.0.2.4 (Base)
Path Id       : None
From          : BGP Base
Res. Protocol : SR-ISIS           Res. Metric    : 100
Res. Nexthop  : 192.0.2.4 (ISIS Tunnel)
--- [snip] ---
Community     : 64496:2249
Cluster       : 192.0.2.10
Originator Id : 192.0.2.4         Peer Router Id : 0.0.0.0
Fwd Class     : None             Priority       : None
Origin        : IGP
Flags         : Used Valid Best Leaked In-RTM
Route Source  : Leaked from Base
AS-Path       : No As-Path
Route Tag     : 0
--- [snip] ---

```

In the second example, prefix 2001:db8:fe01::/48 is learnt by IP-VRF “one” (routing instance 1) and will be Leaked into IP-VRF “two” (routing instance 2). *Output 13-32* shows the necessary configuration to identify this prefix (and only this prefix) as a candidate for BGP RIB leaking. The **policy-statement** “*VRF1-LEAKABLE-PREFIXES*” explicitly references a

prefix-list and **protocol name** of **bgp** to identify the IPv6 route. The **action-type** is configured as **accept**, followed by setting the **bgp-leak** command to **true**. The policy is thereafter applied as an import policy towards the CE router neighbor at 2001:db8:3001:2101::2 who is advertising the IPv6 prefix.

Output 13-32: RIB Leaking configuration in VPRN "one"

```

policy-options {
  prefix-list "VRF1-LEAKABLE-PREFIXES" {
    prefix 2001:db8:fe01::/48 type exact {
    }
  }
  policy-statement "VRF1-LEAKABLE-PREFIXES" {
    entry 10 {
      from {
        prefix-list ["VRF1-LEAKABLE-PREFIXES"]
        protocol {
          name [bgp]
        }
      }
      action {
        action-type accept
        bgp-leak true
      }
    }
  }
}
service {
  vprn "one" {
    autonomous-system 64496
    bgp {
      group "CE-EBGP" {
        family {
          ipv6 true
        }
      }
      neighbor "2001:db8:3001:2101::2" {
        group "CE-EBGP"
        import {
          policy ["VRF1-LEAKABLE-PREFIXES"]
        }
      }
    }
  }
}

```

Output 13-33 shows the truncated RIB entry for prefix 2001:db8:fe01::/48 after the policy is applied as an import policy to the BGP neighbour in IP-VRF “one”. As a result of the import route-policy, the **Flags** field identifies the route as **Leakable**. The route source is external, and the **Nextthop** is the IPv6 address of the adjacent CE interface “link-to-CE1”.

Output 13-33: IP-VRF "one" BGP RIB entry for prefix 2001:db8:fe01::/48

```

A:admin@R1# show router 1 bgp routes 2001:db8:fe01::/48 ipv6 detail
=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                l - leaked, x - stale, > - best, b - backup, p - purge,
                w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete

```



```

=====
BGP IPv6 Routes
=====
Original Attributes

Network       : 2001:db8:fe01::/48
Nexthop       : 2001:db8:3001:2101::2
Path Id       : None
From          : 2001:db8:3001:2101::2
Res. Protocol : LOCAL           Res. Metric    : 0
Res. Nexthop  : 2001:db8:3001:2101::2
Local Pref.   : n/a             Interface Name : link-to-CE1
--- [snip] ---
Origin        : IGP
Flags         : Used Valid Best Leakable In-RTM
Route Source  : External
AS-Path       : 64512
Route Tag     : 0
Neighbor-AS   : 64512
--- [snip] ---

```

Prefix 2001:db8:fe01::/48 now belongs to a common set of Leakable routes that can be imported into any target routing instance. In this example, the target routing instance is IP-VRF “two”. *Output 13-34* shows the configuration applied at VPRN “two” to import the prefix 2001:db8:fe01::/48. The **policy-statement** “VRF1-to-VRF2-LEAKABLE-PREFIXES” explicitly references a **prefix-list** containing the target prefix and the **protocol name** of **bgp**. Following this the **policy-statement** has an **action-type** of **accept**. Within the VPRN instance, the **rib-management** context selects the **ipv6** Address Family and uses the **leak-import** command to reference the route policy.

Output 13-34: RIB management import for IP-VRF “two”

```

policy-options {
  prefix-list "VRF1-to-VRF2-LEAKABLE-PREFIXES" {
    prefix 2001:db8:fe01::/48 type exact {
    }
  }
  policy-statement "VRF1-to-VRF2-LEAKABLE-PREFIXES" {
    entry 10 {
      from {
        prefix-list ["VRF1-to-VRF2-LEAKABLE-PREFIXES"]
        protocol {
          name [bgp]
        }
      }
      action {
        action-type accept
      }
    }
  }
}
service {
  vprn "two" {
    autonomous-system 64520
    bgp {
      admin-state enable
      rib-management {
        ipv6 {
          leak-import {
            policy ["VRF1-to-VRF2-LEAKABLE-PREFIXES"]
          }
        }
      }
    }
  }
}

```

```

    }
  }
}

```

The imported prefix is now visible in the IPv6 route-table of IP-VRF “two”, or routing instance 2. The protocol owner is BGP, and the Next-Hop is the IPv6 address of the adjacent CE router in VPRN “one”. This emphasizes the point that all path attributes, including the Next-Hop attribute, are retained with the Leaked prefix.

Output 13-35: VPRN “two” IPv6 route-table entry for leaked prefix

```

A:admin@R1# show router 2 route-table ipv6 2001:db8:fe01::/48

=====
IPv6 Route Table (Service: 2)
=====
Dest Prefix[Flags]                                Type    Proto    Age          Pref
  Next Hop[Interface Name]                        Metric
-----
2001:db8:fe01::/48                                Remote  BGP       01h55m26s   170
  2001:db8:3001:2101::2                            0
-----
No. of Routes: 1
Flags: n = Number of times nexthop is repeated
      B = BGP backup route available
      L = LFA nexthop available
      S = Sticky ECMP requested
=====

```

As with the first example, within the target routing instance, the leaked prefix can be displayed as a BGP route. As before, the `Flags` field shows the route as a `Leaked` route. There are multiple indications of the source routing instance:

- The `NextHop` field retains the Next-Hop attribute value of 2001:db8:3001:2101::2 received from the adjacent CE router in VPRN 1 together with an indication of the source routing instance (VPRN 1).
- The `From` field provides an indication of the source routing instance of BGP VPRN 1.
- The `Route Source` also indicates the source routing instance and shows `Leaked` from VPRN 1.

Other BGP attributes such as the `AS_PATH` and `Origin` are again retained with the `Leaked` route.

Output 13-36: BGP prefix 2001:db8:fe01::/48 in the target routing instance

```

A:admin@R1# show router 2 bgp routes 2001:db8:fe01::/48 ipv6 detail

=====
BGP Router ID:192.0.2.1      AS:64520      Local AS:64520
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete

=====
BGP IPv6 Routes
=====

```

Original Attributes

```

Network      : 2001:db8:fe01::/48
Nexthop      : 2001:db8:3001:2101::2 (VPRN 1)
Path Id      : None
From         : BGP VPRN 1
Res. Protocol : LOCAL                      Res. Metric      : 0
Res. Nexthop  : 2001:db8:3001:2101::2
Local Pref.   : 100                      Interface Name : link-to-CE1
--- [snip] ---
Origin        : IGP
Flags         : Used Valid Best Leaked In-RTM
Route Source  : Leaked from VPRN 1
AS-Path       : 64512
Route Tag     : 0
Neighbor-AS   : 64512
--- [snip] ---

```

The final output shows the relevant fields of the Leaked prefix being re-advertised by VPRN “two” into base BGP VPN-IPv6. The Route-Distinguisher and Route-Target for VPRN “two” are 64496:2. The former can be seen in the `Route Dist.` field, and the latter can be seen in the `Community` field expressed as `target:64496:2`. The `Nexthop` of `::ffff:192.0.2.1` is an IPv4-mapped IPv6 address used when advertising IPv6 prefixes over an IPv4 backbone.

Output 13-37: VPN-IPv6 RIB-Out entry for prefix 2001:db8:fe01::/48 in VPRN “two”

```

A:admin@R1# show router bgp routes 2001:db8:fe01::/48 vpn-ipv6 hunt rd 64496:2 | match
'Network|Nexthop|Route Dist.|Community'
Network      : 2001:db8:fe01::/48
Nexthop      : ::ffff:192.0.2.1
Route Dist.   : 64496:2                VPN Label      : 524274
Res. Nexthop  : n/a
Community     : target:64496:2

```

BGP Shortcuts

With conventional IP forwarding every router in the forwarding path makes a routing decision, which in turn implies that every router must have adequate routing information available in both its RIB and FIB. In the case of Internet routing, it means that every router in the forwarding path must hold the Internet routing table. Although this is acceptable to some providers, others prefer that the core is “BGP-free”.

BGP shortcuts provide a mechanism for a forwarding router to resolve the BGP Next-Hop of a given prefix to an MPLS LSP. When the traffic has been encapsulated inside the LSP, any subsequent transit routers only perform a label-swap operation on any packets. When the destination router pops the label stack it can execute an IP routing decision before forwarding the traffic to the appropriate external peer. Essentially the BGP shortcut provides an MPLS tunnel between ingress router and egress router allowing transit routers to be BGP-free.

In *Figure 13-6*, routers R1 and R6 are IBGP clients of Route-Reflector RR1. R1 advertises the prefix 172.16.32.0/20 with a Next-Hop of its system address 192.0.2.1. All physical

adjacencies in the topology are configured to support both SR-ISIS and RSVP control planes, and all link metrics are 100. For the MPLS data-plane, router R6 has two equal-cost SR-ISIS LSPs to R1 and a traffic-engineered RSVP path.

Output 13-38: R6's tunnel-table entries for R1

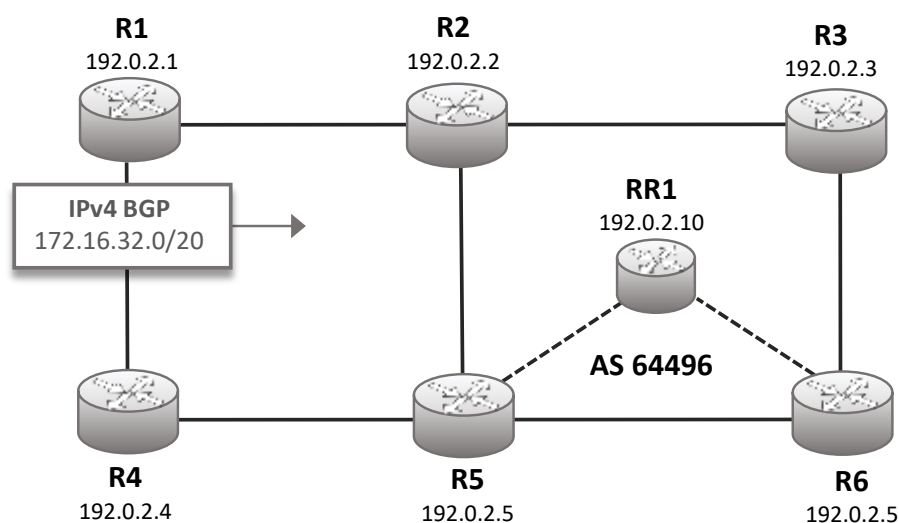
```
A:admin@R6# show router tunnel-table 192.0.2.1/32
```

```
=====
IPv4 Tunnel Table (Router: Base)
=====
```

Destination Color	Owner	Encap	TunnelId	Pref	Nexthop	Metric
192.0.2.1/32	rsvp	MPLS	1	7	192.168.0.17	300
192.0.2.1/32	isis (0)	MPLS	524391	11	192.168.0.17	300
192.0.2.1/32	isis (0)	MPLS	524391	11	192.168.0.25	300

```
-----
Flags: B = BGP or MPLS backup hop available
       L = Loop-Free Alternate (LFA) hop available
       E = Inactive best-external BGP route
       k = RIB-API or Forwarding Policy backup hop
=====
```

Figure 13-6: IGP shortcuts



BGP shortcuts is enabled using the **next-hop-resolution shortcut-tunnel** context. Within this context, the **family** command is used to enable different tunnel selection criteria for IPv4 and IPv6 routes, and when entered opens a new context for the selected family, **ipv4** or **ipv6**. Within the **family** context, the **resolution** command has options for **filter** or **any**. When **any** is selected, the system does a lookup in the tunnel-table and selects the tunnelling mechanism that is the most preferred (lowest numeric value). When **filter** is selected, the **resolution-filter** context is used to filter tunnelling protocols to only those that are permitted. In this initial example, **sr-isis** is the only selected tunnelling mechanism. In addition to the **shortcut-tunnel**, **ecmp 2** and **ibgp-multipath** in order to be able to load-balance traffic over the available paths towards R1.

Output 13-39: Configuration for BGP shortcuts at R6

```

router "Base" {
  bgp {
    next-hop-resolution {
      shortcut-tunnel {
        family ipv4 {
          resolution filter
          resolution-filter {
            sr-isis true
          }
        }
      }
    }
  }
}

```

With shortcuts enabled for **sr-isis**, *Output 13-40* shows the LFIB entries for the BGP Next-Hop 192.0.2.1/32 (R1). As there are two ECMP paths towards R1 from R6, there are two entries, both using an MPLS label (SR-MPLS SID) of 14001 representing R1's Node-SID. The first entry has a NextHop of 192.168.0.17 (link-to-R5) with egress interface 1/1/c2/1:100 and the second entry has a NextHop of 192.168.0.25 (link-to-R3) with egress interface 1/1/c1/1:100. Although BGP itself will only use one of these Next-Hops for control plane traffic (the interface with the lowest IP address), the data-plane will utilise both of these ECMP Next-Hops.

Output 13-40: FP-tunnel-table entry for R1 (192.0.2.1/32)

```

A:admin@R6# show router fp-tunnel-table 1 192.0.2.1/32 protocol isis
=====
IPv4 Tunnel Table Display

Legend:
label stack is ordered from bottom-most to top-most
B - FRR Backup
=====
Destination                                Protocol      Tunnel-ID
Lbl/SID
NextHop                                     Intf/Tunnel
Lbl/SID (backup)
NextHop      (backup)
-----
192.0.2.1/32                               SR-ISIS-0     524391
14001
192.168.0.17                               1/1/c2/1:100
14001
192.168.0.25                               1/1/c1/1:100
-----
Total Entries : 1
=====

```

The route-table entry for prefix 172.16.32.0/20 has a protocol owner (proto) of BGP with a Next Hop of 192.0.2.1 (router R1). The Next-Hop indicates that it is tunnelled using SR-ISIS with a Tunnel-ID of 524391 (tunneled:SR-ISIS:524391). This Tunnel ID correlates with the Tunnel ID shown in *Output 13-40*.

Output 13-41: Route-table entry for prefix 172.16.32.0/20 advertised by R1

```
A:admin@R6# show router route-table 172.16.32.0/20

=====
Route Table (Router: Base)
=====
Dest Prefix[Flags]                                Type    Proto    Age          Pref
  Next Hop[Interface Name]                        Metric
-----
172.16.32.0/20                                     Remote  BGP       00h33m17s    170
  192.0.2.1 (tunneled:SR-ISIS:524391)              300
-----

No. of Routes: 1
Flags: n = Number of times nexthop is repeated
      B = BGP backup route available
      L = LFA nexthop available
      S = Sticky ECMP requested
=====
```

The **resolution-filter** configuration is then modified to add the use of the RSVP-TE LSP from R6 to R1 using the **rsvp** keyword in the **resolution-filter** as well as **sr-isis**. Following this, the route-table entry changes so that it is tunnelled using RSVP (**tunneled:RSVP:1**). This is because when multiple tunnelling techniques are specified in the **resolution-filter**, the system will select and use the one with the lowest preference first. RSVP has a preference of 7, whilst SR-ISIS has a preference of 11, therefore RSVP is chosen. Should the RSVP-TE LSP fail for any reason, the SR-ISIS LSPs will be chosen in its place, thereby acting as a backup tunnelling mechanism.

Output 13-42: Route-table entry for prefix 172.16.32.0/20

```
A:admin@R6# show router route-table 172.16.32.0/20

=====
Route Table (Router: Base)
=====
Dest Prefix[Flags]                                Type    Proto    Age          Pref
  Next Hop[Interface Name]                        Metric
-----
172.16.32.0/20                                     Remote  BGP       00h00m14s    170
  192.0.2.1 (tunneled:RSVP:1)                      300
-----

No. of Routes: 1
Flags: n = Number of times nexthop is repeated
      B = BGP backup route available
      L = LFA nexthop available
      S = Sticky ECMP requested
=====
```

Peer Groups

BGP peer groups are a set of peers that have a common administrative configuration. The general idea behind peer groups is that associating a number of peers together can dramatically improve RIB-Out processing. Some BGP implementations achieve this by advertising the same Update message across all peers of a peer group, which improves RIB-Out Updates/second but can suffer from head-of-line blocking when the advertising BGP speaker encounters a 'slow peer'. In addition, implementations that are optimised to send

the same Update message across all peers of a peer group struggle in the presence of Route Target Constraint (RTC) or Outbound Route Filtering, where per-peer filters may be required.

SR-OS allows for peers to be associated with a peer group as shown in *Output 13-43*, however, this grouping is provided for operational/configuration ease because the grouping here does not necessarily dictate the forming of peer groups. Instead, the system dynamically organises peers into groups known as *tribes*. A tribe can consist of up to 64 peers that are subject to common RIB-Out processing such as common export-policy, and other configuration information.

To fully utilise multi-core processors and symmetric multi-core Processing (SMP), peers are distributed across a number of cores via I/O helper tasks that offload the process of writing to the TCP socket from the BGP process and other RIB-Out work (the actual number of cores is CPM-dependent, with a minimum of seven).

Output 13-43: Peer-Group configuration

```
router "Base" {
  bgp {
    group "EBGP-to-AS64514" {
      peer-as 64514
      family {
        ipv4 true
      }
    }
    neighbor "192.168.2.2" {
      group "EBGP-to-AS64514"
    }
  }
}
```

The grouping of peers into peer groups or tribes has a direct effect on memory consumption as a copy of the RIB-Out is maintained per peer group. The more peer groups there are, the more copies of the RIB-Out are maintained, and the more memory is consumed. It follows therefore that some due diligence is required on platforms that have a high BGP peer count. Wherever possible, configuration aspects such as export policies, split-horizon, etc, should be common for peer groups. Visibility of the number of internal tribes can be displayed as shown in *Output 13-44* with an indication of the current number of tribes and a high-water mark shown as Current Internal Groups and Max Internal Groups.

Output 13-44: Displaying internal peer groups (tribes)

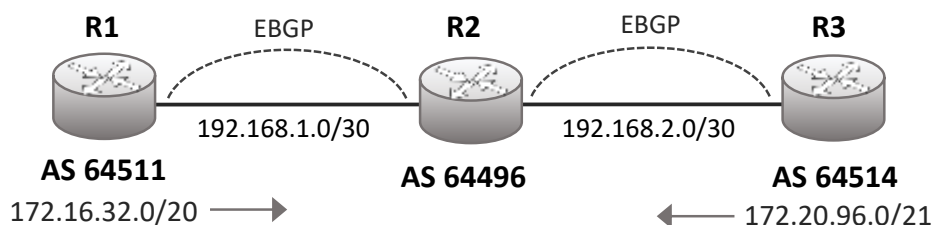
```
A:admin@R1# show router bgp summary | match 'Total Peer Groups|Total VPN Peer
Groups|Internal Group'
Total Peer Groups      : 6          Total Peers           : 8
Total VPN Peer Groups  : 2          Total VPN Peers       : 3
Current Internal Groups : 3          Max Internal Groups   : 3
```

Split-Horizon

To ensure a consistent view of routing information within an AS, a full-mesh of IBGP is provided (or Route-Reflectors are used). However, to avoid routing loops, a split-horizon rule is applied for IBGP in which prefixes learnt from an IBGP peer are never advertised to another

IBGP peer. This is the most commonly understood example of split horizon in BGP, but there is another interesting case that often causes some confusion. Consider the very simple topology in *Figure 13-7* where router R1 in AS 64511 advertises prefix 172.16.32.0/20 towards its external peer R2, and similarly router R3 in AS 64514 advertises prefix 172.20.96.0/21 towards its external peer R2.

Figure 13-7: Split-Horizon



From router R2's perspective, *Output 13-45* shows the prefix 172.16.32.0/20 received from router R1 in AS 64511 with a Next-Hop of 192.168.1.2, and it is installed as Valid, Used, and Best.

Output 13-45: R2's BGP routes received from R1

```

A:admin@R2# show router bgp neighbor 192.168.1.2 received-routes
=====
BGP Router ID:192.0.2.2      AS:64496      Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP IPv4 Routes
=====
Flag  Network                               LocalPref  MED
      Nexthop (Router)                     Path-Id     IGP Cost
      As-Path                               Label
-----
u*>i  172.16.32.0/20                         n/a        None
      192.168.1.2                           None        0
      64511                                  -
-----
Routes : 1
=====
  
```

Again, from router R2's perspective, *Output 13-46* shows the advertised routes towards router R1 in AS 64511. The second entry is prefix 172.20.96.0/21 that has an AS_PATH of {64496 64514}. It was originally advertised by router R3 to R2 and is now being advertised by R2 towards R1. The first entry however is prefix 172.16.32.0/20 that router R2 learnt from R1 in AS 64511 and has reflected back to R1, adding its own AS path in the AS_PATH attribute and setting the Next-Hop to itself (192.168.1.1).

Output 13-46: Router R2's RIB-Out entries towards R1

```

A:admin@R2# show router bgp neighbor 192.168.1.2 advertised-routes
=====
BGP Router ID:192.0.2.2          AS:64496          Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                  l - leaked, x - stale, > - best, b - backup, p - purge,
                  w - unused-weight-only
Origin codes   : i - IGP, e - EGP, ? - incomplete
=====
BGP IPv4 Routes
=====
Flag  Network                               LocalPref  MED
     Nexthop (Router)                       Path-Id     IGP Cost
     As-Path                                Label
-----
i     172.16.32.0/20                         n/a        None
     192.168.1.1                             None        0
     64496 64511                             -
i     172.20.96.0/21                         n/a        None
     192.168.1.1                             None        0
     64496 64514                             -
-----
Routes : 2
=====

```

Router R1 of course drops this prefix because its own AS appears in the AS_PATH attribute. The same behaviour can be observed for prefix 172.20.96.0/21, where it is advertised by router R3 to R2, and router R2 reflects the prefix back to R3 in addition to advertising it to router R1. The rationale for this behaviour is simply that by reflecting the route back to the advertising peer and not subjecting it to a different (split-horizon) RIB-Out policy some improvements can potentially be gained in the time to propagate the Update message. In addition, memory can also be conserved by not maintaining a different RIB-Out for this peer. However, this behaviour is sometimes considered undesirable, and in such an environment it is possible to disable this behaviour using the **split-horizon** command.

Output 13-47: Split-Horizon configuration at R2

```

router "Base" {
  bgp {
    group "EBGP-to-AS64511" {
      peer-as 64511
      split-horizon true
    }
    group "EBGP-to-AS64514" {
      peer-as 64514
      split-horizon true
    }
    neighbor "192.168.1.2" {
      group "EBGP-to-AS64511"
      family {
        ipv4 true
      }
    }
    neighbor "192.168.2.2" {
      group "EBGP-to-AS64514"
      family {
        ipv4 true
      }
    }
  }
}

```

```

    }
  }
}

```

The prefix 172.16.32.0/20 is again advertised from router R1 to router R2, but with **split-horizon** enabled, the prefix is not advertised back to R1. The only advertised prefix towards R1 is the prefix 172.20.96.0/21 originated in AS 64514.

Output 13-48: Routes advertised by R2 to R1 with split-horizon enabled

```

A:admin@R2# show router bgp neighbor 192.168.1.2 advertised-routes
=====
BGP Router ID:192.0.2.2          AS:64496          Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                  l - leaked, x - stale, > - best, b - backup, p - purge,
                  w - unused-weight-only
Origin codes   : i - IGP, e - EGP, ? - incomplete
=====
BGP IPv4 Routes
=====
Flag  Network                               LocalPref  MED
      Nexthop (Router)                     Path-Id     IGP Cost
      As-Path                               Label
-----
i     172.20.96.0/21                        n/a        None
      192.168.1.1                          None        0
      64496 64514                          -
-----
Routes : 1
=====

```

The benefits of enabling **split-horizon** are immediately obvious. It does however come with caveats. In the previous section on *Peer Groups*, it was described that the system creates a copy of the RIB-Out per tribe. If a single peer has **split-horizon** enabled, it cannot be grouped into the same tribe as a peer that has **split-horizon** disabled. In this scenario the system would create a RIB-Out for each peer even if all other peering parameters are equal. If **split-horizon** is enabled at the base BGP level, a RIB-Out per tribe is maintained, and multiple peers with the same peering parameters can be grouped into the same tribe, even with **split-horizon** enabled. The suppressing of reflected Update messages to the originating peer adds nominal overhead as the code-path already exists.

In summary, the main caveat with the use of **split-horizon** is when a configuration includes mixture of peers with **split-horizon** enabled and **split-horizon** disabled. In this scenario the possibility of being able to group peers into tribes is reduced, therefore potentially increasing the number of RIB-Out copies and consuming additional memory. In addition, it is recommended that **split-horizon** is not used on IBGP sessions where the Next-Hop remains unchanged. Split-horizon is done implicitly for these sessions without the need for the **split-horizon** command, which could cause the extra creation of extra tribes and RIB-Out copies for no reason.

Dynamic Peer Groups

By default, the configuration parameters of a BGP peer need to be locally configured at BGP, group, or neighbour level and the IP address of the peer needs to be specified in a **neighbor** configuration command. When this configuration is in place (and providing the session is not configured as passive), the system will attempt an outgoing TCP connection toward the neighbour IP address, and an incoming TCP connection (to port 179) is accepted from a source IP address if it matches the configured neighbour IP address.

The BGP finite state machine optionally allows the acceptance of BGP peers connections from neighbours that are not pre-configured. However, it is not explicit about what “unconfigured” means, and as a result, a BGP security vulnerability analysis [65] concluded that it is not possible to conduct an analysis of the security issues of this feature. It should however be reasonably clear; “unconfigured” can only realistically be partially unconfigured. There needs to be a way to associate parameters such as security attributes to a given peer or group of peers.

SR-OS allows for IPv4 and IPv6 BGP sessions to be established from unconfigured peers in the base and VPRN routing instances, using a feature known as *dynamic peers*. Dynamic (or unconfigured) peers are placed into a peer-group where a prefix-list can optionally be used to limit the source IP addresses of peers that can dynamically establish a BGP session. Dynamic peers can also co-exist with configured peers within a given peer-group. The session parameters defined within the group are common to all peers; dynamic and configured. Dynamic BGP sessions are permitted when they are detected by either of the following mechanisms:

- The source IP address of an incoming BGP TCP connection matches an IP prefix associated with dynamic BGP sessions.
- An ICMPv6 Router Advertisement message is received from a potential BGP router on an interface listed as a dynamic-neighbor interface.

This section will describe two use-cases for dynamic BGP peers that include both methods. The first method with a source IP address matching an IP prefix associated with dynamic BGP peering is used at a Route-Reflector but could equally be used on any device that terminates many BGP sessions. The second method of receiving an ICMPv6 Router Advertisement message is used in a data centre that uses IPv6 EBGp between directly connected peers to build its underlay topology.

Source-Address as Dynamic Peer Trigger

Within most Service Provider networks, Route-Reflectors are sacrosanct. Configuration changes are kept to a minimum simply because if something goes wrong the impact can potentially be felt network-wide. That said, configuration changes including adds/changes/deletes are always necessary, and to help minimise the amount of manual configuration during these changes, dynamic BGP peering could be used.

Output 13-49 provides an example of dynamic BGP peer configuration on a Route-Reflector that has six IBGP client peers with peering addresses of 192.0.2.1 through to 192.0.2.6. Neighbours 192.0.2.1, 192.0.2.2, and 192.0.2.3 are statically configured. Neighbours 192.0.2.4, 192.0.2.5, and 192.0.2.6 are dynamic peers. The split between the statically configured and dynamic peers serves merely to highlight that they can co-exist. The **dynamic-neighbor** context is configured at **group** level and provides a match context where one or more IPv4 and/or IPv6 prefix/length entries can be configured that constitute the peers permitted to establish a BGP session with this router. In effect, the **dynamic-neighbor match prefix** puts the router into a passive state for peers that fall within the specified prefix/length. Only inbound connections are established. The purpose of the **dynamic-neighbor-limit** command should be reasonably clear and places a boundary on how many dynamic BGP sessions the router will accept. Once this limit is reached, the router will not allow any further BGP sessions to be established. Note that any other mandatory peering parameters (**family**, **peer-as**, **authentication-key** if used) need to be configured at group level to provide sufficient information to allow the dynamic peers to be established.

Output 13-49: Dynamic Peer configuration

```
router "Base" {
  bgp {
    group "CLIENTS" {
      authentication-key "C3i5c6IP3VMhQFGff3RCMznwfvRC06c= hash2"
      peer-as 64496
      dynamic-neighbor-limit 256
      family {
        ipv4 true
        ...etc
      }
      cluster {
        cluster-id 192.0.2.10
      }
      dynamic-neighbor {
        match {
          prefix 192.0.2.0/24 {
          }
        }
      }
    }
  }
  neighbor "192.0.2.1" {
    group "CLIENTS"
    authentication-key "C3i5c6IP3VMhQFGff3RCM6O4qzE0tDo= hash2"
  }
  neighbor "192.0.2.2" {
    group "CLIENTS"
    authentication-key "C3i5c6IP3VMhQFGff3RCM3Kf+/MN6G8= hash2"
  }
  neighbor "192.0.2.3" {
    group "CLIENTS"
    authentication-key "C3i5c6IP3VMhQFGff3RCM4yMtmvHSUM= hash2"
  }
}
```

Output 13-50 shows the (truncated) output of all established BGP peers, with the (D) flag used to indicate that the peer is dynamically established. The command “show router bgp group “CLIENTS” will also provide a list of dynamically established BGP peers.

Output 13-50: BGP summary with static and dynamic neighbours

```

A:admin@rr1# show router bgp summary
=====
BGP Router ID:192.0.2.10      AS:64496      Local AS:64496
=====
BGP Admin State      : Up      BGP Oper State      : Up
Total Peer Groups    : 5      Total Peers         : 39
Total VPN Peer Groups : 0      Total VPN Peers     : 0
Current Internal Groups : 6      Max Internal Groups : 6
Total BGP Paths       : 184     Total Path Memory    : 74632

--- [snip] ---

=====
BGP Summary
=====
Legend : D - Dynamic Neighbor
=====
Neighbor
Description
          AS PktRcvd InQ Up/Down State|Rcv/Act/Sent (Addr Family)
          PktSent OutQ
-----
192.0.2.1
          64496 5285 0 01d04h17m 2/0/4 (IPv4)
                               5/0/10 (VpnIPv4)
                               1/0/6 (VpnIPv6)
192.0.2.2
          64496 41594 0 07d00h54m 0/0/5 (IPv4)
          20918 0      0/0/15 (VpnIPv4)
                               0/0/7 (VpnIPv6)
192.0.2.3
          64496 29421 0 10d04h40m 3/0/4 (IPv4)
                               4/0/11 (VpnIPv4)
                               2/0/5 (VpnIPv6)
192.0.2.4 (D)
          64496 38 0 00h00m55s 1/0/5 (IPv4)
          77 0      3/0/12 (VpnIPv4)
                               2/0/5 (VpnIPv6)
192.0.2.5 (D)
          64496 12 0 00h00m36s 0/0/5 (IPv4)
          70 0      0/0/15 (VpnIPv4)
                               0/0/7 (VpnIPv6)
192.0.2.6 (D)
          64496 29 0 00h00m58s 1/0/5 (IPv4)
                               3/0/12 (VpnIPv4)
                               2/0/5 (VpnIPv6)

```

ICMPv6 Router Advertisement as Dynamic Peer Trigger

In the data centre native IP is the predominant technology, utilizing network virtualization overlay (NVO) technologies such as virtual extensible LAN (VXLAN) or generic network virtualization encapsulation (GENEVE). The use of BGP for routing in data centres is becoming increasingly commonplace. Whereas BGP was initially used to create these overlay networks, it is now being widely adopted for creating the data centre underlay network [81] simply because it can scale ‘horizontally’, it has a simple implementation in terms of programming code complexity, it minimises network churn as its flooding overhead is less compared to link-state IGP, and it allows for traffic-engineering and ECMP.

Data centres typically use a hierarchical clos-based network design consisting of two or three tiers. Leaf nodes are used at lowest tier of the hierarchy and connect to servers, virtualised network functions (VNFs), containerised network functions (CNFs), and general compute nodes. Leaf nodes are also connected to spine nodes which form the middle tier of the hierarchy, and in turn these spine nodes are connected to data centre gateways, sometimes via border-leaf nodes. (In very large data centre networks, a super-spine layer may exist adding a fourth tier.) EBGp single-hop sessions are established over direct point-to-point links interconnecting the nodes. No multihop is used, even in the case of multiple links between the same pair of nodes. Private use ASNs from the range 64512-65534 are used to avoid ASN conflicts, and each node in tier two and three of the hierarchy is allocated a unique AS number. Nodes at tier 1 of the hierarchy are allocated a single AS number. To facilitate automation, these directly connected EBGp peers can be automatically detected without the need for manual configuration of IPv4 or IPv6 addressing by using IPv6 link-local addressing.

Before configuring the unnumbered IPv6 with dynamic BGP peering the unnumbered interface must first be configured as shown in *Output 13-51*. No explanation is needed of the parameters configured here, except to say that the IPv6 context is present and enabled, but there is no explicit IPv4 or IPv6 addressing. In addition to the IP interface configuration, the **ipv6 router-advertisement** context enables ICMPv6 Router Advertisement over the configured IP interface and puts it into an **admin-state** of **enable**. Recall that the reception of a Router Advertisement message is required over this interface as a trigger for a potential BGP dynamic peer. This configuration therefore reciprocates and sends Router Advertisement messages over the interface at the specified interval for the adjacent peer. The **min-advertisement-interval** and **max-advertisement-interval** commands simply place a minimum and maximum interval between the sending of subsequent Router Advertisement messages.

Output 13-51: Unnumbered IPv6 interface

```

service {
    ies "DC-UNDERLAY" {
        interface "ethernet-1/1/c6/1-AS64513" {
            admin-state enable
            sap 1/1/c6/1:6 {
                admin-state enable
            }
            ipv6 {
            }
        }
    }
}
router "Base" {
    ipv6 {
        router-advertisement {
            interface "ethernet-1/1/c6/1-AS64513" {
                admin-state enable
                max-advertisement-interval 10
                min-advertisement-interval 4
                router-lifetime 3600
            }
        }
    }
}

```

With a similar configuration applied at the peer node, *Output 13-52* shows the operational state of the unnumbered IPv6 interface. Although there are no explicit IPv4 or IPv6 addresses, the interface is up (Oper (v4/v6) for IPv6 due to the presence of the IPv6 link-local address fd80::201:ffff:fe00:0/64.

Output 13-52: Operational state of IPv6 unnumbered interface

```
A:admin@leaf-64512# show router interface "ethernet-1/1/c6/1-AS64513" ipv6

=====
Interface Table (Router: Base)
=====
Interface-Name      Adm      Opr (v4/v6)  Mode      Port/SapId
IP-Address          PfxState
-----
ethernet-1/1/c6/1-AS64513  Up      Down/Up      IES      1/1/c6/1:6
fe80::201:ffff:fe00:0/64  PREFERRED
-----
Interfaces : 1
=====
```

A BGP group supporting dynamic IPv6 peering is then configured and starts by enabling both the **ipv4** and **ipv6** Address Families. In a pure IPv6 data centre fabric, the IPv4 Address Family would not be required to build the underlay topology, but it is included here to illustrate that building an IPv4 fabric with IPv6 BGP peering is possible. This is largely enabled using the commands **advertise-ipv6-next-hops ipv4** and **extended-nh-encoding ipv4**. The latter instructs the BGP peer to exchange the Extended Next-hop Encoding BGP capability during the Open exchange to indicate that it is capable of accepting IPv4 prefixes with an IPv6 Next-Hop. The latter instructs the system to advertise IPv4 prefixes with an IPv6 Next-Hop. (Advertising IPv4 reachability over an IPv6 core is described in detail later in this chapter.) The peer **group** context contains as much information as required and will be applied to any dynamic peer established within this group context. Within the **dynamic-neighbor** context the **interface** command specifies the previously configured IPv6 unnumbered interface. The **allowed-peer-as** command then specifies the peer AS number or AS numbers that can be accepted over this interface and which the system expects to see as the peer AS in the Open exchange. Finally, the **max-sessions** parameter limits the number of dynamic peers on this interface to 1 since this is a point-to-point link.

Output 13-53: Dynamic IPv6 BGP peering

```
router "Base" {
    autonomous-system 64512
    bgp {
        group "DC1-Dynamic-Peers" {
            family {
                ipv4 true
                ipv6 true
            }
            ebgp-default-reject-policy {
                import false
            }
            export {
                policy ["Export-System-IP"]
            }
            extended-nh-encoding {
                ipv4 true
            }
        }
    }
}
```

```

    }
    advertise-ipv6-next-hops {
        ipv4 true
    }
    dynamic-neighbor {
        interface "ethernet-1/1/c6/1-AS64513" {
            allowed-peer-as ["64513"]
            max-sessions 1
        }
    }
}
}
}
}

```

For completeness the route-policy referenced as an export policy above is shown in *Output 13-54* and simply allows both the /32 IPv4 and /128 IPv6 system addresses to be advertised into BGP whilst setting the Origin attribute to IGP.

Output 13-54: Route-policy used to export IPv4 and IPv6 system addresses

```

policy-options {
    prefix-list "System" {
        prefix 192.0.2.1/32 type exact {
        }
        prefix 3fff:ffe::1/128 type exact {
        }
    }
    policy-statement "Export-System-IP" {
        entry 10 {
            from {
                prefix-list ["System"]
            }
            to {
                protocol {
                    name [bgp]
                }
            }
            action {
                action-type accept
                origin igp
            }
        }
    }
}
}
}

```

The status of the dynamic BGP can be summarised as shown in *Output 13-55*. The neighbour fe80::5054:ff:fee4:2780-“ethernet-c1/1/c6/1-AS64513” is the link-local address of the adjacent peer and is denoted with a (D) flag to indicate that it is a dynamic neighbour. The IPv4 and IPv6 Address Families have been negotiated over this session, and routes have been exchanged in both directions as the (State|Rcv/Act) field shows non-zero values for both families.

Output 13-55: Summary of dynamic BGP peer

```

A:admin@64512# show router bgp summary
=====
BGP Router ID:192.0.2.1          AS:64512          Local AS:64512
=====
BGP Admin State      : Up          BGP Oper State      : Up
Total Peer Groups    : 7           Total Peers          : 9
Total VPN Peer Groups : 1           Total VPN Peers      : 2
Current Internal Groups : 3         Max Internal Groups  : 3

```



```

Total BGP Paths      : 232          Total Path Memory      : 87944

--- [ snip ] ---

=====
BGP Summary
=====
Legend : D - Dynamic Neighbor
=====

Neighbor
Description
          AS PktRcvd InQ Up/Down   State|Rcv/Act/Sent (Addr Family)
          PktSent OutQ
-----
--- [ snip ] ---
          0      0
fe80::5054:ff:fee4:2780-"ethernet-c1/1/c6/1-AS64513" (D)
          64513    231    0 01h49m54s 6/1/1 (IPv4)
          232     0      3/2/1 (IPv6)

```

BGP Monitoring Protocol (BMP)

For many years operators requested a method with which they could have better visibility of the BGP control plane including neighbour state and route visibility. As BGP provided no inherent mechanism to provide out-of-band visibility, the typical approach was reduced to 'screen scraping'. The solution came in the form of the BGP Monitoring Protocol (BMP). BMP provides information regarding the Adj-RIB-In of a peer, including an initial dump of all routes received from that peer as well as an ongoing dump of statistics that can be used by the BMP receiver as an indication of the level of activity of a given peer or peers. There are also notification messages to indicate when a peering session has come up, and similarly when a peering session has gone down.

The initial specification in [62] describes the characteristics of the protocol with its message formats and describes visibility of the Adj-RIB-In. This specification was further enhanced in [63] that added visibility of the Adj-RIB-Out to BMP, and then again in [64] to provide visibility of the Loc-RIB. SR-OS supports only visibility of the Adj-RIB-In detailed in the original specification hence Adj-RIB-Out and Loc-RIB are not discussed further in this section.

BMP operates over TCP, and all options are controlled by configuration on the monitored router(s). That is, there is no well-known port allocated to BMP. The router configures the destination port of the monitoring station, and the monitoring station is instructed to listen for BMP messages on that port. It is a unidirectional protocol in as much as BMP messages are only sent from the router to the monitoring station. No BMP message is ever sent from the monitoring station to the router. Once the TCP session is established, the router sends BMP messages over it. There is no initialization phase, handshaking, or capability negotiation – messages are simply sent as soon as the connection is established.

- The router begins by sending an Initiation message and follows that by sending a Peer Up message for each of the monitored BGP peers that is in the Established state.
- This is followed by sending the contents of its Adj-RIBs-In, which can be pre-policy, post-policy, or both. Once it has sent all the routes for a given peer, it sends an End-of-RIB message for that peer, and when an End-of-RIB message has been sent for each of the monitored BGP peers, the initial table dump is complete.
- Following the initial table dump, the router sends incremental updates encapsulated in Route Monitoring messages. If any new monitored BGP peer becomes established, a Peer Up message is sent. Conversely, if a monitored BGP peer transitions out of the Established state, a Peer Down message is sent.

A BMP session ends when the TCP session over which it is carried is closed. If it is a graceful termination, the router may send a BMP Termination message prior to closing the TCP session.

The common header shown in *Figure 13-8* appears in all BMP messages and immediately follows the TCP header. The version is set to 3, with versions 1 and 2 used in draft versions of the BMP specification. The format of the remainder of the message depends on the value of the message type, with the possible types listed in *Table 13-1*.

Figure 13-8: Common Header

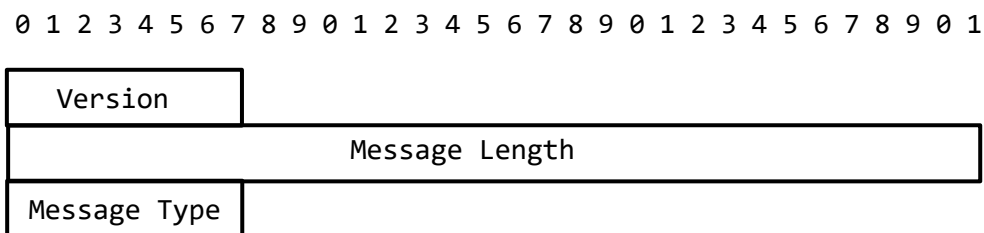


Table 13-1: BMP Message types

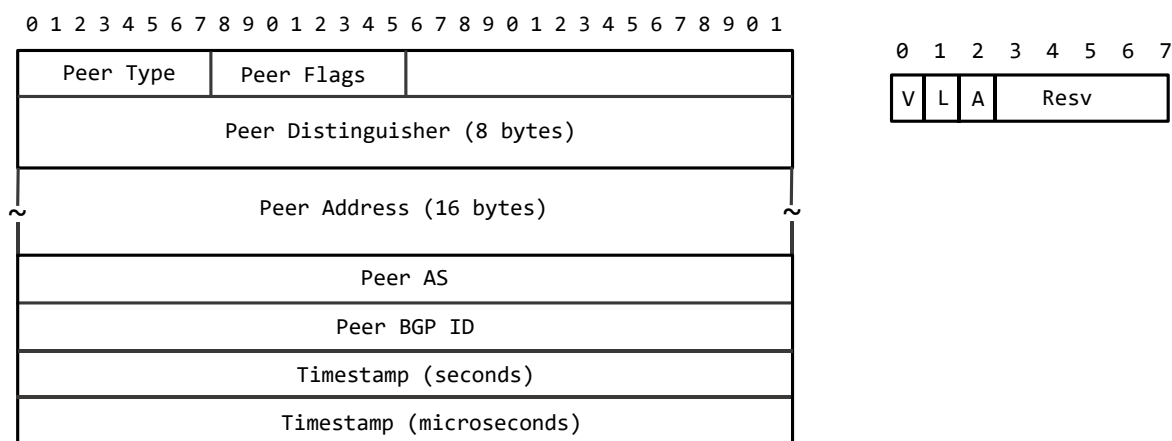
Message Type	BMP Message
0	Route Monitoring
1	Statistics Report
2	Peer Down Notification
3	Peer Up Notification
4	Initiation Message
5	Termination Message
6	Route Mirroring Message

The per-peer header follows the common header for most BMP messages. The peer type identifies the type of peer with three types currently defined: Global Instance peer (type 0), RD Instance peer (type 1), and Local Instance peer (type 2). The peer flags field includes three flags. The V flag when set to 1 indicates that the peer address is an IPv6 address and when

not set indicates that it is an IPv4 address. The L flag when set to 1 indicates that the message content reflects the post-policy of the Adj-RIB-In (that is, after import policy has been applied). When not set it indicates that the message content reflects the pre-policy Adj-RIB-In (before import policy has been applied). Locally sourced routes also carry an L flag of 1. The A flag when set to 1 indicates that the message is formatted using 2-byte AS_PATH format, and when set to zero indicates that it is formatted using the 4-byte AS_PATH format.

The peer distinguisher is used such that the combination of the peer type and peer distinguisher field provide sufficient information to disambiguate peers for which overlapping address space might exist. For BGP peers that reside within an L3VPN it carries the Route-Distinguisher of that particular IP-VRF instance. If the peer is in the base routing instance this field is zero-filled. The peer address is the address of the remote end of the TCP session carrying the BGP session and is 4-bytes long for an IPv4 address and 16-bytes long for an IPv6 address. The Peer AS is the AS number identified by the peer in its Open message, and if a 2-byte ASN is carried in this field the most significant bits are zero-filled. The BGP Peer ID represents the Router ID of the remote peer. Finally, the timestamp indicates the time when routes from this peer were installed in the Adj-RIB-In expressed in seconds and microseconds since midnight January 1, 1970.

Figure 13-9: Per-Peer Header



The necessary configuration to enable a BMP monitoring station is shown in *Output 13-56* and begins in the **bmp** context. One or more monitoring stations are defined using the **station** command which defines the characteristics of the TCP session to support BMP and how the router reports BGP information. The **initiation-message** simply allows for a message to be displayed in the Initiation message originated at the router at the start of the BMP session. It will be described further below. The **stats-report-interval** is defined in seconds and defines how often the router reports statistics. Currently there are fourteen statistic types defined in [62] and they are listed in *Table 13-2*, with all of them supported in SR-OS with the exception of statistic type 13 (Number of duplicate update messages received). The **report-local-routes** command specifies how the router should report routes that are locally originated. That is, routes that have been sourced by the reporting router that may have been redistributed from another protocol or some other reason. In BMP these are modelled

as having been sent by the router to itself, and it places its own address in the Peer Address field of the Per-Peer header. The **connection** context provides options on how the TCP session should be established and maintained and includes **local-address**, **router-instance**, **connection-retry**, **tcp-keepalive**, as well as a **station-address** context. Within this context the **ip-address** contains the address of the monitoring station as well as a destination TCP **port**, which in this case is set to 5000. The **tcp-keepalive** option can also be selected on a per-station basis and in this case is enabled. Lastly, the **family** context allows the user to define the Address Families for which reporting is enabled, which in this example is VPN-IPv4 and VPN-IPv6.

Output 13-56: BMP station configuration

```

bmp {
  admin-state enable
  station "ubuntu2-dot29" {
    admin-state enable
    initiation-message "hello"
    stats-report-interval 30
    report-local-routes true
    connection {
      router-instance "Base"
      station-address {
        ip-address 192.168.254.18
        port 5000
      }
      tcp-keepalive {
        admin-state enable
      }
    }
  }
  family {
    vpn-ipv4 true
    vpn-ipv6 true
  }
}

```

Table 13-2: BMP Statistic Types

Stat Type	Statistic
0	Number of prefixes rejected by inbound policy
1	Number of (known) duplicate prefix advertisements
2	Number of (known) duplicate withdraws
3	Number of updates invalidated due to CLUSTER_LIST loop
4	Number of updates invalidated due to AS_PATH loop
5	Number of updates invalidated due to ORIGINATOR_ID
6	Number of updates invalidated due to a loop found in AS_CONFED_SEQUENCE or AS_CONFED_SET
7	Number of routes in Adj-RIBs-In
8	Number of routes in Loc-RIB
9	Number of routes in per-AFI/SAFI Adj-RIB-In
10	Number of routes in per-AFI/SAFI Loc-RIB

11	Number of updates subjected to treat-as-withdraw
12	Number of prefixes subjected to treat-as-withdraw
13	Number of duplicate update messages received

BMP monitoring can be enabled at **group** or **neighbor** level within the base routing instance or within an IP-VRF. In the example shown in *Output 13-57* it is enabled at **group** level in the base routing instance of a Route-Reflector with a number of clients. (A topology is not important here, just an acknowledgement that there are multiple clients advertising VPN-IPv4/IPv6 routes.) Within the **monitor** context, the **route-monitoring** command allows for Report Monitoring messages to be **pre-policy** or post-policy and in this case is set to the former. The **station** command references the previously configured monitoring station, and the **monitor** context is placed into an **admin-state** of enable.

Output 13-57: Monitor configuration at BGP level

```

router "Base" {
    bgp {
        group "CLIENTS" {
            peer-as 64496
            split-horizon true
            family {
                vpn-ipv4 true
                ...etc
            }
            cluster {
                cluster-id 192.0.2.10
            }
            monitor {
                admin-state enable
                route-monitoring {
                    pre-policy true
                }
                station "ubuntu2-dot29" { }
            }
        }

        neighbor "192.0.2.10" {
            group "CLIENTS"
        }
        neighbor "192.0.2.3" {
            group "CLIENTS"
        }
        ...etc
    }
}

```

The operational status of the BMP session to the monitoring station can be displayed as shown in *Output 13-58*. In addition to displaying the operational state of the configuration parameters, the output provides a useful snapshot of the state of the BMP session together with the number of BMP messages sent to the monitoring station. There is also a useful output of the number of monitored peers and running peers, where a running peer is in the Established state.

Output 13-58: Operational status of BMP monitoring station

```
A:admin@RR1# show router bmp station "ubuntu2-dot29"
```

```

=====
BMP Station "ubuntu2-dot29" (monitoring router "Base")
=====
Admin State      : enabled                      Global BMP State : enabled
Station Address  : 192.168.254.18                Station Port     : 5000
Local Address    : 192.0.2.10                    Local Port       : 57758
Via Router       : Base
Stats Report     : 60 seconds
Connect Interval : 120 seconds                    Local Routes     : reporting
Initiation Msg   : hello
Reported families: ipv4 vpn-ipv4 vpn-ipv6
TCP Keepalive    : enabled                      Multiplier       : 4
Idle             : 600 seconds                    Interval        : 15 seconds

Session State    : ESTABLISHED                    Last State Change: 10/09/2025 08:25:16
Reason Last Down : never down yet                 Last Msg Sent    : 10/09/2025 08:44:21
Routes Timer     : 3 seconds left                  Stats Timer      : 54 seconds left
Connect Timer    : not running
Initiation Msgs  : 1                               Goodbye Msgs     : 0
Peer Up Msgs     : 7                               Peer Down Msgs   : 0
Route Report Msgs: 6553                           Stat Report Msgs : 114
Bytes Sent       : 947605                          Output Queue     : 0/5

Monitored Peers  : 8                               Running Peers    : 6
Peer 1           : 192.0.2.1                       Peer 2           : 192.0.2.2
Peer 3           : 192.0.2.3                       Peer 4           : 192.0.2.4
Peer 5           : 192.0.2.5                       Peer 6           : 192.0.2.6
Peer 7           : 192.0.2.7                       Peer 8           : 192.0.2.12
=====

```

Once the TCP session is established the router starts to send unsolicited BMP messages, and the first message is the Initiation Message (type 4). It is intended to inform the monitoring station of its vendor, software version, and other ancillary information. It consists of the common BMP header followed by two or more Information TLVs containing information about the monitored router. Currently there are three Information TLVs defined for this purpose, a sysDescr TLV (Type 2), a sysName TLV (Type 1), and a sysString TLV (Type 0). The sysName contains the hostname of the reporting router, while SR-OS uses the sysDescr to report the software version. The sysString TLV is freeform text and is used to send the message configured in the **initiation-message** command. *Figure 13-10* shows an example of the Initiation message sent by the router to the monitoring station.

Figure 13-10: BMP Initiation Message

```

> Frame 28: 362 bytes on wire (2896 bits), 362 bytes captured (2896 bits) on interface br1100, id 0
> Ethernet II, Src: RealtekU_36:8b:30 (52:54:00:36:8b:30), Dst: RealtekU_f0:e2:d4 (52:54:00:f0:e2:d4)
> 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 29
> Internet Protocol Version 4, Src: 192.0.2.10, Dst: 192.168.254.18
> Transmission Control Protocol, Src Port: 57758, Dst Port: 5000, Seq: 1, Ack: 1, Len: 292
> BMP Monitoring Protocol, Type: Initiation Message
  Version: 3
  Length: 292
  Type: Initiation Message (4)
  Information Types, Type sysName, Type String, Type sysDescr, Type String
    - Type: sysName (2)
      Length: 3
      Information: rri
    - Type: String (0)
      Length: 33
      Information: Base Router, router-id 192.0.2.10
    - Type: sysDescr (1)
    - Type: String (0)

```

After the Initiation message the router sends a Peer Up Notification (type 3) message for each of the monitored BGP peers that is in the Established state. The Peer Up message does this by encapsulating a copy of the information exchanged in the Open message sent to and received from the peer when the session was established. *Debug 13-1* shows an example of the Peer Up Notification message containing the details of local peer 192.0.2.10 and remote peer 192.0.2.1

Debug 13-1: Peer Up Notification message

```
27 2025/10/09 13:33:00.646 BST minor: DEBUG #2001 Base BMP
BMP: PKT
  Station: ubuntu2-dot29 - PEER-UP, 492 bytes
    Peer: 192.0.2.1 (G) - flags 0x00, asn 64496, bgp-id 192.0.2.1
    Up at: 2025/10/08 14:35:46.465 BST
  Lcladdr: 192.0.2.10, local portnr 179, remote portnr 50429
    Open: (sent) asn 64496, ht 90, bgp-id 192.0.2.10, optlen 196
    Open: (rcvd) asn 64496, ht 90, bgp-id 192.0.2.1, optlen 170
```

The Peer Up Notification message is also shown in *Figure 13-11* where it is clear that the Per-Peer header follows the common BMP header. This is followed by two Open messages representing each end of the peering session and includes any capabilities negotiated as part of the session establishment process. Should any peer being monitored transition from the Established state, a Peer Down Notification is immediately sent. The Peer Down Notification message is simple in structure and encodes a Reason field with some standard and generic codes provided in the specification. A Peer Down message implicitly withdraws all routes that were associated with the peer in question.

Figure 13-11: Peer Up Notification message

```
* Frame 30: 1982 bytes on wire (8656 bits), 1982 bytes captured (8656 bits) on interface br100, id 0
* Ethernet II, Src: RealtekU_36:8b:30 (52:54:00:36:8b:30), Dst: RealtekU_f0:e2:d4 (52:54:00:f0:e2:d4)
* 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 29
* Internet Protocol Version 4, Src: 192.0.2.10, Dst: 192.168.254.18
* Transmission Control Protocol, Src Port: 57758, Dst Port: 5000, Seq: 293, Ack: 1, Len: 1012
* BGP Monitoring Protocol, type Peer Up Notification
  Version: 3
  Length: 492
  Type: Peer Up Notification (3)
  Per Peer Header
    Unused: 00000000000000000000000000000000
    Local Address: 192.0.2.10
    Local Port: 179
    Remote Port: 50429
  Border Gateway Protocol - OPEN Message
    Marker: ffffffffffffffffffffffffffffffffff
    Length: 225
    Type: OPEN Message (1)
    Version: 4
    My AS: 64496
    Hold Time: 90
    BGP Identifier: 192.0.2.10
    Optional Parameters Length: 196
    Optional Parameters
      Optional Parameter: Capability
  Border Gateway Protocol - OPEN Message
    Marker: ffffffffffffffffffffffffffffffffff
    Length: 199
    Type: OPEN Message (1)
    Version: 4
    My AS: 64496
    Hold Time: 90
    BGP Identifier: 192.0.2.1
    Optional Parameters Length: 170
    Optional Parameters
      Optional Parameter: Capability
```

When Peer Up Notification messages have been sent for all peers in the Established state, route Monitoring messages are used to provide a snapshot of the Adj-RIB-In of each monitored peer. This is achieved by sending all routes stored in the Adj-RIB-In of those peers using standard BGP Update messages encapsulated in Route Monitoring messages. When the initial dump is complete for a given peer, an End-of-RIB marker is sent for that peer encapsulated in the BMP header. *Debug 13-2* shows an example of a Route Monitoring message for a VPN-IPv6 address from the Adj-RIB-In of peer 192.0.2.3.

Debug 13-2: Route Monitoring message for a VPN-IPv6 prefix

```
34 2025/10/09 13:33:03.644 BST minor: DEBUG #2001 Base BMP
BMP: PKT
Station: ubuntu2-dot29 - ROUTE-MONITORING, 172 bytes
  Peer: 192.0.2.3 (G) - flags 0x00, asn 64496, bgp-id 192.0.2.3
Rcvd at: 2025/09/29 14:12:54.244 BST
Withdrawn Length = 0
Total Path Attr Length = 101
Flag: 0x40 Type: 1 Len: 1 Origin: 0
Flag: 0x40 Type: 2 Len: 0 AS Path:
Flag: 0x40 Type: 5 Len: 4 Local Preference: 100
Flag: 0xc0 Type: 16 Len: 24 Extended Community:
  target:64496:1
  vrf-import:192.0.2.3:2
  source-as:64496:0
Flag: 0x80 Type: 14 Len: 57 Multiprotocol Reachable NLRI:
  Address Family VPN_IPV6
  NextHop len 24 Global NextHop ::ffff:192.0.2.3
  2001:db8:3001:2101::4/126 RD 64496:1 Label 524282 (Raw label 0x7fffa1)
```

A packet capture of a Route Monitoring message is shown in *Figure 13-12* to aid understanding of the encoding. Following the common BMP header, the message starts with a Per-Peer header that identifies the peer as 192.0.2.1, which is a Global Instance peer (Peer Type is set to 0). The peer ASN is 64496. The Per-Peer header is followed by a standard BGP Update message containing a number of mandatory and discretionary path attributes, one of which, the MP_REACH_NLRI contains a VPN-IPv4 prefix. In short, Route Monitoring messages encapsulate standard BGP Update messages identifying entries in the Adj-RIB-In for each peer.

Figure 13-12: Route Monitoring message

```

* Frame 42: 1082 bytes on wire (8656 bits), 1082 bytes captured (8656 bits) on interface Br1100, id 0
* Ethernet II, Src: RealtekU_36:8b:36 (52:54:00:36:8b:36), Dst: RealtekU_f0:e2:d4 (52:54:00:f0:e2:d4)
* 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 29
* Internet Protocol Version 4, Src: 192.0.2.10, Dst: 192.168.254.10
* Transmission Control Protocol, Src Port: 57758, Dst Port: 5000, Seq: 8838, Ack: 1, Len: 1012
* [2 Reassembled TCP Segments (148 bytes): #41(48), #42(100)]
* BMP Monitoring Protocol, Type: Route Monitoring
  Version: 3
  Length: 148
  Type: Route Monitoring (0)
  - Per Peer Header
    Type: Global Instance Peer (0)
    * 0000 0000 = Flags: 0x00
    Peer Distinguisher: 0:0
    Unused: 000000000000000000000000
    Address: 192.0.2.1
    ASN: 64496
    BGP ID: 192.0.2.1
    Timestamp (sec): 1759938549
    Timestamp (msec): 824256
  - Border Gateway Protocol - UPDATE Message
    Marker: ffffffffffffffffffffffffffffffff
    Length: 188
    Type: UPDATE Message (2)
    Withdrawn Routes Length: 0
    Total Path Attribute Length: 77
    - Path attributes
      * Path Attribute - ORIGIN: IGP
      * Path Attribute - AS_PATH: empty
      * Path Attribute - LOCAL_PREF: 100
      * Path Attribute - EXTENDED_COMMUNITIES
      * Path Attribute - MP_REACH_NLRI
      * Flags: 0x00, Optional, Non-transitive, Complete
      Type Code: MP_REACH_NLRI (14)
      Length: 33
      Address family identifier (AFI): IPv4 (1)
      Subsequent address family identifier (SAFI): Labeled VPN Unicast (128)
      * Next hop: RD=0:0 IPv4=192.0.2.1
      * Number of Subnetwork points of attachment (SNPA): 0
    - Network Layer Reachability Information (NLRI)
      * BGP Prefix
        Prefix Length: 118
        Label Stack: 524294 (bottom)
        Route Distinguisher: 64496:1
        MP Reach NLRI IPv4 prefix: 192.168.100.0

```

Ongoing monitoring is achieved by propagating route changes in BGP Updates and forwarding them to the monitoring station encapsulated in BMP. This includes Withdraws (MP_UNREACH_NLRI) as well as Updates (MP_REACH_NLRI).

Statistics Report messages are sent by the router if a **stats-report-interval** is defined. If there is no **stats-report-interval** configured, Statistics Report messages are not generated. If a **stats-report-interval** is configured, then Statistics Report messages are generated at the interval defined for each monitored peer. The statistics include those listed in *Table 13-2*, where type 8 (Number of routes in Loc-RIB) and 9 (Number of routes in per-AFI/SAFI Adj-RIB-In) are per-Address Family negotiated by the peer. *Debug 13-3* shows an example of a Statistics Report message generated for peer 192.0.2.3. Although there are only 13 statistic types of which 12 are supported by SR-OS, the report contains a total of 39 statistics counters with per-AFI/SAFI counters for VPN-IPv4, VPN-IPv6, and the default IPv4.

Debug 13-3: Statistics Report message

```

1146 2025/10/09 16:25:12.644 BST minor: DEBUG #2001 Base BMP
BMP: PKT
Station: ubuntu2-dot29 - STATISTICS, 568 bytes
Peer: 192.0.2.3 (G) - flags 0x00, asn 64496, bgp-id 192.0.2.3
Sent at: 2025/10/09 16:25:12.644 BST
Stats: 39 counters
duplicate withdrawals rcvd: 0                cluster-list loops: 0
as-path loops: 0                            invalid originator: 0
as-confed loops: 0                          updates treat as withdraw: 0
duplicate prefixes rcvd: 0                   IPV4 in adj-rib-in: 3
IPV4 in loc-rib: 0                          IPV6 in adj-rib-in: 1
IPV6 in loc-rib: 0                          VPN_IPV4 in adj-rib-in: 4
VPN_IPV4 in loc-rib: 0                      VPN_IPV6 in adj-rib-in: 2
VPN_IPV6 in loc-rib: 0                      L2VPN in adj-rib-in: 5

```


closed (reason 0), unspecified reason (reason 1), out of resources (reason 2), redundant connection (reason 3), and session permanently administratively closed (reason 4).

Output 13-59: Termination message

```
25 2025/10/09 13:32:22.794 BST minor: DEBUG #2001 Base BMP
BMP: PKT
  Station: ubuntu2-dot29 - TERMINATION, 35 bytes
    string: rr1 says: Bye bye !
    reason: admin shutdown (0)
```

BMP is an extremely powerful tool that provides operators with visibility of how their BGP control plane is performing. It can be used to identify issues with flapping prefixes, monitor route-table sizes, perform trending analysis, and much more. The unseen can be seen with BMP.

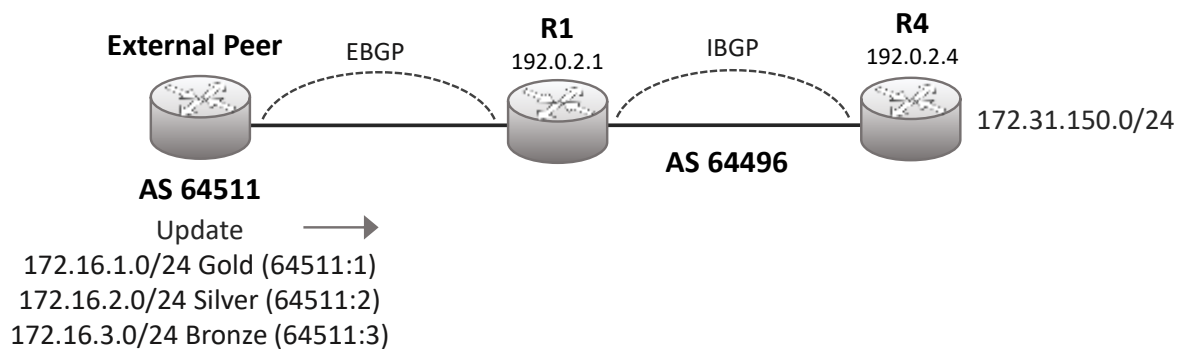
QoS Policy Propagation using BGP (QPPB)

QoS Policy Propagation using BGP (QPPB) is a feature that allows a route to be installed in the route-table with an associated Per-Hop-Behaviour Group (internally referred to as a Forwarding Class, or FC) and priority so that packets matching the route receive the relevant QoS treatment. Using QPPB the QoS class that should be associated with a given prefix is typically signalled using a Community attribute, although any path characteristics of a BGP Update message (such as AS_PATH) can be used if it can be referenced in route policy, where the relevant FC and priority are configured as an ‘action’ within each entry. Although the same behaviour can be achieved using regular QoS policies, QPPB removes some of the operational overhead associated with manually updating IP ‘match’ criteria associated with those policies.

An example application of QPPB is inter-AS co-ordination of QoS policies, where an operator in AS ‘*m*’ may signal to a peer in AS ‘*n*’ that traffic destined towards prefix ‘P1’ should receive ‘Gold’ QoS treatment, but traffic destined towards prefix ‘P2’ should receive ‘Bronze’ QoS treatment. Another application may be where an operator wants to provide differentiated QoS treatment to different traffic flows with known source/destination routes within its own administrative boundary.

To illustrate the use of QPPB a simple and hypothetical scenario is used as shown in *Figure 13-14* where routers R1 and R4 are in AS 64496 and R1 is connected to an external peer in AS 64511. Router R4 advertises prefix 172.31.150.0/24 whilst AS 64511 advertises three prefixes and has an arrangement with AS 64496 that traffic sourced from those prefixes will receive differential QoS treatment within that AS. Prefix 172.16.1.0/24 is tagged with community 64511:1 indicating ‘Gold’ QoS treatment. Prefix 172.16.2.0/24 is tagged with community 64511:2 indicating ‘Silver’ QoS treatment, and prefix 172.16.3.0/24 is tagged with community 64511:3 indicating ‘Bronze’ treatment.

Figure 13-14: QPPB Source-Lookup



Firstly, a simple QoS policy is configured at R1 and applied on the ingress from the external peer at AS 64511. This is simply for traffic differentiation and would have to be applied at all potential router hops if the Per-Hop-Behaviour (PHB) characteristics were to be maintained throughout the Autonomous System. The objective is simply to show how the mechanism operates, so it is applied at ingress only and consists of three queues: queue 1 for Best Effort (BE), queue 2 for Assured Forwarding (AF), and queue 3 for Expedited Forwarding (EF). When traffic has been classified into the appropriate FC/priority at ingress using QPPB, that marking is retained throughout the domain and subsequently used for FC to queue mapping on any egress interfaces.

Output 13-60: SAP-ingress QoS policy at R1 towards AS 64511

```

qos {
  sap-ingress "Classify-QPPB" {
    policy-id 10
    queue 1 {
      queue-type best-effort
    }
    queue 2 {
      queue-type best-effort
    }
    queue 3 {
      queue-type expedited
    }
    fc "af" {
      queue 2
    }
    fc "be" {
      queue 1
    }
    fc "ef" {
      queue 3
    }
  }
}

```

At the service level, the QoS policy is applied at SAP ingress. In addition, QPPB is enabled at the interface level using the **qos-route-lookup** command followed by the **source** or **destination** arguments, in this case **source**. This tells the system to lookup the source (or destination) IP address in the forwarding table associated with the routing context (base or IP-VRF), and if the longest match prefix in the forwarding table has an associated FC and priority, to set the FC and priority based on that match. In the event the packet matches both

QPPB **qos-route-lookup** criteria and some other statically configured criteria, the QPPB-derived classification overrides any statically configured criteria such as DSCP/IP precedence/802.1p.

Output 13-61: QPPB configuration

```

service {
  ies "EBGP-to-AS64511" {
    interface "EBGP-to-AS64511" {
      sap 1/1/c3/1:5 {
        ingress {
          qos {
            sap-ingress {
              policy-name "Classify-QPPB"
            }
          }
        }
      }
    }
    ipv4 {
      qos-route-lookup source
    }
  }
}

```

Finally, the route policy is configured matching standard community values, setting the appropriate FC and priority, and assigning each FC to a queue. As previously indicated, any BGP path characteristic that can be referenced in policy can be used for QPPB classification.

Output 13-62: Example route policy for QPPB

```

policy-options {
  community "AS64511-Bronze" {
    member "64511:3" { }
  }
  community "AS64511-Gold" {
    member "64511:1" { }
  }
  community "AS64511-Silver" {
    member "64511:2" { }
  }
  policy-statement "QPPB-AS64511" {
    entry 10 {
      from {
        community {
          name "AS64511-Gold"
        }
      }
      action {
        action-type accept
        forwarding-class {
          fc ef
          priority high
        }
      }
    }
    entry 20 {
      from {
        community {
          name "AS64511-Silver"
        }
      }
      action {

```

```

        action-type accept
        forwarding-class {
            fc af
            priority high
        }
    }
}
entry 30 {
    from {
        community {
            name "AS64511-Bronze"
        }
    }
    action {
        action-type accept
        forwarding-class {
            fc be
            priority low
        }
    }
}
}
}

```

The route policy is then applied at import at R1 against the external peer in AS 64511.

Output 13-63: Applying policy as import towards AS 64511 at R1

```

router "Base" {
    bgp {
        group "EBGP-to-AS64511" {
            peer-as 64511
            split-horizon true
            import {
                policy ["QPPB-AS64511"]
            }
        }
        neighbor "192.168.2.2" {
            group "EBGP-to-AS64514"
            authentication-key "JK/6NxCqPrX7NU7cD3w1PgntbBRVTQM= hash2"
            family {
                ipv4 true
            }
        }
    }
}

```

Output 13-64 shows how the QPPB configuration applied at R1 can be verified for each prefix (the “show router bgp routes <prefix> detail” command can be used for the same purpose).

Output 13-64: Route-table QoS settings

```
A:admin@R1# show router route-table 172.16.0.0/22 longer qos
```

```

=====
Route Table (Router: Base)
=====
Dest Prefix[Flags]                                Type   Proto   Age           Pref
Next Hop[Interface Name]                          Metric
QoS
-----
172.16.1.0/24                                     Remote BGP      00h01m33s    170
    192.168.1.2                                     0
    ef, high
172.16.2.0/24                                     Remote BGP      00h01m33s    170
    192.168.1.2                                     0

```

```

        af, high
172.16.3.0/24                                Remote  BGP      00h01m33s  170
        192.168.1.2                          0
        be, low
-----
No. of Routes: 3
Flags: n = Number of times nexthop is repeated
      B = BGP backup route available
      L = LFA nexthop available
      S = Sticky ECMP requested
=====

```

To verify that the QPPB classification is functioning correctly a ping is initiated from AS 64511 to the destination prefix 172.31.150.0/24 connected to R4. The source address used for the ping is from the 172.16.1.0/24 range, which was previously advertised with the Gold community 64511:1 and classified as FC EF at R1. *Output 13-66* shows the ingress QoS statistics at R1 after completion of the ping test, showing that all traffic was correctly mapped into queue 3.

Output 13-65: Generate 'Gold' traffic from AS 64511 towards R4

```

*A:AS64511# ping router 5 172.31.150.1 source 172.16.1.1 count 100
PING 172.31.150.1 56 data bytes
64 bytes from 172.31.150.1: icmp_seq=1 ttl=63 time=4.88ms.
64 bytes from 172.31.150.1: icmp_seq=2 ttl=63 time=4.02ms.
64 bytes from 172.31.150.1: icmp_seq=3 ttl=63 time=4.15ms.
--- [snip] ---
64 bytes from 172.31.150.1: icmp_seq=99 ttl=63 time=4.20ms.
64 bytes from 172.31.150.1: icmp_seq=100 ttl=63 time=4.14ms.

---- 172.31.150.1 PING Statistics ----
100 packets transmitted, 100 packets received, 0.00% packet loss
round-trip min = 2.89ms, avg = 4.14ms, max = 6.82ms, stddev = 0.520ms

```

Output 13-66: SAP-Ingress QoS statistics with QPPB classification at R1

```

A:admin@R1# show service id 5 sap 1/1/c3/1:5 detail | match "Ingress Queue" post-lines 6
Ingress Queue 1 (Unicast) (Priority)
Off. HiPrio           : 0                      0
Off. LowPrio          : 0                      0
Dro. HiPrio           : 0                      0
Dro. LowPrio          : 0                      0
For. InProf           : 0                      0
For. OutProf          : 0                      0
Ingress Queue 2 (Unicast) (Priority)
Off. HiPrio           : 0                      0
Off. LowPrio          : 0                      0
Dro. HiPrio           : 0                      0
Dro. LowPrio          : 0                      0
For. InProf           : 0                      0
For. OutProf          : 0                      0
Ingress Queue 3 (Unicast) (Priority)
Off. HiPrio           : 100                    10600
Off. LowPrio          : 0                      0
Dro. HiPrio           : 0                      0
Dro. LowPrio          : 0                      0
For. InProf           : 0                      0

```

QPPB is supported for the IPv4/IPv6 and Label-IPv4/IPv6 Address Families.

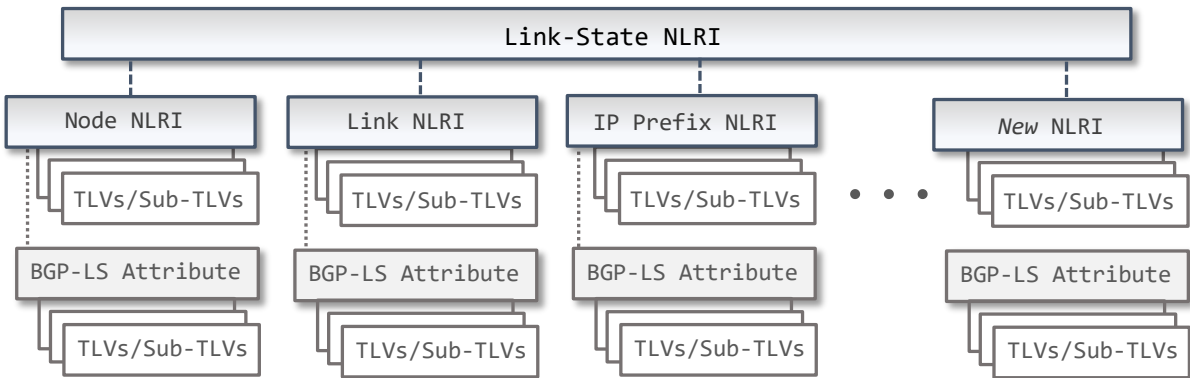
Link State Information Distribution using BGP

The contents of a link-state database (such as topology, metrics, reservable bandwidth, SRLGs, etc.) are in general constrained to an IGP area. If a router has a requirement to compute an end-to-end path across multiple IGP areas (or levels) the Traffic Engineering Database (TED) contains the TE attributes only for the area that the computing router belongs to. The computing router has no visibility of the topology of the adjacent area(s). The intention of the BGP Link-State Address Family [69] is to distribute link and node information of a network topology between BGP speakers. The purpose is to provide real-time Traffic Engineering information to BGP speakers in other Autonomous Systems, or more likely centralised policy servers such as an Application Layer Traffic Optimisation (ALTO) Server or Path Computation Element (PCE) that could make informed and optimal end-to-end path computation decisions regardless of inter-AS or inter-area boundaries. It can also be used by centralised Route-Reflectors that implement Optimal Route Reflection (ORR). A BGP Link-State (BGP-LS) speaker essentially extracts network topology information together with its associated traffic engineering parameters learnt through the IGP and encodes it into Link-State NLRI. The original specification described in [69] defined four types of NLRI: Node NLRI, Link NLRI, IPv4 Topology Prefix NLRI, and IPv6 Topology Prefix NLRI:

- Node NLRI use Node Descriptor TLVs to encode local nodal parameters such as IGP Router-ID and AS number. An optional non-transitive BGP attribute known as the BGP-LS attribute is attached to the NLRI to encode additional node attributes in TLVs such as Node Name, IPv4/IPv6 Router-ID, and Multi-Topology Identifier.
- Link NLRI use local/remote Node Descriptor TLVs to encode information about the routers either side of a given link together with Link Descriptor TLVs to encode link information such as local and remote link identifiers, and IP addressing. More accurately, Link Descriptor TLVs describe one side of a network link such that two Link NLRIs are required in order to fully describe a point-to-point link. The BGP-LS attribute is again used to encode optional link attributes such as TE information.
- IPv4 and IPv6 Topology Prefix NLRI encode IP reachability information and the protocol through those prefixes are learned.

While the above four Link-State NLRI types remain the prevalent NLRI types, whenever new objects and TLVs are added to BGP or IS-IS/OSPF, BGP-LS also requires the subsequent extension if the information needs to be propagated northbound to a controller. These extensions may be additional NLRI TLVs/sub-TLVs, BGP-LS Attribute TLVs/sub-TLVs, or indeed entirely new NLRI types. *Figure 13-15* shows the structure of the Link State NLRI with its associated TLVs and the BGP-LS attribute.

Figure 13-15: Link State NLRI structure



Among the more recent extensions to BGP Link-State are the extensions for Segment Routing (SR) [70], SRv6 [71], and the extensions for Flexible Algorithm [72] together with its corresponding Application-Specific Link attributes [73].

Link-State NLRI

All non-VPN link, node, and prefix information uses AFI 16388 and SAFI 71, while VPN link, node, and prefix information is encoded using AFI 16388 and SAFI 72. SR-OS currently only supports the non-VPN AFI 16388 SAFI 71. The format of the Link-State NLRI is shown in Figure 13-16. Below it, Table 13-3 lists the possible NLRI types encoded in the first two octets of the Link-State NLRI.

Figure 13-16: Link-State NLRI format

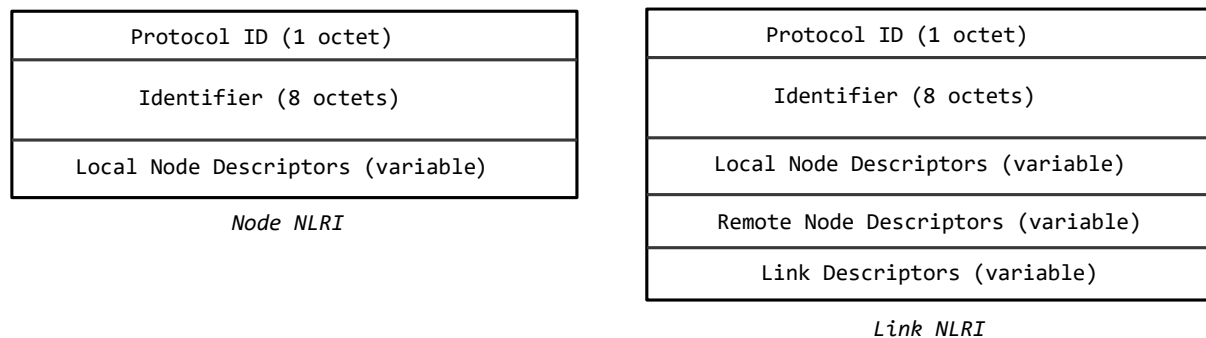
NLRI Type (2 octets)	Total NLRI (2 octets)	NLRI Type (2 octets)	Total NLRI (2 octets)
Link-State NLRI (variable)		Route Distinguisher	
Link-State AFI 16388 / SAFI 71		Link-State NLRI (variable)	Link-State AFI 16388 / SAFI 72

Table 13-3: Link-State NLRI types

Type	NLRI Type
1	Node NLRI
2	Link NLRI
3	IPv4 Topology Prefix NLRI
4	IPv6 Topology Prefix NLRI
5	SR Policy Candidate Path NLRI
6	SRv6 SID NLRI
7	Stub Link NLRI

The format of the Node NLRI (Type 1) and Link NLRI (Type 2) is shown in *Figure 13-17*. The Protocol ID identifies the protocol from which the link-state information was obtained (for example, IS-IS Level 1 or 2, OSPFv2, OSPFv3, static configuration), whilst the Instance field identifies a particular instance of that protocol (for the scenario where a router is running multiple instances). The Node and Link Descriptor fields contain information identifying a given node/link encoded as TLVs.

Figure 13-17: Node and Link NLRI format



The Local Node Descriptors field in the Node NLRI contains sub-TLVs used to describe the node's Router ID and the Autonomous System to which the router belongs. The BGP-LS Identifier is a 32-bit identifier that in conjunction with the AS number uniquely identifies the BGP-LS domain and must be globally unique. All BGP-LS speakers within scope of an IGP domain must use the same {AS number, BGP-LS ID} tuple.

Table 13-4: Node Descriptor sub-TLVs

Code-Point	Sub-TLV
512	Autonomous System
513	BGP-LS Identifier
514	OSPF Area-ID
515	IGP Router-ID

The Link NLRI has similar Local and Remote Node Descriptor TLVs containing sub-TLVs that are used to describe the pair of routers that ‘*anchor*’ a given link. Many types of Router-ID formats are possible, but there must be at least one “like” Router-ID pair of a Local Node Descriptor and Remote Node Descriptor per-protocol. The Link Descriptor TLVs uniquely identify a link between two routers, where both sides of the link must advertise a corresponding NLRI for it to be considered valid. In other words, a link described by the link Descriptor TLVs is actually a “half-link”, and in order to fully describe a single logical link, two originating routers advertise a “half-link” each. Hence, two Link NLRIs are advertised for a given point-to-point link.

The Multi-Topology Identifier carries one or more IS-IS or OSPF Multi-Topology IDs for a link, node, or prefix.

Table 13-5: Link Descriptor sub-TLVs

Code-Point	Sub-TLV
258	Link Local/Remote Identifiers
259	IPv4 interface address
260	IPv4 neighbour address
261	IPv6 interface address
262	IPv6 neighbour address
263	Multi-topology Identifier

The format of the IPv4 and IPv6 Prefix NLRI (Type 3 and Type 4) use the same format as shown in *Figure 13-18*. The Prefix Descriptor TLVs uniquely identify an IPv4 or IPv6 prefix originated by a node and use the sub-TLVs listed in *Table 13-6*. The IP Reachability Information field contains one IPv4 or IPv6 address originally advertised in the IGP topology by the advertising node. It serves to correlate a BGP service NLRI to a given node in the link state database by virtue of its BGP Next-Hop address. Hence, a given node should advertise an IP Prefix NLRI for each of its BGP Next-Hops.

Figure 13-18: IPv4/IPv6 Prefix NLRI

Protocol ID (1 octet)
Identifier (8 octets)
Local Node Descriptors (variable)
Prefix Descriptors (variable)

Table 13-6: Prefix Descriptors

Code-Point	Sub-TLV
263	Multi-Topology Identifier
264	OSPF Route Type
265	IP Reachability Information

BGP-LS Attribute

The BGP-LS attribute is an optional, non-transitive BGP attribute used to carry link, node, and prefix parameters and attributes. It is encoded as a set of TLVs and as previously described, there are multiple extensions of the base definition including, but not limited to, Segment Routing SIDs [70], SRv6 SIDs [71] as well as Application-Specific Link Attributes [73]. The code-points for BGP-LS attribute TLVs are maintained by IANA any new allocated TLVs can be found at [BGP-LS Attribute TLVs](#). Similarly, SR-OS supports numerous BGP-LS

attributes, but it would be futile to attempt to list them here as the list would very soon be out-of-date. *Table 13-7* lists the Node attribute TLVs from the original Link-State specification in [69] that may be encoded in the BGP-LS attribute associated with a Node NLRI.

The Node Flags field contains numerous optional bits to advertise node level IGP parameters such as Overload bit, Attached Bit, External bit, ABR bit, and V6 (OSPFv3) bit. If a node is part of multiple IS-IS areas then multiple TLVs are used to encode each of them. The Opaque Node attribute is used to transparently carry optional Node attribute TLVs specific to the Protocol ID field of the NLRI header. It can be used to advertise proprietary information or a newly defined link-state attributes that do not yet have the relevant BGP-LS extensions defined.

Table 13-7: Node Attribute TLVs

Code-Point	Sub-TLV
263	Multi-Topology Identifier
1024	Node Flag bits
1025	Opaque Node Attribute
1026	Node Name
1027	IS-IS Area Identifier
1028	IPv4 Router-ID of Local Node
1029	IPv6 Router-ID of Local Node

The originally specified Link attributes that may be carried in a BGP-LS attribute with a Link NLRI are listed in *Table 13-8*. Again, this list has been extended by numerous standards but serves to outline the nature of the information that is carried. Many of the original fields in the table carry Traffic Engineering (TE) attributes originally defined for OSPF [74] and IS-IS [75]. Extensions to the attributes defined below include the ability to carry TE Metric information for IS-IS [76] and OSPF [77], Segment Routing SIDs [70], SRv6 SIDs [71] as well as Application-Specific Link Attributes [73].

Table 13-8: Link Attribute TLVs

Code-Point	Sub-TLV
1028	IPv4 Router-ID of Local Node
1029	IPv6 Router-ID of Local Node
1030	IPv4 Router-ID of Remote Node
1031	IPv6 Router-ID of Remote Node
1088	Administrative Group
1090	Max. reservable link bandwidth
1091	Unreserved bandwidth
1092	TE Default Metric
1093	Link Protection Type

1094	MPLS Protocol Mask
1095	IGP Metric
1096	Shared Risk Link Group
1097	Opaque Link Attribute
1098	Link Name

Prefixes may be learned from the IGP topology with a set of IGP attributes such as metric, route-tags, and numerous extensions to prefix attributes that must be advertised into the BGP-LS attribute with a Prefix NLRI (Type 3 and 4). The IGP Flags field contains a number of prefix flags such as IS-IS Up/Down bit, OSPF ‘no unicast’ bit, OSPF ‘local address’ bit, and OSPF ‘propagate NSSA’ bit. The Route Tag and Prefix Metric fields are self-explanatory. The Opaque Prefix Attribute TLV allows a router to encode information specific to the protocol advertised in the NLRI header Protocol ID field. It’s primary use is to allow for encoding of new prefix attributes before the relevant protocol extensions to BGP-LS have been defined.

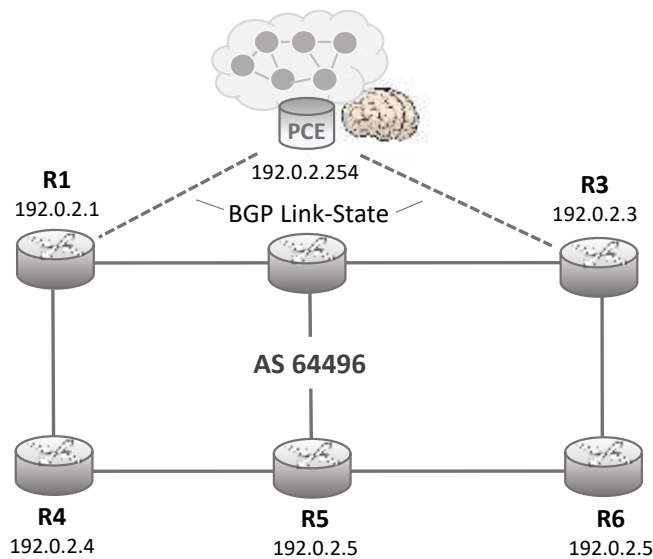
Table 13-9: Prefix Attributes TLV

Code-Point	Sub-TLV
1152	IGP Flags
1153	IGP Route Tag
1154	IGP Extended Route Tag
1155	Prefix Metric
1156	OSPF Forwarding Address
1157	Opaque Prefix Attribute

Configuration and Operation

The topology in *Figure 13-19* is used to demonstrate the use of BGP-LS. Routers R1 to R6 are in a non-hierarchical IS-IS Level 2 network belonging to Autonomous System 64496. The network is dual-stack IPv4/IPv6 and is configured to use both SR-MPLS and SRv6 in addition to Flex-Algorithm with the associated Application-Specific Link Attributes (ASLAs). Routers R1 and R3 peer in BGP-LS with a Path Computation Element (PCE) that is used to calculate traffic engineered LSPs across the network whilst adhering to the required metrics and objectives of each calculated path. The topology of the network is largely irrelevant as the objective is not to show that BGP-LS passes all the relevant node, link, and prefix NLRI to the PCE – that can largely be assumed. The objective is merely to illustrate the configuration requirements for BGP-LS and the structure of each of these NLRI formats as they are advertised.

Figure 13-19: Topology for BGP-LS



Output 13-67 shows the configuration applied at routers R1 and R3 to enable advertisement of link-state information in BGP-LS towards the PCE. Under the ISIS or OSPF context, the **database-export** context is used to populate the extended TE database with specific parameters prior to export in BGP-LS. Within this context there are **igp-identifier** and **bgp-ls-identifier** parameters. The **igp-identifier** parameter is used to indicate the particular IGP instance if multiple instances are in use. The default value is zero, and as the test topology uses instance zero this parameter is not shown. The **bgp-ls-identifier** is used to uniquely identify the BGP-LS domain and all BGP-LS speakers within the flooding scope of the IGP must use the same BGP-LS identifier. The use of the BGP-LS identifier has been deprecated by [78], however, implementations should continue to support it for backward compatibility. SR-OS uses a default value of zero, and [78] suggests this is the recommended value if this sub-TLV is advertised when advertising information into BGP-LS.

Under the BGP context, the **link-state-route-import true** command imports the contents of the extended TE database into the BGP RIB. The Address Family **bgp-ls** is added on the peering session towards the PCE (192.0.2.254), after which the contents of the imported routes will be advertised in BGP.

Output 13-67: Configuration of BGP-LS

```
router "Base" {
  isis 0 {
    database-export {
      bgp-ls-identifier {
      }
    }
  }
  bgp {
    link-state-route-export true
    group "Northbound-to-PCE" {
      peer-as 64496
      family {
        bgp-ls true
      }
    }
  }
}
```

```

neighbor "192.0.2.254" {
    group "Northbound-to-PCE"
    authentication-key "C3i5c6IP3VMhQFGff3RCM3Ptgr+/+Uw= hash2"
}
}

```

By checking the BGP summary at R1 it can be observed that there are 120 Update messages sent towards the PCE in BGP-LS in the short space of time that the BGP session has been established. This represents all of the node, link, and prefix NLRI within the network topology that R1 has taken from the IGP TE database.

Output 13-68: Link State BGP towards the PCE at R1

```

A:admin@R1# show router bgp summary
=====
BGP Router ID:192.0.2.1          AS:64496          Local AS:64496
=====
BGP Admin State      : Up          BGP Oper State      : Up
Total Peer Groups    : 5           Total Peers          : 38
Total VPN Peer Groups : 0           Total VPN Peers      : 0
Current Internal Groups : 7         Max Internal Groups  : 7
Total BGP Paths       : 155        Total Path Memory    : 62968

--- [snip] ---
Total Link State Rem Rts: 120      Total Link State Rem Act Rts: 0

=====
BGP Summary
=====
Legend : D - Dynamic Neighbor
=====
Neighbor
Description
          AS PktRcvd InQ Up/Down  State|Rcv/Act/Sent (Addr Family)
          PktSent OutQ
-----
--- [snip] ---
192.0.2.254
          64496   2517   0 20h57m18s 0/0/120 (LinkState)
          2626   0
-----

```

Output 13-69 shows the output of a Link-State Node NLRI Update advertised by R1 towards the PCE, and starts with the NLRI fields Protocol and Identifier (*Figure 13-17*), followed by a number of Local Node Descriptors from *Table 13-4* including Autonomous System number represented in dotted decimal (0.0.251.240 equates to 64496) and IGP Router-ID (0x192000002002 is router R2). The generic BGP attributes are then followed by the Link-State attribute TLVs, which include values from *Table 13-7*, such as Node Flags, Node Name, IS-IS Area Identifier, and IPv4 and IPv6 Router-IDs. There are also a number of more recent Node attribute TLVs such as SR Capabilities, SR Algorithm, SRv6 Capabilities, S-BFD Discriminator, and Node Maximum Segment Depth (MSD).

Output 13-69: BGP-LS Node NLRI for router R2 advertised by R1

```

A:admin@R1# show router bgp routes bgp-ls hunt node
=====
BGP Router ID:192.0.2.1          AS:64496          Local AS:64496
=====
Legend -

```

```

Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge
Origin codes  : i - IGP, e - EGP, ? - incomplete

=====
BGP-LS Node NLRIs
=====

-----
RIB Out Entries
-----

--- [snip] ---
Network:
Type           : NODE-NLRI
Protocol        : ISIS Level-2          Identifier      : 0x0
Local Node descriptor:
  Autonomous System : 0.0.251.240
  Link State Id      : 0
  IGP Router Id      : 0x192000002002
Nexthop         : 192.0.2.2
To               : 192.0.2.254
Res. Nexthop     : n/a
Local Pref.      : 100                  Interface Name : NotAvailable
Aggregator AS    : None                 Aggregator     : None
Atomic Aggr.     : Not Atomic           MED            : None
AIGP Metric      : None                 IGP Cost       : n/a
Connector        : None
Community        : No Community Members
Cluster          : 192.0.2.10
Originator Id    : 192.0.2.2            Peer Router Id : 192.0.2.254
Origin           : IGP
AS-Path          : No As-Path
Route Tag        : 0
Neighbor-AS      : n/a
DB Orig Val      : N/A                  Final Orig Val  : N/A
Source Class     : 0                    Dest Class      : 0
-----

Link State Attribute TLVs :
Node MSD :
  MSD-type 1 MSD-value 12
  MSD-type 2 MSD-value 15
  MSD-type 41 MSD-value 10
  MSD-type 42 MSD-value 9
  MSD-type 44 MSD-value 7
  MSD-type 45 MSD-value 9
Node Flag Bits : 0x10
Node Name      : R2
IS-IS Area Identifier (length 3) : 490001
IPv4 Rtr-ID of Local Node : 192.0.2.2
IPv6 Rtr-ID of Local Node : 3fff:ffe::2
S-BFD Discriminators : 524288
SR Capabilities : Flags: 0xc0
  Label 12000 range 8000
SR Algorithm : 0 128 129
SRV6 Capabilities (length 4) : 00000000
-----

--- [snip] ---

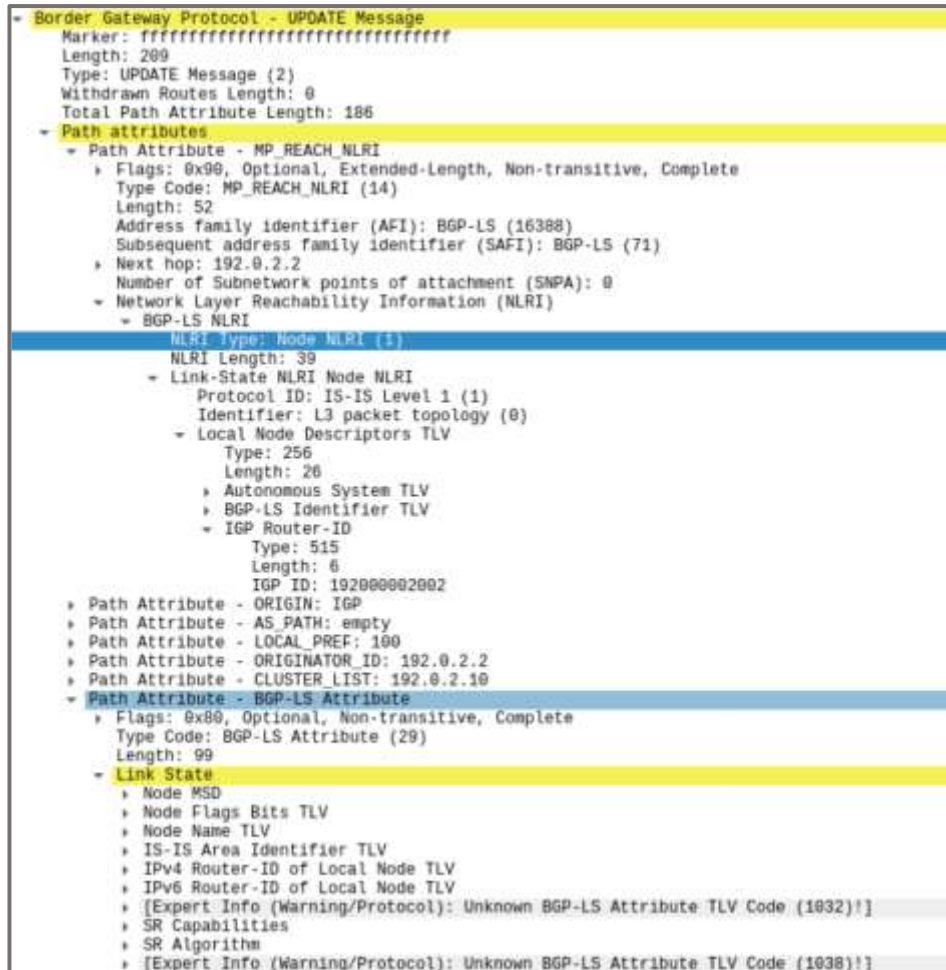
Routes : 16
=====

```

Figure 13-20 shows a packet capture of the same BGP-LS Node NLRI with the NLRI and BGP-LS attributes expanded to aid understanding of the encoding. The Node NLRI itself contains

some very basic and fundamental node-level parameters, while the BGP-LS attribute TLVs are typically much more about nodal capabilities. Note that there are two BGP-LS attribute TLVs that Wireshark failed to decode. TLV 1032 is the S-BFD Discriminators attribute, while TLV 1038 is the SRv6 Capabilities attribute.

Figure 13-20: Packet capture of Node NLRI for router R2 advertised by R1



The next example is a Link NLRI, which in this case is for the R1-R2 link. It begins with the Type field to identify it as Link NLRI (Type 2), and is followed by the Protocol ID, Identifier fields, Local/Remote Node Descriptors, and the Link Descriptors as illustrated in *Figure 13-17*. The BGP-LS attribute TLVs follow the generic BGP attribute TLVs and include a number of TLVs from *Table 13-8* including MPLS TE attributes such as Maximum reservable link bandwidth, unreserved bandwidth, and TE default metric. The attributes also include SR-MPLS (code-point 1099) and SRv6 Adjacency-SIDs (code-point 1106) as well as Application-Specific Link Attribute TLVs (code-point 1122) for Flexible-Algorithm.

Output 13-70: BGP-LS Link NLRI for link R1-R2 advertised by R1

```
A:admin@R1# show router bgp routes bgp-ls hunt link
=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
```

```

                l - leaked, x - stale, > - best, b - backup, p - purge
Origin codes   : i - IGP, e - EGP, ? - incomplete

=====
BGP-LS Link NLRIs
=====

-----
RIB Out Entries
-----

--- [snip] ---

Network:
Type           : LINK-NLRI
Protocol        : ISIS Level-2          Identifier      : 0x0
Local Node descriptor:
  Autonomous System : 0.0.251.240
  Link State Id      : 0
  IGP Router Id      : 0x192000002002
Remote Node descriptor:
  Autonomous System : 0.0.251.240
  Link State Id      : 0
  IGP Router Id      : 0x192000002001
Link descriptor:
  IPV4 Interface Addr: 192.168.0.2
  IPV4 Neighbor Addr : 192.168.0.1
  IPV6 Interface Addr: 2001:db8:33ac:2200::4:2
  IPV6 Neighbor Addr : 2001:db8:33ac:2200::4:1
Nexthop        : 192.0.2.2
To              : 192.0.2.254
Res. Nexthop    : n/a
Local Pref.     : 100
Aggregator AS   : None
Atomic Aggr.    : Not Atomic
AIGP Metric     : None
Connector       : None
Community       : No Community Members
Cluster         : 192.0.2.10
Originator Id   : 192.0.2.2
Peer Router Id  : 192.0.2.254
Origin          : IGP
AS-Path         : No As-Path
Route Tag       : 0
Neighbor-AS     : n/a
DB Orig Val     : N/A
Source Class    : 0
Interface Name  : NotAvailable
Aggregator      : None
MED             : None
IGP Cost        : n/a
Final Orig Val  : N/A
Dest Class      : 0

-----
Link State Attribute TLVs :
Maximum link bandwidth : 10000000 Kbps
Max. reservable link bandwidth : 10000000 Kbps
Unreserved bandwidth0 : 10000000 Kbps
Unreserved bandwidth1 : 10000000 Kbps
Unreserved bandwidth2 : 10000000 Kbps
Unreserved bandwidth3 : 10000000 Kbps
Unreserved bandwidth4 : 10000000 Kbps
Unreserved bandwidth5 : 10000000 Kbps
Unreserved bandwidth6 : 10000000 Kbps
Unreserved bandwidth7 : 10000000 Kbps
TE Default Metric : 100
IGP Metric : 100
Adjacency Segment Identifier (Adj-SID) : flags 0x70 weight 0 label 524284
Adjacency Segment Identifier (Adj-SID) : flags 0xf0 weight 0 label 524283
SRV6 End.X SID : Flags 0x80 Algo 0 weight 0 epBhvr 53 sid 3fff:3ea:2:e013::
SRV6 SID Structure : lbLen 32 lnLen 16 funLen 16 argLen 64
Unidirectional Link Delay Min/Max : MIN 100000 MAX 100000

-----
Application Specific Link Attribute TLVs :
SAIBML 4 UDAIBML 0

```


The final example is an IPv4 Prefix NLRI, again advertised by router R1 and this time for the prefix 192.0.2.4/32 representing router R4's system IP address. Following the Type field (Type 3) the Protocol ID and Identifier fields are part of the NLRI, and this is followed the Local Node Descriptor TLV and the Prefix Descriptor TLV shown in *Figure 13-18* with an address and mask of 192.0.2.4/32. The Link-State attribute TLVs include the IGP Flags and the Prefix Metric from *Table 13-9*, as well as two Prefix-SIDs (code-point 1158) and a Prefix Attribute Flags TLV (code-point 1157).

Output 13-71: IPv4 Prefix NLRI advertised for R4's System IP by R1

```
A:admin@R1# show router bgp routes bgp-ls hunt ipv4-prefix
=====
BGP Router ID:192.0.2.1          AS:64496          Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP-LS Ipv4 NLRIs
=====

-----
RIB Out Entries
-----

--- [snip] ---

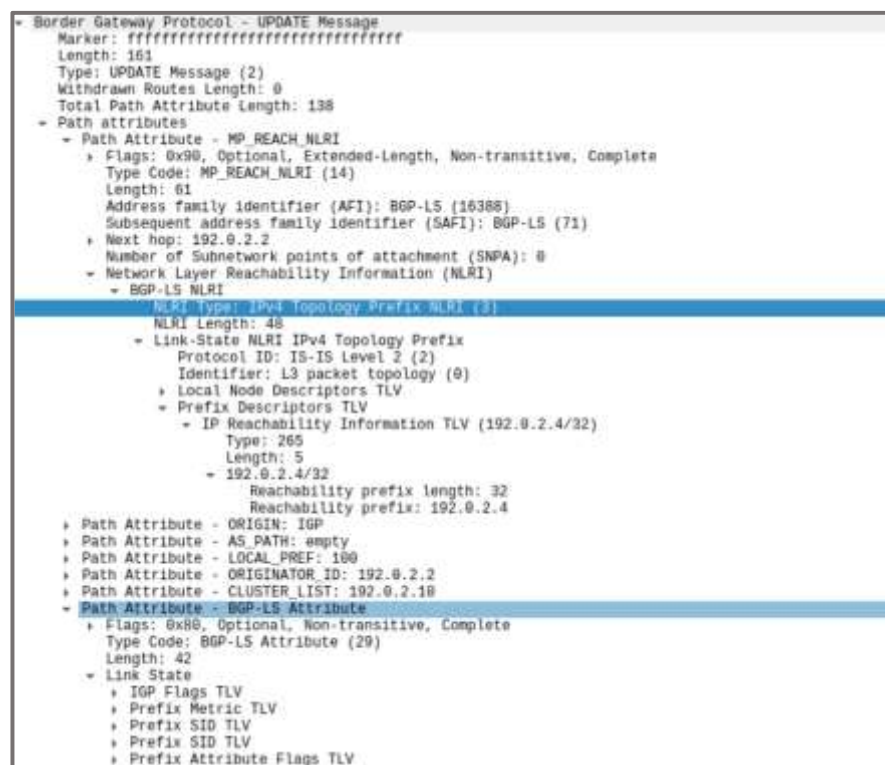
Network:
Type           : IPv4-Pfx-NLRI
Protocol       : ISIS Level-2          Identifier      : 0x0
Local Node descriptor:
  Autonomous System : 0.0.251.240
  Link State Id      : 0
  IGP Router Id      : 0x192000002004
Prefix descriptor:
  IPv4-Addr/Pref-Len : 192.0.2.4/32
Nexthop        : 192.0.2.2
To              : 192.0.2.254
Res. Nexthop    : n/a
Local Pref.     : 100
Interface Name  : NotAvailable
Aggregator AS   : None                 Aggregator      : None
Atomic Aggr.    : Not Atomic           MED             : None
AIGP Metric     : None                 IGP Cost        : n/a
Connector       : None
Community       : No Community Members
Cluster         : 192.0.2.10
Originator Id   : 192.0.2.2            Peer Router Id  : 192.0.2.254
Origin          : IGP
AS-Path         : No As-Path
Route Tag       : 0
Neighbor-AS     : n/a
DB Orig Val     : N/A                  Final Orig Val  : N/A
Source Class    : 0                    Dest Class      : 0
-----
Link State Attribute TLVs :
IGP Flags (length 1) : 00
Prefix Metric (length 4) : 00000000
Prefix SID : flags 0x60 algorithm 0 sid 2004
Prefix SID : flags 0x60 algorithm 128 sid 5004
IGP Prefix Attributes (length 1) : 30
```

```
-----
--- [snip] ---
-----
```

```
Routes : 58
=====
```

Once again a packet capture of the same IPv4 Prefix NLRI is shown in *Figure 13-22*, once again expanded to show the NLRI and BGP-LS attribute TLVs. The IPv4 Topology Prefix NLRI shows the IP Reachability Information TLV encoded within the Prefix Descriptors TLV as shown in *Figure 13-18* and *Table 13-6*, and as one would expect, the contents are encoded as a prefix and length. The BGP-LS attribute TLV shows the attributes previously described.

Figure 13-22: Packet capture of IPv4 Prefix NLRI for R4's System IP advertised by R1



Advertising IPv4 Reachability over an IPv6 Core

When advertising Network Layer Reachability Information (NLRI) using Multi-Protocol BGP, the specification [6] assumes that the Address Family of the Next-Hop is determined by the AFI/SAFI of the advertised MP_REACH_NLRI or MP_UNREACH_NLRI message. In other words, IPv4 unicast/multicast, labeled IPv4, and VPN-IPv4 NLRI should have an IPv4 Next-Hop, and IPv6 unicast/multicast, labeled IPv6, and VPN-IPv6 NLRI should have an IPv6 Next-Hop.

The ability to carry IPv6 NLRI over an IPv4 network has existed for many years in the form of BGP/MPLS IP-VPN (VPN-IPv6) and 6PE, both of which use an IPv4-mapped IPv6 address to ensure that the Next-Hop is of the same Address Family as the advertised NLRI. However, in order to carry IPv4 NLRI with an IPv6 Next-Hop there is clearly no way to map an IPv6 address inside an IPv4 address, hence some extensions to Multi-Protocol BGP were required. This

came in [66], which specified that the address of the Next-Hop for IPv4 NLRI or VPN-IPv4 NLRI may be of the IPv4 or IPv6 Address Family, but the capability to exchange this form of encoding between peers required negotiation. Consequently, a new BGP Capability known as the Extended Next Hop Encoding Capability (capability code 5) was introduced, the purpose of which was to allow MP-BGP peers to negotiate and discover whether they can exchange IPv4 NLRI and VPN-IPv4 NLRI with an IPv6 Next-Hop with a peer. The format of the Extended Next Hop Encoding capability is simple and consists of one or more {AFI, SAFI, Next-Hop AFI} tuples as shown in *Figure 13-23*.

Figure 13-23: Extended Next Hop Encoding Capability

NLRI AFI – 1 (2 octets)
NLRI SAFI – 1 (2 octets)
Nexthop AFI – 1 (2 octets)
.....
NLRI AFI – 1 (2 octets)
NLRI SAFI – 1 (2 octets)
Nexthop AFI – 1 (2 octets)

As the specification is only concerned with the advertisement of IPv4 and VPN-IPv4 NLRI with an IPv6 Next-Hop, the only allowed values in the Next Hop Encoding capability are as follows:

- NLRI AFI = 1 (IPv4)
- NLRI SAFI = 1 (Unicast), 2 (Multicast), 4 (NLRI with MPLS label), or 128 (MPLS-labeled VPN address)
- Nexthop AFI = 2 (IPv6)

The use-case for this form of encoding could potentially be a network transitioning to IPv6 (or dual-stack IPv4/IPv6), a greenfield deployment running IPv6 only (perhaps in conjunction with SRv6), or simply some IPv4 islands in a predominantly IPv6 network. The necessary configuration to enable advertisement of IPv4 and VPN-IPv4 with an IPv6 Next-Hop is shown in *Output 13-72* and can be applied at base BGP level, group level, or neighbour level. The **extended-nh-encoding** command instructs the system to send an Extended Next Hop Encoding capability in the Open message sent to the relevant peer(s) to which the command is applicable during session negotiation. Any peer should not send IPv4 or VPN-IPv4 NLRI with an IPv6 Next-Hop unless it has received this capability. In this example both **vpn-ipv4** and **ipv4** are set to **true**, so the capability message should contain two {AFI, SAFI, Nexthop AFI} tuples. The **advertise-ipv6-next-hops** command instructs the system to advertise the configured Address Families with an IPv6 address when originating a route or imposing **next-hop-self** on a route. It also allows export policies to change the Next-Hop of an configured Address Family with an IPv6 address. The example allows the advertisement of **evpn**, **vpn-ipv4**, and **ipv4** Address Families with an IPv6 Next-Hop. The EVPN Address Family is not within

the scope of [65] and therefore does not require negotiation using the Extended Next Hop Encoding capability – it may however require to be advertised with an IPv6 Next Hop and is therefore a configuration option. As one would expect, the **advertise-ipv6-next-hops** command has no impact on BGP sessions established with IPv4 transport. Finally, although it may seem obvious it's worth stating that the relevant IPv4 Address Families need to be enabled under the IPv6 peering session before the system will send the Extended Next Hop Encoding capability.

Output 13-72: Configuration for IPv4 NLRI with IPv6 Next-Hop

```
router "Base" {
  bgp {
    group "IBGP-AS64496" {
      family {
        ipv4 true
        vpn-ipv4 true
        ...etc
      }
      extended-nh-encoding {
        vpn-ipv4 true
        ipv4 true
      }
      advertise-ipv6-next-hops {
        evpn true
        label-ipv4 true
        ipv4 true
      }
    }
  }
}
```

Using the above configuration, SR-OS applies the following with regard to setting of the Next-Hop address:

- If the peer to which (IPv4/Label-IPv4/VPN-IPv4) routes are being advertised uses IPv4 transport for the BGP session, the Next-Hop is the local address used to set up the session.
- If the peer to which (IPv4/Label-IPv4/VPN-IPv4) routes are being advertised uses IPv6 transport for the BGP session, and the Extended Next-Hop Encoding capability was received from the peer with the relevant {AFI, SAFI, Nexthop AFI} tuple, then the Next-Hop is the local address used to set up the session.

Debug 13-4 shows an Open message sent from a PE router using IPv6 BGP peering towards a Route-Reflector that includes the Extended Next Hop Encoding (EXT-NH-ENCODING) capability. The capability encodes support for two Address Families with the {0x0 0x1 0x0 0x1 0x0 0x2} string equating to support for IPv4 unicast Address Family, and the {0x0 0x1 0x0 0x80 0x0 0x2} string equating to support for the Labeled VPN unicast Address Family, both of which support the use of IPv6 Next-Hops.

Debug 13-4: Extended Next-Hop Capability

```
25 2025/10/16 09:58:40.344 BST minor: DEBUG #2001 Base BGP
BGP: OPEN
Peer 1: 3fff:ffe::a - Send (Passive) BGP OPEN: Version 4
  AS Num 64496: Holdtime 90: BGP_ID 192.0.2.1: Opt Length 98 (ExtOpt F)
  Opt Para: Type CAPABILITY: Length = 96: Data:
```



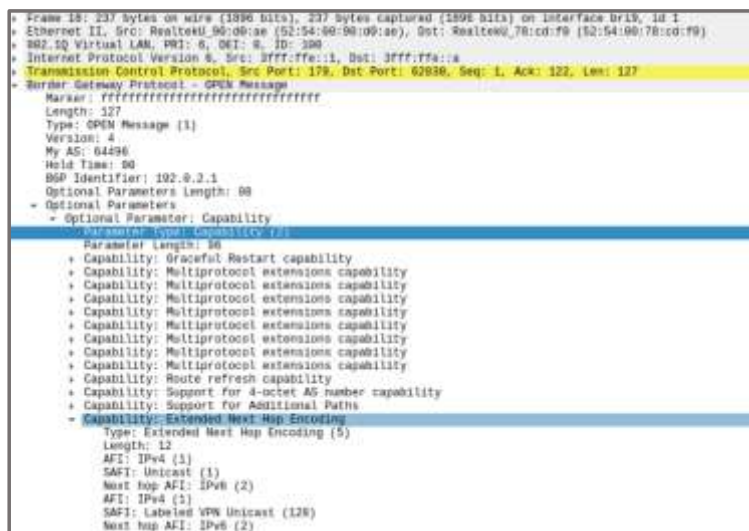
```

Cap_Code GRACEFUL-RESTART: Length 18
  Bytes: 0x40 0x78 0x0 0x1 0x1 0x0 0x0 0x2 0x1 0x0 0x0 0x1 0x80 0x0 0x0 0x2 0x80
0x0
Cap_Code MP-BGP: Length 4
  Bytes: 0x0 0x1 0x0 0x1
Cap_Code MP-BGP: Length 4
  Bytes: 0x0 0x2 0x0 0x1
Cap_Code MP-BGP: Length 4
  Bytes: 0x0 0x1 0x0 0x80
Cap_Code MP-BGP: Length 4
  Bytes: 0x0 0x2 0x0 0x80
Cap_Code MP-BGP: Length 4
  Bytes: 0x0 0x19 0x0 0x46
Cap_Code MP-BGP: Length 4
  Bytes: 0x40 0x4 0x0 0x47
Cap_Code MP-BGP: Length 4
  Bytes: 0x0 0x1 0x0 0x49
Cap_Code MP-BGP: Length 4
  Bytes: 0x0 0x2 0x0 0x49
Cap_Code ROUTE-REFRESH: Length 0
Cap_Code 4-OCTET-ASN: Length 4
  Bytes: 0x0 0x0 0xfb 0xf0
Cap_Code ADD-PATH: Length 4
  Bytes: 0x0 0x1 0x1 0x3
Cap_Code EXT-NH-ENCODING: Length 12
  Bytes: 0x0 0x1 0x0 0x1 0x0 0x2 0x0 0x1 0x0 0x80 0x0 0x2

```

Figure 13-24 shows the same Open message expanded to show the Extended Next Hop Encoding capability. As can be seen, the capability code is 5, and it carries two {AFI, SAFI, Nexthop AFI} tuples, one for IPv4 unicast (AFI 1, SAFI 1) and one for Labeled IPv4 unicast (AFI 1, SAFI 128).

Figure 13-24: Extended Next Hop Encoding capability



An indication of the capabilities negotiated during the Open exchange can be seen in *Output 13-73*, which shows both the local capabilities sent during the exchange and the remote capabilities received during the exchange.

Output 13-73: Output of BGP neighbour showing Capability support

```
A:admin@R1# show router bgp neighbor "3fff:ffe::a" | match Capability
```



```
Local Capability      : RtRefresh MPBGP 4byte ASN ExtNhEncoding
Remote Capability    : RtRefresh MPBGP 4byte ASN ExtNhEncoding
```

Lastly, after the Extended Next Hop Encoding capability is exchanged *Output 13-74* shows a VPN-IPv4 prefix advertised over an IPv6 peering session where the Next-Hop is the IPv6 address 3fff:ffe::1.

Output 13-74: VPN-IPv4 route advertised with IPv6 Next-Hop

```
A:admin@R1# show router bgp routes 172.31.1.0/24 vpn-ipv4 hunt
=====
BGP Router ID:192.0.2.1      AS:64496      Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP VPN-IPv4 Routes
=====
-----
RIB Out Entries
-----
Network       : 172.31.1.0/24
Nexthop       : 3fff:ffe::1
Route Dist.   : 64496:1      VPN Label      : 524284
Path Id       : None
To            : 3fff:ffe::a
Res. Nexthop  : n/a
Local Pref.   : 100
Aggregator AS : None        Interface Name : NotAvailable
Atomic Aggr.  : Not Atomic  Aggregator    : None
AIGP Metric   : None        MED           : None
Connector     : None        IGP Cost      : n/a
Community     : target:64496:1
Cluster       : No Cluster Members
Originator Id : None        Peer Router Id : 192.0.2.10
Origin        : IGP
AS-Path       : 64512
Route Tag     : 0
Neighbor-AS   : 64512
DB Orig Val   : N/A        Final Orig Val : N/A
Source Class  : 0          Dest Class     : 0
-----
Routes : 1
=====
```

Optimal Route Reflection (ORR)

There are three methods that operators use to create a BGP mesh within an Autonomous System (AS): full-mesh, confederations, and route reflection. Of those three, route reflection [67] is the most commonly deployed approach for route distribution within an AS. However, while the route reflection approach addresses the BGP peer scaling issue, it has the effect of summarising routing information and if the best path is selected based on IGP cost, only

reflecting its best path to its clients. The consequence of this path selection is that the choice of exit point for a route reflector and its clients will be the exit point that is preferred for the route reflector and not necessarily the exit point that is optimal for its clients. In some situations, this can lead to the selection of non-optimal paths, which is problematic in networks where the objective is for traffic to egress the network as close as possible to the point it entered the network (sometimes referred to as ‘hot potato routing’).

Route reflectors are frequently control-plane only devices, outside of any forwarding plane, and often placed in arbitrary locations in the core of medium-large networks. When multiple exit points exist for a given prefix and the best path decision comes down to IGP cost (to the BGP Next-Hop), the decision taken by the route reflector is not reflective of the best path that a client might choose. Section 11 of [67] suggests that a way to make route selection the same as it would be with a full IBGP mesh is to ensure that route reflectors are never forced to perform the BGP route selection based on IGP metrics (to the BGP Next-Hop) that are significantly different from the IGP metrics of their clients. To achieve this the solution known as Optimal Route Reflection (ORR) [68] was introduced. ORR allows the operator to specify the IGP location for which the route reflector should calculate IGP cost to the Next-Hop. This location is identified as an IP address of a node (a system or loopback address) and may be configured on a per-route-reflector basis, per set of clients, or on a per-client basis. Typically, a set of clients are associated with each defined location that are within reasonably close IGP proximity to that location IP address. The route reflector will thereafter base its calculation of IGP cost to the Next-Hop not from the perspective of itself, but rather from the perspective of the position of the location or locations. This allows the route reflector to select and advertise to a given set of clients BGP routes with the shortest distance to the Next-Hop(s) from the position of the selected IGP location.

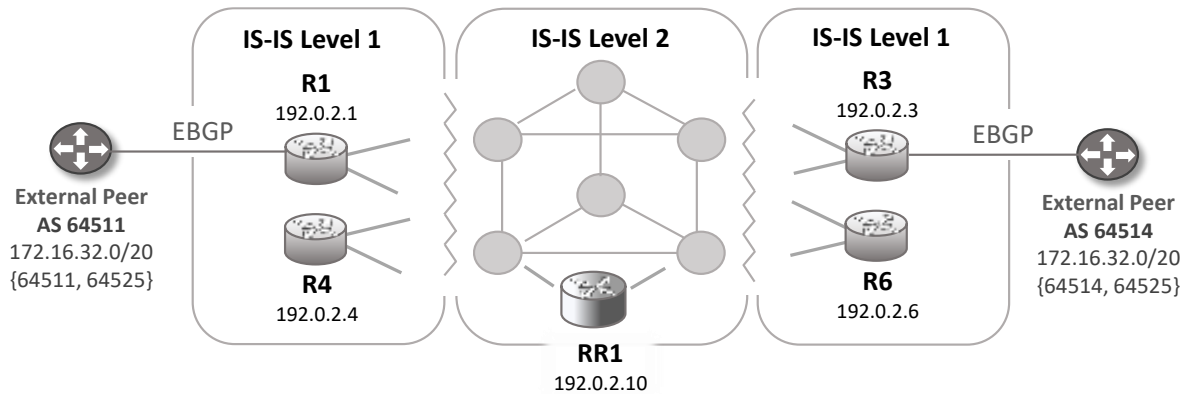
The ORR specification therefore makes two significant changes to the base BGP path selection process:

- The first is related to the IGP cost to the BGP Next-Hop, which with the ORR solution uses the IGP cost from a different IGP location than the route reflector itself.
- The second change is to extend the granularity of the BGP decision process to allow for running multiple decision processes using different perspectives (or policies). That is, the route reflector will run multiple subsets of the BGP decision process (one for each configured location).

The topology used in *Figure 13-25* is used to illustrate both the problem and the solution. The topology represents a multi-level IS-IS domain belonging to a single Autonomous System; AS 64496. Routers R1 and R4 belong to one Level 1 domain, while routers R3 and R6 belong to a second Level 1 domain. Both are connected to the Level 2 core domain via Level 1/2 routers and loopback prefixes are leaked between Level 2 and Level 1. Routers R1, R3, R4, and R6 are IBGP clients of the route reflector, RR1, which is situated in the core Level 2 area. As a result, RR1 has reachability of loopback/system addresses in the Level 1 areas but it does not have complete visibility of the topology of those areas. This is simply because the associated IS-IS TLVs only have area scope and are not subject to domain-wide distribution. Routers R1 and R3 are externally peered with AS 64511 and AS 64514

respectively. Both routers learn the prefix 172.16.32.0/20 from their respective EBGP peers and redistribute the prefix internally within AS 64496. The topology has SR-ISIS enabled throughout, which is used for tunneling of BGP shortcuts.

Figure 13-25: Test topology for Optimal Route Reflection



Illustrating the problem is relatively straightforward. Routers R1 and R3 both learn prefix 172.16.32.0/20 from their respective external peers and advertise it in IBGP towards the route-reflector RR1. As RR1 receives two paths for the prefix 172.16.32.0/20 it executes the best-path selection algorithm and forwards only its best path to its other clients. *Output 13-75* shows the outcome of this, truncated to show only the relevant fields. There are two entries for network 172.16.32.0/20. The first entry is from Next-Hop 192.0.2.1 (R1) and the second entry is from Next-Hop 192.0.2.3 (R3). The result of the best-path selection process is shown in the `TieBreakReason` field only for the prefix that was not selected. As all other attributes in both paths were equal (AS_PATH, MED, LOCAL_PREF, ORIGIN, etc.), the route from R1 is preferred due to the IGP cost to the Next-Hop (`TieBreakReason : NHCost`).

Output 13-75: Best path selection at RR1 for prefix 172.16.32.0/20

```
A:admin@RR1# show router bgp routes 172.16.32.0/20 detail | match
'Network|Nexthop|TieBreakReason'
Network      : 172.16.32.0/20
Nexthop      : 192.0.2.1
Res. Nexthop : 192.168.0.53

Network      : 172.16.32.0/20
Nexthop      : 192.0.2.3
Res. Nexthop : 192.168.0.62
TieBreakReason : NHCost
```

The result of this best-path selection taken from the perspective of RR1 is evident at router R6. As shown in *Output 13-76* it receives the prefix 172.16.32.0/20 from RR1 with the Next-Hop of 192.0.2.1 (R1) and installs this as valid, used, and best. In short, traffic from router R6 towards prefix 172.16.32.0/20 must traverse both the core domain and the both Level 1 domains to exit the network, despite the fact that router R3 has an exit point with a shorter IGP cost. ADD-PATH would certainly help in this example by providing more visibility of the available paths, but this is not always feasible to deploy, either because of the additional memory requirements of holding additional paths, or the added complexity, or some other reason.

Output 13-76: Preferred Next-Hop for prefix 172.16.32.0/20 at R6

```

A:admin@R6# show router bgp routes 172.16.32.0/20
=====
BGP Router ID:192.0.2.6          AS:64496          Local AS:64496
=====
Legend -
Status codes  : u - used, s - suppressed, h - history, d - decayed, * - valid
                  l - leaked, x - stale, > - best, b - backup, p - purge
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP IPv4 Routes
=====
Flag  Network                      LocalPref  MED
      Nexthop (Router)              Path-Id    IGP Cost
      As-Path                      Label
-----
u*>i  172.16.32.0/20                100        None
      192.0.2.1                    None        300
      64511 64525                   -
-----
Routes : 1
=====

```

Enabling ORR at the route reflector allows it to make better-informed decisions about best-paths based on the location of its clients, but before it can do that it needs to understand the topology of the network. For a flat non-hierarchical IGP this is simple as it constitutes a single domain where the topology is known throughout. In short, there is a single traffic-engineered (TE) database known by all routers in the domain. Many networks however implement a hierarchy that has multiple domains. This can be implemented using the hierarchy that is inherent with the IGP (OSPF areas, IS-IS areas/levels), or by using multiple instances of the IGP (where each instance may itself include hierarchy). Either way, there are multiple domains with each containing its own autonomous TE database describing the topology of that domain. For the route reflector to learn the topology of these autonomous domains, BGP Link-State (BGP-LS) can be used. The use of BGP-LS is described earlier in this chapter and essentially extracts network topology information together with its associated traffic engineering parameters learnt through the IGP and encodes it into Link-State NLRI. In short, the BGP-LS Address Family can be used by the route reflector to learn the topology of the constituent parts of its Autonomous System.

In the topology shown in *Figure 13-25*, there are four Level 1/2 routers that connect the Level 2 domain to the Level 1 domains. Two connect to the Level 1 domain on the left of the figure and two connect to the Level 1 domain on the right of the figure. These four routers peer in BGP-LS with RR1 to provide it with the topology information of the three domains that constitute the network. The configuration required to enable topology information using the BGP-LS Address Family has already been covered earlier in this chapter so is not covered again in detail here, but *Output 13-77* shows the necessary configuration to enable BGP-LS at RR1. The **link-state-route-export** command is used to export link-state information from the BGP-LS NLRI into the local Traffic Engineering Database (TED) where it can be used for SPF calculations by RR1. A single **neighbor** example is shown where the **bgp-ls** family is enabled, with three other neighbours having similar configuration.

Output 13-77: BGP-LS configuration at RR1

```

router "Base" {
  bgp {
    link-state-route-export true
    neighbor "192.0.2.101" {
      group "TOPOLOGY"
      family {
        bgp-ls true
      }
    }
    neighbor etc...
  }
}

```

The BGP-LS peering sessions to the four Level 1/2 routers are established and link-state information advertised by the four peers to RR1.

Output 13-78: BGP-LS neighbours at RR1

```

A:admin@RR1# show router bgp summary
=====
BGP Router ID:192.0.2.10      AS:64496      Local AS:64496
=====
BGP Admin State      : Up      BGP Oper State      : Up
Total Peer Groups    : 7        Total Peers          : 38
--- [snip] ---
=====
BGP Summary
=====
Legend : D - Dynamic Neighbor
=====
Neighbor
Description
          AS PktRcvd InQ Up/Down  State|Rcv/Act/Sent (Addr Family)
          PktSent OutQ
-----
--- [snip] ---
          0      0
192.0.2.101
64496      345    0 01h46m24s 301/0/0 (LinkState)
192.0.2.103
64496      352    0 01h46m24s 291/0/0 (LinkState)
192.0.2.104
64496      345    0 01h46m24s 301/0/0 (LinkState)
192.0.2.106
64496      352    0 01h46m24s 291/0/0 (LinkState)

```

Enabling ORR requires configuration on the route reflector(s) only and is purely local. There is no configuration requirement on the IBGP clients. SR-OS supports ORR for the IPv4, Label-IPv4, Label-IPv6, VPN-IPv4, and VPN-IPv6 Address Families.

Output 13-79 shows the necessary configuration to enable ORR at RR1 and begins in the **optimal-route-reflection** context. Within this context the **spf-wait** context allows the user to tune the initial, second, and maximum waits that the router should use when executing an SPF. The router needs to run an SPF calculation to each known BGP Next-Hop from each configured location, and until these SPF runs are complete the route reflector does not have sufficient information to be able to accurately reflect routes. Equally, a topology change, or failure could trigger one or more SPF computations to be rerun, so SR-OS allows these

parameters to be configurable. The **location** context allows for configuration of up to 255 ORR locations. Each **location** requires the configuration of a **primary-ip-address** or **primary-ipv6-address** representing a node in the topology that serves as root for SPF calculations by the route reflector. These addresses must correspond to loopback or system IP addresses of routers that are known in the IGP. The **location** context also allows for configuration of secondary and tertiary IPv4 or IPv6 addresses which provide redundancy in the event that the primary IP address is removed from the topology. In the example of *Output 13-79*, **location 1** contains a **primary-ip-address** of R1 (192.0.2.1) and a **secondary-ip-address** of R4 (192.0.2.4), while **location 2** contains a **primary-ip-address** of R3 (192.0.2.3) and a **secondary-ip-address** of R6 (192.0.2.6). In this simple topology it's very easy to group the ORR clients into locations based on their association with each Level 1 area, but in larger and more complex topologies the placing of routers into locations will require consideration.

A peer **group** is then configured for each of the defined locations, under which the **cluster** context is used to configure the **cluster-id** and the **orr-location** for that group. The **allow-local-fallback true** command allows the route reflector to advertise the best path from its own topology location if no BGP routes are valid from the **orr-location** of the client. The **neighbor** contexts then associate each peer with the relevant group, and therefore the relevant ORR location. As shown, routers R1 and R4 are associated with ORR location 1, and routers R3 and R6 are associated with ORR location 2.

Output 13-79: Configuration for ORR at RR1

```
router "Base" {
  bgp {
    optimal-route-reflection {
      spf-wait {
        max-wait 16
        initial-wait 1
        second-wait 4
      }
      location 1 {
        primary-ip-address 192.0.2.1
        secondary-ip-address 192.0.2.4
      }
      location 2 {
        primary-ip-address 192.0.2.3
        secondary-ip-address 192.0.2.6
      }
    }
  }
  group "ORR-GROUP-LOC1" {
    cluster {
      cluster-id 192.0.2.10
      orr-location 1
      allow-local-fallback true
    }
  }
  group "ORR-GROUP-LOC2" {
    cluster {
      cluster-id 192.0.2.10
      orr-location 2
      allow-local-fallback true
    }
  }
  neighbor "192.0.2.1" {
    group "ORR-GROUP-LOC1"
  }
}
```

```

neighbor "192.0.2.3" {
    group "ORR-GROUP-LOC2"
}
neighbor "192.0.2.4" {
    group "ORR-GROUP-LOC1"
}
neighbor "192.0.2.6" {
    group "ORR-GROUP-LOC2"
}
}

```

With the above configuration in place at RR1, *Output 13-80* shows both the BGP Next-Hop information for each configured location at the route reflector. Each location and its primary, secondary and tertiary IPv4/IPv6 addresses are shown at the start of the output. This is followed by information on the SPF calculations the router has made to each of the BGP Next-Hops configured with an **orr-location** from each configured **location**. The first Next-Hop is router R1 with Next-Hop 192.0.2.1. Under this entry there is a row for each configured location (1 and 2), with each location showing the computed metric to the Next-Hop as well as how it was learned. For example, the SPF from location (Loc) 1 to prefix 192.0.2.1/32 was learned through BGP-LS and is shown as Type Local and Proto Local since the same prefix is used as the **primary-ip-address** in the **location** configuration. It can be considered an SPF to itself and the metric for this is therefore 0. The second row for Next-Hop 192.0.2.1 is from location 2 and is again learned through BGP-LS, but this time the Type field shows Remote and the Proto field is ISIS. The resulting IGP metric is 400. All of the BGP peers configured with an **orr-location** are listed in the output and follow the same format of an SPF result for each BGP Next-Hop from each configured location.

Output 13-80: ORR Next-Hop information at R1

```

A:admin@RR1# show router bgp optimal-route-reflection bgp-nh-info

=====
ORR BGP-NH Table (Router: Base)
=====
Location 1:
  Primary      : 192.0.2.1 [active]
  Secondary    : 192.0.2.4
  --- [snip] ---
Location 2:
  Primary      : 192.0.2.3 [active]
  Secondary    : 192.0.2.6
  --- [snip] ---

-----
Next Hop
  Loc    Dest-Prefix
-----
                                DB-Source  Type      Proto    Metric    Pref
-----
192.0.2.1
  1      192.0.2.1/32
                                BGP-LS    Local     Local     0          0
  2      192.0.2.1/32
                                BGP-LS    Remote    ISIS     400        15
192.0.2.3
  1      192.0.2.3/32
                                BGP-LS    Remote    ISIS     400        15
  2      192.0.2.3/32

```

		BGP-LS	Local	Local	0	0
192.0.2.4						
1	192.0.2.4/32	BGP-LS	Remote	ISIS	200	15
2	192.0.2.4/32	BGP-LS	Remote	ISIS	400	15
192.0.2.6						
1	192.0.2.6/32	BGP-LS	Remote	ISIS	400	15
2	192.0.2.6/32	BGP-LS	Remote	ISIS	200	15

No. of BGP-NHs: 4						
=====						

With the relevant information on BGP Next-Hops and ORR locations, *Output 13-81* shows the RIB-In and RIB-Out entries at RR1 for prefix 172.16.32.0/20, previously advertised internally by router R1 and R3. The output is truncated to show only the relevant fields for conciseness and begins with two RIB-In entries for prefix 172.16.32.0/20, one with a Next-Hop of 192.0.2.1 (R1) and one with a Next-Hop of 192.0.2.3 (R3). This is followed by four RIB-Out entries that in addition to the network show the Next-Hop (Nexthop) and the peer to which the Update is advertised (To). The first two RIB-Out entries for prefix 172.16.32.0/20 are for the path with Next-Hop 192.0.2.3 (R3), with the first entry being advertised to 192.0.2.3 (R3) and the second entry being advertised to 192.0.2.6 (R6). These routers are in ORR location 2 together with the Next-Hop address of R3. The third and fourth RIB-Out entries for prefix 172.16.32.0/20 are for the path with Next-Hop 192.0.2.1 (R1), with the third entry being advertised to 192.0.2.1 (R1) and the fourth entry being advertised to 192.0.2.4 (R4). Both of these routers are in ORR location 1 together with the Next-Hop address of R1. The BGP best-path selection process therefore contained two subsets of path selection: one for ORR location 1, and one for ORR location 2 for each of the configured peers. With all other BGP attributes being equal (AS_PATH, MED, ORIGIN, LOCAL_PREF, etc.) the prefix was thereafter advertised to each peer (Next-Hop) based on its IGP cost from the location.

Output 13-81: RIB-In and RIB-Out at RR1 for prefix 172.16.32.0/20

```
A:admin@RR1# show router bgp routes 172.16.32.0/20 hunt | match 'RIB
In|Network|Nexthop|From|RIB Out|To|---'
-----
RIB In Entries
-----
Network      : 172.16.32.0/20
Nexthop      : 192.0.2.1
From         : 192.0.2.1
Res. Nexthop : 192.168.0.53
-----
Network      : 172.16.32.0/20
Nexthop      : 192.0.2.3
From         : 192.0.2.3
Res. Nexthop : 192.168.0.62
-----
RIB Out Entries
-----
Network      : 172.16.32.0/20
Nexthop      : 192.0.2.3
```



```

To          : 192.0.2.3
Res. Nexthop : n/a
-----
Network     : 172.16.32.0/20
Nexthop     : 192.0.2.3
To          : 192.0.2.6
Res. Nexthop : n/a
-----
Network     : 172.16.32.0/20
Nexthop     : 192.0.2.1
To          : 192.0.2.4
Res. Nexthop : n/a
-----
Network     : 172.16.32.0/20
Nexthop     : 192.0.2.1
To          : 192.0.2.1
Res. Nexthop : n/a
-----

```

Although the result should be evident from the above output, in order to do a before and after comparison with *Output 13-76* before ORR was configured, *Output 13-82* shows the prefix 172.16.32.0/20 at router R6 with a Next-Hop of 192.0.2.3 (R3). Hence optimal routing is now achieved.

Output 13-82: Preferred Next-Hop for prefix 172.16.32.0/20 at R6

```

A:admin@R6# show router bgp routes 172.16.32.0/20
=====
BGP Router ID:192.0.2.6      AS:64496      Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP IPv4 Routes
=====
Flag  Network                               LocalPref  MED
      Nexthop (Router)                     Path-Id     IGP Cost
      As-Path                               Label
-----
u*>i  172.16.32.0/20                         100        None
      192.0.2.3                             None        200
      64514 64525                             -
-----
Routes : 1
=====

```

In networks where exiting the network at the nearest exit point is a requirement, the benefits of ORR are clear. It is self-contained at the route reflector and requires no protocol extensions. In hierarchical networks, BGP-LS is required to allow the route reflector(s) to learn the topology. The route reflector(s) will need sufficient CPU resource to be able to deal with the additional load placed on it by ORR. The router needs to run SPF calculations to each ORR-enabled BGP Next-Hop from each configured location and may need to repeat one or more in the event of a topology change. This of course requires computing resource, and the more locations there are the more computing power is required. There is therefore a trade-off between the optimality of the best path selection by the route reflector, and the computing power required. The most granular approach yielding the most optimal exit point

is a location for each BGP Next-Hop, but it is heavy on CPU resource. The coarsest approach yielding a less optimal exit point is two locations shared between all BGP Next-Hops, but this is the least CPU intensive. Some consideration is required for each network size and topology.

Route Policy Framework

It would not be possible to detail and illustrate all of the possibilities and capabilities made available in the SR-OS route-policy framework within this section. The aim therefore is simply to provide an overview of how policy is configured and referenced whilst illustrating some fairly common use-cases when using policy with BGP.

The default BGP behaviour without any policy applied is to accept all BGP routes and announce all BGP learned routes (subject to the best-path selection algorithm), but not to advertise any IGP, static, or local/connected routes. To advertise these types of routes, route-policy must be used. This allows the user to control how routes are redistributed between protocols. In addition, it provides the ability to manipulate route characteristics such as metrics and path attributes.



Note: There is an exception to the above rule. In an IP-VRF context that uses the **vrf-target <target:community>** command to specify the Route Target extended community to append to VPN-IPv4/IPv6 routes, all routes learned locally in the VRF route-table will be advertised as VPN-IPv4/IPv6 routes.

Route policies are configured in a **policy-statement**, consisting of one or more policy entries. Each entry contains some match criteria followed by an action. Prefixes are parsed through the **policy-statement** on an entry-by-entry basis in numerical order. If a match is found, the specific entry action is executed (accept or reject) and the prefix exits the policy evaluation. If no match is found in the first entry, the prefix is parsed through the next entry in the policy-statement and continues to look for a match. If no match is found after all of the policy-statement entries have been evaluated, a **default-action** defines what action should be taken. If no default-action is configured, the default action for the selected prefix is taken. In other words, if the prefix is a BGP route learnt through BGP then by default that route would be re-advertised by BGP, but if the prefix were learnt from another protocol, then by default it would not be advertised by BGP. The **policy-statement** entries also allow multiple actions to be executed on the same prefix by nesting entries or even **policy-statements** using an entry action of **next-entry** or **next-policy**. The policy-statement also references objects outside the actual policy itself including AS-path lists, prefix-lists, community lists, and other (embedded) policies. This section looks at some examples of these.

An example of a fairly basic routing policy is shown in *Output 13-83*. The intention of the policy is to redistribute a directly connected prefix 172.31.1.0/24 into BGP whilst appending a standard community value of 64501:100. The **community** list 'STD-COMM' defines the standard community value 64501:100. The **prefix-list** 'LOCAL' defines the prefix

172.31.0.0/24 with a **type** of **exact** meaning that the prefix length that is evaluated must match the configured prefix length exactly. Other **type** options include a **longer** prefix length, or a length range either using the commands **through end-length** (for example, 172.31.8.0/22 **through end-length** 24) or a prefix length range using the specified prefix/mask followed by **to through-length** (for example, 172.31.1.0/32 **to through-length** 32).

Output 13-83: Basic redistribution and community appending

```

policy-options {
  community "STD-COMM" {
    member "64501:100" { }
  }
  prefix-list "LOCAL" {
    prefix 172.31.1.0/24 type exact {
    }
  }
  policy-statement "BGP-EXPORT" {
    entry 10 {
      from {
        prefix-list ["LOCAL"]
      }
      to {
        protocol {
          name [bgp]
        }
      }
      action {
        action-type accept
        community {
          add ["STD-COMM"]
        }
      }
    }
  }
}

```

The configured policy can be applied at BGP level, group level, or neighbour level, both within the base BGP instance or within a VPRN BGP instance. *Output 13-84* shows an example of the previously configured policy being applied at the neighbour level. Applying policy to peers in this manner is relatively simple. To avoid repetition, this section provides no further configuration examples.

Output 13-84: Applying route policy to BGP peers

```

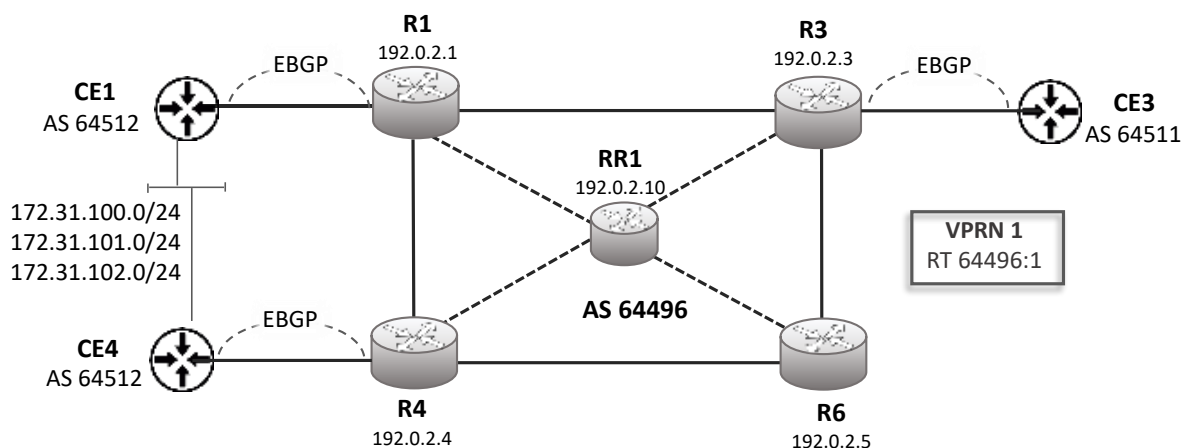
router "Base" {
  bgp {
    neighbor "192.168.1.10" {
      export {
        policy ["BGP-EXPORT"]
      }
    }
  }
}

```

To provide some examples of commonly used route policies the topology depicted in *Figure 13-26* will be used. R1, R3, and R4 belong to AS 64496 and provide an IP-VPN service (VPRN 1) to CE1, CE3, and CE4. CE1 and CE4 represent a dual-homed site in AS 64512 whilst CE3 is a singled-homed site in AS 64511. All CE routers peer in EBGp with their adjacent PE routers.

Different Route distinguishers are used at R1 and R4 to ensure that common IPv4 prefixes advertised by CE1 and CE4 are propagated through the Route-Reflector as different VPN-IPv4 prefixes. A VPRN is used purely to demonstrate the use of policy with the IPv4 and VPN-IPv4 Address Families using both standard and extended community attributes. CE1 and CE4 advertise prefixes 172.31.100.0/24, 172.31.101.0/24, and 172.31.102.0/24 which will be used to manipulate path attributes through route-policy at various points.

Figure 13-26: Test topology for Route Policy



Basic Path Attribute Manipulation

To illustrate path attribute manipulation, an example is shown of the configuration requirements to modify the MED and LOCAL-PREF attributes at R1 with the assumption that CE1 advertises prefix 172.31.100.0/24 with the standard community attributes 64512:200, 64512:300, and 64512:400. When R1 receives the prefix 172.31.100.0/24 from CE1 there are two options regarding where policy can be applied:

- To the RIB-In which dictates the use of an import policy on the IPv4 BGP peering session towards CE1.
- To the RIB-Out which dictates the use of an export policy on the VPN-IPv4 peering session towards the Route-Reflector RR1.

The RIB-In approach is described first, with the MED attribute being added to the prefix upon import. *Output 13-85* shows an example of the route-policy configuration at R1. The policy initially identifies prefixes with the standard communities added by CE1 by listing them in the **community** list 'CE1-STD-COMM', which in its current format represents a logical AND. That is, all members (standard communities) defined in the **community** list must be present before a match is made. Alternative ways of configuring this community list are illustrated later in this section. The **policy-statement** has a single entry (10) that looks for prefixes from **protocol bgp** with the **community** values defined by the previously configured **community** list and accepts them whilst setting an absolute **metric** (MED) value of 50. Other options with MED are to add or subtract from the existing MED value.

Output 13-85: RIB-In policy at R1

```

policy-options {
  community "CE1-STD-COMM" {
    member "64512:200" { }
    member "64512:300" { }
    member "64512:400" { }
  }
  policy-statement "CE1-IMPORT" {
    entry 10 {
      from {
        community {
          name "CE1-STD-COMM"
        }
        protocol {
          name [bgp]
        }
      }
      action {
        action-type accept
        metric {
          set 50
        }
      }
    }
  }
}

```

The route-policy is applied as an import-policy at the BGP neighbour level within VPRN 1. By validating the RIB-In for prefix 172.31.100.0/24 within VPRN 1, the original attribute contains no MED value (default) whilst the modified attributes (post RIB-In policy) contains the MED value of 50 imposed by the route-policy.

Output 13-86: MED imposition at R1

```

A:admin@R1# show router 1 bgp routes 172.31.100.0/24 detail | match 'Original
Attributes|MED|Modified Attributes'
Original Attributes
Atomic Aggr.   : Not Atomic          MED           : None
Modified Attributes
Atomic Aggr.   : Not Atomic          MED           : 50

```

Attributes can also be manipulated in the RIB-Out where the prefix 172.31.100.0/24 is advertised in IBGP towards RR1 using the VPN-IPv4 Address Family (the actual VPN-IPv4 prefix is 64496:1:172.31.100.0/24). Using the same prefix from CE1, the RIB-In policy is removed, and a RIB-Out policy is defined that sets the necessary extended community Route Target values to allow the prefix to be propagated throughout the sites of the VPN. In addition, if either community 64512:200 or 64512:300 or 64512:400 is present in the Update from CE1 the LOCAL-PREF attribute should be set to 200, but any other prefixes should have the default LOCAL-PREF (100).

Output 13-87 shows the required configuration. In this example the **community** list 'CE1-STD-COMM' uses regular expressions bounded by the ^ and \$ symbols to indicate that the community AS number must be 64512, but the community value following the AS number can start with either 2 or 3 or 4 and then have two zeroes. Entry 10 then uses **protocol name bgp** and **community name 'CE1-STD-COMM'** as match criteria in the **from** context, followed by **to protocol name bgp-vpn**. The **action** context provides to add the **community** list 'VRF1-

EXPORT' (in this case consisting of a single Route Target value of 'target:64496:1') and a **local-preference** value of **200**. Entry 20 then takes any other prefixes learnt through BGP and just appends the Route Target values defined in the **community** list 'VRF1-EXPORT'. A **default-action** of **reject** ensures that nothing else is advertised into VPN-IPv4 for this VPRN service.

Output 13-87: RIB-Out policy at R1

```

policy-options {
  community "CE1-STD-COMM" {
    member "^64512:2-400$" { }
  }
  community "VRF1-EXPORT" {
    member "target:64496:1" { }
  }
  policy-statement "VRF1-EXPORT" {
    entry 10 {
      from {
        community {
          name "CE1-STD-COMM"
        }
        protocol {
          name [bgp]
        }
      }
      to {
        protocol {
          name [bgp-vpn]
        }
      }
      action {
        action-type accept
        local-preference 200
        community {
          add ["VRF1-EXPORT"]
        }
      }
    }
    entry 20 {
      from {
        protocol {
          name [bgp]
        }
      }
      to {
        protocol {
          name [bgp-vpn]
        }
      }
      action {
        action-type accept
        community {
          add ["VRF1-EXPORT"]
        }
      }
    }
    default-action {
      action-type reject
    }
  }
}

```

Because the policy appends Route Target extended community values to advertise into IBGP, the policy statement is applied at the VPRN level using the command **vrf-export policy** followed by the **policy-statement** name. It isn't shown in this example output because it isn't relevant to the prefix manipulation, but the **vrf-export** command is typically accompanied by a corresponding **vrf-import** statement and an associated import policy. The **vrf-export** command overrides any configured **vrf-target export** Route Target values, as will the **vrf-import** command override any configured **vrf-target import** Route Target values.

Output 13-88: Applying policy at VPRN level

```

service {
    vprn "one" {
        bgp-ipvpn {
            mpls {
                vrf-export {
                    policy ["VRF1-EXPORT"]
                }
            }
        }
    }
}

```

The prefix 172.31.100.0/24 advertised by R1 into VPN-IPv4 towards RR1 can be validated by checking the RIB-Out as shown in *Output 13-89*. The standard community values imposed by CE1 are accompanied by an extended community Route Target value of 64496:1, whilst the LOCAL-PREF value has been set to 200.

Output 13-89: Verification of RIB-Out policy at R1

```

A:admin@R1# show router bgp routes 172.31.100.0/24 vpn-ipv4 hunt | match "RIB Out
Entries" post-lines 22
RIB Out Entries
-----
Network      : 172.31.100.0/24
Nexthop      : 192.0.2.1
Route Dist.   : 64496:1          VPN Label      : 524284
Path Id      : None
To           : 192.0.2.10
Res. Nexthop : n/a
Local Pref.   : 200             Interface Name : NotAvailable
Aggregator AS : None           Aggregator    : None
Atomic Aggr.  : Not Atomic      MED           : None
AIGP Metric   : None            IGP Cost      : n/a
Connector     : None
Community     : 64512:200 64512:300 64512:400 target:64496:1
Cluster       : No Cluster Members
Originator Id : None           Peer Router Id : 192.0.2.10
Origin        : IGP
AS-Path       : 64512
Route Tag     : 0
Neighbor-AS   : 64512
DB Orig Val   : N/A             Final Orig Val : N/A
Source Class  : 0               Dest Class     : 0

```

Nested Policies

To illustrate the use of nested policies, assume that both CE1 and CE4 advertise prefixes 172.31.101.0/24 and 172.31.102.0/24. When CE1 advertises the prefix 172.31.101.0/24 it includes the standard attribute 64512:100, and similarly when CE4 advertises the prefix 172.31.102.0/24 it includes the standard attribute 64512:200. The routes are propagated into VPN-IPv4 by R1 and R4 without any path attribute manipulation, but both PE routers impose the Route Target values 64496:1 and 64496:10.

The objective of this policy is that when the routes are received at R3 they are imported into VPRN 1 if either Route Target 64496:1 or 64496:10 is present. In addition, a requirement is that R3 prefers the prefix 172.31.101.0/24 (with standard community 64512:100) via R1, and prefix 172.31.102.0/24 (with standard community 64512:200) via R4. Lastly, for operational reasons many operators do not like to mix extended communities and standard communities within the same policy, so two policies are used: the first for Route Target import into the VPRN and the second for path attribute manipulation based on standard communities.

Output 13-90 shows the required configuration to deliver these requirements. In this example the **community** list 'CE1-STD-COMM' matches the community value imposed by CE1 for prefix 172.31.101.0/24, whilst the **community** list 'CE2-STD-COMM' matches the community value imposed by CE2 for prefix 172.31.102.0/24. The **community** list 'VPRN1-RT-IMPORT' uses the **expression** keyword to implement a logical OR so that the Route Target values can be either target:64496:1 OR target:64496:10. Conventional regular expression characters cannot be used with extended community attributes when importing into a VPRN, but the use of the **expression** keyword together with the ability to specify a logical AND, OR, or NOT against Route Target values provides the ability to override the default community list logical AND behaviour. This in turn, can avoid a requirement for multiple policy statement entries. The **policy-statement** 'VRF1-IMPORT' is used for importing VPN-IPv4 prefixes. The **from** context references the previously configured **community** list 'VRF1-RT-IMPORT'. The **action** context indicates an **action-type** of **next-policy**, and exactly which policy that should be is determined through the manner in which the policies are applied to the BGP peer (for example, policy 1, policy 2). The policy-statement 'VRF1-IMPORT' concludes with a **default-action** specifying an **action-type** of **reject** to ensure that no other Route Target values are imported into VPRN 1.

The **policy-statement** 'VRF1-STD-COMM-IMPORT' is the second policy. Entry 10 uses the **from** context to reference the **community** list 'CE1-STD-COMM' and set the **local-preference** to 250 within the **action** context. Given the default LOCAL-PREFERENCE value of 100 this ensures that R1 is the preferred Next-Hop for the 172.31.101.0/24 prefix. Similarly, entry 20 uses the **from** context to reference the **community** list 'CE2-STD-COMM' and set the **local-preference** to 250 again within the **action** context. This ensures that R4 is the preferred Next-Hop for the 172.31.102.0/24 prefix. The **policy-statement** concludes with **default-action** **accept-type** of **accept** to allow prefixes that do not have any standard attributes attached to be imported. This is necessary in this scenario because the CE routers impose standard attributes only for those prefixes for which they are trying to attract traffic. That is,

CE1 is advertising prefix 172.31.102.0/24 and CE4 is advertising prefix 172.31.101.0/24 without any standard attributes attached. If, for example, the link between R1 and CE1 fails and R1 withdraws the 172.31.101.0/24 prefix (carrying standard community 64512:100), R4 must become the new Next-Hop for that same prefix (without any standard community value).

Output 13-90: VRF import policy at R3

```

policy-options {
  community "CE1-STD-COMM" {
    member "64512:100" { }
  }
  community "CE2-STD-COMM" {
    member "64512:200" { }
  }
  community "VRF1-RT-IMPORT" {
    expression {
      expr "target:64496:1 OR target:64496:10"
    }
  }
  policy-statement "VRF1-IMPORT" {
    entry 10 {
      from {
        community {
          name "VRF1-RT-IMPORT"
        }
      }
      action {
        action-type next-policy
      }
    }
    default-action {
      action-type reject
    }
  }
  policy-statement "VRF1-STD-COMM-IMPORT" {
    entry 10 {
      from {
        community {
          name "CE1-STD-COMM"
        }
      }
      action {
        action-type accept
        local-preference 250
      }
    }
    entry 20 {
      from {
        community {
          name "CE2-STD-COMM"
        }
      }
      action {
        action-type accept
        local-preference 250
      }
    }
    default-action {
      action-type accept
    }
  }
}

```

Again, because the policy involves the import of Route Target extended community values, the policy statements are applied at the VPRN level using the command **vrf-import** followed by the **policy** names in the order in which they should be evaluated. In this case, the **policy-statement** 'VRF1-IMPORT' is followed by the **policy-statement** VRF1-STD-COMM-IMPORT.

Output 13-91: VRF import

```

service {
  vprn "one" {
    bgp-ipvpn {
      mpls {
        vrf-import {
          policy ["VRF1-IMPORT" "VRF1-STD-COMM-IMPORT"]
        }
      }
    }
  }
}

```

Finally, a validation is made that the nested policies have implemented the required route imports and carried out the path attribute manipulation. *Output 13-92* shows the original and modified attributes for the prefix 172.31.101.0/24 in the RIB-In of R3 summarised to the relevant lines of interest (Network, Next-Hop, Community, and Local Preference). The first two entries for network 172.31.101.0/24 show the original and modified attributes with a Next-Hop of R1 (192.0.2.1). In addition to the extended community Route-Target attributes, the entries have the standard community 64512:100 attached, which matches entry 10 of the **policy-statement** VRF1-STD-COMM-IMPORT. As a result, the original and modified Local Preference values differ from 100 in the original attributes to 250 in the modified attributes. The second two entries for 172.31.101.0/24 show the original and modified attributes with a Next-Hop of R4 (192.0.2.4) and do not carry any standard community values. As a result, the original and modified Local Preference values are the same using the default value of 100, but nonetheless the prefix is eligible for import into VPRN 1. As the Local Preference value from R1 is higher, it is the preferred path.

Output 13-92: Nested policy verification for prefix 172.31.101.0/24 at R3

```

A:admin@R3# show router bgp routes 172.31.101.0/24 vpn-ipv4 detail | match
'Network|Nexthop|Community|Local Pref.|Original|Modified'
Original Attributes
Network       : 172.31.101.0/24
Nexthop       : 192.0.2.1
Res. Nexthop  : 192.168.0.5
Local Pref.   : 100                               Interface Name : link-to-R2
Community     : 64512:100 target:64496:1 target:64496:10
Last Modified : 02h12m52s
Modified Attributes
Network       : 172.31.101.0/24
Nexthop       : 192.0.2.1
Res. Nexthop  : 192.168.0.5
Local Pref.   : 250                               Interface Name : link-to-R2
Community     : 64512:100 target:64496:1 target:64496:10
Last Modified : 02h12m52s

Original Attributes
Network       : 172.31.101.0/24
Nexthop       : 192.0.2.4
Res. Nexthop  : 192.168.0.5

```

```

Local Pref.      : 100                               Interface Name : link-to-R2
Community       : target:64496:1 target:64496:10
Last Modified   : 01h54m24s
Modified Attributes
Network         : 172.31.101.0/24
Nexthop         : 192.0.2.4
Res. Nexthop    : 192.168.0.5
Local Pref.     : 100                               Interface Name : link-to-R2
Community       : target:64496:1 target:64496:10
Last Modified   : 01h54m24s

```

A similar logic to the preceding output exists for the 172.31.102.0/24 prefix, and the desired behaviour is seen in *Output 13-93* showing R3's route-table for VPRN 1. Here the prefix 172.31.101.0/24 is preferred via 192.0.2.1 (R1) whilst the prefix 172.31.102.0/24 is preferred via 192.0.2.4 (R4). Because both CE routers advertise both prefixes but only attach a standard community value for the prefix for which they are attracting traffic, a backup path for both prefixes (with no standard community attached) remains in the RIB-In in case of failure of the primary path.

Output 13-93: R3 route-table for VPRN 1

```

A:admin@R3# show router 1 route-table 172.31.101.0/22 longer

=====
Route Table (Service: 1)
=====
Dest Prefix[Flags]                                Type   Proto   Age           Pref
Next Hop[Interface Name]                          Metric
-----
172.31.100.0/24                                   Remote BGP VPN  02h03m58s    170
192.0.2.1 (tunneled:SR-ISIS:524337)              200
172.31.101.0/24                                   Remote BGP VPN  02h26m34s    170
192.0.2.1 (tunneled:SR-ISIS:524337)              200
172.31.102.0/24                                   Remote BGP VPN  02h26m34s    170
192.0.2.4 (tunneled:SR-ISIS:524306)              300
-----
No. of Routes: 3
Flags: n = Number of times nexthop is repeated
      B = BGP backup route available
      L = LFA nexthop available
      S = Sticky ECMP requested
=====

```

Sub-Routines

Although multiple policies can be nested and evaluated in a serial manner as shown previously, this approach has the potential to make the process of configuration rather cumbersome when policies become complex. Instead of nesting policies in a serial manner, the use of subroutines allows the user to configure a **policy-statement** entry that calls another policy (sub-policy) as part of its evaluation process. This section will illustrate the use of subroutines, and for completeness will include the entry **action-type** of **'next-entry'**, which is used when the operator wants to modify an attribute within an entry, but does not want that prefix to exit the policy evaluation after the entry in question.

To demonstrate the use of subroutines and **next-entry** actions, assume that R1 in AS 64496 belongs to a non-transit AS taking Internet transit from an ISP (formerly CE1) in AS 64512. The transit ISP is advertising a default-route to AS 64496 and because of some configuration error is also mis-announcing the bogon prefix 172.31.100.0/24 towards AS 64496. For conciseness the illustrated policies contain only a handful of entries, and a production policy is likely to be significantly more extensive, but they serve to illustrate the function.

Output 13-94 shows the policy that forms the subroutine at R1's ingress from AS 64512. This is commonly referred to as a *sub-policy* but is configured in exactly the same way as any other policy. In this example, the subroutine implements a generic prefix rejection that can be referenced by various external policies towards other external peers. It consists of a **policy-statement** 'SUB-ROUTINE' with two entries. Entry 10's **from** context references the prefix-list 'BOGON-PREFIXES-IPv4' containing a few bogon prefixes, whilst entry 20's **from** context references **as-path** 'AS_PATH>100' matching any Update message containing more than 100 AS paths in the AS_PATH attribute. If the purpose of this subroutine is to drop prefixes that match this criterion why do both entries have an **action-type** of **accept**? When a policy calls a sub-policy for match criteria the meaning of accept and reject in the sub-policy become true and false when returned to the calling policy. In other words, accept=true=match, and reject=false=no match. Therefore, if entry 10 of the sub-policy matches a prefix in the listed bogon prefix range it returns a true/match indication, and it is then up to the calling policy to execute the appropriate action. The **default-action action-type** is **reject**, meaning that all prefixes that do not match entry 10 and 20 return false/no match to the calling policy.

Output 13-94: Sub-Policy for transit peering at R1

```
policy-options {
  as-path "AS_PATH>100" {
    expression ".{100,}"
  }
  prefix-list "BOGON-PREFIXES-IPv4" {
    prefix 10.0.0.0/8 type longer {
    }
    prefix 172.16.0.0/12 type longer {
    }
    prefix 192.168.0.0/16 type longer {
    }
  }
  policy-statement "SUB-ROUTINE" {
    entry 10 {
      from {
        prefix-list ["BOGON-PREFIXES-IPv4"]
      }
      action {
        action-type accept
      }
    }
    entry 20 {
      from {
        as-path {
          name "AS_PATH>100"
        }
      }
      action {
        action-type accept
      }
    }
  }
}
```

```

    }
    default-action {
        action-type reject
    }
}

```

The calling-policy that references the sub-policy of *Output 13-94* is shown in *Output 13-95*. The policy-statement 'EBGP-to-AS64512' has four entries, and these are described as follows:

- Entry 10 uses a subroutine and calls the previously configured sub-policy 'SUB-ROUTINE'. Any prefix that is returned by the sub-policy with a 'true' flag indicating a match is discarded due to the **action-type** of **reject** command (in this case, any listed bogon prefixes and any prefixes with more than 100 entries in the AS_PATH attribute).
- Entry 20 has no **from** context and therefore has no explicit match criteria. As a result, entry 20 is applicable to all prefixes being parsed through the policy and will **remove** all standard communities defined in the **community** list 'ALL-INTERNAL-STD-COMM'. This community list uses regular expressions to list all standard communities in the range 64496:1000 to 9999 that the operator uses for AS-internal policy application. The **action-type** of **next-entry** tells prefixes not to exit the policy, but to proceed to the next configured entry in the policy.
- Like entry 20, entry 30 has no **from** context and is applicable to all prefixes being parsed through the policy. It references the community list 'ALL-RT-EXT-COMM', which uses regular expressions to match any Route Target extended community attribute, and then removes them. As shown previously, the **action-type** of **next-entry** tells prefixes to continue through the policy.
- Entry 40 uses a **from** context to match a standard community value 64512:100 defined by the **community** list 'EXPENSIVE-TRANSIT-ISP'. Prefixes with this community are permitted by the **action-type** of **accept**, but because this transit ISP is more expensive than AS 64496's other transit ISP's, the import policy appends the peering AS number in the AS_PATH two more times using the **as-path-prepend** command to influence internal routing decisions based on the AS_PATH length. The **default-action action-type** of **reject** means any other prefixes that do not have the standard attribute 64512:100 are rejected at R1.

Output 13-95: Calling policy for transit peering at R1

```

policy-options {
    community "ALL-INTERNAL-STD-COMM" {
        member "^64496:1-9...$" { }
    }
    community "ALL-RT-EXT-COMM" {
        member "target:^\.*$&^\.*$" { }
    }
    community "EXPENSIVE-TRANSIT-ISP" {
        member "64512:100" { }
    }
    policy-statement "EBGP-to-AS64512" {
        entry 10 {
            from {
                policy "SUB-ROUTINE"
            }

```

```

        action {
            action-type reject
        }
    }
    entry 20 {
        action {
            action-type next-entry
            community {
                remove ["ALL-INTERNAL-STD-COMM"]
            }
        }
    }
    entry 30 {
        action {
            action-type next-entry
            community {
                remove ["ALL-RT-EXT-COMM"]
            }
        }
    }
    entry 40 {
        from {
            community {
                name "EXPENSIVE-TRANSIT-ISP"
            }
        }
        action {
            action-type accept
            as-path-prepend {
                as-path 64512
                repeat 3
            }
        }
    }
    default-action {
        action-type reject
    }
}
}

```

When using sub-routines, only the calling policy (*Output 13-95*) needs to be configured as the import/export policy. The sub-policy does not need to be applied to any peer. After applying the policy 'EBGP-to-AS64512' to the EBGP peering session between R1 and AS 64512, a check is made of the routes being received by R1 from the peer. In *Output 13-96*, the transit peer is announcing the default-route, and also the mis-announced prefix 172.31.100.0/24. However, as shown in *Output 13-97*, only the default-route is accepted into the route-table in VPRN 1 because the prefix 172.31.100.0/24 is dropped by entry 10 of the import policy. Recall that this is the entry implementing subroutines and references a sub-policy to match these routes.

Output 13-96: Received routes from external peer in AS 64512 at R1

```

A:admin@R1# show router 1 bgp neighbor "192.168.100.2" received-routes
=====
BGP Router ID:192.0.2.1          AS:64496          Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete

```

```

=====
BGP IPv4 Routes
=====
Flag   Network                               LocalPref  MED
      Nexthop (Router)                 Path-Id    IGP Cost
      As-Path                           Label
-----
u*>i  0.0.0.0/0                             n/a        None
      192.168.100.2                     None        0
      64512                              -
i     172.31.100.0/24                   n/a        None
      192.168.100.2                     None        0
      64512                              -
-----
Routes : 2
=====

```

Output 13-97: R1's route-table for VPRN 1 after import policy

```

A:admin@R1# show router 1 route-table protocol bgp

=====
Route Table (Service: 1)
=====
Dest Prefix[Flags]                Type   Proto   Age           Pref
      Next Hop[Interface Name]                Metric
-----
0.0.0.0/0                        Remote BGP      00h07m26s  170
      192.168.100.2                        0
-----
No. of Routes: 1
Flags: n = Number of times nexthop is repeated
       B = BGP backup route available
       L = LFA nexthop available
       S = Sticky ECMP requested
=====

```

Taking a closer look at the prefix 0.0.0.0/0 received by the transit peer, it is possible to verify that the policy has had the desired effect. In *Output 13-98*, the first entry for network 0.0.0.0/0 shows the original attributes as received from AS 64512. Of note is the fact that the Update has the standard community 64512:100 but also contains one of the standard community values (64496:1004) that AS 64496 uses for internal policy application. The AS_PATH attribute also contains only one entry, that of the transit ISP itself. The second entry for network 0.0.0.0/0 shows the modified attributes after policy has been applied to the RIB-In. In this entry the community 64496:1004 has been stripped as a result of entry 20 of the import policy, which removes all standard community values in the range 64496:1000-9999, and the AS_PATH attribute now contains the AS number 64512 four times in total. In summary, the policy is functioning as expected.

Output 13-98: Post-import RIB-In for prefix 0.0.0.0/0 at R1

```

A:admin@R1# show router 1 bgp routes 0.0.0.0/0 detail | match
'Network|Nexthop|Community|AS-Path|Original|Modified'
Original Attributes
Network       : 0.0.0.0/0
Nexthop       : 192.168.100.2
Res. Nexthop  : 192.168.100.2
Community     : 64512:100 64496:1004
AS-Path       : 64512

```

```

Last Modified   : 00h01m48s
Modified Attributes
Network        : 0.0.0.0/0
Nexthop        : 192.168.100.2
Res. Nexthop   : 192.168.100.2
Community      : 64512:100
AS-Path        : 64512 64512 64512 64512
Last Modified   : 00h01m48s

```



Note: Only a single level of policy nesting using subroutines is possible. For example, policy 1 can reference policy 2, but policy 2 cannot reference policy 3.

Logical AND, OR, and NOT Operators

By default, community lists containing multiple community values operate a logical “AND” function where all configured community values must be present for the prefix(es) to match. For example, the following community list would require that a prefix had community values 64510:1000 and 64510:1001 present before a match was made.

Output 13-99: Community-List AND function

```

policy-options {
  community "AS-64510" {
    member "64510:1000" { }
    member "64510:1001" { }
  }
}

```

If the use of the default AND operator within community lists is maintained, creating an OR function would require multiple entries in a **policy-statement**, where each entry refers to a different community-list containing one or more community values. Clearly this has the potential to bloat the configuration, so a better alternative is to use regular-expression AND, OR, or NOT operators within the community list. Taking the example from *Output 13-99* the default AND behaviour can be modified to implement an OR function using the **expression** context together with the **expr** keyword and the **OR** operator as shown in *Output 13-100*.

Output 13-100: Community list logical OR operation

```

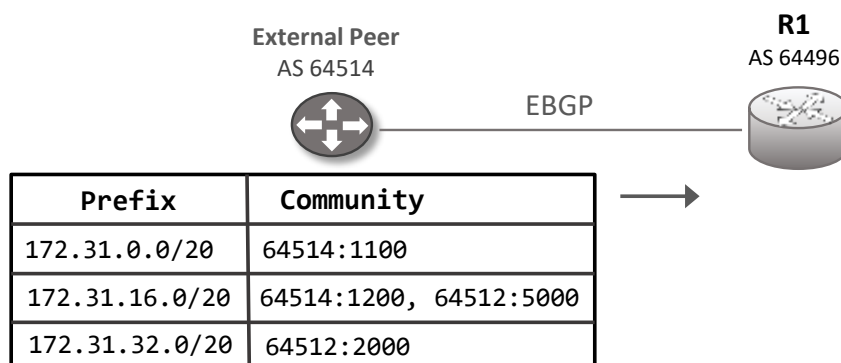
policy-options
community "AS-64510" {
  expression {
    expr "64510:1000 OR 64510:1001"
  }
}

```

The NOT operator becomes useful if the requirement is to implement an “except for...” function. For example, in *Figure 13-27* the BGP speaker in AS 64514 advertises three prefixes to router R1 in AS 64496, each with different community values. The requirement is that router R1 should accept any prefix from AS 64514 with or without standard community attributes. When a standard community is present, the AS portion of the community must

be AS 64514. If it is, the community value can be advertised upstream transparently. If it is not, the community value should be stripped before advertising the prefix upstream.

Figure 13-27: AS 64514 to router R1 advertised prefixes and communities



The following policy is therefore applied as an import policy at router R1, where the community “NOT-AS64514-COMM” again uses the **expression** context together with the **expr** keyword and the **NOT** operator to match any standard communities that do not start with 64514. Entry 10 of the **policy-statement** then accepts all prefixes and removes all communities covered by that community list. That is, all communities are removed, except those beginning with 64514 as a result of the logical “NOT”.

Output 13-101: Policy using logical NOT operator

```

policy-options {
  community "NOT-AS64514-COMM" {
    expression {
      expr "NOT(64514:*)"
    }
  }
  policy-statement "EBGP-to-AS64514" {
    entry 10 {
      action {
        action-type accept
        community {
          remove ["NOT-AS64514-COMM"]
        }
      }
    }
  }
}

```

Prefix 172.31.0.0/20 advertised by AS 64514 with community 64514:1100 is accepted by R1’s policy, and no changes are made to the received community attributes.

Output 13-102: RIB-In entry for prefix 172.31.0.0/20 at R1

```

A:admin@R1# show router bgp routes 172.31.0.0/20 detail | match
'Network|Community|Flags|Original|Modified'
Original Attributes
Network       : 172.31.0.0/20
Community    : 64514:1100
Flags        : Used Valid Best Group-Best In-RTM
Last Modified : 00h23m27s

```

```

Modified Attributes
Network       : 172.31.0.0/20
Community     : 64514:1100
Flags         : Used Valid Best Group-Best In-RTM
Last Modified : 00h23m27s

```

Prefix 172.31.16.0/20 advertised by AS 64514 with community values 64514:1200 and 64512:5000 is accepted by R1's policy, but the community 64512:5000 is removed because it does not have AS 64514 in the AS portion of the community value. The community 64514:1200 remains intact.

Output 13-103: RIB-In entry for prefix 172.31.16.0/20 at R1

```

A:admin@R1# show router bgp routes 172.31.16.0/20 detail | match
'Network|Community|Flags|Original|Modified'
Original Attributes
Network       : 172.31.16.0/20
Community     : 64512:5000 64514:1200
Flags         : Used Valid Best Group-Best In-RTM
Last Modified : 00h25m36s
Modified Attributes
Network       : 172.31.16.0/20
Community     : 64514:1200
Flags         : Used Valid Best Group-Best In-RTM
Last Modified : 00h25m36s

```

Prefix 172.31.32.0/20 advertised by AS 64514 with community value 64512:2000 is accepted by R1's policy, but the community 64512:2000 is removed because it does not have AS 64514 in the AS portion of the community value.

Output 13-104: RIB-In entry for prefix 172.31.32.0/20 at R1

```

A:admin@R1# show router bgp routes 172.31.32.0/20 detail | match
'Network|Community|Flags|Original|Modified'
Original Attributes
Network       : 172.31.32.0/20
Community     : 64512:2000
Flags         : Used Valid Best Group-Best In-RTM
Last Modified : 00h28m55s
Modified Attributes
Network       : 172.31.32.0/20
Community     : No Community Members
Flags         : Used Valid Best Group-Best In-RTM
Last Modified : 00h28m55s

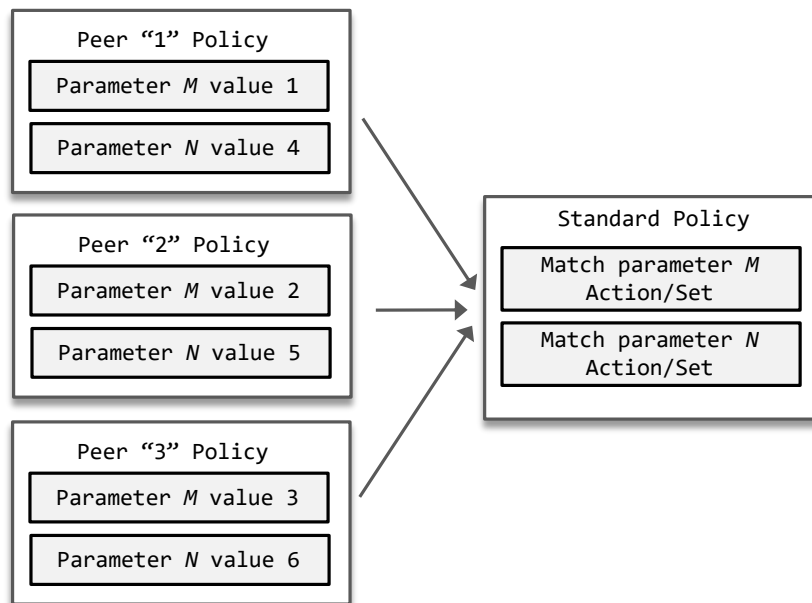
```

Parameterisation

Parameterisation is a feature that builds on the parent/child policy approach used by Sub-Routines. Parameterisation allows for a parent policy to reference parameters in child policies in a generic manner, even though the value of those called parameters is different in each child policy. The parent policy also uses the same generic approach to manipulate path attributes in each of the child policies. In *Figure 13-28* the parent policy (called “Standard Policy”) is referenced by three peer-specific child policies. The Standard Policy matches on parameter “*M*”, even though “*M*” has a value of 1 in peer 1's child policy, a value of 2 in peer 2's child policy, and a value of 3 in peer 3's child policy. A similar approach is carried out for the second entry, matching on parameter “*N*”, where “*N*” has a value of 4 in peer 1's child

policy, a value of 5 in peer 2's child policy, and a value of 6 in peer 3's child policy. Criteria that can be matched in this manner includes Community-Lists, Prefix-Lists, and AS-Paths.

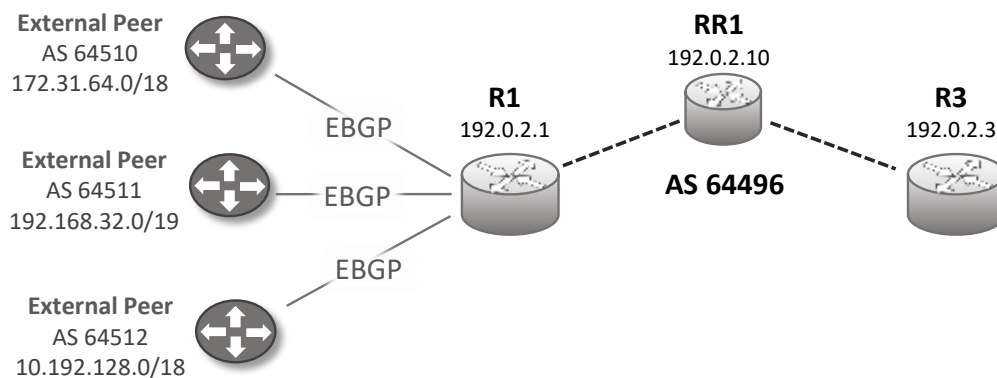
Figure 13-28: Parameterisation concept



To illustrate the use of parameterisation the example shown in *Figure 13-29* is used, where router R1 is externally peered with AS 64510, AS 64511, and AS 64512. These three external peers advertise prefixes 172.31.64.0/18, 192.168.32.0/19, and 10.192.128.0/18 respectively. From the perspective of AS 64496, they are all external private peers, and therefore a common set of policy rules could be applied against them. For this parameterisation example, a very simple policy is illustrated:

- i. Prefixes from AS 64510 should be marked with community 64496:1001 and the well-known community no-export by router R1 before advertising those prefixes internally into AS 64496
- ii. Prefixes from AS 64511 should be marked with community 64496:1002 and the well-known community no-export by router R1 before advertising those prefixes internally into AS 64496
- iii. Prefixes from AS 64512 should be marked with community 64496:1003 and the well-known community no-export by router R1 before advertising those prefixes internally into AS 64496

Figure 13-29: Parameterisation example



Output 13-105 shows the child policy “Peer-AS64510-In” that is applied to the external peer in AS 64510. Entry 10 of this policy uses the **from** criteria **policy-variables** to enable parameterisation. Within the **policy-variables** context, there are two entries beginning with **name**. The **name** can be up to 32 characters of free text that contains the variable prefixed and suffixed with an “@” sign. The first **name**, “@comm-value@”, has a **value** of “AS64510-Comm” which references a community list containing the community “64496:1001”. (Although the example shows the “@” sign at the beginning and end of the name, it’s equally acceptable for them to appear mid-string.) The second **name**, “@peer-as@”, has a **value** of “AS64510” which references an **as-path** name with a regular expression allowing any prefix with AS 64510 as the last (left-most) AS in the AS-PATH attribute. The final entry in the **from** criteria begins with **policy** and references the parent policy “Standard-Policy-In”, which will be discussed shortly. Entry 10 completes with an **action-type** of **accept**.

Output 13-105: Child policy for AS 64510

```

policy-options {
  as-path "AS64510" {
    expression "(64510 .*)"
  }
  community "AS64510-Comm" {
    member "64496:1001" { }
  }
  policy-statement "Peer-AS64510-In" {
    entry 10 {
      from {
        policy "Standard-Policy-In"
        policy-variables {
          name "@comm-value@" {
            value "AS64510-Comm"
          }
          name "@peer-as@" {
            value "AS64510"
          }
        }
      }
      action {
        action-type accept
      }
    }
  }
}

```

}

Output 13-106 shows the child policy “Peer-AS64511-In” that is applied to the external peer in AS 64511, which takes a similar form to the **policy-statement** previously described for AS 64510. Entry 10 of this policy uses the **from** criteria **policy-variables** to enable parameterisation. Within the **policy-variables** context, there are two entries beginning with **name**. The first **name**, “@comm-value@”, has a **value** of “AS64511-Comm” which references a community list containing the community “64496:1002”. The second **name**, “@peer-as@”, has a **value** of “AS64511” which references an **as-path** name with a regular expression allowing any prefix with AS 64511 as the last (left-most) AS in the AS-PATH attribute. Note that both the names used in this policy are identical to the names used in the policy for AS 64510. The final entry in the **from** criteria begins with **policy** and references the parent policy “Standard-Policy-In”. Entry 10 completes with an **action-type** of **accept**.

Output 13-106: Child policy for AS 64511

```

policy-options {
  as-path "AS64511" {
    expression "(64511 .*)"
  }
  community "AS64511-Comm" {
    member "64496:1002" { }
  }
  policy-statement "Peer-AS64511-In" {
    entry 10 {
      from {
        policy "Standard-Policy-In"
        policy-variables {
          name "@comm-value@" {
            value "AS64511-Comm"
          }
          name "@peer-as@" {
            value "AS64511"
          }
        }
      }
      action {
        action-type accept
      }
    }
  }
}

```

It almost doesn’t warrant describing but is covered for completeness. *Output 13-107* shows the child policy “Peer-AS64512-In” that is applied to the external peer in AS 64512, which takes a similar form to both of the policy-statements previously described. Entry 10 of this policy uses the **from** criteria **policy-variables** to enable parameterisation. Within the **policy-variables** context, there are two entries beginning with **name**. The first **name**, “@comm-value@”, has a **value** of “AS64512-Comm” which references a community list containing the community “64496:1003”. The second **name**, “@peer-as@”, has a **value** of “AS64512” which references an **as-path** name with a regular expression allowing any prefix with AS 64512 as the last (left-most) AS in the AS-PATH attribute. Note that both the names used in this policy are identical to the names used in the policy for AS 64510 and AS 64511. The final entry in the **from** criteria begins with **policy** and references the parent policy “Standard-Policy-In”. Entry 10 completes with an **action-type** of **accept**.

Output 13-107: Child policy for AS 64512

```

policy-options {
  as-path "AS64512" {
    expression "(64512 .*)"
  }
  community "AS64512-Comm" {
    member "64496:1003" { }
  }
  policy-statement "Peer-AS64512-In" {
    entry 10 {
      from {
        policy "Standard-Policy-In"
        policy-variables {
          name "@comm-value@" {
            value "AS64512-Comm"
          }
          name "@peer-as@" {
            value "AS64512"
          }
        }
      }
      action {
        action-type accept
      }
    }
  }
}

```

Output 13-108 shows the parent policy that was referenced by each of the three child policies. Entry 10 has **from** criteria of **as-path** “@peer-as@”. With parameterisation enabled this means the following to the child policies that reference this policy:

- For the child policy “PeerAS64510-In” applied to the external peer in AS 64510, match any as-path with AS 64510 as the last (left-most) AS in the AS-PATH attribute, as this was the value of “@peer-as@” within that child policy.
- For the child policy “PeerAS64511-In” applied to the external peer in AS 64511, match any as-path with AS 64511 as the last (left-most) AS in the AS-PATH attribute, as this was the value of “@peer-as@” within that child policy
- For the child policy “PeerAS64512-In” applied to the external peer in AS 64512, match any as-path with AS 64512 as the last (left-most) AS in the AS-PATH attribute, as this was the value of “@peer-as@” within that child policy

This is followed with an **action-type** of **next-entry** and followed by a **community add** “@comm-value@” to any matching prefixes. With parameterisation enabled this means the following to the child policies that reference this parent policy:

- For the child policy “PeerAS64510-In” applied to the external peer in AS 64510, add the standard community 64496:1001 as this was the value of “@comm-value@” within that child policy. Do not leave the policy evaluation but move to the next entry (entry 20).
- For the child policy “PeerAS64511-In” applied to the external peer in AS 64511, add the standard community 64496:1002 as this was the value of “@comm-value@”

within that child policy. Do not leave the policy evaluation but move to the next entry (entry 20).

- c. For the child policy “PeerAS64512-In” applied to the external peer in AS 64512, add the standard community 64496:1003 as this was the value of “@comm-value@” within that child policy. Do not leave the policy evaluation but move to the next entry (entry 20).

Entry 20 then accepts all prefixes (by virtue of the fact that it has no explicit match criteria) and adds the well-known community no-export.

Output 13-108: Parent policy for parameterisation

```

policy-options {
  community "NO-EXPORT" {
    member "no-export" { }
  }
  policy-statement "Standard-Policy-In" {
    entry 10 {
      from {
        as-path {
          name "@peer-as@"
        }
      }
      action {
        action-type next-entry
        community {
          add ["@comm-value@"]
        }
      }
    }
    entry 20 {
      action {
        action-type accept
        community {
          add ["NO-EXPORT"]
        }
      }
    }
  }
}

```

Each of the child policies is applied to the relevant external peer as an import policy. That is, the policy is applied to the Adj-RIB-In. The results are shown in the following outputs, which contain the pertinent lines of the original (as received) and modified (post RIB-In policy) attributes for each peer/prefix. *Output 13-109* shows the prefix 172.31.64.0/18 received from AS 64510 with the community 64535:3500 in the original attributes while the modified attributes show that the community values no-export and 64496:1001 have been appended as a result of the parameterised policy.

Output 13-109: Prefix 172.31.64.0/18 from AS 64510

```

A:admin@R1# show router bgp routes 172.31.64.0/18 detail | match 'Network|AS-
Path|Community|Original|Modified Attr'
Original Attributes
Network          : 172.31.64.0/18
Community        : 64535:3500
AS-Path          : 64510 64535
Last Modified    : 00h01m09s

```

```

Modified Attributes
Network       : 172.31.64.0/18
Community    : no-export 64496:1001 64535:3500
AS-Path      : 64510 64535

```

Output 13-110 shows the prefix 192.168.32.0/19 received from AS 64511 with no community members in the original attributes, but with community values no-export and 64496:1002 added in the modified attributes as a result of the parameterized policy.

Output 13-110: Prefix 192.168.32.0/19 from AS 64511

```

A:admin@R1# show router bgp routes 192.168.32.0/19 detail | match 'Network|AS-
Path|Community|Original|Modified Attr'
Original Attributes
Network       : 192.168.32.0/19
Community    : No Community Members
AS-Path      : 64511
Modified Attributes
Network       : 192.168.32.0/19
Community    : no-export 64496:1002
AS-Path      : 64511

```

Finally, *Output 13-111* shows the prefix 10.192.128.0/18 received from AS 64512 with no community members in the original attributes, but with community values no-export and 64496:1003 added in the modified attributes as a result of the parameterized policy.

Output 13-111: Prefix 10.192.128.0/18 from AS 64512

```

A:admin@R1# show router bgp routes 10.192.128.0/18 detail | match 'Network|AS-
Path|Community|Original|Modified Attr'
Original Attributes
Network       : 10.192.128.0/18
Community    : No Community Members
AS-Path      : 64512
Modified Attributes
Network       : 10.192.128.0/18
Community    : no-export 64496:1003
AS-Path      : 64512

```



Note: The supported **policy-variables** include as-path, community, prefix-list, as-path expression, and as-path-group. The supported action-types when using **policy-variables** include aigp-metric, as-path-prepend, local-preference, metric, next-hop, damping, origin, preference, tag, and type.

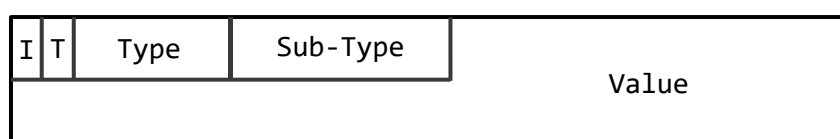
Extended Community Filtering using Hex

When filtering on extended community values, community lists are typically prefixed with Route-Target (target:), Route-Origin (origin:), and Link Bandwidth (bandwidth:). Origin validation state could also be used, which is defined using a generic **ext** value followed by a hex value of 4300 (i.e. **ext:4300**), where 0x43 represents a one-byte type code specifying a non-transitive extended community, and 0x00 represents a one-byte subtype specifying prefix origin validation state). Whilst filtering on extended community values in this manner is suitable for most environments, there may be occasions where a more generic approach

is required that allows for filtering of arbitrary values. For example, perhaps there is a requirement to filter on a new extended community value that is not currently supported in SR-OS.

To support more general matching of Extended Communities, SR-OS uses hexadecimal values to match against the type/sub-type and value fields of an extended community. Before illustrating some examples, it's worth a reminder of the structure of the extended community to help understand how these hex filters are composed. An extended community is encoded as an 8-byte quantity, consisting of a 1-byte Type field, a 1-byte Sub-Type field, and a 6-byte Value field.

Figure 13-30: Two-Octet AS-Specific Extended Community



The Type field has two flags in the high-order bits:

I flag IANA authority bit. A value of 0 indicates that the Type field is an IANA-assigned. A value of 1 allows for partitioning the Type field into IANA assigned types and Experimental use.

T flag Transitive bit. A value of 0 indicates the community is transitive. A value of 1 indicates the community is non-transitive.

The remaining 6-bits of the Type field define the format of the Value field. There are numerous formats defined, but the more commonly used are either a Two-octet AS Specific extended community, an IPv4 Address Specific extended community, or an Opaque extended community. These formats are defined by the Type field and should be considered as “templates” for the construction of the Value field. The type field does not define a particular extended community; that is the purpose of the Sub-Type. *Table 13-10* lists the more commonly used extended community formats together with the associated Value field construction. The full list of extended community Types and Sub-Types is maintained at IANA and can be found here [Extended Community Type/Sub-Type](#).

Table 13-10: Extended Community Formats

Type	EXT COMM Format	Value Field Construction
0x00 (transitive) or 0x40 (non- transitive)	Two-octet AS-Specific extended community	Value field consists of two sub-fields. A 2-octet Global Administrator field containing an AS number, followed by a 4-byte Local Administrator field that can be used by the owner of the AS number in the Global Administrator field to encode any information.
0x01 (transitive) or 0x41 (non- transitive)	IPv4 Address Specific extended community	Value field consists of two sub-fields. A 4-octet Global Administrator field containing an IPv4 address, followed by a 2-byte Local Administrator field that can be used by the

		owner of the IP address in the Global Administrator field to encode any information
0x02 (transitive) or 0x42 (non- transitive)	Four-octet AS-Specific extended community	Value field consists of two sub-fields. A 4-octet Global Administrator field containing an AS number, followed by a 4-byte Local Administrator field that can be used by the owner of the AS number in the Global Administrator field to encode any information
0x03 (transitive) or 0x43 (non- transitive)	Opaque extended community	Value field is generic and defined by Sub-Type (IANA assigned)

The Sub-Type field is used to indicate a particular type of extended community. Hence, when used together with the Type field it is possible to interpret the format of the Value field together with the extended community type. *Table 13-11* lists some commonly used Extended Communities together with their Type and Sub-Type codes.

Table 13-11: Extended Communities

Ext Comm	Type	Sub-Type	Remarks
Route Target	0x00	0x02	Global Administrator field contains an AS number. Local Administrator field contains a number.
	0x01	0x02	Global Administrator field contains an IP address. Local Administrator field contains a number.
	0x02	0x02	Global Administrator field contains an AS number. Local Administrator field contains a number.
Route Origin	0x00	0x03	Global Administrator field contains an AS number. Local Administrator field contains a number.
	0x01	0x03	Global Administrator field contains an IP address. Local Administrator field contains a number.
	0x02	0x03	Global Administrator field contains an AS number. Local Administrator field contains a number.
Link Bandwidth	0x40	0x04	Global Administrator field contains an AS number. Local Administrator field contains a number.
Origin Validation	0x43	0x00	Last octet encodes route's validation state: value 0: "valid", value 1: "not found", value 2: "invalid"
Color	0x03	0x0b	Value field encoded as 2-byte Flags field followed by 4-octet Color value. Two flags are defined (C and O) that are used for Color-Only BGP destination steering

When using the generic extended community format for filtering, the format is as follows:

ext:<value1><value2>

Where:

ext	A keyword (similar to target or origin)
value1	Exactly 4 hex digits to produce any value in the range 0x0000 - 0xffff. Leading digits must not be suppressed in <i>value1</i> . This represents match criteria for the Type and Sub-Type fields.
value2	Between 1 and 12 hex digits to product any value in the range 0x0 and 0xffffffffffff. Leading digits may be suppressed in <i>value2</i> . This represents match criteria for the Value field.

By way of a simple example, a VPN-IPv4 prefix 64496:1:172.31.3.0/24 with extended community Route-Target 64496:1 is injected towards router R1. A local VPRN (1) is configured at R1 with a **vrf-import policy** "VRF1-IMPORT". That policy accepts prefixes with the community "*vrf1-ext-comm-rt*" and has a **default-action** of **reject**.

Output 13-112: Route Target Hex Community at R1

```

policy-options {
  community "vrf1-ext-comm-rt" {
    member "ext:0002:fbf000000001" { }
  }
  policy-statement "VRF1-IMPORT" {
    entry 10 {
      from {
        community {
          name "vrf1-ext-comm-rt"
        }
      }
      action {
        action-type accept
      }
    }
    default-action {
      action-type reject
    }
  }
}

```

The community "*vrf1-ext-comm-rt*" is constructed as follows:

ext	Keyword
value1	Hex value 0002 representing Type/Sub-Type of Route-Target (<i>Table 13-11</i>) with a Global Administrator field containing an AS number and a Local Administrator field containing a number.
value2	Hex value fbf000000001 representing a Value field of 64496 (fbf0) in the Global Administrator field and 1 (1) in the Local Administrator field.

Verification can be made that the community value in the policy has successfully matched the VPN-IPv4 prefix and that it is imported into the relevant VPRN. (Modified attributes are not shown for conciseness and remain the same as the original attributes.)

Output 13-113: Verification of hex Route Target match at R1

```

A:admin@R1# show router bgp routes 172.31.3.0/24 vpn-ipv4 detail | match
'Network|Community|Flags|VPRN Imported'
Network      : 172.31.3.0/24

```

```
Community      : target:64496:1
Flags          : Used Valid Best Group-Best
VPRN Imported  : 1
```

As a further example, an IPv4 prefix 10.174.96.0/21 with extended community Link bandwidth 64496:1000 and Standard Community 64511:100 is advertised to router R1. Router R1 would like to match on the Link Bandwidth extended community and set the Local-Preference attribute to 200. The following policy is therefore configured at R1 and applied as a BGP import policy.

Output 13-114: Link Bandwidth hex community at R1

```
policy-options {
  community "Bandwidth-EXT-COMM" {
    member "ext:4004:fbf04cee6b28" { }
  }
  policy-statement "BGP-import" {
    entry 10 {
      from {
        community {
          name "Bandwidth-EXT-COMM"
        }
      }
      action {
        action-type accept
        local-preference 200
      }
    }
    default-action {
      action-type accept
    }
  }
}
```

The community “*bandwidth-ext-comm*” is constructed as follows:

ext	Keyword
value1	Hex value 4004 representing Type/Sub-Type of Link Bandwidth (<i>Table 13-11</i>) with a Global Administrator field containing an AS number and a Local Administrator field containing a number.
value2	Hex value fbf04cee6b28 representing a Value field of 64496 (fbf0) in the Global Administrator field and a value of 4cee6b28 in the Local Administrator field. Link Bandwidth is expressed as IEEE floating point format in units of bytes. The 1Gb/s bandwidth is equivalent to 125000000 bytes/second, which in IEEE floating point format has a hex value of 4cee6b28.

Output 13-115 shows the original and modified attributes after the import policy has been applied. From this output it can be observed that the hex extended community has made a match and modified the Local Preference attribute to 200.

Output 13-115: Verification of hex Link Bandwidth match at R1

```
A:admin@R1# show router bgp routes 10.174.96.0/21 detail | match 'Network|Local
Pref|Community|Flags|Original|Modified Attr'
Original Attributes
Network          : 10.174.96.0/21
```

```

Local Pref.      : 100                               Interface Name : NotAvailable
Community       : 64511:100 bandwidth:64496:1000
Flags           : Used Valid Best Group-Best In-RTM

Modified Attributes
Network         : 10.174.96.0/21
Local Pref.     : 200                               Interface Name : NotAvailable
Community       : 64511:100 bandwidth:64496:1000
Flags           : Used Valid Best Group-Best In-RTM

```

Regular expressions can also be used with hex Extended Communities to provide a flexible way to remove multiple Extended Communities from a BGP Update. When using regular expressions for extended community filtering, the format is as follows:

ext:<regex1><regex2>

Where:

ext	A keyword (similar to target or origin)
regex1	A regular expression that can match a pattern of four hex digits representing the match criteria for the Type and Sub-Type fields. Leading zeroes are not stripped when the first two bytes are passed to the regex engine. The regex parser considers each hex digit to be an elemental term.
regex2	A regular expression that can match a pattern of between 1 and 12 hex digits representing the match criteria for the Value field. Leading zeroes are stripped when the last six bytes are passed to the regex engine. The regex parser considers each hex digit to be an elemental term.

When regular expressions are used for hex filtering of Extended Communities, it is not possible to add an extended community using the format ext:<regex1><regex2>. The intention is to provide flexibility for removing Extended Communities. When a match is found during policy entry parsing, all communities that match ext:<regex1><regex2> will be removed.

As an example, assume an operator that provides an IP-VPN service using BGP/MPLS IP-VPNs where PE-CE peering uses EBGp. For security purposes, the operator would like to ensure that all Route-Target and Origin Extended Communities are removed in Updates received from the CE. The policy shown in *Output 13-116* is therefore applied. In reality, this example policy would probably be part of a wider import policy, but it serves to illustrate the use of regular expressions with hex filtering. The regular expressions in the community “Remove-EXT-COMM” match Type field 00, 01, and 02 with a single ‘.’ and Sub-Types 02 (Route-Target) and 03 (Origin). The Value field can be any value. The policy-statement “PE-CE-Import” then references the community list as **from** criteria and accepts the routes removing any community that matches the same regular expression.

Output 13-116: Extended Community Hex filtering with regular expressions

```

policy-options {
  community "Remove-EXT-COMM" {
    expression {

```

```

        expr "(ext:0.02&.*) OR (ext:0.03&*)"
    }
}
policy-statement "PE-CE-Import" {
    entry 10 {
        from {
            community {
                name "Remove-EXT-COMM"
            }
        }
        action {
            action-type accept
            community {
                remove ["Remove-EXT-COMM"]
            }
        }
    }
    default-action {
        action-type accept
    }
}
}

```

The policy is applied at the PE to an EBGp peering with a CE router. The CE router is subsequently configured to advertise prefix 172.31.1.0/24 with Extended Communities Route-Target 64512:100, Route-Target 64512:200 and Origin:64512:300. *Output 13-117* shows the original attributes received with the prefix containing the Extended Communities, together with the modified attributes showing the communities removed by the import policy.

Output 13-117: Verification of hex regular expression

```

A:admin@R1# show router 1 bgp routes 172.31.1.0/24 detail | match
'Network|Community|Original|Modified Attr'
Original Attributes
Network      : 172.31.1.0/24
Community    : origin:64512:300 target:64512:100 target:64512:200

Modified Attributes
Network      : 172.31.1.0/24
Community    : No Community Members

```

BGP Conditional Advertisement

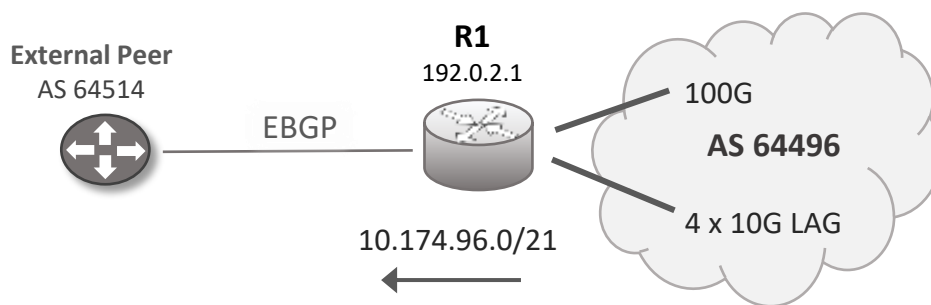
In general, when a BGP route is the best route to a given destination it is installed in the FIB and advertised to other BGP peers (subject to RIB-Out policy). Conditional BGP route advertisement is a feature that allows the user to specify whether or not a BGP route should be advertised based on the existence or non-existence of another route in the routing table. For example, advertise prefix P1 only if prefix P2 exists in the route-table. Or advertise prefix P3 only if prefix P4 does not exist in the route-table.

The BGP conditional route advertisement feature is implemented using the route-policy framework that has been discussed throughout this section. However, up to this point each entry within a **policy-statement** has primarily been formed of three parts. The **from** context

is used to define match criteria, the **to** context is used to define who/what can receive the prefix(es), and the **action** context is used to accept/reject and/or manipulate prefixes that match the **from/to** criteria. In order to implement BGP conditional route advertisement, a new criteria is introduced known as the **route-exists** condition. The **route-exists** condition adds a fourth criteria to each route-policy entry. The to/from/action criteria defined within each policy entry is only evaluated if the route defined in the route-exists criteria matches.

As an example, consider the case in *Figure 13-31*. Router R1 in AS 64496 peers externally with its transit provider in AS 64514. Router R1 advertises prefix 10.174.96.0/21 to AS 64514, which it learns internally with community value 64496:100 attached. Router R1 has two internal IP interfaces to the core; a 4 x 10G LAG and a 100G link. The administrator in AS 64496 would like to advertise prefix 10.174.96.0/21 as long as the 100G link is operational. If the 100G link should fail, router R1 should make the prefix 10.174.96.0/21 preferable through some other peering router (not shown).

Figure 13-31: BGP conditional route advertisement example



Output 13-118 provides an example of the required policy at R1 to implement the conditional route advertisement. The **prefix-list** “100G-link-to-Core” contains a single prefix which equates to the /30 subnet used for the 100G interface towards the core. The **community** list “Internal-Routes” is used to match internal prefixes advertised in IBGP to R1, which R1 should advertise externally. The **policy-statement** “AS64514-Export-Policy” thereafter contains two entries.

- **Entry 10** begins with a **conditional-expression** context, which allows for the definition of the **route-exists** criteria. In this case, the **route-exists** criteria references the **prefix-list** “100G-link-to-Core” containing the IP subnet of the 100G link to the core. If this route does not exist in the route-table, entry 10 is no longer evaluated. If the route does exist, any prefixes with community “Internal-Routes” are accepted and eligible for route advertisement by entry 10. The **route-exists** configuration line has an optional **all** or **none** argument following the **prefix-list** reference. The keyword **all** specifies that all prefixes in the referenced **prefix-list** must exist in the route-table in order for policy evaluation of the entry to continue. The keyword **none** specifies that none of the prefixes in the referenced **prefix-list** must exist in the route-table in order for the policy evaluation of the entry to continue. If neither the **all** or **none** arguments are used the match logic used is ANY, meaning that the policy evaluation of the entry will continue if any prefix in the specified **prefix-list** is active in the route-table.

- **Entry 20** is configured in case the **route-exists** criteria within entry 10 is not met. That is, if the 100G link (192.168.0.0/30) does not exist in the route-table and entry 10 is not evaluated. In this event, entry 20 accepts any prefixes from the community list “Internal-Routes” and will advertise them to AS 64514 prepending AS 64496 to the AS_PATH attribute three times. This ensures that R1 is not the preferred entry point within AS 64496 for prefix 10.174.96.0/21.

The policy is applied as an export policy towards the transit provider. It should be noted that policies with conditional entries are only supported when used in a BGP export policy or a VRF export policy. It is supported in the base routing context, and within an IP-VRF.

Output 13-118: Conditional route advertisement policy at R1

```

policy-options {
  community "Internal-Routes" {
    member "64496:100" { }
  }
  prefix-list "100G-link-to-Core" {
    prefix 192.168.0.0/30 type exact {
    }
  }
}
policy-statement "AS64514-Export-Policy" {
  entry 10 {
    conditional-expression {
      route-exists "[100G-link-to-Core] all"
    }
    from {
      community {
        name "Internal-Routes"
      }
    }
    action {
      action-type accept
    }
  }
  entry 20 {
    from {
      community {
        name "Internal-Routes"
      }
    }
    action {
      action-type accept
      as-path-prepend {
        as-path 64496
        repeat 3
      }
    }
  }
}
}

```

When the 100G link to the core is operational at R1, the prefix 172.16.32.0/20 is advertised to the transit provider without any AS_PATH manipulation (other than pushing on AS 64496) as shown in *Output 13-119*.

Output 13-119: RIB-Out entry for prefix 10.174.96.0/21 with 100G link operational

```

A:admin@R1# show router bgp routes 10.174.96.0/21 hunt | match "RIB Out" post-lines 21
pre-lines 1

```



```

-----
RIB Out Entries
-----
Network       : 10.174.96.0/21
Nexthop       : 192.168.2.1
Path Id       : None
To            : 192.168.2.2
Res. Protocol : INVALID                      Res. Metric    : 0
Res. Nexthop  : n/a
Local Pref.   : n/a                        Interface Name : NotAvailable
Aggregator AS : None                      Aggregator    : None
Atomic Aggr.  : Not Atomic                MED           : None
AIGP Metric   : None                      IGP Cost      : 200
Connector     : None
Community     : 64496:100
Cluster       : No Cluster Members
Originator Id : None                      Peer Router Id : 192.0.2.101
Origin        : IGP
AS-Path       : 64496
Route Tag     : 0
Neighbor-AS   : 64496
DB Orig Val   : NotFound                  Final Orig Val : N/A
Source Class  : 0                        Dest Class     : 0

```

If a failure of the 100G link to the core is simulated, the prefix 192.168.0.00/30 is no longer active in the route-table. As a result, entry 10 of the policy shown in *Output 13-118* is not evaluated. As prefixes are parsed through policy statement on an entry-by-entry basis in numerical order, the prefix moves to entry 20, where the prefix is accepted but AS 64496 is prepended three times (in addition to the normal inclusion of AS into the AS_PATH attribute) as shown in *Output 13-120*. When the failed link is restored, entry 10 will once again be evaluated and only a single imposition of AS 64496 will be made in the AS_PATH. In other words, the **route-exists** condition provides a revertive behaviour.

Output 13-120: RIB-Out entry for prefix 10.174.96.0/21 with 100G link down

```

A:admin@R1# show router bgp routes 10.174.96.0/21 hunt | match "RIB Out" post-lines 21
pre-lines 1
-----
RIB Out Entries
-----
Network       : 10.174.96.0/21
Nexthop       : 192.168.2.1
Path Id       : None
To            : 192.168.2.2
Res. Protocol : INVALID                      Res. Metric    : 0
Res. Nexthop  : n/a
Local Pref.   : n/a                        Interface Name : NotAvailable
Aggregator AS : None                      Aggregator    : None
Atomic Aggr.  : Not Atomic                MED           : None
AIGP Metric   : None                      IGP Cost      : 400
Connector     : None
Community     : 64496:100
Cluster       : No Cluster Members
Originator Id : None                      Peer Router Id : 192.0.2.101
Origin        : IGP
AS-Path       : 64496 64496 64496 64496
Route Tag     : 0
Neighbor-AS   : 64496
DB Orig Val   : NotFound                  Final Orig Val : N/A
Source Class  : 0                        Dest Class     : 0

```



Note: SR-OS also supports a form of conditional route advertisement that generates a static-route based on the existence or non-existence of a prefix or list of prefixes. Once the static-route is active, it can be exported into any protocol using the route-policy framework. Whilst this feature will cover many scenarios, it does not preserve BGP attributes where the route is learnt in BGP and conditionally advertised in BGP.

Policy Verification (BGP Policy-Test)

Applying policy to existing BGP neighbours can sometimes lead to unexpected situations such as a disruption in service or suboptimal routing, if, for example, prefixes are rejected that weren't intended to be or are advertised with incorrect attributes. This could be as a result of a configuration error in the policy, or perhaps just a basic misunderstanding of how the policy would be implemented. The BGP policy test is a useful tool that allows a user to evaluate the effects of a configured policy that exists in the running configuration before actually applying that policy to one or more peers. Evaluating the policy before it is applied allows the operator to verify that the policy does exactly what it is intended in a non-disruptive manner.

By way of a simple example, assume that router R1 has an external peer in AS 64511 that advertises three prefixes:

- Prefix 172.16.0.0/20 with community value 64511:100
- Prefix 172.16.16.0/20 with community value 64511:100
- Prefix 172.16.32.0/20

The following simple policy is configured and committed at router R1 but is not applied to any peer. Entry 10 of the policy rejects prefixes in the range 172.16.0.0/16 or longer that have community value 64511:100. Entry 20 of the policy then accepts any other prefixes in that same range.

Output 13-121: Policy to be evaluated at router R1

```

policy-options {
  community "test-comm" {
    member "64511:100" { }
  }
  prefix-list "test-prefix" {
    prefix 172.16.0.0/16 type longer {
    }
  }
  policy-statement "test-policy" {
    entry 10 {
      from {
        prefix-list ["test-prefix"]
        community {
          name "test-comm"
        }
      }
      action {
        action-type reject
      }
    }
  }
}

```

```

    entry 20 {
        from {
            prefix-list ["test-prefix"]
        }
        action {
            action-type accept
        }
    }
}
}

```

The BGP policy-test tool is then used to run the prefix range 172.16.0.0/16 or longer through the above policy before the policy is applied to any peer. The result is shown in *Output 13-122*. When using the policy-test tool, by default, only prefixes that are accepted are shown with a summary of the number of rejected prefixes, so this example uses the optional argument “**display rejects**” to also include prefixes that are rejected. As can be seen from the output, the prefixes 172.16.0.0/20 and 172.16.16.0/20 carrying the community attribute 64511:100 are both rejected by entry 10 of the policy, whereas the prefix 172.16.32.0/20 is accepted.

Output 13-122: Result of BGP policy-test at router R1

```

A:admin@R1# show router bgp policy-test plcy-or-long-expr "test-policy" prefix
172.16.0.0/16 longer display-rejects
=====
BGP Router ID:192.0.2.1          AS:64496          Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP IPv4 Routes
=====

```

Network	LocalPref	MED
Nexthop	Path-Id	Label
As-Path		

Rejected by Policy test-policy Entry 10		
172.16.0.0/20	None	None
192.168.1.2	None	0
64511		-
Rejected by Policy test-policy Entry 10		
172.16.16.0/20	None	None
192.168.1.2	None	0
64511		-
Accepted by Policy test-policy Entry 20		
172.16.32.0/20	None	None
192.168.1.2	None	n/a
64511		-

Total Routes : 3 Routes rejected : 2		
=====		

Of course, this simple example only evaluates prefixes in the range 172.16.0.0/16 or longer, but it is frequently a requirement to evaluate all BGP routes in the route-table. To do this, the operator simply needs to modify the above example to use the prefix **0.0.0.0/0 longer**. The policy-test tool works for both IPv4 and IPv6 address families using the **family** argument and

works in the base or IP-VRF routing instances, with the base router being the default. The optional **detail** extension shows the original attributes together with any modified attributes as a result of the exercised policy. It is also possible to run the policy-test tool only for routes learnt from a particular neighbour by using the **neighbor** argument.

The policy-test is an extremely useful tool where there is a requirement to implement “on-the-fly” changes to policy such as at Internet peering points.

Triggered Policy

By default, when a user enters the **commit** command after modifying route-policy, any newly configured and/or edited policies are applied immediately to peers configured with those policies. For IPv4/IPv6 prefixes, SR-OS keeps a copy of any routes rejected by import policy in the Adj-RIB-In. As a result, when new or updated policy is applied the only requirement is to run the policy across the routes in the Adj-RIB-In without the need for any kind of Route-Refresh mechanism. For VPN-IPv4/IPv6 routes, the default behaviour is to discard routes that are not imported into any VRFs and use the Route-Refresh mechanism when policy is changed, but this behaviour can be changed using the **mp-bgp-keep** command under the global BGP context as described in Chapter 3.

This behaviour of applying policy immediately is suitable for most environments, particularly as the policy can be validated before applying it using the policy-test command. However, where a common policy is applied to a number of peers it may be desirable to activate a newly-configured policy or policy change on a peer-by-peer basis. This is the purpose of triggered policies. When triggered policies are enabled, new or updated policies only take affect after the peer undergoes a soft-reset.

Triggered policies are enabled under the global router context using the command **triggered-policy true**. When enabled, new or updated export policies are applied to peers using the command “**clear router bgp neighbor <ip-address> soft**”, while new or updated import policies are applied to peers using the command “**clear router bgp neighbor <ip-address> soft-inbound**”.

Policy Accounting

Policy Accounting allows IP interface statistics to be collected and retrieved on the basis of source and/or destination IP traffic. It classifies the measured traffic based on BGP attributes such as communities or AS-PATH. These accounting statistics can provide useful information to an operator for:

- Traffic accounting: Accounting for traffic from a particular source AS, which may be useful for billing/service agreements, or peering arrangements.
- Traffic Analysis: Statistics provide useful information for analysing traffic patterns and modelling what-if scenarios.

- Policy enforcement: Once traffic is classified, it can be used for other purposes such as rate enforcement.
- Troubleshooting: Statistics may prove useful for ad-hoc troubleshooting.

Policy accounting associates ‘classes’ with static or BGP learnt IPv4 and IPv6 routes. Traffic based on source criterion is defined by a “source-class” index, whilst traffic based on destination criterion is defined by a “destination-class” index. Valid routes are installed into the route table with the associated source-class or destination-class index attached.

Both source-class and destination-class indices are contained within a policy accounting template. When ingress policy accounting is enabled on an IP interface, and the referenced policy accounting template stipulates that statistics for source-class *s* should be collected, every ingress IPv4 and IPv6 packet on that interface that matches source-class *s* is counted. A packet is considered to match a source-class *s* if the route that would be used to (theoretically) forward the packet towards the source address is associated with source-class index *s*. Equally, if the referenced policy accounting template stipulates that statistics for destination-class *d* should be collected, every ingress IPv4 and IPv6 packet on that interface that matches destination-class *d* is counted. A packet is considered to match destination-class *d* if the route that would be used to actually forward the packet is associated with destination-class index *d*.

The gathered statistics require a repository such that an incremental count of packets and bytes can be maintained. In this case, the repository are policers which allow for statistics collection and optionally traffic policing:

- Per-interface, per-class traffic statistics are supported on linecards of FP2 or later generations.
- Per-interface, per-FP, per-class traffic policing is supported for destination classes and only by linecards of FP4 or later generations

This section focuses on the former use case of traffic statistic collection.

Before policy accounting can be enabled on an IP interface, each FP on the linecard supporting that IP interface must be configured to allocate some policer statistics resources and source/destination class indices for policy accounting, as shown in *Output 13-123*. For policy accounting, one policer statistic resource is required for every source-class and/or destination-class index listed in the template applied to the interface; where each policer statistic resource represents a single 8-byte counter for packets forwarded, and a single 8-byte counter for bytes forwarded. The **classes** parameter refers to the source and destination class indexes, each of which has global meaning. For example, a destination-class of 2 in template one refers to the same set of routes as a destination-class of 2 in template two. Conversely, policer indexes have local scope to the referring template. In template one destination-class 2 could use policer index 4 whereas destination-class 2 in template two could use policer index 24. SR-OS enforces that the number of **classes** must be equal to or greater than the number of configured **policers**.

Output 13-123: FP policer and classifier resource allocation

```

card 1 {
    fp 1 {
        ingress {
            policy-accounting {
                policers 1024
                classes 4096
            }
        }
    }
}

```

Policer statistics are a common resource pool shared by policing and policy accounting. The total number of available policer statistics resources available and allocated on a given line-card FP complex can be displayed as shown in *Output 13-124*.

Output 13-124: Per-FP policer resource allocation

```

A:admin@R1# tools dump resource-usage card 1 fp 1 | match 'Total|Policer'

```

	Total	Allocated	Free
Ingress Policers	393215	1025	392190
Ingress Policer Stats	524255	2048	522207
Egress Policers	393215	1	393214
Egress Policer Stats	524255	0	524255
Dynamic Policers +	0	0	0
Dynamic Policers (in use by Ingress) -		0	
Dynamic Policers (in use by Egress) -		0	
Dynamic Policer Stats +	0	0	0
Dynamic Policer Stats (in use by Ingress) -		0	
Dynamic Policer Stats (in use by Egress) -		0	
Egress User Policers	393215	0	393215

Equally, it is possible to verify how many of the statistics resources allocated to policy accounting are actually in use.

Output 13-125: Policy-accounting resource consumption

```

A:admin@R1# show card 1 fp 1 policy-accounting

```

```

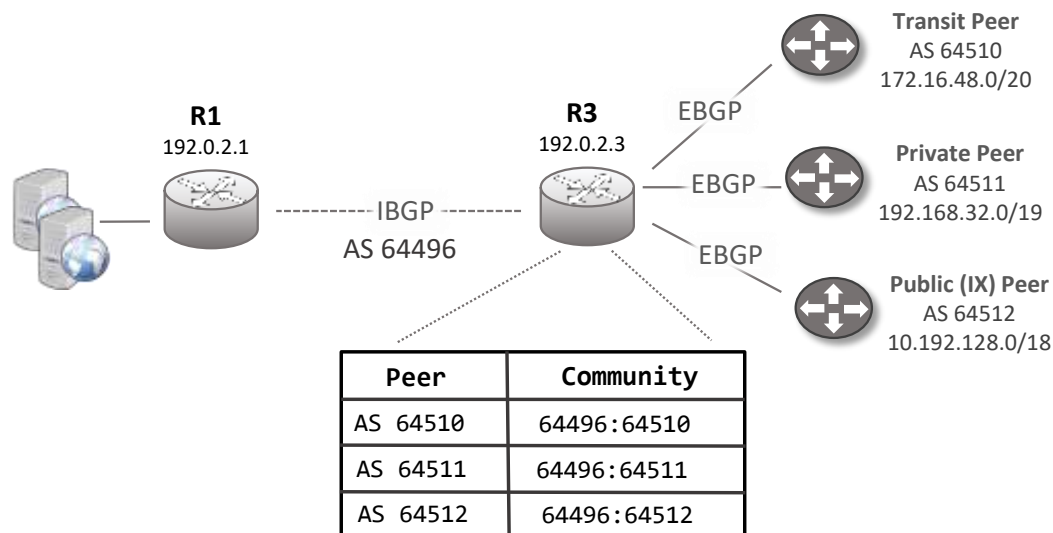
=====
Card : 1 Forwarding Plane(FP) : 1
=====
Policy Accounting Stats Pool Limit : 1024
Stats Resources currently in use : 0
Policy Accounting Policer Limit : 1024
Policer Resources currently in use : 0
=====

```

To illustrate the use of policy accounting, a simple and hypothetical situation is demonstrated where it might be considered useful. In *Figure 13-32*, Router R1 in AS 64496 is locally connected to a web server farm. Router R3 in the same AS is a peering router and advertises the full Internet table to R1 in IBGP. Traffic from router R1 to R3 is forwarded using BGP shortcuts with SR-ISIS LSPs. The operator of AS 64496 would like to understand how much traffic from the server farm is forwarded to each of the peers. The first step is to make routes from each peer distinguishable from each other, so router R3 is configured to tag routes received from each of the peers with a community value before advertising in IBGP to R1:

- Prefixes received from the transit peer AS 64510 (example used is 172.16.48.0/20) are tagged with community value 64496:64510.
- Prefixes received from the private peer AS 64511 (example used is 192.168.32.0/19) are tagged with community value 64496:64511.
- Prefixes received from the Internet Exchange public peer AS 64512 (example used is 10.192.128.0/18) are tagged with community value 64496:64512.

Figure 13-32: Policy accounting example topology



At router R1, the first step is to define a policy accounting template with the relevant source-class and/or destination-classes and associated policers. Because different sets of source and destination classes may be of interest on different interfaces, up to 1024 different **policy-acct-templates** can be supported, and within each template up to 255 source-class and/or destination-class indices may be configured. In this example, the requirement is to identify the amount traffic forwarded from the server farm towards each one of three destinations, so three **destination-class** indices are defined, and each one is associated with a policer.

Output 13-126: Policy Accounting template

```

routing-options {
  policy-accounting {
    policy-acct-template "POL-ACCT-TEMP-1" {
      destination-class 1 {
        policer 1
      }
      destination-class 2 {
        policer 2
      }
      destination-class 3 {
        policer 3
      }
      policer 1 {
      }
      policer 2 {
      }
      policer 3 {
      }
    }
  }
}

```

```

    }
  }
}

```

The next step is to associate routes learned at router R1 with one of the pre-defined destination-classes. To facilitate this, the route-policy framework is used to match the relevant community value set by router R3 and map it to the appropriate **dest-class**. The example policy is shown in *Output 13-127* where community 64496:64510 (AS 64510) is associated with **dest-class 1**, community 64496:64511 (AS 64511) is associated with **dest-class 2**, and community 64496:64512 (AS 64512) is associated with **dest-class 3**.

Output 13-127: Route-policy with source/destination classes

```

policy-options {
  community "AS64510" {
    member "64496:64510" { }
  }
  community "AS64511" {
    member "64496:64511" { }
  }
  community "AS64512" {
    member "64496:64512" { }
  }
  policy-statement "POLICY-ACCT" {
    entry 10 {
      from {
        community {
          name "AS64510"
        }
      }
      action {
        action-type accept
        dest-class 1
      }
    }
    entry 20 {
      from {
        community {
          name "AS64511"
        }
      }
      action {
        action-type accept
        dest-class 2
      }
    }
    entry 30 {
      from {
        community {
          name "AS64512"
        }
      }
      action {
        action-type accept
        dest-class 3
      }
    }
    default-action {
      action-type accept
    }
  }
}

```


When a route-policy has been configured to include an association with a source-class or destination-class, that policy can only be applied as a RIB-In (import) policy. It can be applied in the following scenarios:

- Base BGP import policies (either at base level, group level, or neighbour level).
- VRF import policies.
- VPRN BGP import policies (either at VPRN BGP level, group level, or neighbour level)

In the example of *Figure 13-32*, the route-policy is applied at R1 within the base BGP instance on the IBGP session towards R3 in AS 64496. Note that the **default-action** has an **action-type** of **accept** so that the setting of these **dest-class** indices does not impact any other received routes. *Output 13-128* shows the outcome at R1 on the prefixes received from R3 after the policy has been applied, with the modified attributes showing an association with the appropriate dest-class.

Output 13-128: Source/Destination-Class associated with routes

```
A:admin@R1# show router bgp routes 172.16.48.0/20 detail | match 'Network|Community|Dest
Class|Original|Modified Attr'
Original Attributes
Network      : 172.16.48.0/20
Community    : 64496:64510
Source Class : 0
Dest Class   : 0
Modified Attributes
Network      : 172.16.48.0/20
Community    : 64496:64510
Source Class : 0
Dest Class   : 1

A:admin@R1# show router bgp routes 192.168.32.0/19 detail | match
'Network|Community|Dest Class|Original|Modified Attr'
Original Attributes
Network      : 192.168.32.0/19
Community    : 64496:64511
Source Class : 0
Dest Class   : 0
Modified Attributes
Network      : 192.168.32.0/19
Community    : 64496:64511
Source Class : 0
Dest Class   : 2

A:admin@R1# show router bgp routes 10.192.128.0/18 detail | match
'Network|Community|Dest Class|Original|Modified Attr'
Original Attributes
Network      : 10.192.128.0/18
Community    : 64496:64512
Source Class : 0
Dest Class   : 0
Modified Attributes
Network      : 10.192.128.0/18
Community    : 64496:64512
Source Class : 0
Dest Class   : 3
```

The source- and/or destination-class associated with a given prefix can also be viewed in the route-table using the **accounting-class** extension.

Output 13-129: Route-table entry with accounting class

```
A:admin@R1# show router route-table 172.16.48.0/20 accounting-class

=====
Route Table (Router: Base)
```

```

=====
Dest Prefix[Flags]                                Type      Proto      Age          Pref
      Next Hop[Interface Name]                                Metric
                                      Src-Class Dest-Class
-----
172.16.48.0/20                                     Remote    BGP        00h29m03s    170
      192.0.2.3 (tunneled:SR-ISIS:524298)                200
                                      0          1
-----
No. of Routes: 1
Flags: n = Number of times nexthop is repeated
      B = BGP backup route available
      L = LFA nexthop available
      S = Sticky ECMP requested
=====

```



Note: As previously mentioned, in addition to BGP, policy accounting is also supported for static routes. Although no explicit configuration example is provided for static routing, the only derivation from policy accounting for BGP is that the source/destination-class is appended to the route as an additional argument within the **static-route** context as opposed to using route-policy. An example of the command structure would be **prefix 192.168.100.0/24 unicast next-hop 192.168.0.2 dest-class 1**.

The final step in enabling policy accounting is to apply the **policy-acct-template** to the relevant IP interface. Policy accounting is supported on network interfaces, IES interfaces, VPRN interfaces, and spoke-SDP interfaces, but is currently only supported on ingress. *Output 13-130* shows the **policy-acct-template** applied at router R1 on the interface connecting the server farm, using the command **ingress policy-accounting**.

Output 13-130: Applying policy accounting template

```

service {
  ies "one" {
    interface "to-Server-Farm" {
      ingress {
        policy-accounting "POL-ACCT-TEMP-1"
      }
    }
  }
}

```

The collected statistics can be viewed through CLI as shown in *Output 13-131*, and/or can be collected through YANG state or SNMP using the associated MIB. It should be noted that the statistics include IP payload only, but do not include any Layer-2 overhead. Equally, the statistics do not distinguish or separate between IPv4 and IPv6 packets that match a particular source-class or destination-class but rather represents them as an aggregate. (If separate counters are a real requirement, IPv4 and IPv6 routes could be mapped to discreet non-overlapping classes).

Output 13-131: Viewing collected statistics

```

A:admin@R1# show router interface "EBGP-to-ExternalPeer" policy-accounting

```

```

=====
Accounting Statistics
=====
If Name           : EBGp-to-ExternalP*
Admin State       : Up                               Oper State (v4/v6) : Up/Up
-----
Class             : dest-class 1                     Policer           : 1
Rx Fwd Packets    : 509                             Rx Fwd Bytes      : 256356
-----
Class             : dest-class 2                     Policer           : 2
Rx Fwd Packets    : 328                             Rx Fwd Bytes      : 239524
-----
Class             : dest-class 3                     Policer           : 3
Rx Fwd Packets    : 400                             Rx Fwd Bytes      : 272800
=====
Policer Stats
-----
Policer           : 1
Drop Packets      : 0                               Drop Bytes        : 0
Fwd Packets       : 509                             Fwd Bytes         : 267554
-----
Policer           : 2
Drop Packets      : 0                               Drop Bytes        : 0
Fwd Packets       : 329                             Fwd Bytes         : 239524
-----
Policer           : 3
Drop Packets      : 0                               Drop Bytes        : 0
Fwd Packets       : 400                             Fwd Bytes         : 281600
=====
* indicates that the corresponding row element may have been truncated.

```

If required, policy accounting statistics can be cleared and reset to zero using the command ***clear router interface <name> policy-accounting***. This command can be extended to identify only a particular source- or destination-class index.

BGP in Residential Networks

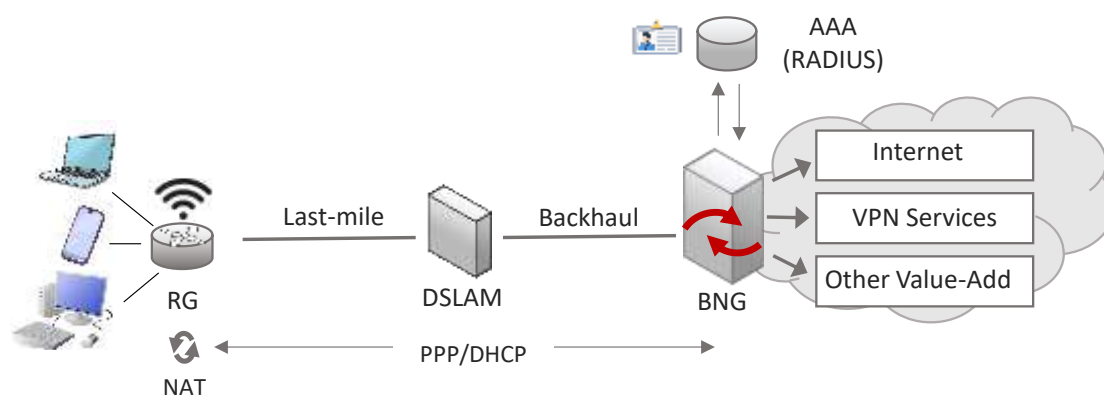
Access to the Internet using fixed broadband infrastructure has been an area of significant development over the last two decades. The growth is not just in its widespread accessibility and availability to residential users (to the point where router in a home is considered just another ‘utility’ like a refrigerator or freezer), but also last-mile access speeds have significantly improved because of the increased rollout of fibre (FTTC, FTTH), the advancement of DSL (VDSL, G.Fast), and in some regions the availability of Fixed Wireless Access (FWA). The result is that broadband infrastructure has become an appealing last-mile access mechanism for either primary or backup lines for enterprise businesses.

Broadband subscriber sessions (business and residential) are logically terminated on a Broadband Network Gateway (BNG) that provides the interface between the Layer-2 access network and Layer-3 IP services. For residential subscribers the Layer-3 service is typically Internet and/or local caching, but for business users this also extends to other IP-VPN services. The BNG interacts with the subscriber using PPP or DHCP, and because both protocols dictate a degree of state maintenance, they are typically used to implement ‘session awareness’. Session awareness is the ability to authenticate and identify a given subscriber, typically through interaction with RADIUS or DIAMETER, apply policy to that

subscriber, and maintain state for the period of the subscriber session. Policies that can be applied to subscribers are typically dynamic and can implement a number of criteria including, for example, QoS, IP filtering, IP address management, and data accounting for billing purposes.

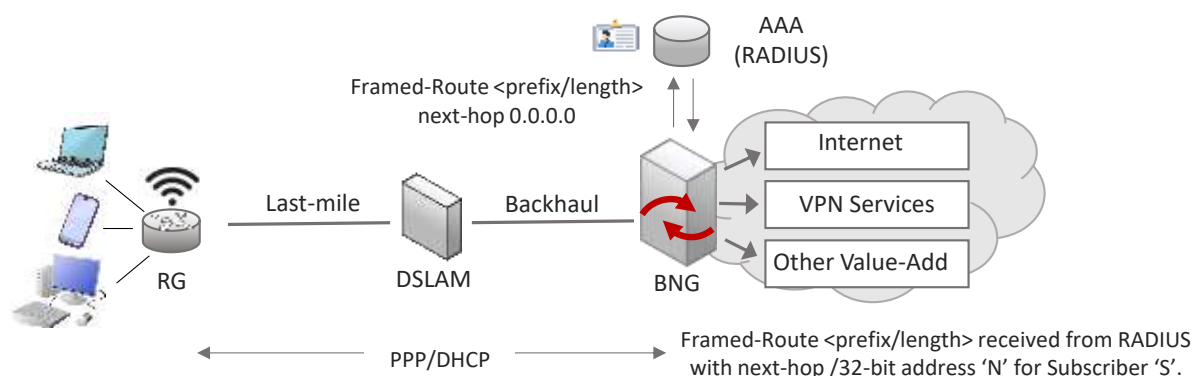
Typically, a residential user is allocated a single IPv4 registered IP address using DHCP or PPP (IPCP), which is applied to the DSL (WAN) side of the home router or residential gateway (RG). The residential user may also be allocated a DHCPv6 Delegated Prefix if the operator is offering dual-stack IPv4/IPv6 to its users. For IPv4, devices that are connected to the LAN side of that RG subsequently use private (RFC1918) addressing and the RG implements Network Address and Port Translation (NAPT) to provide connectivity to the Internet for multiple hosts using a single, visible, registered IP address. The BNG holds an entry in its routing table for that 32-bit IPv4 address for that given subscriber and uses the PPP session, and/or the SVLAN/CVLAN combination, and/or the unicast MAC/IP as a demultiplexer for that subscriber.

Figure 13-33: Residential subscriber using NAT



To attract business services onto its broadband infrastructure, the challenge for the operator is to keep prices competitive whilst offering a level of differentiation over residential users. Much of this differentiation can be achieved with higher levels (or better granularity) of QoS, or lower levels of over-subscription for business users, but enterprises have other requirements that must be considered. One of those requirements is the ability to exchange routing information between sites of its enterprise network without the use of NAT. A method commonly used to achieve this is by using the RADIUS attribute 'Framed-Route'. The 'Framed-Route' attribute looks like a static-route, and takes the form of <prefix/length> and a Next-Hop. The <prefix/length> and Next-Hop tuple may be appended with optional metrics/tags and is passed to the BNG from RADIUS during subscriber activation. (The Next-Hop 0.0.0.0 is a special case that essentially tells the BNG to use the IP address that it allocates to the subscriber from a local address pool as the next-hop for the framed-route.) The RG continues to run PPP/DHCP to obtain an IP address for its DSL (WAN) side, and the BNG uses this WAN-side IP address as the next-hop for the 'Framed-Route', which is used on the LAN side of the RG. Once again the BNG uses the PPP session and/or SVLAN/CVLAN and/or unicast MAC/IP as a demultiplexer for that subscriber.

Figure 13-34: Broadband Subscriber with Framed-Route



A key point with the use of Framed-Route is that there is no control plane interaction between the BNG and RG with regard to the prefix defined in the Framed-Route attribute. The RG LAN is statically configured with the corresponding <prefix/length>. The BNG installs the Framed-Route <prefix/length> into its routing table when the subscriber is activated and removes it from the route-table when the subscriber is de-activated. The point is that they act entirely independently and autonomously.

Framed-Route is widely used for enterprise service delivery over broadband infrastructure, but it has drawbacks. For example, if the enterprise consumer wants to add an additional subnet on the RG LAN side, it is reliant on the service provider to make the change (add another Framed-Route attribute to its RADIUS infrastructure), which may be subject to some delay. Or the RG could potentially have 20 or 30 subnets, which makes the use of Framed-Route cumbersome, or even prohibitive. In this case, the ability to exchange dynamic routing information between the RG and the BNG is needed, and this is obviously where BGP becomes useful. However, this BGP session between the BNG and RG must be dynamic; it is required only when the subscriber is active and can be removed when the subscriber state is removed. That is the purpose of dynamic BGP policies.

Dynamic BGP policies provide the ability to automatically set up a peering session based on parameters from pre-configured BGP policies and is supported for either IPv4 or IPv6 BGP peering. When the subscriber is successfully authenticated, RADIUS returns one or more Vendor Specific Attributes (VSAs) that define the peering policy and parameters. *Table 13-12* lists the available attributes that can be returned from RADIUS. The 'Alc-BGP-Policy' (IPv4 peering) or 'Alc-BGP-IPv6-Policy' (IPv6 peering) attribute is the only mandatory attribute and must be returned when using dynamic BGP peering. Dynamic BGP peering attributes are ignored if the subscriber does not terminate within a VPRN. This makes sense given that the target for the feature is business/enterprise users.

Table 13-12: RADIUS VSAs for dynamic BGP peering

Attribute-ID	Attribute Name	Description
26-6527-55	Alc-BGP-Policy	Mandatory attribute to set up a dynamic BGP peer. References the policy
26-6527-208	Alc-BGP-IPv6-Policy	

		configured under subscriber-mgmt bgp-peering-policy <name>
26-6527-56	Alc-BGP-Auth-Keychain	Optional attribute reference for a TCP-AO keychain
26-6527-209	Alc-BGP-IPv6-Auth-Keychain	
26-6527-57	Alc-BGP-Auth-Key	Optional attribute reference for using MD5 authentication
26-6527-210	Alc-BGP-IPv6-Auth-Key	
26-6527-58	Alc-BGP-Export-Policy	Optional attribute reference for using a BGP route export policy
26-6527-211	Alc-BGP-IPv6-Export-Policy	
26-6527-59	Alc-BGP-Import-Policy	Optional attribute reference for using a BGP route import policy
26-6527-212	Alc-BGP-IPv6-Import-Policy	
26-6527-60	Alc-BGP-PeerAS	Optional attribute for the Autonomous System number for the remote peer
26-6527-213	Alc-BGP-IPv6-PeerAS	

As indicated in *Table 13-12* the 'Alc-BGP-Policy' refers to a BGP peering policy that is configured within the **subscriber-mgmt** context using the somewhat intuitive command **bgp-peering-policy**. Within the policy all the conventional BGP commands are available for use, but the policy does not permit any neighbour group definition.

Output 13-132: Dynamic BGP peering policy configuration

```
subscriber-mgmt {
  bgp-peering-policy "BGP-PEERING-POL" {
    as-override true
    authentication-key "JK/6NxCqPrX7NU7cD3w1PgJOrobFreo= hash2"
    local-address 10.121.0.1
    peer-as 64530
    export ["BGP-VPN-to-BGP-EXPORT"]
    family {
      ipv4 true
    }
  }
}
```

Within the associated VPRN context, there is a requirement to define an **autonomous-system** number, which will be used as the ASN in the Open exchange. Within the VPRN **bgp** context a **group** is defined with the command **static-group** set to **false**. If necessary, additional BGP parameters can be applied here if they are not already defined in the BGP peering policy. The order for BGP parameter selection is as follows:

- Use BGP peering parameters returned in Radius VSAs.
- If BGP peering parameters are not available from RADIUS, use those configured in the **bgp-peering-policy**.
- If BGP peering parameters are not configured in the **bgp-peering-policy**, use those configured in the VPRN BGP group with **static-group** set to **false**.
- If BGP peering parameters are not configured in the group with **static-group** set to **false**, use those configured in the VPRN service BGP context.
- If BGP peering parameters are not configured in the VPRN service BGP context, use the defaults.

Output 13-133: VPRN configuration for dynamic BGP peering

```

service {
    vprn "CP-VRF" {
        autonomous-system 64496
        bgp {
            group "ESM-DYN-PEER-GROUP" {
                admin-state enable
                static-group false
            }
        }
    }
}

```

So far, all the relevant parameters for peering have been defined, but what about the IP addresses that will be used to establish the TCP session? For dynamic BGP peering SR-OS uses the IP address allocated to the subscriber during activation as the neighbour peering address. Using other addresses on the RG such as a loopback (for example, EBGp Multihop) are possible, but because dynamic BGP peering always uses the IP address allocated to the subscriber during activation, peering to a loopback requires that the peer is statically configured. In addition, it also requires that the RG sets the Next-Hop attribute on any Update messages to the subscriber IP address. From the perspective of the RG (or CPE), how the neighbour peering address is derived may change depending on the access method. For example, if PPP is used as an access method, during the IPCP exchange the BNG will send a Configure-Request for a locally configured IP address under the subscriber interface that can be used as the neighbour peering address. If the access mechanism is DHCP, a default-gateway address will be passed to the RG by the BNG in a DHCP Offer. This can be used as a neighbour peering address, or as a Next-Hop to a configured loopback interface. Either way, it is likely that the CPE must be configured with a neighbour address and potentially a static-route to that neighbour address. That address however can be common to all RGs/CPEs that connect to a given BNG. In the example **bgp-peering-policy** shown in *Output 13-132* the **local-address** is derived from the address configured in the subscriber interface.

In this example, RADIUS is configured to pass only the mandatory attribute 'Alc-BGP-Policy' with the pre-configured policy name 'BGP-PEERING-POL' to the BNG when the subscriber is successfully authenticated. *Debug 13-5* shows the RADIUS Access Accept message for a PPP subscriber containing the BGP Policy VSA. Note also that the Framed-IP-Address attribute has a value of 255.255.255.254, which instructs the BNG to use a local pool for address allocation for this subscriber. This locally assigned IP address constitutes the neighbour peering address.

Debug 13-5: RADIUS Access-Accept message with BGP Policy Attribute

```

140 2025/10/20 08:20:31.375 BST minor: DEBUG #2001 vprn901 RADIUS
RADIUS: Receive
Access-Accept(2) id 13 len 218 from 192.168.254.10:1812 vrid 11 pol AAA-SUB-MGMT
MESSAGE AUTHENTICATOR [80] 16 0x2cf85c4bad3251cca60bade6574d9cae
VSA [26] 20 Nokia(6527)
SUBSC ID STR [11] 18 cpe-site1@acme.net
VSA [26] 14 Nokia(6527)
SUBSC PROF STR [12] 12 ESM-SUB-PROF
VSA [26] 14 Nokia(6527)
SLA PROF STR [13] 12 ESM-SLA-PROF
VSA [26] 6 Nokia(6527)

```



```

MSAP SERVICE ID [31] 4 2
SERVICE TYPE [6] 4 Framed(2)
FRAMED PROTOCOL [7] 4 PPP(1)
FRAMED IP ADDRESS [8] 4 255.255.255.254
CLASS [25] 4 0x5342523c
VSA [26] 6 Nokia(6527)
    PRIMARY DNS [9] 4 83.192.47.7
VSA [26] 6 Nokia(6527)
    SECONDARY DNS [10] 4 83.193.47.134
VSA [26] 17 Nokia(6527)
    BGP POLICY [55] 15 BGP-PEERING-POL
VSA [26] 25 Nokia(6527)
    SUBSCRIBER QOS OVERRIDE [126] 23 e:q:7:pir=5000,cir=5000

```

In this subscriber activation, the BNG allocates an IP address of 10.121.0.10 to the subscriber from a local pool, and the subscriber set-up is successful. Note that when using dynamic BGP peering the **anti-spoof** command under the **group-interface sap** or **msap-policy ies-vprn-only-sap-parameters** must be set to **next-hop-ip-and-mac-addr**. Describing the configuration parameters of all the necessary components needed for subscriber management would be non-trivial and would be a document of significant size. It is therefore beyond the scope of this reference as a BGP guide.

Output 13-134: BNG subscriber activation

```

A:admin@BNG1# show service active-subscribers

=====
Active Subscribers
=====
-----
Subscriber cpe-sitel@acme.net
            (ESM-SUB-PROF)
-----
(1) SLA Profile Instance sap:[lag-40:400.1062] - sla:ESM-SLA-PROF
-----
IP Address
-----
MAC Address      Session      Origin      Svc      Fwd
-----
10.121.0.10      00:10:94:00:00:7b  PPP 5      IPCP      2        Y
-----
Number of active subscribers : 1
=====

```

Once the subscriber is active, the command shown in *Output 13-136* can be used to validate that the configured BGP peering policy was selected and successfully installed.

Output 13-135: BGP Peering policy activated for subscriber

```

A:admin@BNG1# show service id 2 ppp session user-name "cpe-sitel@acme.net" detail |
match Bgp
Bgp Peering Policy      : BGP-PEERING-POL
Bgp Peer Status         : installed

```

When the subscriber has been activated and the subscriber IP address is known, the BGP session can be set up, and Update messages exchanged.

Output 13-136: Dynamic BGP peer established

```

A:admin@BNG1# show router 2 bgp summary
=====
BGP Router ID:192.0.2.121      AS:64496      Local AS:64496
=====
BGP Admin State      : Up      BGP Oper State      : Up
Total Peer Groups    : 1      Total Peers          : 1
Current Internal Groups : 1      Max Internal Groups  : 1
Total BGP Paths       : 17     Total Path Memory    : 6288

Total IPv4 Remote Rts : 1      Total IPv4 Rem. Active Rts : 1
Total IPv6 Remote Rts : 0      Total IPv6 Rem. Active Rts : 0
Total IPv4 Backup Rts : 0      Total IPv6 Backup Rts   : 0
Total LblIPv4 Rem Rts : 0      Total LblIPv4 Rem. Act Rts : 0
Total LblIPv6 Rem Rts : 0      Total LblIPv6 Rem. Act Rts : 0
Total LblIPv4 Bkp Rts : 0      Total LblIPv6 Bkp Rts   : 0
Total Supressed Rts   : 0      Total Hist. Rts       : 0
Total Decay Rts       : 0

Total McIPv4 Remote Rts : 0
Total McIPv4 Rem. Active Rts: 0
Total McIPv6 Remote Rts : 0      Total McIPv6 Rem. Active Rts: 0

Total FlowIPv4 Rem Rts : 0      Total FlowIPv4 Rem Act Rts : 0
Total FlowIPv6 Rem Rts : 0      Total FlowIPv6 Rem Act Rts : 0
Total FlowVpnv4 Rem Rts : 0      Total FlowVpnv4 Rem Act Rts : 0
Total FlowVpnv6 Rem Rts : 0      Total FlowVpnv6 Rem Act Rts : 0
Total Link State Rem Rts: 0      Total Link State Rem Act Rts: 0
Total SrPlcyIPv4 Rem Rts: 0      Total SrPlcyIPv4 Rem Act Rts: 0
Total SrPlcyIPv6 Rem Rts: 0      Total SrPlcyIPv6 Rem Act Rts: 0

=====
BGP Summary
=====
Legend : D - Dynamic Neighbor
=====
Neighbor
Description
                AS PktRcvd InQ Up/Down State|Rcv/Act/Sent (Addr Family)
                PktSent OutQ
-----
10.121.0.10
                64530      57   0 00h26m24s 1/1/2 (IPv4)
                58      0
-----

```

Operators delivering business services through a BNG often use import and export BGP policies similar to those used on PE to CE peering sessions. More precisely, an import and export policy is required on the dynamic BGP peer in order to accept and advertise routes respectively whilst complying with the security behaviour in [12]. As with other parameters, import/export policies can be derived from multiple sources, so a priority order is also defined for them as follows:

- Use import/export policies returned in RADIUS VSA attributes and append any policies configured in the **bgp-peering-policy**.
- If import/export policies are not available from RADIUS and not configured in the **bgp-peering-policy**, use the policies configured in the VPRN BGP group with **static-group false**.
- If import/export policies are not configured in the VPRN BGP group with **static-group false**, use the policies configured in the VPRN service BGP context.

When the subscriber state is removed, the BGP session is torn down and all resources associated with that session are released.

Appendix A: Path Selection Process

This appendix is intended to detail the BGP path selection process and provide some additional information regarding MED comparison based on parameter settings.

Path Selection Process

Table 0-1 lists the BGP best path decision process. If there is a tie between paths at any step, the path selection process proceeds to the next step.

Table 0-1: BGP Best-Path decision process

Step	Path Selection	Selection Criteria
1	Select a valid route over an invalid route	If a BGP route is invalid because its Next-Hop cannot be resolved, it may still be advertisable if there are no valid routes (for example, by a Route-Reflector)
2	Prefer a route that has been installed in the FIB over a route that has not	Routes may not be installed in the FIB because route-table-install is set to false
3	Prefer a non-stale route over a stale route	Routes could be stale owing to Graceful Restart
4	A default route generated by the default-originate command is less preferred than a default route programmed by any other means	
5	Select the route with the lowest origin validation state	Valid<Not-Found<Invalid
6	Select the route with the numerically lowest route-table preference	For VPN-IP routes this also considers the number of VPRNs that imported the route
7	Select the route with the highest local preference	
8	Select the route with an AIGP metric	If both paths have an AIGP metric, select the route with the lowest sum of a) AIGMP metric value stored with the LOC-RIB copy of the route, or b) route-table or tunnel-table cost between the calculating router and the BGP Next-Hop in the received route
9	Select the route with the shortest AS_PATH	AS numbers in AS_CONFED_SEQ and AS_CONFED_SET elements do not count toward the AS path length. This step is skipped if as-path-ignore is configured for the Address Family
10	Select the route with the lowest Origin	IGP<EGP<Incomplete

11	Select the route with the lowest MED	The following considerations apply: a) By default, MED is only compared between routes for the same neighbour AS, but this comparison can be changed by setting the strict-as option of the relevant always-compare-med command to false b) By default, a missing MED is considered equivalent to a MED of 0. This can be modified by setting the missing-med-infinity option of the relevant always-compare-med context. c) The relevant always-compare-med configuration for a route comparison depends on the types of routes that are involved: – When comparing BGP VPN routes (with the same RD or different RDs), the always-compare-med configuration in the base route BGP configuration applies. – When comparing a BGP PE-CE route with a BGP VPN route that has been imported into a VPRN, the always-compare-med configuration in the VPRN BGP configuration applies.
12	Select the route with the lowest owner type	BGP<BGP-Label<BGP-VPN
13	Prefer routes learned from EBGP peers over routes learned from IBGP and confed-EBGP peers	
14	Select the route with the lowest route-table or tunnel-table cost to the Next-Hop	This step is skipped if ignore-nh-metric is configured, or if the routes are associated with different RIBs. For VPN-IP routes received by a router without any configured VPRN services, Next-Hop is determined from route-table cost.
15	Select the route with the lowest Next-Hop type	Resolutions made in the route-table are preferred to resolutions made in the tunnel table. This step is skipped if ignore-nh-metric is configured, or if the routes are associated with different RIBs.
16	Select the route received from the peer with the lowest router ID	Router ID derived from ORIGINATOR_ID attribute if present, or the BGP identifier of the peer. If ignore-router-id is configured, keep the current best path and skip the remaining steps
17	Select the route with the shortest CLUSTER_LIST length	
18	Select the route received from the peer with the lowest IP address	

19	For VPN-IP routes imported into a VPRN, select the route with the lowest RD value	
----	---	--



By default, MED values of VPN-IPv4/IPv6 routes are always compared even if **always-compare-med** is disabled (default). This behaviour is historic and allows for sites of the same VPN to belong to different Autonomous System numbers. If this behaviour is undesirable it can be disabled by setting the **always-compare-med strict-as** command to **true**.

Always-Compare-MED

As indicated previously in step 11 of *Table 0-1*, the MED attribute is by default only used in the decision process if both routes have the attribute present and come from the same neighbouring AS. The **always-compare-med** context does however allow for modification of this behaviour to suit environments.

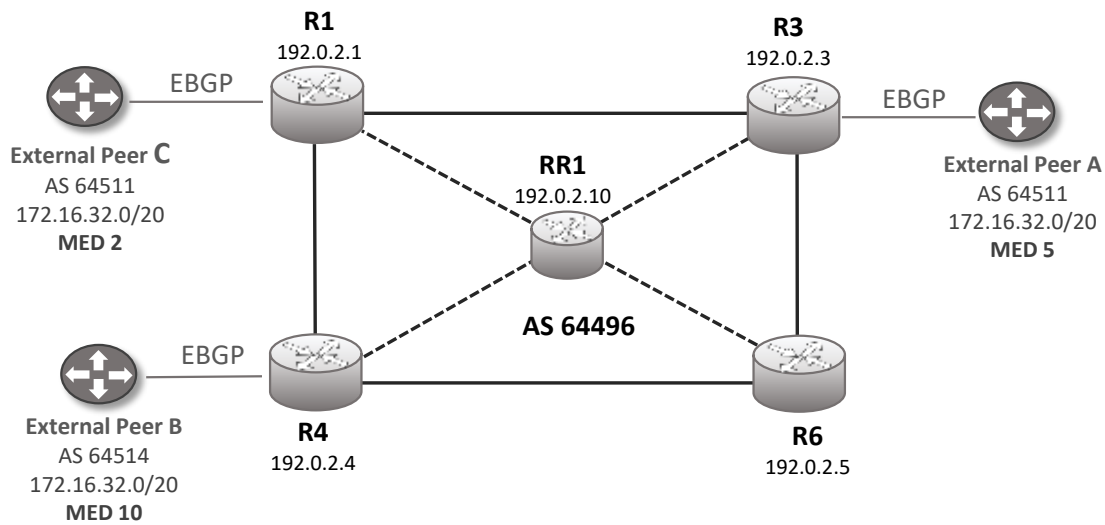
Table 0-2: Always-Compare-MED configuration options

Parameter	MED Comparison
always-compare-med	Only compare the MED of two paths (whether or not they are from the same neighbour-AS) if they both have MED attribute. If a route does not contain a MED attribute, it is handled the same as though it had a MED attribute with the value 0.
always-compare-med med-value zero	Always compare the MED of two paths, even if they are from a different neighbour AS. If one or both paths do not have a MED attribute, consider the MED to be zero. Without this command, this behaviour is inferred (in other words, this is default behaviour).
always-compare-med med-value infinity	Always compare MED of two paths, even if they are from a different neighbour-AS. If one or both paths do not have a MED attribute, consider the MED to be infinite.
always-compare-med strict-as	Only compare the MED of two paths if they come from the same neighbour-AS.

Deterministic MED

In some environments the outcome of the BGP path selection process can be unpredictable and potentially lead to route oscillation because it depends on the order in which routes are learnt. Consider the example shown in *Figure 0-1* where three external peers are advertising the prefix 172.16.32.0/20 with different AS paths and MED values.

Figure 0-1: Deterministic MED



Using router R6 as the calculating router, assume that routes are learnt from peers in the order A, then B, then C. When route A is received, it is the only route to the prefix 172.16.32.0/20, so it is automatically the best route. When route B arrives, it is compared to route A (the current best path). Because the neighbour AS of routes A and B are different the **always-compare-med** configuration option determines whether the MEDs in the two routes are comparable or not. For the sake of example, the **always-compare-med** option is not enabled, so route A remains the best path because of its lower Originator ID. When route C arrives, it is compared to route A, and because the neighbour AS are the same, route C is selected as the new best path because it has the lowest MED value.

Output 0-1: Best Path selection with routes received A, B, then C

```
A:admin@R6# show router bgp routes 172.16.32.0/20
```

```
=====
BGP Router ID:192.0.2.6      AS:64496      Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP IPv4 Routes
=====
```

Flag	Network Nextthop (Router) As-Path	LocalPref Path-Id	MED IGP Cost Label
u*>i	172.16.32.0/20 192.0.2.1 64511	100 187	2 300 -
*i	172.16.32.0/20 192.0.2.3 64511	100 177	5 100 -
*i	172.16.32.0/20 192.0.2.4 64514	100 181	10 200 -

```
-----
```

```
Routes : 3
```

Now consider an example where the routes are received in the order A, then C, then B. When route A is received, it is the only route to the prefix 172.16.32.0/20, so it is automatically the best route. When route C arrives, it is compared to route A (the current best path), and because the neighbour AS is the same, the route from C is installed due to the lower MED. When route B arrives, it is compared to route C. Because the neighbour AS is different and **always-compare-med** is not enabled, the MED is not compared, and route B becomes the best path because of IGP cost to the Next-Hop.

Output 0-2: Best Path selection with routes received A-C-B

```
A:admin@R6# show router bgp routes 172.16.32.0/20
=====
BGP Router ID:192.0.2.6          AS:64496          Local AS:64496
=====
Legend -
Status codes : u - used, s - suppressed, h - history, d - decayed, * - valid
               l - leaked, x - stale, > - best, b - backup, p - purge,
               w - unused-weight-only
Origin codes  : i - IGP, e - EGP, ? - incomplete
=====
BGP IPv4 Routes
=====
```

Flag	Network Nexthop (Router) As-Path	LocalPref Path-Id	MED IGP Cost Label
u*>i	172.16.32.0/20	100	10
	192.0.2.4	212	200
	64514		-
*i	172.16.32.0/20	100	2
	192.0.2.1	199	300
	64511		-
*i	172.16.32.0/20	100	5
	192.0.2.3	195	100
	64511		-

```
-----
Routes : 3
=====
```

Hence the preferred path is non-deterministic because it depends on the arrival order of the routes. In first example when routes arrived in the order A-B-C, route C was preferred. In the second example when routes arrived in the order A-C-B, route B was preferred. The Deterministic MED feature overcomes this problem and changes how MED comparisons are done to ensure deterministic best path selections. The main change is to always group received routes by neighbour AS (first AS in the AS_PATH or the local AS if the AS_PATH is empty). Within each group BGP selects the best path. (The configuration of **always-compare-med deterministic med** does not matter for this step.) Next, BGP compares all the 'group-best paths', and for this step the configuration of **always-compare-med** is relevant. If one path remains after this final MED comparison step, this is the overall best path. If multiple paths remain, further rules of the decision must be evaluated.

Consider again the preceding example. Router A and C belong to the same neighbour AS group and the comparison of these two paths always selects route C as the group-best

(lowest MED). With **always-compare-med** disabled, the MEDs of the group-best-paths cannot be compared so further rules must be evaluated. Route B is ultimately selected over route C as the best path in this example because of the lowest IGP cost. When deterministic MED is enabled route B will always be selected as best regardless of the arrival order of routes A, B, and C. The **deterministic-med** function and **always-compare-med** are both enabled in the **best-path-selection** node of the base BGP context. As indicated previously, enabling deterministic MED can be considered best practice in order to provide deterministic path selection and also avoid potential route oscillation.

Output 0-3: Deterministic MED configuration

```
router "Base" {  
    bgp {  
        best-path-selection {  
            deterministic-med true  
            always-compare-med {  
                med-value missing-med-zero  
            }  
        }  
    }  
}
```


References

- | | | |
|----|--|--|
| 1 | RFC4271 | A Border Gateway Protocol (BGP-4) |
| 2 | RFC5942 | Capabilities Advertisement with BGP-4 |
| 3 | RFC2918 | Route Refresh for BGP-4 |
| 4 | RFC4486 | BGP Cease Notification Message Subcodes |
| 5 | RFC7606 | Revised Error Handling for BGP Update Messages |
| 6 | RFC4760 | Multiprotocol Extensions for BGP-4 |
| 7 | RFC4364 | BGP/MPLS IP-VPNs |
| 8 | RFC4360 | BGP Extended Communities Attribute |
| 9 | RFC3107 | Carrying Label Information in BGP-4 |
| 10 | RFC2918 | Route Refresh Capability for BGP-4 |
| 11 | RFC5291 | Outbound Route Filtering Capability for BGP-4 |
| 12 | RFC5295 | The TCP Authentication Option |
| 13 | RFC8212 | Default External BGP (EBGP) Route Propagation Behavior without Policies |
| 14 | RFC4291 | IPv6 Addressing Architecture |
| 15 | RFC4684 | Constrained Route Distribution for BGP/MPLS IP-VPNs |
| 16 | RFC6164 | Using 127-bit Prefixes on Inter-Router Links |
| 17 | RFC3756 | IPv6 Neighbor Discovery (ND) Trust Models and Threats |
| 18 | RFC4761 | Virtual Private LAN Service (VPLS) Using BGP for Auto-Discovery and Signalling |
| 19 | draft-ietf-bess-vpls-multihoming | BGP based Multi-homing in Virtual Private LAN Service |
| 20 | RFC6624 | Layer 2 VPNs using BGP for Auto-Discovery and Signalling |
| 21 | RFC7432 | BGP MPLS-Based Ethernet VPN |
| 22 | RFC8365 | A Network Virtualization Overlay Solution Using Ethernet VPN (EVPN) |
| 23 | RFC8214 | Virtual Private Wire Support in Ethernet VPN |
| 24 | RFC9784 | Virtual Ethernet Segments for EVPN and Provider Backbone Bridge EVPN |
| 25 | draft-ietf-bess-evpn-df-election-framework | Framework for EVPN Designated Forward Election Extensibility |
| 26 | RFC9785 | Preference-Based EVPN Designated Forwarder Election |
| 27 | RFC9744 | EVPN Virtual Private Wire Service (VPWS) Flexible Cross-Connect (FXC) Service |
| 28 | RFC8317 | Ethernet-Tree (E-Tree) Support in Ethernet VPN (EVPN) and Provider Backbone Bridging EVPN (PBB-EVPN) |
| 29 | RFC9136 | IP Prefix Advertisement in Ethernet VPN |

30	RFC9135	Integrated Routing and Bridging in Ethernet VPN
31	RFC9012	The BGP Tunnel Encapsulation Attribute
32	draft-ietf-bess-evpn-dpath	Domain Path (D-PATH) for Ethernet VPN (EVPN) Interconnect Networks
33	draft-ietf-mpls-seamless-mpls	Seamless MPLS Architecture
34	RFC4798	Connecting IPv4 Islands over IPv4 MPLS Using IPv6 Provider Edge Routers (6PE)
35	RFC8669	Segment Routing Prefix Segment Identifier Extensions for BGP
36	RFC8277	Using BGP to Bind MPLS Labels to Address Prefixes
37	RFC9256	Segment Routing Policy Architecture
38	RFC9830	Advertising Segment Routing Policies in BGP
39	RFC8029	Detecting MPLS Data-Plane Failures
40	RFC5512	The BGP Encapsulation Subsequent Address Family Identifier (SAFI) and the BGP Tunnel Encapsulation Attribute
41	RFC8669	Segment Routing Prefix Segment Identifier Extensions for BGP
42	RFC7911	Advertisement of Multiple Paths in BGP
43	draft-ietf-rtgwg-bgp-pic	BGP Prefix Independent Convergence
44	RFC4724	Graceful Restart Mechanism for BGP
45	RFC6513	Multicast in MPLS/BGP IP VPNs
46	RFC6514	BGP Encodings and Procedures for Multicast in MPLS/BGP IP VPNs
47	RFC6037	Cisco Systems' Solution for Multicast in BGP/MPLS IP VPNs
48	RFC6625	Wildcard in Multicast VPN Auto-Discovery Routes
49	RFC4724	Graceful Restart Mechanism for BGP
50	RFC7606	Revised Error Handling for BGP Update Messages
51	RFC8955	Dissemination of Flow Specification Rules
52	RFC7674	Clarification of the FlowSpec Redirect Extended Community
53	draft-ietf-idr-flowspec-interface-set	Applying BGP flowspec rules on a specific interface-set
54	RFC3704	Ingress Filtering for Multihomed Networks
55	RFC5082	The Generalized TTL Security Mechanism (GTSM)
56	RFC5925	The TCP Authentication Option
57	RFC6811	BGP Prefix Origin Validation
58	RFC6481	A Profile for Resource Certificate Repository Structure
59	RFC6480	RPKI Architecture
60	RFC6810	The Resource Public Key Infrastructure (RPKI) to Router Protocol

61	RFC8097	BGP Prefix Origin Validation State Extended Community
62	draft-ietf-idr-link-bandwidth	BGP Link Bandwidth Extended Community
62	RFC7854	BGP Monitoring Protocol
63	RFC8671	Support for Adj-RIB-Out in the BGP Monitoring Protocol (BM)
64	RFC9069	Support for Local RIB in the BGP Monitoring Protocol (BMP)
65	RFC4272	BGP Security Vulnerabilities Analysis
66	RFC5549	Advertising IPv4 Network Layer Reachability Information with an IPv6 Next Hop
67	RFC4456	BGP Route Reflection
68	RFC9107	BGP Optimal Route Reflection
69	RFC7752	North-Bound Distribution of Link-State and Traffic-Engineering Information using BGP
70	RFC9085	BGP Link State Extensions for Segment Routing
71	RFC9514	BGP Link State Extensions for Segment Routing over IPv6
72	RFC9351	BGP Link State Extensions for Flexible Algorithms Advertisement
73	RFC8919	Application-Specific Link Attributes Advertisement using the BGP Link State
74	RFC5305	IS-IS Extensions for Traffic Engineering
75	RFC3630	Traffic Engineering Extensions for OSPF Version 2
76	RFC8570	IS-IS TE Metric Extensions
77	RFC7471	OSPF Traffic Engineering Metric Extensions
78	draft-ietf-idr-rfc7752bis	North-Bound Distribution of Link-State and Traffic Engineering (TE) Information using BGP
79	RFC8584	Framework for Ethernet VPN Designed Forwarder Election Extensibility
80	RFC4577	OSPF for BGP/MPLS IP VPNs
81	RFC7938	Use of BGP for Routing in Large-Scale Data-Centres
82	RFC8538	Notification Support for BGP Graceful Restart
83	RFC8654	Extended Message Support for BGP
84	RFC879	TCP Maximum Segment Size
85	RFC1191	Path MTU Discovery

Glossary

ABR	Area Border Router
AC	Attachment Circuit
AD	Auto-Discovery
AGI	Address Group Identifier
AGN	Aggregation Node
ALTO	Application Layer Transport Optimisation
AN	Access Node
ARF	Automatic Route Filtering
ARP	Address Resolution Protocol
ASBR	Autonomous System Border Router
ASM	Any Source Multicast
ARP	Address Resolution Protocol
ASBR	Autonomous System Border Router
ASN	Autonomous System Number
BD	Broadcast Domain
BMP	BGP Monitoring Protocol
BNG	Broadband Network Gateway
BSM	Bootstrap Messages
BSR	Bootstrap Router
BUM	Broadcast, Unknown Unicast, and Multicast
CCM	Circuit Continuity Message
CE	Customer Edge
CM	Containerised Machine
CNF	Containerised Network Function
CPE	Customer Premise Equipment
CPM	Control and Processing Module
CSV	Circuit Status Vector
DDOS	Distributed Denial of Service
DF	Designated Forwarder
DHCP	Dynamic Host Configuration Protocol
DSL	Digital Subscriber Line
ECMP	Equal Cost Multipath
EGP	Exterior Gateway Protocol
EOR	End of RIB
ES	Ethernet Segment
ESI	Ethernet Segment Identifier
EVI	Ethernet VPN Instance
EVPN	Ethernet Virtual Private Network
FC	Forwarding Class
FDB	Forwarding Database
FEC	Forwarding Equivalence Class
FIB	Forwarding Information Base

FPE	Forward Processing Engine
FSM	Finite State Machine
FTTC	Fibre to the Cabinet
FTTH	Fibre to the Home
FWA	Fixed Wireless Access
FXC	Flexible Cross Connect
GIB	Global Information Base
GTSM	Generalised TTL Security Mechanism
HMAC	Hash Message Authentication Code
IIF	Incoming Interface (List)
ILM	Incoming Label Map (table)
IMM	Integrated Media Module
IOM	Input Output Module
IRB	Integrated Routing and Bridging
IX	Internet Exchange
IXP	Internet Exchange Point
LACP	Link Aggregation Control Protocol
LAG	Link Aggregation Group
LAN	Local Area Network
LB	Label Block
LDP	Label Distribution Protocol
LFIB	Label Forwarding Information Base
LIB	Local Information Base
LS	Link State
LSA	Link State Advertisement
MAC	Media Access Control
MD5	Message Digest 5
MEF	Metro Ethernet Forum
MEP	Maintenance End Point
MKT	Master Key Tuple
MPLS	Multiprotocol Label Switching
MPLSCP	Multiprotocol Label Switching Control Protocol
MRAI	Minimum Route Advertisement Interval
MSD	Maximum Segment Depth
MSDP	Multicast Source Discovery Protocol
MSS	Maximum Segment Size
MTU	Maximum Transmission Unit
MVPN	Multicast VPN
NAT	Network Address Translation
NATP	Network and Port Address Translation
NCP	Network Control Protocol
ND	Neighbour Discovery
NLRI	Network Layer Reachability Information
NSF	Non-Stop Forwarding

NSR	Non-Stop Routing
NSSA	Not-So-Stubby Area
NTP	Network Time Protocol
NVE	Network Virtualisation Edge
NVO3	Network Virtualisation Overlay over Layer 3
OIL (or OIF)	Outgoing Interface (List)
ORF	Outbound Route Filtering
ORR	Optimal Route Reflection
OSPF	Open Shortest Path First
PCE	Path Computation Element
PHB	Per-Hop Behaviour
PIC	Prefix Independent Convergence
PIM	Protocol Independent Multicast
PIM-SM	PIM Sparse Mode
PIM-SSM	PIM Source Specific Mode
PMSI	P-Multicast Service Interface (Tunnel)
PMTUD	Path MTU Discovery
PNF	Physical Network Function
POS	Packet over Sonet
PPP	Point-to-Point Protocol
PTP	Precision Time Protocol
PVC	Permanent Virtual Circuit
QOS	Quality of Service
QPPB	QoS Policy Propagation using BGP
RADIUS	Remote Access Dial-In User Service
RD	Route-Distinguisher
RG	Residential Gateway
RIR	Regional Internet Registry
RM	Route Monitoring
ROA	Route Origin Authorisation
RP	Rendezvous Point
RPF	Reverse Path Forwarding
RPKI	Resource Public Key Infrastructure
RPT	Rendezvous Point Tree
RSVP	Resource Reservation Protocol
RT	Route-Target
RTM	Route-Table Manager
SA	Source Active
SAII	Source Attachment Individual Identifier
SAP	Service Access Point
SBD	Supplementary Broadcast Domain
SBFD	Seamless Bidirectional Forwarding Detection
SDP	Service Distribution Point
SMP	Symmetric Multi-core Processing

SPF	Shortest Path First
SR	Statistics Report
SRGB	Segment Routing Global Block
SRH	Segment Routing Header
SRLB	Segment Routing Local Block
STP	Spanning Tree Protocol
TAII	Target Attachment Individual Identifier
TCAM	Tertiary Content Addressable Memory
TED	Traffic Engineering Database
TCP	Transmission Control Protocol
TCP-AO	TCP Authentication Option
TE	Traffic Engineering
TED	Traffic Engineering Database
TLV	Type Length Value
TTL	Time-to-Live
UCMP	Unequal Cost Multipath
UDP	User Datagram Protocol
UMH	Upstream Multicast Hop
URPF	Unicast Reverse Path Forwarding
VBO	VE Block Offset
VBS	VE Block Size
VE	VPLS Edge
VLAN	Virtual Local Area Network
VDSL	Very high-bit-rate Digital Subscriber Line
VID	VLAN Identifier
VPLS	Virtual Private LAN Service
VPWS	Virtual Private Wire Service
VM	Virtual Machine
VNF	Virtual Network Function
VNI	VXLAN Network Identifier
VPN	Virtual Private Network
VRF	Virtual Routing and Forwarding instance
VRP	Validated ROA Payload
VSA	Vendor Specific Attribute
VSI	VPLS Switch Instance
VTEP	VXLAN Tunnel Endpoint
VXLAN	Virtual Extensible Local Area Network
WAN	Wide Area Network

Change Control

Date	Version	Remarks
7 th November 2025	Draft	General rewording, cleansing, and updating of text throughout. Removed: Dynamic Multi-Segment Pseudowire and BGP Anycast. All configuration outputs and show commands done with MD-CLI. Added the following: Chapter 3: Added section OSPF as a PE-CE protocol. Chapter 6: EVPN including EVPN-VPLS, and EVPN-VPWS, FXC. Chapter 7: RIB management, IPv6 PE. Chapter 8: Segment Routing and BGP. Chapter 9: Delayed Route Advertisement. Chapter 10: Wildcard S-PMSI. Chapter 12: Updated FlowSpec section for new AFs, added TCP-AO. Chapter 13: Sections on BGP RIB-leaking, BMP, Advertising IPv4 reachability over an IPv6 core, ORR, and BGP Link-State.
26 th November 2025	Issue 0.1	All chapters reviewed by author. Some chapters require peer review.
11 th January 2026	Issue 0.2	Added section on TCP Maximum Segment Size in Chapter 9. Minor updates to section on MRAI.
16 th January 2026	Issue 1	Minor comments from reviewers. Tidy-up nits. First issue.

NOKIA